

Transactions on Computer Systems and Networks

KCS Murti

Design Principles for Embedded Systems

 Springer

Transactions on Computer Systems and Networks

Series Editor

Amlan Chakrabarti, Director and Professor, A.K.Choudhury School of Information
Tech, Kolkata, West Bengal, India

Transactions on Computer Systems and Networks is a unique series that aims to capture advances in evolution of computer hardware and software systems and progress in computer networks. Computing Systems in present world span from miniature IoT nodes and embedded computing systems to large-scale cloud infrastructures, which necessitates developing systems architecture, storage infrastructure and process management to work at various scales. Present day networking technologies provide pervasive global coverage on a scale and enable multitude of transformative technologies. The new landscape of computing comprises of self-aware autonomous systems, which are built upon a software-hardware collaborative framework. These systems are designed to execute critical and non-critical tasks involving a variety of processing resources like multi-core CPUs, reconfigurable hardware, GPUs and TPUs which are managed through virtualisation, real-time process management and fault-tolerance. While AI, Machine Learning and Deep Learning tasks are predominantly increasing in the application space the computing system research aim towards efficient means of data processing, memory management, real-time task scheduling, scalable, secured and energy aware computing. The paradigm of computer networks also extends its support to this evolving application scenario through various advanced protocols, architectures and services. This series aims to present leading works on advances in theory, design, behaviour and applications in computing systems and networks. The Series accepts research monographs, introductory and advanced textbooks, professional books, reference works, and select conference proceedings.

More information about this series at <http://www.springer.com/series/16657>

KCS Murti

Design Principles for Embedded Systems

 Springer

KCS Murti (emeritus)
Central Electronics Engineering Research
Pilani, Rajasthan, India

ISSN 2730-7484 ISSN 2730-7492 (electronic)
Transactions on Computer Systems and Networks
ISBN 978-981-16-3292-1 ISBN 978-981-16-3293-8 (eBook)
<https://doi.org/10.1007/978-981-16-3293-8>

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2022

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Singapore Pte Ltd.
The registered company address is: 152 Beach Road, #21-01/04 Gateway East, Singapore 189721, Singapore

*Dedicated to my parents, my wife Rajeswari,
kids and grandkids*

Preface

I, being an embedded systems developer in the industry for two decades, had to struggle to collect the right knowledge from multiple books for designing robust systems (no internet existed at that time!). The scenario has not changed much now except people collect from an internet search for just-in-time learning. Meanwhile, I had the opportunity to move to the software industry where I was exposed to software product development and software engineering methodologies. After experiencing varied exposures, I confirmed that serious embedded system designers need knowledge of electronics, processors, software development, and engineering methods in a formal way.

During my career, I was lucky enough to pass over my hardware and software experience to students at BITS, Pilani through courses, like microprocessors, embedded systems design, software for embedded systems, etc. These courses were tailored to electronics and computer science students and also for engineers from the industry. I thought of compiling the essential methodologies covered in these courses as a book with a clear objective to bridge the gap between electronics and computer science students providing complementary knowledge essential for designing embedded systems.

Most of the universities/colleges teach this subject as “Embedded systems design” for ECE students. Mostly, this course covers programming microcontrollers, microprocessors with some practical examples. Additional knowledge is acquired by students through electives covering one single topic of their interest like real-time systems, modeling, networking, software engineering, etc. While all this knowledge is required for an embedded system design, one cannot take up all these specialized electives or study all these books. This textbook is my sincere effort to provide all these essential concepts tailored for embedded system design and transform the students as *Embedded System Architects*!

In today’s scenario, most of the educational institutes are Deemed-to-be-Universities that are free to introduce new courses and modernize the syllabus of existing courses. An appeal to the faculty is to update the course of “Embedded System Design” with state-of-the-art topics as per the industry needs.

The objective of this textbook is to bridge the gap between electronics and computer science students providing complementary knowledge essential for designing an embedded system. In a nutshell, our goal is to impart essential formal methodologies to design complex embedded systems.

Chapter 1 defines embedded systems (ES) and classifies them. The focus is to understand the basic strategy to be adapted in development based on market requirements, required quantity, time to market, and all such factors. This chapter introduces the basic characteristics of the system and identifies metrics to be considered in the design. We will broadly discuss different technologies used in designing ES. After reading this chapter, one gets a feel of real intricacies and strategies to be adapted in successfully developing an embedded system.

Majority of customers have difficulties in expressing what they require. A dialog between the developer and customer in a structured way helps in visualizing the system use. Chapter 2 discusses structured methodology in developing use cases which becomes the basis for requirements, documentation, and contracts. After studying this chapter and doing exercises, one can smoothly start developing use cases for any ES project.

The heart of any complex system design is to analyze the real-world problem by transforming it into an appropriate model. Chapter 3 discusses extensively the structural and behavioral models which are mostly reactive and work in real time and are frequently used in ES design. Students should practice all exercises to get real experience to handle any type of problem. I suggest the students on using any CASE tool to represent the model diagrammatically and analyze it. This topic must be done by both streams as CS students might have not done problems in the ES domain.

Once you are comfortable in modeling, you should get acquainted with one of the executable specification languages (ESL) in which the models are verified. Chapter 4 introduces SystemC as ESL. Most of the problems are extensions to those of Chap. 3 so that the models developed here can be implemented in SystemC and verified.

As the embedded systems are becoming more complex, you will have components in upper layers which have to be implemented in object-oriented languages like C++ and Java and in databases. Chapter 5 introduces UML for representing models at different stages of a project.

The heart of an embedded system is how efficiently the system can handle real-time events. This subject is covered normally as a one-semester course. Extensive mathematical analysis and algorithmic knowledge are involved. Chapter 6 introduces this topic with the essential knowledge required to design practical real-time embedded systems. After going through this chapter, students can assess the type of real-time events and decide what type of scheduling is needed and which real-time operating system (RTOS) product is appropriate to be used.

After studying the characteristics of a real-time system and the reference model, Chap. 7 introduces how these concepts are implemented in RTOS. This chapter touches generic RTOS concepts and in detail on the Posix-4 standard which is a real-time extension of POSIX and major features of pThreads.

Chapter 8 introduces the networking aspects of the embedded systems keeping the real-time constraints in mind. Most of the embedded systems are not stand-alone. They are distributed and networked to execute a common task. Broadly, NES is classified into automotive, industrial automation, building automation, and wireless sensor networks based on the real-world applications and networking requirements. This chapter discusses the network architectures and protocols which have been standardized for each of these segments.

Man-machine interface is important for the design of embedded systems. In the case of embedded systems, the interaction is quite different with variety of sensory systems, actuators, and affordances. Chapter 9 covers the essential human physiological system, its strengths, and limitations. Design rules and modern interface devices are explained briefly. Popular interaction models are explained.

As the complexity of embedded systems is increasing, design and implementation challenges are increasing. This leads to system-level design, aborting the old concept of HW and SW design separately. The current concept is function-level analysis, which breaks down hierarchically to a certain leaf level reaching certain granularity and allocates functionality to either software or hardware based on the specification constraints. Chapter 10 takes the basis of system-level modeling and analysis of Chap. 3, verification techniques, and system-level design and synthesis tools of Chap. 4 and introduces co-design concepts. Major emphasis is on different partitioning algorithms with case studies.

Millions of embedded systems are now battery-operated. They are smart and highly functional with millions of transistors compacted into processors, memory, peripherals, and SoCs. Power consumption increases heavily due to such dense architectures. Optimal design with contradicting constraints of high performance and less power is challenging. Chapter 11 discusses the basic concept of power dissipation at the transistor level and technics like *dynamic voltage scaling* (DVS) for energy optimization.

The processor architectures are advancing day by day with the advancements of VLSI technology. Chapter 12 introduces the basic trends in processor architecture at the conceptual level. Most of the commercially available processors, whether low- or high-ended, are designed and developed based on these concepts. After studying this chapter, the readers will be able to understand the internal architecture of any processor which helps in selecting a processor for individual requirements.

While complete systems-on-chip (SoC) is getting built, communication among cores and multiple heterogeneous peripherals is done through standard interfaces. Chapter 13 discusses some important peripheral interconnects and bus architectures that lead to efficient embedded platforms. After going through this chapter, the readers will get a good knowledge of how to select and configure an appropriate platform for a given application.

With increased functionalities in smart embedded systems, the complexity of the design increases and the vulnerability to attacks increases. Chapter 14 introduces the security principles, the security issues in embedded systems, and the methodology to solve them. In embedded systems, the challenge lies in securing not only the software

but also the firmware and hardware. Privacy, trust, and security are to be managed in the entire embedded system. After going through this chapter, the readers will be able to add the dimension of security at each stage of the system development life cycle.

Pilani, India

KCS Murti

Acknowledgments

While I was working at the Central Electronics Engineering Research Institute (CEERI), Pilani, in the area of electronic systems, (late) Prof. I. J. Nagrath, my teacher from BITS, Pilani, asked me to frame a course in microprocessors and teach in the year 1983. I did it for about 15 years as adjunct faculty teaching related courses. This model worked extremely well with academics synergized by industrial experience. At the outset, I would like to thank (late) Prof. I. J. Nagrath, for his vision and for allowing me to share my industrial research experience with students through teaching. I thank (late) Dr. G. N. Acharya, Prof. R. N. Biswas, and Dr. Chandra Shekhar who supported me to take up this task.

Being an embedded systems developer at CEERI, I have become a multi-disciplinary man with more focus on software. This made me switch to the software industry, Intergraph Consulting. Thanks to Dr. Sita Devulapalli, who placed me in challenging software projects, allowing me to impart my knowledge in software engineering and software quality.

I went back to teaching at the Hyderabad campus of BITS after I retired from Intergraph. Thanks to Prof. V. S. Rao, Director, BITS, Hyderabad campus; Prof. L. K. Maheshwari, Vice-Chancellor (VC); and Prof. M. B. Srinivas (HoD) for assigning me to teach ME embedded systems and microprocessor-related courses. I had a great opportunity to structure the courses of embedded systems with an equal focus on electronics and computing sciences.

This book is the result of my course content on software for embedded systems. This content must have not been possible without the opportunities given to me by all visionary stalwarts mentioned below.

My colleagues from the systems group of CEERI, Dr. R. S. Shekhawat, Prof. M. V. Karthikeyan, Dr. P. Bhanu Prasad, Dr. C. R. K. Prasad, Dr. Eranna, and Dr. Soumya J. from BITS have given me very constructive suggestions while I was authoring this book. Thanks to all of them.

While this job of authoring this book must have been done long back, I woke up only after I retired and had plenty of time during lockdown. I am sorry for that.

Thanks to Springer for continuous support and publishing the book.

Last but not least, thanks to my family members for encouraging me.

About This Book

This book is authored as a textbook for undergraduate students of electronics and computer science. Embedded systems design is a multi-disciplinary subject. The basic objective of this book is to impart hardware/software/engineering methodologies so that readers can design end-to-end complex embedded systems.

The purpose of this book is to bridge the gap between electronics and computer science students providing complementary knowledge essential for designing embedded systems. Readers learn basic strategies of implementation in terms of time-to-market, cost, and architecture selection. A structured approach to writing use cases and deriving requirements for un-ambiguous implementation is covered.

Design and analysis are covered in two chapters with a focus on model-driven approach and verification by executable system specification languages (SystemC). The major segment of embedded systems need hard real-time requirements. The book covers real-time concepts, including algorithms and real-time operating system standards like POSIX-4 and pThreads.

Embedded systems are mostly distributed and networked for deterministic responses. The book covers how to design networked embedded systems with appropriate protocols for real-time requirements. The major industrial segments for automotive, industrial automation, home automation, and WSNs are taken as examples for illustrating network protocol standards.

The modern concept is to design systems in a hardware/software agnostic way. The HW/SW co-design approach is introduced as one chapter. The human interface plays a major role in the success of the product. Different qualitative and quantitative approaches used are covered in one chapter.

Most of the commercially available processors, whether low- or high-ended, are designed with complex architectures. Two chapters introduce processor architectures and platforms and the concepts behind them. This enables the readers to evaluate and select appropriate processors and platforms for embedded systems design.

Millions of embedded systems are now battery-operated. They are smart and highly functional with millions of transistors compacted into processors, memory, peripherals, and SoCs. Power consumption increases heavily due to such dense architectures. Optimal design with contradicting constraints of high performance and less

power is challenging. The basic concept of power dissipation at the transistor level and techniques like *dynamic voltage scaling* (DVS) for energy optimization are covered in one chapter.

Last but not the least, security in embedded systems has become the most important topic of the day. Embedded systems like IoT and WSNs are no more stand-alone but distributed. Security is needed at the hardware, firmware, OS, and application levels in embedded systems. These aspects are covered in the last chapter.

The book includes case studies and exercises in each chapter for the students to practice. Once a reader completes all chapters, one appreciates the systematic approach needed for the end-to-end design of an embedded system.

Contents

- 1 The Strategy** 1
 - 1.1 Definition 2
 - 1.2 Common Characteristics 3
 - 1.3 Some Quality Metrics in ES Design 5
 - 1.4 Versatility Factors for ES Product 9
 - 1.4.1 Case Study: 1-1 10
 - 1.5 Technologies Involved 12
 - 1.5.1 Processors 12
 - 1.5.2 Platforms 14
 - 1.5.3 Devices-IC Technology 16
 - 1.6 Hardware/Software Co-design 17
 - 1.7 Summary 18
 - 1.8 Further Reading 18
 - 1.9 Exercises 18
 - References 19
- 2 Use Cases** 21
 - 2.1 History 21
 - 2.1.1 What Are Use Cases? 22
 - 2.1.2 Casual Versus Structured Version 23
 - 2.1.3 Black Box Versus White Box 25
 - 2.1.4 Hub and Spoke Model 25
 - 2.2 Details of the Use Case Model Entities 26
 - 2.2.1 Actor 26
 - 2.2.2 Stakeholder 26
 - 2.2.3 Primary Actor 27
 - 2.2.4 Supporting Actor 27
 - 2.2.5 Scope 27
 - 2.2.6 Scenarios 28
 - 2.2.7 Levels 28

2.2.8	Use Case Entities and Their Relation	28
2.2.9	When Are We Done?	30
2.2.10	Standard Use Case Template	30
2.3	Best Practices	31
2.4	Summary	32
2.5	Further Reading	32
2.6	Exercises	32
	References	34
3	Models and Architectures	35
3.1	Representation of a Design	36
3.1.1	Behavioral Representation	36
3.1.2	Structural Representation	36
3.1.3	Physical Representation	37
3.1.4	What Is a Model?	38
3.1.5	Case Study: 3-1	39
3.1.6	What Is Architecture?	40
3.1.7	Relation Between Model and Architecture	41
3.2	Model Taxonomy	42
3.2.1	State-Oriented Models	42
3.2.2	Activity-Oriented Models	42
3.2.3	Structure-Oriented Models	42
3.2.4	Data-Oriented Models	42
3.2.5	Heterogeneous Models	43
3.3	Finite-State Machine (Mealy) Model	43
3.3.1	Finite-State Machine (Moore Model)	44
3.3.2	Finite-State Machine with Data Path (FSMD)	45
3.3.3	Case Study: 3-2	46
3.3.4	Case Study: 3-3	47
3.3.5	Summary: Finite-State Machines	49
3.4	Petri Nets	49
3.4.1	Modeling of System Characteristics by Petri Nets	50
3.4.2	Properties of Petri Nets	52
3.4.3	Case Study: 3-4	54
3.4.4	Case Study: 3-5	55
3.4.5	Summary: Petri Nets	56
3.5	Hierarchical Concurrent FSMs	56
3.5.1	Summary: HCFSM	58
3.5.2	Case Study: 3-6	58
3.6	Activity-Oriented Data Flow Graphs	60
3.6.1	Case Study: 3-7	61
3.6.2	Solution	61
3.6.3	Summary: Data Flow Model	62

3.7	Control Flow Graphs (Flowchart)	62
3.7.1	Summary: CFG	62
3.8	Structure-Oriented Models	63
3.8.1	Summary: Structure Diagrams	63
3.9	Data-Oriented Entity-Relationship Model	63
3.9.1	Case Study: 3-8	64
3.10	Jackson's Structured Programming Model	65
3.10.1	Summary: Jackson's Model	66
3.11	Heterogeneous Models	66
3.11.1	Control/Data Flow Graph (CDFG)	66
3.11.2	Summary: CDFG	66
3.11.3	Case Study: 3-9	67
3.11.4	Object-Oriented Model	69
3.11.5	Encapsulation	70
3.11.6	Inheritance	70
3.11.7	Polymorphism	70
3.11.8	Case Study: 3-10	70
3.11.9	Case Study: 3-11	71
3.11.10	Program State Machines	74
3.11.11	Communication in PSM	76
3.11.12	Case Study: 3-12	77
3.12	Summary: Models and Architectures	77
3.13	Further Reading	78
3.14	Exercises	78
	References	83
4	Specification Languages: SystemC	85
4.1	An Example	86
4.2	Characteristics of ESL for Embedded Systems	87
4.2.1	Concurrency	87
4.2.2	Data-Driven	87
4.2.3	Control Flow Driven	88
4.2.4	Hierarchy of Behaviors	88
4.2.5	Completion of Behaviors	89
4.2.6	Shared Communication	89
4.2.7	Synchronization	90
4.2.8	Exception Handling	92
4.2.9	Summary: Specification Languages	92
4.3	SystemC	93
4.3.1	What Is SystemC?	95
4.3.2	SystemC Features	96
4.3.3	SystemC 2.0 Language Architecture	97
4.3.4	Module	98
4.3.5	Module Declaration	98
4.3.6	Module Ports	99

4.3.7	Module Constructor	99
4.3.8	Module Signals	99
4.3.9	Positional Connection	100
4.3.10	Named Connection	101
4.3.11	Member Functions	102
4.4	Processes	102
4.4.1	Method (SC_METHOD)	102
4.4.2	Thread (SC_THREAD)	102
4.5	Case Study: 4.1	103
4.5.1	Solution	103
4.5.2	Half-Adder Module	103
4.5.3	Full-Adder Module	103
4.5.4	Driver Module	104
4.5.5	Monitor Module	105
4.5.6	The Test Bench	106
4.6	Case Study: 4.2	106
4.7	Objects in SystemC	107
4.7.1	Sc_clock	107
4.7.2	Data Types	108
4.7.3	Wait Until	108
4.7.4	Sc_Start	108
4.7.5	Sc_Event	109
4.7.6	Wait	109
4.8	Models of Computation in SystemC	110
4.8.1	Untimed Functional Model	111
4.8.2	Timed Functional Model	111
4.8.3	Transaction-Level Model	111
4.8.4	Behavior Hardware Model	111
4.8.5	Register-Transfer Level Model	112
4.9	Interface	112
4.10	Channel	112
4.10.1	Primitive Channels	113
4.10.2	Hierarchical Channels	114
4.11	Summary: SystemC	115
4.12	Further Reading	116
4.13	Exercises	116
	References	117
5	UML for Embedded Systems	119
5.1	Motivation	120
5.2	Typical Tasks and Roles in System Engineering	121
5.3	UML Diagrams	121
5.4	Structural Diagrams	123
5.4.1	Class Diagram	124
5.4.2	Association	125

5.4.3	Association Class	126
5.4.4	Case Study 1	127
5.4.5	Aggregation	128
5.4.6	Composition	129
5.4.7	Generalization	129
5.4.8	Case Study 2	130
5.4.9	Interface	131
5.4.10	Signals	133
5.4.11	Component	133
5.4.12	Deployment Diagram	134
5.5	Behavioral Diagrams	135
5.5.1	Use Case Diagram	136
5.5.2	State Diagram	137
5.5.3	Activity Diagram	142
5.5.4	Sequence Diagram	143
5.5.5	Case Study 4	144
5.6	Other Diagrams	147
5.7	Summary—UML	147
5.8	Further Reading	148
5.9	Exercises	148
	References	153
6	Real-Time Systems	155
6.1	Definition and Examples	156
6.1.1	A Digital Controller	156
6.2	Broad Classification of RTS	157
6.2.1	Periodic	157
6.2.2	Mostly Periodic	157
6.2.3	Aperiodic but Predictable	158
6.2.4	Asynchronous and Unpredictable	158
6.3	Terms in RT Systems	158
6.3.1	Hard RT Systems	159
6.3.2	Soft RT Systems	160
6.3.3	Scheduler	160
6.3.4	Preemptivity	161
6.3.5	Criticality	161
6.3.6	Laxity Function	161
6.4	Periodic Schedule	162
6.4.1	Modeling Periodic Tasks	162
6.4.2	Task Utilization	163
6.4.3	Response to External Events	163
6.5	Precedence Constraints and Dependencies	164
6.5.1	Precedence Graph	164
6.5.2	Task Graph	165
6.5.3	Task Dependencies	165

6.5.4	Resource Graph	168
6.5.5	Scheduling Process	169
6.5.6	Valid and Feasible Schedule	169
6.6	Scheduling Algorithms—Classification	170
6.6.1	Static Scheduling	170
6.6.2	Dynamic Scheduling	171
6.6.3	Static Priority Scheduling	171
6.6.4	Dynamic Priority Scheduling	171
6.7	Clock-Driven Scheduling	171
6.7.1	Notation	171
6.7.2	Pseudo Algorithm	173
6.7.3	Slack Time	173
6.7.4	Handling Sporadic Jobs in Clock-Driven Scheduling	175
6.7.5	Merits and Demerits	175
6.8	Priority-Driven Periodic Tasks	176
6.8.1	Rate Monotonic Algorithm (RMA)	176
6.8.2	Deadline-Monotonic (DM) Algorithm	177
6.9	Dynamic Priority Algorithms	177
6.9.1	Earliest Deadline First (EDF)	178
6.9.2	Least Slack Time First Algorithm (LST)	178
6.9.3	Case Study 1	179
6.9.4	Case Study-2	180
6.9.5	Case Study 3	181
6.10	Scheduling Sporadic Jobs	181
6.11	Resource Access and Contention	182
6.11.1	Priority Inversion	183
6.11.2	Priority Inheritance	184
6.12	Summary	184
6.13	Further Reading	185
6.14	Exercises	185
	References	186
7	Real-Time Operating Systems (RTOS)	189
7.1	Introduction	189
7.2	RTOS Concepts	190
7.2.1	Task and Task States	191
7.2.2	RTOS—Basic Organization	192
7.2.3	Re-entrancy	192
7.2.4	Semaphore	193
7.2.5	Mutex	195
7.3	Basic Design Using RTOS	195
7.3.1	Case Study 1	196
7.4	Concept-Process and Threads	198
7.4.1	Kernel	199

7.4.2	Process	200
7.4.3	Thread	201
7.5	Posix	202
7.6	pThreads	202
7.6.1	Create Thread	203
7.6.2	Thread Synchronization	204
7.6.3	Cancel Thread	205
7.6.4	Schedule Threads	206
7.7	Thread Synchronization	210
7.7.1	Mutex	210
7.7.2	Semaphore	211
7.7.3	Condition Variable	213
7.7.4	Reader's/Writers Lock	214
7.7.5	Spin Locks	215
7.7.6	Barrier	215
7.8	Design Strategies	216
7.8.1	Master-Slave	216
7.8.2	Thread per Client	217
7.8.3	Thread per Request	217
7.8.4	Work Queue	217
7.8.5	Pipeline	217
7.9	Summary RTOS	217
7.10	Further Reading	218
7.11	Exercises	218
	References	223
8	Networked Embedded Systems (NES)	225
8.1	Introduction	226
8.2	Characteristics	226
8.2.1	Design Aspects	227
8.3	Broad Segments of NES	228
8.3.1	Automotive NES	228
8.3.2	NES in Industrial Automation	229
8.3.3	NES in Building Automation	230
8.3.4	Wireless Sensor Networks (WSN)	230
8.4	Automotive NES	231
8.4.1	Event-Triggered Protocols	231
8.4.2	Time Triggered Protocols (TT)	232
8.4.3	Example TT Protocols	232
8.4.4	Fundamental Services of TT Protocol	232
8.5	CAN (Controller Area Network)	233
8.5.1	CAN Frame	234
8.5.2	CAN Messages	235
8.5.3	CAN Physical Layer	235
8.5.4	CAN Media Access and Arbitration	237

8.5.5	CAN Protocol Stack	238
8.5.6	CAN Information Exchange	239
8.6	Time-Triggered CAN (TTCAN)	239
8.7	NES in Industrial Automation	241
8.7.1	Network Hierarchy	241
8.7.2	Fieldbus	243
8.7.3	Fieldbus Topology	243
8.8	NES in Building Automation	246
8.8.1	BACNET—Building Automation and Control Network	247
8.8.2	LON Works	248
8.8.3	ZigBee	250
8.9	Wireless Sensor Networks (WSN)	252
8.9.1	Structure	252
8.10	Summary-NES	257
8.11	Further Reading	258
	References	258
9	Human Interaction with Embedded Systems	261
9.1	Motivation	262
9.2	Overview	263
9.2.1	Study Users for Good Interface	263
9.2.2	Evaluate the Design	264
9.3	Human System	264
9.3.1	Vision	265
9.3.2	Touch	269
9.3.3	Movement	269
9.3.4	Memory	270
9.3.5	Cognitive System	270
9.4	Physical System	272
9.4.1	Handwriting Recognition	272
9.4.2	Speech Recognition	272
9.4.3	Eye Gaze	273
9.4.4	Virtual Reality	273
9.4.5	Sensing Position in 3D Space	274
9.4.6	Augmented Reality (AR)	274
9.5	Interaction Concepts	275
9.5.1	Interaction Model	276
9.5.2	Donald Norman's Model	276
9.5.3	Case Study-1	277
9.5.4	Ergonomics	278
9.5.5	Physical Design	279
9.6	Recent Paradigms in Computer Interaction	279
9.6.1	Metaphors	279
9.6.2	Multimodality	280

9.6.3	System Supported Cooperative Work	280
9.6.4	Human-Agent Interaction	280
9.6.5	Ubiquitous Computing	281
9.6.6	Implicit Interface	281
9.7	Design for Usability	282
9.7.1	Goals of Usability Engineering	282
9.7.2	Design Patterns	284
9.8	Evaluation	285
9.8.1	Cognitive Walkthrough	285
9.8.2	Heuristic Evaluation	285
9.8.3	Evaluation Through User Participation	286
9.8.4	Model-Based Evaluation	286
9.8.5	Case Study-3	290
9.9	Summary	292
9.10	Further Reading	292
9.11	Exercises	292
	References	293
10	HW-SW Co-design	295
10.1	Introduction	296
10.2	Factors Driving Co-design	297
10.3	Co-design Problems	298
10.4	Conventional Model for HW-SW Design Process	299
10.5	Integrated Co-design Process	300
10.6	System Partitioning	301
10.7	Partitioning Algorithms	304
10.8	Summary	313
10.9	Further Reading	313
10.10	Exercises	314
	References	316
11	Energy Efficient Embedded Systems	317
11.1	Introduction	318
11.1.1	Activity Allocation	318
11.1.2	Activity Mapping	319
11.1.3	Activity Scheduling	319
11.1.4	Energy Management	320
11.2	Energy Dissipation in Devices	321
11.2.1	Power Efficiency	322
11.3	Techniques for Energy Minimization	323
11.3.1	Dynamic Power Management (DPM)	323
11.3.2	Dynamic Voltage Scaling (DVS)	324
11.3.3	DVFS in Heterogeneous Processing Elements (PEs)	325
11.4	Energy-Aware Scheduling	325
11.4.1	Case Study-1	326

11.4.2	Static DVFS Scheduling	327
11.4.3	Case Study-2	328
11.5	Advanced Configuration and Power Interface (ACPI)	330
11.5.1	ACPI Components	330
11.5.2	ACPI System	333
11.5.3	ACPI System States	333
11.6	Typical Guidelines for Power Management	336
11.6.1	Power Management at Design Time	336
11.6.2	Power Management at Run Time	337
11.7	Summary	337
11.8	Further Reading	338
11.9	Exercises	338
	References	339
12	Embedded Processor Architectures	341
12.1	Introduction	342
12.2	Memory Hierarchy	343
12.3	Cache Basics	344
12.3.1	Direct Mapped Cache	345
12.3.2	Fully Associative Cache	345
12.3.3	Set Associative Cache	346
12.3.4	Writing to Cache	347
12.3.5	Replacing a Block During Cache Miss	347
12.3.6	Basic Cache Optimizations	348
12.4	Virtual Memory	348
12.4.1	Case Study-1	349
12.4.2	Few Techniques for Cache Performance	351
12.4.3	Compiler Optimizations	352
12.5	RISC Processors	353
12.5.1	Instruction Cycle for RISC	354
12.6	Pipelining	355
12.6.1	Hazards in Pipelining	356
12.6.2	Pipeline in MIPS Processor	358
12.6.3	Pipeline in Arm Cortex-A8	359
12.7	Data-Level Parallelism	361
12.7.1	Vector Architectures	361
12.7.2	Basic Structure	361
12.7.3	Vector Instructions	362
12.7.4	Lanes	363
12.7.5	Vector Length Register	364
12.7.6	Vector Mask Registers	364
12.7.7	Memory System and Memory Banks	364
12.7.8	Stride	364
12.7.9	Gather–Scatter	365
12.8	SIMD Architecture	366

12.9	Graphic Processing Units (GPU) SIMT Architecture	367
12.9.1	Programming Model	370
12.10	Thread-Level Parallelism (TLP)	371
12.10.1	Multicore Processor with Shared Memory	372
12.10.2	Multi-processor System with Distributed Memory	372
12.10.3	Multi-threaded Execution	373
12.11	Reconfigurable Computing—FPGAs	375
12.11.1	Generic PLD Architecture	376
12.11.2	Generic FPGA Architecture	377
12.11.3	Dynamic Configuration of FPGA	377
12.11.4	System on Chip with FPGA	378
12.11.5	Typical Mapping Flow of FPGA	379
12.11.6	Logic Design with CLB	379
12.11.7	Case Study–2	381
12.11.8	Connectivity Programming	382
12.11.9	Exploiting Reconfigurability	382
12.12	Summary	385
12.13	Further Reading	385
12.14	Exercises	386
	References	389
13	Embedded Platform Architectures	391
13.1	Introduction to Bus	392
13.2	Data Transfers	393
13.2.1	Synchronous Transfers	393
13.2.2	Asynchronous Transfers	393
13.2.3	Burst Transfers	394
13.2.4	IO Addressing	394
13.3	Typical ARM Platform with AMBA Bus	394
13.3.1	Advanced High-Performance Bus	394
13.4	Generic Interrupt Controller (GIC)	396
13.5	Modern IO Interfaces	399
13.5.1	Universal Serial Bus (USB)	399
13.5.2	Bluetooth	403
13.5.3	Low-Performance Device Interconnects	407
13.6	IOT Platform for Embedded Systems	409
13.6.1	Privacy in IoT	410
13.6.2	Security in IoT	411
13.6.3	IoT Architecture	411
13.6.4	Communication in IoT	412
13.7	Summary	416
13.8	Exercises	416
13.9	Further Study	417
	References	417

14	Security in Embedded Systems	419
14.1	Motivation	419
14.2	Introduction	420
14.2.1	Terminology	420
14.2.2	Cyber-Attacks on Embedded Systems	421
14.2.3	Security Policies	422
14.3	Security Vulnerabilities in ES	424
14.3.1	Buffer Overflow	424
14.3.2	Improper Input Data Validation	425
14.3.3	Improper Authentication	425
14.3.4	Out of Bounds Memory Access	426
14.3.5	DMA Attacks	426
14.3.6	Platform Reset Attacks	426
14.4	Basic Security Algorithms	426
14.4.1	Symmetric Ciphers	426
14.4.2	Secure Hash Algorithms	427
14.4.3	Asymmetric Algorithms	428
14.5	Security Protocols for Embedded Systems	429
14.6	Guidelines for Secure Systems	430
14.6.1	Trust Across Devices	431
14.6.2	Secure Firmware Updates and Critical Data	432
14.6.3	Secure Boot	432
14.6.4	UEFI Security Guidance	433
14.6.5	Trusted Zone	433
14.6.6	Secure OS	435
14.6.7	Secure Storage and Memory	435
14.6.8	System Recovery	435
14.6.9	Security Life Cycle	435
14.7	Security Standards for Embedded Systems	437
14.7.1	Digital Rights Management	437
14.7.2	Fast Identity Online Alliance (FIDO)	438
14.8	Typical Secured Platform Architecture	439
14.9	Summary	440
14.10	Further Reading	440
14.11	Exercises	441
	References	441
	Index	443

About the Author

KCS Murti has 44 years of industry, research, and academics experience. He has published over 50 papers at national and international conferences. He completed his B.E. from Andhra University in 1972 and M.E. from Birla Inst. of Tech & Science, Pilani, in 1974. He retired from BITS, Pilani, Hyderabad campus, in July 2018. His areas of specialization are real-time industrial networks, Geographic Information Systems, and real-time embedded systems. He worked as a research officer (Indian Engineering Services) with All India Radio for six years, as Assistant Professor at the Military College of Telecommunication Engineering (MCTE, MHOW) for three years, and as scientist C to F at CEERI, Pilani for 17 years and also held various positions at Intergraph Consulting.

Chapter 1

The Strategy



Abstract This chapter introduces embedded systems (ES) and discusses real design challenges. Section 1.1 defines an embedded system based on the basic traits of such systems. Embedded systems have common characteristics. These systems have unique functionality, driven by exclusive user requirements, have to be compact, energy-efficient, and reactive. The majority of them have to possess real-time behavior. Section 1.2 discusses these characteristics in detail. When embedded systems are designed, there should be a way of measuring the quality of the product. We should be able to measure quantitatively the metrics of the design. Let us call them design metrics which are measurable features of the system implementation. Section 1.3 will discuss these important metrics. These are common metrics but more can be added depending upon the application and emerging modern technologies. Some qualitative parameters define the versatility of the product. Section 1.4 explains the features which improve the versatility of a product. Technology is the manner of implementing a product. In our context, the platform used and the methods of hardware and software implementation are major strategies to optimize the cost and marketability of the product without compromising the quality. Decisions will be based on choosing the design around general-purpose processors, or ASICs, ASIPs, FPGAs, SoC, and so on. Nonrecurring engineering costs, time to market, quantity required, and the final marketable cost will decide the strategy to be adapted. Sections 1.5, 1.6, 1.7, 1.8 and 1.9 discuss several options of selecting a proper platform, processors, and IC technology for a strategic decision. This chapter concludes with an important statement that “Customer requirements” is the prime design and implementation factor. After reading this chapter, one gets a feel of real intricacies in successfully developing an embedded system and the strategy to be adapted. To summarize, the strategy of developing an embedded system is extremely complex and needs the customer’s involvement. Based on the customer’s requirements, the product has to be designed cost-effectively with the needed performance by properly selecting the metrics. This involves deciding the type of technology to be used for implementation. Chapter 2 discusses how we should interact with customers and extract the user requirements using the USE-CASE methodology.

Keywords Response time • Reactive systems • Real-time • Non recurring engineering costs (NRE) • User interface • Time to market • Ubiquity • Context aware • Hardware software co-design

1.1 Definition

Let us loosely define what an embedded system is at this stage. We can refine it as we go ahead. You find from the time you get up till the time you sleep, you will come across several embedded systems in your home. A list of some devices is shown in Fig. 1.1 You must try to count all of them, which have the following traits:

- It has certain processing capabilities.
- It reacts to the input taken from the environment.
- It responds with processed data.

Thus, any device that has in-built processing power with certain input and output capability and has certain memory built-in is a very generic way of defining an embedded system.

Very simple devices like a clock, an alarm, and a stopwatch are simple systems. Modern automobiles which have roughly 8–10 processors inside, communicating with each other, controlling the driving capability, and providing real comfort to the drivers are examples of complex embedded systems.

Now let us define a system. A system is a block with known behavior and has processing capability for a given input and generating desired output. A system can be further divided into a number of subsystems where each subsystem itself is a system. It is a typical hierarchical definition of a system.

As the system's complexity increases, the hierarchy of the subsystems increases. Their interaction is extremely important when you are trying to design such a complex system. In such cases, you require several formal methodologies in designing such

Fig. 1.1 A shortlist of embedded systems



systems. One way of defining an embedded system which you find in some books is that anything which is not a personal computer, laptop, or Mainframe or the server is an embedded system. This may be a rough way of defining an embedded system.

In today's scenario, billions of such devices are being produced. The success of such units depends upon several factors which are necessary to meet customer's requirements.

1.2 Common Characteristics

Let us discuss the common characteristics of an embedded system.

Unique functionality

Most of them have unique functionality. The system exhibits repeatedly the programmed behavior. Most consumer electronic devices like a clock, a stopwatch, a simple alarm device, a refrigerator, and a washing machine are classical examples. They all perform pre-programmed functionality.

User-driven requirements

The system is characterized by a very tightly constrained design. The user's requirements may not be technical but there may be several constraints. The constraints will go on changing from one type of device to another. Some devices have to be very low cost, particularly the consumer applications. A simple digital wristwatch or small digital thermometer are typical examples. But some users may need wristwatches that can withstand 10 m depth of water. The design and engineering of the products change for the same functionality.

Energy-efficient

Devices should consume very low power as most of them may be battery-based. This is the major criteria for most hand-held devices. The devices may be placed in locations where they may have to be self-powered.

Compact

Consumer devices should be compact enough so that they can be held in pockets and palm and taken along with them. Most of the wearable devices can be listed in this class of devices.

Response time

Devices need to be fast enough to provide the desired response. Fastness depends on the type of application that is necessarily needed. Fastness is always relative and depends upon the requirements.

Reactive

All embedded systems have reactive property. They continuously react to the changes in the system's environment and respond to inputs given from the end-user or the environment. Example: An air conditioner, humidity control machine in your home. They react to the environment and behave. The amount of time it takes for the reaction depends upon the response time and desired fastness of the device.

Real time

Majority of the devices need real-time behavior. A very rough way of defining real time is that time is a factor in output validity. It means that the output has to be available at a stipulated time. Time is a factor in designing the system. People also define this as a faster system as it should compute the results in real time without delay. But that's not really a true definition.

Let us see an example digital watch whether the four common characteristics are met (see Fig. 1.2). It is a single-function device managing real-time clock and other auxiliary functions like stopwatch, date, and time display. It is a tightly constrained device. It has to be low cost, consumes very less power, must be small and handy, and you should be able to perform the operations in real time. It is reactive to the press of buttons and displays desired functionality in real time.

Let us see another example of a smart washing machine (Fig. 1.3) and study which common characteristics are met. It is single-function device managing different wash cycles by sensing the buttons and actuating the valves and motors. It is a tightly constrained device. It has to be of moderate cost, consume moderate power, and you should perform the operations in real time. It is reactive to the press of buttons and processes the desired functionality in real time.

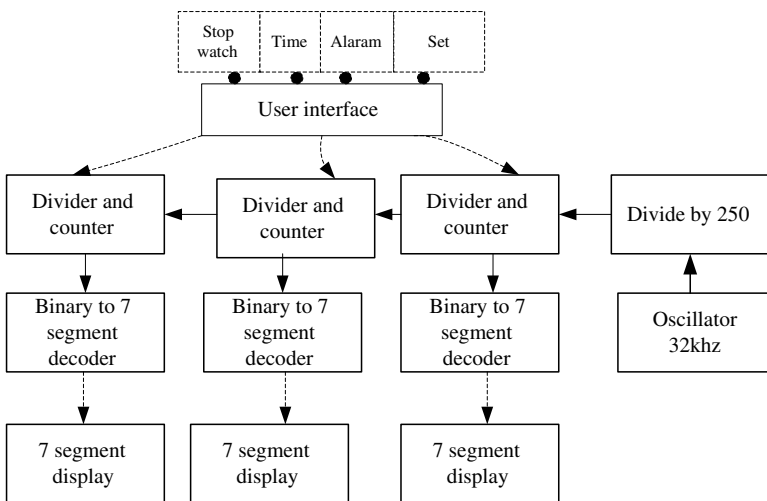


Fig. 1.2 A simple digital clock

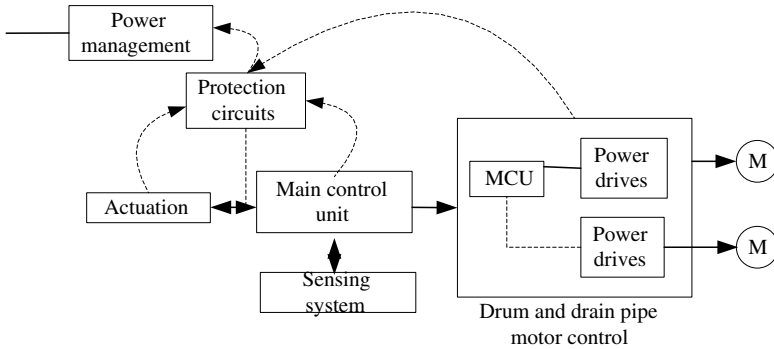


Fig. 1.3 A smart washing machine

1.3 Some Quality Metrics in ES Design

We have mentioned the major characteristics of the system. At the same time, there should be a way of measuring the quality of a product because it depends upon the design. We should be able to measure quantitatively the metrics of the design. Let us call them design metrics which are measurable features of the system implementation. We will discuss important metrics in the below paragraphs. These are common metrics but more can be added depending upon the application and emerging modern technologies.

Non-recurring engineering costs (NRE)

When you start developing a product, you will make a conceptual design for developing the prototype. Necessary tools for the development will be procured, a design is implemented, and the functionality is verified. You get formal approval from the customer if the system is for a specific customer. The costs involved in developing an engineered version of the system which is ready for production are one-time nonrecurring engineering costs. (NRE) These costs have to be absorbed in the marketing cost of the product. If you want the product to be brought into the market quickly, you may plan rapid development tools. If the initial prototype is designed with high quality (high quantified metrics) the cost and time to develop will increase. Effectively, NRE costs increase. A balance has to be achieved through proper optimization.

Unit cost

Unit cost is computed from the total NRE cost and the production costs. Effectively unit cost depends on the quantity required. NRE costs and quantity decide the unit costs. The actual unit cost of production has to be optimized based on the above factors.

Performance

Every consumer needs high performant systems irrespective of whether they really use that functionality. We have to properly optimize major functionality to the truly desired accuracy and response times required by the users. Performance can be in terms of response time, functionality, and several factors.

Energy efficiency

The desired metric for consumer applications like handheld devices, mobiles, wearables, etc. has to be designed so that they consume very less power and the charging rate is reduced. Consumers need systems driven from mains to consume less power for saving energy costs. That's why you find five stars to two-star ratings given to ACs, TVs, refrigerators, etc.

Functional updates

Customers need functional updates on existing systems. The design has to be flexible enough to add new features into the product, which are not thought of at the time of inception. This concept was tough in earlier days. But now, as the systems are processor-driven and connected through the internet and intelligent, new functionality can be added silently through software updates.

User interface

Customers need ease of interface with the system. They expect ease of interaction. A lot of research is being done to provide implicit interfaces, voice-based, pervasive, and several other paradigms (which we will discuss in subsequent chapters).

Size

Users need compact devices as much as possible. However, it requires major optimization in terms of cost. As you try to miniaturize by using ASICs and programmable logic devices, NRE costs go high. So, strike a balance between size and cost and time-to-market.

Time-to-market

This metric depends upon the time taken to prototype the unit, testing, and verification. Subsequently getting to production and releasing to the market. This process has to be optimized so that the product is available in the market at the right time even before your competitors try to bring in a similar item. Optimization depends upon the type of design you are planning and the time needed for this process. As an example, if you want to develop a low power and compact device, you may plan to develop an ASIC. But this involves time-to-market to be very high. By the time you prototype and bring it into the market, you may lose the market or your market share gets reduced. Hence you have to properly optimize the design. This is a design challenge.

Maintainability

Once the product is in the field, it has to be maintained from all defects and provide minor improvements in the field itself. Today varieties of techniques are available to make the system maintained remotely and the design itself should have sufficient features by which we can easily maintain systems in the market very quickly.

Ruggedness

Generally, it is thought of as the physical ruggedness of the system. But in embedded systems, it is measured in functional ruggedness, viz., recovery from unexpected conditions, correctness in measurements in harsh environments, etc.

Trueness

The trueness of the system is mostly observed through correctness in measurements. To some extent, it depends upon accuracy. Accuracy depends upon the cost. As an example, if you are making a simple digital weighing machine, the accuracy has to be just sufficient to get true results from the customer. If you want to make it too accurate which is not really required by the consumer, you are effectively increasing the cost of the system. So, here comes the requirements optimization scenario.

Safety

Personal safety is the highest priority. You might have heard that some mobile devices blasted while it is in use. The desired safety aspects have to be introduced in the design itself. Safety is the utmost factor so that the users and the installed site are not damaged and no loss of personal life. There are certain standards and international regulations to be adapted in product design by which safety aspects have to be complied.

Optimizations needed

You have seen in the above paragraphs, the common metrics by which you can quantitatively estimate the quality of the embedded system. However, it is very tough to get all the metrics optimized to high levels because the factors conflict with each other (see Fig. 1.4). You plan to bring out an energy-efficient, compact, and highly efficient system at a low cost. This has to be done at the expense of NRE costs through innovative design and development which increases time-to-market.

Let us see some of the metrics which have good interaction, viz., NRE cost dominates the cost of the unit if the device is planned to be very compact. In such a case, you need the development of ASICs to replace a lot of discrete hardware. This increases the cost of the system. Hence, a serious decision has to be taken to what extent the size of the system is affordable. Similarly, if you plan a very low power device, the design may involve specialized integrated circuits and complex hardware and software design to reduce the power. Again, the judicial decision has to be taken to what level the power has to be reduced. One of the metrics which everyone needs is high performance. It means that you have to use higher-ended processors with complex designs to meet the performance. You can see from the

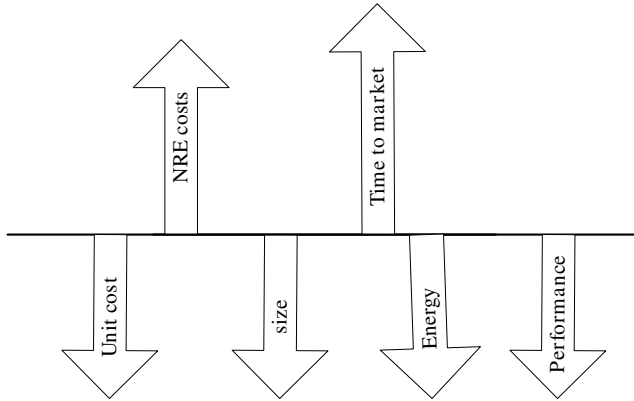


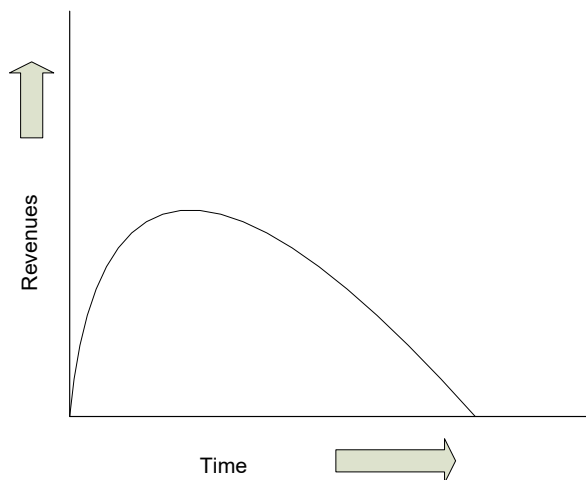
Fig. 1.4 Relation across metrics for developing a product with good metrics

above sample examples that the metrics are interrelated and a judicial decision has to be taken in setting the desired metrics and estimate the overall cost.

Time-to-market

Time-to-market is an important metric. A product has to compete with existing similar products in the market. A product’s life looks close to a bell-shaped curve (see Fig. 1.5). Initially, it has to pick up competing products. Sales will increase if it has versatile metrics relative to competitive products. However, the product’s consumption slowly fades as consumers find new competitive products with improved metrics. Hence, you find the life cycle of a consumer product follows roughly a bell-shaped curve. If a product’s entry into the market is delayed, effectively total sales go down. Revenue obtained is the area of the bell curve.

Fig. 1.5 Product life cycle



If you enter the market in a delayed fashion you pick a small share of the product market, but as the product gets into to diminishing slope, your product also gets diminished. This can be shown in Fig. 1.5 where the area of the curve shows the overall market gained by your product. The area of the curve is equal to the revenue obtained from the product.

The relation between NRE costs, the unit cost, and the quantity needed in the market have a close relation. As an example, if you want less number of units, say 2,000, and in another case you need 200,000 units, you can observe that the NRE cost can be easily absorbed when the number of units needed is high. So, the thumb rule is when you have estimated requirements in high volumes, you can absorb NRE costs easily. Bringing a quality product with less number of required units, unit costs will be high and the development is very challenging.

1.4 Versatility Factors for ES Product

We have discussed important metrics by which we can evaluate quantitatively a product. Some qualitative parameters which define the versatility of the product are listed and explained below in short (Danillo et al. 2006).

- Speed is an important factor that always improves the product's versatility. A high-speed processor will always be a better option and more functionality can be added in the future without loss of performance.
- As you add more and more features, you will gain the market from your competitor. Though all the features may not be really important, this is a psychological factor for the end-users. They always look for more features whether they use or not.
- Openness in your device helps to upgrade your system in the future by which new features can be added easily without deprecating the existing product. You can retain the product in the market by this feature.
- Expandability can be really provided in your system if the system is designed with open architecture. In today's scenario, this is essential because the product's life cycle can be extended heavily without getting obsolete. Also, the product can be customized.
- Customizability is another factor of the product. The product can be customized for different users with different functionality or different user interface. By this, you can attract different segments of people to utilize your product.
- Upgradability is another factor that can be thought of at the time of design. It is possible by designing in a modular fashion so that current modules can be replaced and new modules can make your system upgraded. This applies to hardware and also to software.
- Modularity is the mechanism by which the modules can be replaced and the system can be upgraded in a modular fashion. Modular designs consume more space and involve more design costs. Sometimes the reliability also reduces. All these factors have to be kept in mind before going to the modular design.

- In certain applications, an abrupt fail you're maybe detrimental for some applications like industrial instrumentation. Need graceful degradation, meaning that the product works with limited functionality before it fails.
- The accuracy of your system is a vital factor but it has to be judicially decided. System requirement specifications should mention the accuracy needed by the product. Over-providing the accuracy increases the cost of the system. Effectively it may not find a place in the market.
- Smart systems are the talk of today. All products are tagged with smart labels like "smart TV," "smart refrigerators," "smart sensors," "smart watch," and so on. It is extremely difficult to define "smartness" in this context. We can roughly state that "smart systems" try to outweigh the human operator behavior and increase their satisfaction level.
- Ubiquity means "being everywhere hidden." Ubiquitous systems enable computer-based services to be made available everywhere. They support intuitive human usage. They appear to be invisible to the user.
- Machine intelligence is the hot topic of today. Systems are getting designed approaching human intelligence, in some cases even exceeding human cognitive capabilities.
- Context-aware paradigm allows systems to take decisions sensing the current context. Human context, location context, and environmental context are some typical contexts.

Above is a list of features that can be upgraded depending upon the type of application. These factors must be reviewed and considered well before freezing requirements.

Till now, we have seen different metrics and some important parameters which have to be strategically considered before initiating conceptualized embedded system design. Now we will get into different technologies which are most commonly used in implementing the system (Marman [2010](#)).

1.4.1 Case Study: 1-1

The following is a list of four embedded systems. List of different crucial parameters mentioned above in the priority order 1, 2, 3, 4 and add comments for your decision.

Solution

S. no	System	1	2	3	4	Comments
1	Automatic brake control	<i>Safety/reliability</i>	<i>Speed of operation</i>	<i>Size</i>	<i>Unit cost</i>	System safety and speed of operation are very crucial because failure in the accident situation can cause human loss. This is a hard real-time application
2	Automatic teller machines	<i>User interface</i>	<i>Reliability</i>	<i>Maintainability</i>	<i>Ruggedness</i>	As ATM centers are operated by both educated and un-educated people, the user interface is very important
3	Radar tracker	<i>Performance</i>	<i>Accuracy</i>	<i>Speed of operation</i>	<i>Ruggedness</i>	Performance and speed of operation are important in radar tracking because missing the deadline can miss the target, hence this is a hard real-time application

(continued)

(continued)

S. no	System	1	2	3	4	Comments
4	Cell phones	Features	Unit cost	Power	Size	The features of mobile and its unit cost are important because attractive features can only attract public

1.5 Technologies Involved

Technology is the manner of implementing a task especially, in our context, the platform used, and the methods of hardware and software implementation. Basically, we will focus on the following paradigms of implementation which are important in our context.

- Processors
- Platforms
- Devices
- Designs.

1.5.1 Processors

General-purpose processors

Given the problem and basic logic of implementation (algorithm), the logic can be implemented in multiple ways. This can be explained by taking a simple example of implementing a 32-bit multiplier. A simple way of implementation is by writing a simple program on a general-purpose microprocessor-based system (see Fig. 1.6). It works perfectly fine. Even complex algorithms involving complex input and output patterns can be easily implemented on general-purpose microprocessor-based systems. This approach has several merits, viz., the NRE cost will be extremely low because the required hardware is readily available. Only the programming aspects have to be implemented. You have reduced time-to-market heavily. As the NRE costs are very less, the product unit cost will also be low. Another advantage is that the system expansion flexibility, modularity, and majority of the metrics we have discussed earlier get complied. Even the required volumes are very low; the cost of

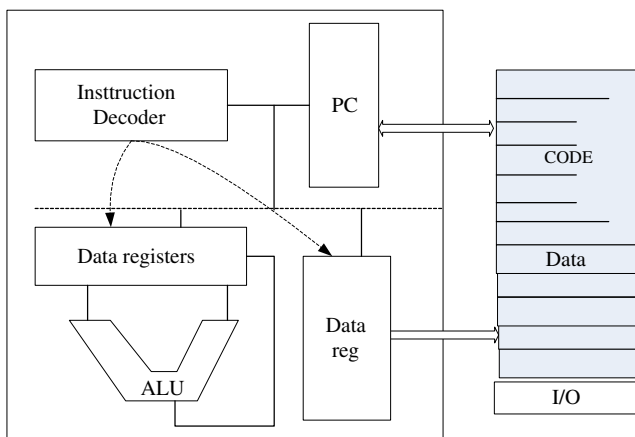


Fig. 1.6 General-purpose processor

the system will not increase. This is the reason why most of the generic products are conceptually designed and produced on general-purpose processors.

Custom processors

However, general-purpose processors cannot perform at the speed of a single clock cycle. As an example, if you desire the multiplication explained above is to be done in one nanosecond, it is not possible to implement the problem on a general-purpose processor.

Another drawback of going ahead with a general-purpose processor is that when you are producing in bulk you are not taking the advantage of reducing the unit cost by making customized hardware. Because of these reasons, you will go with customized processors and customized hardware.

If you see the architecture of a general-purpose processor it has a program memory, from where the instructions are fetched and executed by the control logic one instruction after the other in a cyclic fashion of instruction cycle. Hence it is a general-purpose instruction execution engine. If performance is essential and you are processing for a single application, one mechanism is to remove the general instruction cycle and have a control logic that is hard-wired and execute the specific instructions at the clock cycle level. However, it has similar general-purpose processor architecture except it is not general purpose. The performance will be extremely high because the instruction cycle can be one single cycle. Hence, this technique is used for customized and fast executing applications. Some examples are graphic accelerators, communication controllers, smartwatches, etc. You will get very high performance at low power and even at the smallest size. In spite of all the advantages, it has the demerit of very high NRE cost because the whole system has to be designed from scratch. For certain applications, people strategically use a combination of

general-purpose processors with such customized processors. Certain computationally intensive jobs are offloaded to the custom processor and the generic processing is done on the general-purpose processor.

Sometimes, such single-task designs may not be of much use compared to the cost involved in development. Hence, another branch of processors called application-specific processors has evolved. Examples are digital signal processors and microcontrollers.

These processors have a similar architecture of general-purpose, except they have a custom arithmetic unit and a much-tailored instruction set. The internal data paths of these processors are optimized for the type of functionality they perform. As an example, they have 64-bit and 128-bit data paths to perform mathematical computations.

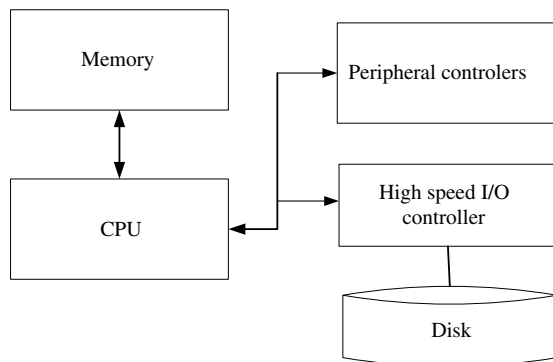
The advantage of application-specific processors is to provide high performance for certain computationally intensive jobs needing optimized hardware. The size and power consumed are also well within limits. In today's scenario, such processors are heavily used for strategic applications. One of the demerits of these processors is that they need specialized tools for programming and debugging.

1.5.2 Platforms

Dedicated systems.

Systems needing compact size, dedicated functionality, and little expandability are designed as single-board computers (see Fig. 1.7). Off-the-shelf boards are used which are close to their requirement and systems are developed around it. The major drawback is when certain components get outdated, the whole board has to be changed. Maintenance issues also crop up. Advantages are low cost, low NRE, and quick time-to-market.

Fig. 1.7 Dedicated systems



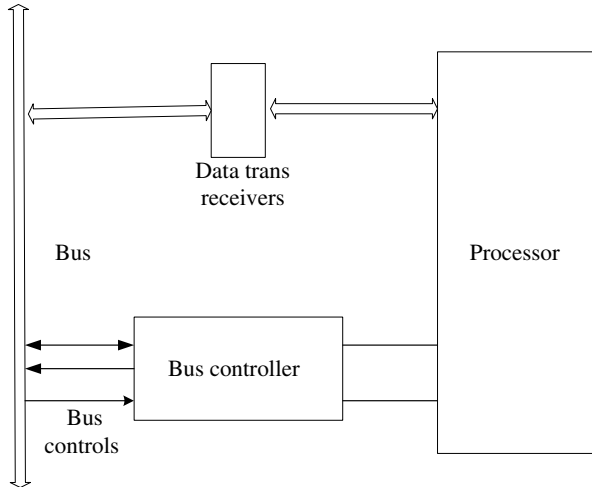


Fig. 1.8 Bus-based systems

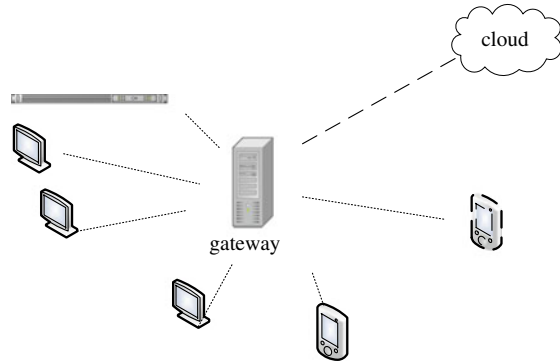
Bus-based systems

When systems with medium and high complexity have to be developed, the hardware functionality is modularized and each module is designed and fabricated separately (see Fig. 1.8). All the modules communicate across the bus. The majority of the systems are developed around standard bus specifications, like VME, PCI, ISA, etc. This helps in replacing a module with third-party products available in the market. Other advantages are modular expansion, upgradability, and easy maintenance. Most industrial systems and rack-based computers are designed as bus-based which comply with the environmental standards.

Distributed systems

Systems connected over wireless and communicate over Internet protocols to execute jobs in a distributed way are becoming the emerging paradigm (see Fig. 1.9). Wireless sensor networks, mobile computing, and the Internet of Things are important technologies in this direction. The end devices can be any smart device starting from a watch to a car. These devices compute locally and communicate with other peer devices through specified protocols and exchange data. Certain devices on the net can be servers. As all devices may not be using the same protocol, the gateway transforms relevant protocols appropriately.

Fig. 1.9 Distributed platforms



1.5.3 *Devices-IC Technology*

We have discussed the processor-based implementation. But sometimes customized applications have to be designed in discrete hardware. There are three major IC technologies used for this purpose. The first one is a full custom VLSI bass design in which case the whole application will be synthesized using high-level logic building tools like VHDL, and chips are fabricated using full custom. Such a solution provides very high performance, low cost, low power, and small size and covers the majority of metrics. However, the strategy of such implementation is practically adapted when the requirement is in bulk. For example, USB controllers, Bluetooth controllers, communication controllers, and image processors are normally fabricated as full custom chips.

As the cost and time-to-market become extremely large in the case of full custom, people adopt semi-custom application-specific integrated circuits using gate arrays and standard cells by which development time is drastically reduced.

Still, the current IC technologies are adopting programmable logic devices where the internal logic blocks are interconnected through programming. This can be done on site itself. Field programmable gate arrays (FPGAs) and programmable logic devices (PLDs) are major examples of this technology. Adapting to PLD-based design, we are able to make custom products in very short timings. NRE costs are made very less and achieve customized high performance. The current methodology of system designs adapts FPGA and general processors by offloading computationally intensive jobs on the FPGA hardware and remaining on general-purpose processors.

Another emerging concept is reconfigurable computing, wherein the field programmable devices can be dynamically configured at run time to realize different hardware blocks by re-mapping the routing. This is very close to the concept of dynamically linked libraries in software (DLLs), where the object code is dynamically loaded into memory when it has to be executed. After execution, it is offloaded with the next functional block. Applying this concept, once the functionality with a

hardware block is done, the same gate array gets reconfigured to new hardware for the next execution. By this, you can efficiently utilize available FPGA fabric.

1.6 Hardware/Software Co-design

Another major decision that a designer has to take is whether the task has to be implemented through software programming or implementing the logic in hardware. Taking the same multiplication example as above, a very simple program can be written to implement the multiplication and getting the result in software but if required speed cannot be achieved, this has to be implemented in discrete logic. The logic remains the same except for the way the implementation is done. The current methodology of designing systems does not discuss whether the problem is implemented on hardware or software. It makes a system-level design where the logic is agnostic to the way of its implementation. Once the logic is designed and tested at the system level, the designer decides whether to implement the logic completely on software, or completely on hardware or partially on hardware and partially on software. This process is called hardware/software co-design which we will deal with this topic subsequently. This concept is shown in Fig. 1.10 at a broad level.

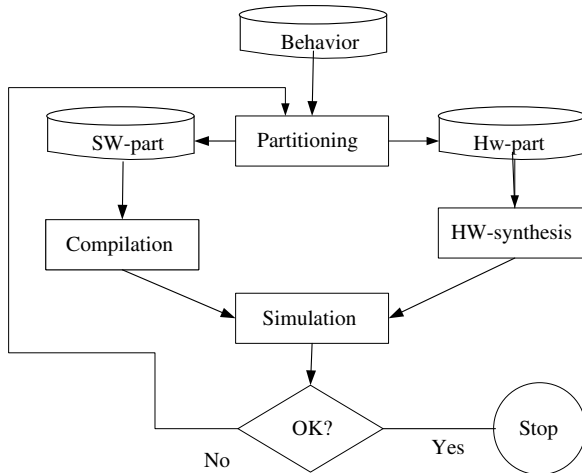


Fig. 1.10 Hardware/software co-design

1.7 Summary

To summarize, the strategy of developing an embedded system is extremely complex and needs the customer's involvement. Based on the customer's requirements, the product has to be designed cost-effectively with the needed performance by properly selecting the metrics. This involves deciding the type of technology to be used for implementation.

State-of-the-art designs can be done with a unified view of hardware and software. This topic will be dealt with in detail subsequently.

This chapter concludes with an important statement that "Customer requirements" is the prime design and implementation factor. The next chapter discusses how should we interact with customers and extract the user requirements using the USE-CASE methodology.

1.8 Further Reading

Several international conferences are conducted regularly on advances in embedded systems and emerging trends. References Danillo et al. (2006) and Koopman (1996) discuss in detail the emerging trends. Books by Vahid and Givargis (2006) and Wolf (2013) discuss elaborately the strategy to develop embedded systems.

1.9 Exercises

1. List all embedded systems found in your house and characterizes them.
2. From your study of those products, list the metrics to be considered in designing those products.
3. Mention what technologies/architectures you use in designing the subsystems needed.
4. You are planning to design a surveillance system with a maximum of 8 cameras and a digital video recorder. You want to develop and market this product. The estimated quantity is about 10,000 per year. Design your strategy.
5. You are asked to design the following gadgets. List three important parameters (in the order of priority) to be considered for each. What sort of processing technology (general-purpose, ASIP, custom) you prefer for each?
 - a. Washing machine
 - b. Automotive braking system
 - c. Railway signaling system
 - d. Camera
 - e. USB pen drive.

References

- Danillo et al (2006) Application-oriented system design as an embedded systems development strategy: a critical analysis. In: IEEE conference on emerging technologies
- Koopman PJ Jr (1996) Embedded system design issues. Preprint of paper published in: proceedings of the international conference on computer design (ICCD 96). Embedded System Design Issues
- Marman et al (2010) The new embedded system design methodology for improving design process performance. Int J Comput Sci Inf Secur
- Vahid F, Givargis T (2006) Embedded system design: a unified hardware/software approach, Student edition. Wiley
- Valvano J, Yerraballi R (2016) Embedded systems shape the world
- Wolf W (2013) Computers as components: principles of embedded computing system design. Elsevier

Chapter 2

Use Cases



Abstract After looking into the details of the design challenges of an embedded system, the next challenge is to capture detailed requirements of the system under development (SUD). The majority of the customers do not know how the proposed system looks like or able to make detailed requirements. Even the system designer cannot capture customer's requirements without having detailed interaction with the customers. Most of the projects fail because user's requirements are not captured properly. Certain customers can provide detailed technical requirements themselves, in which case, designers can get into the implementation phase. This chapter discusses a structured methodology of capturing use cases which becomes the nucleus to frame requirements. This topic is part of structured analysis and structured design (SASD) discussed in software engineering, which is very essential in developing embedded systems also. A use case is an agreement or contract between the stakeholders in the entire system. It states a sequence of actions and interactions between the users and the systems to achieve the desired goal. It describes how the system behaves and reacts to a request from one of the stakeholders. The actor who initiates the request is the primary actor. Use cases are not requirements. They do not state the required performance in qualitative or quantitative terms, nor the user interface or internal system design. There are several benefits to starting the project by framing structured use cases. If they are framed up to a granular level, the system's complexity is exposed. The system requirements can be extracted from these use cases systematically. Premature designs can be avoided. We focus on what the system should do rather than how it should do it. This chapter covers structured methodology to develop use cases and the best practices adopted in the industry.

Keywords Use cases · System under development (SUD) · Actor · Stake holder · Primary actor · Success scenario · Scope · Precondition · Hub and spoke model · Supporting actor · Scope

2.1 History

Software engineering experts have made a detailed study and formulated several methodologies for requirement analysis and design. Ivar Jacobson has introduced

the concept of use cases in 1960. The concept is extended with actors and goals in 1995 by Cockburn (Writing effective use cases). These concepts are introduced in UML specifications in 1999.

2.1.1 What Are Use Cases?

This can be explained by an example. Let us say, a customer wants a washing machine to be developed and introduced into the market with automatic features. But he is not clear on the way to specify its functionality. But he can explain the way the system is intended to be used by the end-user. He says, “the operator opens the front door, places the cloths to be washed, closes the door, opens the detergent box, puts the detergent, sets the type of wash and starts the system. The system will not start if the front door is not closed or the detergent is not placed. The system displays error messages if there is fault in the system and advises a solution to rectify the problem.”

From this text, you can capture certain **behaviors** of the entities involved. The first one is the washing machine which starts the wash cycle, once the start switch is pressed. The washing machine is the system under development (**SUD**) and is also an **actor**. There is a front door sensor and detergent sensor whose behavior is to detect the door closure and send a signal to other actors. The operator is another actor. He has a specific **interest**, i.e., to get the clothes washed. This is his **goal**. There is another actor who detects any fault in the system [fault detection unit (**FDU**)]. It is also an actor with fault detection as its behavior. FDU has a **specific interest** in keeping the system faultless and advises the fault rectification. This actor is a **stakeholder** in the system as he has a specific interest to keep the system faultless.

Similarly, the operator has a specific interest in washing his clothes. He is also a stakeholder. But he is the **primary actor** who **triggers** the activity to achieve his goal. Hence, he is called the primary actor. The **success scenario** of the operator's goal is to get the wash cycle complete without faults. The success scenario of another stakeholder, FDU is that the wash cycle has been completed. This is the **top-level use case**. As you go hierarchically down and detail their goals, the system use cases become comprehensive. The door closure unit is the actor of this subsystem which has a specific goal of placing clothes and closing the door. Another subgoal is placing the detergent. The terminating condition for the use case definition is the behavior of all actors is explained. Use case analysis revolves around this concept. The developer captures the use cases which become the input to detailed system requirements.

From the simple example above, we have introduced basic entities. Let us explore further.

- A use case is an agreement or contract between the stakeholders in the entire system. It states a sequence of actions and interactions between the users and the systems to achieve the desired goal.
- A use case describes how the system behaves and reacts to a request from one of the stakeholders. The actor who initiates the request is the primary actor.

- Fundamentally, the use cases can be framed as free-flow text, or flowcharts, sequence charts, Petri nets, and other methods. But we will introduce Alistair's method (Writing effective use cases) of structured representation of use cases.
- Use cases are a powerful method to capture and model the known functional requirements. Use cases do not form complete functional requirements. They are fractional. They describe only the behavioral portions.
- Use cases describe:
 - a business' work process,
 - to focus discussion about upcoming software system requirements, but not the requirements description,
 - the functional requirements for a system, or
 - how to document the design of the system.
- Use cases
 - Do not state the required performance in qualitative or quantitative terms.
 - Do not talk about the user interface how the user interacts through different dialogs and keypad interface, etc.
 - Do not talk about the internal designs, models used, concurrency, and the internal data definitions, data flow, and data interactions.

Benefits of use cases

- As the use cases are framed completely up to a granular level, the system's complexity is exposed.
- The real system requirements can be extracted from these use cases systematically.
- All the stakeholders will be involved in the use case formulation, so the goals and interests will be comprehensive. No requirements will be missed and no surprises of missing functional specifications after product development.
- Premature design can be avoided by focusing on what the system should do rather than how it should do it.

2.1.2 *Casual Versus Structured Version*

An example of writing use case as free-flow text (casual) to structured (fully dressed version) is given below:

- Casual version:
 - The customer initiates a request to purchase an item and its quantity to the agent. The customer makes a prepaid payment for the quantity ordered. The customer selects the address where it has to be delivered. The agent confirms the receipt of payment. The agent passes the request to the supplier. The supplier confirms the availability and date by which he will deliver the item. The agent confirms the same to the customer. The customer receives the items. He releases the payment to the supplier.

- Structured version:
 - Primary actor: Customer
 - Goal in context: Customer buys something through the system, gets it. Pays for it online.
 - Scope: Business: The overall purchasing mechanism
 - Level: Summary
 - Stakeholders and interests:
 - The customer wants the item that he has ordered.
 - The agent wants to distribute the orders to suppliers and get his commission.
 - The supplier wants to get paid for the goods delivered.
 - Precondition: None
 - Minimal guarantees: Every purchase request is closed properly.
 - Success guarantees: Every purchase request sent by the customer is executed successfully and delivered.
 - Trigger: Customer decides to buy something.
 - Main success scenario:
 - Customer: Initiate a request and gets the item.
 - Agent: Verifies money pre-paid by the customer, finds the supplier, and passes the order.
 - Supplier: Verifies availability, sets a delivery date and delivers items, and gets the money from the agent.

The above text compares the casual version of writing a use case with the structured version proposed by Cockburn (Writing effective use cases). If you study the paragraph in the casual version, the underlined entities are stakeholders in the system. They have a specific interest, as shown in the workflow. The workflow clearly states the initiation of the use case by the **customer** who proposes to purchase an item. The whole process goes through the actors/stakeholders and gets processed as per the behavior of these actors. The final success is to get the proposed item by the requester.

It is very difficult to study and extract requirements from such unstructured content. The fully dressed version provides a structured version that is more elegant and understandable.

The primary actor in this use case is the customer because this actor has triggered the use case. The next line explains the goal in this context, i.e., the primary actor wants to buy an item. The next line explains the scope of the use case, i.e., agent-based item purchase (like Amazon). The next line explains the level at which the use case is stated. In this example, this structure is at the topmost level and hence the level is mentioned as a summary. The use case can be further drilled down with the behavior of actors involved into detailed levels below. The next three lines explain each stakeholder and their interests when the use case is executed. The customer wants to get the item, the agent wants to get the commission, and the supplier delivers the item and gets a payment.

The next lines state when the stakeholder's interests are satisfied. These are successful scenarios. This use case does not explain failure scenarios, e.g., what happens when payment is not received? The concept of different scenarios will be explained in subsequent paragraphs. The next line states whether any preconditions have to be met before executing this use case. As there are no conditions, it is shown as null. The main success scenario states whether the individual stakeholder's interests are met and also the primary actor's interests are successful.

The final success scenario does not show different scenarios under which any of the stakeholder's interests are not met and the way it has to be handled by another use case.

2.1.3 Black Box Versus White Box

Whenever a new system is to be designed, the use cases are written without discussing the internals of the system. It is a black box use case. In the case of the washing machine example, the black box use case defines the sequence of user actions needed to start a wash cycle. It does not explain how it internally washes.

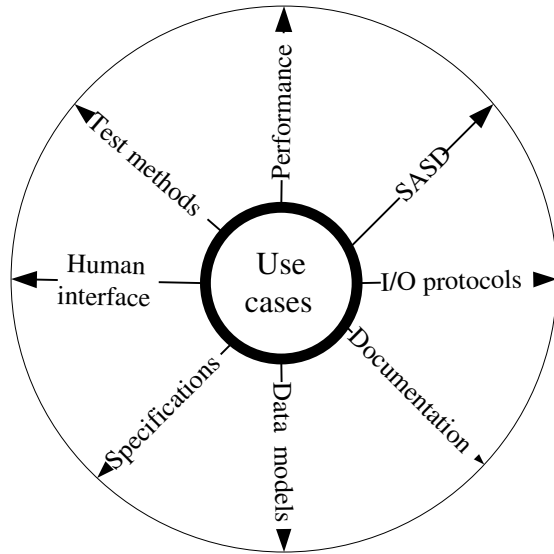
Business process designers can write white box use cases, showing how the company or organization runs its internal processes as a part of the use case.

2.1.4 Hub and Spoke Model

Figure 2.1 is the diagram that depicts the relationship between use cases and other system design activities. If one starts with use cases and gets to its deepest level considering all possible success and failure scenarios, the system design becomes ready to a major extent. Use cases do not talk about detailed user requirements, user interface, data formats, input and output requirements, timing requirements, performance requirements, communication methodologies across subsystems. But the basis for the design of all the above aspects is based on the use cases because every stakeholder's interests (sort of requirements) are covered in this.

This analysis becomes the starting point to judge the complexity of the system and estimate the rough costs. Outside the requirement document, they help structure project planning information such as release dates, teams, priorities, and development status. If use cases are designed to satisfying all stakeholder's interests, there is little possibility of having surprises at the end of development that certain requirements are not thought of in the beginning. Hence, the use cases act as the hub of a wheel, and the other information acts as spokes leading to different directions.

Fig. 2.1 Hub and spoke model



2.2 Details of the Use Case Model Entities

2.2.1 Actor

An actor is anyone or anything with behavior. Actors have goals. An actor might be a person, a company, or organization, a computer program or a computer system, hardware or software, or both.

2.2.2 Stakeholder

A stakeholder is someone or something with interest in the behavior of the system under development (SuD). A stakeholder has interests. To satisfy the interests of the stakeholders, we shall need to describe three sorts of actions:

- An interaction between two actors to further a goal
- A validation to protect a stakeholder
- An internal state change on behalf of a stakeholder.

A stakeholder is someone who participates in the contract.

2.2.3 Primary Actor

The primary actor has a certain goal. This actor initiates interaction with the system to achieve the goal. The primary actor is also one of the stakeholders in the system as he has a specific interest. This actor triggers the use case. This calls upon the interaction between different actors in the system and finally achieves the goal (success scenario). The use case manages different scenarios in case of failure.

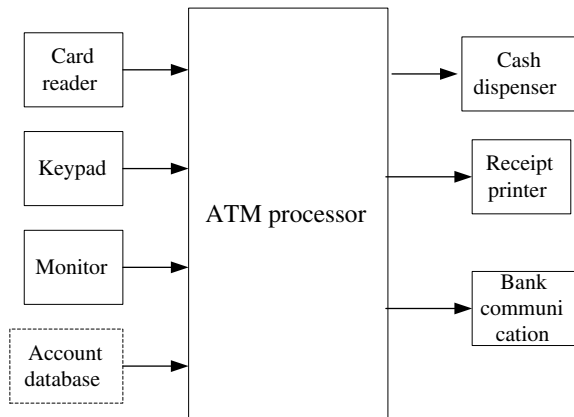
2.2.4 Supporting Actor

A supporting actor of a use case is an external actor that provides a service to the system under development. For example, a web service, a printer, etc. To carry out its job responsibility, the system formulates subgoals. A supporting actor can carry out some subgoals externally. This supporting actor may be a printing subsystem or a third-party module you are adapting to your system. It is an actor which is not part of the system under development (SuD).

2.2.5 Scope

The scope is the extent to be discussed and designed in the system to be developed. A well-defined scope sets expectations among the project stakeholders. It identifies the external interfaces between the system and the rest of the world. Before the use cases are framed, we should call upon the boundary in which the systems involved are to be developed. Else the design becomes out of bounds. As an example, in Fig. 2.2, when ATM is being designed, the dotted modules are out of the scope of design.

Fig. 2.2 Scope of the system under development



2.2.6 *Scenarios*

A scenario is a sequence of actions and interactions that occurs under certain conditions. Each scenario is a straight description of one set of circumstances with one outcome. A use case collects scenarios. Each scenario contains a sequence of steps showing how their actions and interactions unfold. Each scenario or fragment starts from a triggering condition that indicates when it runs and goes until it shows completion or abandonment of the goal it is about.

The primary actor has a goal. The system should help the primary actor to reach that goal. Some scenarios show the goal being achieved, some end with it being abandoned. Each scenario contains a sequence of steps showing how their actions and interactions unfold. A use case collects all those scenarios together, showing all the ways that the goal can be accomplished or fail.

2.2.7 *Levels*

When a problem is complex, the concept of hierarchy is essential to solve the problem. Divide and conquer is a famous concept in computing algorithms. Hence, when writing a major goal, it can be divided as subgoals and each subgoal is handled at one level below.

At the top level, there will be only a few use cases for the entire SuD. There may be only one use case even. An example is the ATM operation.

The second level is still a high level, providing an overview and summary of goals. This level may have unit-level operations. Examples would be a cash transaction, repair and service, cash replenishment, etc.

The third level is usually created for more detailed implementation of modules with several scenarios with success and failure scenarios to be handled. As an example, ATM cash transaction is expanded into multiple uses cases, viz., user authentication, balance inquiry, cash withdrawal, cash deposit, etc.

The lowest levels are subfunctions which are common re-usable use cases by upper levels. Card sensing, logon, bank communication, card dispense, etc. are some low-level use cases required by upper-layer use cases. These use cases are “included” or “referenced” in upper-level use cases (Fig. 2.3).

2.2.8 *Use Case Entities and Their Relation*

Figure 2.4 explains the relation across different use case entities. The figure explains the inheritance (Is-A) relation across these entities. The most generic entity is the actor. An actor has one or more behaviors. Behavior can be explained as the way the actor reacts to certain inputs. An actor is a person or an abstract entity like a black

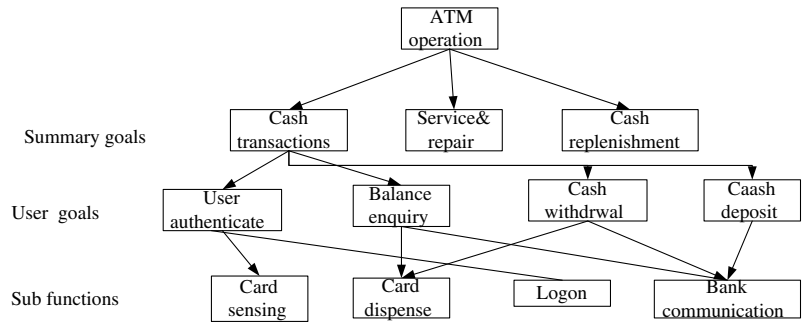


Fig. 2.3 Use cases at different levels

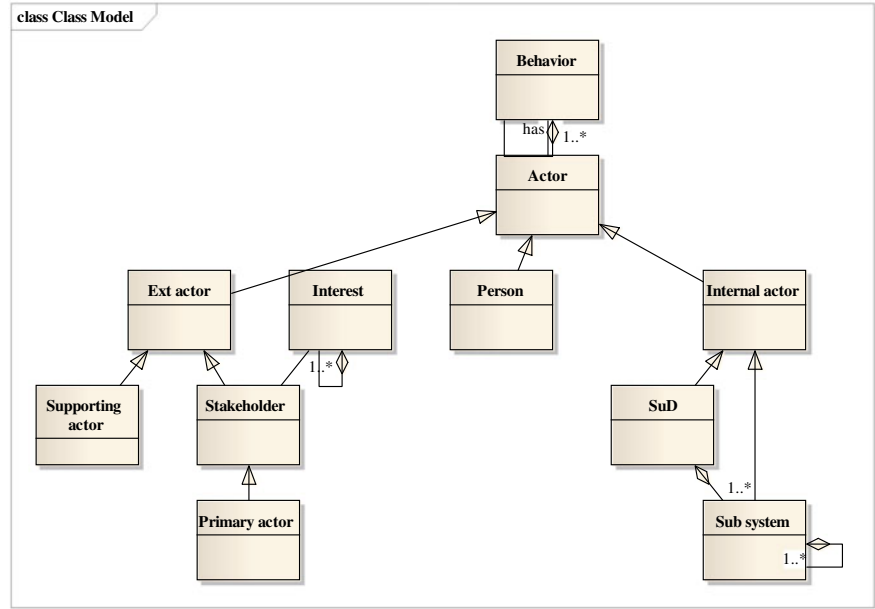


Fig. 2.4 Is-A relation across use case entities (Courtesy Cockburn (Writing effective use cases))

box or a software module or which has finite behavior. Explaining in another way, any system having known behavior can be called an actor. Actors can be classified as external or internal actors. An internal actor is a constituent part of the system under development. The system under development itself is an actor as it has a known behavior. A system has multiple subsystems aggregated and each one has its own behavior. The granularity can be further extended till the subsystem is represented by multiple objects and each object is an actor with known behavior.

External actor is an actor which is not the system under development. The stakeholders, the external systems, the operators of the system, the users of the system

can all be classified under external actors. In one way the external actors consume the behavior of the internal actors.

Under the external actors, the stakeholder is an important actor entity. The stakeholder has one or more interests. A primary actor is a stakeholder who triggers use cases. The concept of a supporting actor is nothing but a submodule or module or an actor who is out of the purview of the current development. This actor’s behavior cannot be changed. Mostly the supporting actors are used to consume their behavior. Some modules which are external to the system under development which are borrowed and whose behavior is well-known can be classified under supporting actors. Examples can be a DSP module in a camera design.

2.2.9 When Are We Done?

Below is the list of actions to be completed before we successfully complete the use case analysis for the project.

- Named all the primary actors and all the user goals with respect to the system.
- Captured every trigger condition to the system either as a use case, trigger, or extension condition.
- All possible success and failure scenarios are dealt with.
- Written all the user-goal use cases,
 - Each use case is clearly enough written so that the sponsors agree they will be able to tell whether or not it is fully dealt with.
 - The users agree that they perceive the proposed system’s behavior.
 - The developers agree they can actually develop that functionality.

2.2.10 Standard Use Case Template

This is one of the templates proposed by Alistair and followed in the majority of projects. There is no standardization. So you can alter as per your requirement.

USE CASE #	<the name is the goal as a short active verb phrase>
Goal in context	<a longer statement of the goal in context if needed>
Scope and level	<what system is being considered black box under design> <one of: Summary, Primary Task, Sub function>
Preconditions	<what we expect is already the state of the world>
Success end condition	<the state of the world upon successful completion>
Failed end condition	<the state of the world if goal abandoned>
Primary, secondary actors	<A role name or description for the primary actor> <other systems relied upon to accomplish use case>

(continued)

(continued)

Trigger	<the action upon the system that starts the use case>	
Description	Step	Action
	1	<put here the steps of the scenario from trigger to goal delivery, and any cleanup after>
	2	<...>
	3	
Extensions	Step	Branching action
	1a	<condition causing branching>: <action or name of sub.use case>
Sub-variations		Branching Action
	1	<list of variations>
Related information	<Use case name>	
Priority	<how critical to your system/organization>	
Performance	<the amount of time this use case should take>	
Frequency	<how often it is expected to happen>	
Channels to actors	<e.g. interactive, static files, database, timeouts>	
Open issues	<list of issues awaiting decision affecting this use case>	
Due date	<date or release needed>	
...any other management information...	<...as needed>	

2.3 Best Practices

- List which actors and their goals will the system support. It should be comprehensive. Must be framed with the close association of end-users (Larson 2004).
- Sketch the main success scenario of all use cases. All the pre and postconditions must be thought of at this stage.
- Brainstorm all failure conditions. All possible success and failure scenarios and how they have to be acted upon must be thought at this stage. These activities must be done through brainstorming.
- Write how the system is supposed to respond to each failure.
- Keep the GUI out. Use cases do not specify the user interface, data formats, data design, etc.

- Make the use case easy to read. Structured way as shown in the templates tailored will make them readable.
- Work breadth-first.

2.4 Summary

A good start for a system design is to have very crisp and unambiguous specifications. This is the most difficult job. Practically, neither the customer nor system designer can do it alone. Customers have the domain knowledge and vaguely know how the system is visualized. Developers know how to develop but have little knowledge of the domain. Use case design bridges the gap and robust specifications can be formulated. As discussed in the wheel and spoke diagram, good use cases are the highway for a successful project. Vouching again with my practical experience!

2.5 Further Reading

For detailed understanding and to practice, read the book by Alistair Cockburn titled, “Writing effective use cases,” Addison-Wesley (2000). Most of the CASE tools provide support in developing use cases. Any book on UML also covers developing use cases but not to an extent in the book named as above. This topic can be appreciated when you take up a real-world project, write use cases, and then derive detailed requirements. Usability.gov (2021) provides excellent tips in system design.

2.6 Exercises

Note: When you are doing the use case exercises, form a group of two. One will take the role of end-user/customer and the other will be a developer. The customer can totally change the broad features given below and add novel features. Below are a few to make you start.

1. A vending machine has to be developed with the following features. Write detailed use cases for this project.
 - The system accepts three (one rupee) coins one after the other.
 - If the total time of dropping the coins exceeds one minute, all pending coins will be released.
 - The system validates each coin as and when it is dropped. If a coin is invalid, all pending coins will be released.
 - The system releases the item by operating a relay after the final validation of the three dropped coins.

- Inter arrival of coins can be as low as 1 sec.
 - The system can accept coins and validate the coins concurrently when the item is being released.
2. A stopwatch has to be designed with broad features as below. Write the use cases.
 - Two-digit display of seconds.
 - Resets to zero if it exceeds 99.
 - Has two input buttons: reset and start/stop.
 - When the reset is pressed count resets to zero and starts counting.
 - The start/stop button toggles to stop and resume the counting.
 3. An electronic door access system has to be developed. A reader is attached to each door which reads the thumb impression, validates, and sends the information to the server for registration. The server accepts messages, registers access, and acknowledges to the reader. Around 100 such readers are served by the server.
 4. An electronic voting machine has to be developed. The machine should sense the fingerprint of the person. It should have the facility to select from four candidates.
 5. A remotely controlled trolley (RCT) moves on a straight path. When it is ON it moves from stop 1 to stop 4 in the forward direction and from stop 4 to stop 1 in the reverse direction. It stops for 2 min at each stop and **resumes** motion. When it reaches stop 4, it reverses its motion automatically. The RCT senses the arrival of a stop from stop sensors and parking position from the Park sensor. RCT receives two commands from remote user PARK and START as two messages. When a PARK message is received, the RCT moves to the park position and switches off its motor. When a START message is received, RCT puts on the motor and resumes its motion. The RCT has a controller unit (CU) which has the following functionality: (a) receives the two messages (details of media and communication can be ignored); (b) senses the park and stops; (c) takes a decision on motor movement, by turning the motor ON/OFF and setting the direction Forward/Reverse; identifies the actors, stakeholders, and top-level use cases; expands one of the use cases with detailed success and failure scenarios; represents each use case as a structured template (Fig. 2.5).

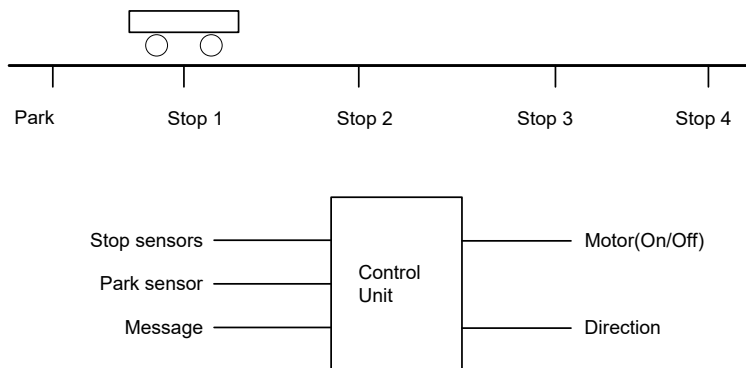


Fig. 2.5 Remotely controlled trolley (RCT)

References

Alistair Cockburn A (2000) Writing effective use cases. Addison-Wesley
Improving user experience usability.gov (2021)

Larson E, Larson R (2004) Use cases: what every project manager should know. Paper presented at PMI® Global Congress 2004—North America, Anaheim, CA. Newtown Square, PA: Project Management Institute

Chapter 3

Models and Architectures



Abstract Once you have framed the use cases and then made detailed requirements, you jump to the design of the system. The question is how do we represent the design. There are three basic representations that are used in the design process. One or more is required, based on the scope of design. The first one is behavioral representation. The system is represented as a black box. The behavior of the box is represented as a function of inputs and outputs. The second one is a structural representation where black boxes are shown interconnected without describing the functionality of each block. The third one is a physical representation where physically the organization and connectivity are described. Model is an abstract version of representing the physical problem, do any analysis, and derive results. You can transform a physical model into an abstract model, do the analysis, and transform it back to the physical model. Hence, modeling is an excellent mechanism for problem-solving. Once the design is modeled and proven, the system is implemented using appropriate architecture. This section covers several models used in embedded systems design. Broadly they are classified into state-oriented, activity-oriented, structure-oriented, data-oriented, and heterogeneous models. State-oriented models, viz., finite state machines, Petri nets, and hierarchical FSMs, are covered in Sects. 3.3–3.5. Activity-oriented models, viz., data flow models are covered in Sect. 3.6; control flow graphs are covered in Sect. 3.7; structure-oriented models are covered in Sect. 3.8, data-oriented models, viz., ER diagrams are covered in Sect. 3.9. Section 3.10 covers heterogeneous models, viz., OOP and program state machines. This chapter covers extensively most of the models used in embedded system design. These models help in understanding, organizing, and defining the system's functionality. These are abstract models. Depending on the complexity of the system, the designer may choose a subset of these models in defining and analyzing the system. Once you define the system, we need tools to verify the proposed model's behavior. Here, executable specification languages come into play. In the next chapter, we will deal with the specification language characteristics, and a couple of executable specification languages to verify model behavior.

Keywords Model · Architecture · State oriented models · Finite state machines · Petri nets · Activity oriented models · Data flow models · Control flow graphs · Structure oriented models · ER diagrams · Heterogeneous models · Program state

machines · Reachability · Boundedness · Liveness · Control/data flow graph ·
Object oriented model · Encapsulation · Inheritance · Polymorphism

3.1 Representation of a Design

3.1.1 Behavioral Representation

In this representation, the design is viewed as a black box. The functional representation of the box is not specified. However, the behavior of the box is represented as a function of inputs and outputs. As an example, if you want to design a logic shifter (Fig. 3.1) the behavior is represented in terms of the input shifted by the number of bits.

Another example is a multiplier (Fig. 3.2) designed to multiply two numbers a and b , which are the inputs. Here the behavior is represented as output $c = a * b$. A complex system can be represented as multiple functional blocks where the inputs and outputs of these black boxes are interconnected to finally generate the desired functionality of the given inputs. Hence behavioral representation is hierarchical. You can go on representing the functionality into multiple blocks to an extent where each box is implementable.

3.1.2 Structural Representation

In this representation, the black boxes are shown interconnected without describing the functionality of each block. As an example, if you want to design an 8 * 8-bit multiplier, the blocks are shown interconnected to generate the desired functionality (see Fig. 3.3).

Fig. 3.1 Bit shift logic

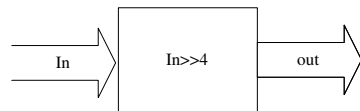
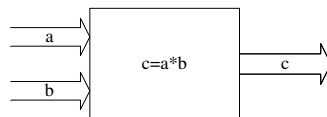


Fig. 3.2 A multiplier



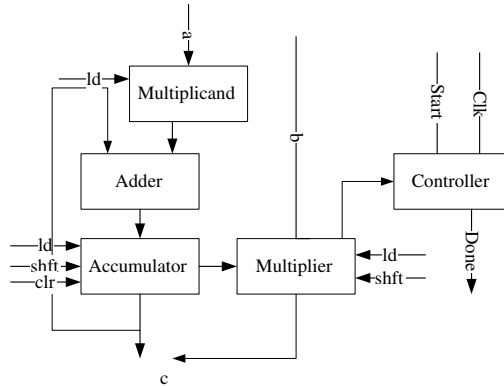


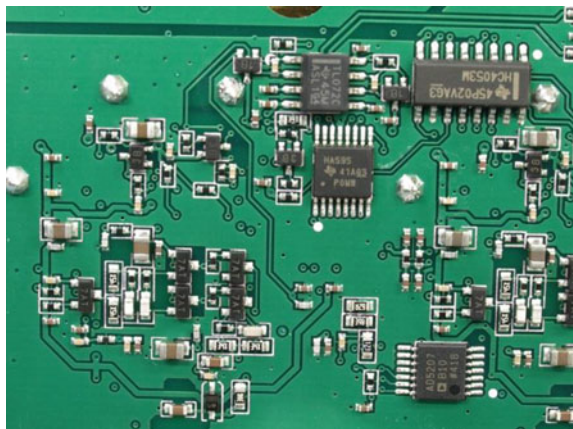
Fig. 3.3 Structure diagram of simple multiplier ($c = a * b$)

3.1.3 Physical Representation

In a system, all electronic and mechanical components are physically organized and connected. As an example, when a printed circuit board (PCB) is to be fabricated, the layout of the IC chips and the copper connectivity in the layer become the physical representation (see Fig. 3.4).

In order to design a system irrespective of electronic or mechanical or any micro-level (integrated circuits) or macro-level (PCB) device, all the three ways of representations as explained above are essential in the design process. If, as an example, we want to design a microprocessor-based board, we will start with the behavioral representation and define the black boxes with each block. Further functionally is divided hierarchically to the smallest unit of the block by which it is readily implementable. Once you verify the overall functionality through these interconnected

Fig. 3.4 Physical layout of a PCB (“Rigol DS1054Z Oscilloscope Teardown PCB” by eevblog is licensed under CC BY 2.0)



blocks, you represent the design structurally how they have to be interconnected and modularized. The next step is the physical layout of the system of these implementable modules on a PCB. The interconnection thus realizes the final product of our interest. With this introduction to the design process, we will discuss the models and architecture.

3.1.4 What Is a Model?

Let us understand the necessity of modeling in solving any practical problem by an example. Let us say you have to travel from one place to another place in a city. You have a physical map only where the roads are shown interconnected. You want to find the shortest path of moving from your source to the destination. You can start thinking brutally by finding out all the possible routes, computing their distance, and deciding the best path. That's not the way anyone will do. The correct approach is you have to use some computational algorithm to derive the correct route. You will start exploring the mechanism for this and you come across a graph model to solve the problem by using an algorithm. A graph is constituted by a set of vertices that are connected by certain edges. The algorithm finds the best path from one vertex to the other through the edges. Now you have the problem of converting your roads on the map to edges and the crossings to vertices. This transformation of a map to the graph is done for doing routing analysis because the analysis is possible only on the graph model.

Now let us introduce the term model, which means to identify a particular domain in which analysis is possible. So we will extract a graph model from the map. We will start making a graph from the map by keeping the crossings as the vertices and each road connecting the crossings as edges. What we did in this process is we created a graph model from the map model. After we compute the optimal path using the algorithm on the graph model, we have to re-transform the vertices and edges of the graph model to crossings and roads on the map so that we know the path that we have to follow. In summary, if you want to do a certain analysis of a physical problem, you transform the physical model to another appropriate model where you can do the analysis and re-transform the results to the physical model. Thus, models play a major role in every aspect of design and analysis.

The explanation given above is useful to understand what a model is. Model is thus an abstract version of representing the physical problem, do any analysis, and derive results. You can transform a physical model into an abstract model, do the analysis, and transform it back to the physical model. Hence modeling is an excellent mechanism for problem-solving architecture.

3.1.5 Case Study: 3-1

Someone wants to construct a house with broad specifications as below. He has to frame the topological placement of rooms with proper access. They need one living room (L) and dining room (D) and a kitchen (K). All guests come to the guest room (G) and have access to the living room only. The living room has access to a toilet (T3). Two bedrooms (B1 and B2) have exclusive access to toilets (T1 and T2). Both B1 and B2 have access only from L through a lobby (LB) if necessary. They need a strong room (St) which has exclusive access from one of the bedrooms (see Fig. 3.5).

Based on the broad specifications we have to place the rooms optimally and provide access (through doors). Let us model the problem by mapping each room as a vertex and connectivity to adjacent rooms through edges. The graph looks as shown in Fig. 3.6. You can verify the specifications from the model defined below.

Once the connectivity and accessibility are established, let us map the connectivity by placing the rooms adjacently and keep doors to map the edges (see Fig. 3.7).

Fig. 3.5 The list of rooms needed

K	L	D	B1	B2	St	G	LB
T1	T2	T3					

Fig. 3.6 Connectivity graph

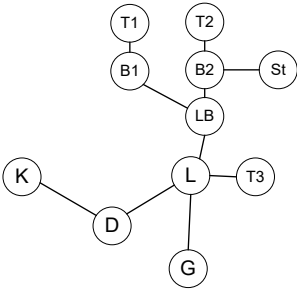
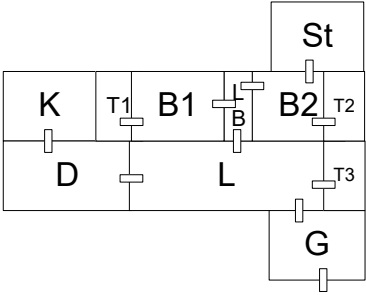


Fig. 3.7 Implementation from connectivity model



Though the dimension of the problem here is small, this model can be applied to big industries and complexes. This case study thus explains the effectiveness of model-driven designs.

3.1.6 What Is Architecture?

After you analyze the problem by transforming the physical problem to a model and getting back the results, the problem has to be implemented using the analyzed model. While a model is an abstract way of analyzing the problem in a domain, the real implementation is done by selecting suitable architecture. As an example, I have modeled a 64-bit multiplier by writing an algorithm and functionally verifying it on a computational model (a software program). The next step is to realize the device for implementation. You can implement it in multiple ways.

One simple mechanism is to implement the model through the software on a processor and get the results. Another way of implementation is by using discrete devices (IC chips), making a PCB, and achieving the results. Another approach is to design a simple sequential machine with freezed control logic to implement the same. Another approach is to implement the same in a programmable logic device. These are all different architectures by which a specific model is implemented. The model remains the same but implementation is done in various ways. Once we understand the concept of a model and architecture, we will further dwell onto the relation between these two. Figure 3.8 illustrates three different architectures for implementing a multiplier as explained above.

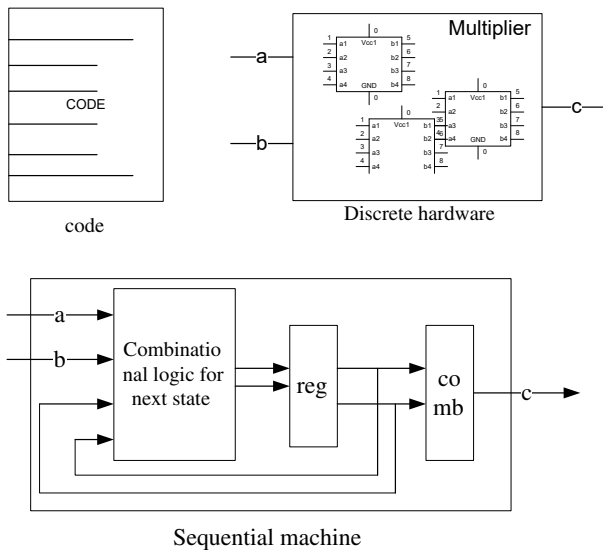


Fig. 3.8 Different architectures for a multiplier

3.1.7 Relation Between Model and Architecture

Model provides a way to analyze the problem and design in an abstract way. Each type of problem needs a specific type of model to analyze and get the results. As an example shown in case study 3-1, if you are constructing your house and you want to provide rooms like the kitchen, restrooms, bedrooms, and living rooms, you have to provide a connectivity across them. The analysis here is through a topological model. Once the analysis is done and you decide the positions where to place the rooms you will enter into the architectural aspects of implementing the house. You can implement the construction as a mud house or a brick house or precast structures, and so on. Similarly, if you want to provide lighting in an auditorium, you will use an illumination model and decide the position of light sources to get uniform illumination. Similarly, if you want to provide safety features in a complex building you will use evacuation models to design the location of access points.

The same architecture is suitable for implementing certain models. Designing and manufacturing technologies will have a great influence on the choice of architecture. Models can be specified, executed, and analyzed in different languages. A language can capture different models. Verilog is an example of synthesizing and analyzing the behavior of an electronic circuit. Implementation of the analyzed VHDL code can be done on discrete devices, ASIC, PLD, or FPGAs. The graph-theoretic model, which we discussed above, can be implemented in software or purely by mathematical analysis. If it is implemented in software, it is called a software model, whereas the one implemented through mathematical analysis is an analytical model.

Designers choose different models in different phases of the system design. As an example, acoustic models analyze different types of materials for providing good acoustics in auditoriums, evacuation models compute optimal access points for congestion-less evacuation in complex buildings, hydrological models provide solutions for proper water flow, and terrain models provide solutions to ground-related heights, slopes, and contours of the ground, visibility, etc. Hence models allow representing different views of a system thereby exposing its different characteristics.

In summary, models are a set of functional objects and rules for composing these objects. They are used for describing the system. Different models represent different views of the system, thereby exposing different characteristics. As an example, if a PCB is designed, the thermal model gives a view of the heat generated and the way it gets dissipated. The testability model provides the extent to which a system can be testable.

3.2 Model Taxonomy

3.2.1 State-Oriented Models

A system will always be in one stable state. The system switches from one stable state to another based on an allowable input event. The system generates output based on the input, during this state transition and switches to the next state. A finite-state machine (FSM) is an example of this type of model. Other models like Petri nets and hierarchical FSMs exist which are based on the states, transitions, and inputs.

3.2.2 Activity-Oriented Models

The whole system is modeled as a set of activities. An activity accepts the given data, processes, and generates the output. The output data is input to other activities. Effectively the activities are organized in such a way that the input to the system gets processed by orchestrated set of activities and finally generates the output. This is very akin to the way raw material is processed by different jobs in a workshop. The data flow model is an activity-oriented model.

3.2.3 Structure-Oriented Models

These models explain the structure of the system. It explains how the internal subsystems are interconnected to achieve the desired functionality. It does not explain the activity of each internal subsystem. The behavior of the system is not defined. Schematic diagrams and system block diagrams are some examples.

3.2.4 Data-Oriented Models

Defines all the data entities in the system, their relation, and the properties of each entity. ER diagrams are examples. These models are useful in data definitions of the entire system from the specifications. This model becomes the basis to design database schemas or complex data structures and persistence mechanisms in the system.

3.2.5 *Heterogeneous Models*

These models represent the data entities as objects. It associates the object's behavior for input events and the way the objects are related to each other. A very good example of this is object-oriented paradigm, control/data flow graphs, program state machine, etc. Every system has three basic properties, i.e., data, activity, and control. This model is more close to real-world entities and is hence used in modeling very frequently.

3.3 Finite-State Machine (Mealy) Model

We use the term “machine” for the system you want to represent by convention. Every machine will be in one stable state or the other (2011). Stable means that it gets transitioned to another stable state when a valid input is applied; else it remains in the same state indefinitely. The machine thus moves into multiple states based on the inputs given to the machine. It is the designer's task to define the possible states of the machine. The designer has to decide what possible valid inputs to the machine are. The next step is to define the behavior of the machine, how it transits its state for each valid input. If there are M states and N inputs, $M \times N$ is the total transitions that are to be defined. The system's definition is complete only when all possible transitions are defined. The machine generates output during the transition when it moves from one stable state to the next one. In Fig. 3.9 the machine has three states: q_0 , q_1 , and q_2 . Possible inputs are 0 and 1, hence possible transitions are 6 and possible outputs are 6 (Fig. 3.10).

The same is represented as below:

Let the states are represented as $s = (s_1, s_2, s_3)$.

Let the inputs represented as $I = (i_1, i_2, i_3)$.

Let the outputs are represented as $O = (o_1, o_2, o_3, o_4)$.

Then:

Fig. 3.9 Finite-state machine (Mealy model)

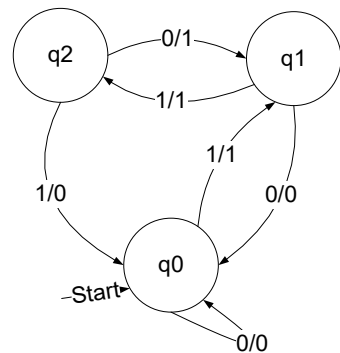
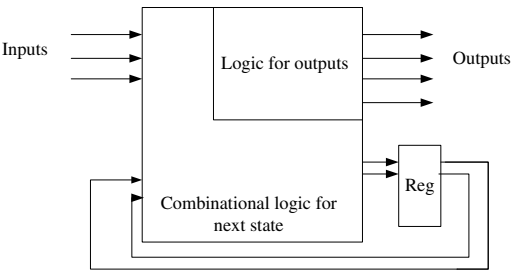


Fig. 3.10 Realization of Mealy model



F: $S \times I \rightarrow S$ (for a given input, machine transforms from current to another state) It can remain in the same state also. F is the state transition function.

H: $S \times I \rightarrow O$ (for a given input and current state, it transforms to another state generating an output O). H is the output function.

Mealy FSM is a very versatile model to define a complex machine’s temporal behavior, which makes you think of all possible states and behavior with all possible inputs. This helps in making a robust design. If you have not considered all possible inputs in a state, the system’s behavior is not defined. The possible faults in a system are mostly due to this. Also, the FSM is useful to analyze how to reach a state. This is reachability analysis.

3.3.1 Finite-State Machine (Moore Model)

Input	Current state	Next state	Output
Reset		A	0
0	A	B	0
1	A	C	0
0	B	B	0
1	B	D	1
0	C	E	1
1	C	C	0
0	D	E	1
1	D	C	0
0	E	B	0
1	E	D	1

In this model, see Figs. 3.11 and 3.12. The output is a function of the state. State-based FSM may require few more states because in transition-based FSM multiple arcs with different outputs may be pointing to the same state. Mealy has different outputs on arcs (n^2) rather than outputs on states (n). Moore machines are safer to use

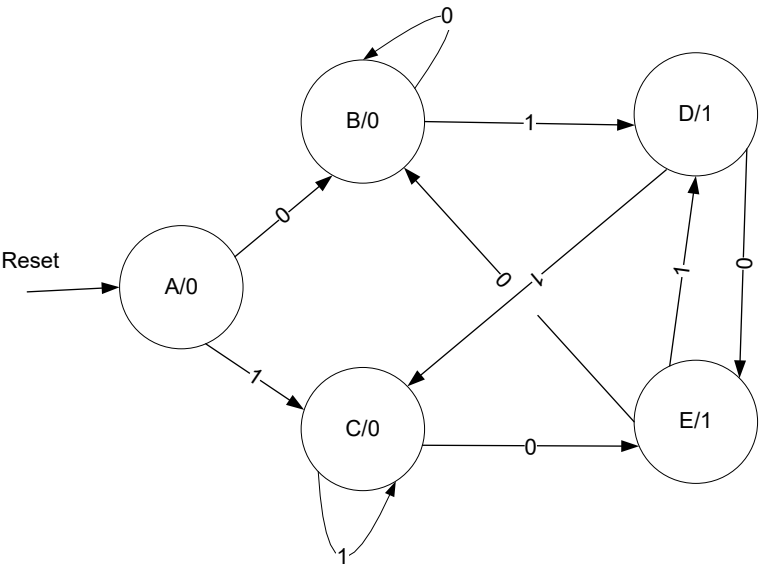


Fig. 3.11 Moore model finite-state machine and state transitions in a table

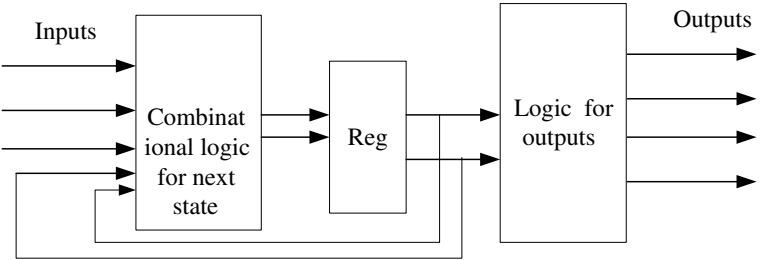


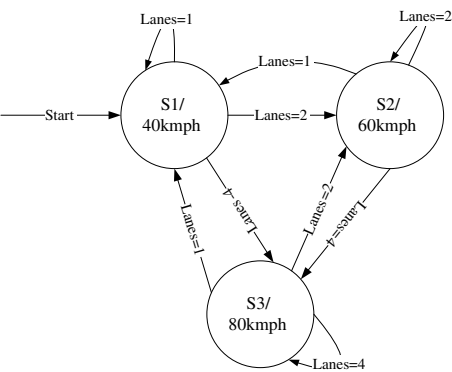
Fig. 3.12 Realizing Moore model as a sequential circuit

as the outputs change at the clock edge (always one cycle later). In Mealy machines, input change can cause output change as soon as logic is done—a big problem when two machines are interconnected—asynchronous feedback.

3.3.2 Finite-State Machine with Data Path (FSMD)

FSMs do not use arithmetic expressions or data values as input which causes the state transitions. In FSM the output in a state is only a value. In FSMD, the states and transitions may include complex expressions and these expressions may include complex inputs and generate complex outputs and also include variables. If the input

Fig. 3.13 FSM with the data path



expression is satisfied, the state gets transferred to next. By such representation, FSMD is suitable for control and computation-dominated systems.

As an example, let us say a vehicle *s* is moving on a different type of road with different lane widths. The vehicle speed changes based on the lane width. The vehicle travels on 1, 2, and 4 lanes. The vehicle speeds are 40, 60, and 80, respectively. In this example, the state of the machine is in which lane it is moving. The inputs are lane widths. The outputs are the speed of vehicles. The FSMD for this example is shown in Fig. 3.13.

3.3.3 Case Study: 3-2

We wish to implement a finite-state machine (FSM) that recognizes two specific sequences of applied input symbols, namely four consecutive 1s or four consecutive 0s. There is an input *w* and an output *z*. Whenever *w* = 1 or *w* = 0 for four consecutive clock pulses (sensed at raising edge), the value of *z* has to be 1; otherwise, *z* = 0. Overlapping sequences are allowed so that if *w* = 1 for five consecutive clock pulses the output *z* will be equal to 1 after the fourth and fifth pulses. Figure 3.14 illustrates the required relationship between *w* and *z*.

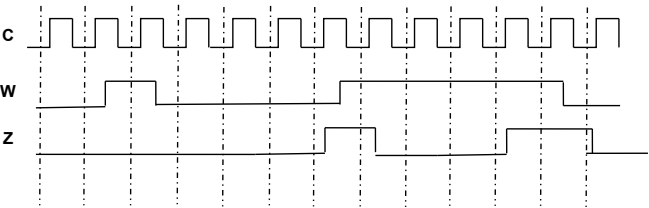
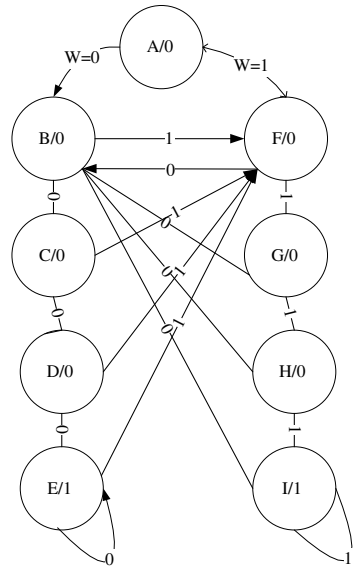


Fig. 3.14 Pulse sequence

Fig. 3.15 FSM for pulse detection



Solution

The module is designed to exist in any of the nine states in the diagram in Fig. 3.15. A, F, G, H, I are used to detect 1111... sequence, whereas A, B, C, D, E are used to detect 0000... pattern in each state, and the transition conditions are mentioned as follows:

Input/output, for example, 0/0 implies that with input 0 the output is also 0 in this state.

From the diagram shown in state I after detecting four consecutive 1s, the output is one, and if the input w is one again, the output z is also one until the input is zero. This implies that this supports **overlapping sequence** detection and if in this state the input is zero, the output is transitioned to state B. The same conditions apply to zero detection in state E. In between states like G, H if the pattern breaks and in input w zero appears the state is transitioned to B to detect consecutive zeros.

3.3.4 Case Study: 3-3

A pulse detector system is to be designed. The input to the system is a pattern of pulse sequence as in Fig. 3.16 with 60 ms ON, followed immediately by 40 ms OFF (used in old telephone pulse dialing systems). The system counts such continuous pulses to detect a dialed digit. For example, if a digit 7 is dialed, 7 pulses each of 60–40 ms pattern are generated. The quiescent state of the input pulse level is low. Use a finite-state machine to design the system.



Fig. 3.16 Pulse sequence

Let us solve this problem using FSM (see Fig. 3.17). Let us sample the data at 1 ms interval.

The problem is to detect a 60 ms pulse of level one followed by a 40 ms pulse of level zero. Once this pattern is detected, this is counted as one digit. As the data is in the range of milliseconds, sample the data at 1 ms interval by using a periodic clock and keep the state machine updated for each 1 ms based on the data is 0 or 1. At any time the sample will be either in 60 ms range or in the 40 ms range.

When the system is idle, i.e., when there is long silence of 0 or 1, the system will be in the *idle* state. Now the strategy is to observe when the data moves from the idle state and start counting 1 s whenever it is in the *in-60 ms* state. When you get 1, increment the count by 1 until you reach 60. During this state, if a zero is detected, the pulse is less than 60 ms and hence the digit becomes zero and the count also resets to zero. It will go back to the ideal state. This path is shown in the state diagram by the transitions going from idle to in-60 ms.

Once a count of 60 has been reached in *in-60ms* state, you expect it will move to *in-40 ms* state in which it does a similar count of 40. Due to some fault, if one is found before reaching a count of 40, the state gets back to Ideal. If a 1 is detected after the count is 40, it has successfully completed 60 ms and 40 ms. The digit count gets incremented by one. This is shown in the state diagram as a successful transition from in-40 ms to in-60 ms state. You find in this example, the output is an expression; the state transition is also a conditional expression.

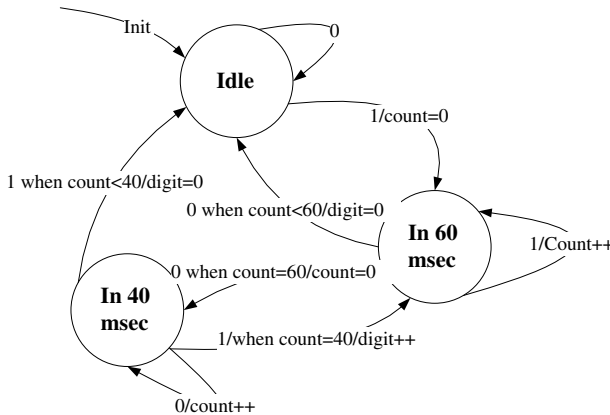


Fig. 3.17 FSM for digit detection

3.3.5 Summary: Finite-State Machines

- The most popular model used for modeling reactive systems with finite behavior.
- Also, the temporal behavior can be captured as a suitable model.
- Explicitly useful for control-dominated systems. Any real-time system can be modeled.
- Can be used in non-reactive systems also. If a system is modeled with a finite set of states and possible transitions, the behavior in all possible states and all transitions can be exhaustively designed to avoid any failure of the system because of missing events. Thus exhaustiveness of the design can be managed.
- They are the basis for more extensive models, like hierarchical concurrent FSMs, program state machines, etc.
- The only limitation is that FSMs cannot represent concurrency.

3.4 Petri Nets

Petri nets were invented in August 1939 by **Carl Adam Petri** at the age of 13 for the purpose of describing chemical processes. Today, it is the most powerful tool to design, analyze, and validate distributed systems; in fact, any concurrent system.

Petri net is a state-oriented model (see Fig. 3.18). Here, the state is not a lumped value but distributed across possible satisfying conditions represented as tokens and possible transitions which can occur. Let us study the basic entities, their properties, and behaviors. A *place* is represented by a round circle like an empty plate. A token is very similar to coins or tokens used at counters. One token represents a condition. When this token is placed in a place, one condition is satisfied for the place. A place may require one or more tokens to enable it for a *transition*. Hence every place has a count of enabling tokens. The places are connected by a transition. The transition is represented as a small flat strip. The transition can hold one or more input places and one or more output places.

Fig. 3.18 Petri net entities

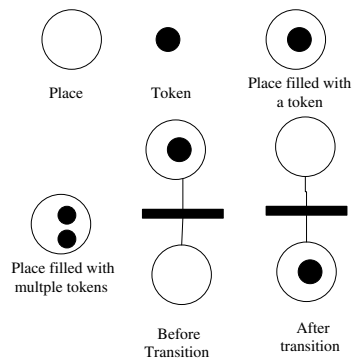
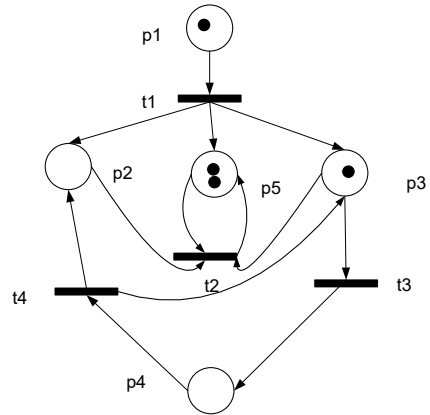


Fig. 3.19 Representation of a sample Petri net



A transition *fires* when all the input places are full of enabling tokens. When the transition fires, the enabling tokens from input place(s) are removed and one token is placed in the output place(s). The structure of a transition with input places and out places is called a *marking*.

Let us understand the implication of the above story. It depends upon how you interpret this. In system design, the input places hold multiple conditions. When a place is filled with the enabling tokens, this condition is ready. Similarly, other conditions may get satisfied. When all the input places are ready, the transition to which all these are connected will fire, meaning that the event has fired. The transition represents an n event. This causes the output places to get filled, which may be the conditions for a different transaction. This becomes a sort of chain reaction representing multiple activities being executed when the inputs are ready. As multiple transitions can fire concurrently, concurrent systems can be modeled.

One marking (structure of input places, transitions, output places, and tokens as shown in Fig. 3.18) can be thought of as one state of the system.

Let us study the Petri net in Fig. 3.19. It has five places and four transitions.

P = places (p1..p5)

T = Transitions (t1..t4)

M1 = Marking initial = {1, 0, 1, 0, 2}

M2 = M1->t3(t3 fires) = {1, 0, 0, 1, 2}

M3 = M2->t4(t4 fires) (1, 1, 1, 0, 2}.

3.4.1 Modeling of System Characteristics by Petri Nets

Varieties of systems from chemistry to computer sciences to safety-critical systems can be modeled using Petri nets. We are covering the tip of the iceberg in this chapter (Murata 1989; Gajski and Vahid 2009).

Figure 3.20a models sequential actions. When transition t_1 fires, then only t_2 can fire. Figure 3.20b models non-deterministic branching. Either t_1 or t_2 can fire. Certain non-deterministic events can be modeled by this pattern.

Figure 3.20c models synchronization. Here t_1 can fire when both the input places have enabled tokens. This is used for synchronizing two processes. Let both the processes execute at their own pace. Only when both the processes complete execution and place respective tokens in the two places, the subsequent process can start. This is like two persons plan to meet at one place and take a cab. One has to wait for the other to arrive.

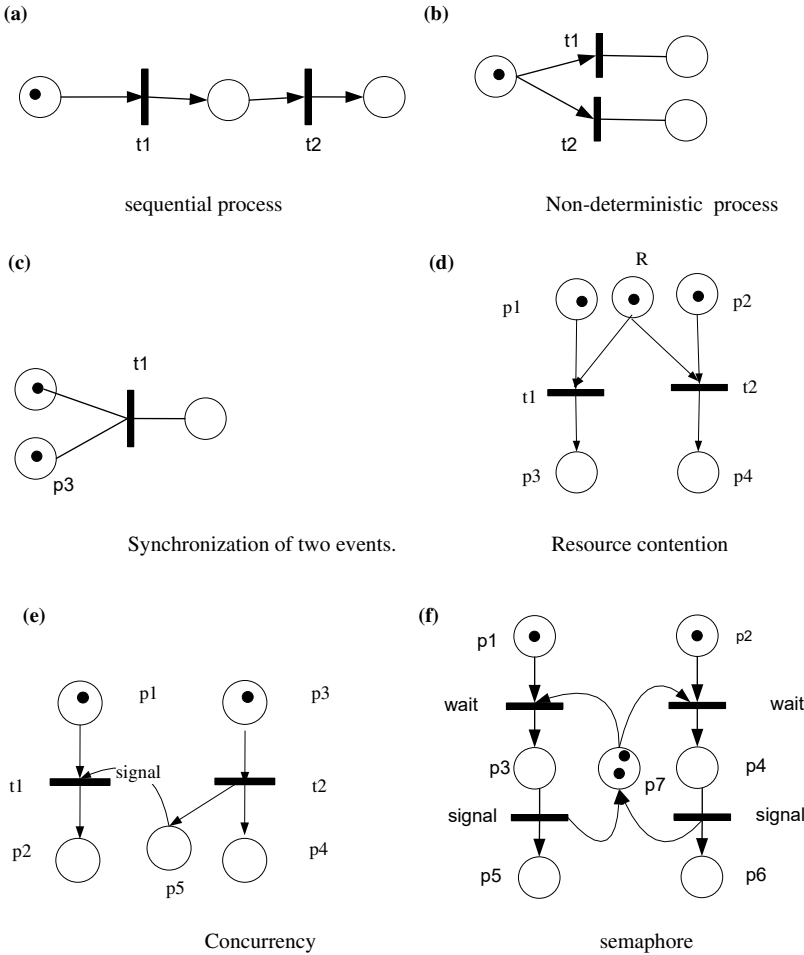


Fig. 3.20 a Sequential process. b Non-deterministic process. c Synchronization of two events. d Resource contention. e Concurrency. f semaphore

Figure 3.20d models resource contention. There is one available resource (R). When this resource is available (a token placed in R) and the other places p1, p2 have tokens, it means two processes are waiting for this resource. So either t1 or t2 fires. When t1 fires, p2 will not get the resource and vice versa.

Figure 3.20e models concurrency. The right side is a process RP (p3) fired on t2. Once RP fires it leaves a token in the middle. Left process p1 waits for this token and fires when this token is ready. It means LP always follows the execution of RP. When multiple processes are running concurrently they cannot execute randomly. The prerequisite processes have to be executed and make the data ready for the next process to execute. We will deal with this concept further in the real-time system design chapter.

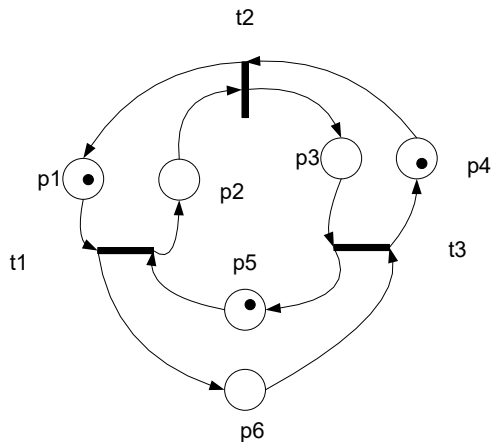
Figure 3.20f is a simple representation of a multi-valued semaphore. It has two resources shown as tokens in p7. Left and right processes p1 and p2 wait for the availability of tokens in p7. Once p1 gets a token, execution proceeds (p3 has token). The token is released by the transition at the signal.

3.4.2 Properties of Petri Nets

3.4.2.1 Reachability and Deadlock

A Petri net is reachable from a marking M_s to M_d if after a finite number of transitions, the net moves from M_s to M_d . Reachability can be analyzed by building a reachability graph (see Fig. 3.21). The initial marking is set as M. When t1 fires the new marking is M' . When t2 fires the marking changes to M'' . When t3 fires the marking returns back to M. If none of the transitions can fire and change the marking, the system is deadlocked. A marking roughly represents the state of a system.

Fig. 3.21 Markings and reachability



Initial marking $M = \{1, 0, 0, 1, 1, 0\}$

$M' = M \rightarrow t_1 = \{0, 10, 1, 0, 1, 1\}$

$M'' = M' \rightarrow t_2 = \{1, 0, 1, 0, 0, 1\}$

$M''' = M'' \rightarrow t_3 = \{1, 0, 0, 1, 1, 0\} = M$

A reachability graph is formed with nodes representing markings and edges representing transitions between two markings. The graph is constructed by finding all possible transitions from the initial marking. This gives a set of markings reachable from the initial one, then all possible transitions from the previously discovered markings, and so on. This may lead to extremely large graphs.

3.4.2.2 Boundedness

As discussed above, a transition fires when the input places have required enabling tokens. Once it fires it places one token in the output places. That becomes one marking. It means the output place has one condition ready and can be processed by the next transition. It generates the next marking. Let us say, the next transition is not ready to fire; the input place goes on accumulating tokens and the tokens increase indefinitely. The number of tokens gets unbounded, i.e., the number of tokens in each place should be bounded by k . This is similar to congestion in networking. The system becomes unsafe. Such behavior of the system can be validated by the pertinent model. A Petri net is structurally (inherently) bounded if all of its initial markings are bounded. In other words, no reachable state can at any place contain more than k tokens. This property is useful for modeling limited (bounded) resources. Figure 3.22 shows an unbounded net where the number of tokens in place p_2 increases by one for each cycle; see M_1 and M_4 .

The sequence of markings is listed below:

$M_1 = (1, 0, 0, 0, 0)$

Fig. 3.22 The Petri net is unbounded at place p_2

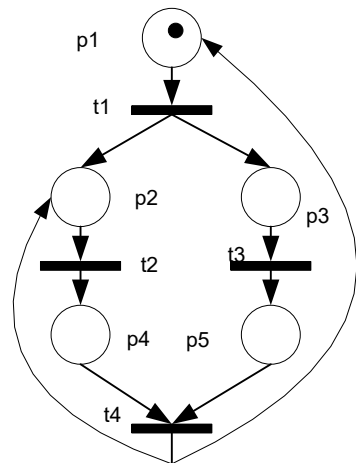
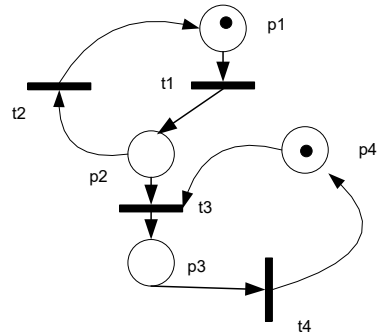


Fig. 3.23 Deadlock

$$M2 = (0, 1, 1, 0, 0)$$

$$M3 = (0, 0, 0, 1, 1)$$

$$M4 = (1, 1, 0, 0, 0)$$

$$M5 = (0, 2, 1, 0, 0).$$

3.4.2.3 Liveness

Liveness is a property that retains the property of each transition to fire after a sequence of firings by other transitions. Liveness is equivalent to deadlock-free. As a corollary, if a transition cannot fire indefinitely, the Petri net is not live. It means that the condition to fire a transition is missing which indicates a fault in design. A Petri net is structurally live, if any initial marking is live. Liveness may be used to model the occurrences of deadlocks. In the net in Fig. 3.23, the markings are (1, 0, 0, 1) (0, 1, 0, 1) (0, 0, 1, 0), and (0, 0, 0, 1). When the marking reaches to {0, 0, 0, 1} no more transitions can fire and the system is deadlocked.

3.4.3 Case Study: 3-4

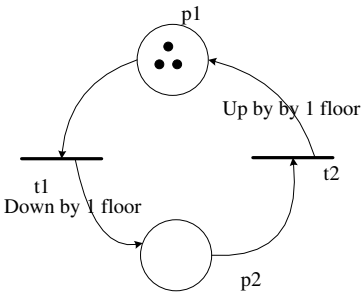
An elevator serves three floors (ground/first/second/third). At any time, the elevator transits up or down by one floor. Represent the state and transitions of the elevator as a pertinent model which can be useful to find the following:

- In which floor the elevator is?
- Can it go upward?
- Can it go downward?

Solution:

See Fig. 3.24 for lift status. A simple way to model the problem is to have two transitions. T1 fires when it goes down by one floor. T2 fires when it goes up by one

Fig. 3.24 Lift status through Petri nets



floor. When the bottom place p2 is empty it is on the topmost floor. Similarly, when all the tokens are in p2, t1 cannot fire to go down. The total tokens cannot increase more than the number of floors. The difference in the number of tokens in p1 and p2 shows the position of the lift.

3.4.4 Case Study: 3-5

Design a communication across two systems using wait for ack communication protocol. Represent the same in the Petri net.

Solution:

The diagram Fig. 3.25 represents the communication across the two processes.

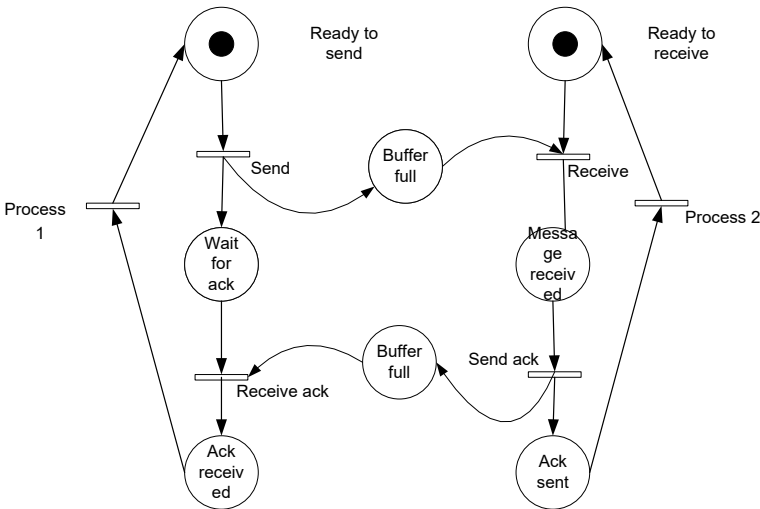


Fig. 3.25 Communication protocol model using Petri nets (Murata 1989)

Process-1 prepares the message when it received the previous acknowledgment. The message will be posted into transmit buffer when the buffer is empty. When process-2 is ready to receive the next message and the buffer is full, the receive process will start. When the message is received, an acknowledgment is sent to the transmitter. As process-1 is waiting for acknowledgment and the buffer is ready with acknowledgment, the ack is read and updates its status to ack received. The ack buffer becomes empty. The next message is processed by p1 after reading the ack received. Hence the synchronization is established across the two processes.

Process-1: The transition fires when the token is received into ack received. And places tokens in messages ready to send.

Send message: The transition fires when a message-ready token is placed. This places token in message buffer full and also in wait for ack.

Receive message: The transition fires when the receiver-ready token is placed and the buffer-full token is placed. After reading the message both tokens are removed and the message received token is placed.

Send ack-transition is fired when the message received token is in place. Sets the ack buffer token and ack sent token.

Process-2 transition starts processing the received message when ack sent token is in place. Once processed, it places a ready-to-receive token in place.

Receive ack-transition is fired when the ack buffer has token in place and p1 is waiting for ack. After receiving the ack, the received token is placed so that p1 can process the next message.

Thus both the processes synchronize their activities by using tokens as status placeholders.

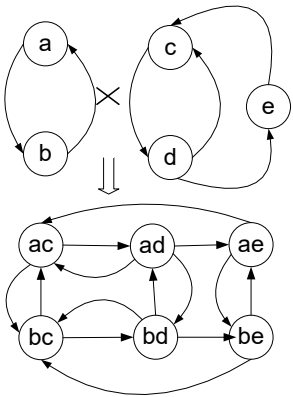
3.4.5 Summary: Petri Nets

- A powerful model to represent concurrent activities.
- The model can be used to verify safe liveness, boundedness, and reachability.
- Several safety-critical systems are modeled using Petri nets to verify their safety.
- The basic Petri net model is extended to timed Petri nets, Petri nets with priority, colored Petri nets, etc. to map certain class of problems.
- Major applications include protocol specification, verification for behavioral properties, reachability analysis, and performance analysis.
- The model is flexible. For different applications, the places and transitions may have different interpretations by which you are mapping the problem to Petri nets.
- As the related concurrent processes grow, representation and analysis can be too complex.
- The only demerit is the model is flat and tough to represent hierarchically.

3.5 Hierarchical Concurrent FSMs

Let us start with an example as shown in Fig. 3.26.

Fig. 3.26 Explosion of states due to concurrency



Let there be two processes p1 and p2 which are running concurrently. Let p1 be represented as left FSM with two states and p2 be represented as the right FSM with three states. The two FSMs are meaningful if you consider them independently. But when a parent process P has these two processes running concurrently, the overall state of P is represented in the FSM in the lower portion of Fig. 3.26. The concurrent FSM has multiple states and multiple transitions. The number of states and transitions explodes with the number of processes and their state machines in the order of $p1 \times p2$. So we should have a way to represent the FSM of such a machine with multiple concurrent processes. Hierarchical concurrent FSM (HCFSM) is an extension of FSM model. It adds support of hierarchy and concurrency to the FSM model (see Fig. 3.27).

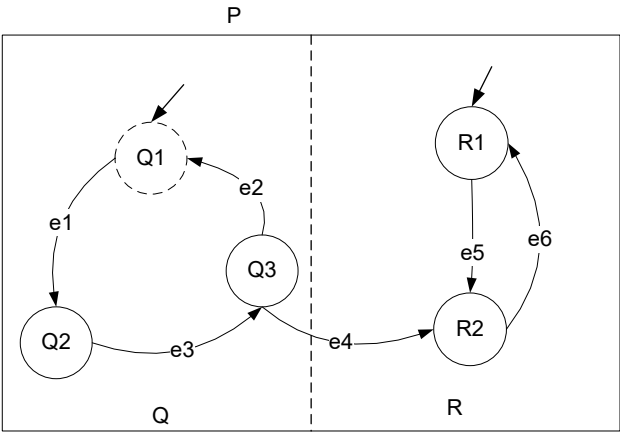


Fig. 3.27 A hierarchical and concurrent FSM

A state P can be decomposed into multiple substates (Q and R in this example). The substates execute concurrently. In the above example, substate Q has three states Q1 to Q3. The initial state for substate Q is Q1. Substate R has two states: R1 and R2. The initial state of substate of R is R1. The substates Q and R communicate by global variables. In this model, a transition occurs when an event occurs. When a parent FSM is concurrently running as two FSMs, the initial state when they are created the way they communicate across the boundary and the way they terminate are complex issues. We will cover all of them when studying program state machines.

3.5.1 Summary: HCFSM

- HCFSM supports both hierarchy and concurrence. Thus complex systems can be represented easily.
- The exponential growth of states can be avoided.
- This concentrates only on modeling control aspects and not data and activities.

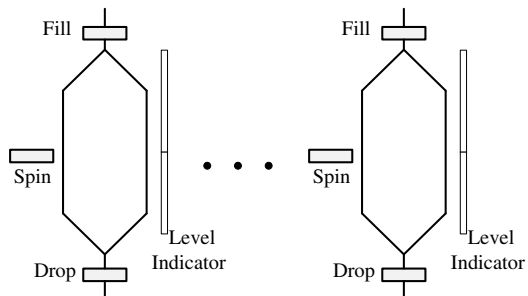
3.5.2 Case Study: 3-6

There are four milk spinning machines that run concurrently (see Fig. 3.28). The machines get filled from a reservoir (not shown in the figure) by opening the “Fill” valve. A level sensor detects whether the milk is filled up to the desired level. Once the milk is filled, the spinner is made ON by operating the “Spin” relay. The spin time is fixed. After the spin is over the “Drop” relay is operated to drop the toned milk. Drop is ON till milk is released. The main constraint is that only one machine can spin due to load conditions.

The relay operations are given below:

- Fill = ON to open the “fill” valve. A level sensor senses that milk is filled to the desired level in the machine.
- Spin = ON to spin the machine for a fixed time. The spin time is to be 10 min.

Fig. 3.28 Concurrent milk spinning machines



- Drop = ON to release the milk. The drop valve is to be open till the milk is empty.

Constraints:

- Only one machine can spin at a time.
- The machine cannot spin till the milk is to the desired level.

For representation by HCFSM, see Figs. 3.29 and 3.30.

All the machines work independently. There is no dependency except that a machine cannot spin when any other machine is spinning. This event has to be shared across the four FSMs. This can be easily done by keeping one global variable

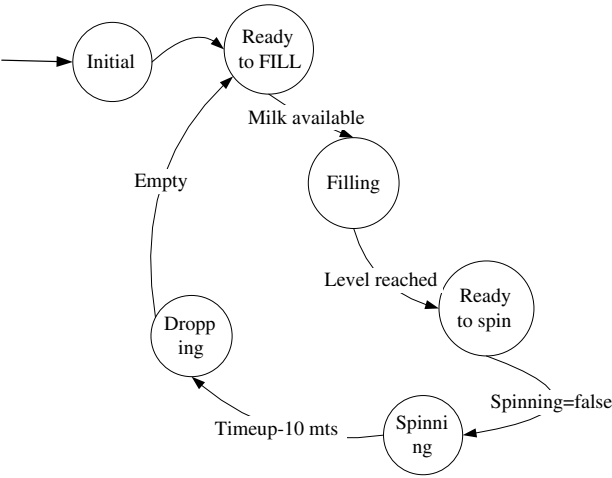


Fig. 3.29 FSM for each machine

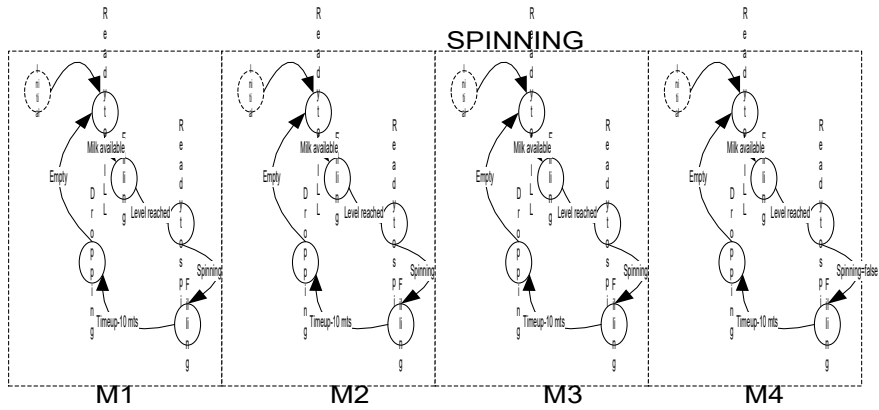


Fig. 3.30 Concurrent operation using HCFSM

shared across all four FSMs. Let the global variable SPINNING is sensed by each FSM and wait for an event spinning = false. It sets the variable to TRUE and starts spinning. Once spinning is over it sets to false.

3.6 Activity-Oriented Data Flow Graphs

An embedded system may not be processing only real-time events always in a reactive way. It will have certain components where the captured data need to be processed either offline mode or as a part of the system in non-real-time mode. This task has to be done like any data processing application. As an example, an industrial system collecting sensor data in real time needs to be processed by data analysis and extracting non-redundant data for historic storage. Such data-oriented activity is represented by data flow graphs.

Data flow graphs are very commonly used for transformational systems which process input data and generate the desired output. See Fig. 3.31, where a door access system is represented as a data flow model. Data flow graphs are not state-oriented or event-oriented. This model is a graph. It is represented by nodes and directed edges. The node represents either input data or output data or an activity. These are connected by edges. An activity node processes the data from input nodes and posts over the output nodes. Hence, a complex activity can be broken into multiple activities and connected through input and output nodes. This is very similar to job processing in a manufacturing workshop. The model is hierarchical as an activity can be broken down into subactivities connected by a subgraph. The model does not represent in which sequence the data is processed.

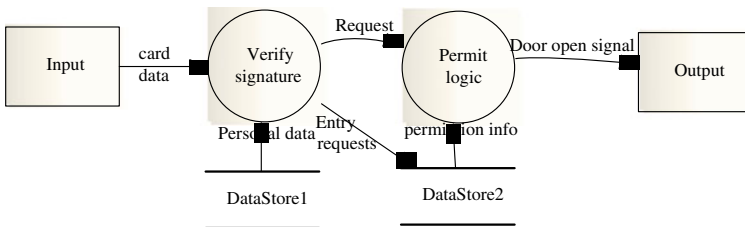


Fig. 3.31 A data flow model

3.6.1 Case Study: 3-7

A system has to be designed to monitor the temperatures of an industrial process. Detailed specifications are given below. Draw a data flow graph.

1. The temperature sensors (TS) are intelligent devices installed close to the physical location of each process.
2. Each TS reads the temperature at a set sampling rate (each TS has its own rate) and sends data to the processing station (PS) serially. There is no acknowledgment from PS for each data sent.
3. PS sends control messages serially to the TS whenever its operating parameters have to be set.
4. The only operating parameter of the TS to be set is the sampling rate.
5. PS reads the serial data from each TS and computes the average of the last 100 samples for each TS.
6. PS monitors the upper limits for each TS and raises an alarm if high. The alarm has to be reset by the operator by pressing a push-button switch.
7. Assume there is no other user interface except a push button switch to reset the alarm.
8. The number of TS can be taken at any arbitrary value.
9. All the activities go concurrently in real time.

3.6.2 Solution

See Fig. 3.32.

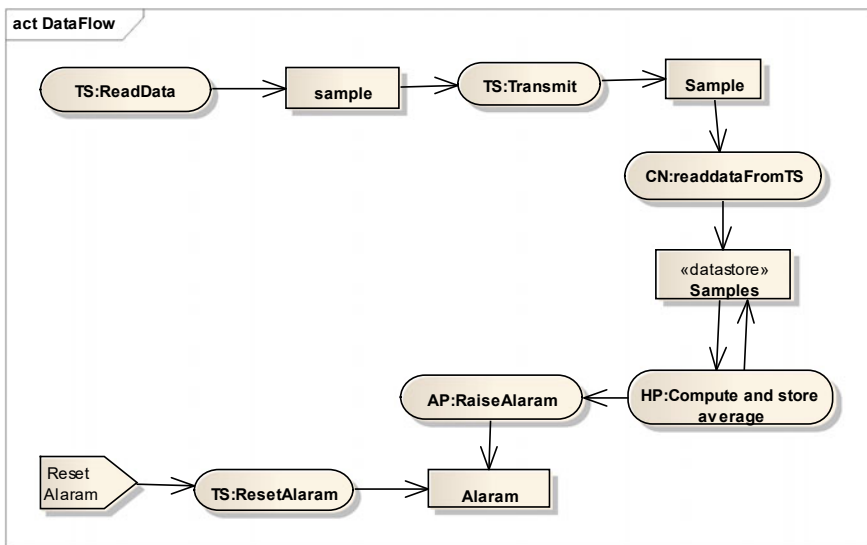


Fig. 3.32 Data flow diagram. Legend: TS: temp sensor; CN: controller; HP: history processor; AP: alarm processor

3.6.3 Summary: Data Flow Model

- Very suitable for specifying complex transformational systems. Most of the embedded systems have some components where such transformation processing is needed.
- Complex problems can be broken down hierarchically with subactivities.
- Represent problem-inherent data dependencies.
- They do not express temporal behaviors or control sequencing. It is a demerit.
- Weak for modeling complete embedded systems but used for certain components of the system.

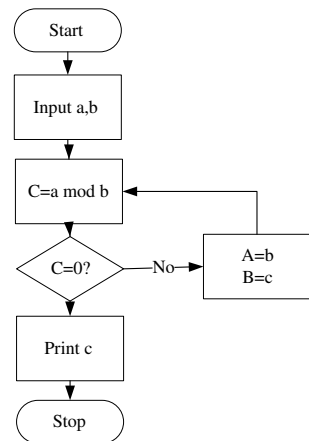
3.7 Control Flow Graphs (Flowchart)

See Fig. 3.33 for the control flow graphs. We have studied FSMs where an event is a conditional expression of data or an external event. This event changes the state of the machine. In the control flow graph, an activity completion triggers the flow of activities. When a system is viewed as a sequence of activities and the sequence is controlled by the completion of an activity, this model is used. We are all much acquainted with this as flowcharts to write sequential programming.

3.7.1 Summary: CFG

- CFG is useful when systems are designed based on certain activities, and the flow of these activities is to be controlled.
- Has no concept of state or data flow.

Fig. 3.33 A control flow graph



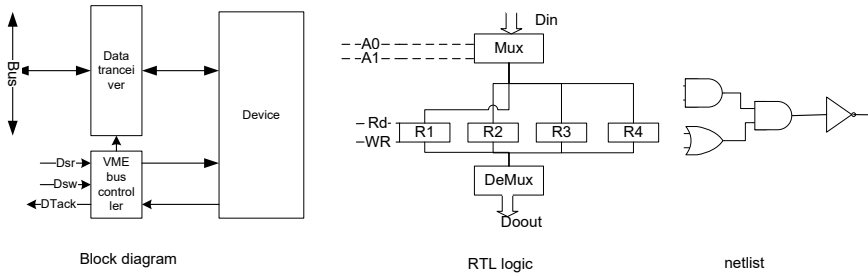


Fig. 3.34 Sample of structure-oriented models

3.8 Structure-Oriented Models

When you are designing a software module or hardware module, they have a specific structured connectivity (see Fig. 3.34). Structure-oriented models are nothing but diagrams to represent the structural aspects. Block diagrams, schematic diagrams, and interconnectivity of cells in FPGAs IC layout are all structure diagrams. Effectively, they represent a set of system components and their interconnectivity.

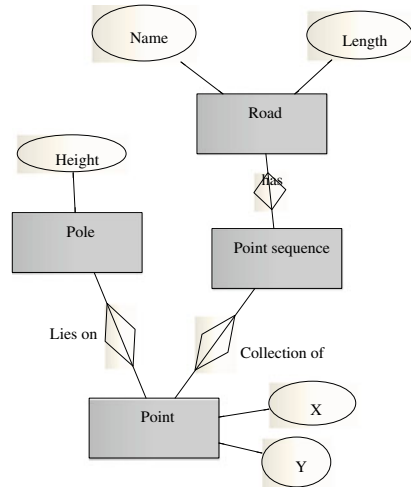
3.8.1 Summary: Structure Diagrams

- Structure diagram is very essential to describe a system’s physical structure as opposed to its functionality.
- It is useful in subsequent parts of design-like product engineering.
- Also, it is useful to decide whether the design is implementable.

3.9 Data-Oriented Entity-Relationship Model

An entity denotes an abstract or a physical real-world object. Such an entity has certain properties which are represented as its attributes. In Fig. 3.35 the rectangle blocks represent the entities; for example, a point is an entity. The connected ovals are its attributes. A point can have attributes like its coordinates X and Y. An electric pole is an entity. It has attributes like height. Certain entities have a relation to each other. In the diagram below, an electric pole “*lies on*” a point. The association is shown by a diamond block named “*lies on*.” Other entities in the diagram below are a *point sequence* and a *road*. Point sequence “*is a collection of*” points. Road “*has*” point collection. The road has attributes name and length. So relationships reveal the facts across entities. Hence ER diagram represents complex relations across data elements by mapping. Once the model is established, it can be implemented in multiple ways. ER models can be transformed by representing each entity as a table with the attributes as fields. The relation across entities becomes the relational table related by keys. So ER models are best used to model real-world data and

Fig. 3.35 An entity-relationship model



implemented as a relational database. However, the temporal behavior of the system cannot be modeled.

Here are some points for converting an ER diagram to the table. The tables can be in a relational database or even mapped to memory-based structures. Tables are mapped as flat structures. Relations are mapped as pointers. Key values can be mapped as indexes. Using these rules, you can convert the ER diagram to tables and columns and assign the mapping between the tables.

- Entity type becomes a table.
- All single-valued attribute becomes a column for the table
- A key attribute of the entity type is represented by the primary key.
- The multi-valued attribute is represented by a separate table.
- Composite attribute represented by components.
- Derived attributes are not considered in the table.

3.9.1 Case Study: 3-8

Employees work in departments; each department is managed by an employee; a child must be identified uniquely by name when the parent (who is an employee; assume that only one parent works for the company) is known. We are not interested in information about a child once the parent leaves the company. Draw an ER diagram that captures this information. The solution is given in Fig. 3.36.

Solution

See Fig. 3.36.

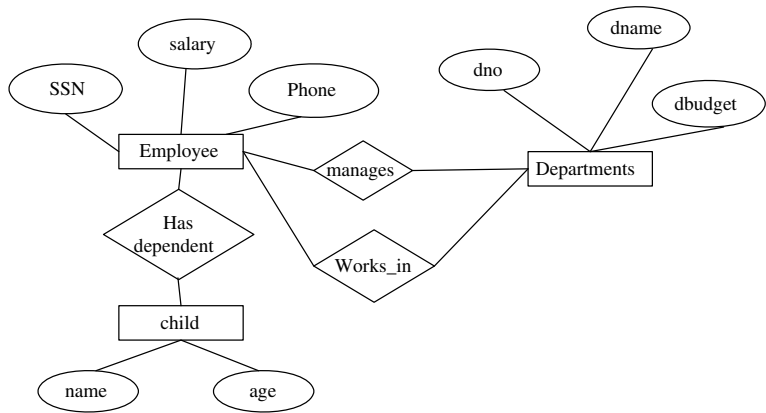


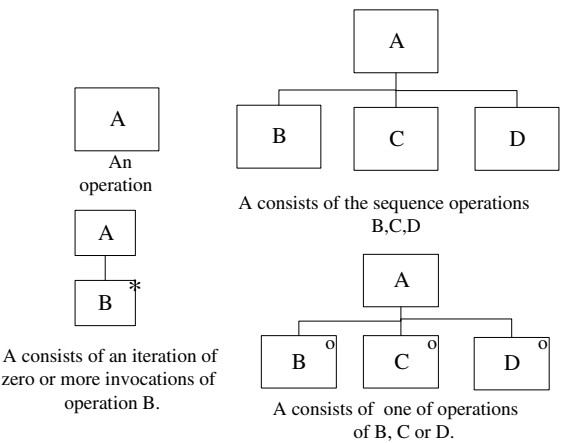
Fig. 3.36 The ER model

3.10 Jackson's Structured Programming Model

JSP is modeled toward programming at the level of control structures (see Fig. 3.37). The implemented designs use just primitive operations, sequences, iterations, and selections. JSP is a method for structured control flow. It uses diagramming notation to describe the structure of inputs, outputs, and programs, with diagram elements for each of the fundamental component types.

It structures by decomposing that data into sub-data. It forms tree-type structure. The leaf nodes become basic data types or primitive operations. Non-leaf nodes are composite types obtained through various operations like sequences (AND), selection (OR), and iteration (*). Sequences generate a type of data incorporating two or more subtypes. Selection generates data by selecting one of these types. Iteration generates data by replicating certain elements of its type.

Fig. 3.37 Jackson's structured programming model



3.10.1 *Summary: Jackson's Model*

- Suitable for representing data having a complex composite structure or structured programming components.
- This model does not describe any functional or temporal behavior of the system.

3.11 Heterogeneous Models

3.11.1 *Control/Data Flow Graph (CDFG)*

We have studied data flow graphs (DFGs) where certain activities are networked. Data is input to each activity, processed, and the output flows to other activities. In this model, the activities are not controlled by any one activity. The overall flow is hard-wired. But in majority of situations, decisions have to be taken on how the activities have to be executed. This may depend upon the type of control generated in the process itself or by any external commands. It is very much like orchestrating the musicians based on the song. The orchestrator controls the flow of data to the activities, makes the activities ON and OFF, and takes the inputs from the activities and from any external sources to control. The orchestrator explained above is the control flow graph or FSMD or FSM which we studied earlier. So, a CDFG is nothing but a data flow graph controlled by a control flow graph or a state machine. Hence, it is called a controlled data flow graph (CDFG).

Let us study this by an example in detail.

Let the simple program shown in Fig. 3.38 is to be realized. Based on the value of X either ADD activity (A1) or subtract activity (A2) has to be executed. In the figure, the CFG represented as FSMD gets two events $X > 0$ or $X \leq 0$. Based on the event, it controls the two activities A1 and A2 by enabling one of them. The enable and disable signals are fed to the add/subtract processes. Effectively the CFG is controlling the two activities A1 and A2 based on the value of X .

CDFG is a heterogeneous model combining the advantages of CFG and DFGs. The control constructs that you find in any language are mapped onto control flow nodes. The activities in the DFGs process basic blocks of data. CFGs and DFGs are connected by the control line. Based on the control line signal, associated activities get executed.

CDFGs can be used to implement complex activities and control actions required by the system. In Fig. 3.38 CFG is represented as FSMD. The FSMD responds to internal data (as events) and external events and controls the execution of DFG.

3.11.2 *Summary: CDFG*

- CDFG corrects the inability of DFG in representing the control of a system.
- It corrects the inability of CFG to represent data dependencies.
- Well suited for real-time systems and behavioral synthesis of ASICs.

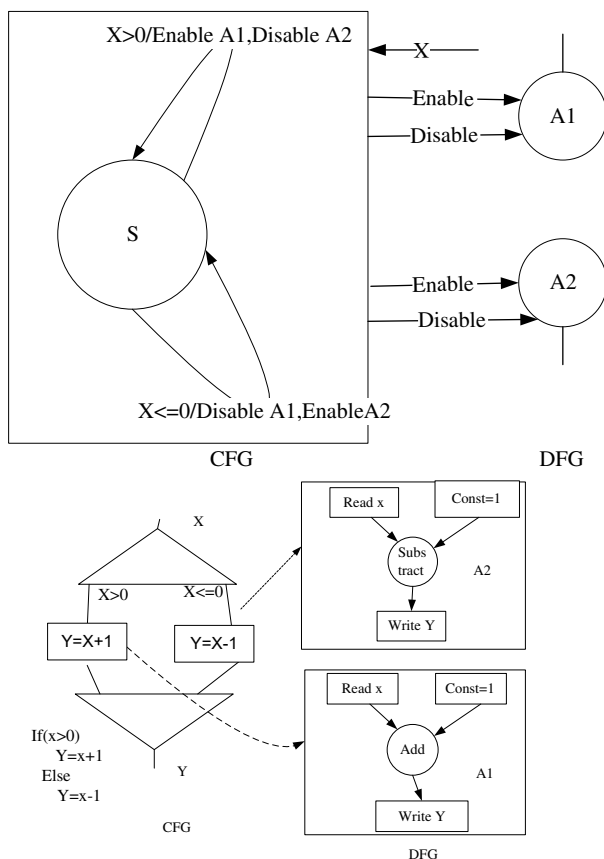


Fig. 3.38 An example of CDFG—mapping a C-program

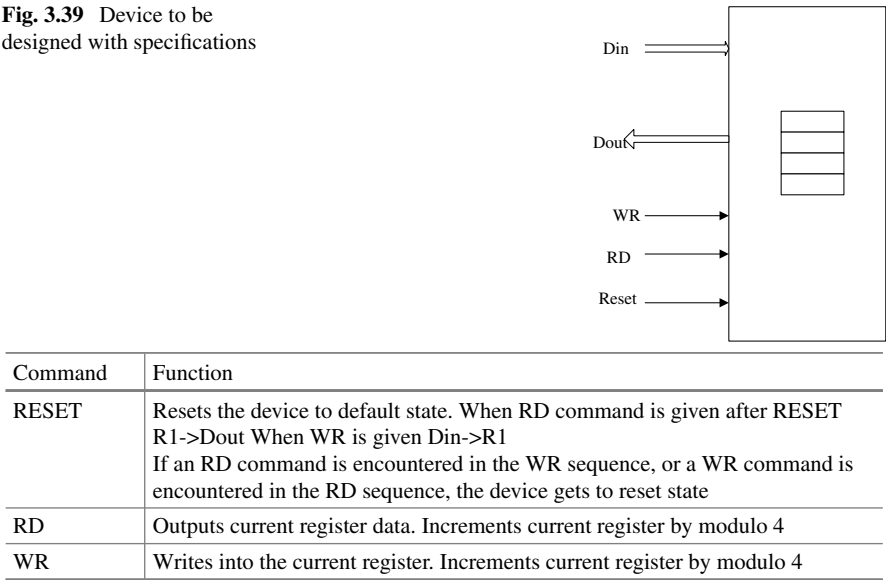
3.11.3 Case Study: 3-9

A device has four registers R1 to R4 of 8-bit width. The external interface to write and read the device is WR and RD signals, respectively. A RESET signal resets the system. Data is input and output using DIN and DOUT.

The four registers cannot be accessed randomly as there is no address input for random selection. They have to be written or read sequentially. For each WR command in sequence, it writes to the next register. Similarly, for RD command. If an RD command is encountered in the WR sequence, or a WR command is encountered in the RD sequence, the device gets to reset state and starts operations from R1. The behavior of the device for different commands is given below.

Design the device using the CDFG model systematically (starting from logic in C to final CDFG) (Fig. 3.39).

Fig. 3.39 Device to be designed with specifications



Solution

The data path contains the four registers. Din is routed using a two-bit control address generated by the controller. The output of R1 to R4 is routed to Dout using the same control address. The controller is an FSM that generates proper control address. The typical end result is shown (see Fig. 3.40).

The DFG has four registers R1 to R4. One register is selected by input mux and rd/wr signals read or write data. Written data is output from the selected register. The problem specification is to read the four registers cyclically. Similarly, write the data cyclically. Once the cyclic read is interrupted by write, the write cycle starts. This complex mechanism is managed by the control logic on the left side. The control

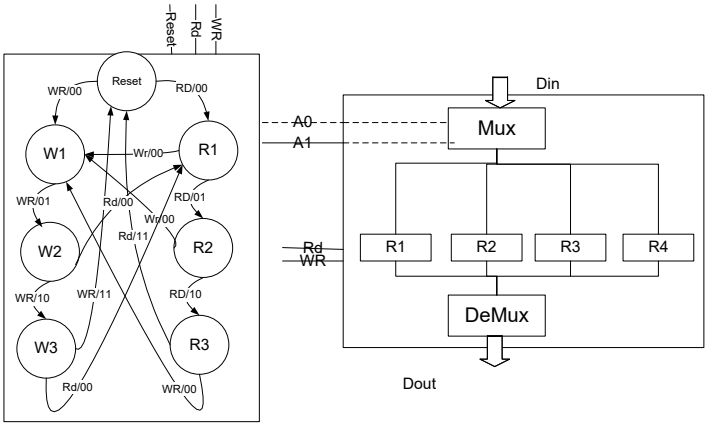


Fig. 3.40 The sample CDFG solution

logic advances its state depending on the read cycle and write cycle. R1-R2-R3-Reset is the read cycle and W1-W2-W3-Reset is the write cycle states. Address A0, A1 is placed by the controller based on the current state. If the cycle skips, the controller jumps to res, W1 or R1 by placing the first address.

You can see the power of the controller orchestrating the data flow of the system. Current processor architectures have a very similar concept of controller design and data path design.

3.11.4 Object-Oriented Model

This model is very popular and well known to all people solving real-world programs through object-oriented programming. Hence, we will touch on this topic very briefly with more case studies that are related to embedded systems. In this model, the real-world object is modeled as an object. Several objects having the same behavior are marked as a class of objects. The behavior of those objects is the same. Hence, certain activities (implemented as procedures) are assigned to the class definition. An object persists certain data (in a structured way). Hence every class definition holds data with it. This is called encapsulation. When an object is instantiated from class T, the object is created with defined data elements. As in the real world, a class of objects is related to the other class of objects through aggregation, containment. Thus the objects can use the services of related classes. The object-oriented programming approach encourages modularization, where the application can be decomposed into modules, and software re-use, where an application can be composed from existing and new modules.

Few concepts of OOP are described below (Fig. 3.41).

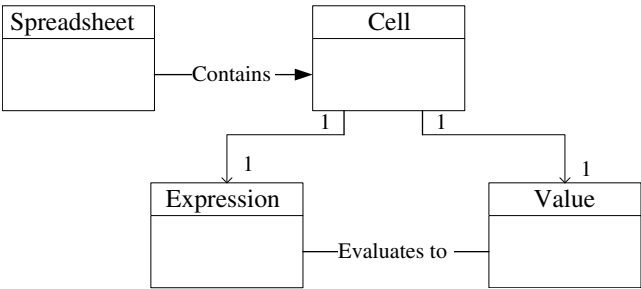


Fig. 3.41 Conceptual example of class diagram

3.11.5 Encapsulation

The object-oriented paradigm encourages encapsulation. It is a method of binding the data and methods. Encapsulation is used to hide the mechanics of the object, allowing the actual implementation of the object to be hidden so that we don't need to understand how the object works. All we need to understand is the interface that is provided for us. We can identify a level of "hiding" of particular methods or states within a class using the public, private, and protected keywords.

3.11.6 Inheritance

If we have several descriptions with some commonality between these descriptions, we can group the descriptions and their commonality using inheritance to provide a compact representation of these descriptions as "parent class." Any new class can inherit the properties and behavior of the parent class in addition to its own. This is "sub class" inherited from the "parent class."

3.11.7 Polymorphism

Polymorphism means "multiple forms." In OOP these multiple forms refer to multiple forms of the same method, where the same method name can be used in different classes, or the same method name can be used in the same class with slightly different parameters.

3.11.8 Case Study: 3-10

A customer has described the problem as a textual statement as given below. Identify the classes and their associations from the statement. Define the properties and methods of identified classes.

A digitizing tablet (DT) captures certain entities from an image by the process of digitization. Captures Electric poles as points when the mouse is clicked on a pole. It captures the coordinates of the point when pressed. Each pole's height is also entered into the system in this process. A road is captured by digitizing a sequence of points. The road name is captured in this process. System computes the length of road and stores with each road....

Solution:

See Fig. 3.42 for the OO model. The entity is a generalized class representing the physical objects on the ground. The inherited entities with different properties are

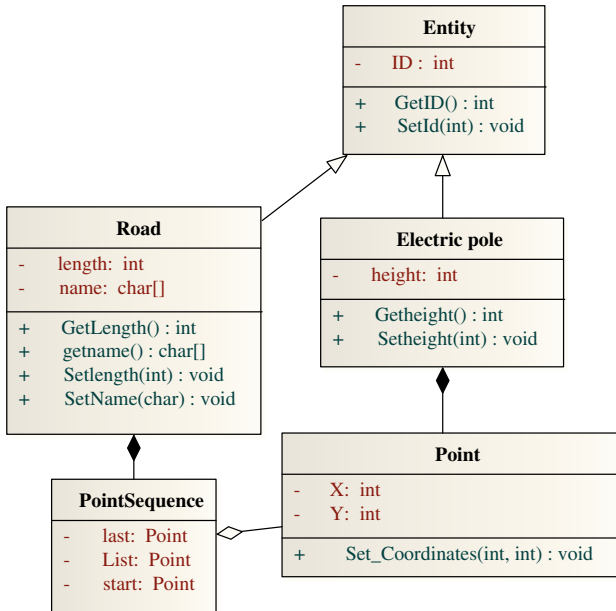


Fig. 3.42 OO model for the digitizer

the electric pole and road. The pole has height and is located at a point. Point is a generic class holding coordinates x, y as its attributes. Electric pole **has a** point in its class (shown as aggregation. It is *has-a* relation). Similarly, road class has a name, length as its properties and **has-a** sequence of points class.

3.11.9 Case Study: 3-11

A system has to be designed to monitor the temperatures of an industrial process. Detailed specifications are as follows:

1. The temperature sensors (TS) are intelligent devices installed close to the physical location of each process.
2. Each TS reads the temperature at a set sampling rate (each TS has its own rate) and sends data to the processing station (PS) serially. There is no acknowledgment from PS for each data sent.
3. PS sends control messages serially to the TS whenever its operating parameters have to be set.
4. The only operating parameter of the TS to be set is the sampling rate.
5. PS reads the serial data from each TS and computes the average of the last 100 samples for each TS.

6. PS monitors the upper limits for each TS and raises an alarm if high. The alarm has to be reset by the operator by pressing a push button switch.
7. Assume there is no other user interface except a push button switch to reset the alarm.
8. The number of TS can be taken at any arbitrary value.
9. All the activities go concurrently in real time.

Problem:

1. Draw use cases from the above description of the problem and represent them diagrammatically.
2. Represent the complete system at an architectural level (as a diagram) and explain briefly the strategy.
3. Define a structural model to represent the entities as objects and their association. Define the attributes, methods, and events of the classes.
4. Draw a data flow model for the entire process (as a diagram) and explain briefly.

Solution:

Use cases:

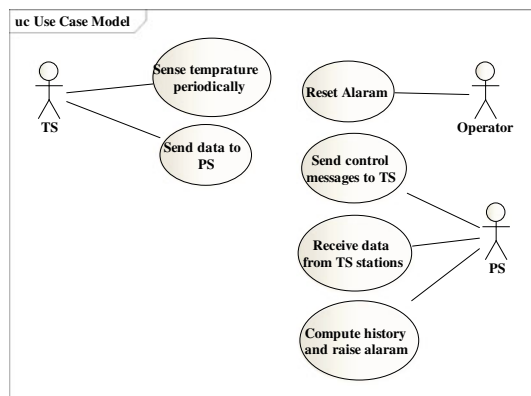
The actors are the operator, TS and PS. The typical use case diagram is as shown in Fig. 3.43.

Physical architecture

Each TS is an autonomous hardware. PS is the processing station and has an IO interface. It can be designed as a multi-processor system for higher scalability (see Figs. 3.44, 3.45 and 3.46).

For the above hardware architecture, alarm management, and history processing, communication runs on concurrent processes (threads/tasks...) and TS executes data acquisition and communication processes. Logically all these processes can be shown

Fig. 3.43 Simplified use cases for sensor system



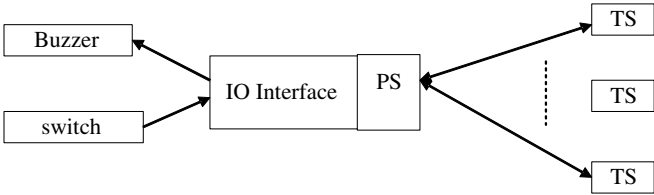


Fig. 3.44 Physical interface for the sensor system

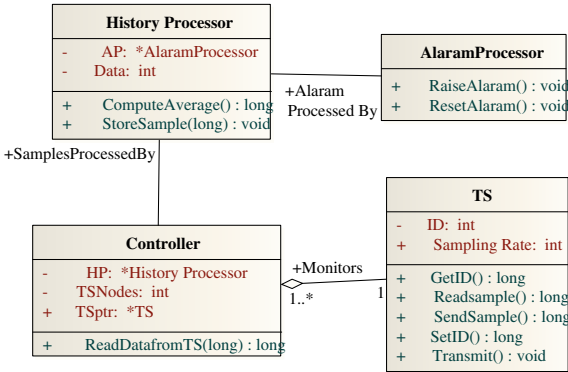


Fig. 3.45 OO model for the sensor system

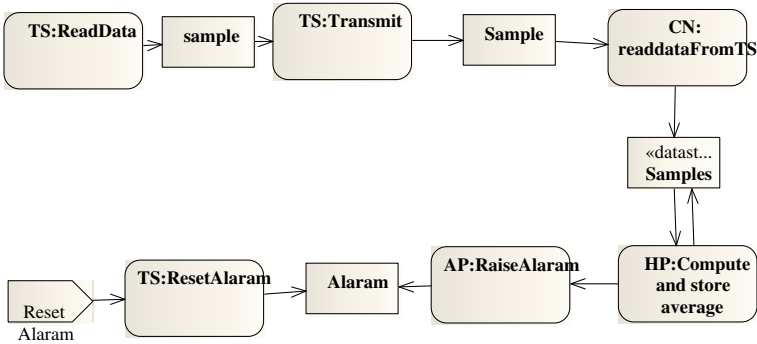


Fig. 3.46 DFG for sensor system

as a multi-tasking system. One can use any other alternative mechanism or use containers or proxy, etc.

Object model

This is a simple controller pattern. The controller sets all the TS. Receives samples. Gets stored and processed using HP (history processor). AP (alarm processor) raises and resets the alarm.

Data flow

Figure 3.46 is the data flow across the processes.

3.11.10 Program State Machines

We have studied hierarchical concurrent finite-state machines (HCFSM) where multiple activities execute concurrently. Each activity is represented by a state machine. It is a set of state machines executing concurrently. Each activity can be further broken concurrently. Such concurrent activities exchange events and data through global variables or events at the same level. Program state machines are an extension of HCFSMs, which integrate a programming language paradigm, i.e., one of the activities instanced can be a program. Each program state represents a distinct mode of computation. Like an event causes a state transition from one state to another, an event can cause a transition to execute another program. Once it is in a program state, it exits and switches to another state either by generating an event or terminating the condition of the program.

In FSM, the machine remains in one state at any time. In HCFSM, the machine will be in one state in each concurrency. Similarly in PSM, a subset of programs will be active which are the currently active states in the concurrency. A state can create multiple concurrent states where some of the concurrent states are represented as programs. Hence hierarchy, state-based, program-based states, concurrency are all modeled in PSMs. The PSM states can be classified as compound or leaf states. Composite program states can be further decomposed into either concurrent or sequential program substates. A compound state can be hierarchically broken down till a leaf level program state is reached. If the program substates are sequential, they will be active one after the other. At each point of time, only a subset of program states is active, and thus performs their computations.

Let us observe this behavior in the below models. In Fig. 3.47a p1...p3 are leaf program states. They execute sequentially p1 to p3. When p1 reaches a terminating condition p1 exits and transits to p2, and so on. After p3 completes execution the state is exited. In Fig. 3.47b the entry state fires s1 and transits around all the states depending on the events and finally when s6 reaches exit state, FSM exits. In Fig. 3.47c when the concurrent program (CP) is entered the subprograms p1...p3 execute concurrently. CP exits when all the concurrent programs reach their exit state. This is similar to the FORK and JOIN pattern.

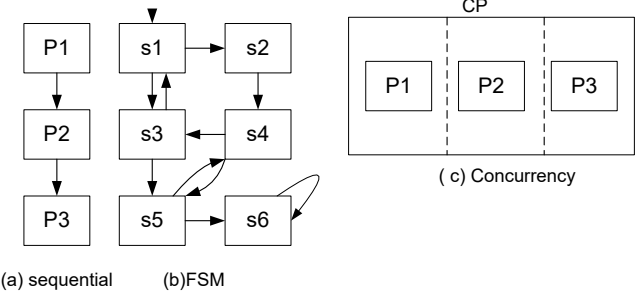


Fig. 3.47 PSM composite behaviors

Fig. 3.48 Hierarchical PSM

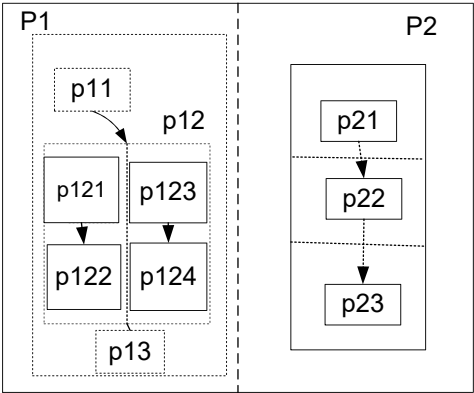


Figure 3.48 illustrates how a composite behavior is broken into multiple concurrent and hierarchical behaviors. The transitions shown above are due to data-driven or termination-driven.

See Fig. 3.49 for the concept of PSM. The state p1 forks for concurrent execution of states P11 and P12. The entry program of p11 is p111. p111 and p112 execute sequentially with transitions based on certain events. The black square shown in a state is a terminating condition. When p111 reaches the terminating condition and evt1 event occurs, the p111 state transits to p112. If a condition evt2 occurs anywhere while executing p112, it enters p111 again. The terminating condition of p112 transits to the terminating condition of state P11. When both the terminating conditions of P11 and P12 are reached, the system reaches the terminating condition of P1.

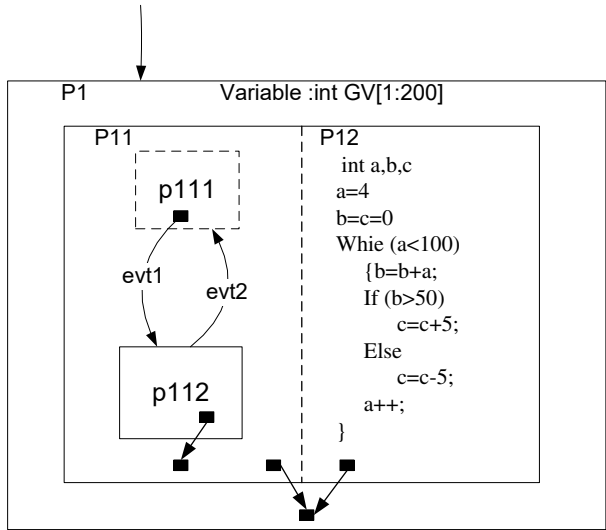


Fig. 3.49 Concept of PSM explained

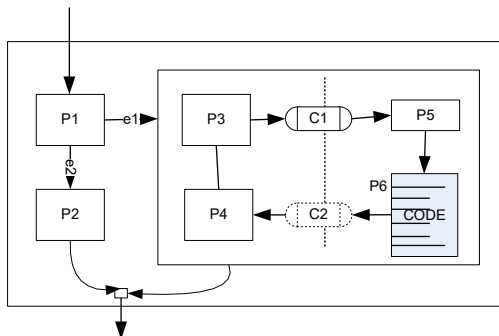
3.11.11 Communication in PSM

While designing embedded systems, one major principle to be adopted is to separate communication with computation. This has to be done by defining an abstract mechanism that provides this principle. PSM provides channels as abstract communication channels across activities. This enables the implementation of communication at different custom levels of abstraction.

A channel implements a certain interface that defines the communication primitives. These abstract communication primitives can be used by behaviors that represent the computational parts of the design. Communication is initiated on ports which can be part of behavior.

In Fig. 3.50 two concurrent programs (P3, P4) and (P5 and P6) execute concurrently. P3 is connected to P5 by channel C1. P5 receives data using `c1.receive(..)`.

Fig. 3.50 Channels in PSM



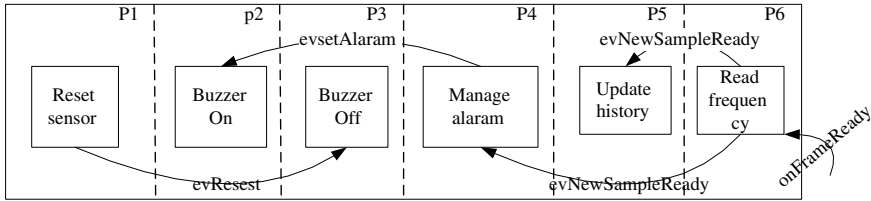


Fig. 3.51 PSM model for sensor subsystem

P6 after further processing sends to P4 by channel c2 as `c2.send (...)`. PSM allows representing each entity as a state, process, or program code. In this example, P6 is represented as code.

3.11.12 Case Study: 3-12

Refer to case study 3-11 of industrial temperature sensor and processor.

Represent the complete process as a program state model with all behaviors (as a diagram) (Fig. 3.51).

Solution

The problem is a real-time multi-tasking system which we will study subsequently. Main task forks into six programs P1 to P6 executing simultaneously. P6 is responsible to set the frequency of reading temperature samples. When *onFrameReady*, the sample is read by P6 and generates an event to P4 and P5 to the alarm manager and history processor. Both of the processes do their responsible jobs. P5 processes the sample and updates the last 100 samples. P4 manages alarm and executes its logic on whether to generate an alarm. If the alarm has to be generated, it generates *evsetAlarm* event to P2 which sets the buzzer as ON. When the user physically operates the reset button, P1 sends *evreset* event to P3 which resets the buzzer. The terminating conditions are not shown as all the processes are run indefinitely.

3.12 Summary: Models and Architectures

We have gone through different models to represent the behavior of a system. Different models help in defining certain characteristics only. A complex embedded system has to be characterized by state, concurrency, data flow, hierarchy, intercommunication, and control flow aspects. Models help in understanding, organizing, and defining the system's functionality. These are abstract models. Depending on the complexity of the system, the designer may choose a subset of these models in defining and analyzing the system. Once you define the system, we need tools

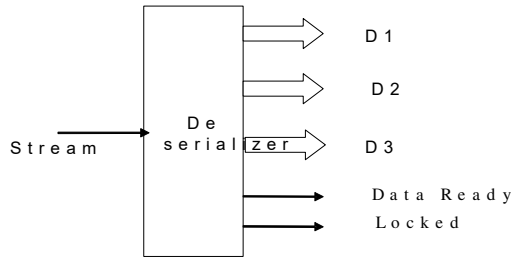
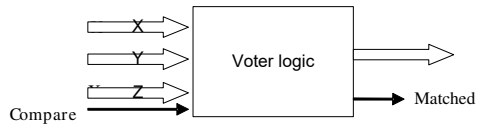
to verify the proposed model's behavior. Here, executable specification languages come into play. In the next chapter, we will deal with the specification language characteristics and a couple of executable specification languages to verify model behavior.

3.13 Further Reading

For further reading on models please refer to multiple books written by Gajski and Vahid (2009) and Wolf (2008). On Petri nets please go through the excellent paper by Murata (1989). Other excellent books covering these topics are Marwedel (2006) and Lavagno (1998). For practicing the designs, you can select any CASE tool which supports all the models covered in this chapter. However, for Petri nets, Math works™ provides pertinent tool box useful for simulation, analysis, and synthesis of discrete-event systems based on Petri net models. Modelling time can be studied in paper by Furia (2010).

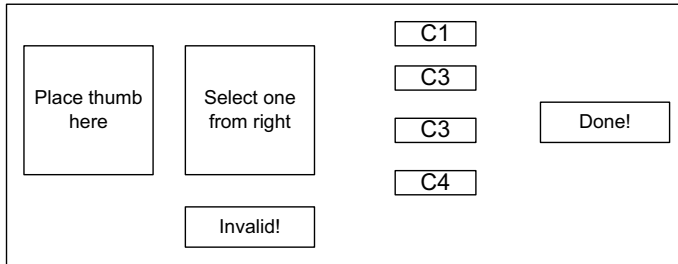
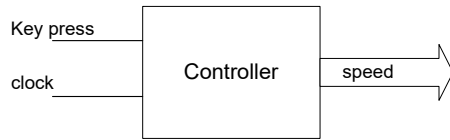
3.14 Exercises

1. A string processing module analyzes the characters in sequence and counts the number of isolated words. Two words are said to be isolated if they are separated by a “blank” or “tab” character. Design an algorithm using an FSM model.
2. A stopwatch has to be designed with the below specs: Design a custom processor with the controller and data path.
 - Displays seconds (2 digits)
 - Counts every 1 s
 - Can be reset to zero
 - Has start and stop buttons.
3. A stream of three audio channels is encoded and transmitted as frames. Each channel is encoded as 8-bit PCM data. Each frame contains one header and three audio channels. The header is 5 bits and is coded as 10001. So the frame contains 29 bits in total. The header pattern may also be contained in the PCM data. So the system has frame synchronization logic. The block shown below has to read the frames and get locked to a frame. Once locked it generates a locked signal and the three channels are de-serialized and outputted. Model the frame synchronization logic appropriately to generate “Locked” signal from the bitstream. Represent the model diagrammatically (Fig. 3.52).
4. Below is a majority voting logic block. The input is X, Y, Z each of 4 bits. If the data on two or more inputs is the same, then the result will be the matching data and the “matched” output is true. If the three input data are different, the

Fig. 3.52 De-serializer**Fig. 3.53** Voter logic

matched output is false and data on “result” is irrelevant. The match operation takes place when input to “compare” is TRUE. When compare is not TRUE the output is irrelevant. Design the CDFG model (Fig. 3.53).

5. A communication system implements a simple STOP and WAIT protocol. The sequence is as follows:
 - Message is transmitted.
 - Waits for Acknowledgement ACK.
 - Once ACK is received, the system sends the next message.
 - Assume the sender has infinite messages to send.
 - Model the sending module using (a) Petri net and also as (b) FSM.
6. The telephone exchange detects the status of the handset and processes a call. Define broadly the states and possible transitions in a call processing using FSM.
7. A variable-speed motor has to be developed. The controller shown below controls the speed of the motor. The motor speed is proportional to the byte value posted by the controller on its output, i.e., when you want 100 rotations/sec (RPS) you have to post a 064H. When the user keeps the input key pressed (“key press” as shown in the diagram), the speed of the motor raises gradually at the rate of 1 RPS reaching a maximum of 255 RPS. When the “key press” is released, the motor gradually slows down at the rate of 1RPS. Assume the frequency of the input clock to the controller of a suitable value.
 - a. Represent the algorithm in a sequential pseudo-language.
 - b. Draw the state machine.
 - c. Design the controller and data path (Fig. 3.54).
8. Below is the interface of an electronic voting machine (EVM). The EVM has five buttons and four LEDs to glow. When the system resets or is ready to accept a vote the “place thumb here” LED glows. It is also a push button. Voter

Fig. 3.54 Speed controller**Fig. 3.55** EVM

has to press it for minimum 5 s. During this time the EVM senses and analyzes the fingerprint and generates a 16-bit signature. The processor compares it with internal data. If the signature is valid and not voted, the next light “select one from right” glows. The voter has to press one of C1 to C4 for 5 sec. If it is valid “done” lamp glows for 15 sec and voter selection is registered. System goes back to ready state again. Invalid button glows when the signature could not be generated or buttons pressed for shorter time or for any abnormal operation. System resets back to acceptance state after “Invalid” is on for 15 s (Fig. 3.55).

- Represent the system as FSM at the top level.
 - Expand each state hierarchically as an FSM so that the complete system is represented as a hierarchical FSM.
9. A digital watch has four modes of display. (Mode 1) HH:MM:SS, (Mode 2) HH (only hours), (Mode 3) MM (only minutes), and (Mode 4) SS (only seconds). One can go to any mode of display by keeping the mode switch pressed for more than 2 s. After releasing the mode switch press, the system changes the mode. The change is cyclic, i.e., 1 > 2 > 3 > 4 > 1; when HH, MM, or SS is displayed, one can increment the displayed time by keeping the SET button pressed.
 - Analyze the problem using an object-oriented approach.
 - Define the classes (methods and attributes).
 - Explain the interactions to set and display the time using an interaction diagram.
 10. There are two traffic lights posts. Each has red-green-yellow signals. The normal switching is from red to green to yellow and back to red. Both the

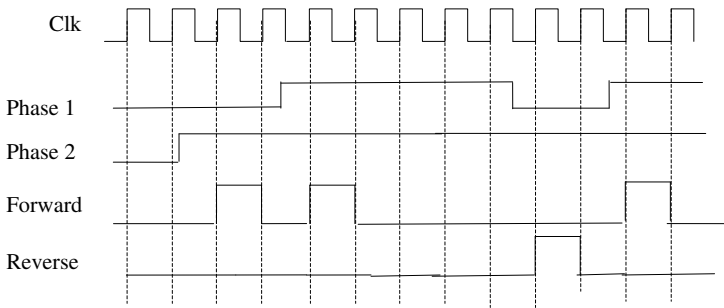


Fig. 3.56 Rotary encoder

light posts signal independently but both of them cannot signal green at the same time. Represent the problem using Petri nets.

11. A rotary encoder has two outputs $\Phi 1$ and $\Phi 2$. When the encoder rotates clockwise, it generates a sequence of pulses 00->01->11->10->00 for each step it moves. The sequence is reverse if it rotates anti-clockwise. We have to design a module that generates a forward pulse for each step clockwise and a reverse pulse for each step anti-clockwise as shown in the diagram below. Assume clk frequency is high compared to the rotating speed. Represent it as a state machine (Fig. 3.56).
13. A radar system detects flying objects and generates one pulse for each object detected. The width of the pulse is 100 ± 1 ms. However, due to noise, the received signal contains extraneous pulses which are out of this pulse width range. The system has to ignore the noise and count the number of objects tracked in a minute (Fig. 3.57).

Problem:

- i. Represent the logic in sequential programming logic.
 - ii. Design the hardware from the above logic using the CDFG model systematically.
14. Below is a state diagram with five states and nine event transitions. Represent this in a hierarchical fashion using an appropriate model (Fig. 3.58).
 15. A vending machine accepts combinations of 1, 2, and 5 rupee coins to get a coke. The cost of coke is 15 rupees. The machine validates the coins you place and releases the coke if the amount placed is Rs. 15 or more. If an invalid coin is placed it aborts all coins placed. Assume the machine does not return the surplus amount you inserted. Represent the problem as a state machine.

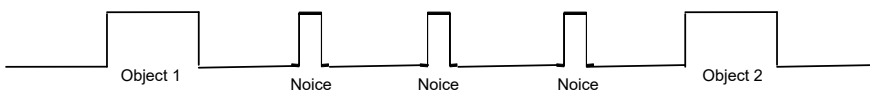
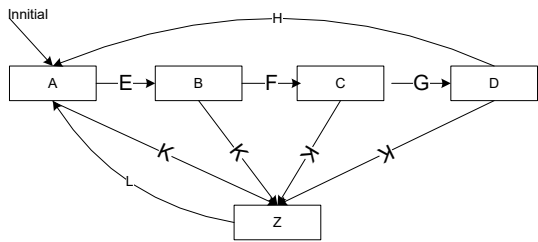


Fig. 3.57 Object detection

Fig. 3.58 State diagram



- 16. Identify whether below Petri nets are (a) bounded and (b) Live (Figs. 3.59 and 3.60).
- 17. Four lights L1 to L4 are to be blinked each for 3 sec. The sequence of blinking is 0, 1, 3, 2, 0, 1, 3.... The transition from one blink to the other is 2 sec. Design an FSM to represent the model and realize the FSM.
- 18. One traffic light has three lamps: red, yellow, and green. There can be only one valid transition allowed. Red to green to orange to red. Represent this as a Petri net model and use this to realize the circuit.
- 19. A digital circuit accepts a bitstream and stores the last three bits at any time. Model this as an FSM.

Fig. 3.59 Petrinet

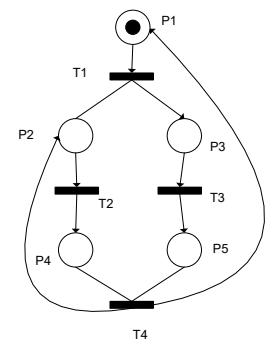
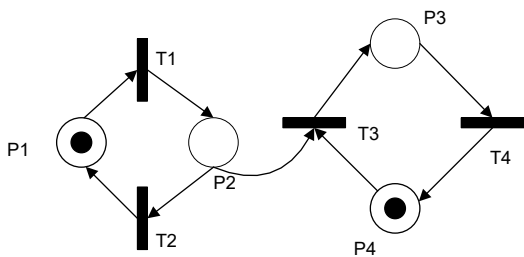


Fig. 3.60 Petrinet



References

- Chapter 4: State machines, 6.01SC Introduction to Electrical Engineering and Computer Science
Spring 2011, 6.01—Spring 2011—April 25, (2011)
- d'Ascq V (2007) Safer European level crossing appraisal and technology. In: First Workshop “Level crossing appraisal”, May 16th 2007
- Furia CA (2010) Modeling time in computing: a taxonomy and a comparative survey. *ACM Comput Surv* 42(2), Article 6
- Gajski DD (2009) Embedded system design, modeling, synthesis and verification. Springer
- Gajski DD, Vahid F (2009) Specification and design of embedded systems. Prentice Hall
- Heath S (2000) Embedded systems design. Newnes
- High-level Petri Nets—concepts, definitions and graphical notation. Final Draft International Standard ISO/IEC 15909, Version 4.7.1, October 28, 2000
- Lavagno L (1998) Models of computation for embedded systems design
- Marwedel P (2006) Embedded system design. Springer
- Murata T (1989) Petri nets: properties, analysis and applications. *Proc IEEE* 77(4)
- Oshana R (2013) Software engineering of embedded and real-time systems. Elsevier Inc.
- Radojevic I, Salcic Z (2011) Embedded systems design based on formal models of computation. Springer
- Wolf W (2008) Computers as components. Elsevier

Chapter 4

Specification Languages: SystemC



Abstract We have studied various models extensively in the last chapter. We transformed real-world problems into different domains through appropriate models to analyze certain characteristics. We have adopted different types of models for analyzing different characteristics for the same problem. However, it is a theoretical representation. We should now explore methods to verify that the model will behave as expected. The models we studied are represented as diagrams. Manual analysis of this diagrammatic representation is possible for a smaller size of the problems. We need a concrete form to represent the model so that all possible characteristics can be analyzed. Mostly, it is done by a specification language that captures these models in a concrete form. The language captures the functionality of the model in a machine-readable form. Once transformed to a language, it can be executed like any programming language and obtain results for different inputs. These are executable specification languages (ESLs). In modern design and development, ESLs play a major role as you can totally analyze the proposed model of the real-world problem, analyze using the ESL, and verify your design for intended functionality before you implement it. The ESL becomes the synthesis tool for design. Section 4.2 discusses important characteristics needed for the ESL for the design of embedded systems. The language has to capture the concurrent and hierarchical behaviors as processes, procedures, or state machines. Every behavior must have a mechanism to indicate that the activity is completed. ESL should support resource and activity synchronization primitives. The ESL should be executable and verify the behavior in a simulated environment. Once the results are verified, the ESL construct should be synthesizable to the desired implementation platform. SystemC is an executable specification language (ESL) at the system level. Sections 4.3, 4.4, 4.5, 4.6, 4.7, 4.8, 4.9, 4.10 and 4.11 discuss the details of SystemC with example implementations.

Keywords Executable specification languages (ESL) • Concurrency • Data driven concurrency • Control driven concurrency • Behavioral hierarchy • Completion of behaviors • Synchronization • Exception handling • SystemC • Interface • Channel

4.1 An Example

As an example, see Fig. 3.6, representing house topology as a graph. You want to find the average number of doors you have to cross to move from any room to the other. It is possible doing it manually. Similarly, look into any of the state machine diagrams we studied. We want to analyze, for example (a) the events needed from transitioning from one state to the other. (b) Can we reach every state of the machine from any other state? (C) Does the machine define state transition for every possible event? These are the possible analysis we have to do.

In Fig. 4.1, an ESL representing the input FSM in a language is executed. After analysis and verification for correctness, the ESL synthesizes a sequential circuit representing the model input. The ESL is nothing but some hardware description language (HDL) tool which is a very common executable specification language used in electronic design. The next advantage is that the ESL is a way of documenting the model. ESL is a good medium of exchange of model information. The designed models can be stored as components for re-use in different applications. Thus ESL is a way of mapping a model to a language. As different conceptual models have different characteristics, a unique ESL may not be possible to support all types of models. Similarly, an ESL may not support all characteristics of a model. Let us see in detail.

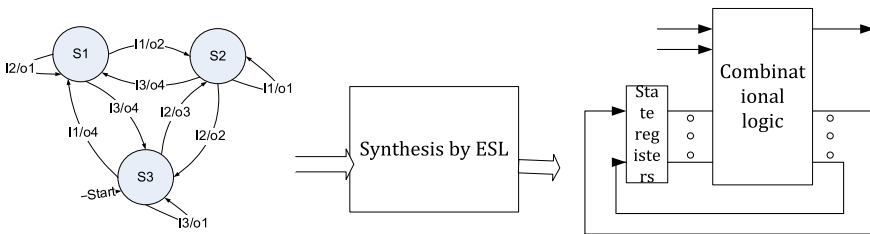


Fig. 4.1 Synthesizing a sequential circuit from an FSM

4.2 Characteristics of ESL for Embedded Systems

Before we dwell onto SystemC as one of the executable specifications in detail, let us see what characteristics an ESL should support. This will be helpful before we select an ESL for the embedded system design process.

4.2.1 Concurrency

A system can be composed of multiple components which execute concurrently (see HCFSM, PSM, ...) the concurrent components that interact, by communicating either by events, or certain conditions of data. The language has to capture concurrent and hierarchical behaviors as processes, procedures, or state machines.

4.2.2 Data-Driven

Figure 4.2 presents the data-driven concurrency.

See Fig. 4.2 for data-driven concurrency. The basic inputs to the system are a, b, c. The operations execute when the data is ready.

To compute $(a^2 - b^2)$, two concurrent operations $(a+b)$ and $(a-b)$ are possible. Once the results are available, the multiplication process takes place and computes $(a^2 - b^2)$. The next operation is possible when $(a^2 - b^2)$ is computed which is added to C. So the concurrent operations are only two. In data-driven concurrency, the operations are scheduled when the data is ready to be processed. Dependencies between operations decide the execution order.

Fig. 4.2 Data-driven concurrency

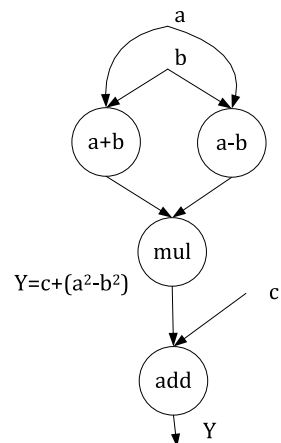
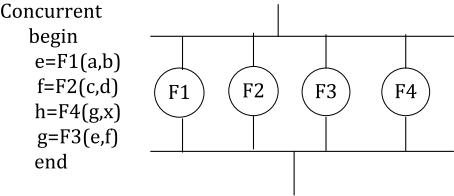


Fig. 4.3 Control-driven concurrency



4.2.3 Control Flow Driven

The concurrency is driven by the control flow. In Fig. 4.3, explicit construct begins concurrent execution of F1 to F4. At the task level, Fork and Join are typical examples to create concurrency.

4.2.4 Hierarchy of Behaviors

Please refer to HCFSM and PSM models discussed in Chap. 3. They are most frequently used for representing concurrent and hierarchical behaviors where the behaviors exchange data and events. These models de-compose a behavior into sub-behaviors. A sample language construct is shown on the left portion of Fig. 4.4. Sequential behaviors are stated as sequential types. Similarly, concurrent behaviors are stated as concurrent types. In this model, P has two concurrent behaviors P1 and P2. P1 and P2 are initial sub-behaviors. P11, P12, and P13 are the sub-behaviors of P1. P2 has three concurrent sub-behaviors. P12 has two concurrent sub-behaviors, each having two sequential sub-behaviors. The sample list partially represents how the hierarchical behavior representation is represented in a specification language.

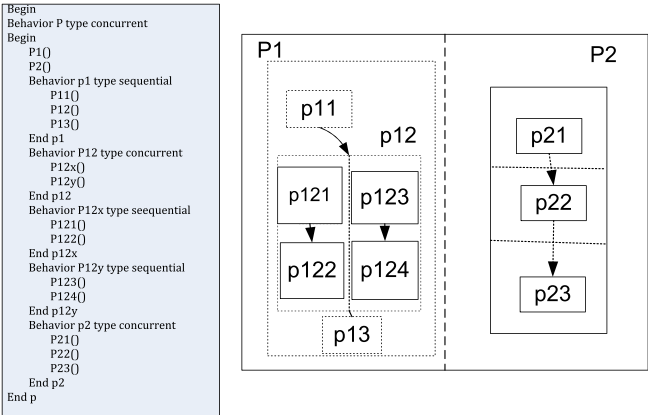


Fig. 4.4 Behavioral hierarchy

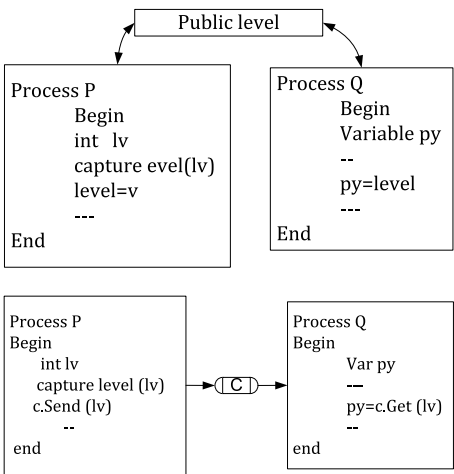
4.2.5 Completion of Behaviors

Please refer to Fig. 3.46 where PSM behavior is explained. Every behavior must have a mechanism to indicate that the activity is completed. The terminating condition (TOC) is shown as a rectangular square box. In FSM when the machine reaches a designated end state, the activity is completed. In a sequential program when the program reaches an exit condition or the last statement of the construct, the activity is completed. In PSM when the termination condition occurs, the same is represented as TOC. Then the activity is considered completed.

4.2.6 Shared Communication

Please refer to Fig. 3.47 where concurrent behaviors communicate using channels. A channel is an abstract entity with a virtual interface defined. Concrete implementation of the abstract interfaces is done based on the application environment. A channel is realized by a bus (serial or parallel) and a protocol to stream the data. The data transfer can be uni- or bidirectional. Two behaviors can be connected one-to-one or one-to-many behaviors. Communication across the participating activities continues when both are ready to transmit and receive, else the communication is blocked. In the case of non-blocking mode, the communication is asynchronous, with the data written into a queue and received from the queue. Another general way of communication is through shared variable globally. Typical implementation in a language is shown in Fig. 4.5.

Fig. 4.5 Implementation of communication



4.2.7 Synchronization

Multiple activities (you may call them processes, programs, or behaviors, tasks/jobs at this stage) execute independently but they have to coordinate based on a certain behavioral status of the activities. This process is called synchronization. The type of synchronization can be (a) wait for other activities to complete; (b) restart all activities to their initial state; (c) suspend it and wait for an event to occur, and so on.

Fundamentally synchronization is classified into two categories: resource synchronization and activity synchronization. When a resource is shared by multiple threads or activities, resource synchronization indicates whether an activity can access it safely. It means that no other activity is using the resource. When multiple activities are executing, one activity should know the state of the other activity so that they can synchronize their activities. Activity synchronization indicates the state of an activity when it has reached a certain state.

4.2.7.1 Resource Synchronization

If a resource is common (e.g., shared memory) and accessed by multiple tasks, the related tasks must be synchronized to maintain the integrity of a shared resource. This process is called *resource synchronization*. Most of the programming languages including the specification languages provide constructs to support this. Let us see this concept with critical sections and mutual exclusions as examples.

Mutual exclusion is a provision by which only one task at a time can access a shared resource. The critical section is the extent of code by executing of which the shared resources are accessed.

Below is an example where two activities are sensing and displaying shared sensor data. The sensor task reads from IO devices and writes into the shared memory. Display task reads the sensor data from shared memory and displays. Both the activities should share the resource through synchronization. Both have to mutually exclude each other while accessing the shared memory. The common design pattern of using shared memory is shown in Fig. 4.6.

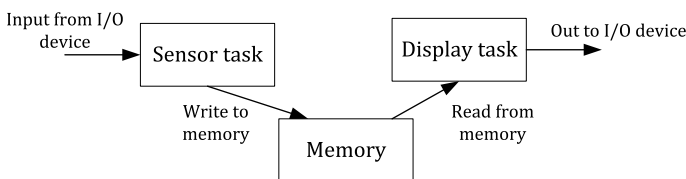


Fig. 4.6 Resource synchronization

4.2.7.2 Activity Synchronization

In general, a task must synchronize its activity with other tasks to execute when concurrent activities are executing. *Activity synchronization* is also called *event synchronization* or *sequence control*. Activity synchronization ensures that the correct sequence of execution among the tasks involved. The synchronization can be either synchronous or asynchronous.

4.2.7.3 Control-Dependent Synchronization

In control-dependent synchronization, the events generated by the activities in the concurrent system bring the activities of the complete system to a certain desired state. Each specification language provides constructs for this type of event synchronization (like (FORK/JOIN), Barrier, and so on). In the PSM example (Fig. 4.7a), the two concurrent tasks A1 and A2 have s1 and s3 as initial tasks. In the normal flow of execution both A1 and A2 terminate after executing s2 and s4 and reach the terminating state of A. But under certain conditions when event evt2 occurs, the event causes both A1 and A2 to start and begin execution from their respective initial state. This is close to EXIT statement in a normal sequential program. Hence A1 and A2 synchronize from start by the control event evt2. See the behavior in Fig. 4.7b, where the event evt2 occurred somewhere in task 1 makes A2 transit to the initial state and causes A1 to enter state S2. So by the available synchronization constructs in the ESL, an event can reset the state of the complete machine to the desired one. The event definition is the same and can be as complex as conditional data expressions.

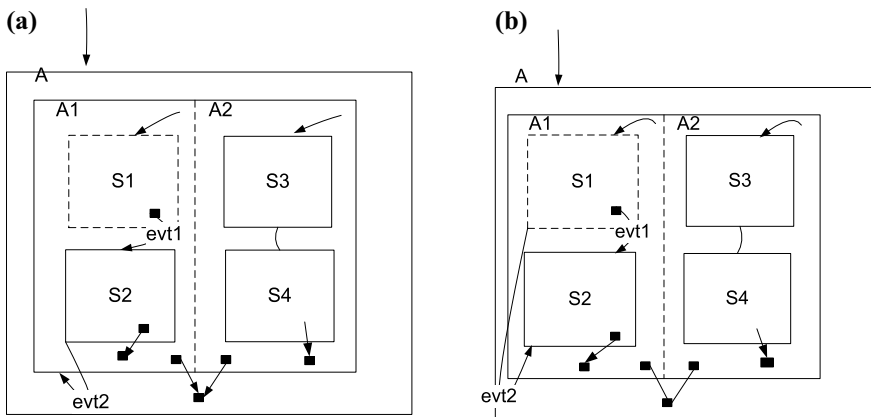
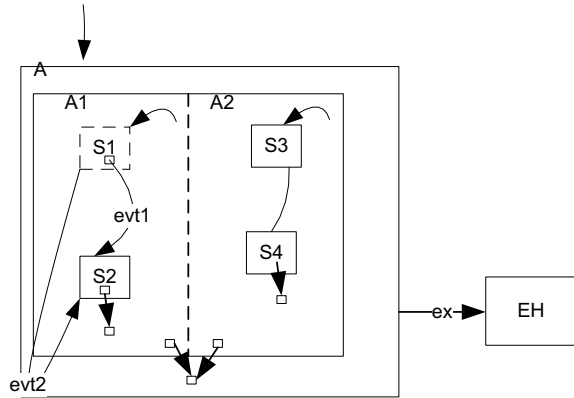


Fig. 4.7 a and b Control-dependent synchronization

Fig. 4.8 Exception handling

4.2.7.4 Data-Dependent Synchronization

Synchronizing shared memory access by multiple events is data-dependent synchronization. The processes requesting access to the shared memory are suspended until the process accessing the shared memory relinquishes. Several other data-dependent synchronizations are caused by events related to the data held by other processors and the status of processes itself, like (a) message passing, (b) the status of other process has changed, (c) a global variable reached to certain value, and (d) an event generated by a data expression.

4.2.8 Exception Handling

A very common construct available in most of the programming languages and in ESLs also is exception handling. Every behavior should be able to get terminated abruptly. The exit behavior to whom the control has to be transferred is to be explicit. ESL should support exception handling by allowing transitions at higher levels of hierarchy and terminate all computations at lower levels (see Fig. 4.8). In the entire system when an exception event *ex* occurs, the system transits to process EH and exits.

4.2.9 Summary: Specification Languages

We have seen in the above paragraphs that the abstract model definitions are mapped to suitable specification language which can be executable to verify the overall behavior and synthesize the system. All the model characteristics must be able to be transformed to the ESL. Once such ESL is selected, the ESL should be executable

and the behavior in a simulated environment is verified. Once the results are verified, the ESL construct should be synthesizable to the desired implementation platform.

4.3 SystemC

With a basic overview of the executable specification languages, we should explore what ESLs are available for designers, for what purpose they are used, and what are their limitations. Basically, the designer views the problem hierarchically. Let us start from bottom to top. Hardware design at the chip level is simulated and verified with clock cycle level accuracy. Cycle accurate models provide an accurate description of the state of the model on each clock cycle. As such, they represent a mid-point between traditional event-driven simulation and high-level transaction models. Cycle accurate models are of particular value because they reflect the level of detail seen by a software engineer using a chip. The software engineer generally cannot see what is happening within clock cycles.

These are hardware description languages like VHDL and VERILOG, and so on. Chip-level design is too cumbersome and time-consuming. Let us go one level up, say at register transfer level (RTL), where you model the design as register transfers, and mapping behaviors to data transfers at the register level. At this level, design verification and synthesis to chip level is less cumbersome and faster. Current HDL tools support both RTL level and synthesize down to chip level. Even we can use C/C++ to represent the design and simulate. However, our interest is to represent the complete system, simulate and verify the design at the system level, and seamlessly pass the verified design to lower RTL and chip levels. At this level, we are not interested in whether the design is implemented as hardware or software but the system behavior is modeled.

SystemC is an executable specification language (ESL) at the system level. The objective is to pass the verified design to lower levels for implementation in hardware. The hardware/software co-design concept is implemented through SystemC level design. See Fig. 4.9 for the three levels of design.

Fig. 4.9 Exploring different architectures at different levels

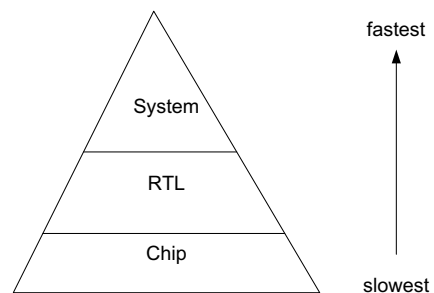
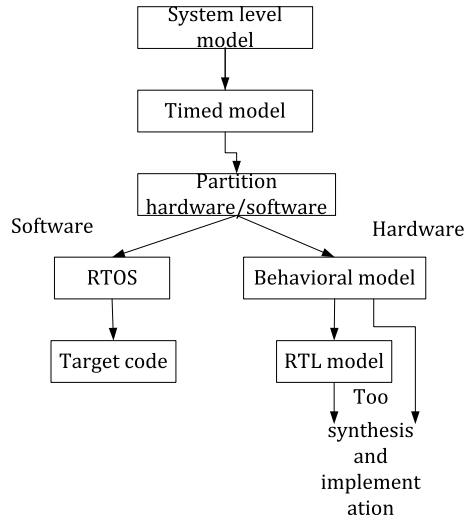


Fig. 4.10 Basic system-level design process



SystemC initiative was formed in 1999 with the active participation of multiple companies. SystemC1.0 was released in the year 2000. SystemC2.0 was released in 2001. This is standardized by *IEEE 1666–2011: SystemC language*. Currently, Accellera systems initiative is coordinating all development and standardization initiatives in this direction (About SystemC: the language for system-level modeling, design and verification). Current release version is SystemC 2.3.3, which includes transaction-level modeling.

The objective is to enable system-level modeling, which finally includes software, hardware, or both. System-level modeling should support a wide range of models of computation at different abstraction levels and different methodologies. Figure 4.10 explains the basic system-level methodology.

At the system level (top layer in a pyramid of Fig. 4.9), you use any programming language, execute basic algorithms in the design, and verify against specifications. At this level, you are not verifying whether the design works in real time. The design elements are abstract. You have verified only the functionality. At this stage, you have to move the verified design to an event-driven timed model by transforming the behavior/algorithms to different architectures to verify the temporal behavior. Verified designs at the middle level will be partitioned into software and hardware suitable for implementation. Partitioned timed model to software gets implemented on real-time operating systems (RTOS) and target code in any language. The hardware partition gets verified using RTL models and hardware design is implemented.

Let us see how the design process explained in Fig. 4.11 is executed in non-SystemC and a SystemC approach and how it makes the difference.

The system designer implements the system using C/C++ or any programming language to verify the overall system behavior. The verification is done with respect to the specifications. Verified implementation is handed over to an RTL designer. The RTL designer cannot input the tested model into the RTL design tool. First of

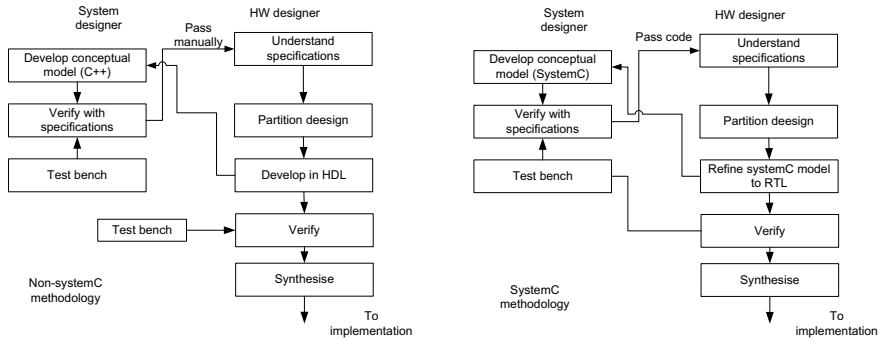


Fig. 4.11 Compare non-SystemC and SystemC methodologies (Courtesy Bhasker J (Based on Fig. 1.3 and 1.4. of a system-C primer))

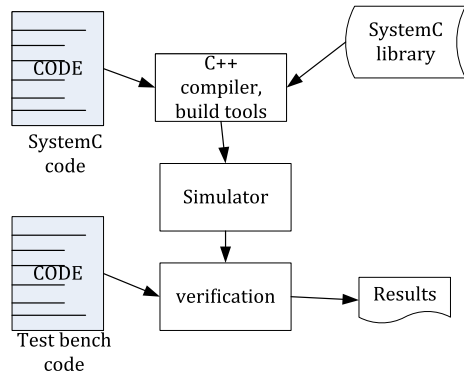
all, the tested model has to be partitioned with certain modules to be implemented in hardware. The design partition is done and the hardware portion is written in HDL and goes through the design/verification and testing phase. If the verification process needs certain changes at the system design level, the whole process is repeated. Finally, RTL synthesis tools produce the design for implementation in hardware. We can see the major problem. The specification language at the system level design and at lower levels is not seamless.

In SystemC methodology, a system designer develops the conceptual model in SystemC language and verifies the design with respect to specifications through simulation tests provided in SystemC. Once the system designer is satisfied, the SystemC code is passed to the RTL designer. The SystemC code is partitioned for appropriate implementation in hardware; the code is mapped to RTL for hardware synthesis. If any changes at the RTL level are needed, they are done in the SystemC level code. Effectively, you observe, once a system is modeled in SystemC at the system level, the process is seamless till hardware design and software design are done. This realizes hardware/software co-design in a seamless way.

4.3.1 What Is SystemC?

SystemC is a C++ class library with a set of objects used to implement, simulate, and verify an integrated system that has software modules, hardware with complex architecture, and interface elements. Finally, it will be C++ code with certain portions which can be synthesized into hardware and some as software. Using SystemC and C++ development tools, one can develop code to create an integrated system in a software/hardware agnostic way.

Fig. 4.12 SystemC in C++ development environment



SystemC is an executable specification which is essentially a C++ program that exhibits the same behavior as the system when executed. With today's modern HDL frameworks, you can mix SystemC with an HDL design to make a more complex efficient model of a piece of hardware, or make a more robust test bench.

See Fig. 4.12 for SystemC in C++ development environment. The SystemC Class Library provides all the primitive elements needed to implement hardware like modules, signals, channels, etc. with necessary timing and concurrency. Each primitive element has specified behavior with which we can construct complex hardware integrated with C++ code for generic algorithm implementation.

4.3.2 SystemC Features

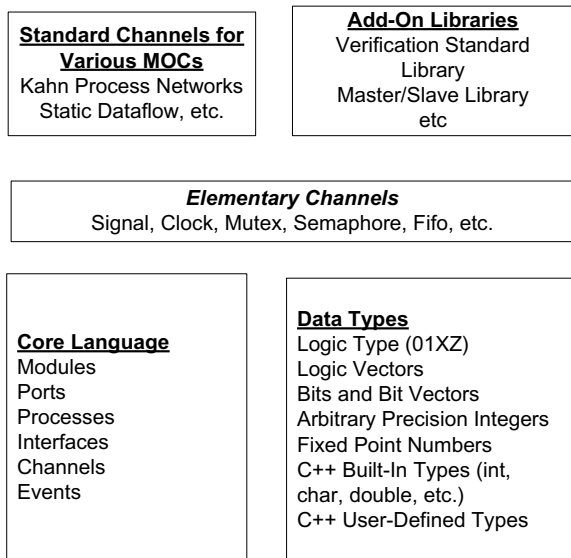
- SystemC is both system-level and hardware description language.
- SystemC supports hardware/software co-design and the description of the architecture of complex systems consisting of both hardware and software components. It supports the description of the hardware, software, and interfaces in a C++ environment.
- SystemC allows it to be used as a co-design language.
- Multi-level logic supports both two-valued and four-valued data types.
- Clocks support multiple clock objects, with an arbitrary phase relationship.
- Cycle-based simulation: SystemC includes an ultra-light-weight cycle-based simulation kernel that allows high-speed simulation.
- Allows you to model at the algorithmic level. Supports various models of computation and methodologies.
- SystemC 1.0 provided RTL and behavioral HDL modeling capabilities.

- HW is modeled using zero-delay semantics for combinational logic. Signals are modeled using 01XZ. “C” data types and complex data types can also be used within signals.
- SystemC 1.0 includes good support for fixed-point modeling.
- SystemC 1.1 beta and 1.2 beta provided some limited communication refinement capabilities.
- SystemC 2.0 has more general system-level modeling.
- Capabilities with channels, interfaces, and events.
- SystemC 3.0 will focus on software and scheduler modeling (more later).

4.3.3 SystemC 2.0 Language Architecture

The language is built on C++ as template library, extended data types, and component library. In Fig. 4.13 the upper layers are built on lower ones. Lower layers can be used without using upper layers. Core language supports the structure, concurrency, communication, and synchronization primitives. Data types are separate from the core language. The commonly used communication mechanisms and MOC are built on top of the core language.

Fig. 4.13 SystemC2.0 language architecture
(*Courtesy* Accellera systems initiative™)



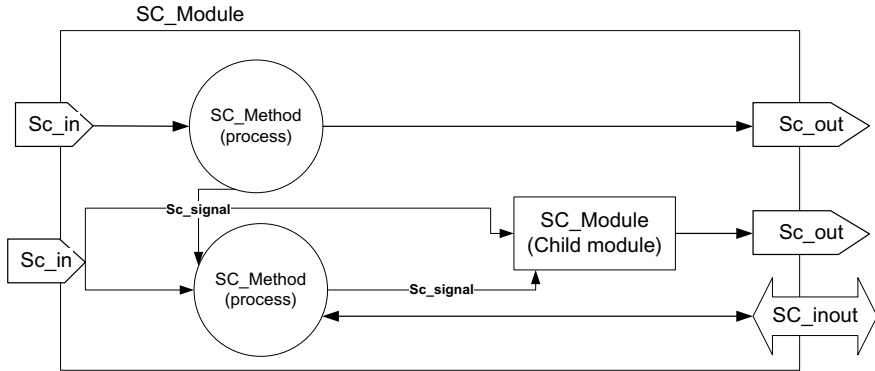


Fig. 4.14 Module (Courtesy J Bhasker (Based on Fig. 1.3 and 1.4. of a system-C primer))

4.3.4 Module

Module is the basic entity to represent certain functionality. Modules are the basic building blocks within SystemC to partition a design. Modules allow designers to break complex systems into smaller more manageable pieces. Modules help split complex designs among a number of different designers in a design group. Modules allow designers to hide internal data representation and algorithms from other modules. Modules are interconnected through ports. Modules can contain modules and processes. A module is described by `SC_Module`. A functional block can be declared using the `SC_MODULE` macro. This makes defining a C++ Class more like HDL. Much like declaring an HDL module, the ports and member functions (Processes) are defined. The `SC_CTOR` constructor defines the sensitivity lists of the processes (Fig. 4.14).

4.3.5 Module Declaration

A module declaration in SystemC is essentially the same as a Class definition in C++. It represents functionality as an Entity/Architecture does in VHDL.

The module declaration contains

- Port declarations.
- Local variables.
- Methods to define the process behavior.
- `SC_CTOR` is the constructor which initializes variables and specifies sensitivity lists.

As an example the module declaration for a d-flip-flop is given in Fig. 4.15.

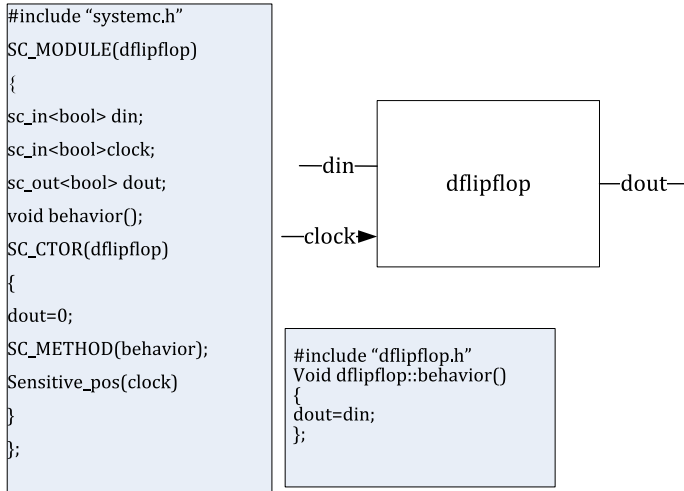


Fig. 4.15 Declaration of a d-flip-flop

4.3.6 Module Ports

Module ports pass data to and from the processes of a module. You declare a port mode as **in**, **out**, or **inout**. You also declare the data type of the port as any C++ data type, SystemC data type, or user-defined type.

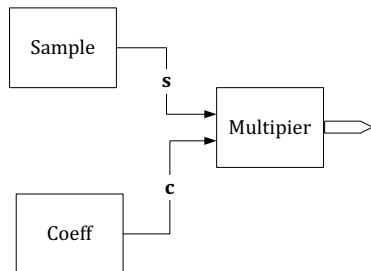
4.3.7 Module Constructor

The module constructor function `SC_CTOR` transforms C++ from a sequential language into a cycle-based system. The `SC_CTOR` constructor serves double duty. It acts as a class constructor, giving initial values to the variables in the module (class). It also converts the member functions into processes.

4.3.8 Module Signals

Signals can be local to a module, and are used to connect ports of lower-level modules together. These signals represent the physical wires that interconnect devices on the physical implementation of the design. Signals carry data, while ports determine the direction of data from one module to another. Signals aren't declared with a mode such as in, out, or inout. The direction of the data transfer is dependent on the port modes of the connecting components. In Fig. 4.16 there are three lower-level

Fig. 4.16 Sc_module
coefficient multiplier (cmult)



modules instantiated in the coefficient multiplier design, sample, coeff, and mult modules. The module ports are connected by two local signals *s* and *c*. There are two ways to connect signals to ports in SystemC.

4.3.9 Positional Connection

See Fig. 4.17. The submodules *s1*, *c1*, and *m1* are defined in filter module. Signals *s*, *c* are defined along with submodules. The constructor of the coefficient multiplier module contains the creation of new submodules *s1*, *c1*, and *m1*. The connectivity of the signals to the modules is defined while creating the submodules. The connectivity is defined by position. The second statement (**s1*) (*s*) states that signal *s* is connected to *dout*, i.e., each signal in the mapping matches the port of the instantiated module on a positional basis.

Fig. 4.17 Positional
connection

```
#include "systemc.h"
#include "mult.h"
#include "coeff.h"
#include "sample.h"
SC_MODULE(cmult) {
    sample *s1;
    coeff *c1;
    mult *m1;
    sc_signal<sc_uint<32>> s, c;
    SC_CTOR(filter) {
        s1 = new sample ("s1");
        (*s1)(s);
        c1 = new coeff ("c1");
        (*c1)(c);
        m1 = new mult ("m1");
        (*m1)(s,c);
    }
}
```


4.3.10 Named Connection

See Fig. 4.18a for a named connection. In named connection, the signal to port connections need not be in the specified order. You can define one explicit connection at any time. The first named connection connects port dout of module s1 to signal s of module filter. Using a named connection, the designer can create the signal to port connections in any order.

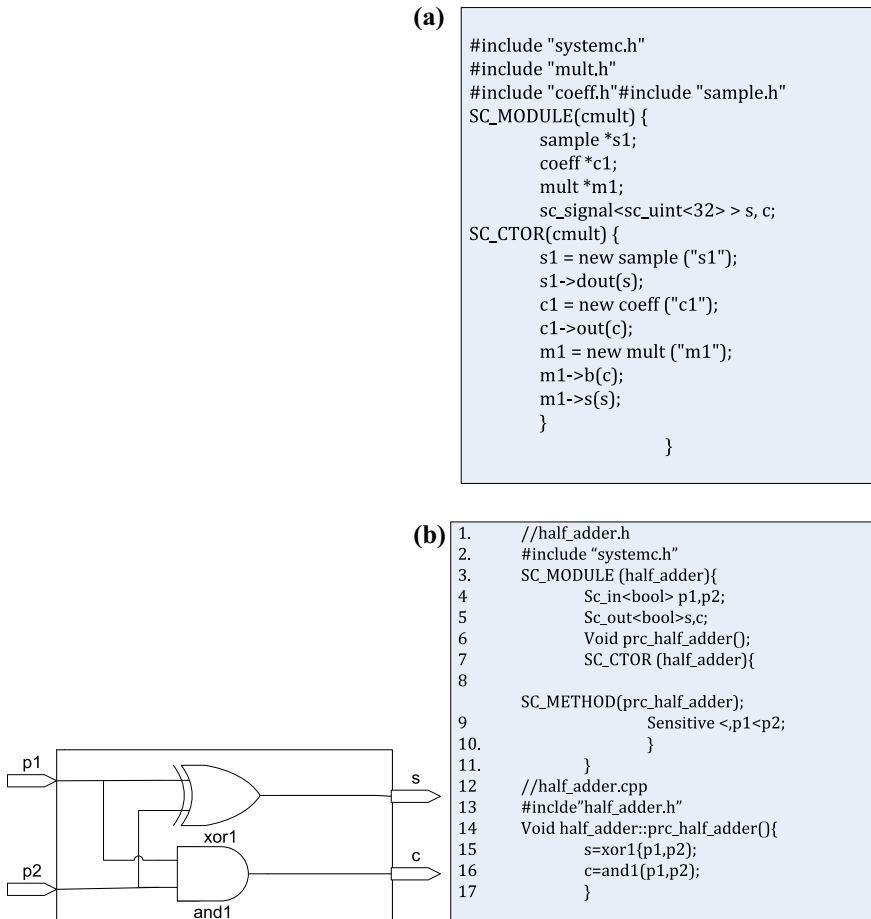


Fig. 4.18 a Named connection. **b** Half-adder using SystemC

4.3.11 Member Functions

The functionality is described using a member function, just as in C++. A member function is declared as part of the module declaration. The actual functionality can be described in line with the module declaration or, as was shown earlier, just the prototype is defined in the modules declaration. The functionality is then defined separately. For example, in Fig. 4.15, the function *behavior* is declared in the module definition. The functionality is defined separately in `dflipflop::behavior ()`.

4.4 Processes

Processes are the basic unit of execution within SystemC. The processes are called to emulate the behavior of the target device or system. Three types of SystemC processes are available:

- Methods (SC_METHOD)
- Threads (SC_THREAD)
- Clocked Threads (SC_CTHREAD).

4.4.1 Method (SC_METHOD)

When events (value changes) occur on signals that a process is sensitive to, the process executes. A method executes and returns control back to the simulation kernel. When a method process is invoked, it executes until it returns. Methods are like event processors that respond to an event. Processes assign value to the ports or generate signals and terminate to the simulator. In this case, the event is the value of the input signal which is declared sensitive to this method. Methods are never in infinite loops. If so, control will never be returned back to the simulator. As an example, see Fig. 4.15; the d-flip-flop module has one single SC_METHOD named as *behavior*. It has one sensitive input which is the positive edge of the clock. When this event occurs *behavior* executes and *dout* will be equal to *din*.

4.4.2 Thread (SC_THREAD)

Thread is a process that is continuously executing. The thread execution can be suspended on an event. It can be resumed again at an event. The thread is set to be sensitive in certain events. A thread changes its states and can wait for an event. The sensitivity list (the events on which the thread acts) is defined at the construction of the thread. You can correlate this as one concurrent program in the PSM model.

The thread process is the most general process and can be used to model nearly anything. A SC_METHOD process to model this same design would be difficult to understand and maintain. The thread process can implement a state machine (FSM) completely with interaction with other thread processes, thus implementing a concurrent FSM. Hierarchy can be established by multiple submodules and each submodule hosts multiple thread processes. A complete PSM model can thus be implemented.

4.5 Case Study: 4.1

Design a full adder and test the module with test inputs and verify the design (Courtesy Bhasker J (Based on Fig. 1.3 and 1.4. of a system-C primer)).

4.5.1 Solution

See Fig. 4.18b.

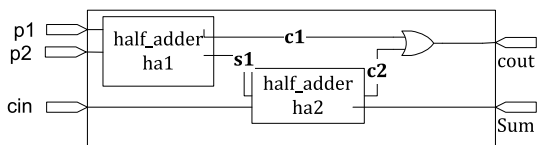
4.5.2 Half-Adder Module

A full-adder constitutes two half-adders. Let us design the half-adder as SC_Module. The SystemC source is shown in the right part of Fig. 4.18b. The SC module by name half_adder is declared at line 3. The input ports p1, p2 are defined at line 4. The output ports s, c are defined at line 5. The module has one SC_method by name prc_half_adder as declared at line 6. The module is constructed at line 7. The constructor instantiates the SC_method (line 8). The method responds to the change of inputs at p1, p2 (line 9). The behavior of SC_method is defined in lines 14–17. Generates sum as xor (a,b) and carry as a&b.

4.5.3 Full-Adder Module

Full-adder logic is known to all. Figure 4.19 shows the schematic of constructing a full-adder from two half-adders. Let us write SystemC code to create a full-adder SC_Module using half-adder submodules (Fig. 4.20).

Fig. 4.19 Full-adder using SystemC



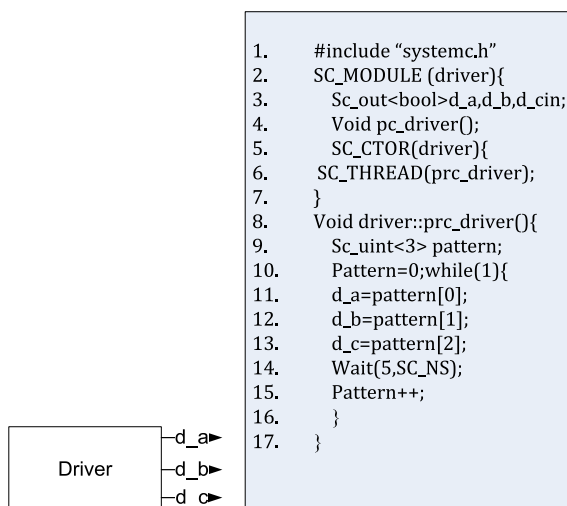
<pre> 1. //full_adder.h 2. #include "half_adder.h" 3. SC_MODULE (full_adder){ 4. Sc_in<bool>p1,p2,cin; 5. Sc_out<bool>sum,cout; 6. Sc_signal<bool>c1,s1,c2; 7. Void prc_or(); 8. Half_adder *ha1_ptr,*ha2_ptr; 9. SC_CTOR(full_adder){ 10. ha1_ptr=new half_adder(*ha1); 11. ha1_ptr->p1(p1); 12. ha1_ptr->p2(p2); 13. ha1_ptr->s(s1); </pre>	<pre> 14. ha1_ptr->c(c1); 15. Ha2_ptr=new half_adder("ha2"); 16. (*ha2_ptr) (s1,cin,sum,c2); 17. SC_METHOD(prc_or); 18. Sensitive<<c1<<c2; 19. } 20. //destructor 21. ~full_adder(){ 22. Delete_ha1_ptr; 23. Delete ha2_ptr; 24. } 25. }; 26. //full_adder.cpp 27. #include "full_adder.h" 28. Void full_adder::prc_or(){ 29. cout=c1 c2; 30. } </pre>
--	--

Fig. 4.20 SystemC source for full-adder

Full_adder module is defined (3) with the inputs: p1, p2, cin (4) and sum, cout as outputs (5). The connectivity of half_adders needs signals c1, s1, and s2. These can be assumed as internal nodes of wiring (6). Full-adder constructor is defined from line 9. First, half-adder is instantiated (10) as ha1_ptr. The inputs and outputs to ha1 are associated by named association (11–14). The second half-adder ha2 is instantiated (15) and its inputs/outputs are done through positional association (16). Full adder has two submodules (ha1 and ha2) and one SC_method to perform OR operation of carry. This is defined at 17–19. The constructor of full adder is completed. At this stage, SC_MODULE (full_adder) is ready which can be instantiated. Need a module to create a stimulus to test and check the results.

4.5.4 Driver Module

A driver module is to be connected to full adder and inject a pattern of data into full adder. Hence a SC_MODULE has to be developed. See Fig. 4.21, where the module has no inputs but generates continuously three outputs, d_a, d_b, and d_c as three sets and repeats the same. Hence the module will contain one SC_Thread to generate continuously. The module driver is defined (2) with three outputs (3). The module constructor (5) contains a single statement defining a SC_Thread with behavior defined in the prc_driver. Prc_driver behavior (8) contains three patterns which are written to outputs (11 to 13) and repeats. The same is shown as a block diagram.

Fig. 4.21 Driver module in SystemC

4.5.5 Monitor Module

The monitor in Fig. 4.22 is SC_Module which reads the inputs given to the full adder by the driver and the response from the full adder. The module simply prints each vector. For automated testing, the module can be extended to verify the input and out vectors and verify the results and pass or fail the design.

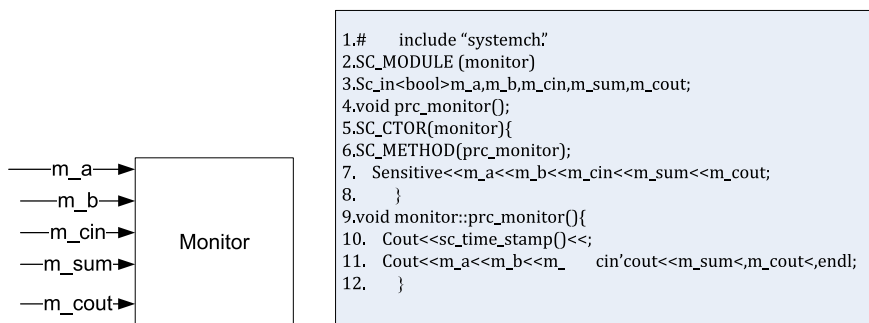
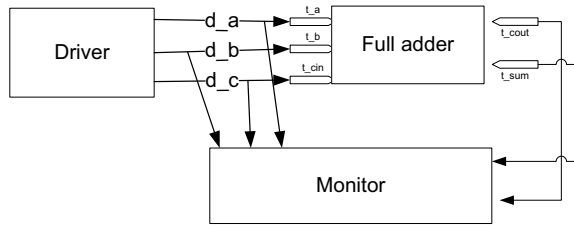
**Fig. 4.22** The monitor in SystemC

Fig. 4.23 The test bench

4.5.6 The Test Bench

You have developed three modules: full adder, driver, and monitor. They are defined in respective header files. For the complete simulation cycle, they have to be instantiated, interconnected, and executed. The interconnections are shown in Fig. 4.23.

```

1.  #include "driver.h"
2.  #include "monitor.h"
3.  #include "full_adder.h"
4.  Int sc_main(int argc,char*argv[]){
5.    SC_Signal <bool t_a,t_b,t_cin,t_sum,t_cout;
6.    Fulladder f1()
7.    F1<<t_a<<t_b<<t_cin<<t_sum<<t_cout;
8.    Driver d1();
9.    D1.d_a(t_a);
10.   D1.d_b(t_b);
11.   D1.d_cin(t_cin);
12.   Monitor mo1();
13.   Mo1<<t_a,t<<t_b<<t_cin<<t_sum<<t_cout;
14.   Sc_start(100,sc_ns);
15.   Return(0);
16.  }

```

The SystemC source includes the respective headers (1–3); the main program looks similar to `main ()` of C program (4). The interconnection signals are defined (5). Full adder `f1` is instantiated (6). The inputs and outputs to `f1` are wired-up (7). Driver `d1` is instantiated (8). `D1` is wired to input signals of `f1` (9–11). Monitor is instantiated (12). The monitor is hooked to input and output signals of `fulladder-f1` (13). All the five signals will thus be input to the monitor. Execution starts (14).

4.6 Case Study: 4.2

Design an up-down counter. This needs `SC_Thread` as it is a continuous process. The counter has control signals up count, down count, and stop. The counter can be set to a count and the current count has to be displayed.

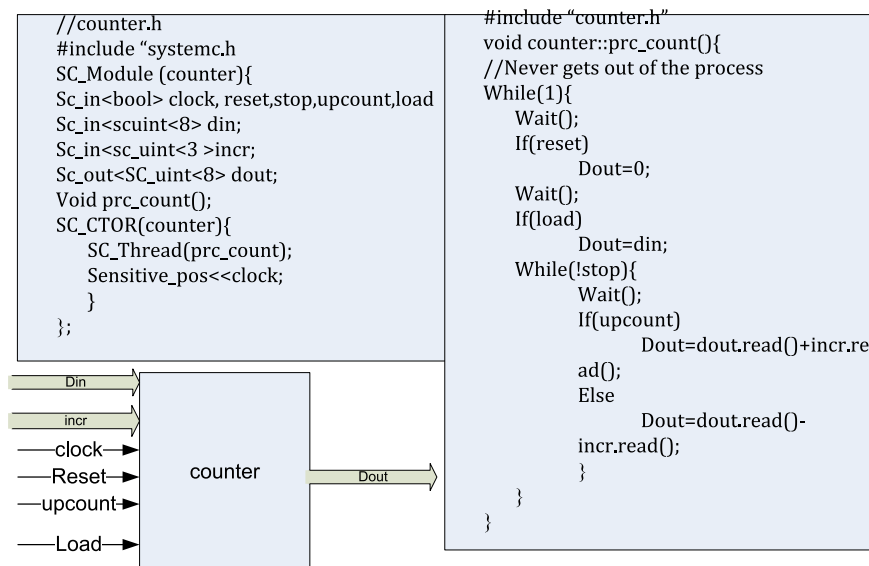


Fig. 4.24 Up-down counter

Solution

See Fig. 4.24.

4.7 Objects in SystemC

After looking into important objects like SC_module, SC_method, SC_threads, and a couple of examples, let us study some important objects in SystemC. Study the references attached at the end of the chapter to be a serious SystemC developer (1666–2011—IEEE Standard for Standard SystemC Language Reference Manual Revision of IEEE Std 1666–2005; Based on Fig. 1.3 and 1.4. of a system-C primer).

4.7.1 Sc_clock

Clock generates timing signals used to synchronize events in the simulation. Clocks order events in time so that parallel events in hardware are properly modeled by a simulator on a sequential computer. A clock object has a number of data members to store clock settings and methods to perform clock actions. To create a clock object, use the following syntax, something like:

```
sc_clock clock1 ("clock1", 25, 0.5, 3, true);
```

This declaration will create a clock object named `clock1` with a period of 25 time units, a duty cycle of 50%, the first edge will occur at 3 time units, and the first value will be true. All of these arguments have default values except for the clock name. The period defaults to 1, the duty cycle to 0.5, the first edge to 0, and the first value to true.

4.7.2 Data Types

SystemC provides the designer the ability to use any and all C++ data types as well as unique SystemC data types to model systems. The SystemC data types include the following:

- `sc_bit`—2 value single bit.
- `sc_logic`—4 value single bit.
- `sc_int`—1 to 64 bit signed integer.
- `sc_uint`—1 to 64 bit unsigned integer.
- `sc_bigint`—arbitrary sized signed integer.
- `sc_biguint`—arbitrary sized unsigned integer.
- `sc_bv`—arbitrary sized 2 value vector.
- `sc_lv`—arbitrary sized 4 value vector.
- `sc_fixed`—templated signed fixed point.
- `sc_ufixed`—templated unsigned fixed point.
- `sc_fix`—untemplated signed fixed point.

4.7.3 Wait Until

In a `SC_THREAD` process `wait_until()` methods can be used to control the execution of the process. The `wait_until()` method will halt the execution of the process until a specific event has occurred. This specific event is specified by the expression to the `wait_until()` method.

4.7.4 Sc_Start

Once the instantiation of the lower level modules has been coded, and the clocks setup, the simulation is moved forward using the `sc_start` method. If an argument is given, then the simulation will move forward by that many time ticks. If an argument of `-1` is given then the simulation will run forever.


```
#include "counter1.h"
Int sc_main(int argc, char* argv[]) {
    Sc_signal<bool> Rst;
    Sc_signal<sc_int, 8> cval;
    Sc_clock CLOCK("clock", 20);
    Counter1 u1("count1");
    U1.Clk(CLOCK);
    U1.reset(Rst);
    U1.Ctr_val(cval);
    Sc_start(500);
    Return(0);
}
```

4.7.5 *Sc_Event*

Sc_Event is the fundamental synchronization primitive. It does not have a type, neither has it transmitted any value. Event transfers control from one process to the other. An event notification causes the processes sensitive to this event to be resumed. Effectively, this is akin to an event triggering a program in the PSM model. An event can be declared explicitly by using the SC_Event type.

EX: Sc_Event write_back;
 Write_back.Notify (); Trigger for immediate notification.
 Write_back.Notify (20, SC_NS); Delayed Notification.
 Write_back.Cancel (); cancel a scheduled and un-occurred event.

The owner of the event is responsible for reporting the change to the event object. The event object is responsible to keep a list of processes sensitive to it (Fig. 4.25).

4.7.6 *Wait*

Wait function makes a process wait on an event. Examples are given below.

- sc_time t(200, SC_NS);
- Wait (t); // wait for 200 ns.
- Wait (t, e1); // wait on event e1, timeout after 200 ns.
- Wait (t, e1 | e2 | e3); // wait on events e1, e2, or e3, timeout after 200 ns.
- Wait (t, e1 & e2 & e3); // wait on events e1, e2, and e3, timeout after 200 ns.
- wait (200); // wait for 200 clock cycles.

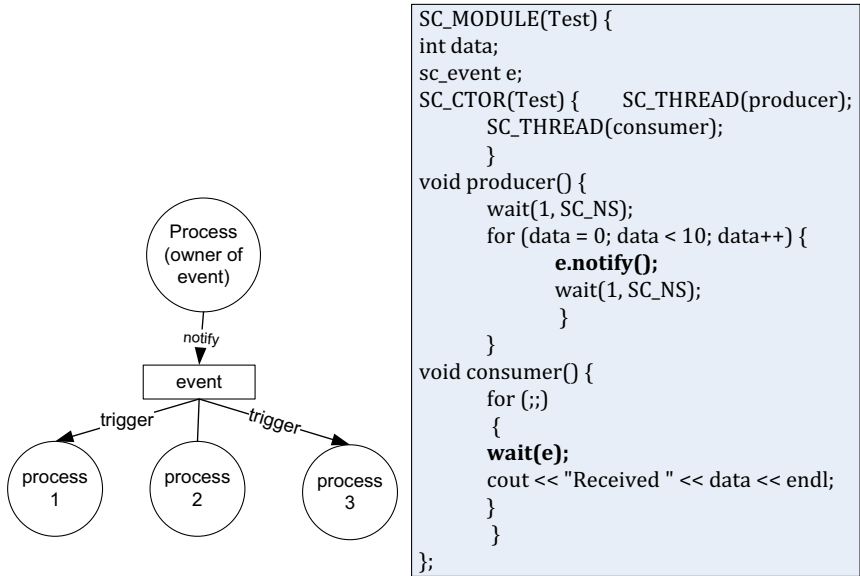
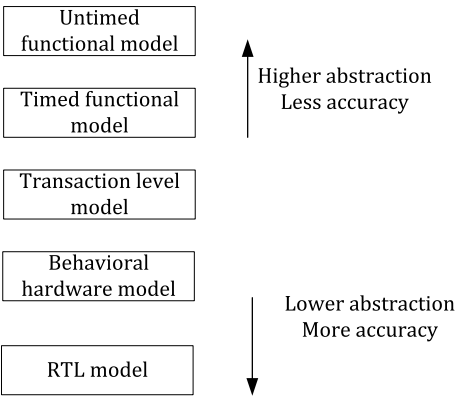


Fig. 4.25 Notification process

4.8 Models of Computation in SystemC

Five models of computation supported by SystemC are shown in Fig. 4.26. These five models of computation are an untimed functional model, a timed functional model, a transaction-level model, a behavior-level model, and a register-transfer model. In terms of abstraction and accuracy, the hierarchy of the models of computation supported by SystemC can be seen in Fig. 4.26.

Fig. 4.26 Models of computation in SystemC



4.8.1 Untimed Functional Model

The untimed functional model is a functional specification of the target system. As the name implies, the model has no notion of timing. When the untimed functional model is simulated, only the functional results get verified. Functionality may be hierarchically broken down into modules. Communication between modules is done implicitly as there are no communication links or buses being modeled in this layer of abstraction. This model of computation is much like a prototype of a system written entirely in a software programming language.

4.8.2 Timed Functional Model

The timed functional model is functionally the same as the untimed functional model, but includes the notion of timing during simulation. Approximate timing constraints are annotated so that the computation delays associated with the target implementation can be estimated. No details regarding the communication between modules are defined at this level since it is still done implicitly. All other aspects in comparison with the untimed functional model remain the same.

4.8.3 Transaction-Level Model

The transaction-level model defines the communication between modules by using function calls. This models the accurate functionality of the communication protocol and isolates the communication details from the computational functionality. The transaction-level model has approximated timing annotations in both communication functions and computational modules to indicate a rough estimate of the timing characteristics of the system. In the transaction-level model, the modules represent computational components and the function calls related to communication represent the communication buses of the target implementation.

4.8.4 Behavior Hardware Model

The behavior hardware model has detailed implementations of the communication buses of the target system. The communication protocols of the target implementation are inserted into the processing elements. Instead of the abstract communication interfaces used in the transaction-level model, wires represent the communication buses and pins are added to the processing elements so that they may be connected to the wires. The computational timing is approximate-timed, so the key difference

between the transaction-level model and the behavior-hardware model is whether the communication aspects are abstract or accurate.

4.8.5 *Register-Transfer Level Model*

The register-transfer level model (RTL) is the most accurate model supported by SystemC. All of the communication, computation, and architectural aspects of the target system are defined explicitly. Timing characteristics of both the computational and communication elements are clock-cycle accurate. At this layer of abstraction, the SystemC code representing the hardware components may be translated to an HDL that can be synthesized and the SystemC code representing software is translated into the desired software programming language.

4.9 Interface

Though interface is a general concept in all object-oriented programs, let us recap this concept. An interface declares a set of methods (pure virtual functions). An interface is an abstract base class. The pure virtual functions are implemented in concrete class. One can do different implementations adhering to the basic interface definition. In this way, the behavior can be changed by sticking to the way the object is used. The interface is universal, just like the male and female of any connector and the way they communicate. Just like the USB interface. Multiple vendors implement the logic sticking to the USB interface definition.

4.10 Channel

Channel is thus an object that serves as a container for communication and synchronization. To construct complex system-level models, SystemC uses the idea of defining a channel as an object that implements an interface. An *interface* is a declaration of the available methods for accessing a given channel. By distinguishing the declaration of an interface from the implementation of its methods, SystemC promotes a coding style in which communication is separated from behavior, a key feature to promote refinement from one level of abstraction to another. In SystemC if you want modules to communicate via *channels*, you must use ports on the modules to gain access to those channels. A port acts as an agent that forwards method call up to the channel on behalf of the calling module.

Hierarchical channels form the basis of the system-level modeling capabilities of SystemC. They are based on the idea that a channel may contain quite complex behavior; for instance, it could be a complete on-chip bus. Primitive channels, on the

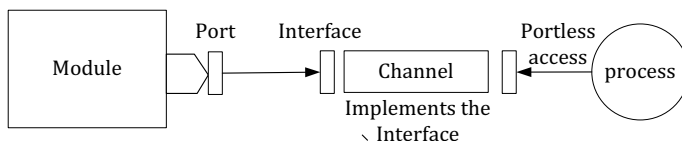


Fig. 4.27 Interprocess communication using a channel

other hand, cannot contain internal structure and so they are normally simpler. For example, `sc_signal` behaves like a primitive channel (Fig. 4.27).

A module calls the interface methods via its *port*. The communication mechanism can be changed by modifying the channel interface implementation. A port can read a channel using `Read` method of the channel interface. Similarly, a port can *write* to a channel using the `write` method of channel interface. Interfaces and ports describe what functions are available in the communication.

4.10.1 Primitive Channels

There are two types of channels *primitive* and *hierarchical*. Primitive channels are atomic in nature. Primitive channels are used if we need to use the request update/update scheme in the implementation. Primitive channels do not contain processes and do not access other channels. An example of a primitive channel is SC-signal as shown in Fig. 4.28. The other examples are `sc_fifo`, `sc_mutex`, `sc_semaphore`...

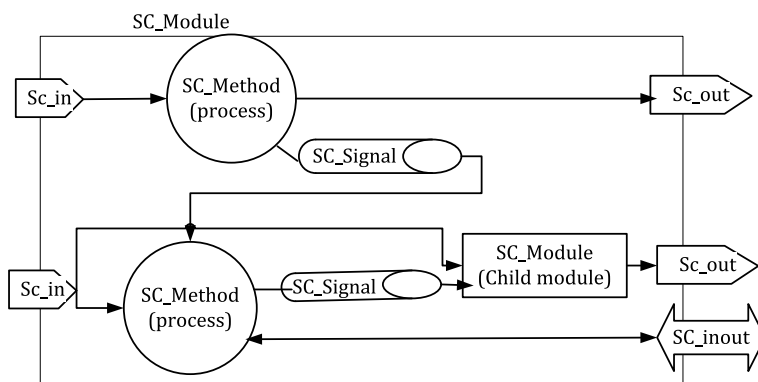


Fig. 4.28 `SC_signal` as a primitive channel

4.10.2 Hierarchical Channels

Hierarchical channels are derived from `sc_channel`. These are modules that can implement one or more interfaces. Like modules, hierarchical channels can have embedded child modules, channels, or processes. They implement methods declared in one or more interface classes defined. Because hierarchical channels encapsulate structural methods, shared data, modules, and multiple channels, complex communication protocols can be implemented.

A template for a hierarchical channel is shown in Fig. 4.29.

The communication across two modules through a channel is shown in Fig. 4.30. The communication is one way, from module 1 to module 2. The `out_port` of module 1 defines the write interface whereas `in_port` of module 2 defines the read interface. The

```

Template <class T>
Class channel_name;
Public sc_channel,
Public interface1,
Public interface2
{
Public:
    //data embers
    //can be ports,channels or variables
    //constructor;
    //instantiation of other channels
    //modules of other processes
    //interface method definitions
private:
    //data members;
Protected:
    //data members
};

```

Fig. 4.29 A template for a hierarchical channel

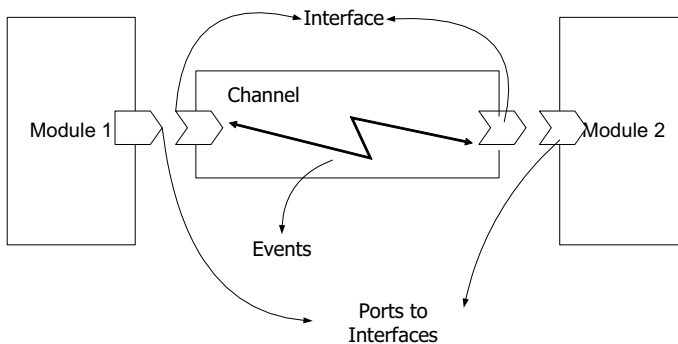


Fig. 4.30 Communication and synchronization

channel shown connecting the module implements both the write and read interfaces. The behavior of the channel and the communication protocol are implemented in the methods of the channel.

4.11 Summary: SystemC

SystemC initiative is announced in the year 1999. With a lot of refinements and versions, IEEE approves it as a standard (IEEE 1666–2011 standard for SystemC). Accellera Systems Initiative advances the SystemC ecosystem with the release of the core language and verification libraries.

SystemC is a good executable specification language for SoC designs, transaction-level modeling (TLM) and hardware/software co-design paradigms. We studied and observed that PSM is a suitable model for embedded systems design and it gets mapped to language constructs effectively through SystemC. The constructs are also synthesizable.

With several programming methodologies, one gets confused about which one to adopt in development. Figure 4.31 depicts different levels of abstraction and supporting programming tools. SystemC lies at the highest abstraction level with software and hardware seamless modeling which is essential for embedded systems. SystemC models from top to RTL level where other methods like Verilog taking gate level and switch level modeling. In an emerging design and verification paradigm, design teams elaborate SystemC-based designs with System Verilog-based RTL as implementation proceeds. They intermingle SystemC and System Verilog to speed up the co-simulation of hardware/software SoC designs.

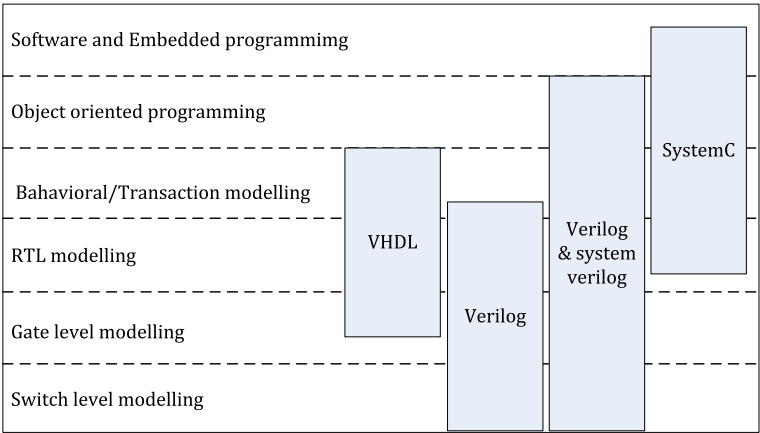


Fig. 4.31 Design language overlap

4.12 Further Reading

The book “A system-C primer” by Bhasker (2002) gives a good startup for learning SystemC in detail. The book “System design with system”-c, Grotker (2002) also covers extensively with good examples. The final reference is from IEEE systemC reference manual (2012). Also, SystemC golden reference guide by Doulos (2012) will be helpful. The system libraries can be downloaded and practiced with real-world projects.

4.13 Exercises

- (1) Develop a model using SystemC for a stopwatch with the below specifications:
 - Two-digit display of seconds.
 - Resets to zero if it exceeds 99.
 - Has two input buttons: reset and start/stop.
 - When reset is pressed, count resets to zero and starts counting.
 - Start/stop button toggles to stop and resume the counting.
- (2) Refer to case study 3.11 where a system has to be designed to monitor the temperatures of an industrial process. Model the system and implement it in SystemC.
- (3) A communication system has a transmitter (TX) and receiver (RX). The system implements a simple protocol of communication as mentioned below:
 - A data message is transmitted by TX.
 - TX waits for an acknowledgment message (ACK).
 - If ACK is received in T_{ack} seconds, TX sends the next message.
 - If ACK is not received in T_{ack} seconds, it re-transmits the message.
 - If no ACK is received for three re-transmissions, the message is aborted and the next message is sent.
 - Rx reads a message when a message is received.
 - RX sends ACK message in response to a data message received.

For simplicity assume

- TX and RX have infinite buffers.
- TX has infinite messages to send.
- Rx does not detect duplicate messages received.

Represent the behavior of TX and RX as a program state machine and implement in SystemC.

- (4) Develop a model and implement using SystemC for the below specifications.
 - A system detects moving vehicles and measures inter-arrival times (IAT) in seconds.

Fig. 4.32 Splitter

- A short pulse is generated when a vehicle passes by and is input to the system.
 - The time between two successive pulses is the inter-arrival time (IAT) of the vehicle.
 - The system computes and displays the average IAT and number of vehicles passed during the last 1 h.
- (5) Below is a module that splits the input data stream into two identical streams. The rate at which data streams-in and streams-out are not exactly the same. Assume the data is 8-bit wide. Implement the module in SystemC (Fig. 4.32).

References

- About SystemC: the language for system-level modeling, design and verification. Accellera Systems Initiative
- Aynsley J, Here's exactly what you can do with the new SystemC standard! Doulos, Ringwood, UK
- Bhasker J (2002) "system-C primer" Star Galaxy Publishers
- Doulos (2012) SystemC golden reference guide
- Dömer R (2000) System-level modeling and design with the SpecC Language. Doctoral dissertation. Department Computer Science, University of Dortmund
- Edwards SA (2003) Design languages for embedded systems. Columbia University, New York
- Gajski D, Vahid F, Narayan S, Gong J (1994) Specification, and design of embedded systems. Prentice, Hall
- Grotker T (2002) System design with system-c. Kluwer Academic Publishers
- Grüttner K, Modelling program-state machines in SystemC, OFFIS Research Institute, Oldenburg, Germany
- IEEE Standard SystemC(R) (2012) Language Reference Manual
- Introduction to systemC tutorial—esperon
- SystemC user guide V2.0
- System C tutorial, John Moondanos Strategic CAD Labs, INTEL Corp
- Walstrom RD, System level design refinement using SystemC. M tech thesis. Graduate College Iowa State University
- 1666–2011—IEEE Standard for Standard SystemC Language Reference Manual Revision of IEEE Std 1666–2005

Chapter 5

UML for Embedded Systems



Abstract One of the models we studied in Chap. 3 is the heterogeneous object-oriented model. The majority of software systems are implemented using this model, as it is close to real-life systems. Unified Modeling Language (UML) is an object-oriented modeling language standardized by Object Management Group (OMG) mainly for software system development. In software development, UML has become de-facto standard as CASE methodology. Now it is invariably used in embedded systems because of the growing complexity of embedded systems. In this chapter, we use the term system engineering as an equivalent CASE for embedded systems. An embedded system is represented in UML using multiple models through different diagrams. Each model describes the system from a distinctly different perspective. In industry, any project cycle has several people involved in different roles with certain tasks assigned. Section 5.2 describes typical tasks and roles in system engineering. Section 5.3 describes different diagrams supported in UML. Section 5.4 describes with examples, different structural diagrams, viz, class, association, concept of aggregation, composition, signals, and interfaces. Section 5.5 explains different behavioral diagrams with examples, viz, use cases, state, activity, and sequence diagrams. To summarize, Unified Modeling Language (UML) is very established standard to represent a system, its static and dynamic behavior. End to end development of systems, whether it is software system or any embedded system, can be specified. The models can be easily exchanged across design teams. The UML models will be very useful for documentation, skeleton code generation, and verifying the behavior well before coding. Though several books are available to learn UML, we have included this chapter just sufficient to start off the design with UML standard and to represent the extensive set of models we have studied in Chap. 3.

Keywords Unified Modelling Language (UML) • CASE methodology • UML elements • UML connectors • UML model diagrams • Class diagram • Association class • Aggregation • Composite aggregation • Generalization • Signals • Component • Behavioral diagrams • Activity diagram • Sequence diagram

5.1 Motivation

Let us recap what we have learned till now. In Chap. 1, we have studied basic characteristics of an embedded system (ES), important metrics to be considered before design, improving versatility in design and the platform in which to architect the system. All this is for good marketability of the product. In Chap. 2, we discussed the structured methodology of developing use cases with customer interaction so that requirements can be framed in a robust way. In Chap. 3, we have discussed different models by which the practical problem is mapped to appropriate model, analyzed, and verified. At this stage, the model is an abstract representation. In Chap. 4, we studied one executable specification language, systemC which is well suited for system-level design of ES. We studied how the selected models are represented in the executable specification language (ESL), execute, and verify the behavior.

One of the models we studied is the heterogeneous object-oriented model. The majority of software systems are implemented using this model, as it is close to real-life systems. All CASE (computer-aided software engineering) tools support modeling, design, and code generation for object-oriented systems. Unified Modeling Language (UML) is an object-oriented modeling language standardized by Object Management Group (OMG) mainly for software system development. UML is used for specifying, visualizing, analyzing, and documenting the artifacts in the software development process. It helps in representing the models in a standard way and helps in understanding the systems to be constructed. It is used to understand, design, browse, configure, maintain, and control information about such systems. UML provides standard diagrams to capture static as well as dynamic behavior of a system.

In software development, UML has become de-facto standard as CASE methodology. Now it is invariably used in embedded systems because of the growing complexity of embedded systems. Moreover, embedded systems are becoming more complex with complex features, which are mostly driven by software. Proven computer-aided software engineering methodologies (CASE) using UML are now adapted to handle upper layers of software in embedded systems. ES designers are resorting to software and system engineering from CASE methodologies adapting too many well-practiced concepts like abstraction.

In this chapter, we use the term system engineering as an equivalent CASE for embedded systems. UML is a standard notational language. Starting from use cases, specifications, model-based analysis, representing them as standard diagrams, and documentation are the parts of software engineering process supported at the design level by UML. Hence, ES designs are adapting UML as the standard modeling language.

Due to extension mechanisms offered by UML, it can be tuned by definition of a set of stereotypes and constraints for embedded applications. UML furnishes a good support for visual modeling, fast design space exploration, and automatic code generation. As UML has matured enough, developers focus on designing at abstract level and go to coding level, which is a healthy practice. UML thus forces strong

disciplinary procedures in ES development. UML tools also allow code generation, once you are through with abstract-level design. So designers will concentrate more on true design rather simply coding. Another advantage of UML is its use in formal verification of the system design at abstract level itself.

A system is represented in UML using multiple models through different diagrams which we will discuss in detail in subsequent paragraphs. Each model describes the system from a distinctly different perspective.

ES development is no more on simple processor architecture and coding some modules in C. As the complexity grows, much development focus is to rise to abstraction levels. This improves productivity. Design errors can be identified at earlier stages, better documentation can be provided, and ES designers can collaborate more effectively.

UML standardizes the syntax and semantics of the model diagrams. They are standards by which they can be exported into another platform. However, UML does not prescribe how to model and implement. This is left to the developer. One thing to note, UML is not a prescriptive process for modeling software systems.

In this chapter, we will study essential UML views and UML diagrams useful in ES design with case studies.

5.2 Typical Tasks and Roles in System Engineering

In industry, any project cycle has several people involved with different roles with certain tasks assigned. The goal is to complete the project successfully. Below is a typical-list of roles and the tasks they perform. This list is just for understanding the role of UML at each stage. The roles and tasks vary considerably in real-world industries. At each stage, UML functionality is going to help for achieving the goal (Table 5.1).

5.3 UML Diagrams

UML tools are developed based on UML standards. Currently, 2.x version is active. The standard defines rules and notations for specifying business and software systems. The notation supplies a rich set of graphic elements for modeling object-oriented systems, and the rules state how those elements can be connected and used. UML is not a software development language. Instead, it is a visual language for defining, modeling, specifying, and communicating.

UML 2.x defines diagrams that contain UML elements connected by UML connectors. UML model diagrams, like state machine diagram, represent various aspects of the system to be developed, environment and business processes of the system, see Fig. 5.1. UML elements represent the objects and actions within the system (like state in state machine diagram), arranged by relationships represented

Table 5.1 Typical tasks and roles in system engineering (Enterprise Architect User Guide 2010)

Role	Tasks
Business Analyst	Responsible for modeling requirements, high-level business processes, business activities, workflows, system behavior
Software Architect	Responsible for mapping functional requirements of the system, mapping objects in real time, mapping the deployment of objects, defining deliverable components
Software Engineer	Responsible for: mapping use cases into detailed classes, defining the interaction between classes, defining system deployment, defining software packages and the software architecture
Database Developer	Responsible for: developing databases, modeling database structures, creating logical data models, generating schema, reverse engineering databases
Tester	Responsible for: developing test cases, importing requirements, constraints and scenarios, creating quality test documentation, tracking element defects and changes
Project Manager	Responsible for: providing project estimates, resource management, risk management, maintenance management
Developer	Responsible for: forward, reverse, and round-trip engineering, visualizing the system states, visualizing package arrangements, mapping the flow of code
Implementation Manager	Responsible for: modeling the tasks in rolling out a project, including network and hardware deployment, assigning and tracking maintenance items on, elements (issues, changes, defects, and tasks)
Technology Developer	Responsible for creating or customizing: UML profiles, UML patterns, code templates, tagged value types, MDG technologies, Add-Ins

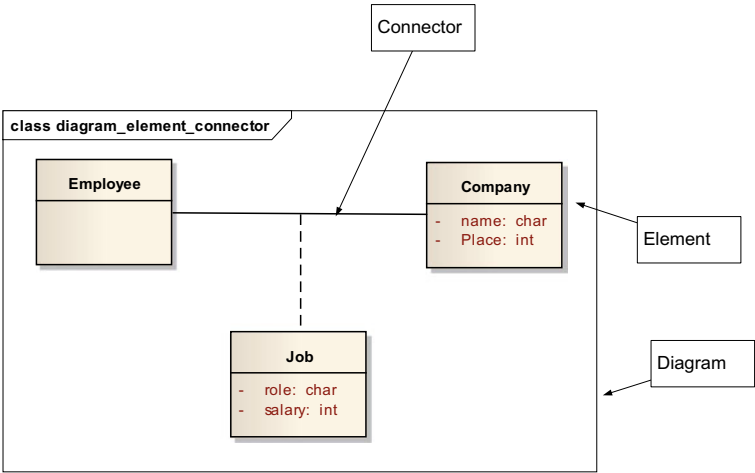


Fig. 5.1 UML diagram—elements and connectors

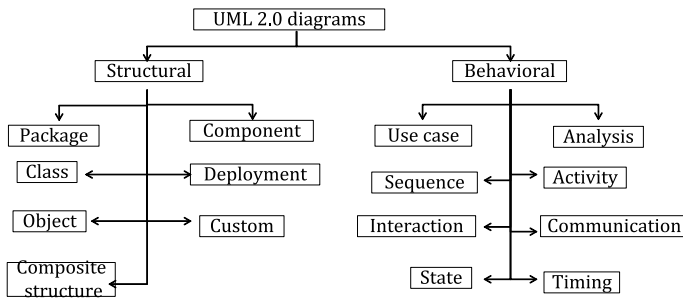


Fig. 5.2 UML2.0 diagrams, Courtesy Sparx systems (Enterprise Architect User Guide [2010](#))

by UML connectors. UML connectors, along with elements, form the basis of a UML model. Connectors link elements together to denote some kind of logical or functional relationship between them. Each connector has its own purpose, meaning, and notation and is used in specific kinds of UML diagrams.

Figure 5.2 shows the structural and behavioral diagrams supported by UML 2.0. As the names explain, the structural group of diagrams depicts the static characteristics. They explain the way the elements are associated, their connectivity and how they are hierarchically contained, etc. The behavioral diagrams depict the dynamic characters of the objects. They explain how they react to the inputs, their interaction with other objects, and the communication across the objects and the state change, etc.

5.4 Structural Diagrams

1. Class diagrams—capture the logical structure of the system.
2. Composite structure diagrams—represent structural organization of the elements and their interconnectivity using element connectors. Each connector has associated semantics with specific meanings attached (Ex: if e1 and e2 are connected, the connector may mean “is a part of”).
3. Component diagrams—represent a component that is a reusable sub-system with clear interface. It is like a physical component like valve with specific functionality and the way it can be connected with other systems.
4. Deployment diagrams—show how and where the system is to be deployed, i.e., its execution architecture.
5. Object diagrams—depict object instances of Classes and their relationships at a point in time.
6. Package diagrams—Depict the organization of model elements into packages and the dependencies amongst them.

Let us get into detail of important structural diagrams keeping embedded system design in view.

5.4.1 Class Diagram

Class model represents the classes and their association. A class has attributes that are the properties of the class. The behavior of the class is represented by methods. PI refers to case study 3–10. The class model for this problem is represented in Fig. 5.3. In the problem definition, we have come across the real-world objects electric pole, road, and digitizer. One way to classify these objects is from their properties. An electric pole can be represented by a point with height as its attribute. The road can be represented as a sequence of points. The road has attributes like its name and length of the road. There should be a way to generalize all the electric poles and roads. They are physical entities on the ground, which is to be digitized. Let us define the class as *Entity*. Electric poles and roads are entities but have different properties. As both types of entities have point data, let us define *point* as a class with attributes *x* and *y* coordinates. Now let us define the electric pole completely by relating it with point class and entity by the statement “*Electric pole is an entity. It has one point object and its attribute is height*”. Hence the relation between entity and point is “*is a*” and the relation between pole and point is “*has a*”.

A road is an entity from the above definition. However, it is represented by a sequence of points. So let us define a class “point sequence” which holds a sequence of points. Now the relation between point and point sequence can be done in multiple

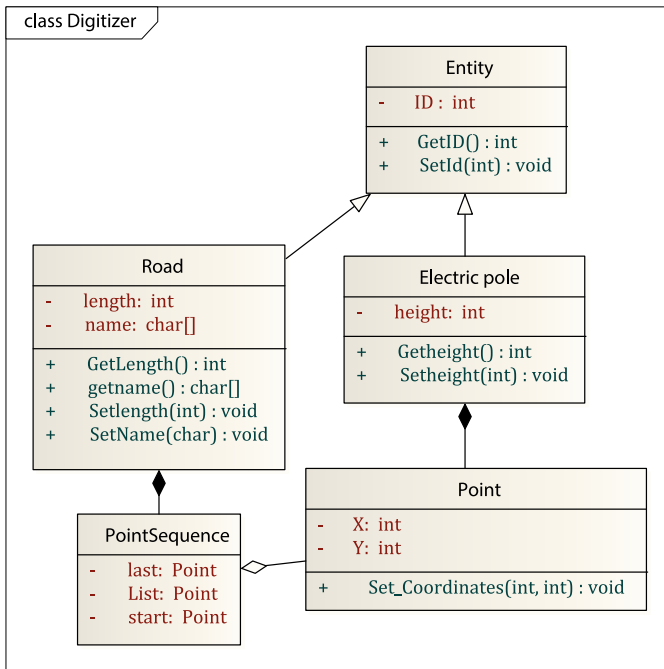


Fig. 5.3 Class diagram

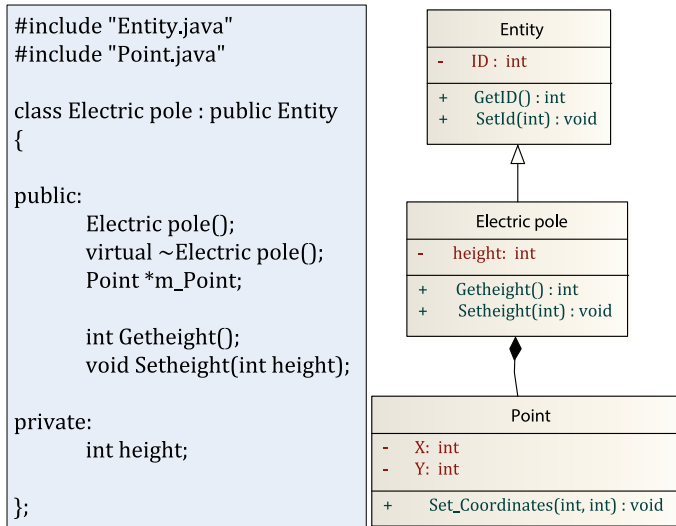


Fig. 5.4 Class diagram to C++ mapping by CASE tool

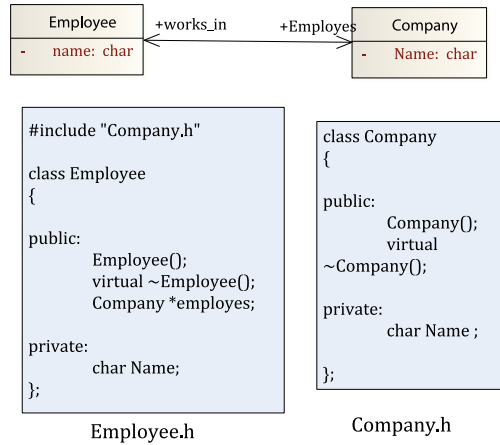
ways. One way is that point sequence contains the start point, an endpoint and remaining is a list of points. The road has a point sequence and has name and length as attributes. Thus the five classes are defined and related by associations. Hence the class diagram captures the logical structure of the system. It describes the problem as a static model and the association across them. Let us see how one of the classes in the class diagram gets converted to skeleton C++ class to implement the logic in the methods. For simplicity, let us not bother how the relations get mapped. Just observe the class structure, properties, and methods. Below shows for the electric pole class (Fig. 5.4).

UML diagrams can be the input for source code engineering. Basic class declaration with constructor method, private, and public variable and methods get declared. The logic for Getheight and Setheight has to be implemented.

5.4.2 Association

Every class has certain properties encapsulated with its behavior. A class provides services that are published by declaring them public. The classes have to be associated to utilize services of other classes. UML provides an association connector to define association between two classes. You can mention the roles of each on the connector ends. Below is a very popular example relating an employee class with the company class where he is working. In this example, employee is the source and his role is “works_in” and target is company, its role being “employees”. In literal language, it can be mentioned “Employee A *works_in* CompanyB” “company B *employees*

Fig. 5.5 Employee is associated with the company



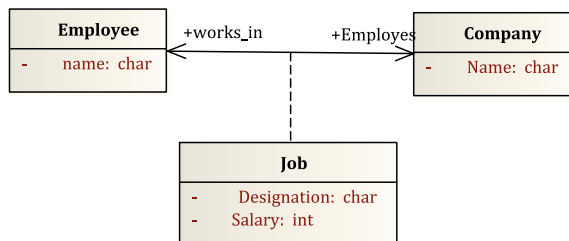
EmployeeA”. The UML association connector get coded in C++ code as shown in the code below. Employee class has a pointer to the Company class. The association in this case is unidirectional from employee to company. One can have association connector bi-directional also in which case each class has pointer to the associated class. This is a 1–1 relation in data models (Fig. 5.5).

5.4.3 Association Class

In the above example, there is an association between the two classes employee and company. The association is shown as “works in”. The association means, it is a job. The association can have attributes like job’s role and job’s salary, etc. So we can associate a class to the association connector itself, explaining what the association describes (Fig. 5.6).

An Association Class connector is a UML construct that enables an Associate connector to have attributes and operations (features). This results in a hybrid relation with the characteristics of a connection and a Class. Association class is thus a model element that has both association properties and class properties. An Association

Fig. 5.6 An association class



Class can be viewed as an association between objects which has class properties. It can also be viewed as a class that has association properties. It not only connects a set of classes but also defines a set of features that belong to the relationship itself and not to any of the classes. When you add an Association Class connection, you are creating a new Class that is connected to the Association. When you delete the Association, the Class is also deleted.

5.4.4 Case Study 1

A sensor can measure multiple parameters like level, flow, etc. A parameter can be measured by different types of sensors. Each such measure has a quality factor (1.4) and certain accuracy. Represent this association as classes in UML and show the engineered code (Fig. 5.7).

A sensor can measure a list of parameters. Similarly, a parameter can be measured by a set of sensors. This is many to many relation. This is representation by the

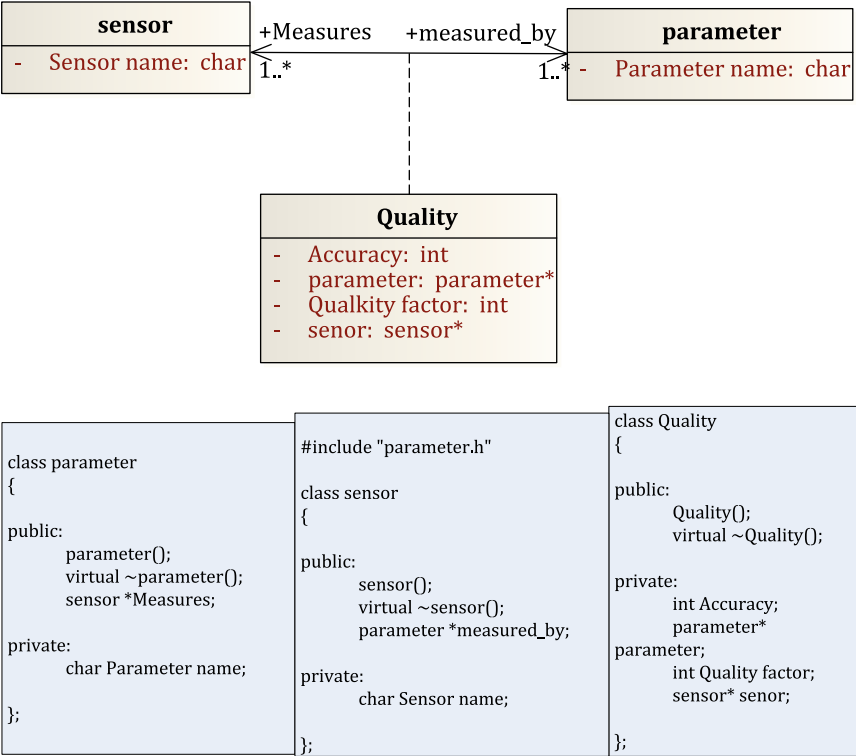


Fig. 5.7 Association class example

association connector cardinality. Code generated for sensor contains a pointer to parameters it can measure. Similarly, the parameter class contains the pointer to sensors it can measure. This is association class. Hence, each measure has a quality entity. The quality class holds the pointer to the sensor and pointer to the parameter it is reading and its quality and accuracy factors. If you are acquainted with databases this is like two tables with a list of sensors and parameters and interconnected by quality table. So this example shows how you can implement many to many association with the attributes of each such association.

5.4.5 Aggregation

Aggregation is-a-part-of relation. The electric pole *lies on* a point. So the electric pole class has the point class as a part of it. The point class and electric pole are independent classes. When the electric pole is deleted, the point class remains. Moreover, multiple classes can aggregate with same point class, i.e., multiple entities may lie at the same point. Aggregation is a type of association between two classes where one is a part of the other both having independent behavior.

Aggregation is used to define complex elements by aggregating other simple or complex elements (for example, a car from wheels, tyres, motor, and so on. Observe: wheel, tyre, and car are independent elements. When tyre is made part of wheel and this complex element is then made part of car for the desired mobility) (Fig. 5.8).

A stronger form of aggregation, known as Composite Aggregation, is used to indicate ownership of the whole over its parts. The part can belong to only one Composite Aggregation at a time. If the composite is deleted, all of its parts are deleted with it.

Fig. 5.8 Aggregation

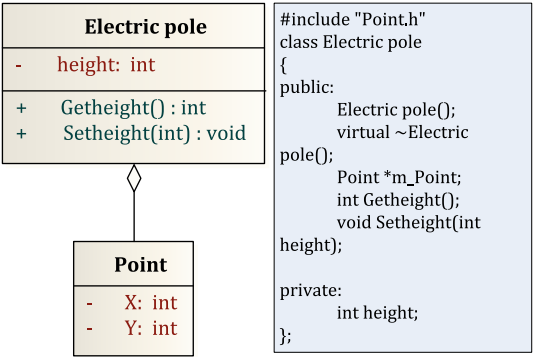
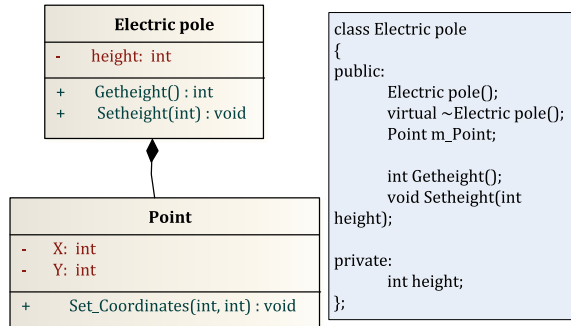


Fig. 5.9 Composite aggregation

5.4.6 Composition

A Composite aggregation is used to depict a complex element that constitutes more granular components. A component can be included in a maximum of one composition at a time. If a composition is deleted, all of its parts are deleted. As a standard example, a carburetor of an automobile is fabricated by composing multiple parts. As an example, a carburetor has fuel injection control. When the carburetor is faulty and is removed, all its components are removed (Fig. 5.9).

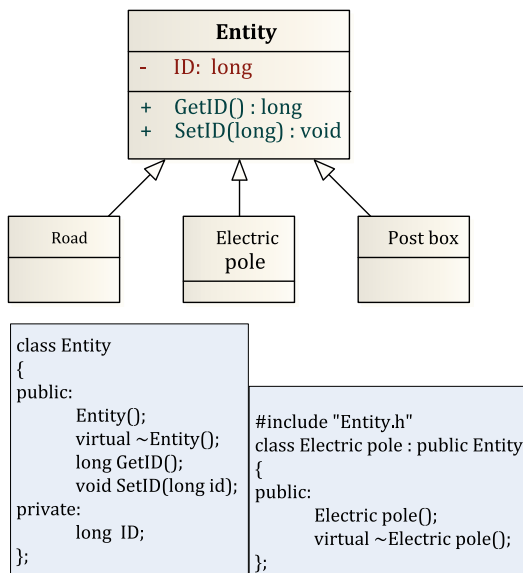
Compositions are transitive, asymmetric relationships and can be recursive. As stated by UML standard, Composite aggregation is a strong form of aggregation. This requires a part instance be included in one composite only at a time. If a composite element is deleted, all of its composed elements are deleted with it.

Composition is a good way of showing properties that own by value or properties that have a strong and somewhat exclusive ownership of particular other components.

5.4.7 Generalization

Generalization is a bottom-up approach in which two lower level classes combine to form a higher level class. In generalization, the higher level class can also combine with other lower level classes to make further higher level class.

It is implemented as Superclass and Subclasses in object-oriented languages. Hence, classes are combined to form a more generalized class, in other words, sub-classes are combined to form a super-class. A sub-class inherits the characteristics of the super class. It can further specialize by overriding the behavior of super class. As per UML specification, *a generalization is a taxonomic relationship between a more general class and a more specific class. Each instance of the specific class is also an indirect instance of the general class.* Thus, the specific class inherits the features of the more general class (Fig. 5.10).

Fig. 5.10 Generalization

5.4.8 Case Study 2

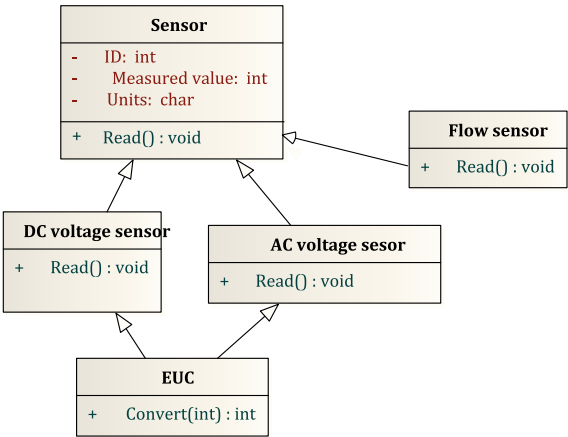
A customer has described the problem as a textual statement as given below. Identify the objects, classes, and their associations from the statement. Define the properties and methods of identified classes.

V1 is a voltage sensor which measures DC voltage in volts. V2 measures AC voltage in volts. Process of measurement of AC and DC voltages is different. F1 and F2 are flow sensors which measure the liquid flow in Engineering Units (EU). EUC is a module which converts volts to EU.

Here, we have four sensors. They have different behavior in the process of sensing. However, all of them belong to class of sensor, see Fig. 5.11. Hence, let us generalize the objects to sensor class. Irrespective of the type of sensor, the common attribute held by the class is the ID of sensor. The value it measured (does not mention how it is measured) and the units of measurement. DC sensor *is a* sensor, similarly other sensors. Hence, the three types of sensors are inherited from the generalized sensor class.

The specialized behavior of each sensor is defined in the read() method. The AC and DC sensor values have another job, i.e., to convert the measured value using read() method has to be converted to Engineering units. So, define a converter class which does this conversion. Let this converter be aggregated in the DC sensor and AC sensor. Both these sensors get the conversion done using this part. In UML language *DC voltage sensor has EU converter as part of it*.

Fig. 5.11 Case study—sensors implementation



5.4.9 Interface

An *Interface* is the specification of the behavior of an abstract class that the implementers agree to meet. The interface is implemented by a concrete class which is inherited from the abstract class. The concrete class guarantees to support the required behavior as specified in the interface. Thus definition and implementation are separated. An Interface cannot be instantiated, i.e., you cannot instantiate an object from an Interface. You must create a Class that “implements” the Interface specification, then you can instantiate the Class.

OGC UML specification states as below.

An interface is a kind of classifier that represents a declaration of a set of coherent public features and obligations. An interface specifies a contract; any instance of a classifier that realizes the interface must fulfill that contract. The obligations that may be associated with an interface are in the form of various kinds of constraints (such as pre- and post-conditions) or protocol specifications, which may impose ordering restrictions on interactions through the interface.

Interfaces are declarations and are not instantiable. Interface definition is implemented by an instantiable class, which means that the instantiable class conforms to the interface specification. A given class may implement more than one interface and an interface may be implemented by different classes.

Figure 5.12 gives an example of defining and implementing an interface. There are different types of shapes like rectangle, circle, and other shapes. We want to define an interface to move any shape by plugging in this interface. The interface definition Move is shown on the top, which has two methods move left and move right by certain units. The methods are pure virtual functions having no implementation of these methods. See the code for move interface in the right part of the figure.

This interface has been plugged in rectangle class, which implements these interface methods. Any class shape needing the move functionality can implement this interface with its own logic.

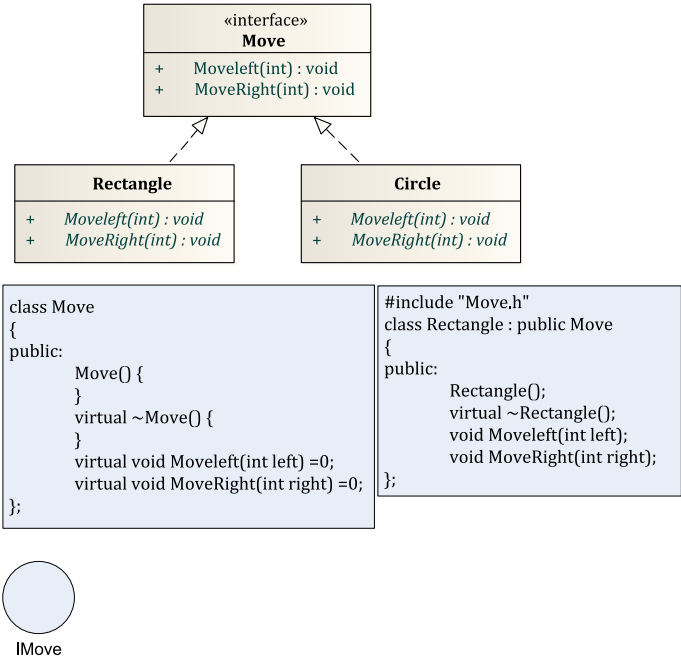


Fig. 5.12 Interface

The right portion of the code shows the rectangle class implementing the interface methods. The rectangle and other shapes can be instantiated. Observe that *Move* class cannot be instantiated as it has no implementation of its methods. They are pre-virtual functions.

The rectangle and circle classes that implemented the interface can be instantiated as shown in Fig. 5.13. The variable *m* can refer to an object of any class that implements the *Move* interface. So *m* synthetically represents the feature of being capable of movement, i.e., *m.Moveleft* and *m.Moveright*.

Fig. 5.13 Instanting interface implementations

```
class Mover {
Move m;
...
m = new Rectangle(...);
m.Move left(17);
m = new circle (...);
m.Moveright25);
```

5.4.10 Signals

In Finite State Machines, events make the system to change from one state to the other. Events thus trigger state changes. Events can be of different types. A signal is generated externally or internally by system. Any change of data or condition occurrence can be an event.

UML allows signals to be represented as stereotyped class. Other events are represented as messages associated with transitions, which cause an object to move from one state to another.

A Signal is the specification of send request communicated between objects. The receiving object handles the Received request. The data carried by a Send request are represented as attributes of the Signal.

5.4.11 Component

A component is a replaceable and executable piece of a system. Literally, it means the same. When any of the components in a physical system fail, you will replace with an *equivalent* component. The meaning of equivalence is very important. The functionality of the new component should be same as that of old one. It should match (interface) correctly as that of old component in connecting with system. The way the new component is internally manufactured is not known. The same applies to a component in embedded system.

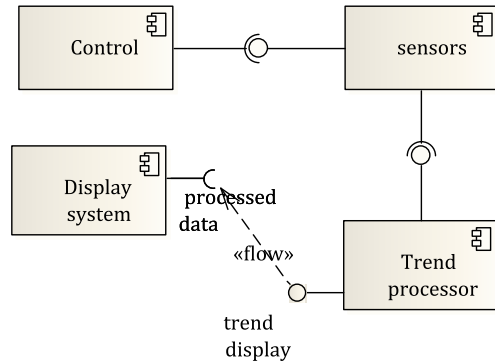
Component is a modular part of a system. Its behavior is defined by the provided interfaces. The implementation of the component is not visible and it will comply to the interface definition. Hence, a software component can be plugged anywhere in the software as far as you get the functionality as per interface definition.

A component can be composed of one or more classes, or one or more components to implement the functionality. As smaller components are integrated to create bigger component, system can be built by assembling such components. The prime advantage of component-based development is its reusability. Component diagrams show the components in the system and their interconnectivity.

In UML, *standard component represents a modular part of a system that encapsulates its contents and whose manifestation is replaceable within its environment. A component defines its behavior in terms of provided and required interfaces.*

Figure 5.14 demonstrates some components and their inter-relationships. *Sensors* component collects data from sensors and provides through the published interface for all its consumers. The internals of the sensor component are hidden except through the interface. The data are provided to the controller for generation of control signals. Both the sensor and controller are connected through assembly. The *assembly* connectors connect the provided interfaces supplied by *sensors* to the required interfaces specified by *control*.

Fig. 5.14 Component diagram



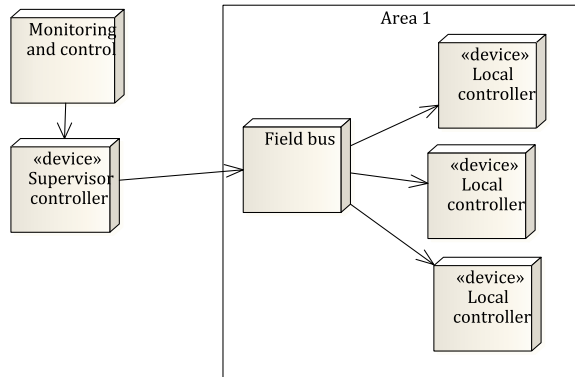
Component diagrams display various components of a software system and other sub-systems and their inter-connectivity. Component diagrams give a clear picture at higher level, how various components together make a single, fully functional system. Before modeling the component diagram, one must know all the components within the system. The working of each component should be mentioned. One should also explore each component in depth to understand the connection of a component to other physical artifacts in the system.

5.4.12 Deployment Diagram

Deployment diagram helps to view the topology of the units (hardware and software components) how they get deployed in the field. This is very essential for system engineers. The customer gets a picture how the system looks like after it is deployed. A *Deployment diagram* shows how and where the system is to be deployed; that is, its execution architecture. Hardware devices, processors, and software execution environments are reflected as nodes, and the internal construction can be depicted by embedding or nesting nodes. Artifacts (components, packages) are allocated to nodes to model the system's deployment. The allocation of the artifacts to nodes is guided by the deployment specifications.

Mostly UML diagrams represent software in terms of states, activities, processes, behavior, etc. Only deployment diagram gives a total picture of how the hardware units and the embedded software components are deployed physically.

A simple Deployment diagram for distributed digital control (DDC) in a process plant is shown below, representing the arrangement of local controllers, their connectivity through fieldbus to supervisory controller and to monitoring and control unit. Before drawing the deployment diagram, one has to identify the nodes and their relation. In the DDC diagram, the local controller node can be a smart sensor, smart actuator, and a single loop PID controller controlling a process. All this class of units has to be deployed very close to the industrial process, like pressure regulator, flow

Fig. 5.15 Deployment diagram

controller, valve closure, etc. Fieldbus is the node constituting the networking hardware and the communication software providing peer-to-peer connectivity among local controller nodes. Hence, it is represented as a node connected with local controller nodes in star topology.

Several fieldbus nodes from different areas get connected to the supervisor controller node, which coordinates all the area controllers. This node also constitutes hardware, communication, and control software components. The upper layer node is a monitoring system for display and control by operators. The topology is hierarchical across the nodes in this example.

Use deployment diagrams.

- to configure overall system
- configure each node
- study physical constraints in deployment
- retrofitting to an existing system
- and so on (Fig. 5.15).

5.5 Behavioral Diagrams

Please refer to Chap. 3 where we have described some example models, which model the dynamic nature of the system. Some of them are use case, activity-oriented, state machines, data-oriented, program state machines, object-oriented models, etc. These models are reactive in nature. They describe how the system reacts to the inputs and events and how the system outputs data and changes its state and generates events. The UML diagrams that are described below are standardized representations of these models. We will discuss very important diagrams relevant to embedded systems design.

5.5.1 Use Case Diagram

Use cases can be represented in a structured way as described in Chap. 2. UML has standardized use case representation diagrammatically. These are used to model the system functionality from the perspective of a system user. The user is called an *Actor* and is drawn as a stick figure, although the user could be another computer system or similar. A *Use Case* is a discrete piece of functionality the system provides, which enables the user to perform some piece of work or something of value using the system.

- The diagram captures Use Cases and relationships between Actors and the subject (system)
- It describes the functional requirements of the system
- It describes the manner in which outside things (Actors) interact at the system boundary and the response of the system.

Figure 5.16 shows a use case diagram. The system functionality is to provide access through biometrics, do needed operation, and logout. Also has facility to close his account. The admin can also do closure of account. The user can use the system with five functionalities as described in the diagram. The admin can do one function, i.e., closure of account. Here two use case connectors have to be explained. One connector is *extends*. By this connector one use case can extend the functionality of another use case. In the above diagram, login functionality can be extended to verify the system is available or not, before login. Other connector is “*include*” by which one use case can get included into another use case. In this example, the admin uses account closure use case while disabling the user account. These connectors will be

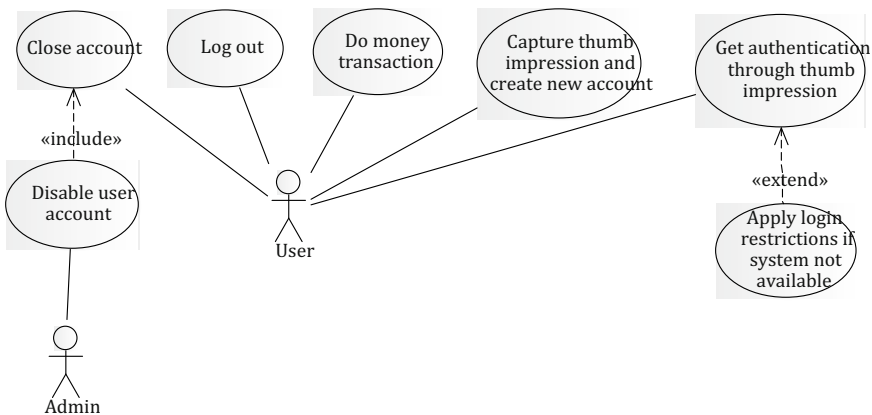


Fig. 5.16 Example use case diagram

very useful to reuse certain use cases repeatedly in different workflows. Please refer to detailed discussion on this subject in “levels” section of Chap. 2.

5.5.2 State Diagram

We have studied Finite state machine models (FSM) in detail in Chap. 3. State diagram illustrates how an entity can move between states when triggered by a set of events. As an example let us take up below stated problem.

5.5.2.1 Case Study 3

PI refers to exercise no.15 in Chap. 3. Same is included here for reference. Let us solve and draw the state machine for the below problem.

A vending machine accepts combinations of 1, 2, and 5 rupee coins to get a coke. The cost of a coke is 15 rupees. The machines validate the coins you place and release the coke if the amount placed is Rs. 15 or more. If an invalid coin is placed it aborts all coins placed. Assume the machine does not return surplus amount you inserted. Represent the problem as a state machine.

The initial state is *Idle*. It transitions to *waiting for coin*. When a coin is placed, it gets into *validating* where the token is verified for validity. If the token is invalid, it gets into *aborting coins* where the coins are purged out. When the purging process is over the *coins_aborted* event occurs which moves to state to idle again for next iteration. If the coin validation is correct, the *valid* event moves the state to *calculating* where the total amount of valid coins is updated. When the *amount < 15* is generated it moves to *waiting for coin* again. If amount is 15 it gets into *dropping item* state where the vending machine drops the item. When the *item_dropped* event occurs, it goes back to idle for next iteration. In the state machine below, all the rectangular blocks represent the state, which represents process occurring in that state. The connectors are state transitions when an event occurs. The events are generated by the processes (Fig. 5.17).

5.5.2.2 Composite States

A composite state either contains one region or is decomposed into two or more orthogonal regions. Each region has a set of mutually exclusive nodes and a set of transitions. Any state within a region of a composite state is called a sub-state. A composite state will have an initial state and a final state. A transition to the composite state represents a transition to the initial state. A transition to a final state represents the completion of activity in the composite state.

A composite state, *coin collector* is represented from the above example whose process is to wait for coins, validate and keep accumulating the amount to 15. It exits

Fig. 5.17 Solution vending machine

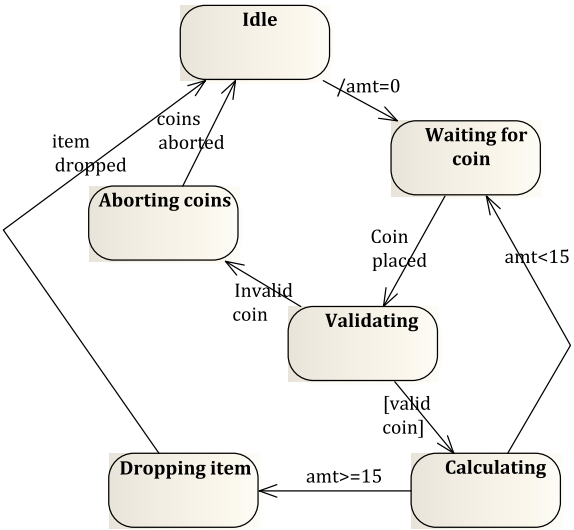
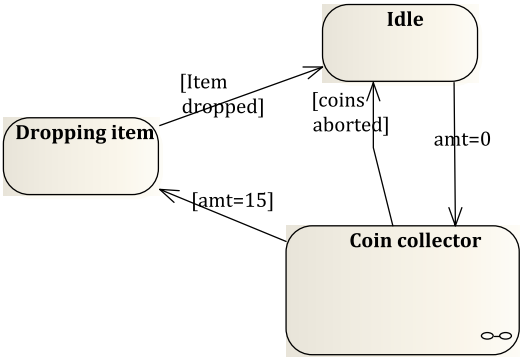


Fig. 5.18 Composite state



from composite state when an invalid coin is collected and the coins are aborted. Generates an event *coins_aborted*. It exits when `amt = 15` for dropping the item. Thus hierarchical representation helps in state machine representation (Fig. 5.18).

5.5.2.3 Pseudo States

A pseudo state is an abstraction that encompasses different types of transient vertices in the state machine graph. Pseudo states are used to express complex transition paths. Pseudo states help in ease of representation and ease of assimilation. Pseudo states are not true physical states of the machine/entity but a mechanism to represent. We will cover just few pseudo states for the sake of illustration.

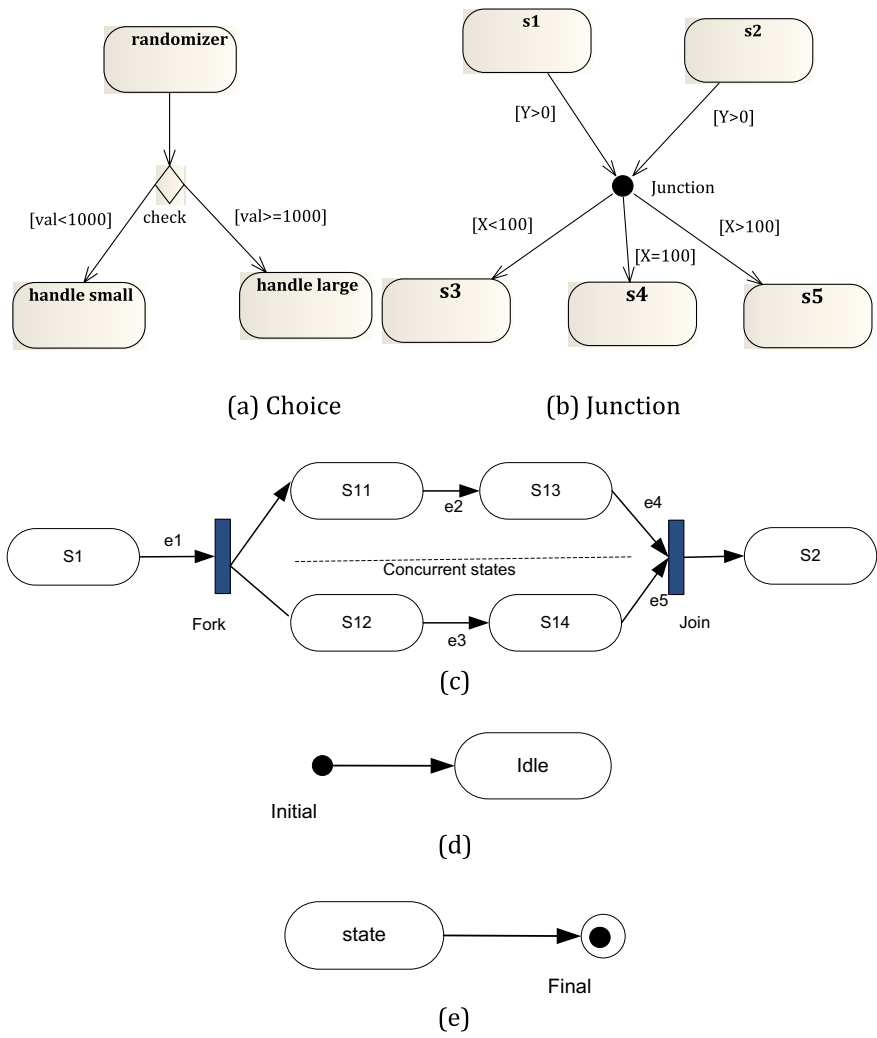


Fig. 5.19 a Choice, b junction, c fork and join, d initial pseudo state, e final pseudo state, f entry and exit points

Choice

The outgoing transition path is decided by dynamic, run-time conditions. Run-time conditions are determined by the actions performed by the State Machine on the path leading to the choice. Choice is like forking into multiple states from one state depending upon an event. In the left diagram Fig 5.19a when the random value is less than 1000, the state transitions *handle small* else to *handle large*. There can be multiple events and the choice diverts them to appropriate states.

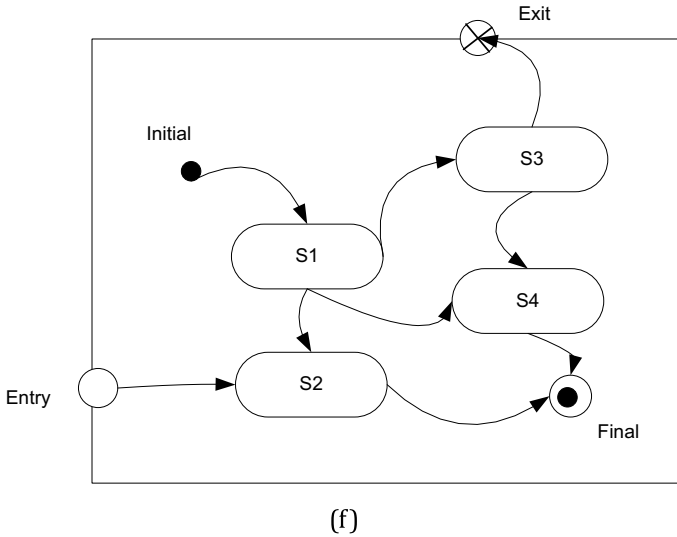


Fig. 5.19 (continued)

Junctions

Junctions are used to combine or merge multiple paths into a shared transition path. Junction can split an incoming path into multiple paths. This is like multiple roads joining at a junction. People come to junction from one path and move to other paths from the junction. Multiple states will transit to a pseudo state depending on different events and transit to physical states depending on certain events. See Fig. 5.19b, the black node at the center is a pseudo node. State 1 and state 2 transit to pseudo state and transit to three physical states from the pseudo state. If the diagram is represented without the pseudo state, it requires $(2 \times 3 = 6)$ transitions whereas with pseudo state it needs $(2 + 3 = 5)$ states.

Fork and Join

Fork and Join are used in state machine diagrams and in activity diagrams. A state on the occurrence of a specific event creates multiple concurrent transitions to different states. The created states transit to different states depending on the events occurring in their respective states. This is represented by *Fork* pseudo state.

Join is the mechanism when multiple states transit to a single state on the occurrence of certain events in the multiple states. In the above diagram on the occurrence of e1, the machine moves into concurrent behavior with two states s11 and s12 running concurrently. Again after certain state transitions, the two states s13 and s14 merge to a single state s2. Both the states s13 and s14 wait for each other to get into e4 and e5, respectively, on the join pseudo state. Though above is explained in state

transition, similar behavior occurs in activity diagram where each rectangle shown is a process.

Initial

Initial pseudo state is used in activity and state machine diagrams. In state machines when the machine is ON or it enters into a composite state, initial pseudo state points to the first state where the machine enters. In the diagram above, the machine enters *idle* state initially. In the activity diagrams, this points to the initial process when the activity is invoked...

Final

Final pseudo state is used in state diagrams and activity diagrams. When the machine reaches to the final state, the activity gets completed. No more transitions take place. A machine may have multiple final states. When it reaches first final, it stops the activity. Same happens in the activity diagram, where no more activity flows occur once it reaches final point.

Entry Point

Entry point is the pseudo state by which the machine (a region or composite state) is entered. It is shown as a circle on the boarder of the region or composite state. It is a transition to a single vertex within in the region, see Fig. 5.19f where entry point transits to s2.

Exit Point

Similar to the entry point, exit point is the point of exit from a composite state. It is shown as a small circle with a cross on the boarder of the composite state.

This is like you entered into a restaurant (entry point), start interacting with others (initial), continue interacting with others and changing your state (active) stop your activity (Final) and exit from restaurant (exit point). So the entry point and exit points provide better encapsulation of composite states and help avoid “unstructured” transitions.

5.5.3 Activity Diagram

Activity diagrams are used to model the behaviors of a system, and the way in which these behaviors are related in an overall flow of the system. This is very close to the data flow model we discussed in Chap. 3, see Fig. 3.28. The logical path a process follows is based on various conditions like concurrent processing, data access, interruptions.

As a simple example, the activities involved in opening the door of ATM room are depicted in Fig. 5.20. Activity diagram has the initial start and final closure states. The process flow starts from initial state. In our example, the first process reads the ATM card swiped. Based on the validity, the process flow will be to *door open* or end activity (if invalid). Once door open process is over, the process forks to two simultaneous processes to *put on the lights* and also *capture the image*. *Capture image* is passed to the save process through an object as information flow. The objects passed across processes are shown at the tip of the process. After saving, the two processes join and finally enter *close the door* process. Then it reaches to end state. This is just a simple example of door access unit to show how the processes communicate and

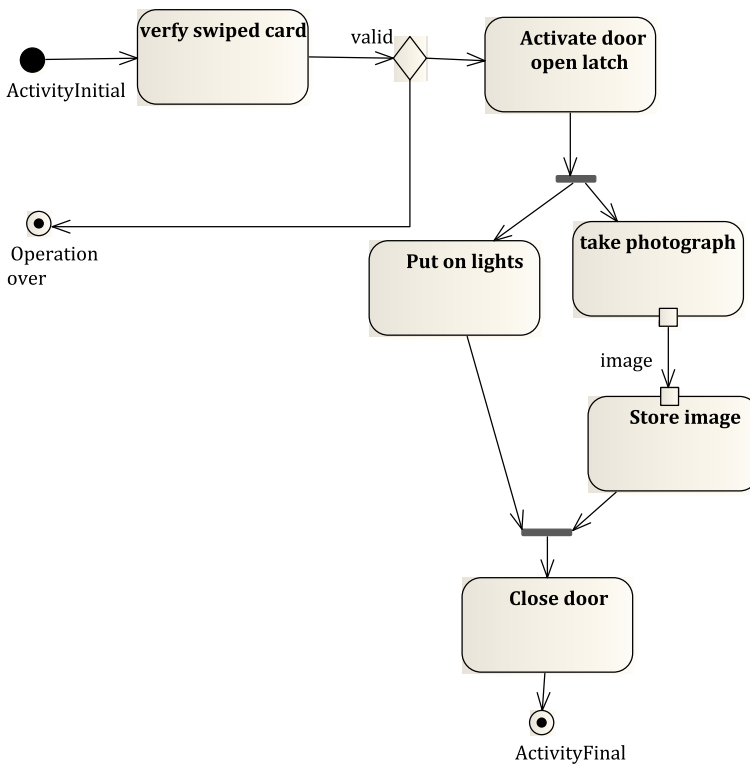


Fig. 5.20 Activity diagram

how the overall processes flow from start to end. You can create composite activities so that analysis can be done hierarchically.

5.5.4 Sequence Diagram

We have structurally analyzed object-oriented representation in structural diagrams. We defined the classes and the relations among them with different types of associations. The associations are necessary so that the instanced objects from the classes communicate and share the services provided by an object to other objects and vice versa. A complex workflow gets initiated by an actor (recollect the use cases we discussed) and initiates it through the method provided by an object. This triggers a chain of actions through communication among objects till the results of the workflow have reached. While the sequential action takes place, any object can create other objects, get the services done and delete the object. This is very similar to how real-world projects like building constructions, roads are executed. You define the sequential process to be executed, depute the persons, do the job, and remove them after that task is over.

Sequence diagram is thus a structured representation of the behavior as a sequence of actions. The objects cooperate by providing services and utilizing the services from other objects to complete the triggered behavior and achieve a result. The objects involved are arranged in a horizontal sequence, with messages passing back and forward between elements. Messages on a Sequence diagram can be of several types. Messages are the operation requests on target objects. An actor element can be used to represent the user initiating the flow of events. Stereotyped elements, such as Boundary, Control, and Entity, can be used to illustrate screens, controllers, and database items, respectively. Each element has a dashed stem called a lifeline, where that element exists and potentially takes part in the interactions.

As a simple example let us take the code below and represent as sequence flow (Fig. 5.21).

Classes A and B are associated. B provides certain services through methods op2 and op3 to class A for its method op1. An actor instances A and calls op1. This

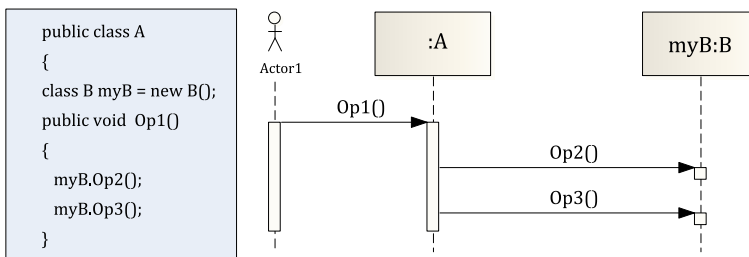


Fig. 5.21 Sequence diagram

instances op1 method, shown as vertical column under A which in turn calls op2 and op3 of B. The horizontal lines are basically way of communication, messaging across objects.

5.5.5 Case Study 4

A communication system has a transmitter (TX) and receiver (RX). The system implements a simple protocol of communication as mentioned below.

- A data message is transmitted by TX
- TX Waits for Acknowledgment message (ACK).
- If ACK is received in T_{ack} seconds, TX sends next message.
- If ACK is not received in T_{ack} seconds, it retransmits the message.
- If no ACK is received for three retransmissions, the message is aborted and next message is sent.
- Rx reads a message when a message is received.
- RX sends ACK message in response to a data message received.

For simplicity assume

- TX and RX have infinite buffers
- TX has infinite messages to send.
- Rx does not detect duplicate messages received.

Questions

- Identify the classes and their relations using structured diagram.
- Define the attributes and methods for each class
- Represent the TX and RX behavior as a FSM. Draw state machine diagrams.
- Represent transmit operation using a sequence diagram.

5.5.5.1 Solution

The state machine for TX and RX is shown in Fig. 5.22, which is self-explanatory.

Figure 5.23 is the typical class diagram for TX and RX. TX class holds message to be sent and ACK messages and the state of ACK for current message is held. The state of current message transmit is also held with it. Uses send service of Xmit class. TX aggregates a buffer class to hold the messages to be transmitted.

RX receives a message using the Receive class services. Verifies if it is ACK message or data message. Post the messages in appropriate buffer. RX has aggregated buffer class to hold received messages. In fact both TX and RX point to same buffer for message handling. Buffer has to manage the message content in the buffer. Buffer management services are through aggregated message class.

Figure 5.24 is the sequence diagram showing the interaction among the defined classes above for message transmission. 1. User puts the message in buffer

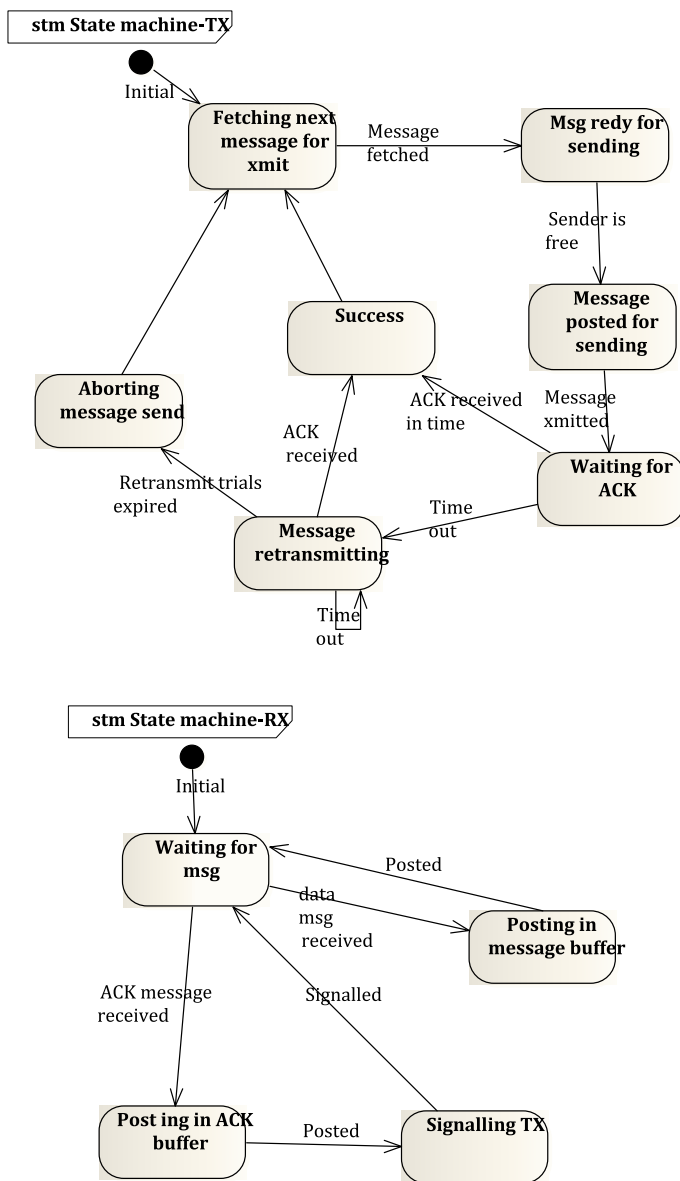


Fig. 5.22 State machine for TX and Rx

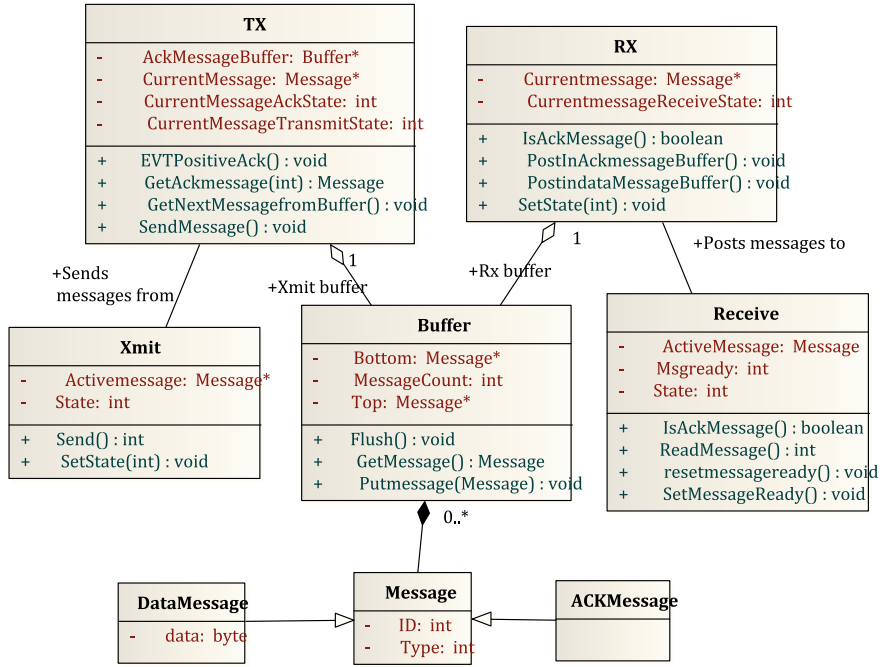


Fig. 5.23 Class diagram for TX and Rx

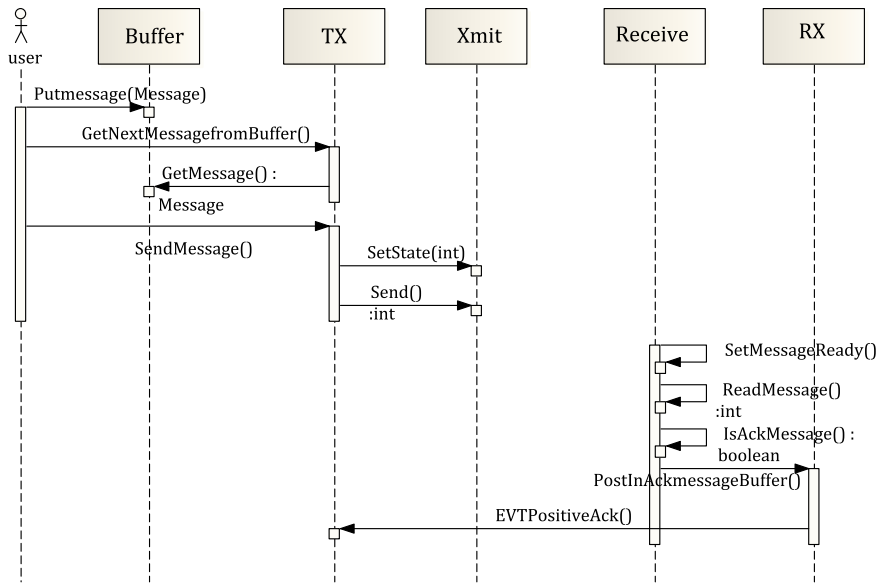


Fig. 5.24 Sequence diagram for message transmit

(buffer.putmessage (message)). 2. User invokes to send next message from buffer. 3. Tx gets message from buffer. 4. User asks TX to send message (tx.send message). 5. Tx uses xmit services (xmit.send).

Now TX is supposed to get the ack message. 1. Receive class receives a message and sets message ready to be read. 2. Rx reads the message (Receive.read message). 3. Checks it is ack message. (receive.ackmessage). 4. RX posts the ack message to buffer. (rx.postackmessage tobuffer). 5. Tx is informed ack message is received.

We can now understand why this diagram is called sequence diagram. It depicts the sequence of operations (methods) invoked by the different objects in a coordinated way to execute one workflow initiated by the actor.

5.6 Other Diagrams

There are more behavioral diagrams that we are not going to be included here for the sake of space and also they are not too frequently used. They are introduced very briefly.

- Analysis diagram is close to activity diagram but at higher level. A simplified activity diagram to capture high-level business processes.
- A communication diagram shows the interactions between elements at run-time in much the same manner as a sequence diagram. It visualizes inter-object relationships, while sequence diagrams are more effective at visualizing processing over time.
- Timing diagrams define the behavior of different objects within a time scale. It provides a visual representation of objects changing state and interacting over time.
- Requirements diagram is a customized diagram to describe system's requirements or features as a visual model

5.7 Summary—UML

Unified modeling language (UML) is very established standard to represent a system, its static and dynamic behavior. End to end development of systems, whether it is software system or any embedded system can be specified. The models can be easily exchanged across design teams. The UML models will be very useful for documentation, skeleton code generation, and verifying the behavior well before coding.

Though several books are available to learn UML, we have included this chapter just sufficient to start off the design with UML standard and to represent the extensive set of models we have studied in Chap. 3. The best way is to practice the system design using CASE tools. Enterprise Architecture is a one of popular CASE tool from Sparx systems for modeling tasks.

Embedded systems are becoming too complex with the total logic partitioned into hardware implementation (needing high-computational requirements), high-level business logic in object-oriented languages and persistent parts in databases. Out of all, we still miss one aspect, i.e., real-time computing where certain systems need deterministic upper time-bound for the execution of a job. Such systems are called real-time systems whose specification includes both logical and temporal correctness of requirements. Next chapter looks into design of such real-time systems.

5.8 Further Reading

To become proficient in this subject, one has to practice through real-world projects implemented using a CASE tool. For getting fundamental concepts, UML Distilled, by Martin Fowler (2003) can be referred. Enterprise Architect, by Sparx systems (2010) supports UML and other extensions like SysML/systemC and other models for developing embedded systems. The user guide and student version of this tool will be helpful for practice.

5.9 Exercises

1. A digital watch has four modes of display. (Mode 1) HH:MM:SS, (Mode 2) HH (only hours), (Mode 3) MM (only minutes), (Mode 4) SS (only seconds). One can go to any mode of display by keeping mode switch pressed for more than 2 s. After releasing the mode switch press, system changes the mode. The change is cyclic, i.e., $1 > 2 > 3 > 4 > 1$. When HH, MM, or SS is displayed, one can increment the displayed time by keeping the SET button pressed.
 - Analyze the problem using object-oriented approach.
 - Define the classes (methods and attributes).
 - Explain the interactions to set and display the time using interaction diagram.
2. A system has to be designed that calculates the centroid of a given shape. A shape is a closed polygon formed by connecting a sequence of vertices by straight lines. Ex: Triangle is formed by connecting three vertices. The system identifies the shape by manually capturing each corner of the shape (its X and Y coordinates) in sequence. User starts capturing by pressing a “start button” user presses “capture” button when he places the pointer on a corner. Once all vertices are captured, user presses a “Compute” button. The system computes the centroid and displays. The design should be generic such that:
 - Any shape with different vertex count can be handled.
 - The vertex capturing device can differ but has same functionality of capturing the X, Y coordinates.

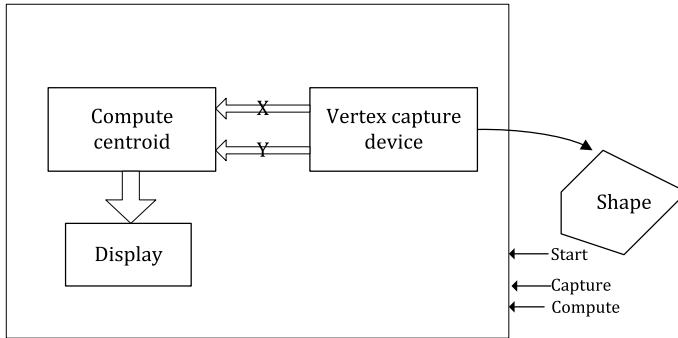
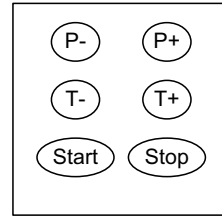


Fig. 5.25 Digitizer

A block diagram of the complete system is shown below (Fig. 5.25).

- Analyze the system using object-oriented approach. Identify the classes and their relations using structured diagram
 - Define the attributes and methods for each class.
 - Draw state machine diagram for the complete system behavior.
 - Represent the complete workflow as a sequence diagram.
3. A controller has to be designed for a microwave oven. The oven has primitive operations as described below.
- When power is on, the oven is ready to be started with default heating time of 10 s and default heating power of 50%.
 - User can change heating power (HP) from 50 to 100% in steps of 10 by using P- and P+.
 - User can change heating time (HT) from 10 to 90 s in steps of 10 by using T- and T+.
 - Oven heats when the start button is pressed and stops automatically when heating time is over.
 - Oven can be stopped while heating by pressing “stop” button.
 - When “stop” button is pressed, HP and HT settings come to default values.
 - When door is opened during heating, the oven behaves as if “stop” button is pressed.
 - When door is opened when heating is off, the HP and HT settings remain intact.
 - Heating can start only when door is closed.
 - User interface is shown below. The system has no display.(for simplicity) (Fig. 5.26).

Questions

Fig. 5.26 Oven interface

- Identify the actors, stakeholders, and top-level use cases. Expand one of the use cases with detailed success and failure scenarios. Represent each use case as a structured template.
 - Represent the system behavior as a FSM. Draw state machine diagram.
 - Make an object model of the control unit of the oven using class diagrams. Define the attributes and methods for each class.
 - Represent the behavior of the control unit for any two commands given by the operator as a sequence diagram.
4. A coffee vending machine (system) has to be designed.
- a. The machine has three tubs for milk, water, and beans.
 - b. System keeps the milk and water at set temperature through ON/OFF control. Temperature sensor and heater are used for this purpose.
 - c. The machine can prepare four variations of coffee V1 to V4. User can select one of them and presses “make” using a 5 button panel.
 - d. When “make” is pressed, beans are released through valve V2, ground for set time and released through valve V4.
 - e. Milk and water are released through valves V1 and V3 after v4 is released.
 - f. Coffee is released by opening V5 after mixing.
 - g. V5 is released only when a cup is placed which is sensed by cup sensor.
 - h. System logs the time at which a cup is filled. Machine supervisor uses this information to get usage profile of the machine.
 - i. Different variations of coffee (V1-V4) are done by preset times of the valve, mixer, grinder operations for each version. Machine operator can set these values in “configuration” mode (Fig. 5.27).
- Draw use cases from above description of the problem at top level. Who are the primary actors?
 - Define a structural model to represent the entities as objects and their association. Define the attributes, methods, and events of the identified classes (class diagram).
 - Explain the behavior of the machine as a state model (FSM diagram).
 - Explain one main operation using interaction or sequence diagram.
5. Remotely Controlled Trolley (RCT) moves on a straight path. When it is ON, it moves from stop 1 to stop 4 in forward direction and from stop 4 to stop 1 in reverse direction. It stops for 2 min at each stop and resumes motion. When it reaches stop 4, it reverses its motion automatically. The RCT senses the arrival

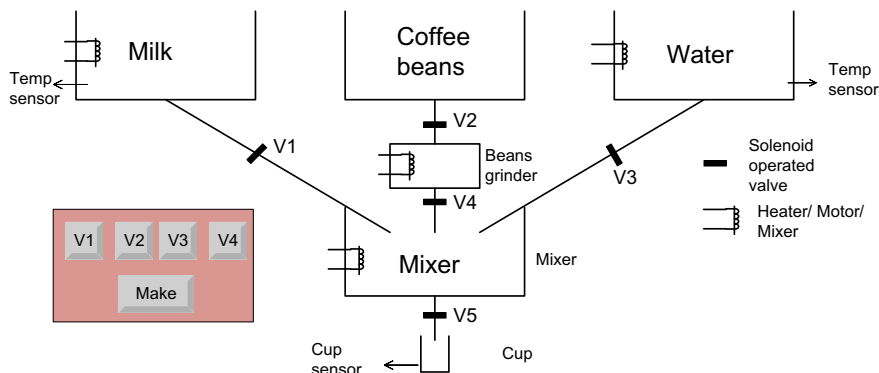


Fig. 5.27 Coffee vending machine

of a stop from stop sensors and parking position from Park sensor. RCT receives two commands from remote user PARK and START as two messages. When a PARK message is received, the RCT moves to park position and switches off its Motor. When a START message is received, RCT puts on the motor and resumes its motion. The RCT has a controller unit (CU), which has the functionality: (a) receive the two messages (details of media and communication can be ignored), (b) senses the park and stops, and (c) take a decision on motor movement, by turning the motor ON/OFF and setting the direction forward/reverse (Fig. 5.28).

Questions

- Identify the actors, stakeholders, and top-level use cases. Expand one of the use cases with detailed success and failure scenarios. Represent each use case as a structured template.
- Represent the RCT behavior as a FSM. Draw state machine diagram.

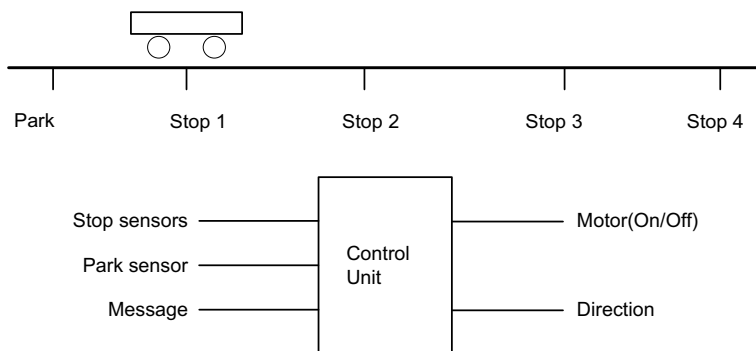


Fig. 5.28 Remotely controlled trolley (RCT)

- Represent the behavior as a sequence diagram for different operations, the system is going to function.
6. An electronic door access system has to be developed. A reader is attached on each door, which reads the thumb impression, validates, and sends the information to the server for registration. Server accepts messages, registers access, and acknowledges to the reader. Around 100 such readers are served by the server. The details of the reader and server functionality are as below.

Sequence of operations:

- a. A green lamp indicates that the reader is ready to accept.
- b. When the thumb is placed on the reader for at least 5 s, reader generates a 16 bit signature and sends it to server as a message with the signature.
- c. Server verifies from internal list of signatures and acknowledges whether it is a valid signature.
- d. Server registers the ID of the person belonging to the signature, the time and access type (IN/OUT) if it is valid
- e. Reader operates door relay for 10 s and sets back to READY state.

A transaction gets aborted

- f. When thumb is placed less than 5 s
- g. No acknowledgment message is received to the reader within 20 s.
- h. Or any abnormal event occurs.

Represent the total behavior as an interaction diagram

7. A system has to be designed to monitor the temperatures of an industrial process. Detailed specifications are as below:
- a. The temperature sensors (TS) are intelligent devices installed close to the physical location of each process.
 - b. Each TS reads the temperature at set sampling rate (Each TS has its own rate) and sends data to processing station (PS) serially. There is no acknowledgment from PS for each data sent.
 - c. PS sends control messages serially to the TS whenever its operating parameters have to be set.
 - d. The only operating parameter of the TS to be set is the sampling rate.
 - e. PS reads the serial data from each TS and computes the average of last 100 samples for each TS.
 - f. PS monitors the upper limits for each TS and raises an alarm if high. The alarm has to be reset by the operator by pressing a push button switch.
 - g. Assume there is no other user interface except a push button switch to reset the alarm.
 - h. The number of TS can be taken any arbitrary value.
 - i. All the activities go concurrently in real time.

Questions

- Draw use cases from above description of the problem and represent diagrammatically.
 - Represent the complete system at architectural level. (As a diagram) and explain briefly the strategy.
 - Define a structural model to represent the entities as objects and their association. Define the attributes, methods, and events of the classes.
 - Draw a data flow model for the entire process (As a diagram) and explain briefly.
 - Explain the behavioral model using interaction or sequence diagram.
8. Observe the behavior of a lift available at your office or in your apartments. You have to take over the roles of customer, system analyzer, designer and developer of the complete system. Following are the tasks and deliverables. The design should include the user interface and system's behavior for each command sequence.

Part	Scope	Deliverables
1	Write use cases stating all scenarios	Use cases
2	Derive the requirements from use cases	Requirements
3	Draw the user interface	UI diagram
4	Design a detailed state machine (FSM) hierarchically satisfying the desired behavior	FSM diagrams
5	Represent the model using OOAD	Structural diagrams (UML)
6	Represent the system behavior as program state machine	Program state machine model
7	Represent important interactions using UML interaction diagram	Interaction diagram

References

- Chen R (2003) Embedded system design using UML and platforms. System specification and design languages. Kluwer Academic Publishers
- Enterprise Architect User Guide (2010) Sparx systems
- Fowler M (2003) UML distilled, 3rd edn. Addison Wesley
- Herrer F. Modeling hardware/software embedded systems with UML/MARTE: a single-source design approach. Handbook of hardware/software co-design
- Kaur A (2012) Application of UML in real-time embedded systems. Int J Softw Eng Appl (IJSEA) 3(2)
- Martin RC (1997) UML tutorial: collaboration diagrams, Engineering notebook column
- Martin RC. UML tutorial: part 1—class diagrams
- Thepade SD. Approaches of using UML for embedded system design
- UML use case diagrams. Engineering notebook. C++ report, Nov-Dec 1998
- UML for modelling and performance estimation of embedded systems. J Object Technol 8(2) (2009)

Chapter 6

Real-Time Systems



Abstract We come across several times the term “real time” tagged before any other noun or verb, like real-time data, real-time monitoring, real-time governance, and so on. Let us understand what real time signifies. After completing this chapter, one will be able to un-tag the term “real time” from many such usages. In this chapter, we will understand the characteristics of a system that qualifies it, to be called as real-time system. Then, we will classify the RT systems based on their traits. We will study the reference model by which we can analyze the system and focus on important aspects of them. We will study scheduling mechanisms through supporting algorithms to reach real-time constraints. Section 6.2 classifies real-time systems (RTS) to periodic, mostly periodic, aperiodic, and predictable and unpredictable (spurious) systems. Section 6.4 deals with models to execute such periodic tasks. Section 6.6 classifies scheduling algorithms. Section 6.7 deals with clock-driven scheduling. Section 6.8 deals with scheduling priority-driven periodic tasks. Section 6.9 deals with scheduling tasks with dynamic priority like Earliest Deadline First (EDF) and Least Slack Time First (LST). Section 6.10 deals with scheduling sporadic tasks. Section 6.11 deals with accessing resources by multiple tasks, handling the contention for resources and how to handle cases of priority inversion. To summarize, aperiodic jobs are soft and can be accommodated by stealing slack times and idle slots. Tasks can be prioritized based on their rate. RMA is a popular protocol. Priorities of jobs can be assigned using early deadlines and also the least slack time. EDF algorithms are most popular. Sporadic jobs are unpredictable with varied properties. Given a context, a sporadic job can be accepted if it is schedulable. Sporadic jobs have to be handled in a separate queue. The above algorithms assume no contention of resources. Resource contention modifies the execution times based on the availability of resources and the critical section of the resources in each job. The most serious problem is priority inversion, which has to be taken care with multiple algorithms like priority inheritance. This chapter becomes the input to the next chapter where we study the architecture of real-time executives, their standardization, and their features.

Keywords Real time systems (RTS) · Periodic · Aperiodic · Earliest deadline first (EDF) · Least slack time first (LST) · Deadline · Tardiness · Usefulness · Job ·

Task · Release time · Response time · Slack time · Scheduler · Laxity function ·
Rate monotonic algorithm (RMA)

6.1 Definition and Examples

A *real-time system (RTS)* is a system whose specification includes both logical and temporal correctness requirements. The majority of systems, we come across, have specifications that demand logically correct outputs. The system behavior can be verified for logical correctness by verifying the output value with respect to the input given.

An RT system has to produce the output at the correct time specified. The RT system is functional when the output is temporally correct. The challenge is to maintain temporal correctness and verification. This chapter is focused on these two aspects.

A simple function $y = f(x)$ will be logically correct if it produces correct value y for a given input x . y can be generated at any time by the system. An RT system specifies that the function $f(x)$ should produce valid data at $(t + 3)$ seconds from the time t the input x is given. The RT system is verified for temporal correctness by verifying the output data at time $(t + 3)$. This simple example illustrates the difference between an RT system and a non-RT system.

Some misconceptions still exist about RT systems. Like “Real-time computing is equivalent to fast computing” is not correct. Generating output quicker is not real time. As per definition, system should generate valid output at a specified time. When there is no such requirement, it is no more RT. “Real-time systems function in a static environment” is not correct. As they have temporal specifications, it is a dynamic system. ““Real time” is performance engineering...” not correct. Is a payroll processing system a real-time system? It has a time constraint: Print the pay checks every 2 weeks by a specified time. Perhaps, it is a real-time system in a definitional sense, but it doesn’t pay us to view it as such.

6.1.1 A Digital Controller

A digital control system is an RT system, see Fig. 6.1. The plant has to behave as per the reference input $r(t)$, which is function of time. The reference signal and the feedback signal are sampled periodically and the controller module computes the actuating signal and feeds it to the system.

Other examples are

- ABS control in a car—the time at which brake pressure is to be controlled for individual wheels is critical. This is based on the sensed wheel speeds, the surface conditions, etc.

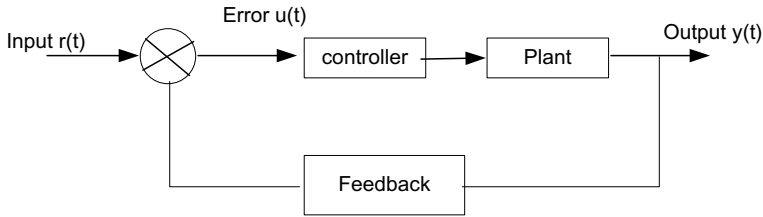


Fig. 6.1 A digital controller

- Autopilot system in a car—the time, at which the speed has to be reduced, wheels to be steered, breaks to applied are highly time bound. Any lapse in time is devastation, an excellent example needing real-time control.
- Missile controls—sensing the target and the time at which the missile has to be released are time bound. Any early release or late release of the missile will cause devastation.

6.2 Broad Classification of RTS

Basically, RT systems can be classified broadly as follows.

6.2.1 *Periodic*

The system is periodically controlled by a clock. When the clock cycle starts, the process takes place. All the actions have to be computed and the outputs to be generated before the clock cycle ends. The process repeats the next clock cycle. Most digital process control systems, health monitoring systems are examples. Multi-rate control systems are another example. At each clock, certain activities are done, and every Nth clock certain activities are done. Effectively, the total activities of the system are done at different rates.

6.2.2 *Mostly Periodic*

The system is driven exactly the same way as purely periodic. However, the system has to respond to certain events, which are totally asynchronous. They are not periodic. The system cannot afford to miss them. As an example, a temperature control system that is periodic gets some sensor failure alarm or fire alarm.

6.2.3 *Aperiodic but Predictable*

The events are not periodic. These are termed as *aperiodic*. The events are asynchronous. Duration between such events is normally predictable. Because of this predictability, the system is planned to complete its job before the next asynchronous event occurs.

6.2.4 *Asynchronous and Unpredictable*

The events are not periodic. They are asynchronous. Duration between such events is unpredictable. Such events are termed sporadic. The design of such systems without losing any such asynchronous events is challenging and highly compute intensive.

6.3 Terms in RT Systems

Before the classification of RT systems, we introduce certain terms related to real-time systems.

Deadline—A deadline is just like a real-life deadline. It is the time by which a system must produce an output. A deadline is hard if the failure to meet the timing constraint, has serious consequences, and can be catastrophic. Deadline is soft if occasional misses of deadline are not harmful and can be tolerated.

Tardiness—Tardiness is a measure of delay with respect to the planned time in executing any task.

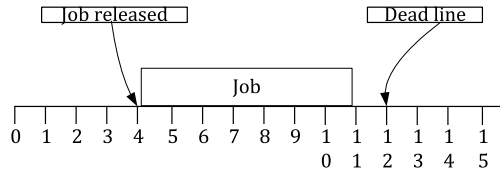
Usefulness—A measure of how the output of an RT system is useful. Usefulness of the output may go abruptly down if deadlines are not met.

Job—A quantum of work that is scheduled and executed by the system. Jobs require resources like processor time, memory, and other resources exclusively to get the job executed. Some examples of job are computation of a control-law, computation of an FFT on sensor data, transmission of a data packet, retrieval of a file, etc.

Task—A set of related jobs that jointly provide some major activities. The jobs have to be scheduled effectively to meet the real-time constraints. Ex: navigation control of a plane, alarm management system, missile guidance system, etc.

Processor—The jobs execute on a processor. The jobs are scheduled to be executed on processor based on several factors like priority, deadlines of each job, etc. This is done by the operating system. There may be multiple processors with different speeds of execution to whom the jobs are assigned.

Fig. 6.2 Example of release and deadlines



Resources—To get a job executed it needs processor time, memory, disk access, and network. The resources should be available exclusively for the job to get executed. A job gets exclusive access to a resource or waits for it. When a job gets access to a resource and completes the operation, the resource is released by the job. The resources are said to be *plentiful* if no job has to wait for a resource.

Release time—When a job is ready to get executed and ready to get scheduled. After the job is released, it waits in the scheduled job list for execution.

Relative deadline: The time at which job must be completed from the time it is released, i.e., $\text{deadline} - \text{release time}$.

Response time—The time at which the job is completed with respect to its release. $\text{Response time} = \text{Completion time} - \text{release time}$.

Slack time—The difference between the relative deadlines to the response time is the slack time. Most of the jobs do not have deterministic execution times because of varied inputs and varied conditions. So some slack time is maintained to manage the variations in response times within the deadline.

In the example at Fig. 6.2

Release time = 4

Absolute deadline = 12

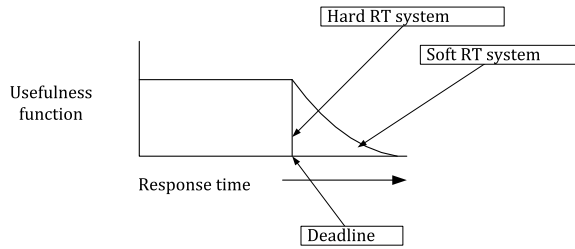
Relative deadline = 8

Response time = 7.

6.3.1 Hard RT Systems

Hard RT systems have hard deadlines. In a hard real-time system, the usefulness of the output degrades abruptly as tardiness increases. Usefulness may abruptly become zero when a deadline is not met or even with little tardiness. The deadline must be met. Else the system is nonfunctional. Before releasing a hard RT system, it must be validated for all possible scenarios, the system meets deadline requirement. In designing the RT system, the relation between the usefulness and tardiness has to be drawn. The design will be based on this aspect. The timing constraint of meeting deadline is not probabilistic. It is deterministic. It will not change. When you validate the system output (temporal validity), the system must always meet. Else it will not

Fig. 6.3 Usefulness function of hard and soft RT systems



qualify to be hard RT system. Another property is that, completing the job early (before deadline) has no additional usefulness.

Example Hard RT systems: Nuclear power plants, ABS in cars, safety bags in cars, railway signaling, fire control systems, missile guidance systems ...

6.3.2 Soft RT Systems

Soft RT systems have soft deadlines. In a soft real-time system, the usefulness degrades gradually as tardiness increases. It is not catastrophic if the system misses the deadline sometimes. Whether it is hard or soft RT system, the quality of the RT system depends upon how you estimate the usefulness function. The timing constraint of meeting deadline is probabilistic. The quality of the system depends on the probability value. The validation of output is statistical in nature and must satisfy the statistical constraint. One can also define a soft RT system's failure to meet deadlines in terms of the utility function. If the response time exceeds deadline, the usefulness function gradually falls in soft RT systems. Soft RT systems are focused on more throughput and can tolerate missing deadlines. One good example is video data transmission. We can afford to miss some frames without much loss of quality. Figure 6.3 illustrates the usefulness function of hard and soft RT systems.

Example soft RT systems: process controllers, multimedia communication systems, surveillance systems, telephone switching ...

6.3.3 Scheduler

A scheduler releases the jobs so that the task is completed without missing any deadlines. In soft RT systems, the scheduler aims to schedule the jobs to get the best usefulness. Scheduler executes appropriate algorithms to achieve the goal.

A scheduler does not schedule a job before it is released. It schedules the jobs such that the job is completed before the deadline (in hard RT systems) or miss the deadline with least probability. The scheduling algorithm must be validated well before it is implemented in field so that all timing constraints are met in all possible

scenarios. Validation always assumes that the required resources are available. The scheduler must consider all these parameters while scheduling.

6.3.4 Preemptivity

Preemption is defined literally as “to seize upon to the exclusion of others: take for oneself.” This is an action temporarily stopping current activity and taking over its place. Jobs have a certain priority. Some jobs may have equal priority. Priority comes into picture when two jobs contend for a resource, like processor, memory or any static resource. The contention is resolved based on the priority of contending jobs. When a higher priority job is released, and a lower priority job is being executed by the processor, the higher priority job preempts the current one. A job is preemptable if its execution can be interrupted in this manner. A job is non-preemptable if it must run to completion once started. Many preemptable jobs have periods during which they cannot be preempted for example when accessing certain resources. The ability to preempt a job or not impacts the scheduling algorithm.

When a job is preempted, the processor has to windup current job and save its status. It then has to load the preempting job to start execution. The time to switch the jobs is called *context switching*. This overhead must be considered while scheduling jobs. Response times will get extended due to the context switching. This may even cause missing deadlines. When such preemptions take place frequently, a job may intelligently omit a certain portion of its functionality (which are not serious) and maintain response times. For soft RT systems, the usefulness may reduce to some extent.

6.3.5 Criticality

In most of the systems, the priority is set as static. In certain occasions, there may be heavy contention across high-priority jobs (some jobs may have the same priority). Due to the heavy overload, scheduling is critical so that all jobs meet deadline. In such occasions, scheduler has to take critical decisions by looking into relative priorities of the jobs and weighted average. These will be discussed in detail to some extent in the scheduling algorithms.

6.3.6 Laxity Function

Laxity literally means some sort of lenience and *slackness*. Slackness is maximum time a task can be delayed on its activation to complete within deadline (absolute deadline-release time-job execution time) and indicates its timing constraints are soft

or hard. Usefulness function gives the value of a job with respect to its tardiness. Certain jobs can never be executed and must be aborted when late (better never than late). The slackness and utility functions make a decision factor.

6.4 Periodic Schedule

Below is an example where a supervisory monitoring system controls the temperature of each room in a building, see Fig. 6.4. The system has a job to read the temperature in the room from multiple locations, decides the temperature to be set, and the duct will be opened as per the read temperature. This is the activity to be done on the job. It takes about 3 units of time. It may vary by 1 unit due to the variations in execution times. Each room has to be attended each 5 units of time.

So the relative deadline is 5 units. Hence, the system is designed with a response time of 3 units and keeps a slack time of 2 units. In a periodic system, the relative deadline is the clock period.

6.4.1 Modeling Periodic Tasks

A task executes certain jobs periodically. The task repeats at that period and executes all the jobs. The total time of the execution of jobs in that task is the execution time of the task. Different tasks will have a different number of jobs. The tasks can have different period of execution. Let us work out a model to execute the tasks. Let us formulate the problem as below:

$T_{i\{i=1\dots m\}}$ = task i out of m tasks

$J_{i,j}$ = Job $\{i, j\}$ is Job j of task i

Phase of task T_i = release time of first job $J_{i,1}$

P_i = period of task T_i

E_i = execution time of T_i is the maximum execution time of all jobs in the periodic task

As an example, let us have two tasks T_1 and T_2 with periods 3 and 5, respectively. Each has one job. Task T_1 has one job with execution time of 1 unit. T_2 has one

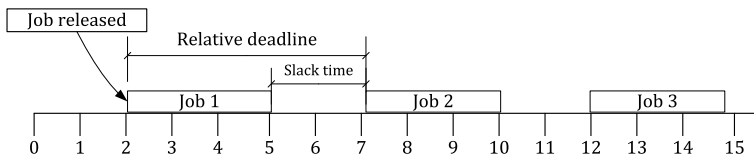


Fig. 6.4 A periodic schedule example

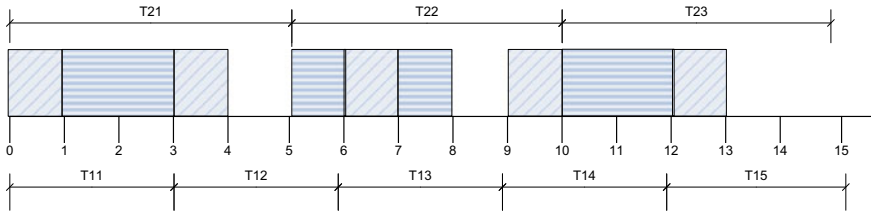


Fig. 6.5 Hyper period of the two tasks with periods 3 and 5

job with execution of 2 units. We have to schedule the jobs of the two tasks so that the jobs are executed once in each period without missing deadlines. In this case, deadline is the period of the task itself, see Fig. 6.5.

The solution is to find the LCM of the two tasks, i.e., $\text{LCM}(3, 5) = 15$. This becomes *hyper-period*. In one hyper period 5 T_1 s get executed and 3 T_2 s get executed. Now the whole system is periodic at hyperperiod (i.e.) 15.

We can schedule the job any time during the task. Even we can split the job to fit into the task period. We will study an algorithm to allot the jobs in the task slot subsequently. Now let us see how we manually allot the jobs. A simple algorithm will be to look into each period of task 1, allot the job in the first slot. Next look into task 2 period and allot its 2 unit jobs where ever space is available. This may become complex with multiple tasks and multiple jobs in each task. During $T_1 J_{11}$ is allotted in first slot and J_{21} in the next two slots. During T_{12} , $J_{1,2}$ is allotted. $J_{2,2}$ cannot be allotted in 4–5 within the same cycle as J_{22} is not released yet. So it is allotted after 5th unit. Continue this allotment with the constraint that one job should only be allotted in each task cycle. The hyper period will contain 5 T_1 jobs and 3 T_2 jobs.

6.4.2 Task Utilization

A task utilizes a portion of the processor time based on the execution time of the job. The utilization factor is defined as

$$U_i = e_i / p_i \text{ for task } t_i.$$

$$\text{The total utilization is } U = \sum U_i = \sum_{i=1}^m \frac{e_i}{p_i}.$$

In this example, $1/3 + 2/5 = 11/15$, meaning that 11 slots are utilized out of 15 slots in the hyper period.

6.4.3 Response to External Events

Many real-time systems are required to respond to external events. The jobs resulting from such events are *sporadic* or *aperiodic jobs*. A sporadic job has hard deadlines.

An aperiodic job has either a soft deadline or no deadline. The release time for sporadic or aperiodic jobs can be modeled as a random variable with some probability distribution, $A(x)$ where $A(x)$ gives the probability that the release time of the job is not later than x . Alternatively, if discussing a stream of similar sporadic/aperiodic jobs, $A(x)$ can be viewed as the probability distribution of their inter-release times. The execution times are also aperiodic.

6.5 Precedence Constraints and Dependencies

The jobs in a task, whether periodic, aperiodic or sporadic, may be constrained to execute in a particular order. This is known as a precedence constraint. The precedence rules can be listed as follows.

- A job J_i is a predecessor of another job J_k , if J_k cannot begin execution until the execution of J_i completes. This is represented by $J_i < J_k$.
- J_i is an immediate predecessor of J_k if $J_i < J_k$ and there is no other job J_j such that $J_i < J_j < J_k$.
- A job with a precedence constraint becomes ready for execution when its release time has passed and when all predecessors have completed execution.

6.5.1 Precedence Graph

Precedence graph is a directed acyclic graph used to show the precedence of jobs, see Fig. 6.6. It consists of nodes and edges. Nodes represent the jobs and the edges represent the flow of execution. A directed edge from node A to node B shows that statement A executes first and then Statement B executes. Let us see the precedence of the jobs below. J_1 and J_2 can be executed concurrently. J_1 and J_2 precede J_3 . J_3 precedes J_4 . It can be shown by precedence diagram.

Fig. 6.6 Precedence graph

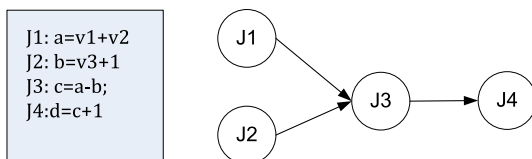
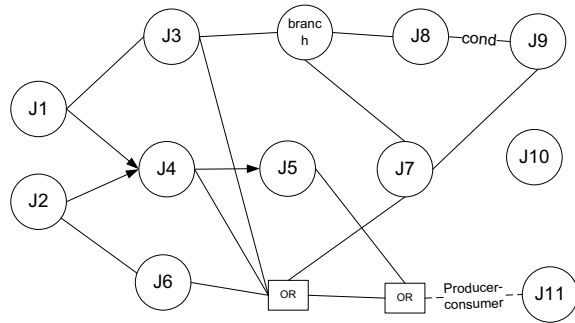


Fig. 6.7 Task graph

6.5.2 Task Graph

Task is an extended precedence graph to describe the system. It can capture many types of interactions and communications not possible by precedence graph. Precedence graph has jobs and precedence only.

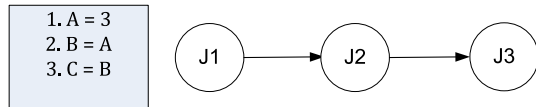
The task graph (see Fig. 6.7) can represent the precedence constraints among jobs in a set $\{J\}$ by using a directed graph $G = (J, <)$. Each node represents a job represented; a directed edge goes from J_i to J_k if J_i is an immediate predecessor of J_k .

Normally, a job must wait for the completion of all immediate predecessors. The jobs not connected by an edge with other jobs can execute after their release. It is implicitly an AND constraint. You can have an OR constraint which indicates that the successor job can start after its release when any one or more of its immediate jobs is completed. A dotted line represents that the two connected jobs behave as producer and consumer. Hence, task graph is very useful to represent the structure of a real-time system.

6.5.3 Task Dependencies

Precedence relations exist when two tasks are dependent. The dependencies occur due to several reasons. They can be classified as

- Data dependency
- Temporal dependency
- AND/OR precedencies
- Conditional branches
- Pipeline relationship.

Fig. 6.8 Flow dependency

6.5.3.1 Data Dependency

Let us take the three instructions in Fig. 6.8 below. The output of next instruction will be valid only when the previous instruction is executed. This is flow dependency. The precedence relation is shown in Fig. 6.8b. Each instruction is mapped as one job.

In Fig. 6.9, J2 must read B, before J3 rewrites B. You have *write after read* dependency. This is avoided by re-naming B and thus A and B can be read in parallel. There are several such dependence cases, which we will not cover here. By proper redesign, data dependencies can be minimized and activities can be parallelized.

6.5.3.2 Temporal Dependency

Jobs can be constrained to complete within a time relative to one another. (Temporal distance) Represented with each node with the time taken to complete the task, the depth of computation is the time taken to reach the final task through any path. The highlighted tasks in Fig. 6.9b show the critical path, which is the longest path from source to the destination. The source node is the one with no incoming edges and the target node is the one with no outgoing edges. The critical path time (in this case 21) is the worst-case time in execution considering all possible conditions.

6.5.3.3 AND/OR Precedencies

All jobs need their precedence to be completed normally. If a task needs all its precedence to be completed, it is AND-only precedence task. In certain occasions, completion of one or more jobs is sufficient for the precedence to be satisfied. They are called OR precedence tasks. As an example, we are interested in designing a voting system for fault tolerance, see Fig. 6.9c. The three tasks v1 to v3 execute with independent logic and output the decision. When two of them complete the job and the result is positive, voting logic (task F) need not wait for the third one to complete. A similar application is when we want to sense whether a switch is closed in a signaling system. If two or more systems complete the job and confirm the switch is closed, the next job can be executed. In these two cases, one out of the three tasks can be skipped. This is called AND/OR skipped.

But in certain cases, a task can proceed when certain precedence constraint is satisfied but it needs the other uncompleted one to complete. This is called AND/OR unskipped. In this example when two precedence tasks (say v1 and v2) complete, F

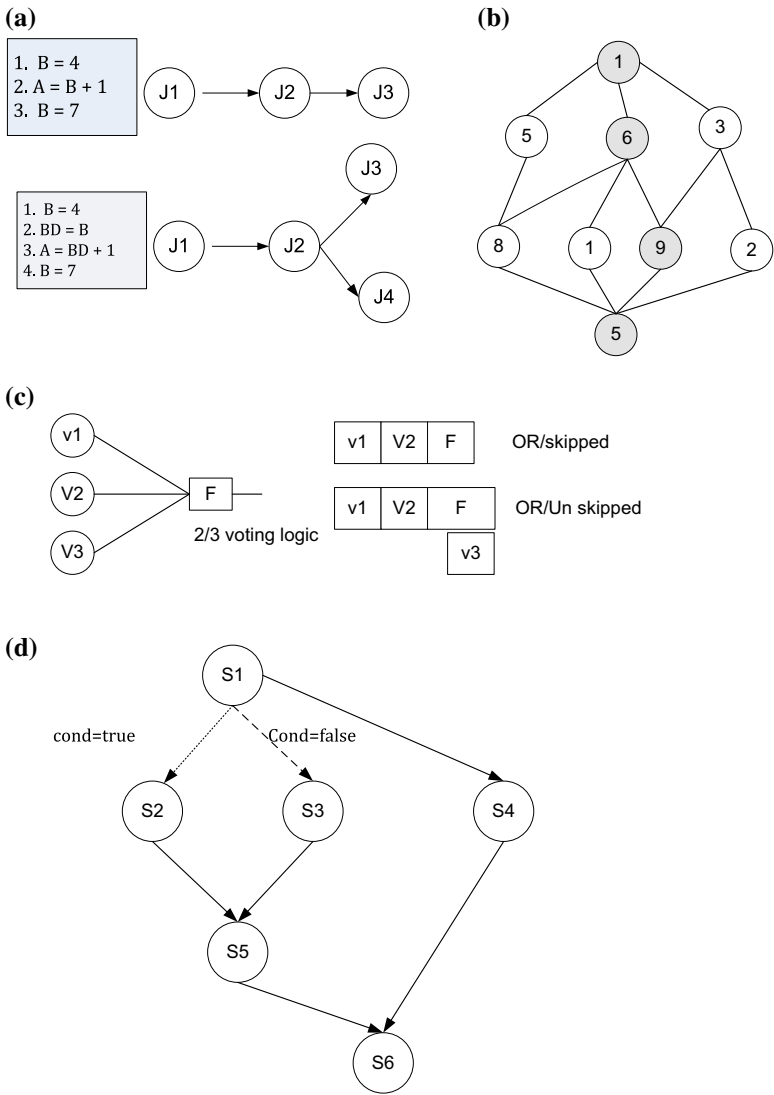


Fig. 6.9 **a**Anti-dependency. **b** Temporal dependency. **c** OR precedence. **d** Conditional branches

can start but before F completes v3 has to complete. This is very intuitive and very useful in real-time systems, to have better response times.

6.5.3.4 Conditional Branches

Only one of all the immediate successors of a job whose outgoing edges express OR constraints is to be executed. Such is a branch job. There is an associated join job for each branch job. Branch to join is called conditional block. Only one conditional branch is executed in each conditional block.

See the example in Fig. 6.9d. The two outgoing edges to successors of S1 are conditional. One of them only can be true. As shown in fig, $S1 \rightarrow S2$ is true. S2 executes. S2, S3 join at S5.

6.5.3.5 Pipeline Relationship

Represented by a dotted edge from producer to consumer. Preceding node J_i produces the data to be processed by succeeding job J_k . The dependency occurs only when J_i has data to be piped to J_k . The connectivity remains till all data from J_i are exhausted, see Fig. 6.7.

6.5.4 Resource Graph

The status of availability of free resources, how many are allocated, any request for the resource is pending by a process can be represented by a resource diagram, see Fig. 6.10. This is represented as a graph. It will be very useful in effective utilization of the resources and avoiding deadlock. However, the resources can be managed by a graph when they are in small numbers. When they are large, they have to be managed by a graph.

Resource graph has vertices and edges as any graph has. The vertices are of two types. Every process will be represented as a process vertex. Generally, the process will be represented with a circle. Every resource will be represented as a resource vertex. It is represented as a box. The box contains a single dot representing how many instances are present for each resource type. Multiple dots will be shown if it is multi-resource instance type resource.

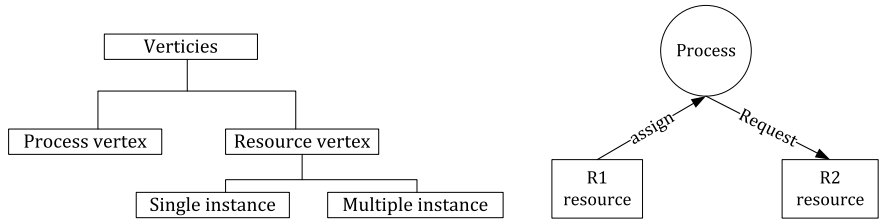
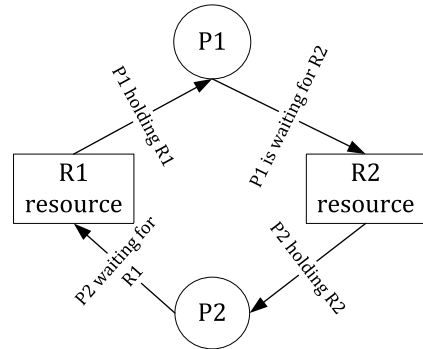


Fig. 6.10 Resource graph notation

Fig. 6.11 An example resource graph representing deadlock



If a process is using a resource, an arrow is drawn from the resource node to the process node. If a process is requesting a resource, an arrow is drawn from the process node to the resource node. Figure 6.11 represents two processors. P1 hold R1 and P2 holds r2. Bothe of them are waiting for the other resource and hence causing deadlock.

6.5.5 Scheduling Process

The scheduling process allocates resources including processor time to the released jobs. The goal is that all jobs meet their deadlines and maximize the job utility. The process is done by a scheduling algorithm suited for the type of jobs and their real-time constraints. The scheduler implements the algorithms and assigns jobs and resources as per the schedule. It implies that the scheduler assigns the jobs to available processors. This scheduler is nothing but part of operating system (RTOS) or the real-time executive.

6.5.6 Valid and Feasible Schedule

A schedule generated by the scheduler is *valid* if it complies with the following conditions.

- Every processor is assigned to one job only at a time.
- Every job is assigned to one processor only at any time.
- A job is scheduled for execution on a processor only after the job is in released state. As a corollary no job is scheduled on a processor before its release.
- The processor time allotted to a job is the maximum time allowed by the schedule so that other jobs get their time. A job may relinquish the processor before the maximum time allowed based on its state change.

- All the precedence relations as per resource graph and task graph are met.

A valid schedule generated by the scheduler is *feasible* if all the jobs complete their execution as per the deadline constraints.

- Given a set of jobs and their constraints, if the scheduler is able to produce a *feasible* schedule, then the jobs are *schedulable*. As a corollary certain set of jobs with hard constraints may not be theoretically schedulable.
- A valid schedule is also a feasible schedule if every job meets its timing constraints.
- A hard real-time scheduling algorithm is optimal if the algorithm always produces a feasible schedule, if the given set of jobs has theoretically feasible schedules. As a corollary, if an optimal algorithm can't find a feasible schedule for a set of jobs, then the jobs can't be feasibly scheduled by any algorithm.

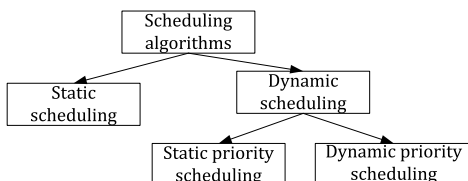
6.6 Scheduling Algorithms–Classification

The scheduling algorithms can be broadly classified with the characteristics explained briefly as below. We will take up few important algorithms in each class and study in detail, see Fig. 6.12.

6.6.1 Static Scheduling

There is a constant number of periodic tasks in the system. The constraints and parameters of the periodic tasks are known apriori. There are some aperiodic jobs whose release times are not known. There are no sporadic jobs. The algorithm constructs static schedule of the jobs off-line. Aperiodic jobs are placed in a queue and released whenever processor is idle.

Fig. 6.12 Classification of scheduling algorithms



6.6.2 *Dynamic Scheduling*

The schedule is not precomputed off-line and not static like in static scheduling. These are on-line schedulers. Scheduler assigns priority to the released jobs dynamically and places them in the ready job queue in priority order.

6.6.3 *Static Priority Scheduling*

Different jobs in a task have the same priority assigned.

6.6.4 *Dynamic Priority Scheduling*

Tasks have dynamic priority. The jobs in the tasks have static priority or dynamic. So dynamic priority systems can have task-level and job-level dynamic priorities or task-level dynamic and job-level static priorities.

6.7 *Clock-Driven Scheduling*

Clock-driven scheduling is static schedule. It is time-driven cyclic process. This algorithm can handle periodic tasks and aperiodic tasks. The assumptions:

- There are N number of periodic tasks.
- The parameters of the periodic task are, i.e., phase, period, and execution time of each job.
- Each job is ready for execution at its release time.
- There are aperiodic jobs that are rereleased arbitrarily.
- There are no sporadic jobs.

6.7.1 *Notation*

A task is represented denoting its parameters as

$T_i (\Phi_i, p_i, e_i, D_i)$, where

T_i = task number

Φ_i = phase at which the task begins

p_i = period

e_i = execution time per job

D_i = relative deadline.

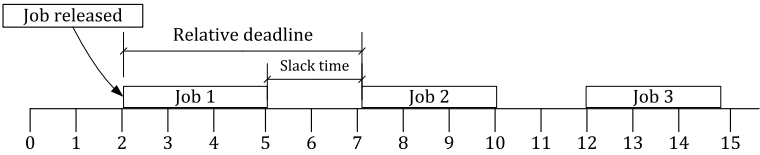


Fig. 6.13 Task notation

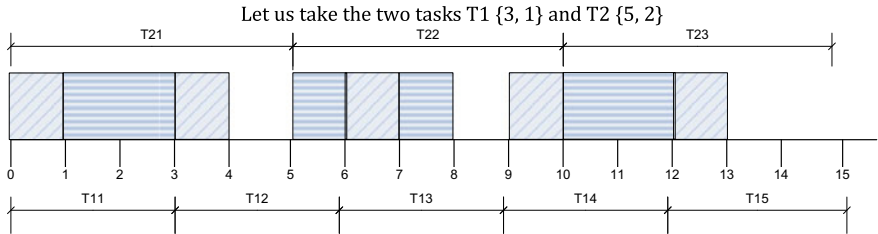


Fig. 6.14 Tasks to be clock-driven schedule

The task shown in Fig. 6.13 is represented as $T(2, 5, 3, \text{ and } 5)$.
If there is no phase, it is represented as $T(5, 3, \text{ and } 5)$.
If period is the relative deadline, the same is represented as $T(5, 3)$.
Let us take the two tasks $T1 \{3, 1\}$ and $T2 \{5, 2\}$ (Fig. 6.14).
The static table as shown in Table 6.1 is computed off-line and stored. The first column shows the job sequence, which is to be executed sequentially. The second column shows the time at which the job is going to be released. The third column shows the task number to which the job belongs. In this schedule, you will find some

Table 6.1 Static table of job scheduled

Job slot (k)	Schedule time (t_k)	Task at $t_k(T_{tk})$
0	0	T1
1	1	T2
2	3	T1
3	4	Idle
4	5	T2
5	6	T1
6	7	T2
7	8	Idle
8	9	T1
9	10	T2
10	12	T1
11	13	Idle
12	14	Idle

empty slots because of the hyper period of the two tasks, which we have discussed earlier. The scheduler utilizes these idle slots to execute the aperiodic jobs. The aperiodic jobs are placed in a queue by the scheduler and allot the processor to these jobs during idle slots.

6.7.2 Pseudo Algorithm

Store pre-computed schedule as a table.

Set the hardware timer to interrupt at the first decision time, $t_k = 0$.

On receipt of an interrupt at t_k .

Set the timer interrupt to expire at t_{k+1}

If previous job overrunning, handle failure

If $T(t_k) = \text{Idle}$ and aperiodic job waiting

Start aperiodic job

Otherwise

Start next job in task $T(t_k)$ executing

6.7.3 Slack Time

Slack time is a term frequently used in project management. It tells you how much time you have to start a particular task to keep the project deadline on time. Slack time = latest time to start an event – earliest time to start. When this concept is applied to real-time systems, the same is explained below.

In Fig. 6.15a, the task started early and completed before the deadline say 2 units before. So the slack is 2 units... At any time during the task execution the task has a slack of 2 units. This reduces linearly as it approaches deadline. In Fig. 6.15b, the task started late by 2 units. Hence, it has no slack time when it started.

Question is when the task T should start. A simple intuitive way is

- Case 1: if there are some aperiodic jobs waiting for any empty slots, then give them a way to execute during the slack period, and task T can start late as shown in Fig. 6.15b.

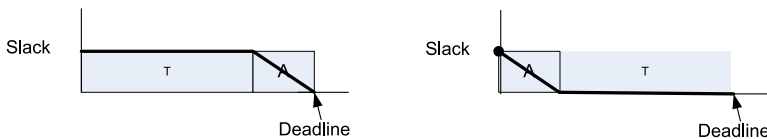


Fig. 6.15 a Slack time with early task start b with late start

- Case 2: When there are no pending aperiodic tasks, T can start early and keep the slack time as shown in Fig. 6.15a.

Fig. 6.15b allowed the aperiodic starts execute 2 units before! This concept is **slack stealing**. Figure 6.16 shows the effect of slack stealing and early execution of aperiodic jobs. It assumes the aperiodic jobs are released already.

Intuitively, it makes sense to give hard real-time jobs higher priority than aperiodic jobs. See Fig. 6.17a where the hard job is executed first and then the aperiodic job. Hard job has deadline of 5, effectively the slack time is 3 units. You observe that there is no advantage in completing the hard job completed first. It can start after aperiodic job and still complete within its deadline. This improves the response time of the aperiodic job.

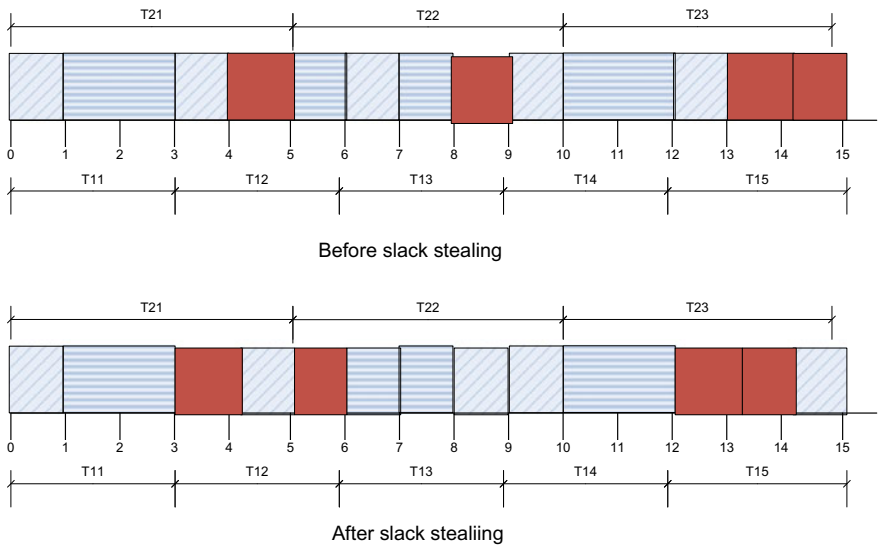


Fig. 6.16 Aperiodic jobs before and after cycle stealing

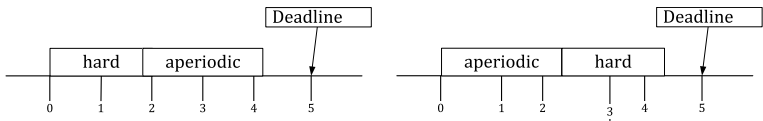


Fig. 6.17 Use slack stealing for better aperiodic response

6.7.4 *Handling Sporadic Jobs in Clock-Driven Scheduling*

Sporadic jobs have hard deadlines. A sporadic job is represented as

$S(d, e)$ where

d = deadline

e = execution time of the sporadic job.

The time when the job gets released and its execution time is not known apriori. The frequency at which the jobs get released may be high. Theoretically, it will be impossible to guarantee the successful execution of hard jobs without missing deadlines.

One can utmostly observe the properties of a hard job when it is released, then make a decision whether the job can be executed successfully in that context. The job is accepted if it can be successfully executed along with other jobs and complete within its deadline, else it is rejected. The strategy of handling sporadic jobs in clock-driven scheduling is as below.

- The scheduler already has a queue of aperiodic jobs. They are accommodated in the idle slots in the static table generated. These are served on first come first serve basis from the FIFO queue.
- Accepted sporadic jobs are placed in a separate queue.
- The order of execution is not FIFO but the earliest Deadline First. (EDF). This is something like a passenger at the security check bypasses the normal queue when his flight is about to leave!! The earliest deadline first.
- The way sporadic jobs are processed is same as aperiodic jobs.
- Aperiodic jobs are executed only when sporadic job queue is empty.

The question of accepting a sporadic job is done when it is released. This is little overhead on the scheduler as it has to switch to acceptance test and back. If there is burst of sporadic jobs and hence burst of acceptance tests, there is risk of regular periodic jobs missing the deadlines due to starving of processor time.

6.7.5 *Merits and Demerits*

- Clock-driven scheduling is conceptually simple.
- Schedule can be represented as tables that are used by the scheduler at run time.
- Relatively easy to validate because when jobs execute as deterministic, the system will not exhibit anomalies.
- As the schedule is a static table, changes to the tasks and jobs will be tough.
- Task parameters must be known ahead.
- Mostly useful for periodic tasks with very less number of aperiodic and sporadic jobs.

6.8 Priority-Driven Periodic Tasks

In the last section, we have scheduled multiple tasks, each having certain jobs that are periodically executing. We assumed all the tasks are of the same priority. In turn, all jobs in the task are of same priority. The way to schedule them was to simply make a static schedule of all the jobs belonging to the tasks in the hyper period of the tasks.

Let us extend the problem where the tasks have different priorities. Most real-world periodic tasks fall in this range, as certain tasks need more processing time and they have to be prioritized with respect to other tasks. Before we describe a scheduling algorithm, we assume

- The tasks have no dependency on other tasks. Hence, their priorities are independent.
- There are no aperiodic and sporadic tasks.
- Scheduling decisions are made immediately after release.
- A job can be preempted by other priority jobs at any time.
- Jobs do not suspend themselves.

In priority-driven scheduling, the task schedule is not statically computed. The priority of a job is assigned after the job is released. The released jobs are placed in a queue in priority order. If the priority of current running job is less than the one at the top of ready job queue, the current job is preempted. This is done at each decision scheduling time of the scheduler. At each time, the decision is made, the queue of the ready job is updated.

Most real-time scheduling algorithms of practical interest assign fixed priorities to individual jobs. The priority of them is assigned on its release and placed in the queue. Its priority will not change any more. The priority at the task level can change. These are task-level dynamic and job-level fixed priority models. We can have algorithms with both task-level and job-level dynamic priorities and only job-level dynamic. We will deal only with task-level dynamic and job-level fixed priority cases.

6.8.1 Rate Monotonic Algorithm (RMA)

Let us take a real-world example of patient health monitoring system. Certain critical parameters have to be sensed at faster rate and non-critical ones at lower rate. The task of critical sensing has jobs with small periods. It is intuitively obvious that a job that is sensing critical parameter (at fast rates) must have higher priority and can preempt other low priority jobs. In such systems, the tasks with higher rates (low periods) are assigned high priority. This is the crux of *Rate Monotonic Algorithm*.

It is task dynamic and fixed-job priority algorithm, each job in a task has same priority. Tasks with lower periods have higher priority. The tasks are indexed with smaller value as high priority, i.e., priority of $T_i > T_k$ where $i < k$.

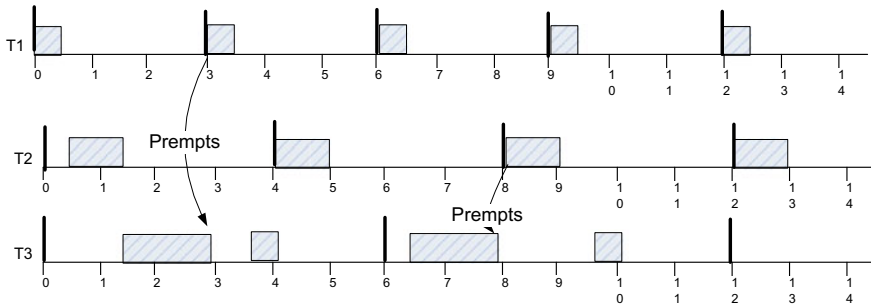


Fig. 6.18 RMA examples

6.8.1.1 RMA Example

See Fig. 6.18, let there be three tasks $T1 = (3, 0.5)$, $T2 = (4, 1)$ and $T3 = (6, 2)$.

So priorities: $T1 > T2 > T3$.

Hyper period = 12.

Utility = $0.5/3 + 1/4 + 2/6 = 0.75$.

T1 has the highest priority. Let $J_{i,j}$ represents j th job of task i . Represent $J11, j21$, and $J31$ will be in queue in the same order. $J11$ released for execution. $J21$ and $J31$ are released in the order. While $J31$ is executing, $J12$ is ready with higher priority. It preempts $J3, 1$. After $j12$, $J3, 1$ is released for the remaining job execution.

Again $J13$ and $J3, 2$ contend with $J13$ getting the processor. After $j13$ executes, $J32$ gets processor and again preempted by $J23$ and completes after $J14$ and $J2, 3$. You can see now, the higher rate $T1$ gets four samples while low priority gets two samples processed.

6.8.2 Deadline-Monotonic (DM) Algorithm

It is the same as RMA but the task priorities are assigned based on the relative deadlines of the task. The task having small deadlines is relatively higher priority. In case the deadlines are same as period of the task, DM is same as RM.

6.9 Dynamic Priority Algorithms

We have studied task-level priority based on the rate of its periodic jobs (RMA) and relative deadlines of the task. The priority of the jobs in the task is fixed. Now we will study algorithms where the priority of a job is decided based on its absolute deadline. (EDF) and on available slack time (LST).

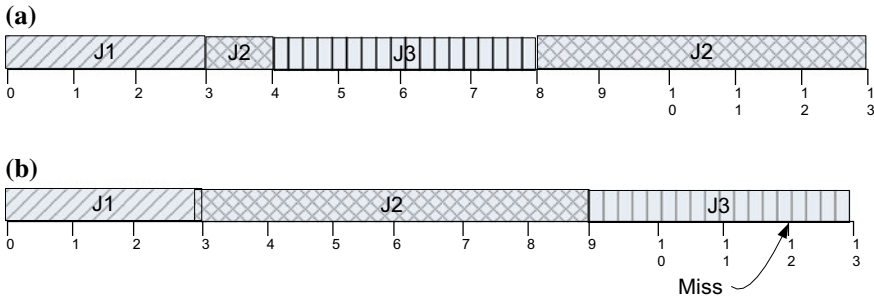


Fig. 6.19 EDF examples. **b** EDF not applied

6.9.1 Earliest Deadline First (EDF)

The priority of a job is dynamic and varies with absolute deadline. Let us say a job must complete at unit 20, available time to complete the job is absolute deadline – current time. The jobs that have least available time to complete its job are given the highest priority. As explained earlier, all passengers standing in a queue for security check in the airport are prioritized with respect to the departure time of their carriers!!

As an example consider three jobs, see Fig. 6.19.

$J1 = (0, 3, 10)$, $J2 = (2, 6, 14)$, and $J3 = (4, 4, 12)$.

Each job's parameters are release time, execution time, and absolute deadline of the job. In this example, J1 is released at time 0. No other job is in queue. J1 gets executed. Meanwhile j2 is released at $t = 2$. As J1 has higher priority ($10 < 14$), it continues and completes job at $t = 3$. J2 takes over. J3 is released at $t = 4$. Now there are two jobs where J3 has higher priority ($12 < 14$). Hence J3 preempts J2 at $t = 4$. J3 completes after its execution time of 4 units. At $t = 8$ J2 takes over and completes.

If there is no such preemption, J3 misses deadline as shown below. So an EDF algorithm provides a feasible solution when jobs are preemptable and there exists a feasible solution.

6.9.2 Least Slack Time First Algorithm (LST)

We have already defined slack time. Jobs execute competing with other priority jobs. They get preempted by higher priority jobs. Though a job starts with sufficient slack time (deadline–response time), they lose the slack time as they get suspended, see Fig. 6.20. Below job has execution of 7 and deadline of 11. Initially, it has slack time of 4. ($11 - 7$) as it got suspended at 3 for 1 unit, the slack has reduced to 3. Again it has reduced to 2 at $t = 9$. In this algorithm, the jobs having least slack time are given higher priority.

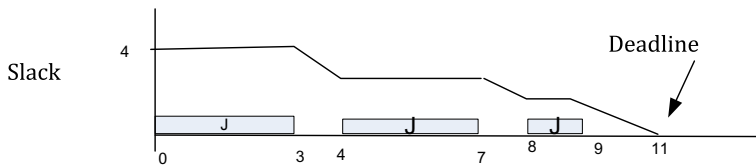


Fig. 6.20 Slack time reduction as jobs gets preempted

However, this algorithm is more complex, as the job's slack time must be known at each instance.

Dynamic priority algorithms take care of dynamic nature of jobs. They have high utilization factor than fixed priority algorithms. The algorithms provide better solutions but can miss the schedules due to unpredicted dynamism of certain jobs. As an example, if some jobs are late and their deadlines are approaching, some other jobs get preempted and they may miss deadline. As an example, a late running train that is about to miss its scheduled arrival (earliest deadline) is released first and another train that is running as per schedule is stopped, the correctly running train may also get delayed!!!!

6.9.3 Case Study 1

Which of the following systems of periodic tasks are schedulable by the rate-monotonic algorithm and/ or the earliest-deadline first algorithm? Explain your answer.

- a. $T = \{(8, 3), (9, 3), (15, 3)\}$
- b. $T = \{(8, 4), (10, 2), (12, 3)\}$

Solution

(a) $U_{RM}(3) \approx 0.780$

$$U = 3/8 + 3/9 + 3/15 = 0.908 > U_{RM}$$

schedulable utilization test is indeterminate. Using time-demand analysis

$$w_1(t) = 3, W_1 = 3 \leq 8, \therefore T_1 \text{ is schedulable}$$

$$w_2(t) = j11 + j21 = 6 \text{ (j21 completes by 6)}$$

$$W_2 = 6 \leq 9, \therefore T_2 \text{ is schedulable}$$

$$w_3(t) = j11 + j21 + 2 + j12 + j22 + 1 = 15 \text{ (j31 completes by 15)}$$

$$W_3 = 15 \leq 15, \therefore T_3 \text{ is schedulable.}$$

All tasks are schedulable under RM, therefore, the system is schedulable under RM.

$$U \leq 1, \text{ and } \therefore \text{the system is schedulable under EDF}$$

(b) $U = 4/8 + 2/10 + 3/12 = 0.95 > U_{RM}(3)$

Schedulable utilization test is indeterminate, use time-demand analysis,

$$w_1(t) = 4, W_1 = 4 \leq 8$$

$\therefore T_1$ is schedulable

$$w_2(t) = j_{11} + j_{21} = 6 \text{ (j}_{21} \text{ completes by 6)}$$

$$W_2 = 6 \leq 10$$

$\therefore T_2$ is schedulable

$$w_3(t) = j_{11} + j_{21} + 2 + j_{12} + j_{22} + 1 \text{ (j}_{31} \text{ completes by 15)}$$

$$W_3 = 15 > 12$$

$\therefore T_3$ misses its deadline

This system is not schedulable under RM.

$U \leq 1 \therefore$ this system is schedulable under EDF

6.9.4 Case Study-2

The two periodic tasks $T_1 = (3, 4, 2)$ and $T_2 = (6, 1)$ are scheduled as EDF with a slack stealer to serve aperiodic tasks. What is the response time of the three aperiodic jobs A1 and A2 released at (4,1) and (6,1) and (8,1) explain the process.

Solution

- 1 See Fig. 6.21. Initially, the slack stealer is suspended because the aperiodic job queue is empty. When A1 arrives at 4, the slack stealer resumes. A1 preempts J11 as J11 has slack of 2 units and can resume after A1.
- 2 A1 executes, aperiodic queue is empty, J11 resumes at 5 and completes at 6.
- 3 A2 gets into aperiodic queue at 6; J22 has slack of 5 units, so A2 gets executed at 6.
- 4 No jobs in aperiodic queue, J12 starts executing at 7. Has slack of 2 units.
- 5 A3 preempts J22 and gets out at 9.
- 6 J12 and J22 get executed as per priority before their deadlines.

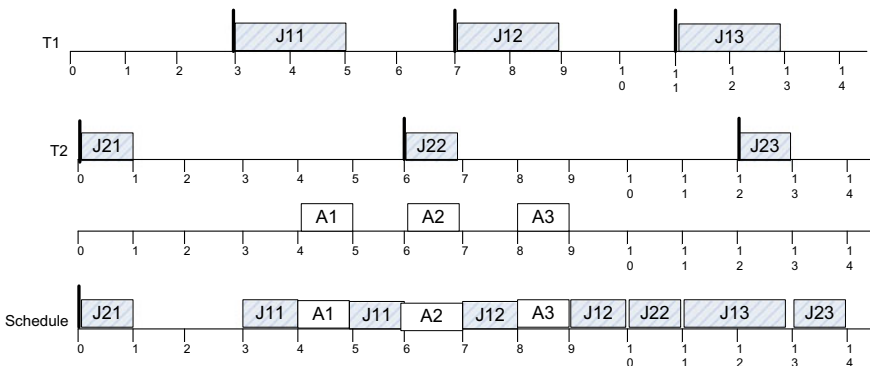


Fig. 6.21 Case study 2

6.9.5 Case Study 3

There are three tasks T1 (2, 0.5), T2 (5, 2), and T3 (6, 2). Schedule them with least sack time first.

Solution

- 1 See Fig. 6.22. Initial the first jobs of T1 to T3 (J11, J21 and J31) are released.
- 2 At $t = 0$; the slack times are $J11 = (2 - 0.5) = 1.5$; $J21 = (5 - 2) = 3$; $J31 = (6 - 2) = 4$ so $T1 > T2 > T3$
- 3 At $t = 0.5$, J21 executes till $t = 2$; completes 1.5 units
- 4 At $t = 2.0$, J12 is released. The slack times are $J12 = (2 - 0.5) = 1.5$, $J21 = (5 - 2 - 0.5) = 2.5$, $J31 = (6 - 2 - 2) = 2$
- 5 J12 executes the job at $t = 2.0$
- 6 At $t = 2.5$, J31 executes which has least slack time
- 7 At $t = 4.0$, J13 is released. Sack times are $J13 = (2 - 0.5) = 1.5$, $J21 = (5 - 4 - 0.5) = 0.5$; $J31 = (6 - 4 - 0.5) = 1.5$
- 8 J21 executes for 0.5 and the job completed.
- 9 At $t = 4.5$, slack times are $J13 = (2 - 0.5 - 0.5) = 1$ and $J31 = (6 - 4.5 - 0.5) = 1$
- 10 The sack times are equal, let J31 completes the job. Then J13.

6.10 Scheduling Sporadic Jobs

The characteristics of sporadic jobs are introduced at the beginning of the chapter as asynchronous and unpredictable jobs. We have studied the way to schedule periodic (clock driven) and aperiodic jobs with static and dynamic priorities. Every real-world application will have sporadic events along with the class of jobs described above. These have hard deadlines. Execution times and frequency of occurrence are not known. Examples are power fluctuations, alarm management under abnormal conditions, safety systems, etc.

The way to handle them is to have separate queues for three types of jobs, see Fig. 6.23. We assume the occurrence of these jobs is independent of each other. We also assume the periodic tasks in the absence of aperiodic and sporadic jobs are

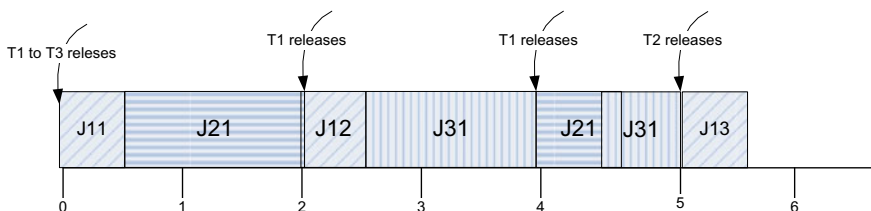


Fig. 6.22 Case study 3

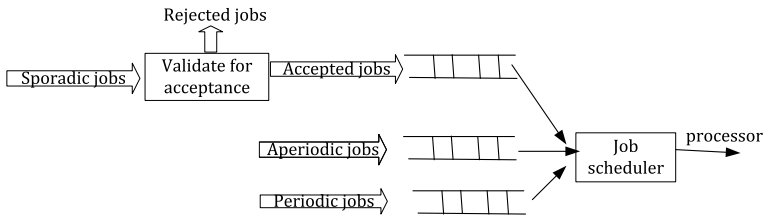


Fig. 6.23 Scheduling sporadic jobs

schedulable to meet the deadlines. The properties of a sporadic job will be known only when they are released.

The sporadic jobs will first go through an acceptance test. The test verifies that if this sporadic job is accepted, current periodic jobs and already accepted sporadic jobs will execute as planned and do not affect their schedule. It may not matter if running aperiodic tasks may get further delayed. If it satisfies, then the job is accepted and placed in the queue. Also, it verifies whether the sporadic job is schedulable meaning that all periodic and accepted sporadic jobs never miss their deadlines.

Schedule

- Scheduler performs an acceptance test on each sporadic job upon its arrival.
- Acceptance tests are performed on sporadic jobs in the EDF order.
- Sporadic jobs are ordered among themselves in the EDF order.
- In a deadline-driven system, they are scheduled with periodic jobs on the EDF basis.
- In any case, no new scheduling algorithm is needed.

6.11 Resource Access and Contention

We have discussed resources and resource graphs in the beginning of the chapter. In all the scheduling protocols, we discussed above, we have not considered the way an active job behaves when it needs a resource and how it affects its execution time. Moreover, we have not considered when multiple jobs need same resource and the protocol to manage the resource contention.

We focus on priority-driven systems. Clock-driven systems do not have these problems as we can avoid resource contention among jobs by scheduling them according to a cyclic schedule that keeps jobs' resource accesses serialized.

When a job needs a resource access, the resource is granted on a non-preemptive basis and used in a *mutually exclusive* manner. Mutually exclusiveness is implemented by *locking* the resource after grant and the job after using the resource *unlocks* the resource for use of others. The time period a job locks and unlocks a resource is *critical section* as shown in black hatched pattern in the below figure. When a lock request fails, the requesting job relinquishes the processor and waits

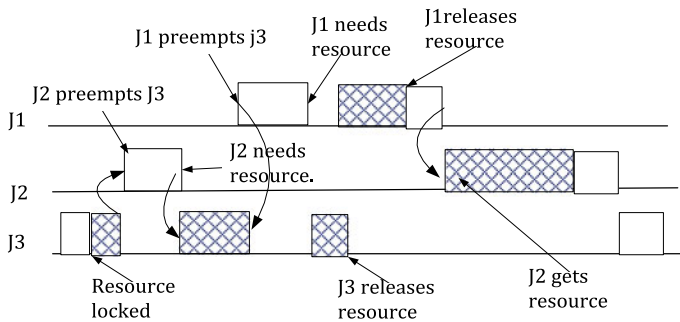


Fig. 6.24 Resource access

for the availability of the resource. The next waiting job in the queue takes over the processor.

The interaction of three jobs (J1...J3) for a resource R is shown in Fig. 6.24. This example shows how the priority jobs get delayed due to resource contention.

J3 executes initially and holds the resource when it needed. J3 gets preempted by J2 and starts executing. Resource is locked with J3. J2 also needs resource and gets blocked for want of resource. So, J3 gets the processor and utilizes the resource. Meanwhile, J3 gets preempted by J1. J1 needs resource after some time and gets blocked. J3 completes its job with resource and releases. J1 now gets the resource and releases once the job is done... Once J1 completes its job, J2 gets the processor and J2 completes its job. Finally, J3 gets processor and completes its job.

6.11.1 Priority Inversion

Resources are allocated to jobs on a non-preemptive basis; a higher priority job can be blocked by a lower priority job when the low-priority job holds the resources needed by high-priority job. See the sequence below: Fig. 6.25

1. J3 becomes ready and executes.
2. J3 requests for R and gets.
3. J1 takes over preempting J3. R is locked with j3,
4. J1 needs R; gets blocked as R is locked with j3. J3 = active with R
5. J2 is released. J2 preempts J3. R is locked with j3.
6. J2 relinquishes. J3 = active
7. J3 releases R. J1 gets resource. J3 is blocked.
8. J1 completes and relinquishes. J3 = active
9. J3 completes.

If you observe, J2 having higher priority over J3, preempted J3 which is holding the resource which is needed by J1. Because of the need for resource, job J1 is

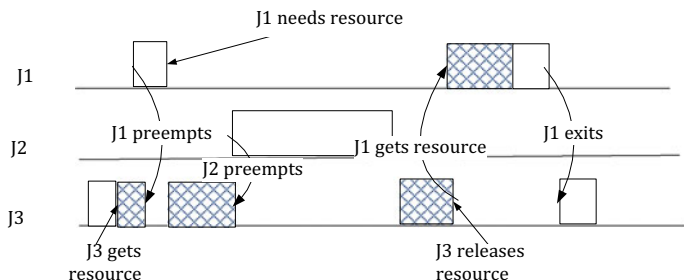


Fig. 6.25 Priority inversion

blocked. Hence J2 has done its job before J1. Effectively, the priority of J1 and J2 gets *inverted*. The real reason: *resources are allocated in a non-preemptive way*.

6.11.2 Priority Inheritance

A common method to avoid this inversion is, “*when a high priority job requests a resource which is locked with a low priority job, the low priority job inherits the high priority of the job requesting the resource.*”

In the example, we studied above, the following actions take place with priority inheritance.

- When J₁ requests resource R and becomes blocked at time 3, job J₃ inherits the priority of job J₁.
- When J₂ becomes ready at time 5, it cannot preempt J₃ because its priority is lower than the inherited priority of J₃.
- As a consequence, J₃ completes its critical section as soon as possible.

6.12 Summary

While this topic is one-semester course, we have browsed the most important concepts in real-time systems. The most important schedulers are static clock-based for periodic tasks.

Aperiodic jobs are soft and can be accommodated by stealing slack times and idle slots. Tasks can be prioritized based on their rate. RMA is a popular protocol. Priorities of jobs can be assigned using early deadlines and also least slack time. EDF algorithms are most popular. Sporadic jobs are unpredictable with varied properties. Given a context, a sporadic job can be accepted if it is schedulable. If not it is rejected. Sporadic jobs have to be handled in a separate queue.

Above algorithms assume no contention of resources. Resource contention modifies the execution times based on the availability of resources and the critical section of the resources in each job. Most serious problem is priority inversion, which has to be taken care with multiple algorithms like priority inheritance.

This chapter becomes the input to the next chapter where we study the architecture of real-time executives, their standardization, and their features. For those, to get in-depth knowledge on this subject, request to go through reference (Jane Liu 2003).

6.13 Further Reading

Several books Laplante (2005), Krishna and Shin (1997), Jane Liu (2003), Douglass (2004), Li and Yao (2003), Cheng (2003), Merz and Navet (2008), and Williams (2005) are authored on real-time systems by reputable authors. As an embedded system developer, one has to learn fundamental scheduling algorithms and how they are supported in commercial real-time operating systems. One has to correlate the theory from this chapter to the commercial RTOS user guides.

6.14 Exercises

1. A system with periodic tasks is scheduled and executed according to a cyclic schedule. Choose an appropriate frame size. The task details are: (8, 1), (15, 3), (20, 4), and (22, 6).
2. A system uses the cyclic EDF algorithm to schedule sporadic jobs. The cyclic schedule of periodic tasks in the system uses a frame size of 5, and a major cycle contains 6 frames. Suppose that the initial amounts of slack time in the frames are 1, 0.5, 0.5, 0.5, 1, and 1. Suppose that a sporadic job S (23, 1) arrives in frame 1, sporadic jobs S2 (16, 0.8) and S3 (20, 0.5) arrive in frame 2. In which frame are the accepted sporadic jobs scheduled? Consider both cases where a) slack stealing is used and b) not used.
3. Which of the following systems of periodic tasks are schedulable by the rate-monotonic algorithm and/or the earliest-deadline-first algorithm? Explain your answer.

$$T = \{(8, 3), (9, 3), (15, 3)\}$$

$$T = \{(8, 4), (10, 2), (12, 3)\}$$

4. The Periodic Tasks (3, 1), (4, 2), (6, 1) are scheduled according to the rate-monotonic algorithm.

Draw time-demand function of the tasks

Are the tasks schedulable? Give reasons.

5. The two periodic tasks $T1 = (2.0, 3.5, 1.5)$ and $T2 = (6.5, 0.5)$ are scheduled as EDF with a slack stealer to serve aperiodic tasks. What is the response time of the two aperiodic jobs released at $(2.8, 1.7)$ and $(5.5, 2.5)$ explain the process.
6. Use the time-demand analysis method to show that the rate-monotonic algorithm will produce a feasible schedule of the tasks $(6, 1)$, $(8, 2)$ and $(15, 6)$.
7. Consider the following tasks $(0, 10, \text{and } 3)$, $(2, 12, 6)$, $(4, 7, \text{and } 4)$. Show the schedules in a graphical way during 0 to 14 units of time for the following cases:
 - a. Non preemptive EDF
 - b. Preemptive EDF
 - c. Non preemptive and non-priority driven.
8. A system of three tasks $T1(3.5,1)$; $T2(4,1)$ and $T3(5,2,7)$ is to be scheduled with clock-driven cyclic executive algorithm.
 - (a) Is the task set schedulable? Justify your answer.
 - (b) What are the hyper period and possible frame size(s)?
 - (c) Choose the largest frame size and draw a Network Flow Graph.
 - (d) Draw a neat timing diagram of up to 20 frames.
9. A system contains three periodic tasks $T_i (P_i, e_i) = \{(7, 3), (12, 3), (20, 5)\}$. The tasks are scheduled by using Rate Monotonic Algorithm. Using iterative method, determine the schedulability of the tasks.
10. A system of three tasks $T1(3.5, 1)$, $T2(4, 1)$, and $T3(5, 2, 7)$ is scheduled with clock-driven cyclic executive algorithm and then with sporadic server with $T2$ as T_{ss} (with RMS algorithm). If a stream of sporadic tasks arrives as follows, can you schedule these tasks?

$S1 (2, 1, 10)$; $S2 (5, 2, 16)$; $S3 (5, 1.5, 13)$ Compare the results in both cases. Use acceptance test in both cases to accept/reject the tasks.
11. There are three $T1: (2, 0.75)$ $T2: (5, 1.5)$ $T3: (5.1, 1.5)$ Schedule them with least slack time first.
12. The two periodic tasks $T1 = (2.0, 3.5, 1.5)$ and $T2 = (6.5, 0.5)$ are scheduled as EDF with a slack stealer to serve aperiodic tasks. What is the response time of the two aperiodic jobs $A1$ and $A2$ released at $(2.8, 1.7)$ and $(5.5, 2.5)$ explain the process.

References

- Cheng AMK (2003) Real-time systems, scheduling, analysis, and verification. University of Houston, Wiley
- Douglass BP (2004) Real time UML: advances in the UML for real-time systems, 3rd edn. Addison Wesley

- JaneLiu WS (2003) Real-time systems. Pearson Education, India
- Krishna CM, Shin KG (1997) Real-time systems. McGraw-Hill
- Laplante PA (2005) Real-time system design and analysis, 3rd edn. PHI
- Li Q, Yao C (2003) Real-time concepts for embedded systems
- Merz S, Navet N (2008) Modeling and verification of real-time systems. Wiley
- Williams R (2005) Real-time systems development. Elsevier

Chapter 7

Real-Time Operating Systems (RTOS)



Abstract In the last chapter, we have studied and made a conceptual framework to handle multiple tasks in real time. When it comes to implementation, one can start from scratch writing in assembly language or in high-level languages, the models we have studied that include, tasks, jobs, and scheduling algorithms. When you follow this approach, the application becomes monolithic, with the application logic merged with commonly usable scheduler. This is the reason why real-time operating systems (RTOS) are commercially developed, which becomes the framework over which the tasks can be defined and executed. RTOS is functionally the same as generic operating system with functionality tailored for real-time embedded systems, viz, management of tasks, their states, memory, processor, etc. Basic concepts of RTOS, viz, tasks and its states, reentrancy, synchronization primitives are explained in Sects. 7.2 and 7.3. Kernel is the computer program that is always resident in memory and interfaces with the hardware resources (processor, memory, I/O, etc.) and with upper layers of applications. The system works either in *user mode* or in *kernel mode*. When a process makes requests of the kernel, it is called a *system call*. Multiple processes can be scheduled on a system. Similarly, a process can have multiple independent executables running concurrently. Each executable is a *thread*. Threads are lightweight and have low-context switching overheads. Standard OS call interface and behavior are standardized by IEEE as POSIX standard. POSIX specification, pThreads (IEEE 1003) is intended for all computing platforms. Sections 7.6 and 7.7 explain in detail Posix threads. The challenge now is how to orchestrate these threads to do a complex job. The design strategies are explained in Sect. 7.8.

Keywords Kernel · Thread · POSIX · pThreads · Re-entrancy · Semaphore · Mutex · Reader's/writers lock · Spin locks · Barrier

7.1 Introduction

In the last chapter, we have studied and made a conceptual framework to handle multiple tasks in real time. When it comes to implementation, one can start from scratch writing in assembly language or in high-level languages, the models we have studied that include tasks, jobs, scheduling algorithms. Any one acquainted

with microprocessors and their programming can do this. The timing aspects are managed through hardware interrupts. When you follow this approach, the application becomes monolithic, with the application logic merged with commonly usable scheduler. This is the reason why real-time operating systems (RTOS) are commercially developed, which becomes the framework over which the tasks can be defined and executed. The operating system is a complex software architecture, which handles multiple tasks, coordinates all the tasks, manages resource access, manages communication among them, and handles events through interrupts. The RTOS keeps the status of each task, their priorities and assigns them on to the processor. Those who have knowledge of operating systems, RTOS is functionally same as generic operating system with functionality tailored for real-time embedded systems, viz, management of tasks, their states, memory, processor, etc.

This chapter introduces conceptually the general structure of a real-time operating system. We use the term “*task*” as the unit of functionality to be executed. The terms job, process, and thread are interchangeable here. When we discuss a specific RTOS, we use the specific terms used by the standard or the commercial RTOS. We will study POSIX 4 basics, which is a portable operating system with real-time features. Then, we will study pthreads, the thread management in RTOS. The goal is to provide source-level portability as the standard defines syntactic and semantic standard.

Every popular operating system provides thread support in their processes.

7.2 RTOS Concepts

All of us have experience with general purpose computer and write applications. In this process, we get acquainted with the operating system and use some of its methods embedded in our application.

There are subtle differences between generic OS and RTOS. RTOS is an extension to the functionality provided by OS to handle real-time events. RTOS can be configured to be ported onto any hardware with minimal memory foot print. When such customization happens, the RTOS and the applications work in the same memory address space and they are integrated. In such tailored operating systems, the application is less protected as the application and OS work in same address space.

Several commercial RTOS are available in market like VxWorks, VRTX, Nucleus, LynxOS, uC/OS, Qnx, etc. Most of these RTOS confirm to the interface standard of IEEE. This is only interfacing standard. Implementation can be done by RTOS implementers differently.

Any commercial RTOS must have certain desirable properties.

- As the RTOS is integrated with the application, it works as a compact embedded system, mostly on firmware and volatile memory without any external hard drives. The memory footprint occupied by RTOS must be very less.

- The integrated application and RTOS have to work on different hardware platforms with different processor architectures. Hence, the RTOS must support different processors.
- RTOS must provide standard application programming interface and debugging tools. Effectively, RTOS is embedded into the application. Debugging should be seamless across the complete system. This is possible when RTOS supports.

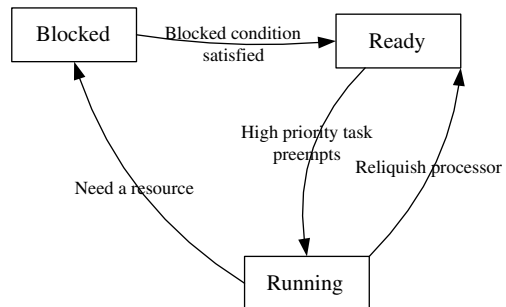
7.2.1 Task and Task States

A task is a finite amount of functionality callable as a subroutine or as a function. The ES application calls the RTOS to start the tasks. RTOS allots stack space, memory for each invoked task.

RTOS maintains the properties of all the tasks including their states. RTOS has a scheduler, which schedules the tasks depending on the task state, task priority, and the scheduling algorithm.

The states maintained for a task depend on the RTOS. Figure 7.1 shows the most essential states a task maintains. When the task is being executed by the processor, it is in “active” or “running” state. The running task relinquishes the processor based on certain conditions or time period or a high-priority task preempts it. When it relinquishes, it remains in a queue which are “ready” for their turn to get processor time. The task is in “ready” state. A task in the ready queue which has higher priority than the running task preempts running task and gets processor time. A “running” task may need certain resource which is locked by another task and is not available. Then the running task gets into “blocked” state waiting for the release of the resource. There may be several conditions on which a task gets into “blocked” state, also called “waiting” state. When the resource becomes available, the “blocked” state transitions to “ready” state waiting for its turn of execution. It is a bad design where all tasks get blocked waiting for some events. The processor becomes idle under the control of scheduler. Similarly, when two tasks having same priority gets unblocked and becomes ready, and their priority is higher than the running task, one of the tasks

Fig. 7.1 Typical states of a task



Task management	Device I/O management	Interrupt &Event handling	Timer management	Memory management	Synchronization & communication
Kernel					

Fig. 7.2 Basic organization of RTOS

has to preempt. This causes non-deterministic state, which is managed by different RTOS in a different way.

Scheduler keeps track of the events on which a task is waiting. When the event occurs (implemented as signal or other way), the scheduler listens and updates the task state.

7.2.2 RTOS—Basic Organization

Kernel is the smallest and central component of the RTOS. Its services include managing memory and devices and to provide an interface for software applications to use the resources. For simple applications, RTOS will be a tiny module and is the kernel. But as complexity increases, the RTOS needs additional modules for networking, I/O, etc. So networking modules, debugging facilities, device I/Os are included in the kernel. An RTOS is generally constituted with two parts: kernel space and user space. RTOS kernel acts as an observation layer between the hardware and the applications. Six types of common services provided by the kernel are shown in Fig. 7.2. Let us see some important characteristics related to multi-tasking in RTOS.

7.2.3 Re-entrancy

In multi-tasking environment, the program’s control flow switches as the tasks switch abruptly. During this switching, the current operation of previous task (task 1) is put on hold and another task (task 2) takes over. If task 1 and task 2 process certain data through a common function, the data get corrupted. Re-entrant functions allow multiple concurrent invocations of a function. This will not interfere with each other’s data. This is extremely necessary in multi-tasking systems.

- Re-entrant functions must satisfy below conditions.
- All the shared variables are used in atomic way unless a separate instance of data is allocated for each instance of the function.
 - A re-entrant function will not call a non-reentrant functions.
 - The function does not use hardware in a non-atomic way because separate instances cannot be allocated.

An atomic operation cannot be interrupted by the processor. As an example, *mov ax,bx* is atomic. It gets executed by the processor as one atomic instruction.

Rule 1 states that in multi-tasking, the operations on shared variables should be atomic.

Now, see the code below by a function

```
temp = val;
temp += 1;
val = temp;
```

While a function is executing the code above, the task A calling this function is switched to task B and same function is called. The variable *val* gets corrupted depending on the state where the function was. So it is a non-entrant function.

We may think by re-coding this as **val++** may make the code atomic as we believe at high-level language, a single instruction is atomic. But atomicity is at assembly language and processor instruction level. Even this, instruction gets coded as

```
mov ax,[val]
inc ax
mov [val],ax
```

C and C++ solve this by instantiating a local variable with each instance of the function. The local values are stored on the stack for each call of this function. i:

```
void some_function(void) {
    sint val;
    val++;
}
```

The best option to eliminate reentrancy is to avoid shared variables. A better approach is to use a mutex to indicate that a resource is busy. Mutexes are simple on-off state indicators whose processing is inherently atomic.

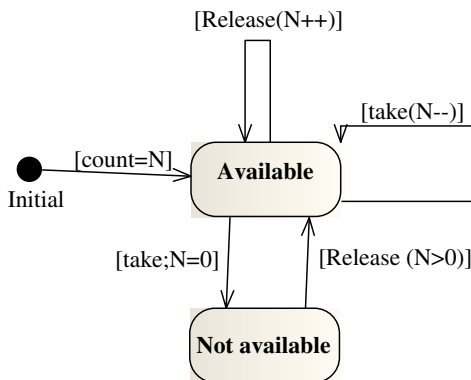
7.2.4 Semaphore

Semaphore is a variable/lock/flag used to control access to a shared resource and avoid shared-data problems in RTOS. It is non-negative and shared between threads. It is a signaling mechanism by which a task waiting on a semaphore can be signaled by another task. It uses two atomic operations, (1) wait and (2) signal for the task synchronization. It is a mechanism that can be used to provide synchronization of tasks, see Fig. 7.3.

Two types of semaphores exist. (1) counting semaphores and (2) binary semaphores.

This type of semaphore uses a count that helps task for a resource to be acquired or released numerous times. If in the initial state, the semaphore is created with a

Fig. 7.3 Counting semaphore



count of N (for this example, can be any value). If the count is > 0 , the semaphore is created with its state as “available.”

Binary semaphores are same as counting semaphores. But their count is restricted to 0 and 1. In binary semaphore, the wait operation occurs if semaphore = 1, and the signal operation occurs when semaphore = 0. It is easy to implement binary semaphores than counting semaphores. Binary semaphores are a type of mutual exclusion but counting semaphores do not provide.

Wait for Operation on semaphore helps you to control the entry of a task into the critical section. **Signal** operation is used to control the exit of a task from a critical section.

One important point to note while using semaphores: a low-priority task can make a high-priority task block on the semaphore. So priority inversion may take place (discussed earlier).

7.2.4.1 Precautions While Using Semaphores

- The initial values of semaphores have to be set properly based on the availability of resources.
- The “symmetry” of takes and releases must match or correspond. Each “take” must have a corresponding “release” somewhere in the ES application.
- “Taking” the wrong semaphore unintentionally will cause in-appropriate use of resources (issue with multiple semaphores).
- Holding a semaphore for too long can cause waiting’ tasks miss deadline.
- Priorities could be “inverted” as explained above.
- Deadlocks can occur. Ex
 - *Task1 and task2 need two resources (A&B locked).*
 - Task1 gets semaphore A.
 - Before it gets semaphore, scheduler switches to task2.
 - Task2 gets Semaphore B.

- Task2 gets blocked while getting semaphore A.
- Both task1 and task2 are now blocked and is a deadlock.

7.2.5 *Mutex*

Mutex is a mutual exclusion object. It is an object that allows multiple tasks to share the same resource, such as file access, but not simultaneously. This is very close to the bank locker system. Mutex resource is complete bank locker. One gets the lock. Others remain in queue till lock is released. When a program is started, a mutex is created for each resource and unique name given. Any thread that needs the resource must lock the mutex if it is in unlocked state. Else it has to wait. The mutex is unlocked when the resource is no longer needed. If the mutex is already locked, a thread needing the resource is typically queued by the system and then given control when the mutex becomes unlocked. The basic difference between semaphore and mutex is that semaphore is a signaling mechanism. The processes perform wait() and signal() operations to indicate that they are acquiring or releasing the resource. In case of mutex, the process has to acquire the lock on mutex object if it wants to acquire the resource and unlock the mutex after the resource is no more needed.

7.3 Basic Design Using RTOS

Commercially, varieties of RTOS products are available. Before selecting one, thorough studies of their features, your requirements and their match have to be studied. In subsequent sections of this chapter, we are going to study Posix-4 and pThread, which are IEEE standards.

Decompose the problem into tasks, keeping the scheduling requirements and real-time responses needed.

Some of the thumb rules to be considered in task decomposition are as below.

- Identify a different task for each different device handling or for different functionality.
- Encapsulate data and functionality within responsible task.
- More tasks offer better control of overall response time. However,
 - More tasks mean more data sharing, hence more protection worries and long response time due to associated overheads.
 - More tasks means inter-task messaging, with overhead due to queuing, mail boxing, and pipe use.
 - More tasks mean more space for task states and messages.
 - More tasks mean frequent context switching (overhead) and less throughput.
 - More tasks mean frequent calls to the RTOS functions.

7.3.1 Case Study 1

There are 255 EVMs that have to be connected to a central processing unit (CP).

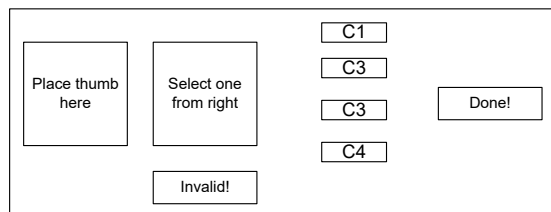
CP holds the IDs of voters, their thumb signatures, and voted option, see Fig. 7.4.

- Communicates with EVMs through messages.
- Role of EVM:
 - When user places thumb, EVM generates a signature.
 - Sends the signature to CP.
 - Receives the assertion/negation message sent by CP.
 - CP asserts if the signature is correct and the user has not voted. CP negates otherwise.
 - Glows “select one from right” and when user selects one option, it pre-validates (i.e., one option is kept pressed for 5 s) and sends the selection to CP.
 - Receives ACK sent by CP in response to EVMs message.
 - EVM glows “done!” and ready to take next user.
- Role of CP:
 - Maintains IDs of voters, their thumb signatures and voted options.
 - Gets messages from all EVMs and responds the action taken.
 - Receives the signature from EVM.
 - Verifies if the signature is correct and the user has not voted. Sends assert/negate message in response to this.
 - Receives candidate selection from EVM and updates the selection and acknowledges to EVM.

Question

- Model the CP functionality as a real-time multi-tasking system. Identify the tasks, their priorities events and objects for inter-task communication/synchronization.
- State the functionality of each task in a descriptive language.
- Draw the execution of the above use case as a sequence diagram.

Fig. 7.4 Electronic voting machine



Solution

Let us identify the tasks and define their behavior. The responsibility for each task and its reaction to the events received are given as bullets. This is not unique solution. Readers should try improved strategies.

Message_Receiver

- Receives the messages from every EVM.
- Puts into received messages queue.
- Suspends itself for next message to arrive.

Message Decoder

- Decodes the received message.
- Updates the action buffer with the information decoded from message.
- Ex: EVM ID, verify operation, signature.
- Adds a session number.
- Set verify event.
- Suspends till the next message to be decoded.

Signature verifier

- Receives a record from the buffer needing an action to verify the signature.
- Interacts with data access layer and verifies the signature.
- Extracts the ID of the person.
- Updates the buffer with the ID.
- Updates the buffer state.
- Posts data to be sent to EVM to message encoder.
- Clear verify event.
- Waits on verify event.

Vote_registration

- Receives a record from the buffer needing to store candidate selection.
- Interacts with data access layer and updates the data.
- Removes the record from buffer once the registration is successful.
- Posts acknowledgment data to be sent to EVM to message encoder.

Message Encoder

- Receives data.
- Frames appropriate message.
- Posts into Transmit Queue.
- Waits for a message to be encoded.

Message_Transmitter

- Pulls a message from transmit queue.
- Transmits to the EVM to be notified.
- Sleeps if there are no messages to be transmitted.

Watchdog

- Checks action buffer every one second for any time outs.
- Generates error message and posts to message encoder.
- Deletes the record from action buffer.

Below are the set of objects that are accessed by the tasks and keep the status.

Objects

Received Queue

- Message receiver posts received message.
- Message decoder waits on this object and pulls the messages for decoding.

Action buffer

- A buffer with one record for each transaction from EVM.
- This can be a FIFO.
- Randomly accessible as the tasks will update each session.
- Message decoder posts/updates a record.
- All action tasks (signature verifier/Vote registration/watch dog.) wait on the buffer data to do actions and update them.

Transmit Queue

- Message encode posts after a message is framed.
- Message transmitter pulls out each message and sends.

Below are the events generated for inter-task communication and coordination.

Events

- OnMessage Received (message receiver waits on this event).
- Onmessageposted in RXQ (message posted in RXQ).
- OnBufferPosted (several action tasks wait on this).
- On verify event (verifier waits on this event).
- Onvoterselect (encoded message is to set the voter selection).
- OnTxQueuePosted (message transmitter waits on this).

Below is the interaction diagram, see Fig. [7.5](#).

7.4 Concept-Process and Threads

Before getting into Posix and multi-threading, let us briefly discuss basic concepts of traditional operating system. For in-depth study, please refer to a Linux book or pThreads primer (Lewis and Berg [1996](#)).

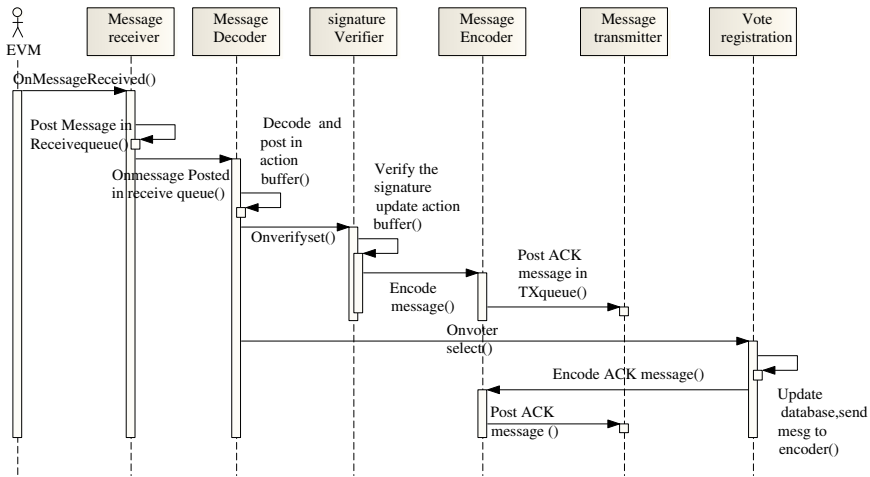
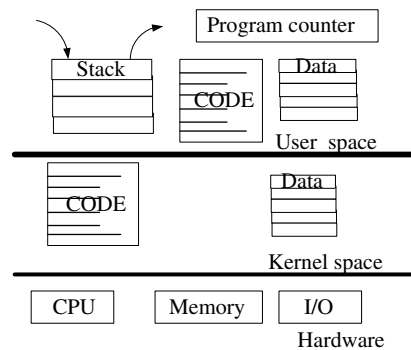


Fig. 7.5 Interaction of multiple tasks

7.4.1 Kernel

Kernel is the computer program that is always resident in memory and interfaces with the hardware resources (processor, memory, I/O, etc.) and with upper layers of applications. The kernel code is loaded in a protected memory segment, which cannot be accessed by application programs. The kernel performs its tasks like process execution, hardware managements, interrupt handling, disc access, etc. The kernel's interface is a low-level abstraction layer. When a process makes requests of the kernel, it is called a *system call*, see Fig. 7.6. The hardware is accessed and controlled by kernel. The application is in separate address space and holds the program counter, stack, and data memory and user code. This layer is called *user space*. When the application needs some services from hardware or from kernel they can be accessed through system calls to the kernel. But in a traditional DOS, the application can

Fig. 7.6 Layered representation of traditional OS



access kernel space. In DoS, the partitioning across kernel and user is implicit and no hardware enforcement.

In the case of multi-tasking systems, there is strict control of resource access across different layers. The user programs cannot access kernel data. The access will be through system calls provided; the user has permissions for it.

The system works in *user mode* or in *kernel mode*. In user mode, the application programs run using the resources from user space only like stack, data, and code. When any user application needs services from lower layers, it can get through system calls of kernel.

When the system is in kernel mode, some special instructions can be run. Kernel mode instructions are like processor interrupts, memory management, I/O, etc. These can be executed by kernel only. When the user program needs some services of kernel, it makes a system call. A system call is basically a function that ends up trapping to routines in the kernel. Hardware traps this instruction, passes to kernel. The kernel figures out what the user wants and whether the user has permission to do so and executes.

7.4.2 Process

The application shown in user space is typically a process. They execute in the user mode. There can be multiple processes running at the same time as shown in Fig. 7.7. As these processes are running under the control of the operating system, the status of each process is maintained in the kernel space. This is a brief structure of the multi-tasking operating system. Each of the processes has its own stack, code, program segments, and necessary virtual registers to execute programs. These processes switch and get scheduled on to the processor by the operating system.

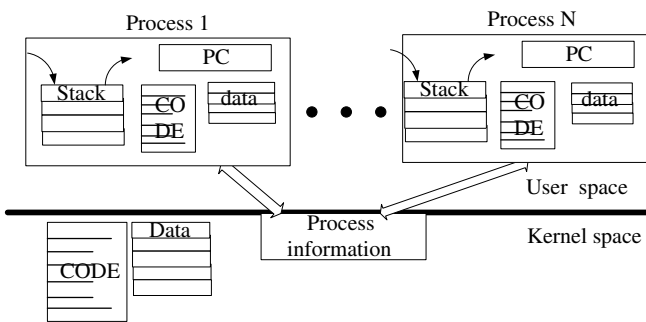


Fig. 7.7 Multi-tasking

7.4.3 Thread

Multiple processes can be scheduled on a system. Similarly, a process can have multiple independent executables running concurrently. Each executable is a *thread*. So a multi-threaded process can execute concurrent threads. Each thread is a different stream of control that can execute its instructions independently. A thread is a lightweight entity, comprising the registers, stack, and some other data. All threads in a process share the state of that process.

Each thread has its program counter, stack, and register set. Each thread maintains its state roughly with the following data

- Thread ID.
- Register state (including PC and stack pointer).
- Stack.
- Signal mask.
- Priority.
- Thread-private storage.

All threads are in the same memory space. They can share data of other threads. Processes are like people working in two rooms. Sharing their activities is tough. Creating a new process is a heavy job like creating a different room and all essentials needed for working. Thread is like a group of persons working in a room. There may be multiple groups (threads) in the same room working on different tasks and sharing their data across them.

Hence, threads are lightweight and low context switching overheads. There are specific advantages in developing multi-threaded (MT) programs as listed below.

- Higher throughput by effective utilization of processor.
- Reducing heavy switching contexts and thus increasing application responsiveness.
- Effective use of system resources.
- Real-time responses through efficient scheduling.
- Same code can be used on multi-processor system for parallel execution. Performance gains from multiprocessing hardware (parallelism).
- Reducing process-to-process communication.
- Efficient use of system resources.
- Simplified real-time processing.
- Simplified handling of signals.
- Inherent ability of concurrency property of distributed objects can be utilized.
- The programs will be well structured and close to the way, we handle in real-world scenario.
- There can be a single source for multiple platforms.

7.5 Posix

Posix (Gallmeister) is the acronym that stands for *portable operating system interface*. When an application is developed in multi-tasking environment, the processes will make system calls as discussed above. The systems calls are from the operating system over which our process is running. When someone designs an application over a commercial operating system and plan to port the application to some other environment where the OS is a different one, the system calls may not be with the same syntax and also the behavior may be different. So the application *will not be portable*.

To confirm portability, standard OS call interface and behavior are standardized by IEEE as POSIX standard. The goal is to provide source-level portability as the standard defines syntactic and semantic standard. Commercial Posix compatible OS implements with standardized source calls. Implementation may be different. Hence, any application written as per Posix calls, can run on any Posix compatible operating systems. The standards are refined with minimal syntactic and semantic changes to support real-time extensions.

- POSIX.4 supports *Real-time extensions* that define interfaces to support the portability of applications with real-time requirements.
- POSIX.4a supports *threads extension* that defines the interfaces to support multiple threads of control inside each POSIX process.
- POSIX.4b supports *additional real-time extensions* that define interfaces to support additional real-time services.

The real-time extensions provided in Posix-4b are

Timeouts: Maximum amount of time that the process may be suspended while waiting for the service to complete.

Execution-Time Clocks: Timers may be defined for each process and each thread. Allow to detect when an execution time overrun occurs.

New scheduling policy: *Sporadic Server*: process aperiodic events at the desired priority level, guaranteeing the timing requirements of lower priority tasks.

Interrupt Control: A process or thread can receive and respond to an interrupt. The process registers a user-written interrupt service routine.

Input/output Device Control: Allows an application program to transfer control information to and from a device driver.

7.6 pThreads

Every popular operating system provides thread support in their processes. Popular ones are.

- POSIX Threads.

- Win32 and OS/2 Threads.
- DCE Threads.
- Solaris Threads.

The POSIX specification pThreads (IEEE 1003) is intended for all computing platforms. Implementations are done commercially on Unix systems and most of real-time operating (RTOS) systems. It is impossible to cover pThreads in one chapter. We will cover most important concepts in brief. For further reading, please refer Lewis and Berg (1996).

7.6.1 Create Thread

The *pthread_create* creates a thread. This call has one of its arguments the thread function with its arguments.

A thread is exited by calling the appropriate *thread exit* function. There is no relation like a parent thread creating child thread and so on. Any thread can create as many threads as it needs.

```
#include <pthread.h>

int pthread_create ( pthread_t *thread_handle, const pthread_attr_t *attribute,
void * (*thread_function) (void *), void *arg);
```

```
pthread_create(&tid, NULL, start_fn, arg);
pthread_exit(status);
```

Attributes of a thread:

Stack Size: Size of the stack.

Stack address: Region of user allocated memory to be used as a stack region.

Detachstate: State of a thread.

Contention scope: How threads compete for resources.

Inheritsched: Whether the thread is created with scheduling parameters inherited from its parent thread.

Schedpolicy: Scheduling policy for the thread.

Example—Create and exit multiple threads

```
#include <pthread.h>
#include <stdio.h>
void *PrintHello(void *threadid)
{
    long tid;
    tid = (long)threadid;
    printf("Hello World! Thread number : %ld!\n", tid);
    pthread_exit(NULL);
}
```

```
int main (int argc, char *argv[])
{
    pthread_t threads [5];
    int rc;
    int i;
    for (i=0; i<=4; i++)
        printf("creating thread _%ld\n", i);
    rc = pthread_create(&threads[i], NULL, PrintHello, (void *)i);
    if (rc){
        printf("ERROR;");
        exit(-1);
    }
}
```

Join threads.

7.6.2 Thread Synchronization

`pthread_join` is a way of synchronizing two threads, see Fig. 7.8a. In the example below, T1 creates T2 and decides at some extent to wait for exit of T2. Then it sleeps

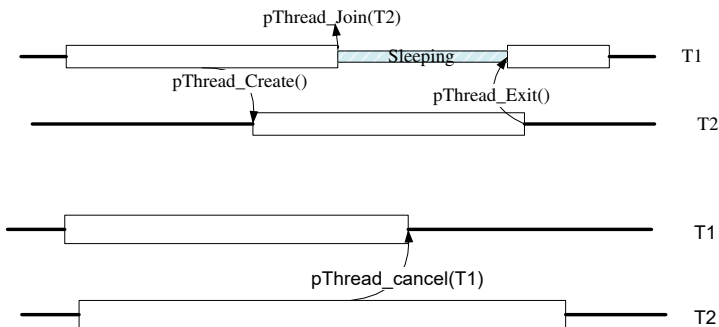


Fig. 7.8 a Join command b Thread cancellation

till T2 exits. When a thread is created, you can set its attribute `set_detach_state` attribute as `PTHREAD_CREATE_JOINABLE` so that the thread is joinable. Only threads that are created with the joinable attribute can be joined. If a thread is created as detached, it cannot be joined. The template code is given below.

```
#include <pthread.h>
--
void * thread_function(void *t)
{
--
}
int main (int argc, char *argv[])
{
    pthread_t threads[5];
    pthread_attr_t attr;
    --
    /* Initialize thread attribute */
    pthread_attr_init(&attr);
    pthread_attr_setdetachstate(&attr,
PTHREAD_CREATE_JOINABLE);
    --
    rc = pthread_create(&threads[t], &attr,
thread_function, (void *)t); //Create pThreads
    --
}
```

7.6.3 Cancel Thread

It is possible for one thread to tell another thread to exit. There is no relationship between the threads. Syntax

```
#include <pthread.h>
nt pthread_cancel(pthread_t thread);
```

The thread `cancel ()` function shall request that thread to be canceled. During the cancel action, the cleanup handlers for the thread will be called, see Fig. 7.8b. When all cleanup handlers complete execution, the thread-specific destructor functions shall be executed. When the last destructor function returns, thread shall be terminated. This is akin to some person leaving the job and someone takes over all the data, systems with him. Without such handlers, data get destroyed and whole system will loose its state.

7.6.4 Schedule Threads

Before we get into the different mechanisms of scheduling multiple threads, we have to get into more details of processes, the threads in each process and the processors available as resources.

A process having multiple threads and built on a kernel with single processor, the threads can execute concurrently. In a multi-processor environment, each thread in the process can run on a separate processor at the same time, resulting in parallel execution.

The threads library uses underlying threads of control called lightweight processes (LWP) that are supported by the kernel, see Fig. 7.9. You can think of an LWP as a virtual CPU that executes code or system calls. We normally are not concerned with LWPs while programming with threads, but the threads in a process execute.

LWPs bridge the user level and the kernel level. Each process contains one or more LWPs. Each LWP runs one or more user threads. The creation of a thread usually involves just the creation of some user context, but not the creation of an LWP. Each LWP is a kernel resource. When the threads are created or scheduled, they are allotted to a LWP. This is transparent to the thread programmer.

See Fig. 7.10. Threads are scheduled on to the kernel resources in three ways. The first technique is “Many-to-One” model. Multiple threads created in the user space by a process will run on one LWP by turn. The second one is “One-to-One” model where kernel allocates one LWP for each thread. This model allows many threads to run simultaneously on different CPUs. This model has the drawback that thread creation involves LWP creation which takes more kernel resources. The third model is “Many-to-Many” model. Multiple number of threads are multiplexed on multiple number of LWPs. Thread creation is done completely in user space. The number of LWPs may be tuned for the particular application and machine. Numerous threads can run in parallel on different CPUs, and a blocking system call need not block the

Fig. 7.9 Process, threads, LWPs, and kernel

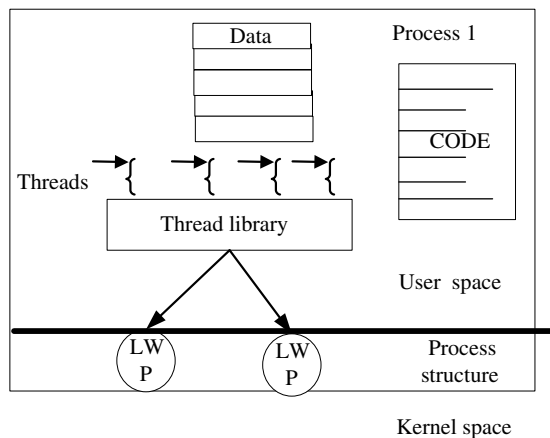
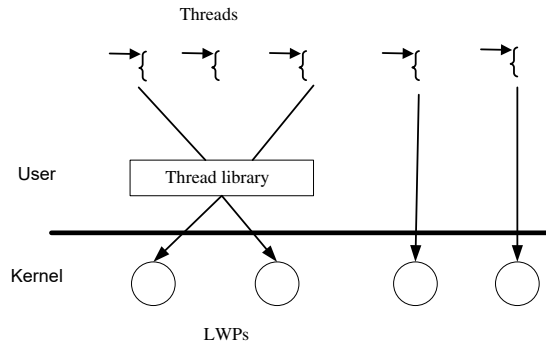


Fig. 7.10 Scheduling threads on LWPs



whole process. In any case, a thread can be directly accessed by another process or moved to another process.

7.6.4.1 Scheduling Policies

Scheduling policies are the same as those defined for processes in POSIX.4. There are three ways of scheduling threads in Posix4, process local scheduling, global scheduling, and mixed scheduling.

Process local scheduling is also known as *Process Contention Scope*. The contention among threads lies within the process. The scheduling mechanism for the thread is local to the process. The threads library has full control over which thread will be scheduled on an LWP. However, the scheduling of the LWP is still global and independent of the local scheduling.

An active thread T 1 switches and relinquishes the LWP to other threads when

- T1 needs a resource and waits on the resource availability. A thread with the highest priority in the queue takes over.
- A thread T2 gets a locked resource and T2 having higher priority, it preempts T1 and takes over LWP.
- The running thread yields itself by making a call `sched_yield()`. The thread waiting in queue with high priority will take over.
- A periodic time slice event occurs to give equal share to other threads.

The other one is *system global scheduling*. It is also known as *System Contention Scope*. In system contention scope, scheduling is done by the kernel.

The third one is *mixed scheduling* where some threads have *global contention scope*, and other threads have *local contention scope*. Scheduling is done at two levels: in the first level, processes and global threads are scheduled; at the second level, local threads within the selected process are scheduled.

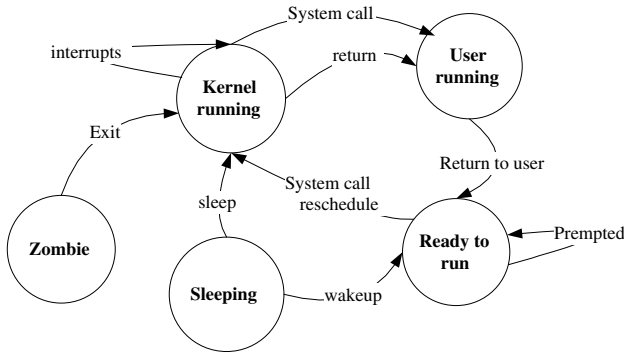


Fig. 7.11 Thread states

7.6.4.2 Thread States

Figure 7.11 shows the typical states in Posix, varies with product to product.

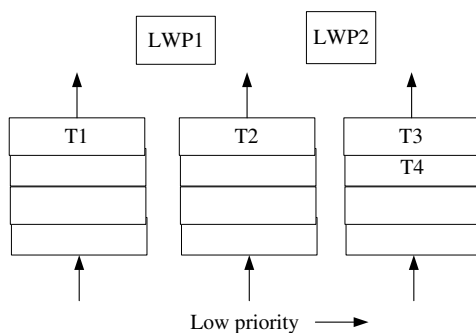
- **User-running:** Process is in user-running.
- **Kernel-running:** Process is allocated to kernel and hence, is in kernel mode.
- **Ready to run:** The process is not executing but is ready to run as soon as the kernel schedules it.
- **Preempted:** Kernel preempts an on-going process for allocation of another process.
- **Sleeping:** Process is sleeping but resides in main memory. It is waiting for the task to begin.
- **Zombie:** Meaning that it is a dead thread and is waiting for its resources to be collected.

7.6.4.3 Scheduling Classes

POSIX specifies three scheduling policies: first-in-first-out (SCHED_FIFO), round-robin (SCHED_RR), and custom (SCHED_OTHER). Both SCHED_FIFO and SCHED_RR are POSIX Real-time extensions. SCHED_OTHER is the default scheduling policy.

SCHED_FIFO

Sched_FIFO is a queue-based scheduler with different queues for each priority level, see Fig. 7.12. The priority of threads is fixed. First thread at a given priority level will be the First one at that priority level to run (active state on LWP). The thread runs until it voluntarily exits or is preempted by a thread of higher priority.

Fig. 7.12 Sched FIFO queue

High utilization levels can be achieved by using rate-monotonic (RMA) or deadline monotonic priority (EDF) assignments. All you need to know is the frequency at which all your processes need to run. For sporadically interrupting tasks, you can use the worst-case arrival rate as the frequency for the purposes of assigning priorities.

As an example, Fig. 7.12 is shown with three queues for priorities 1–3. Lowest priority queues hold T3 and T4. T1 gets LWP1. Assume T2 gets blocked waiting for a mutex. T3 is in the top of the queue and gets LWP2. When T1 gets blocked due to resource unavailability T4 gets LWP1. When T2 unblocks T4 relinquishes LWP1 and T2 acquires it. This process continues.

SCHED_RR

SCHED_RR is like Sched_FIFO except that each thread has an execution time quota. It is a round-robin scheduler where the threads have only the time quantum to run before it gets shuffled back to the end of the queue for their priority level. This gives another thread with the same priority a chance to run. SCHED_RR uses a system-provided quantum value that you cannot alter.

SCHED_OTHER

POSIX puts no limits on the behavior of this option. Commercially, it is open for different schedule implementations.

Sample code snippet to set schedule policy is below.

```
#include <sched.h>

int sched_setscheduler(pid_t pid, int policy,
const struct sched_param *param);

int sched_getscheduler(pid_t pid);

struct sched_param {
    ...
    int sched_priority;
    ...
};
```

7.7 Thread Synchronization

We have introduced synchronization earlier in this chapter. When multiple threads execute independently, they have to coordinate in accessing commonly shared variables. When a thread is operating and updating shared data, other threads should wait for updating the shared data. In the below example, thread1 has the task of reading the levels of multiple tanks and update an array of tank levels. It has to signal the thread2 that all tanks are scanned. Thread-2 computes the average of tank levels and goes to sleep till the next scan is done. Here there is signaling across the two threads and also mutual exclusion in accessing level data. This is provided in pThreads by synchronizing primitives mutex, condition variables, Semaphores, reader-writer locks, and many more.

```
//thread-1
While(true)
{
    Level=readlevel(tankid)
    Store (level,tanki id, tank_data)
    Tankid++
    If(tankid>max)
        {Tankid=0
        Alltanks scanned=true}
}
```

```
//Thread-2
While (true)
{
    If (all tanks scanned )
        avglevel=compute average level()
}
```

7.7.1 Mutex

Mutexes synchronize threads for mutually exclusive access to shared resources. It is the most primitive synchronizing variable. It has two calls:

- `pthread_mutex_lock(m)`.
- `pthread_mutex_unlock(m)`.

It provides a single, absolute owner for the section of a code (thus a critical section). The first thread that locks the mutex gets ownership. Any efforts to lock it will fail and the calling thread waits (sleep) till the mutex is unlocked.

A sample code of using mutex is given below. The two functions use the mutex lock for different purposes. The **`add_item()`** function uses the mutex lock simply to ensure a lock when it updates the list with one more item. The **`get_count()`** function uses the mutex lock to guarantee that the items in the list are not accessed by any other so that it can count the items correctly.

```
include <pthread.h>

pthread_mutex_t list_mutex;
int list_count;
struct *list_items

void add_item()
{pthread_mutex_lock(&list_mutex);
 //add items to list
 pthread_mutex_unlock(&list_mutex);
}

int get_count()
{
 int c;

 pthread_mutex_lock(&list_mutex);

 //count list items
 c = count_list_items();
 pthread_mutex_unlock(&list_mutex);
 return (c);
}
```

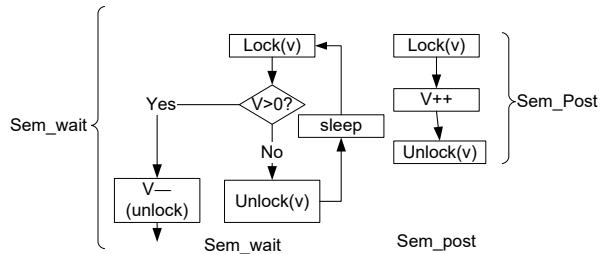
Below are the mutex synchronization protocols

- **`NO_PRIO_INHERIT`**: a mutex is owned by the thread that locked it. The priority of the thread does not change based on the mutex it is owning.
- **`PRIO_INHERIT`**: The thread owning a mutex inherits the priorities of the threads, which are waiting to acquire that mutex (priority inheritance protocol).
- **`PRIO_PROTECT`**: When a thread locks a mutex it inherits the priority ceiling of the mutex, (priority ceiling protocol).

7.7.2 Semaphore

We explained semaphore earlier and now let us look into how it is implemented in posix-4. A semaphore is initialized as an object with a value, see Fig. 7.13. The value

Fig. 7.13 Semaphore locked as hidden mutex



is 0 or 1 for binary semaphores and more than 1 for multi valued. The object state in count can be interpreted by its users (threads) in multiple ways. When a common resource that is mutually exclusive is to be accessed by multiple threads it can be done by the state of the semaphore object.

If there is a single resource to be accessed, if the semaphore is 1, the resource is available and the thread can get by `sem_get()` and set the value to zero. Any other thread wants to get it will wait on the sem object. When the previous thread posts `sem_post()` the waiting thread is released. This is a good synchronizing mechanism, same is represented as flowchart below.

Semaphore looks as a hidden mutex. `sem_wait()` does is lock the mutex. It checks the value. If it is greater than zero, the value is decremented, the hidden mutex is released. If the value of the semaphore is zero, then the mutex will be released, and the thread will go to sleep. `sem_post` locks the mutex, increments the value, releases the mutex, and wakes up one sleeper (if there is one).

A semaphore is initialized by using `sem_init`

```
sem_init (sem_t *sem, int pshared, unsigned int value);
```

To lock a semaphore or wait we can use the `sem_wait` function:

```
int sem_wait (sem_t *sem);
```

To release or signal a semaphore, we use the `sem_post` function:

```
int sem_post(sem_t *sem);
```

```
// sample code - Semaphores
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t sem1;
void* thread(void* arg)
{
    //wait
    sem_wait(&sem1);
    printf("Entered.");
    //critical section
    sleep(4);
    //signal
    printf(" Exiting.");
    sem_post(&sem1);
}
```

```
int main()
{
    sem_init(&sem1, 0, 1);
    pthread_t t1,t2;
    pthread_create(&t1,NULL,thread,NULL);
    sleep(2);
    pthread_create(&t2,NULL,thread,NULL);
    pthread_join(t1,NULL);
    pthread_join(t2,NULL);
    sem_destroy(&sem1);
    return 0;
}
```

t2 created after 2 s after t1. t1 will sleep for 4 s after acquiring the lock. t2 gets critical section after enter $4 - 2 = 2$ s after it is called.

7.7.3 Condition Variable

Condition variables always have an associated mutex, see Fig. 7.14. CV tests the condition under the mutex's protection. If the condition is true, your thread completes its task, releasing the mutex when appropriate. If the condition isn't true, the mutex is released and the thread goes to sleep on the condition variable. No other thread should alter any aspect of the condition without holding the mutex. As long as you can express the condition in a program, you can use it in a condition variable.

Mutexes and Condition Variables are a way for threads to synchronize. Mutexes implement synchronization by controlling thread access to data. Condition Variables allow threads to synchronize based upon a satisfying condition on the data. Without condition variables, threads continually poll to check if the condition is met. The

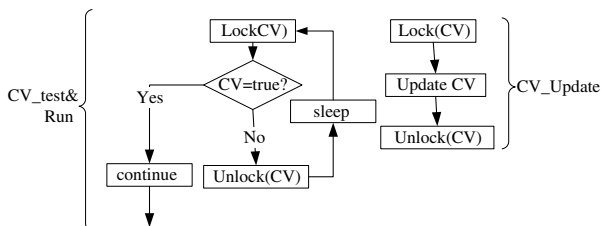


Fig. 7.14 Using condition variable CV

process loses time and is inefficient. CV can do the same job while the process is busy in doing its task. A CV is always used in conjunction with a mutex lock.

- `pthread_cond_t` Cv declaration.
- `pthread_cond_init(condition,attr)` Set condition variable object attributes, ID of the created CV is returned through condition and attr.
- `pthread_cond_destroy()`: free the CV.
- `pthread_cond_wait()` blocks the calling thread till the specified condition is signaled.
- `pthread_cond_signal()` routine wakes up another thread, which is waiting on the CV.

7.7.4 Reader's/Writers Lock

RW lock allows any number of threads to read a shared data. When a thread wants to update the data, it has to wait until all reader threads complete reading. Similarly, all reader threads are blocked until a writer thread completes writing. Reader/writer lock is very efficient because most of the threads are readers and one of them will be updating at any time. If you use mutual exclusion only one can access the shared data though it is not going to update it.

```
// compute Global minimum (GV) of complete list
//Break the list. Let each thread find partial minimum(pv)
//If pv is less than GV update GV by read_write lock.
void *find_min(void *list_ptr) {
    break the list...;
    for (i = 0; i < part; i++)
        find min pv for each part ;
    /* lock the mutex associated with minimum_value
    // and update the variable as required
    mylib_rwlock_rlock(&read_write_lock);
    if (min of part list < GV)
        { mylib_rwlock_unlock(&read_write_lock);
          mylib_rwlock_wlock(&read_write_lock);
          GV = min of partlist;
          mylib_rwlock_unlock(&read_write_lock);
        }
    pthread_exit(0);
}
```

In the above pseudo code, a long list is broken into N partial lists. Each thread computes the minimum element (PV) in its partial list. Each thread attempts a read lock on the global minimum value (GV). If the GV value is greater than the minimum value of the partial result, the GV has to be changed to the new minimum value. So the thread attempts a write lock on GV to update it. By this process of reader lock all partial threads read the value without getting blocked and only that thread which

found lesser than GV tries to get write lock. So the number of blocks on GV will be very less.

7.7.5 Spin Locks

You should hold a lock for the shortest time possible, to allow other threads to run without blocking. If you are going to be blocked for long time, you are not doing any effective work. In such cases, you can peep in and check whether the mutex is unlocked. You can use a *spin lock* and try again. You initialize a counter to some value and do a `pthread_mutex_trylock()`—that takes very less time. If you don't get the lock, decrement the counter and loop. When the counter hits zero, then give up and get blocked. If you get the mutex, then you've saved a bunch of time. If you don't get it, then you've only wasted a little time. Spin locks can be effective in very restricted circumstances. The critical section *must* be short, you *must* have significant contention for the lock.

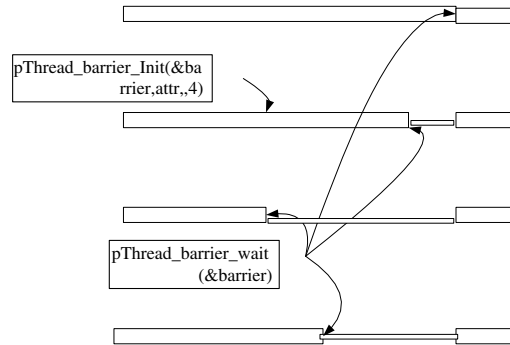
```
spin_lock(mutex_t *m)
{int i;
 for (i=0; i < SPIN_COUNT; i++)
 {if (pthread_mutex_trylock(m) != EBUSY)
  return; /* got the lock! */
 else
  //do some job
 }
 pthread_mutex_lock(m); // give up and block.
 return; } /* got the lock after blocking! */
```

7.7.6 Barrier

A way to make N number of threads do their respective jobs and wait, see Fig. 7.15. Once all N threads complete their jobs and wait till all the threads get unblocked and continue their jobs. This is very close to the analogy of N friends in a hostel deciding to do their jobs and wait for all others to meet at one place. Once all have met, they proceed further.

```
int pthread_barrier_init(pthread_barrier_t *barrier, const pthread_barrierattr_t
*attr, unsigned int count);
```

This creates a barrier object at the passed address, with the attributes as specified by *attr*. The number of threads that must call `pthread_barrier_wait()` is passed in *count*. Once the barrier is created, we then want each of the threads to call `pthread_barrier_wait()` to indicate that it has been completed.

Fig. 7.15 Barrier

```
Int pthread_barrier_wait (pthread_barrier_t *barrier);
```

When a thread calls `pthread_barrier_wait()`, the calling thread gets blocked till the number of threads specified initially in the `pthread_barrier_init()` have called `pthread_barrier_wait()`. When the correct number of threads has called `pthread_barrier_wait()`, all those threads will “simultaneously” unblock.

7.8 Design Strategies

From the study done on threads in this chapter, we can compare threads as workers in a project, or employees in an office. They are given quantum of work, compete for the resources they need, sleep till the time they get resources, communicate with other workers, and they also have liberty to create new workers to offload certain work and at the last remove those workers not needed.

The challenge now is how to orchestrate these threads to do a complex job, akin to managers and executives. Different strategies are followed and we conclude this chapter giving some of them in brief.

7.8.1 Master–Slave

One thread has to do the complete work. This thread creates multiple threads, with assigned jobs and needed coordination. They are slave threads and execute the job assigned and get destroyed after the job is over.

7.8.2 Thread per Client

Basically used when multiple users are to be served, the number of users needing a service is not known apriori. As a service request comes, a thread is created. After the user's service is over the thread gets closed. The thread is dedicated exclusively to the client. This is nothing but client-server architecture. Threads are dynamically created to serve the clients.

7.8.3 Thread per Request

Similar to the dynamic creation of threads by the server. But one thread is created for each service request it gets. Once the response to the service is done, the thread is closed. This is close to service-oriented architecture.

7.8.4 Work Queue

There will be multiple work queues. The queue consists of finite jobs to be done. Does not bother who does the job. Certain threads push the jobs into the queue. Certain threads pull the jobs and execute the jobs. This is similar to reservation counter where a coordinator allots a customer to a counter and pushes him into the queue. One worker at each counter pops the customer from the queue and services him.

7.8.5 Pipeline

The total work is broken into multiple stages. After completing the work at stage 1, it is pushed into a queue as input to stage 2. Stage 2 takes the job from queue, executes, and then pushes to queue for stage 3. One thread at each stage completes the finite amount of work and pushes to the next stage. Very similar to the way a product is manufactured in a workshop by processing stage by stage.

7.9 Summary RTOS

After studying the characteristics of a real-time system and reference model in the previous chapter, we have switched how these concepts are implemented as RTOS in this chapter. The structure of commercial RTOS varies a lot from product to

product. You will find hundreds of RTOS in market with varied features. They can be classified based on the processors over which they can be ported, the minimal memory footprint needed, supporting features and response times, the architecture, whether designed for safety critical systems, compliant to standards like Posix, and so on. So we have touched upon generic RTOS concepts and studied Posix-4, which is real-time extensions and then studied pThreads covering major features.

Most of the popular RTOS comply with Posix-4 real-time extension standards and support Romable versions on popular microprocessors. Users have to make deep study and select RTOS legitimately.

7.10 Further Reading

Several books are authored on POSIX and pThreads. As an embedded systems developer, it is worth putting more focus on multi-threading concepts. Oracle, IBM provides excellent tutorials on use of pThreads. As a book on pThreads Lewis and Berg (1996) covers well and a must read book to get concepts cleared.

7.11 Exercises

1. An electronic door access system has to be developed. A reader is attached on each door, which reads the thumb impression, validates, and sends the information to the server for registration. Server accepts messages, registers access, and acknowledges to the reader. Around 100 such readers are served by the server. The details of the reader and server functionality are as below. Sequence of operations:
 - a. A green lamp indicates that the reader is ready to accept.
 - b. When the thumb is placed on the reader for at least 5 s, reader generates a 16-bit signature and sends it to server as a message with the signature.
 - c. Server verifies from internal list of signatures and acknowledges whether it is a valid signature.
 - d. Server registers the ID of the person belonging to the signature, the time and access type (IN/OUT) if it is valid.
 - e. Reader operates door relay for 10 s and sets back to READY state.
A transaction gets aborted
 - f. When thumb is placed less than 5 s.
 - g. No acknowledgment message is received to the reader within 20 s.
 - h. Or any abnormal event occurs.

Both reader and server have to be implemented using a real-time operating system.

- Identify the tasks and their interaction for the reader.
 - Identify the tasks and their interaction for the server.
 - Represent the total behavior as an interaction diagram.
 - Write pseudo code for each task.
2. A vending machine has to be developed with following features.
- System accepts three (one rupee) coins one after the other.
 - If total time of dropping the coins exceeds one minute, all pending coins will be released.
 - System validates each coin as and when it is dropped. If a coin is invalid, all pending coins will be released.
 - System releases the item by operating a relay after final validation of the three dropped coins.
 - Inter arrival of coins can be as low as 1 ms.
 - System can accept coins and validates coins concurrently when the item is being released.
- Define the components to design a real-time multi-tasking system and pseudo logic to implement.
3. A system has to be designed to monitor and control the temperatures of an industrial process. Detailed specifications are as below:
- a. There are multiple independent heating processes (P_1 to P_{100}) each actuated by a heater (HT) and sensed by a temperature sensor (TS).
 - b. Both HT and TS are intelligent devices. One can communicate bi-directionally with TS and HT through serial commands.
 - c. Commands to HT
 - Put ON heater.
 - Put OFF heater.
 - Send status (working/not working).
 - d. Commands to TS
 - Send status (working/not working).
 - Set scan rate (rate at which temperature is sensed).
 - e. Messages from TS
 - Scanned temperature values.
 - Alarm.
 - f. The Processing Station (PS) manages the overall system and has the following use cases:
 - Presets the scan rate of each TS, i.e., the rate at which TS reads temperature and sends to PS.
 - Reads scanned temperature values sent by TS.
 - Gets alarm messages from TS and takes action.

- Puts heater on/off if the TS is out of range.
- Scans health of each TS and HT every 1 s.
- Processes the average of last 100 samples of each TS and updates history.

Questions

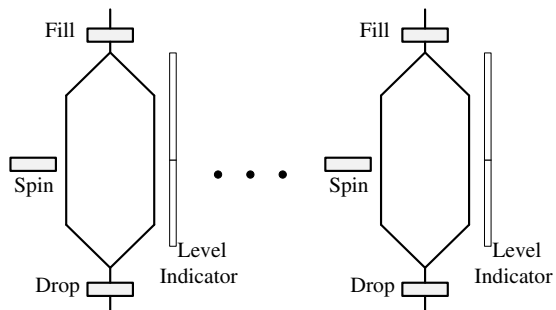
- Identify the tasks, their priorities, and state the functionality of each task in a descriptive language.
 - Identify all events and objects for inter-task communication/synchronization.
 - Take any two use cases (stated at 6), write pseudo code for the tasks.
4. There are 50 milk spinning machines, which run concurrently. All the machines are controlled by a single processing unit (CP). There are three relay-activated operations for each machine.

- Fill = ON to open “fill” valve. A level sensor senses that milk is filled to desired level in the machine.
- Spin = ON to spin the machine for fixed time. The spin time is to be 10 mts.
- Drop = ON to release the milk. Drop valve is to be open for 1 mt for a complete drop.

Constraints

- A maximum of 10 machines can spin at a time.
 - A machine cannot spin till the milk is to the desired level (Fig. 7.16).
 - Model the CP functionality as a real-time multi-tasking system. Identify the tasks, their priorities, events, and objects for inter-task communication/synchronization.
 - State the functionality of each task in a descriptive language or in any RTOS.
 - Represent task-level interaction using appropriate UML diagram.
5. An athletic track (100 m race) has to be automated.
- The track has eight lanes for the athletes to run.

Fig. 7.16 Milk spinning machines



- The system starts the race by generating short beep.
- When the athlete is reaching the Finish line, he passes over a foot switch which generates a pulse of 10 ms width. System senses this pulse (positive edge) for each lane to compute the time taken by each athlete.
- Similar foot switch (start foot switch) is at start line. If the athlete starts before the beep, (sensed by start foot switch), system records as “foul” for the lane.
- If the athlete could not complete the race within 12 s, system records as “Not completed” for the lane.
- The resolution of running time to be recorded is 1 ms.

Problem:

A real-time system has to be designed to compute the completion times of all athletes (or foul/incomplete). System has to store the completion times in order of completion.

- a. Explain the strategy in brief.
 - b. Explain the design in terms of tasks, and objects for synchronization, event capture and management using Posix-4 entities.
 - c. Write pseudo code for each task.
6. A system has three concurrently executing threads A, B, and C created by the main thread. You are asked to create these three threads from main with the following attributes
- Thread A is a detachable non-real-time thread with SCHED_OTHER as scheduling policy.
 - Threads B and C are joinable threads with SCHED_RR as scheduling policy and 10 and 5 as dynamic priorities, respectively. These threads return thread id when they exit (to main thread). Thread C changes its scheduling policy and dynamic priority at runtime (from thread C itself) to SCHED_FIFO and 0.
 - These concurrently executing threads are trying to access a share resource. The order in which they access these shared resources is as follows:
 - Thread A only can start accessing the shared resource. After thread A finishes accessing shared resource once, thread B or thread C (not both together) can access the shared resource. After thread B and thread C accessing the shared resource twice each, thread A will get its next chance. This loop will continue 3 times (shared resource is accessed 3 times by thread A, 6 times each by thread B and C) before all these threads exit out. Write a thread program that implements the above task using POSIX libraries (pthread and semaphore libraries).
 - [Note: Use minimum semaphores to solve the problem. Make sure you are showing initializations of semaphores and thread attributes. Write C program not pseudo code].
7. An online examination system has to be designed.

- a. Each student is given a unit (client), which has four push buttons. The student will answer by pressing one of the push buttons.
- b. All the clients are connected to a server, which controls the examination. The server's role is
 - Projecting the question on OHP by a command or signal.
 - Switch to next question after the time for the current question is over. (Time to answer for each question is not same.)
 - Give a beep when it is switching to the next question.
 - Receive all answers given by the students during the time frame for each question.
 - Compute marks after projecting all questions.
- c. Client communicates with the server by a set of serial messages (media and the protocol are not important).
- d. The server has to be designed in multi-threaded environment.

Problem:

- i. Identify the client-server communication mechanism. Define message content.
 - ii. Identify the threads, synchronization objects in the server and their interaction.
 - iii. Write the pseudo code for the functionality of each thread. Draw the sequence diagram for one use case.
1. Draw a data flow model for the entire process (as a diagram).
 2. Identify the threads and associated entities to implement this as multi-threaded system.
8. A global variable COUNT is incremented by two threads concurrently. When the count reaches a threshold value (say 20), the incrementing thread has to notify a thread waiting for this event. The waiting thread simply prints the time of this occurrence (reaching threshold). Using the Conditional Variable implement this functionality. Write the code in C using pThreads in correct syntax.
 9. A set of tasks (T1 to T3) have to be scheduled with Earliest Deadline First (EDF) philosophy, which states that *"Given a set of N independent tasks with arbitrary arrival times, the algorithm at any instant executes the task with the earliest absolute deadline among all the ready tasks."* The scheduler is preemptive. Given the below tasks with release time, execution time and dead line in the table, draw the sequence in which the tasks will be scheduled as per above policy.

Task	Release time	Execution time	Absolute deadline
T1	0	10	33
T2	4	3	28
T3	5	10	29

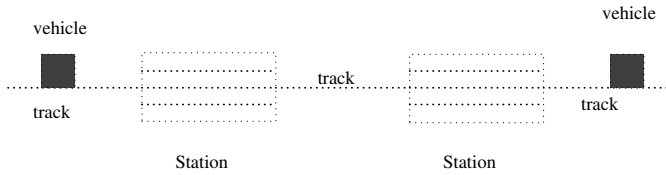


Fig. 7.17 Unmanned vehicle moving system

10. An unmanned vehicle moving system has to be developed with the below features. PI assumes any use cases, which have not been mentioned (Fig. 7.17).
 - a. The vehicles are unmanned and have a node with processing and wireless communication capabilities.
 - b. The vehicles move in both directions.
 - c. They stop at each station.
 - d. Each station has three platforms available.
 - e. Each station is unmanned and has one node with processing and wireless communication capabilities.
 - f. For safety, one track (the segment between two stations) can be occupied by one vehicle.
 - g. Each station node controls the occupancy of its platform.
 - h. The track occupancy is also controlled by a station (to be decided by you in the design).
 - i. The communication across the vehicles and the station nodes control safety rules.

Problem:

 - a. Design the overall strategy and explain with any appropriate model.
 - b. Design the vehicle node and the station node as a multi-threaded system. Identify the threads and associated objects. Explain their behaviors using pseudo code.
 - c. Explain the dynamic behavior using sequence diagrams.

References

- Barney B (2012) Posix thread programming
 Gallmeister BO (1995) POSIX.4: programming for the real world. O'Reilly & Associates
 Labrosse JJ (2002) μ C/OS. The real-time kernel. CRC Press
 Lewine DA (1991) POSIX programmer's guide. Orielly
 Lewis B, Berg DJ (1996) pThreads primer. Guide to multithreaded programming. Prentice Hall
 LWP and scheduling classes. Oracle documentation guide (2010)

Multithreaded programming guide. Oracle (2014)
Robbins D (2016) POSIX threads explained. IBM developer works
Simon DE (1999) An embedded software primer. Addison-Wesley
Ultimate guide to Real-time Operating Systems (RTOS), Blackberry, Qnx
Understanding basic multithreading concepts. Oracle documentation guide (2010)

Chapter 8

Networked Embedded Systems (NES)



Abstract Most of the embedded systems are not stand alone. They are distributed and networked to execute a common task. In such systems, the same real-time constraints have to be applied to the networking protocols, so that data is transmitted within the task's deadline. Network will also become a resource and has to be scheduled. Some characteristics of NES are low data rates, small size of data packets, real-time capabilities, deterministic data transfer, support various communication media, safety critical, etc. No two network designs of NES will be the same. Network architecture includes selection of appropriate communication protocol and communication medium. The node design goes through interfacing to the physical layer and communicates with peer nodes through layered network software. Allocation of priorities for the messages and simulating network performance for required response times are part of network architecture. Most of the NES can be classified into automotive segment, industrial segment, home automation, and wireless sensor networks. Any application can be broadly placed into one of these segments where the characteristics match. Basic assumption of automotive NES is that the nodes communicating each other are contained in close areas like in automobiles, trucks, helicopters, etc. The systems are designed around time-triggered protocols (TTP) based on time division multiple access (TDMA). In this protocol, frames are transmitted at predetermined points of time. Also use event triggered protocols where messages are transmitted to signal the occurrence of significant events or a combination of time triggered and event triggered. Sections 8.4, 8.5 and 8.6 discusses these in detail. Any industry is a collection of independent machinery to manufacture or process a part of the system. Each such system is automated. Such automated systems communicate each other and linked to upper levels for overall control. In automation industry, communication is established at different levels with different requirements. At field level, requirements are in real time with short messages, whereas the communication at supervisory and enterprise level is non-real-time but large data. Evolution of fieldbus technology (ControlNet, PROFIBUS (DP, PA) and Real-Time Ethernet (RTE)) has provided solutions for this; Sect. 8.7 discusses these protocols in detail. While designing large commercial complexes, offices, institutes, etc., several factors like energy savings, heating, ventilation, air condition control (HVAC) safety, surveillance, evacuation, and so on have to be considered and optimized through home automation systems. Seldom, we may require hard real-time requirements.

Typically, the communication is event driven (aperiodic). The timing requirements are much more relaxed. Similar to industrial fieldbus systems, there are a number of bodies involved in the standardization of technologies for building automation, including the field area networks; Sect. 8.8 discusses in detail. Some of the major characteristics of WSNs are self-organizing, ad hoc networks, self-containment, lack of prearranged network topology and the ability to self-heal and self-powered. The communication protocols, reconfiguration mechanisms, data rates are extremely dynamic and distributed among the nodes. Reliability of message delivery is maintained through space, time, and frequency diversity mechanisms because the nodes are in harsh environments with limited resources; Sect. 8.9 discusses in detail.

Keywords Time division multiple access (TDMA) · Time-triggered protocols (TTP) · Media access control · Local Area networks · Wireless sensor networks (WSN) · Event triggered protocols · Controller area network (CAN) · Fieldbus · ZigBee · Localization · Triangulation

8.1 Introduction

We have studied in the last two chapters, real-time concepts and the techniques and algorithms for implementing real-time requirements. We have also studied real-time operating systems that provide real-time extensions to the operating systems.

But most of the embedded systems are not stand alone. They are distributed and networked to execute a common task. In such systems, the same real-time constraints have to be applied to the networking protocols, so that data is transmitted within the task's deadline. Network will also be a resource and has to be scheduled. Normal resources like shared memory, disk access have deterministic times of execution. Certain network responses are not deterministic like Ethernet. They are not useful in designing of real-time systems.

We will study a class of networks having real-time responses. Let us coin the term as “Real-time networks,” which becomes backbone for networking of embedded systems. We will study their architectures starting from their hardware, topology, network protocols, and interconnectivity aspects. We will study how this class of networks provides interconnectivity across heterogeneous embedded systems.

8.2 Characteristics

The type of protocols used to interconnect embedded system nodes will impact whether the communication across the nodes goes in a deterministic way. For instance, protocols based on random media access control (MAC) such as carrier-sense multiple accesses–collision detection (CSMA/CD) are non-deterministic.

Meaning, under heavy traffic conditions, there will be heavy collisions and due to several re-tries, the response will be poor and becomes non-deterministic.

Due to the nature of communication requirements imposed by applications, networks like field area networks tend to have low data rates, small size of data packets, and typically require real-time capabilities that mandate determinism of data transferred. These are totally distinctive characteristics from conventional Local Area networks (LANs).

Design methods for NES fall into the general category of system-level design. They include three aspects, namely, node design, network architecture design, and timing analysis of the whole system.

Networked Embedded Systems are a collection of processing nodes, which are spatially distributed and have varied functionality. They are interconnected by means of wired or wireless media and associated communication protocols. The systems are not only physically distributed but also the functionality is distributed across the nodes.

As the designs are becoming compact due to advances in VLSI, the field devices like sensors and actuators are becoming intelligent and need communication with peer devices and with upper levels. Most of the applications can be classified into automotive electronics, industrial automation, and home automation. All these applications have diverse traits as listed below.

- **Low data rates:** The quantum of data generated and transmitted is very less compared with LANs. There is no transmission of files or images or video data. Most real-time data are samples of sensed data, pre-processed data, commands, and controls.
- **Small size of data packets:** From the applications given above, the quantum of data per message is very short. Some typical data are few data samples collected, a simple message or a command. The size of message normally will be 1 to 256 bytes.
- **Real-time capabilities:** All the data transmitted in NES should reach destination before the deadline. This is the reason why the data frames are very short. Not like images and file transfers in LANs.
- **Deterministic data transfer:** Transfer of data has to be guaranteed by a specific time. The time can vary for different types of data based on the application context.
- **Support various communication media:** Data should be able to be transmitted on different physical media like twisted pair lines, cables, wireless or optical, etc.
- **Safety critical:** Certain applications are highly critical in terms of failure, fault tolerant, fail-safe. The physical and network-level protocols must support needed features at data transmission level itself but not only at upper layers.

8.2.1 Design Aspects

From the classified applications as above, no two network designs of NES will be same. The message structures, data types, data rates, message priorities and message

delivery deadlines, fault tolerance make the designs complex as well as real-time scheduling aspects we have studied in Chap. 6.

The **Network architecture** includes selection of appropriate communication protocol and communication medium. The topology of the network is a part of architectural design. A complete NES may have to be segmented into different regions where each segment will have different architectural implementations.

Node design: Once the topology is decided, the node design goes through interfacing to network (physical layer) communication mechanism with peer nodes and any interface with other networks of different protocols or upper layers through gateways and bridges, etc.

Priorities: Allocation of priorities for the messages originated from communication nodes.

Timing analysis: Simulating the performance of the network at segment level and overall network, based on the expected traffic rate and required response times. Estimating worst case and best case execution times.

8.3 Broad Segments of NES

In industry, most of the NES can be classified into automotive segment, industrial segment and home automation and wireless sensor networks. Any application can be broadly placed into one of the segments, where the characteristics match. In this section, we will briefly explain the characteristics of each segment and get into details.

8.3.1 *Automotive NES*

Modern automobiles, transport vehicles, military tanks, trucks, air planes, helicopters, etc. come into this segment. Each system in this segment will have numerous nodes, which are distributed and interconnected. Each node is intelligent enough to do local processing and closely communicate with its peers.

Some examples of such nodes in a modern car are electronic engine control, anti-locking break system (ABS), active suspension, control of the driving torque through traction control, electric power steering, engine control, telematics, environment control, etc. These are getting extended towards driverless auto-guided autonomous vehicles...

Old systems are used to get mechanical control by hydraulic and pneumatic systems. Modern concept is fly-by-wire, drive-by-wire, steer-by-wire, brake-by-wire, or throttle-by-wire and ultimately X-by-wire aiming to replace mechanical, hydraulic, and pneumatic systems by electrical/electronic systems.

These systems have one class of products like railway signaling, navigation systems, autonomous vehicles, etc., which need failure rates of the order of 10^{-9} to 10^{-12} per hour/system depending on the criticality.

The systems are designed around time-triggered protocols (TTP) based on time division multiple access (TDMA). In this protocol, frames are transmitted at predetermined points of time.

Also event-triggered protocols are used, where messages are transmitted to signal the occurrence of significant events or a combination of time-triggered and event-triggered.

Automotive applications are classified into three classes: A, B, and C based on increasing criticality on real-time constraints and other safety and performance aspects. Class A is low-speed network, with speeds of about 10 Kb/s, which is used for non-critical applications like passenger comfort and other cosmetic features. Class B is medium speed (between 10 and 125 Kb/s) for data transfer across nodes, which is not critical. Some typical examples are information on emission data, environment data like internal temperature, and other data from instrumentation. Class C is high-speed network with greater than 125 Kb/s for real-time control of traction, ABS, safety bags, etc.

8.3.2 NES in Industrial Automation

Any industrial sector, viz, a process industry or manufacturing industry is no more mechanical and manually operated. Every technological advance starting from digital controls to machine intelligence has found place in industry for high productivity, quality and cost control.

Any industry is a collection of independent machinery to manufacture or process a part of the system. Each such system is automated. Such automated systems communicate each other and linked to upper levels for overall control. The concept of distributed digital control (DDC) is adapted. When it comes to unit level, the controlling or supervisor system will be different from other units. Most of them will be heterogeneous. Each unit controller is an embedded system and their connectivity is NES with heterogeneous units.

The architecture of NES for industrial automation is so challenging with different layers of communication protocols. Some such protocols are manufacturing automation protocol (MAP), Technical Office Protocol (TOP), etc. at upper level.

Sensors have become intelligent and as they are installed in the field, networking to communicate with peer sensors and to the controllers installed in control room have become essential. Evolution of fieldbus technology (ControlNet, PROFIBUS (DP, PA), and Real-Time Ethernet (RTE)) has provided solutions for this.

In automation applications, data need not be processed through all OSI layers but needs only 1, 2, and 7. We will study in depth these layers in the respective section.

The use of standard components such as protocol stacks, Ethernet controllers, bridges, etc. allows to mitigate the ownership and maintenance cost. In automation industry, communication is established at different levels with different requirements. At field level, requirements are in real time with short messages, whereas the communication at supervisory and enterprise level is non-real-time but large data. The technologies of communication vary across the levels. At high levels, conventional internet technology protocols are used for integration across business logistic levels. Seamless integration between process automation to upper levels is done through bridges and gateways.

Multiple approaches to integrate with Ethernet technology are adapted in Industrial automation. Three different approaches include.

- Keep the TCP/UDP/IP protocol layers unchanged.
- Bypass TCP/UDP/IP protocols suite. Access Ethernet functionality directly.
- The Ethernet mechanism and infrastructure are modified.

Major focus of NES in industrial automation.

- Operational security requirements to avoid disasters. viz, to plant operators and to the plant itself.
- System/plant availability for 24 h with minimal breakdown and provide
- Security measures to be taken not only at field level but also at the corporate and control network levels.

8.3.3 NES in Building Automation

While designing large commercial complexes, offices, institutes, etc., several factors like energy savings, heating, ventilation, air condition controls (HVAC) safety, surveillance, evacuation, and so on have to be considered and optimized through automation systems. Seldom, we may require hard-real time requirements. Typically, the communication is event driven (aperiodic). The timing requirements are much more relaxed. As with industrial fieldbus systems, there are a number of bodies involved in the standardization of technologies for building automation, including the field area networks.

8.3.4 Wireless Sensor Networks (WSN)

Some of the major characteristics of WSNs are self-organizing, ad hoc networks, self-containment, lack of prearranged network topology and the ability to self-heal and self-powered.

Certain applications need the above-mentioned traits strongly. Those applications cannot have a pre-configured topology like physical bus, star topologies. System

should self-configure appropriate topology logically and interconnect the nodes. The nodes are spread across spatially and have no power source. They are self-powered or limited self-powered. The node's life time may be short. In such cases, the connectivity among the nodes has to be dynamically reconfigured because of some node failures. The nodes must have self-healing capabilities to keep themselves live to maximum extent. The communication protocols, reconfiguration mechanisms, data rates are extremely dynamic and distributed among the nodes. Reliability of message delivery is maintained through space, time, and frequency diversity mechanisms because the nodes are in harsh environments with limited resources.

The applications of WSNs are unimaginable. They provide ultimate and exclusive solution, where the nodes have to be spatially distributed in the field and there is no power source, and no physical connectivity across nodes is possible.

8.4 Automotive NES

Typical applications and traits of automotive NES are explained in the earlier paragraphs. Let us study how these are implemented using different protocols.

8.4.1 *Event-Triggered Protocols*

Main properties of event-triggered protocols are listed below.

- Basic assumption of this protocol is that the nodes communicating each other are contained in close area (the transmission rate depends upon the closeness) such that there is negligible propagation delay from any node to the farthest node to it.
- When an event occurs in a node, it is transmitted to the peers as a message or a frame containing a set of messages.
- All the events are considered independent and asynchronous. System possesses the ability to take into account, as quickly as possible, any asynchronous events such as an alarm.
- The communication protocol defines a policy by which a node is granted access to the media (bus). All other nodes listen only during the transmission. Hence, collisions are avoided.
- The protocol assigns priority to each message, so that the bus grants access to high-priority messages. Assign a priority to each frame and give the bus access to the highest priority frame.
- Event-triggered protocols are very efficient in message transmission because bus is granted only when a node has messages to be sent.

8.4.2 Time Triggered Protocols (TT)

- Frames are transmitted at predetermined time slots.
- Each node gets a time slot that is scheduled by the protocol. This slot is periodic. Hence, TT is useful when nodes generate data periodically to be communicated. Each frame is scheduled for transmission at one predefined interval of time (slot). This method of access to the shared bus is called Time Division Multiple Access (TDMA).
- Nodes have to become active during their slot irrespective of whether they have data to be transmitted or not.
- Other nodes can find a faulty node when a node does not respond in its respective slot.
- TT is inefficient in utilizing the network bandwidth.
- The schedule is always periodic and not possible to schedule to some other.
- Unplanned addition of new transmitting nodes changes complete schedules.
- Well-suited protocol for periodic data communication, fault finding, composability, error detection, and error containment.
- Preferred for safety-critical systems.

8.4.3 Example TT Protocols

Few time-triggered protocols in use are given below.

- **TTP/C**: An architecture for the design and implementation of dependable distributed real-time systems.
- **TTP/A**: used for low-cost networks of sensors and actuators.
- **TTCAN**: Layered on top of an event-triggered Controller Area network (CAN) protocol.
- **TT Ethernet**: Unifies real-time and non-real-time traffic into a single-coherent communication architecture

8.4.4 Fundamental Services of TT Protocol

8.4.4.1 Clock Synchronization

All the nodes must maintain same absolute time. A service has to synchronize the timers of each node to same value (Please remember, all the nodes are in vicinity with negligible propagation delay.) Granularity determines the minimum interval between two adjacent ticks of the global time (time is maintained in microseconds or milliseconds...).

8.4.4.2 Periodic Exchange of State Messages

The service provides periodic exchange of messages carrying the state of the node to all other nodes. TDMA mechanism divides the time frame T into multiple slots $i = 1 \dots n$ and allocates each slot to one node, so that i th node gets t_i to t_{i+1} . As all the nodes maintain same time absolutely, no contention occurs across nodes. The communication activity of every node is managed by the communication module of the node when triggered in its slot.

8.4.4.3 Fault Isolation Mechanisms

System architecture provides mechanism for detection of a faulty node and isolating it and resuming communication in the remaining nodes. Services partition the system into independent regions when a fault is detected. When a fault occurs and it is detected, the faulty region is isolated and communication goes in the remaining nodes. A fault-containment region (FCR) is a subsystem that operates correctly regardless of any arbitrary logical or electrical fault outside the region.

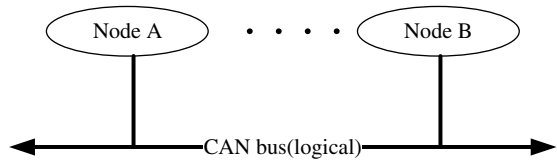
A message failure occurs if the data contained in a message are incorrect. A message timing failure means that the message send or receive instants are not in agreement with the specification. Error containment involves an independent component for error detection and mediation of a component's access to the shared network. Diagnostic services provide replacement of defective node if a failure is permanent.

8.5 CAN (Controller Area Network)

Controller Area Network (CAN) is a bus-based architecture designed for communication across multiple nodes in a contained area. This can be a car, a complex machine with multiple processors, all contained in an enclosed area. It is particularly used for automotive applications. CAN is a two-wire half-duplex high-speed serial network technology.

CAN is invented by Bosch and subsequently standardized into the ISO11898-1 standard (The ZigBeealliance) CAN defines the data link and physical layer of the Open Systems Interconnection (OSI) model, providing a low-level networking solution for high-speed in-vehicle communications. In general, in any automotive vehicle, a minimum of 10–12 processing units have to work independently and communicate with others. Typical examples are engine control, transmission control, active suspension, anti-lock brakes, lighting, power steering, power locks, airbags, etc. Some of them are to be interconnected to communicate among them and coordinate the job. Due to this, the cabling will be messy, very long, becomes costly and reliability reduces. So CAN provides bus architectures, where all the controllers communicate over a single media as shown in Fig. 8.1. In particular, CAN was developed to reduce node-to-node wiring among nodes and all the nodes communicate on a single bus.

Fig. 8.1 Logical configuration of CAN bus



As in Fig. 8.1, a CAN network consists of various nodes. Each node has a host controller which is responsible for the functioning of the respective node. Each node has a CAN controller and CAN transceiver. CAN controller converts the messages to be transmitted to the format of CAN protocol and transmits via CAN transceiver over the CAN bus. CAN does not follow the master–slave architecture. Every node constantly reads the data on the bus and accesses the data marked to it. When a node is ready to send data, it checks availability of the bus and writes a CAN frame onto the network. The arbitration protocol explained in the below sections allows only a high-priority node to transmit (dominates) and other nodes to recede.

8.5.1 CAN Frame

Before understanding CAN frame, let us understand dominant and recessive bits: If at least one node is transmitting the 0 bit level, then the bus is in that state regardless of other nodes have transmitted 1 bit level. 0 is termed the dominant bit value while 1 is the recessive bit value.

A CAN frame is labeled by an identifier, transmitted within the frame whose numerical value determines the frame priority. Non-return-to-zero (NRZ) bit representation is used with a bit stuffing of length 5.

The frame structure of CAN 2.0 is detailed below: see Fig. 8.2.

- SOF—(1 bit) Start of Frame. The frame starts from this point.
- Identifier—(11 bits). The value decides the priority of the message. Lower the binary value (00...0) higher is the priority.
- RTR—(1 bit) 1 = Remote Transmission Request. It is dominant when information is required from another node. Each node receives the request, but only that node whose identifier matches that of the message is the required node. 0 = data frame.
- IDE—(1 bit) Single Identification Extension. If it is dominant, it means a standard CAN identifier with no extension is being transmitted.
- R0—(1 bit) reserved bit.

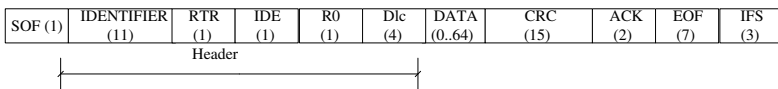


Fig. 8.2 CAN (2.0) Standard frame (Courtesy ISO11898-1) (The ZigBeealliance)

- **DLC**—Data Length Code. It defines the length of the data being sent. It is 4 bit to 64 bit of data, which can be transmitted.
- **Data**—(0–64 bits): data to be transmitted.
- **CRC**—(15 bit) Cyclic Redundancy Check. Contains the checksum of the preceding application data transmitted in the frame. Used for error detection at the receiver end.
- **ACK**—(2 bit) Acknowledge. It is dominant if an accurate message is received. Enables the sender to know that at least one station, but not necessarily the intended recipient, has received the frame correctly.
- **EOF**—(7 bit) End of the Frame. It marks the end of CAN frame and disables bit stuffing.
- **IFS**—(3 bits) Inter Frame Space. The time required between two frames. During this time, the controller moves the received frame to its proper position.

8.5.2 CAN Messages

A node can send any of the following types of messages.

- **Data Frame:** It consists of an arbitrary field, data field, CRC field and the acknowledge fields.
- **Remote Frame:** It requests for transmission of data from another node. Here, the RTR bit is recessive.
- **Error Frame:** It is transmitted when an error is detected.
- **Overload Frame:** It is used to provide a delay between messages. It is transmitted when the nodes become too busy.
- **Valid Frame:** A message is valid if the EOF field is recessive. Else, the message is transmitted again.

8.5.3 CAN Physical Layer

CAN uses bus topology as shown in Fig. 8.1. It is a broadcast bus, so that there is one sender and others are listeners. The contention mechanism of the bus will be discussed in media access section.

8.5.3.1 Network Topology

The most common way of transmission is two-wire balanced signaling scheme. It is also sometimes known as “high-speed CAN.” ISO 11898-3 defines another two-wire balanced signaling scheme for lower bus speeds. It is fault tolerant, so the signaling can continue even if one bus wire is cut. Single wire transmission is also used with common ground as return path.

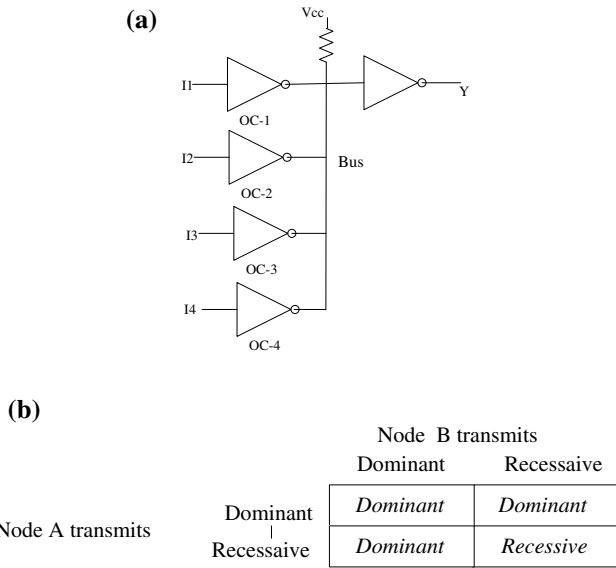


Fig. 8.3 **a** Open collector bus. **b** Bus state when two nodes transmit dominant and recessive data

Maximum transmission rate is 1 Mbit/s. At this rate, end-to-end cable length is 40 m to avoid propagation delays. The node distances can be increased at lower bit rates such as 100 m at 500 kbps and 500 m at 125 kbps.

8.5.3.2 Bit Encoding and Synchronization

Bus Levels

Binary values on the bus in CAN protocol are termed as dominant and recessive bits. CAN defines the logic “0” as dominant bit and logic “1” as recessive bit. Please see Fig. 8.3 for the open collector inverters forming a bus. Any one of the devices oc1 to oc4 is given high (logic 1) the bus is pulled down and bus will be at logic 0. This logic 0 is called dominant bit. When all the inputs are at logic 0, no device pulls the bus and the bus state is called “recessive” logic 1. In CAN, the devices have very similar bus states. When a device puts dominant bit, all other devices listen as dominant bit. If device A places dominant and device B places recessive bit, B listens dominant and infers this as conflict. In the CAN system, dominant bit always overwrites the recessive bit.

CAN bus uses non-return to zero (NRZ) format a of bit transmission. Hence, it has to stuff a 0 after five consecutive ones and a 1 after consecutive zeros, see Fig. 8.4.

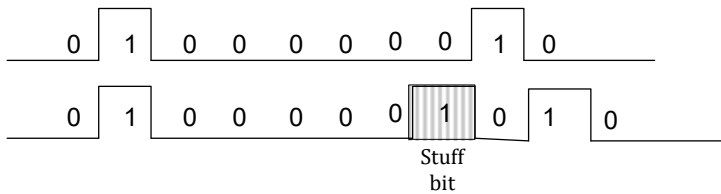


Fig. 8.4 Bit stuffing and NRZ data transmission

8.5.4 CAN Media Access and Arbitration

CAN uses arbitration similar to CSMA/CA (carrier-sense multiple access collision avoidance).

CAN arbitration cycle is accomplished in predictable and constant time. Arbitration starts after a frame is transmitted and start of new frame (SOF). All nodes must wait for this state. CAN does not keep the node address who originates a message and who receives it. Each message is identified by an 11 bit ID, which is the message’s identifier field, see Fig. 8.5. The identifier with the lowest value starting with 0000 has the highest priority. The nodes will understand which message IDs they are interested and have to receive. Bus is considered idle, i.e., free for access, after end of the completely transmitted message followed by the Intermission Field. Node, that transmits a message with the lowest message ID (with dominant bits) will be of high priority. The node wins the arbitration and continues to transmit. Other competing nodes with lower priority (more recessive bits) switch to listening mode. Nodes that lost arbitration will start a new arbitration when the bus becomes free. Thus, CAN provides a non-destructive bus arbitration mechanism also called collision avoidance.

PI refer Fig. 8.6 for the arbitration cycle.
Each node compares the level sensed on the bus to the value of the bit, which is being written out (Node1 and Node 2 send recessive).

- If the node is transmitting a recessive value, whereas the level on the bus is dominant, the node understands that it has lost the contention and withdraws immediately (This happens in the example at 5th bit, where Node 1 sends a dominant bit and Node 2 sends a recessive bit. Node 1 wins and Node 2 backs off).
- A node that has lost the arbitration will wait until the bus becomes free (Node 2 in this example.)

Fig. 8.5 Arbitration fields in CAN message

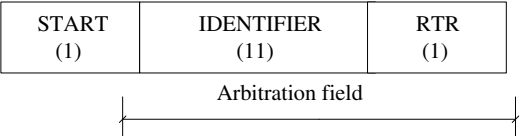
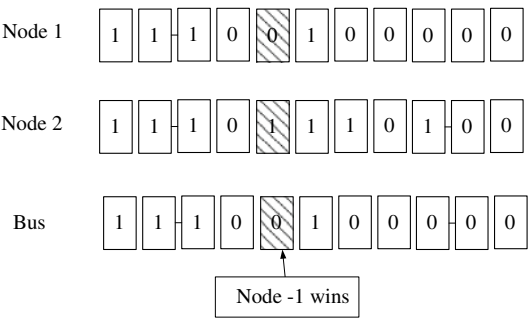


Fig. 8.6 Arbitration of CAN messages from two nodes 1 and 2



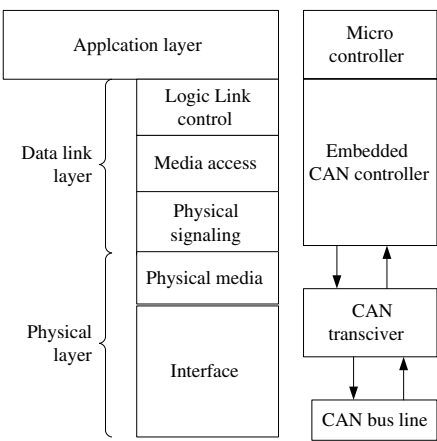
- Sending node monitors the bus while transmitting.
- No two nodes can be transmitting messages concerning the same object at the same time.
- Propagation delay limits the bus bandwidth (40 m at 1 Mb and 250 m at 250 kbps.)

8.5.5 CAN Protocol Stack

CAN layered architecture consists of three layers. The type of messages to be transmitted and needed functionality for any NES does not need all OSI seven layers, see Fig. 8.7. It consists of

- The *physical layer* that represents the actual hardware. The way the data are encoded, bit transmission, signal levels, etc.
- The *data link layer* that defines the rules for bus access, frame encoding and decoding standards, error checking, signaling, and fault confinement.

Fig. 8.7 CAN protocol stack (Courtesy ISO11898-1)



- The *logically link control layer* (LLC) provides well-defined communication services, which can be used by the application layer.
- The *Application Layer* interacts with the operating system or the application of the CAN device.

8.5.6 CAN Information Exchange

Producer of information encodes the data and transmits the related frame on the bus going through the arbitration protocols. Because of the intrinsic broadcast nature of the bus, the frame is propagated all over the network, and every node reads its content in a local receive buffer, i.e., a node does not transmit a frame to a specific node with a destination address. Only the messages are identified by the message ID. Frame acceptance filtering (FAF) function in each node determines whether or not the information is relevant to the node itself. In the former case, the frame is passed to the upper communication layers. On the contrary, the frame is simply ignored and discarded, see Fig. 8.8.

This is message-based protocol where the frame contains unique message IDs and not any node addresses. Due to this, other nodes can be added without any reconfiguration of the network since the nodes connected over the bus have no identifying information like node address. Hence, there is no change needed in the software and hardware of any of the units connected on the bus.

8.6 Time-Triggered CAN (TTCAN)

CAN protocol explained above is event triggered, which means that each node can compete to acquire bus and send messages whenever it needs to. Since different nodes have different time cycles for their need to send, this gives an effective scheduling on the bus. Bus is utilized effectively with priority messages are delivered first. No idle slots exist on the bus, unless no node has messages to be sent.

However, event-triggered protocols have certain drawbacks. If some nodes frequently send messages because of their inherent priority, the bus is hogged by these messages. Sometime-bound periodic messages of lower priority will never get

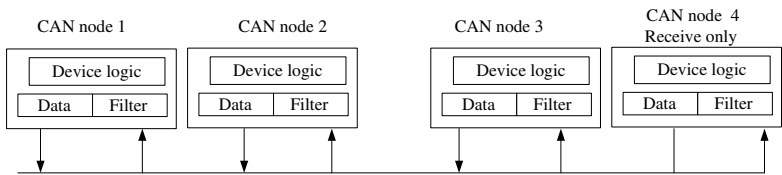


Fig. 8.8 Message based communication in CAN

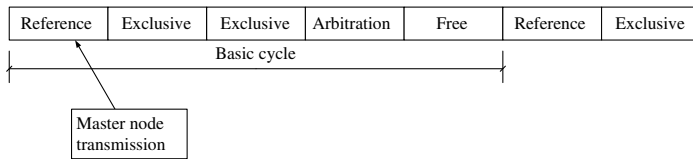


Fig. 8.9 Basic cycle in TTCAN

chance to transmit, and they surely miss their deadline. That is the reason why periodic scheduling is preferred for real-time systems. So TTCAN is designed as a standard over CAN protocol supporting messages to be sent as periodic time-triggered events. With this, the bus is utilized with certain time for periodic messages and certain time for event-driven messages. TTCAN is based on a centralized approach, where a special node called the time master (TM) keeps the whole network synchronized by regularly broadcasting a reference message (RM).

The way TTCAN protocol works as below, see Fig. 8.9.

- The timing (clock) on the bus is controlled by one master node.
- A collection of messages becomes one basic cycle.
- A set of basic cycles form schedule matrix.
- Each basic cycle starts with a reference message. The reference message sets the global time of the system. Each basic cycle consists of a number of transmission slots (also called as columns). The slots can be of three types.
- Reserved for one particular message. It is exclusively reserved to one predefined message, so that collisions cannot occur; they are used for safety-critical data and periodic data that have to be sent deterministically and with no jitters. Ex: Msg B in BC1 to BC4. Msg X in BC1 in the schedule matrix, see Fig. 8.10.
- Free for arbitration, all nodes can compete for transmission (Arbitrated) Arbitrating window is ruled by the standard CAN protocol. See third msg in BC1.
- Free window, not used but reserved for further expansion (free). See 4th msg in BC1.

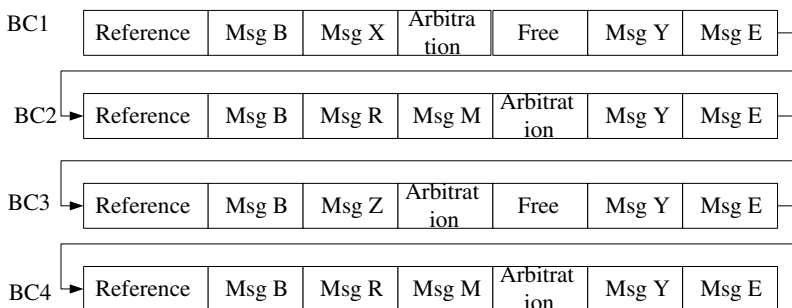


Fig. 8.10 Schedule matrixes TTCAN

- Since the messages have to keep their time slots. There is no retransmission of messages. The slots will have to wait till the next allocated time slot or an arbitration time slot.
- A transmission column has to be of the same size in every basic cycle. So the size is governed by the longest message that is to be sent in that column.
- The protocol also enables the master to stop functioning in TTCAN mode and switch to standard CAN mode. Master node will send a reference message to switch back to TTCAN mode.

8.7 NES in Industrial Automation

Networking of industrial equipment is the backbone for any automation system architecture. This provides a powerful means of data exchange, data controllability, and flexibility to connect, communicate and control various devices.

In previous generations, industry was using proprietary digital communication networks integrated into their manufactured devices itself. Inter-operability with other devices was difficult. Current generation systems are adapting to industrial networking with standard communication protocols between digital controllers, field devices, and various automation-related software tools and also to external systems. Industrial automation is becoming more complex with more number of automation devices on control floor. Today, the trend is toward Open Systems Interconnection (OSI) standards that permit to interconnect any automation devices irrespective of their manufacturer.

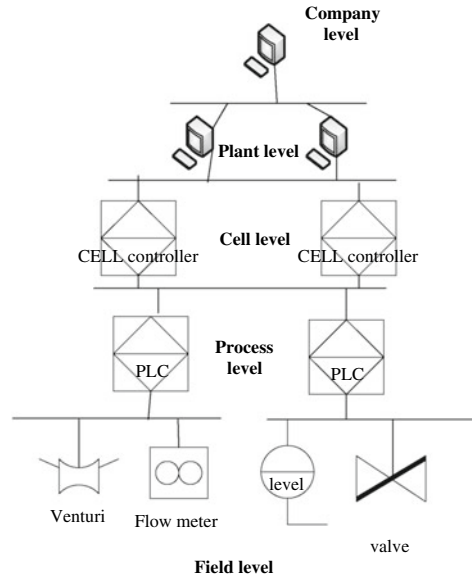
8.7.1 *Network Hierarchy*

The three significant control mechanisms used in industrial automation field include Programmable Logic Controllers (PLCs), Supervisory Control and Data Acquisition (SCADA), and Distributed Control System (DCS). All these control instruments deal with smart field instruments like sensors and actuators, supervisory controllers, process monitors, distributed I/O controllers, etc., see Fig. 8.11.

Different devices at a particular level have to handle with different requirements. So, it is obvious that no single communication network address all requirements needed by each level. Hence, different levels may use different networks based on the requirements such as data volume, data transmission, data security, etc. Based on the functionality, industrial communication networks are classified into different levels, which are discussed below.

Field-level devices are sensors like flow sensor, level sensor, temperature sensor, etc. Actuators are also field devices like valves, thermal heaters, motor control, flow controllers, etc. Most of these are smart with built in intelligence. They communicate

Fig. 8.11 Sensor to plant-level network hierarchy



among peers to share the sensed parameters and their health. They communicate to upper level to the process controllers like PLCs, CNCs and PID controllers.

The information transfer can be digital, analog or hybrid. The measured values may stay for longer periods or over a short period.

As the field devices are becoming smart through built-in intelligence, they have to communicate among peers. This needs distributed network among the field devices itself. Most of the pre-processing of tasks can now be done at the field device level itself. Different types of fieldbus standards have been in practice like HART, Device Net, ControlNet, Profibus, CAN Bus, and Foundation Field Bus. We will study characteristics of one of the popular fieldbuses subsequently.

Most of the plants will be physically spread as independent processes except very small machinery units. These smart field devices are installed in the field physically connected to the process to read the process parameters. Each controller controls the process. The process controller reads the data from field devices and controls the unit process. The controllers are mostly installed in control rooms. The bus at field level is limited to the process region. The tasks at controller level include configuring automation devices, loading of program data and process variables data, adjusting set variables,. At this level, the communication characteristics are low response times, high-speed data rates, short messages, machine synchronization, constant use of critical data, etc.

A set of such connected processes need to coordinate and communicate their status. This is normally called *cell-level* control. Cell-level control coordinates the unit processes for optimal processing and guides unit processes with proper commands and control. The level of communication at cell level will be higher and spread geographically.

Plant-level monitoring and supervision is done at plant level by networking all cell controllers. The majority of jobs at plant level is to monitor the plant process and manual intervention through operator commands. User interface and process visualization are important at this stage. The type of communication is not real time but data driven. This deals with large volumes of data that are neither in constant use or time critical. Ethernet LANs are commonly used as information-level networks for factory planning and management information exchange.

Company-level networking is to monitor multiple plants spread geographically and for administrative reasons. Networking at this stage is through internet.

This is generalized architecture that may not be followed in every industry. Let us see the characteristics of a network at each level. Field-level networking is done using Fieldbus standards, where the number of nodes is limited. Data rates vary from process to process. Critical processes need hard deadlines. The topology varies from star to bus network. Network across process level needs higher data rates with hard real-time responses. Bus networks are mostly preferred at this stage with deterministic response time. Networking at cell level and plane level is normally adapted using conventional local area networks or wireless.

8.7.2 *Fieldbus*

A Fieldbus is a digital, two-way, multi-drop communication link among intelligent measurement and control devices. Fieldbus is a group of protocols that are used in the industrial arena. The Fieldbus protocols have been standardized as IEC61158 (Harmening 2017).

8.7.3 *Fieldbus Topology*

Fieldbus works on a network that permits various physical topologies such as star, ring, branch, and daisy chain, see Fig. 8.12.

Fig. 8.12 Fieldbus topologies

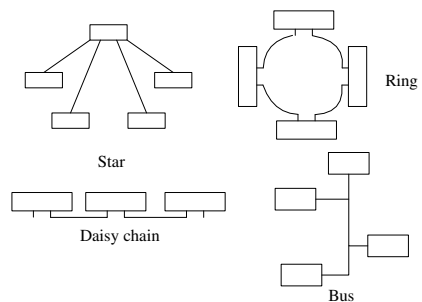
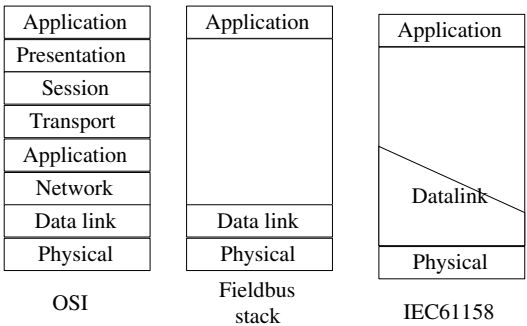


Fig. 8.13 Fieldbus-layered architecture (Courtesy IEC 61,158-2 ed.6.0 “Copyright © 2014 IEC Geneva, Switzerland. www.iec.ch”) (Harmening 2017)



Fieldbus is not a connection type, instead, a description is used to indicate a group of protocols. There are several protocols in the group such as ControlNet in the Allen Bradley family, Modbus, Profibus, EtherCAT, HART, CIP, and many more.

8.7.3.1 Layered Structure of Fieldbus

Numerous fieldbuses are commercially available before standardization, which cannot be listed here. International Electro technical Commission (IEC) framed IEC61158 standard with 29 participating countries. IEC finalized 15 Communication Profile Families. Popular ones are FF1, Profibus-PA, FF HSE, profinet, Modbus, profibus DP, HART, and World-Fip. Physical layer is all digital and the speeds 9KBPS to 32kbps except few which uses 802.3. Maximum distance is up to 1.5 km. Mostly, the number of devices per segment is 32. Datalink layer is mostly token passing media access. Few use master/slave.

Originally, fieldbus systems were not meant to be fully OSI compliant. Fully implemented session and presentation layers are not needed as the type of functionality is basically reliable and short message transfers. Still, certain functions from the layers 3–6 might be needed in reduced form. Rudimentary networking aspects could be required or specific coding rules for messages that are better suited for handling limited resources available in typical fieldbus nodes, see Fig. 8.13.

8.7.3.2 Media Access Strategies in NES

Fieldbus nodes are mostly connected topologically as star, bus, daisy chain, or ring. The node to node communication protocols vary for different topologies. In certain topologies, viz, bus topology, all the nodes use same media. Hence, only one can transmit data and all others have to be listeners. This is also called as broad cast mode. A standard protocol has to be followed to gain access to the media. This is part of datalink layer and is called Media Access Control (MAC). Fieldbus supports

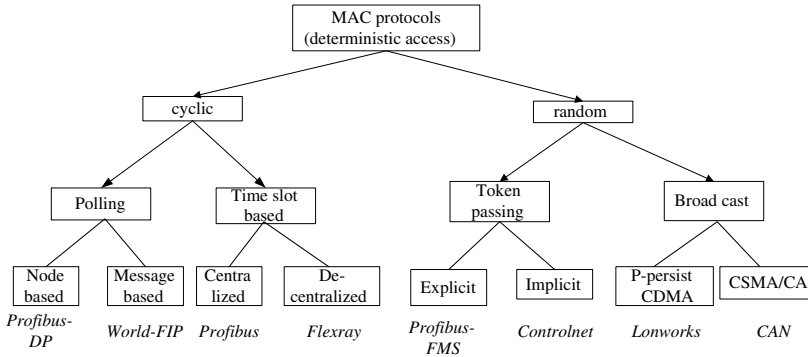


Fig. 8.14 Media access strategies in NES

different MAC strategies for media access by different standards and topologies used. We will cover few important MAC strategies, see Fig. 8.14.

Polling is a master–slave access scheme. A slave node is only allowed to send data when explicitly asked so by a central master. This strategy can be on physical star topology or on bus also. In the case of bus, the access mechanism will be logical. When the master fails, all slaves fail and so the network.

Token passing is the right to control the network. A token (in the form of message) is passed in a specific sequence among the nodes. The node holding the token becomes the bus master and can transmit messages. All others will receive only. Once the token holder has done its job or a time-out occurs, it passes to the scheduled successor.

Time-slot-based in which the available transmission time on the medium is divided into distinct slots, which are assigned to the individual nodes. All the nodes are synchronized with a global clock.

Random access means that a network node tries to access the communication medium whenever it wants to without limitations imposed for instance by any pre-computed access schedule.

8.7.3.3 Fieldbus Gateway to Internet

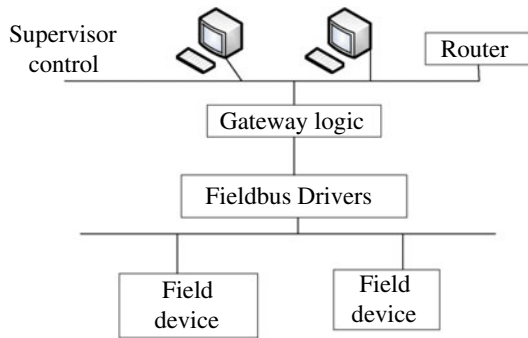
The gateway is a full member of the fieldbus on one side and can be accessed through IP-based mechanisms via internet, see Fig. 8.15.

The use of IP-based networks is a convenient means to remotely access fieldbus systems. In the gateway approach, the access point takes the role of a proxy representing the fieldbus and its data to the outside world. It fetches the data from the field devices using the usual fieldbus communication methods and is the communication partner addressed by the client.

There are several other possibilities to get Ethernet or Internet technologies into the domain currently occupied by fieldbus systems:

- Tunneling of a fieldbus protocol over UDP/TCP/IP.

Fig. 8.15 Fieldbus–Internet interface



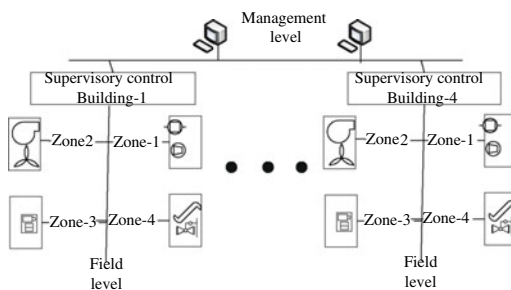
- Tunneling of TCP/IP over an existing fieldbus.
- Definition of new real-time-enabled protocols.
- Limitation of the free medium access in standard Ethernet.

Tunneling in connection with communication networks essentially means that data frames of one protocol are wrapped into the payload data of another protocol without any modification or translation. Thus, two possibilities exist: the fieldbus protocol can be encapsulated in the Internet Protocol on the higher level or a protocol from the IP suite can be passed over the fieldbus. Tunneling is well-known in the office world as a means to set up virtual private networks for secure communication of, e.g., remote offices. Although tunnels in this area normally operated on the data link layer, any PDU of any protocol layer can be tunneled over any other; there is no compelling reason to restrict tunneling to the data link layer.

8.8 NES in Building Automation

Building automation is a special case of process automation, see Fig. 8.16. However, this is not any hierarchical or distributed control system. The real-time requirements are too little. The data rates are also limited. The purpose of the automation is monitoring and control of building services. Major services are

Fig. 8.16 Typical integrated building automation system



- monitoring and control of illumination,
- monitoring and control for energy management,
- heating, ventilation, and air conditioning control (HVAC),
- surveillance and security of building, access control,
- transportation, viz, elevators, escalators, and conveyor belts,
- internal communication through CCTV, voice intercom, private exchanges,
- supply and treatment of drinking water.
- disposal of wastewater,
- power distribution and standby power,
- fire monitoring and control,
- evacuation control, etc.

Integrated building automation systems reduces energy costs and provide safety to all the dwelling persons. Typically, the automation costs are compensated by the energy savings and other savings associated with preemptive maintenance and quick detection of issues.

8.8.1 BACNET—Building Automation and Control Network

Building automation has to be done around a standard protocol, so that equipment supporting the protocol can be installed and maintained easily. The control and monitoring network can be implemented with built-in functionality provided in the compatible devices. Developing from scratch will not be to a particular standard and cannot be supported. Interoperability can be gained for sharing of data across multiple implementations by adapting to standards.

BACnet is approved as ISO16484-5 standard in 2003 (introduction to the LonWorks® Platform, Echelon Corp) The physical layer is RS-485-based 78.4 kbps multi-drop protocol. MAC layer is master-slave/token passing and BACnet proprietary standard at 1 MBps. The network layer provides an unacknowledged connectionless datagram service, see Fig. 8.17.

The data model follows an object-oriented approach. The objects are manipulated via appropriate services, following a client-server model.

BACnet devices can share a common LAN selected from the approved BACnet choices. Because of the way BACnet messages are packaged, other types of

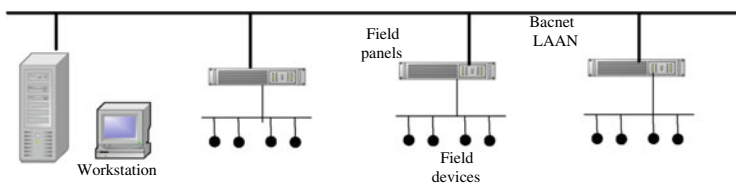


Fig. 8.17 Bactnet—typical implementation

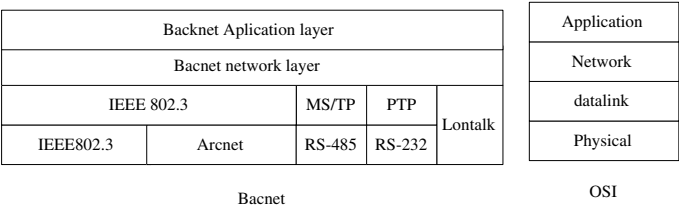


Fig. 8.18 BACnet protocol layers

computers, for example, office PCs and servers, can coexist on the same LAN without interference, see Fig. 8.18.

BACnet provides a standard way of representing the functionality of any device, such as analog and binary inputs and outputs, as “objects.” Each object has a set of “properties” that further characterize it. As an example, each analog input is represented by an analog input object. The object has a set of properties like present value, sensor type, location, alarm limits, and so on. Some of these properties are mandatory while others are optional. One of the object’s most important property is the object identifier, a value that allows BACnet to unambiguously access it.

BACnet defines several message types, or “services,” that are divided into five classes. For example, one class contains messages for accessing and manipulating the properties of the objects described above. A common one is the “Read Property” service request. This message causes the server to locate the requested property of the requested object and send its value to the client.

BACnet is, thus, a protocol defined for reliable and short messaging using object-based messaging over the LAN. Based on the OSI model, BACnet is extended to many other domains outside of the building automation such as management applications and embedded control.

8.8.2 *LON Works*

LON works has widespread use for building automation all over world. This is designed by Echelon Corp as a universal standard for control networks (ISO 11898-1:2015) Currently standardized as EIA 709.1 Lon works names this network as “control network,” which connects peer-to-peer devices like sensors, actuators, and other field devices.

Control network can be configured as master–slave or peer-to-peer network. Physical connection of devices is through a channel. The channel can be of different configurations based on the speed and distance. The LONTALK protocol is the heart of Lonworks communication (The ZigBeealliance). The protocol is broadcast bus based, where one device transmits and others are listeners. The media access is basically CSMA (carrier-sense multiple access). In case of CSMA/CD, the message transmission gets delayed during high traffics because of back off due to collisions.

Lontalk uses *predictive-persistent CSMA* protocol, which dynamically manages the bandwidth during high traffic conditions.

The protocol is given below briefly.

1. If the medium is idle, transmit with probability p , and delay for one unit of time with probability $(1 - p)$.
2. If the medium is busy, continue to listen until medium becomes idle, then go to step 1.
3. If transmission is delayed by one unit of time, continue with step 1.

The devices are grouped in the total network (domain). The groups are called subnets. There can be 255 subnets and each subnet can contain 127 nodes. The protocol defines network variables (NV). An NV is a data (from sensors or any parameter generated by a node), which is of interest globally. The NV is defined globally and can be accessed by a node by directly addressing the NV irrespective of the node where it is originated. It is variable-based addressing rather node-based addressing.

A set of NVs can be logically connected by a “*binding*” process by the application. As an example, a switch is an NV-1 with states *on* and *off*. Similarly, a lamp is an NV-2 with light *on* or *off*. Both the NVs can be logically connected at the time of configuration such that when NV-1 is in ON state, NV-2 will be in ON state and lamp glows. This is a virtual connection without a wire in a building, see Fig. 8.19.

A channel is a physical communication medium connecting the devices. The medium is twisted pair or powerlines. On twisted pair, bus 78 kbps to 1.25 MBPs with 64 devices on the bus maximum distances being 100–500 m. On power line, data rates are 5 KBPS with 500 nodes.

Specialties of Lonworks include multiple media support including power lines, predictable response time’s independent of network size and arbitrary connectivity among nodes and interoperability with multiple physical media and speeds.

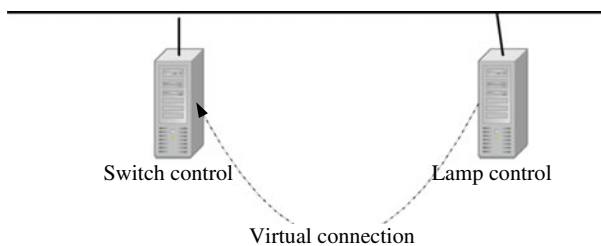


Fig. 8.19 Virtual wire through NV binding

8.8.3 ZigBee

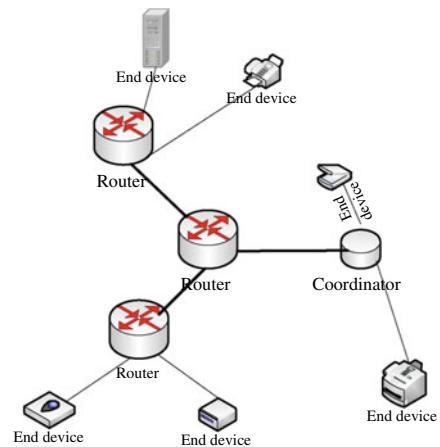
Zigbee is based on the IEEE's 802.15.4 personal-area network (PAN) standard. The specification is more than a decade old and is considered an alternative to Wi-Fi and Bluetooth for some applications. This is published in 2003 under PAN (Personal Area Networks) technology.

Zigbee devices can be divided into two categories, based on the topology and media access control used by the device. Full-Function Devices (FFDs) communicate directly with any other device in the network. FFDs communicate among them on peer to peer basis. In contrast, reduced function devices (RFDs) can communicate only with FFDs. They have no peer-to-peer capability, see Fig. 8.20.

The 802.15.4 standard allows networks to form either a single-hop star topology or a multi-hop peer-to-peer topology. The former is most appropriate in networks with few FFDs. The latter is more robust to node failure when many FFDs are available. Though 802.15.4 defines the allowed topologies, it does not define the layers that actually support them. Routing within these topologies is the responsibility of layers above those defined by IEEE.

One FFD acts as a coordinator node. Media access is coordinated by this node.. This node sends periodically beacons. The interval between these beacons is a multiple of 15.38 ms. It can be up to 252 s. Two beacons form a superframe. The superframe is partitioned into 16 equally-sized timeslots. Members of the PAN may request *guaranteed time slots* (GTSs) in the contention-free period at the end of the superframe, see Fig. 8.21. All other slots form the contention access period, which is accessed using a CSMA-CA scheme. A coordinator node has to be computationally powerful to control the media access, it may not be practical to deploy one in all networks to have a coordinator all the time. When such coordinator is not controlling, media is accessed using CSMA-CA protocol. The media is always subjected to contention.

Fig. 8.20 Zigbee network



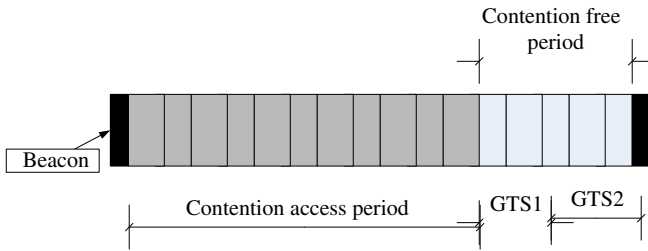


Fig. 8.21 Superframes

The standard, thus, defines two kinds of PANs. They are beacon enabled and non-beacon enabled.

In beacon enabled, the PAN coordinator sends superframes periodically. Superframe is divided into slots. PAN members may request guaranteed time slots (GTSs) in the contention-free period at the end of the superframe. PAN members can use the slots using CSMA-CA.

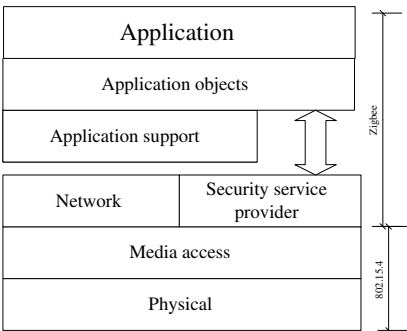
In non-beacon enabled, all PAN members can communicate at any time using CSMA-CA.

ZigBee refines 802.15.4's two device categories into three hierarchical device roles. *Coordinators* are 802.15.4 FFDs that act as 802.15.4 coordinator nodes and maintain ZigBee-specific information about the PAN. *Routers* are FFDs that participate in ZigBee's routing protocols. *End devices* are analogous to RFDs: they must communicate with each other by way of an intermediary coordinator or router.

ZigBee maintains 802.15.4's star topology but divides the peer-to-peer topology into clusters and mesh topologies. Cluster topologies create links between routers and coordinators using a beaconing scheme. Mesh topologies maintain a relatively fixed routing infrastructure, using a simplified version of the Ad hoc On-demand Distance Vector routing scheme proposed for ad hoc networks [RFC3561]. Cluster topologies have the advantage that coordinators and routers may sleep periodically to extend battery life, whereas their counterparts in mesh networks must maintain constant availability. However, the routing delays in cluster topologies are unpredictable and often much higher than those in mesh networks.

ZigBee also provides Bluetooth-like device and service discovery. Manufacturers describe a device's role and capabilities using a static *device object*, see Fig. 8.22. Device objects contain descriptions of the device's type, power profile, and communication endpoints as well as optional "*complex*" fields describing device- or manufacturer-specific information. The service provided by each device is described by using an application object. The object encapsulates its attributes and capabilities. ZigBee devices can perform queries to discover other devices. Through these queries, it identifies other objects to perform services that match each other. ZigBee also supports binding of some types of devices. As an example, a ZigBee-enabled light switch and automated light socket can be logically bound, so that when the light

Fig. 8.22 Zigbee network stack



switch object gets to ON state, the light switch state changes. The link is maintained between devices based on their application profiles.

Zigbee Home Automation is a global standard that makes every home smarter. This enables consumers to manage energy consumption, home security and saves money. Zigbee-enabled sensors, gadgets for home, etc. are in the market for easy installation. One can develop customized products as per the standard using available tools. For any system being used in home automation, the main goal is to reduce human effort by operating various appliances remotely. This is where the various parameters of ZigBee in Home Automation enable key benefits like lighting controls, single touch without obstructions, control using one application, built-in security from 802.15.4.

8.9 Wireless Sensor Networks (WSN)

8.9.1 Structure

Wireless sensor networks commonly called as WSNs are tiny nodes, which are randomly dispersed in a specified area. The position of nodes is not pre-determined. Each sensor node has limited energy and hence limited processing power. Most of these nodes are deployed to sense certain parameters, pre-process the data locally and transmit to the main server or to adjacent nodes for onward transmission by the protocol being followed, see Fig. 8.23.

As the nodes are not placed in pre-determined positions, the connectivity among them is through wireless and each node has to configure itself coordinating with other nodes into a self-configured network for communication among them. This trait of self-configuration with little power dissipation, and communication is a major challenge for WSNs. The network protocols must possess self-organizing capabilities and coordinate with other sensor nodes.

Several applications need such capabilities. Monitoring certain parameters in a hazardous area, disaster management, flood control, pest control in fields, earthquake

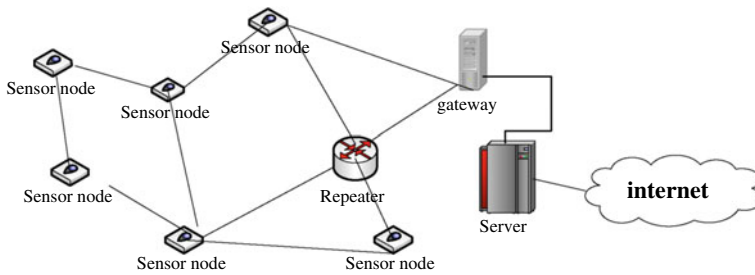


Fig. 8.23 A wireless sensor network

prediction, surveillance in unapproachable areas, controlled irrigation of fields with moisture sensing and so on are some potential applications, where WSNs are the only solution.

8.9.1.1 Differences with Other Networks

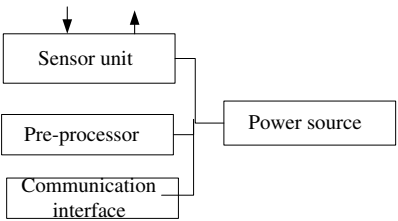
Conventional ad hoc network and wireless network protocols are quite different from the stringent requirements of WSNs. The differences being:

- The number of sensor nodes will be comparatively very high.
- The spatial distribution of the nodes will be highly dispersed, with some nodes very close and some very sparse. The network protocol should cater to different lengths.
- Sensor nodes will fail because of power loss or due to environment. Network has to reconfigure.
- The topology has to change dynamically based on the available nodes, their capabilities, and spatial distribution.
- Sensor nodes mainly use broadcast communication paradigms. Most traditional ad hoc networks are based on point-to-point communications.
- Sensor nodes have limited power, computational capabilities, and memory.
- Large number of nodes may congest the network traffic.
- Sensor nodes need to consume very low power. The network protocols have to be optimized based on these factors. The algorithms have to be built to increase network lifetime at the cost of low throughput and transmission delays without compromising the Quality of Service, see Fig. 8.24.

8.9.1.2 Localization in WSNs

Each node in sensor network must know its location with respect to other nodes to such an accurate extent, so that it can decide to which nodes to connect and set communication parameters. It should also have an image of all the locations of all nodes in the network. Immediately, it comes to our mind that each node should have

Fig. 8.24 Block diagram of a sensor node



a GPS. Because of poor power status, one cannot have. So the nodes communicate each other and apply certain algorithms to compute its local position.

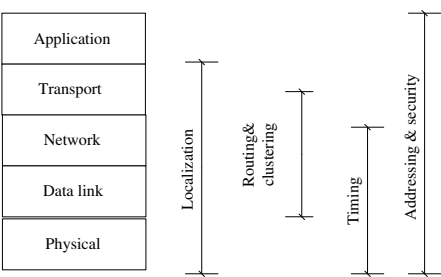
8.9.1.3 Network Protocol

WSN protocol consists of five layers, see Fig. 8.25, it has no session and presentation layers. The services provided by each layer cannot be separated, but major functionalities like localization, coverage, timing, synchronization are cooperatively provided by a collection of layers. The protocol effectively aims at minimizing energy consumption, end-to-end congestion control, end-to-end delay, and maintaining system efficiency. Traditional network protocols are not designed to meet these requirements. Brief functionality of each layer is given below:

Physical layer is responsible for converting bit streams for transmission over the communication medium and receiving from other nodes. It deals with various related issues, like, transmission medium, frequency selection, carrier frequency generation, signal modulation and detection, and data encryption. In addition, it also deals with the design of the underlying hardware and various electrical and mechanical interfaces.

Data link layer is responsible for data stream multiplexing, data frame creation and detection, media access, and error control in order to provide reliable point-to-point and point-to-multipoint transmissions. Major functionality of this layer is medium access control (MAC) by which the communication among nodes is done efficiently to achieve good network performance in terms of energy consumption, network throughput, and delivery latency. Normally a node remains in sleep, receive

Fig. 8.25 Network stack of WSN



and transmit states. This layer has to schedule when the node has to sleep to save power and resume transmission and reception. The layer creates and maintains the list of its adjacent nodes.

Network layer is responsible for routing the data collected by this node (source) to the destination (sink) node. The data cannot be sent in one hop as it needs more power to transmit to far nodes. Mostly, it is done by multi-hop transmission by an efficient route. This layer has the responsibility of dynamically deciding an energy efficient path to be used and forward the message to the next adjacent node in the route.

Transport layer has responsibility of connecting the WSN cluster to an external network. This works as a gateway for interconnectivity, so that WSN data are provided to the external world.

Application layer includes a variety of application—layer protocols that perform various sensor network applications, such as query dissemination, node localization, time synchronization, and network security. It links the user's applications with the underlying layers.

8.9.1.4 Routing in WSNs

This topic is extremely vast and a variety of protocols are being invented day by day. All of them focus on energy awareness. The routing protocols can be classified based on the structure of network (flat, hierarchical, location based, etc.) or protocol based (negotiation based, multi-path hopping, query based, etc.) Let us see one popular routing mechanism based on gradient approach.

A gradient specifies an attribute value and a least cost direction between adjacent nodes. The cost can be the energy consumed, time taken, number of hops it encountered, etc. The strength of the gradient is different towards different neighbors, resulting in different amounts of information flow. This process continues until gradients are set up from the base station (sink) to the source. The gradients are refreshed periodically when it starts to receive data from the source(s).

Base station or sink broadcasts a query and requests data of its interest. Similarly, any node can broadcast a query for data. All neighbors listen and propagate the request to its neighbors. During this propagation, the nodes transmit to the successors several parameters like number of hops from source to it, energy consumed, time, hop count, etc. Each node registers this data and decides the adjacent node having least gradient to whom it will transmit to reach the sink or base station. This process continues till the query reaches to the source from where the data have to be transmitted back to the sink. The source transmits data to the sink using the least gradient adjacent node. The same is depicted in Fig. 8.26.

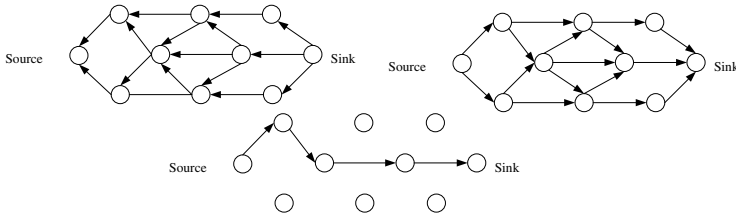
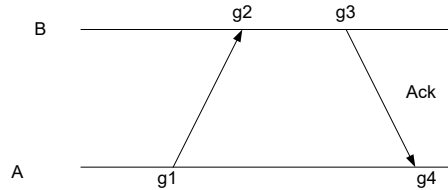


Fig. 8.26 Interest diffusion in WSN

Fig. 8.27 Time synchronization



8.9.1.5 Time Synchronization in WSNs

This topic is again with vast amount of literature with numerous synchronization techniques and is still under active research. Let us touch one technique by way of two-way message handshake method, see Fig. 8.27.

The root node A sends a *time_sync* packet to node B and initializes the time synchronization process. At the end of the handshake at time g_4 node A obtains the times g_1 , g_2 , and g_3 from the acknowledge frame. The times g_2 and g_3 are obtained from the clock of sensor node B while g_1 and g_4 are from the node A. After processing the ACK packet, the node A readjusts its clock by the clock drift value Δ , where $\Delta = ((g_2 - g_1) - (g_4 - g_3))/2$.

As an example, Node A is slower from B by 15 s. A has to sync with B. Let the transmission time between A to B and reverse is 6 s. B acknowledges after 10 s. Then $g_1 = 0$; $g_2 = (15 + 6)$, $g_3 = 15 + 6 + 10$, $g_4 = 22$. So $\Delta = \{21 - (22 - 31)\}/2 = 15$. The correction to A is 15.

8.9.1.6 Localization

The nodes are dispersed in space randomly. The location is not known to the node itself. This is just like spraying several nodes from a helicopter into a forest or a flooded area (This project started with an idea of spraying smart dust!). The location information is very essential to determine neighbors and routing in energy-efficient way.

The nodes are to self-organize initially to form the mesh network and also reconfigure when certain nodes are out of network or more nodes are added in neighborhood. Such reconfiguration is possible only when the position information is available with possible accuracy. Network protocols need location information.

We cannot get absolute location like using GPS because of size, cost and power. The locations can be indirectly computed from neighbors whose location is known and finding the distance from them. So distance measurement becomes key mechanism to compute location. Distance measurement in electronic way is by sending message and the time it reaches. Some techniques with this philosophy are.

- Measuring time of arrival
- Measuring round trip times
- Measuring received signal strength
- Detection of time lags caused by different types of signals.

Once distance is measured, the nodes' location can be computed by several localization algorithms as listed below. Again, we will not cover this vast subject except triangulation.

- Approximate versus exact precision
- Central versus distributed calculation
- Range based versus distance free (or angle)
- Relative versus absolute localization regarding point of reference
- Indoor versus outdoor usage
- Beacon free versus beacon based
- The limits between these characteristics.

Triangulation

Below is an example method for localization using two/three neighborhood algorithm. See the diagram with two circles. When we have two nodes whose locations are known (B1 and B2) and measure the distance from a node (P) whose location is to be computed as d_1 and d_2 , it means that P will lie at the intersection of the two circles at positions p_1 or p_2 , see Fig. 8.28.

If we do measure the distance from three known nodes B1, B2, and B3, the location of P is at the cross-section of the three circles. This method of computing coordinates is called trilateration.

8.10 Summary-NES

In this chapter, we have studied how embedded systems are networked and communicate each other. From the previous chapters, we observe that the requirements of Embedded Systems vary widely. Thus, their networking requirements also vary a lot. However, we have classified them into automotive, industrial automation, building

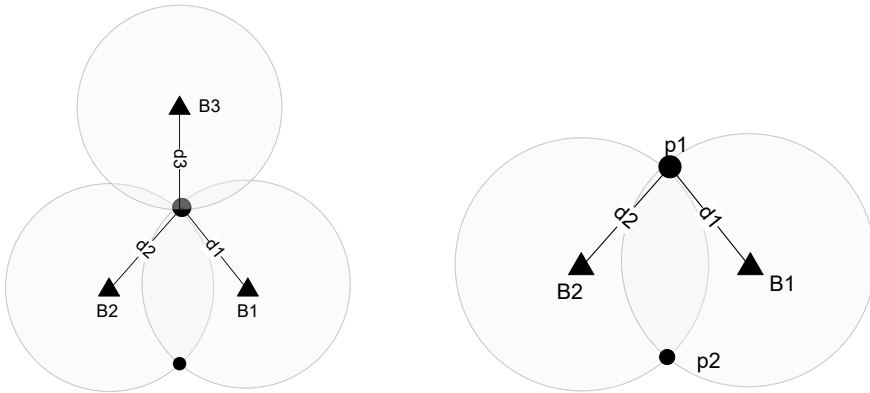


Fig. 8.28 Two or three neighborhood algorithm

automation and wireless sensor networks. Based on the requirements, we have studied the network architectures and the protocols which have been standardized for each class. These network protocols deviate totally from OSI standard. The real-time requirements also change among the classes.

Further research is very active to provide fault tolerance, power optimization, fail safe networks, WSNs, Internet of Things, etc.

8.11 Further Reading

No single book discusses all the protocols introduced in this chapter comprehensively. Most of them are standardized by multiple organizations. Once a decision is taken to adopt a protocol for an application, related standard documents would help in implementation. To get more details of different network protocols, Zurawski's (2017) work is a hand book that is worth reading.

References

- BACnet—a data communication protocol for building automation and control networks-ASHRAE
- Harmening JT (2017) Virtual private networks. Computer and information security handbook, 3rd edn
- IEC 61158: Industrial communication networks—fieldbus specification
- Introduction to industrial networks. Automation network selection: a reference manual, 3d edn
- Introduction to Lonworks platform. An overview of principles and practices, v2.0. Echelon Corporation
- Introduction to the LonWorks® Platform, Echelon Corp
- ISO 11898-1:2015. Road vehicles—controller area network (CAN)—part 1: data link layer and physical signaling

- Jamal N et al. Routing techniques in wireless sensor networks: a survey, Iowa State Univ.
- Kuriakose J, Joshi S (2014) A review on localization in wireless sensor networks. *Advances in signal processing and intelligent recognition systems*, pp 599–610
- Lopes F (2012) Networked embedded systems—example applications in the educational environment. Institute Superior de Engenharia de Coimbra Telecommunication Institute, Portugal
- Martin MA (2012) Overview of wireless sensor network Protocol stack for wireless sensor networks (WSNs). www.WordPress.com
- Sveda M (2009) Design of networked embedded systems: an approach for safety and security. In: 9th IFAC workshop on programmable devices and embedded systems. Roznov pod Radhostem, Czech Republic
- The ZigBeealliance
- Tomar A (2011) Introduction to Zibgbee technology, vol 1. Global Technology Centre
- Watteyne T (2009) Implementation of gradient routing in wireless sensor networks. In: Global telecommunications conference, globecom 2009. IEEE
- Zurawski R (2017) Networked embedded systems. Tayler & Francis

Chapter 9

Human Interaction with Embedded Systems



Abstract All of us are very comfortable to power on our laptop, install and execute any software. The software guides us in executing the next operation to some extent. In case you do some wrong operations, the software suggests a way to recover back. In most of embedded systems, you have no screen or a mouse but certain physical interfaces like buttons, sliders, rotating knobs, sticks, etc. as the interaction devices. If the end user is not conversant with the system and does a wrong operation, it may be very difficult to recover. The design needs careful operator interaction to prevent them to do faults. There are no default interactions on embedded devices. Interaction with an embedded device should work in any harsh environment. The device must be operable by variety of operators, young and old people, experts, and novices. The device should adapt to the operator's capabilities. To summarize, embedded systems need effective interaction satisfying human needs. This needs formal methodologies to be studied and adapted. Good interface helps in operating the system safely, effectively with minimal operations and the user enjoys operating the system. The quality of interface is measured in terms of "*Usability*" of the product. Basic mistake we do is by making an assumption that "*all users are alike and they are like the designer.*" Evaluating the user's physical and cognitive capabilities is essential. Human user's physiology, their capabilities, and limitations in sensing through different channels, memory, cognitive, and motor capabilities have to be studied as explained in Sect. 9.3 before interface design. Section 9.4 details certain physical interfaces used in the embedded systems. The interaction between the human user and the system is to be designed with user in mind and not the other way. Section 9.5 describes the concept of interaction. Interaction models help us to understand how the interaction between user and system is progressing. Section 9.6 reviews recent paradigms in computer interaction. Section 9.7 covers rules for interface design for maximum usability. Section 9.8 covers methods of interface evaluation using cognitive, heuristic, and user participation methods. To summarize that the ultimate factor for success of a product is the usability.

Keywords Usability · Affordances · Retina · Rods and cones · Visual interface · Stereoscopia · Cognitive system · Eye gaze · Virtual reality · Augmented reality · Gulf of execution · Gulf of evaluation · Ergonomics · Metaphors · Ubiquitous computing · Implicit interface · GOMS model · Keystroke Level Model (KLM)

9.1 Motivation

Quotes:

- “*It is easy to make things hard. It is hard to make things easy.*” – Al Chapanis, 1982
- “*Learning to use a computer system is like learning to use a parachute – if a person fails on the first try, odds are he won’t try again.*” – anonymous

All of us are very comfortable to power on our laptop, install and execute any software. We like some applications because of their functionality but we find hard to use them because it is very tough to interact due to bad human interface design. We give it away using it any more in spite of its powerful features. Because of this reason, the subject of human–computer interface has become most important topic in software design. Now comes to embedded systems!

In computer-based applications with a screen in front of you and a mouse, the software guides you in executing the next operation to some extent. In case you do some wrong operations, the software suggests a way to recover back. In case of an embedded system, you have no screen or a mouse but certain physical interfaces like buttons, sliders, rotating knobs, sticks, etc. are the interaction devices. If the end user is not conversant with the system and does a wrong operation, it may be very difficult to recover. *So embedded device design needs careful operator interaction to prevent them to do faults.*

In computer-based applications, certain operations are executed by default. It is easy to log into system using user id and password. Easily recover password through certain operations. If you want to protect the operations from an embedded device you need biometric device added to the system. Errors cannot be displayed without a screen. *There are no default interactions on embedded devices.*

Computer systems work mostly in protected environment. An embedded device should work everywhere. A wristwatch should display the time in deep sun light and also in deep dark. *Interaction with an embedded device should work in any harsh environment.*

Computer systems work in a protected environment and the behavior does not depend on the environment they work. In case of *embedded device, the device has to interact with the environment, take decisions, and behave accordingly.* As an example, when someone is driving, the music system should switch to voice-based operations.

In computer systems, the interaction is only through certain interfaces like keyboard and mouse. *In embedded environment, the interaction is through affordances, i.e., the click of a button, rotation of knob in a particular way, push with certain pressure, etc.*

Computer systems are normally operated with sufficient knowledge of operation. *An embedded device must be operable by variety of operators, young and old people, experts, and novices. The embedded system should adapt to the operator’s capabilities.*

To summarize, embedded systems need effective interaction satisfying human needs. This needs formal methodologies to be studied and adapted. The type of interactions vary with different types of people (aged, differently abled, etc.), different environments. Interactions also vary with different applications and with different platforms. This topic is to be studied independent of technology as the interactions should work with different technologies.

This subject is interestingly multi-disciplinary with cognitive science, psychology, industrial engineering, ergonomics, context sensitive, and human factors. This chapter introduces the principles of human–computer interaction (HCI) and extends the concepts to embedded systems design to take care of the abovementioned challenges. The objective of this chapter is to get readers acquire knowledge and skills needed to create *highly usable* embedded systems. We will study analytical way of designing and evaluating interactive technologies. By the end of this chapter, readers should be able to design highly effective interaction with embedded systems. This chapter focuses only the specific interface techniques necessary for embedded environment and assumes you are conversant with generic human–computer interface. The topic of HCI is one-semester course, only essentials related to embedded systems is covered in this chapter.

9.2 Overview

Typical embedded systems which need strong human interaction are VCRs, household gadgets like washing machine, microwave oven, mobile phone, car dashboard, etc. Professionals may need interaction with industrial equipment, simulators, cockpit of airplane, air traffic controls, etc.

Good user interface is required for all systems whether they are simple or complex systems. Good interface helps in operating the system safely, effectively with minimal operations and the user enjoys operating the system. The quality of interface is measured in terms of “*Usability*” of the product. Several factors which decide usability are ease of learning to operate, quick completion of any task with the product, less errors made by the operator, satisfaction of the user after using the interface, and user retention to use the product again and again and not switch over to another product due to bad interface.

9.2.1 Study Users for Good Interface

Basic mistake we do is by making an assumption that “*all users are alike and they are like the designer.*” This is not true. So, before jumping into interface design, one has to consider the following factors in general.

Depending upon the product class, these factors will vary.

- Study details of the users who are going to use the product, their age, IQ factors, knowledge of the product, physical capabilities, etc.
- Study, in detail, their physical capabilities. A mobile phone interface designed for super senior citizens is different from normal users. Another example is a wheel chair for physically challenged.
- Study the user's cognitive capabilities. This varies with health, age, mental stability, current ailments, etc.
- User's with varied capabilities in operating skills. The designer may have to simulate the interface as a prototype and assess the skills of user segments before design.
- Study user's motivation to use the product. Some may be quite enthusiastic and some may be afraid of using.

9.2.2 Evaluate the Design

Before implementing the design, the interface is worth getting valued by the designated users using a simulated interface design. Some factors of the evaluation will be subjective and some quantitative. Some quantitative metrics are like the time to learn the operations, the speed at which the task is performed, the number of errors made while completing the task, the retention of the sequence of operations to complete the workflow (short-time retention and long-time retention) and overall subjective satisfaction.

Before designing, the human sensory limitations have to be thoroughly understood and taken care in the design. This is the reason why the topic is named globally as human-computer interface. The interface design for embedded systems has more constraints than the paradigms evolved in HCI. So let us understand human sensory limitations.

9.3 Human System

Human user interacts with the product to accomplish certain tasks through a sequence of operations. So human, psychological, and physiological aspects play major role in the design. In this chapter, we will study the characteristics and limitations of human sensory system and how the design has to be adapted to avoid the limitations.

Human system itself is a processor. It has perception through visual (see), auditory (listen), and haptic (touch) sensory systems. It has motor system (actuators) for applying responses based on sensory signal processing. Information is stored in memory. It has sensory memory (for storing immediate sensed information), short-term memory (used to store certain information for a short term), and long-term memory (used to store certain information for a long term). Information is processed

and applied through cognitive capabilities, viz., reasoning, problem-solving, skill acquisition.

9.3.1 Vision

Visual perception of the human eye can be divided into two stages: the reception of the stimulus from the external world, and the processing and interpretation of that stimulus (see Fig. 9.1).

Though we are not biological experts, limited knowledge of anatomy of eye is necessary for interface design. The *cornea* and *lens* focus the image onto retina. *Retina* has photoreceptors which are *rods* and *cones*. Rods are highly sensitive to light and therefore allow us to see under low level of illumination.

Cones are sensitive to different wavelengths of light. They are less sensitive to light than the rods and can therefore tolerate more light. This allows color vision.

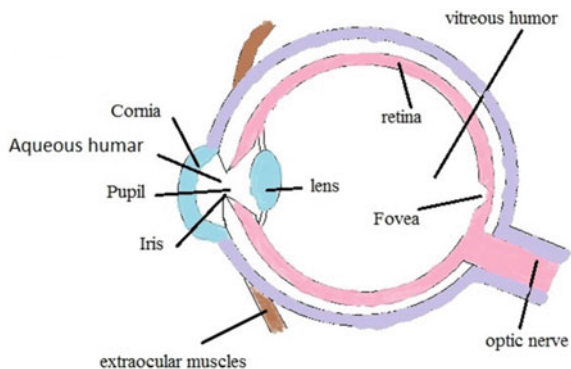
Cones are concentrated on *fovea*. The retina has X-cells which are concentrated in the fovea. They are responsible for early detection of image pattern. The Y-cells are distributed over the entire retina. They perceive movement. Due to this type of distribution, we may not be able to detect the pattern changes in peripheral vision, but movement can be perceived.

Design rule-1:

- *Design the panels such that alarm information is not statically displayed but flashing so that the movement is sensed.*
- *Keep such indications with small movements in the corners of the panel.*

You can observe in most of the computer screens, any flashing messages will pop up in one of the corners. The pop-up movement is easily judged by Y cells.

Fig. 9.1 Eye structure



9.3.1.1 Visual Interface

- Perceiving size and depth, brightness and color, each of which is crucial to the design of effective visual interfaces.
- **Visual angle** indicates how much of view object occupies. Relates to size and distance from eye. In Fig. 9.2, D = distance from fovea to scene and V = visual angle in degrees.

Design rule-2:

- *If a large display panel is designed to display in public or on roads (road signs, directions) the distance from where it must be visible has to be decided and then the size finalized.*

Visual acuity is the ability to perceive the details of the screen or panel. As an example, a single line can be detected above visual angle of 0.5 s. Visual acuity increases with increased luminance. In dim lighting, the rods predominate vision.

Design rule-3:

- *The brightness of the panel lights or display devices (LCDs, LEDs, and small LCD panels) has to be decided based on the distance and the visual angle.*

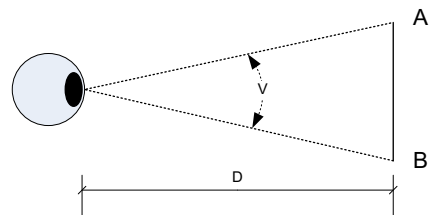
Flicker: As luminance increases, *flicker* also increases. If speed of switching is less than 50 Hz then the light is perceived to flicker. In high luminance, flicker can be perceived at over 50 Hz. Flicker is also more noticeable in peripheral vision. Larger the display the more it will appear to flicker.

Design rule-4:

- *Avoid flicker by deciding the panel size, distance from user, and luminance of the devices on the panel.*

Color is usually regarded as being made up of three components: *hue*, *intensity*, and *saturation*. Hue is characterized by the wavelength of the light spectrum. Blue, green, and red are the order in which the wavelength increases or light frequency reduces. Human eye can differentiate about 150 hues at a time. Intensity is the brightness of the color. Saturation is the amount of whiteness in the color. Humans can remember and identify about 10 colors without much training. Cones in the

Fig. 9.2 The visual angle



eye are sensitive to different wavelengths of light. Hence, eye perceives different colors due to cones. Hence, color vision is best in fovea. Peripheral vision has worst color sensitivity. People from color blindness cannot discriminate between red and green.

Design rule-5:

- *Do not use more than 7–8 colors on the buttons for the users to distinguish.*

Depth perception: Human system has visual ability to perceive depth from monocular cues. Some of them are depth perception from the motion of object, motion parallax, relative sizes of the objects, occlusion, lighting, shading, etc. we will not get into the details of mono-ocular perceptions but discuss binocular cues specifically used in 3D measurement devices.

A **stereoscope** is a device for viewing a stereoscopic pair of separate images, depicting left-eye and right-eye views of the same scene, as a single three-dimensional image. Depth is measured by instruments like photogrammetry by taking two photographs and by the law of similar triangles the distance is measured. In Fig. 9.3, left photograph of point A is taken by left camera with focal point at B. It marks point A on the film at distance d_1 from center. Similarly, the right photograph marks point A at distance d_2 . Once you know d_1 and d_2 , and their focal points and distance BD, AC can be computed. This principle is used in all depth-measuring instruments. Figure 9.4 is a stereo scope where two aerial photographs taken at known distance are placed below the stereoscope and viewed by left eye and right eye. The map features will be seen in 3D and with proper instrumentation the depths of geographic entities are measured using this principle (photogrammetry).

The same concept is used by human perception to estimate the distance by the images formed by the left eye and right eye on the retina. To present stereoscopic pictures, two images are projected on the same screen through polarizing filters. The viewer wears eyeglasses with the left glass and right glass has oppositely polarized filters. Each filter only passes light which is polarized as per projected image on the screen. It blocks the oppositely polarized light. Hence, left eye sees the left image and right eye sees the right image. And hence the 3D effect is achieved. This mechanism

Fig. 9.3 Stereoscopy principle

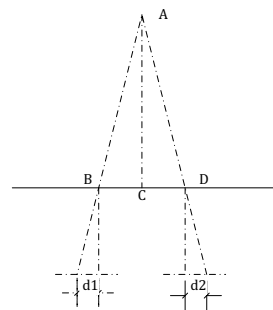


Fig. 9.4 Image of stereoscope (Courtesy usgs.gov U.S. Geological Survey Department of the Interior/USGS)



gets the left image onto left eye and right image onto right eye. Human system perceives third dimension from the stereoscopy principle as explained above.

9.3.1.2 Reading Interface

Any content on a panel or device has lot of text which has to be read. There are several stages in the reading process. First, the visual pattern of the word on the panel/screen is perceived. It is then decoded with reference to an internal representation of language. Further processing is done through cognitive techniques by the brain for language processing which includes syntactic and semantic analysis.

Eye makes jerky movements while reading text. They are called *saccades* followed by *fixations*. During the fixation period, system perceives the read content. This accounts to roughly 94% of the time elapsed. The eye moves forward and backward over the text and is called *regressions*. If the text is complex there will be more regressions. This is the reason why a scrolling text is difficult to read compared to a static text. The jerky movements of eyes and perception will not synchronize with the text rolling speed.

The speed at which text can be read is a measure of its legibility. Standard font sizes of 9–12 points are equally legible. A negative contrast (dark characters on a light screen) provides higher luminance and, therefore, increased acuity than a positive contrast.

Design rule-6:

- Use panels with static text rather scrolling text unless you have large text to be shown.

Design rule-7:

- Adjust font size based on the view distance.

Design rule-8:

- **Use dark characters on a light screen for increased acuity.**

9.3.2 Touch

Also called as *haptic perception*. The system senses the environment by users touch. Touch screens, virtual reality games, and simulators are some examples. Haptic sensing provides feedback on the environment. The stimulus is received via receptors in the skin. Thermo-receptors sense temperature. Mechano-receptors sense intense pressure.

9.3.3 Movement

The operator responds to any event displayed on the system panel or menu highlight on the screen or some mechanical actuation happened (like automatic door latch opened and user has to push to open the door to get access). For all these events, user takes certain time to react and then acts on the event. The total response time of the user is the *reaction time + movement time*. A few examples are as below.

When the user swipes a card to get access into an ATM cabin, a greenlight is displayed and opening sound of the door latch occurs. User has to sense both the events and pushes the door within certain time.

On a mobile an OTP is displayed for finite time and user has to enter into ATM for access grant. All such operations need user's response. Different users will have different reaction times. Reaction time is dependent on stimulus type.

For an average person, visual response is around 200 ms. Auditory response is about 150 ms. The response times and movement times vary with age. Skill and practice can reduce reaction time. If the same type of operation has to be done, fatigue should not occur to the operator.

Design rule-9:

- **Estimate the user's response and action times through user trials.**

Design rule-10:

- **Reduce movements of user actions (like pressing a sequence of buttons, movement of hand or fingers to access button, etc.) to improve response time.**

Design rule-11:

- **Work flows and panel buttons to be organized to reduce user fatigue while responding.**

9.3.4 Memory

User's retention power plays a lot in interface design. Human system has sensory memory which retains visual stimuli (iconic), oral stimuli (echoic memory), and touch (haptic). As an example, when you see fireworks, the image is retained about 0.5 s, similarly the sound of a cracker. The sensed signal from these channels gets updated continuously.

Short-term memory (STM) is something like a scratch pad for immediate recall. A good example is some utters eight digits and you have to listen and repeat the same. You have to retain the digits for sometime. You get important information and loose it after some time. The retention time is about 100–150 ms. An average user can retain 7–10 digits in a short time.

Human system prefers to manage short-term memory by sensing in chunks. This improves short-term memory capacity. As an example try remembering 2537868956. Also try 253-786-8956. It is easy to retain the three *chunks*. Successful formation of a chunk is known as *closure*.

Design rule-12:

- **System should display numbers in chunks. Not as long digit sequence.**

Design rule 13:

- **When a list is displayed for selection or group of items, it should not exceed more than 7–10 items.**

Repeated exposure to a stimulus transfers from STM to LTM (long-term memory). That is, through rehearsals or repeated operations. Information is easy to remember and gets into LTM when the information is structured and meaningful.

Design rule-14:

- **Creating random passwords and trying to recall from LTM is tough. So create a random data from a meaningful data using meaningful rules. You can recall from LTM easily.**

9.3.5 Cognitive System

Human cognition is the activity of observing with selective attention, learning, acquiring knowledge, and applying to solving problems.

Human system can focus on one particular thing in spite of other events occurring around. Certain auditory, visual cues help in such selective attention. Examples are like mobile phone ringing when a call is coming, an attention sound in railway stations before announcements, etc. Systems can use this effectively when user's attention is needed. It has to be judiciously used.

Design rule 15:

- **Use auditory, visual cues to get users' selective attention like beeping and blinking.**

Human system learns procedures by observing (selective attention) to certain actions. Like observing someone playing badminton. Learn by practice (cycling), learning by repetition, learning by observation of several factors, inferencing certain rules, etc. Learning facilitated by analogy, by structure and organization, and by repetition.

Design rule 16:

- **Users can learn the system workflows from the knowledge they have from previous interfaces. So adapt standard interfaces for easy learnability.**

People solve problems more heuristically than algorithmic way through crisp calculations. They learn better strategies by practice and possible better alternatives. They try to solve problems with interest if sufficient cues are given.

Design rule 17:

- **Allow flexible shortcuts from long workflows. Do not force user to follow unique workflow. Allow multiple ways of doing.**

Design rule 18:

- **Provide active help during the workflow rather passive help.**

While operating the system users *slip* what they want to do. They have right intention but fail to do due to physical and mental state. System users make *mistakes* because they did not understand. Whether it is a slip or mistake, users get into stress.

Design rule 19:

- **Avoid stress in users by making esthetic interface design**

To summarize, we can come out with general design rules considering human physiology. An understanding of the capabilities and limitations of the human system helps in designing usable interfaces.

9.4 Physical System

We have reviewed human physiology in brief, covering the sensory, memory, and cognitive system with respect to its limitations and strengths in designing user interfaces for embedded systems. Let us see certain physical interfaces used in the embedded systems.

Different interfaces are used for different types of interaction as listed below:

- Input devices like keyboard for text entry;
- Mouse, digitizer, etc. for pointing a location on screen or paper;
- Screens, digital paper for display of any text, and multimedia information;
- Special devices for virtual reality and augmented reality;
- Special devices for voice-operated devices (speech recognition and synthesis);
- Biometric devices for haptic and biosensing;
- Emotion sensing through eye gazing; etc.

We are very much conversant with computer interfaces like keyboards, mouse, digitizers, etc. They are not discussed here. Only some specialized interfaces designed for embedded systems interface will be dealt in this section.

9.4.1 *Handwriting Recognition*

As the world is getting into paperless office, authentication is through detection of handwritten image. Handwriting recognition does not lie in the shape of the character or line drawing drawn. It remains in the stroke information. The devices which detect signatures must detect stroke information while doing signature not just the final character shape.

The technical challenges include

- capturing all useful information—stroke path, pressure, etc. in a natural manner;
- segmenting into individual letters and joining up;
- interpreting individual letters;
- detection or generalizing different styles of handwriting.

This method is very useful in gesture recognition and signature authentication in embedded systems.

9.4.2 *Speech Recognition*

Speech recognition and synthesis is a promising area for speech-to-text conversion, interactive voice response systems, speech-based authentication, voice commands, etc. Currently, the research has evolved systems with more than 90% confidence

due to machine learning algorithms. Certain issues like external noise interfering, imprecision of pronunciation, large vocabularies, speech by different speakers in different accents, and languages are in advanced research.

Embedded applications with voice-based operations are increasing and becoming essential feature. Very much necessary for physically challenged persons and some applications where hands are occupied to enter data and where keyboard-based input is practically impossible.

9.4.3 Eye Gaze

Emotion detection is becoming state-of-the-art topic. When user views a panel, a screen, or some view area, the system will detect the specific object or segment user is viewing. Several other applications in defence and augmented reality where the operators view is detected by a laser and object detected. Research is in advanced stages.

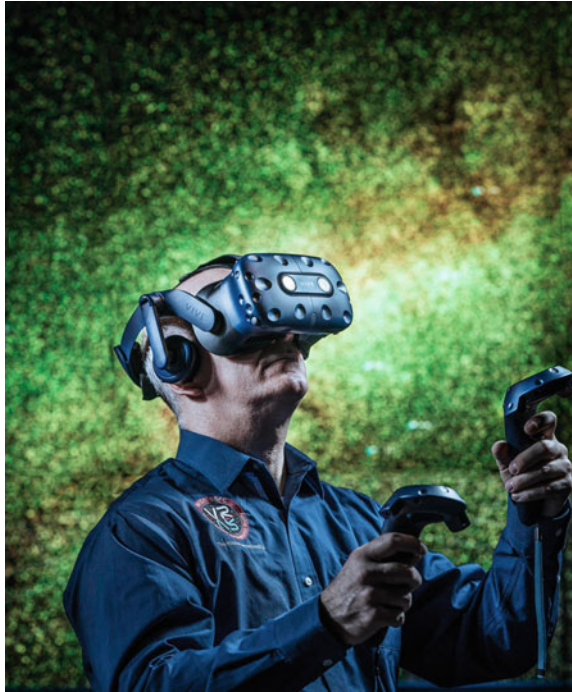
A low-power laser, which does not damage eye, is focused onto eye. The beam gets reflected from the retina. The direction of the beam depends upon the view angle of the eye. Once the beam direction is detected, one knows what object the user is viewing on the screen or what real-world object the user is seeing. This technique is called *eye gazing*.

Applying this technique, several gadgets are emerging for emotion detection, information of objects being viewed, etc. Such systems have high-potential applications for physically challenged people and for workers in environments where it is impossible for them to use their hands, for evaluating advertisement screens, etc.

9.4.4 Virtual Reality

Virtual reality is the use of computer technology to create a simulated environment. Unlike traditional user interfaces, VR places the user inside an experience. Instead of viewing a screen in front of them, users are immersed and able to interact with 3D worlds. By simulating as many senses as possible, such as vision, hearing, touch, even smell, the system is transformed into an artificial world. We get the most out of VR if we can interact with the world just as you would with the real world: with your natural body movements. This gadget is very useful in design of simulators (see Fig. 9.5).

Fig. 9.5 Attachment to mobile for VR display
(*Courtesy “Exploring the Universe in Virtual Reality”*
by NASA Goddard Photo and Video is licensed under
CC BY 2.0)



9.4.5 Sensing Position in 3D Space

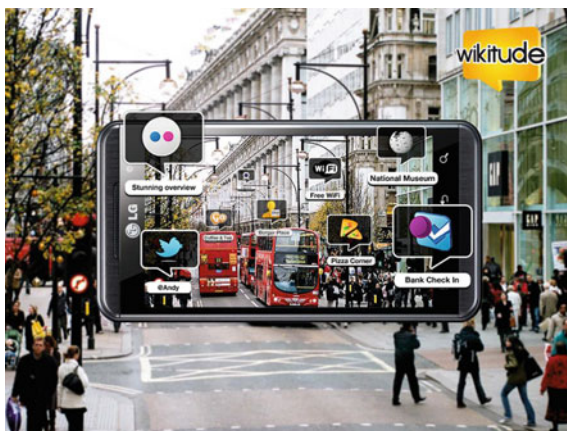
Several applications need to sense the position or movement of an object in 3D space. Gesture recognition recognizes the movements of limbs and infer from it. Detecting limb joint angles, 3D pointing mouse, etc. are few examples.

Several techniques are used to detect 3D position using 3D mouse, 3D gloves uses fiber optics to sense finger positions, VR helmets, etc. are some techniques. Future embedded applications sense human gestures and no interaction through explicit interaction through buttons and pointers.

9.4.6 Augmented Reality (AR)

While virtual reality is creation of simulated environment, augmented reality is a technique by which you add additional information on a real-world scene. The AR gadget senses its position and direction of focus. From this information, the location of focused objects is computed. Once the objects or the 3D location of the area is sensed, the information available in the server of that spatial location is extracted and superimposed on the gadget screen.

Fig. 9.6 Augmented reality capturing data from nearby objects (Courtesy “LG OPTIMUS 3D ADDS ANOTHER INNOVATIVE FIRST WITH 3D AUGMENTED REALITY” by LGEPR is licensed under CC BY 2.0)



Very good example is Google Glass. Google Glass offers an augmented reality experience by using visual-, audio-, and location-based inputs to provide relevant information. For example, upon entering an airport, a user could automatically receive flight status information.

AR applications are limitless arpost (2009). Wearable AR glasses and headsets will help futuristic defense applications by which the personnel can view real-world scenes, and also get strategic information of the viewing objects superimposed. One infrastructure application in use is wear AR glasses on a busy street and observes the underlying drainage pipes superimposed in the view. Also find data of nearby objects on the street (see Fig. 9.6). The real-world objects are superimposed with data.

9.5 Interaction Concepts

In the above two sections, we have seen the human user's physiology, their capabilities and limitations in sensing through different channels, memory, and cognitive and motoring capabilities. We have seen in the next section, how the systems are designed which are capable of interfacing with human user's sensory system and actuation.

The next subject of interest is how the human user interacts with the system to get his task goal done in simplistic and enjoyable way. The interaction between the human user and the system is to be designed with user in mind and not the other way. We will study the concept of interaction in this section below.

9.5.1 Interaction Model

Interaction models help us to understand how the interaction between user and system is progressing. It addresses how the user and system move their states through their responses and actions till the goal of user is achieved. Interaction defines what user wants and what system does. It involves a sequence of steps of interaction till the goal is achieved. As a simple example, user presses power ON button. System responds by self-checking health of complete system and displays “ready.”

Ergonomics looks at the physical characteristics of the interaction and how these influence its effectiveness. In the example above, ergonomics help where the power ON button is to be placed for safe and effective operation, say by placing it on the corner and with radium-based material for nightglow.

The dialog between user and system is influenced by the *style of the interface*. Is it a toggle button, On/Off button, or lock and key type....

The interaction takes place within a social and organizational context that affects both user and system.

9.5.2 Donald Norman’s Model

A popular model by Donald Norman on interaction (2013) (see Fig. 9.7). The user formulates a goal which he wants to achieve. He specifies certain actions at the system interface. He starts executing one action or action sequence to achieve as a part of the goal. This is done by operating the user interface (like pressing a button).

He then observes, perceives, and interprets the state of the system in terms of his expectations. He then evaluates the system state with respect to the goal achieved. He continues this loop, till he achieves the goal. If the system state is not in tune to the user’s goal, change the goal and proceed to the new goal.

Some of the systems are harder to use than others. To reason out the cause, we should understand the concept of “*Gulf of execution*” and “*Gulf of evaluation*.”

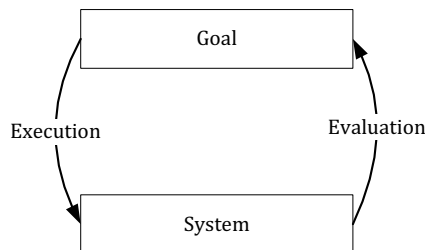


Fig. 9.7 Execution-evaluation model

9.5.2.1 Gulf of Execution

The user formulates certain actions to reach the goal but the system does not allow them to be executed. Gulf of execution is the difference between the user's formulation of the actions to reach the goal and the actions allowed by the system. If the actions allowed by the system correspond to those intended by the user, the interaction will be effective. The interface should therefore aim to reduce this gulf. As an example someone watching a TV channel, the user wants to record current channel. He presses a "Record button." System responds on the screen "enter channel number," then "from-time," and "to-time." User's expectation of current channel recording is not present. This is gulf of execution.

9.5.2.2 Gulf of Evaluation

When the user executes an action, the system state is evaluated. The system state is too far from the user's expectations. The gulf of evaluation is the distance between the physical presentation of the system state and the expectation of the user. If the user can readily evaluate the presentation in terms of his goal, the gulf of evaluation is small. The more effort that is required on the part of the user to interpret the presentation, the less effective is the interaction.

A very simple example is an inverter which is designed to put it in the inverter mode or get power directly from main. For this they provided a push button. When someone pushes, it makes inverter on and when pushed again it makes inverter off. Unfortunately user cannot know what the state of the inverter is, because it is simple push button. The solution is to keep an LED which shows the status of inverter. This is gulf of evaluation. User's expectation and system's presentation are different.

9.5.3 Case Study-1

A person interacts with ATM to draw money. Explain all stages of the interaction model with respect to this example. List all states of interaction in the form of a table.

Solution

Goal is to get cash from ATM. Some cycles are shown below. This can be made exhaustive (Table 9.1).

NB: In the example above, if the ATM card has to be pushed in and gets released after cash transaction, the goal has to be properly changed; collecting card is a sub-goal. If the goal completes after collecting cash, user forgets collecting card. This was happening in old ATMs. Human psychology in goal completion comes into picture. All ATMs are changed now to swipe card.

Table 9.1 Stages of interaction

SNO	Intention	Specify action and execute	Perceive system state	Interpret and evaluate
1.1	Get card accepted	Insert card and swipe	Card accepted and asks for password	Card accepted
1.2			Card rejected message	Card rejected
2.1	Get authenticated	Enter password	Menu displayed for selecting money	Card authenticated for cash withdrawal
			Wrong password message	Authentication failure
3.1	Select amount	Enter amount	Response message to wait for cash	Successful cash selection
3.2			Wrong entry message	unsuccessful cash selection
3.3		Nil	Cash dispensed	Successful cash dispense
3.4	Collect cash	Pull cash from dispenser	System resets and waits for next transaction	Transaction successful

9.5.4 Ergonomics

The field of *ergonomics* addresses issues on the user side of the interface, traditionally the study of the physical characteristics of the interaction. This touches upon human psychology, physical constraints of the user, and system constraints. Ergonomics are good at defining standards and guidelines for designing the systems.

Some examples are given below:

- Arranging the buttons into blocks according to functionality and logical relationship.
- LCD panel in the AC remote to view the AC temperature in dark and also in day light.
- Power on button on the remote should be radium based so that it can be viewed in the dark.
- If lot of data to be shown on panel, adapt graphical display like sliding bars, rather a cluster of numbers.
- All cable connections to the controller box to be on the backside and have no clutter in front.

9.5.5 *Physical Design*

Physical constraints play importance in design of embedded system equipment with lot of buttons, knobs, and displays. Below are some important aspects to be followed.

Ergonomic: Button sizes to be optimum so that they can be easily pressed. Not too big keep occupying more space. Most infrequent switches in the panel back.

Physical: High voltage switches are to be big.

Safety: Child locks in cars

Maintenance: All modules should be able to be opened in sequence and interlocked with power.

Aesthetic: Must look good.

Economic: Must not cost much.

The *value* of a product depends on all factors above like interaction, ergonomics, physical design, etc. A product with good value fulfils all work requirements, very much usable; working with it is enjoyable and fun. It is subjective. This is not dependent on cost. People use something only if it has perceived value and *value exceeds cost*.

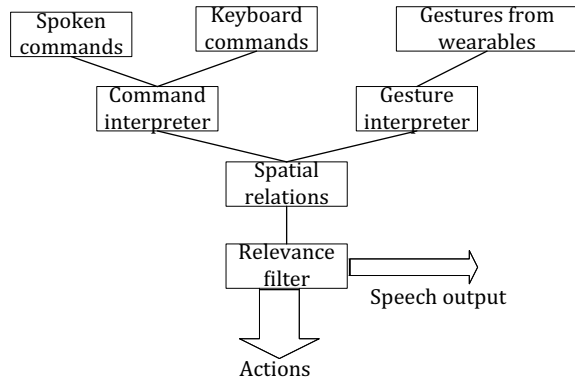
9.6 Recent Paradigms in Computer Interaction

Paradigms are basic framework of understanding, methodology, and application of certain principles which are commonly accepted by users and developers. In HCI, new paradigms are getting evolved with time as technology is advancing. Certain advances are given below.

9.6.1 *Metaphors*

Metaphor is a technique to teach a new concept from an existing concept which is well known. A metaphor is a good technique to increase the initial familiarity between user and computer application. The learning and retention of a system's functionality is considerably facilitated by meaningful and consistent metaphors. As an example, *shopping cart* in web application is a name which encapsulates all data and operations for web commerce. Computers use files, folders, recycle bin, and so on which are well known to every user.

Fig. 9.8 Multimodal system with voice, keyboard, and gesture inputs



9.6.2 Multimodality

Humans quite naturally process information by simultaneous use of different channels. We point by finger to someone and refer to them as “you.” It is only by interpreting the simultaneous use of voice and touch that our directions are easily articulated and understood. For example, we can modify a gesture made with a pointing device by speaking, indicating what operation is to be performed on the selected object. However, genuine multimodal systems rely to a greater extent on simultaneous use of multiple communication channels for both input and output. Figure 9.8 shows an interface with multimodal system with voice, keyboard and gesture inputs, and voice output command and actions.

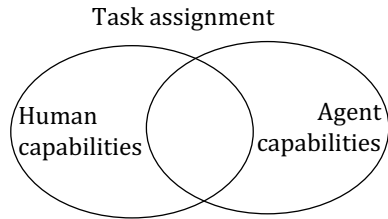
9.6.3 System Supported Cooperative Work

When a task has to be performed across any two users A and B, the task is performed in coordination with a system in between. One simple example is a money transfer by wallet transactions. The transaction is done across two systems with the help of a server in between. Such systems are built as supporting users working in groups.

9.6.4 Human-Agent Interaction

An agent is a computer system that is situated in some environment and that is capable of autonomous action in this environment in order to meet its design objectives. The independent agent provides a flexible way to build a dynamic user interface for the need of wide range of users.

Fig. 9.9 Human–agent interaction concept



As an example, varieties of patients come to hospitals to report their problem and get treated. Doctors cannot handle and interact with all of them. A good solution is to keep certain agents who are capable of interacting with different types of users and able to interact with doctors and serve the customers.

The traditional interface approach has drawback when interacting with users, especially when users are distributed and belong to different organizations. Agent’s properties are introduced and contributed as a flexible approach to complement the disadvantage of direct manipulation when a more general usability criterion is required (see Fig. 9.9).

9.6.5 Ubiquitous Computing

Ubiquitous systems are networked, distributed, and transparently accessible. System interaction with humans is hidden. Systems are aware of environment context. Ubiquitous computing permeates our physical environment so much that we do not notice the system any longer. Such systems are hidden and distributed in the physical world and designed such that they do not demand user’s interaction.

9.6.6 Implicit Interface

Humans are good at recognizing the “context” of a situation and reacting appropriately. When same is done by a system it is classified as “context-aware computing.” Systems sense context, make inferences from past patterns and current *context*, and implicitly execute.

In *context-aware computing*, the interaction is more implicit. Context-aware applications follow the principles of *appropriate intelligence*.

9.7 Design for Usability

Before designing any software, any organization follows certain coding rules. This is mainly to follow uniformity among all developers and maintain quality. Similarly, interface design rules are followed to maintain uniformity. Sometimes, they provide certain guidelines based on previous success stories. Certain design patterns are also available in the literature extracted from success stories. All this is intended to prevent many bad designs, before they begin, or evaluate existing designs on a scientific basis.

Here we will concentrate only on rules for interface design for maximum usability of the product (Dix et al. 2005), Shneiderman (2000), Dix (Norman 2013). Foley have framed design principles for interface designs.

9.7.1 Goals of Usability Engineering

Goal is

- to use the system *effectively* (correctly, accurately) to execute all functionalities.
- to use the system *efficiently* (less effort, quick, and enjoyable) to execute all functionalities.
- to use the system *without errors and safely* (to the system, user, and environment) while executing the functionality.
- easy to use—*user-friendly*.
- *enjoyable* in use—pleasurable experience.

9.7.1.1 Learnability

To achieve the above goals, the designer should consider good learnability of the system by the intended users. Users should quickly adapt to the system through the commands and begin effective interaction and achieve maximal performance in least time.

Interface should be *flexible* enough by which the goal can be achieved by multiple ways of operations.

System interaction should be *robust* enough that any mistake/slip done by the user should not cause the system misbehave or cause catastrophes or shutdown. System should guide the user to avoid getting into that state.

A system is more learnable if the system observes the last operations and observes current operation and predicts the type of interaction and final goal. If this leads to a wrong direction, system should guide the user. This is intelligent online guidance. Online guidance should suggest available alternate operations also. Usability improves if the system proves the operation is done. This can be explained by an example. If you press a button to close a valve, where the valve is remote or hidden, the system operates, closes the valve then senses whether the valve is closed, and

displays on the panel. This is the *principle of honesty* by which interface provides an observable and informative account of such change.

In screen-based interface, certain operations are very common across other products, like file operations. Cut/paste operations use same commands. These cause system learnability to improve. Unfortunately in embedded interfaces, such generalized commands have not yet found. No two TV remotes have similar interfaces.

9.7.1.2 Flexibility

Flexibility is the ability of the system and the user to interact in multiple ways. When system initiates a dialog and asks the user to do certain operation, it is called *system preemptive*. As an example, in a car when the gas level is low, system preempts to put off the AC of the car!

The user may be entirely free to initiate any action toward the system, in which case the dialog is *user preemptive*. Maximize the user's ability to preempt the system and minimize the system's ability to preempt the user.

Task migratability is a way the system takes over certain tasks and executes autonomously. A good example is auto-navigation of cars, cruise control in cars, etc. This provides flexibility in system usage.

Flexibility improves if a goal can be achieved by different combination of tasks, allows different ways of presenting the details (like displaying a clock in digital or analog form). Flexibility also improves if the interface is *customizable* to remove unused commands and keep most frequently used commands.

9.7.1.3 Robustness

A system is robust if it is *recoverable* after observing certain error in the previous operations and the user is able to rectify by another set of operations.

When a system gets into *fault*, the internal system state is *observable* through certain operations. The user can use this information and be able to rectify the fault. (e.g., a car stops mid-way on road. Certain data displayed through commands can help detect the fault).

Systems can get into faulty state when initial setting is faulty. Facility to set a *default initial state* helps the system to get out of faults.

When a system gets into a faulty state, a robust system should advise certain operations to reach into working state. The *reachability* information from the system makes the system robust. (For example, there are two digital watches A and B. Due to moisture, A's display gets blank. User cannot use it. B's display prompts to rotate the right button to rectify the problem. No doubt B is robust!)

9.7.1.4 Responsiveness

The expected response for user command or motor response should be sufficiently fast that the user should not feel sluggish. This varies from user to user. If the responsiveness is adaptable to user, the system is more usable.

9.7.1.5 Task Conformance

Take traffic signaling system is controlled from a remote control room. When the operator puts on green light, the system glows the green light. The traffic signal makes the green light to glow and should respond back that the light is on. A robust conformance occurs by detecting the lamp glowing by an optoelectronic device and returning back the signal. That makes 100% closed loop and perfect task completion conformance. Such type of task conformance is used in safety-critical systems.

9.7.1.6 Design Guidelines

Experts have stated certain golden rules and guidelines for design of highly usable systems. Some are Nielsen's 10 Heuristics (2020), Shneiderman's 8 Golden Rule (2000), and Norman's 7 Principles. Because of paucity of space only Norman's 7 principles is listed below.

Norman's 7 Principles (2009)

1. *Use both knowledge in the world and knowledge in the head.*
2. *Simplify the structure of tasks.*
3. *Make things visible: bridge the gulfs of execution and evaluation.*
4. *Get the mappings right.*
5. *Exploit the power of constraints, both natural and artificial.*
6. *Design for error.*
7. *When all else fails, standardize.*

9.7.2 Design Patterns

Design patterns are very popular in software architecture. Certain success stories which are widely accepted by the community are documented. Design patterns capture common properties from good examples of design. Not worth re-inventing the wheel. Use directly or augment from this solution. Same concept is adapted in human interface solutions.

9.8 Evaluation

Like software testing, the human interface has three facets of testing. The first one is functionality as per the specifications. The second one is usability of the interface. The third one is to study the effect of the interface on the user. Like software testing, the interface has to be evaluated at each phase of the design life cycle. Certain testing can be in the lab, some with users, and some in the field.

Evaluation goes entirely during design life cycle. The mistakes in the design phase are rectified at each design phase through feedback. Interface gets tested after the prototype or product is implemented. At this stage, overall tasks are verified and final interface feedback is generated for corrections before release.

When the product is released, true evaluation can be done by observing the users response, their suggestions, their comfort level, etc. But this phase is too time-consuming and costly. At this stage, certain cognitive testing is done by experts and predicts how users feel the product. Experts evaluate through cognitive walkthrough, heuristic, and review-based techniques.

9.8.1 Cognitive Walkthrough

The evaluation is done by experts using cognitive psychology. Experts walk through the design and identify potential usability problems. They verify from a checklist whether the interface violates any cognitive principles. Experts need a prototype of the system. Each task the user will perform on the system (workflow), the description of each workflow and the operator's traits like their knowledge, age, IQ, etc. In each walk through, expert considers whether it is learnable by the user, any impact on the user (fatigue, etc.).

9.8.2 Heuristic Evaluation

This method is proposed by Nielsen and Molich. They are called "heuristics" because they have broad rules of thumb and do not have specific usability guidelines. The evaluation is done by independent persons and rate in 5-point scale starting no issue to strongly rejected. The ten heuristics are given below:

1. **Visibility of system status:** At any stage of the workflow, user should be able to know to what extent he has done the action, what is the status of the system.
2. **Match between system and the real world:** The messages should be understandable to the user and should not be in terms of system. For example, *Error2403, class23 not found*. A better message would be "*Connectivity to server failed*."

3. **User control and freedom:** If the user gets into unwanted state and he wants to exit, certain mechanisms like “undo, back, exit” have to be provided.
4. **Consistency and standards:** Users are comfortable with the language of messages and consistency.
5. **Error prevention:** Workflow should prevent catastrophic errors to occur. Confirm whether the user wants to go ahead or not. For example, user wants to initialize the system to default state. All existing setup gets lost. This needs confirmation.
6. **Recognition rather than recall:** Users should have all the information needed for operation at this stage. He need not go back and remember data. For example, in mobile phones, you get an OTP to enter. Unfortunately, the message disappears from the current screen in few seconds. User has to go to messages and get the OTP, remember, and switch to the current application.
7. **Flexibility and efficiency of use:** Allow users to tailor frequent actions. For example, setting favorites on your TV remote is an example.
8. **Aesthetic and minimalist design:** Display content and dialog descriptions should be crisp enough to get user’s attention. Else the user’s get distracted. For example, some times when an exception error occurs, you get lot of information displayed, not relevant to user.
9. **Help users recognize, diagnose, and recover from errors:** Error messages should be crisp enough and help users to recover from problem.
10. **Help and documentation:** It is best if system is intuitively designed that the user need not require help document. Still, help should guide users to understand and rectify the problem.

9.8.3 Evaluation Through User Participation

If you have a working prototype and some users are ready to test it in the lab, user participated evaluation can proceed. Users can continuously operate the system without any distractions. The developers can observe their interaction with the system and assess themselves whether their (user’s) interaction is what they (developers) intended. This method is only alternative when the system cannot be tested in field.

9.8.4 Model-Based Evaluation

This method is a way to combine design specifications and evaluation into the same framework. GOMS model (goals, operators, methods, and selection) is a description of the knowledge that a user must have in order to carry out tasks on a device or system. Keystroke-level model is another model based on evaluation where the system is evaluated from the keystroke sequence.

9.8.4.1 GOMS Model

The acronym GOMS stands for goals, operators, methods, and selection rules (Zeepedia.com) Schrepp (2007).

- **Goals** are the functions what user wants to achieve.
- **Operators** are the sequence of actions user performs.
- **Methods** are a series of steps consisting of operators that the user performs.
- **Selection** means a way to select between competing methods.

GOMS stands for *goals–operators–methods–selection*. *Goals* are list of accomplishments user wants to acquire by operating the system (like move some text). *Operators* are the steps the user will execute to accomplish the goal (like select text–get context menu–select copy–move mouse–context menu–paste). The user may accomplish the goal through multiple *methods*. Using ctrl-c method or context menu by *selection* of one of the methods. GOMS analysis constitutes using this paradigm of goals, operators, methods, and selection.

A GOMS example is given below:

GOAL: Select a channel on TV by remote.

. [select GOAL: Channel number entry method;

. Enter first digit;
Observe in channel display;
Enter second digit;
Observe in channel display;
Enter third digit;
Observe in channel display.

GOAL: Select from favorites;

Press favorites button;
Scroll down till required channel is selected.

GOAL: select from menu;

Select menu by pressing menu button;
Select channel type by cursor movement;
Select the desired channel from list.]

For a particular user:

Rule 1: Select channel number entry method if he knows channel number.

Rule 2: Select favorites method if the channel is listed in favorites.

Rule 3: Use menu method otherwise.

9.8.4.2 Case Study-2

In any text editor you are acquainted, a portion of text can be copied and pasted by multiple work flows. Represent the same using GOMS model.

GOAL: copy and paste a block of text.

. [Select GOAL: USE-MENU-METHOD

. Goal: Select text block

1. Move mouse to beginning of text block;
2. Press left button;
3. Drag mouse till end of block.

Goal: Copy text block

4. Move mouse to main menu;
5. Select edit;
6. Select copy option.

Goal: Paste text block

7. Move mouse to the position where text block is to be pasted;
8. Move mouse to main menu;
9. Select edit;
10. Select paste option.

Select GOAL: Use mouse method

. Goal: select text block

11. Move mouse to beginning of text block;
12. Press left button;
13. Drag mouse till end of block.

Goal: Copy text block

14. Place mouse on selected text;
15. Press right mouse button;
16. Select copy option.

Goal: Paste text block

17. Move mouse to the position where text block is to be pasted;
18. Press right mouse button;
19. Select paste.

9.8.4.3 Keystroke-Level Model (KLM)

Keystroke-level model (KLM) is a quantitative approach to compute the efforts to complete a workflow (a sequence of operations) to achieve the goal. This helps in designing efficient workflows by comparing different possible methods. It calculates the efforts needed (in terms of time) to do each operation. An operation may constitute movement of mouse or hand, key press time, mental preparation to do an operation, perceiving result, etc. Depending upon the operation the effort needed is computed.

The model estimates the response from six factors.

Physical motor: (motoring operation by the user)

TK—time to strike a key;

TP—time to pointing a mouse or any other pointing device to a location;

TH—time to return to home position;

TD—time to draw.

Mental

TM—mental preparation for next operation;

System

TR—time for the system to respond.

Response times are empirically determined.

Execution time = TK + TP + TH + TD + TM + TR.

For the GOMS example given above, let us calculate the execution times for the first two methods. Assume desired channel is fourth in the favorite’s iteration.

Assume TK = 0.5, TH = 0.7, TM = 1 TR = 0.4 (in seconds).

Channel number entry method	Time
Enter first digit (TK + TR)	0.9
Observe in channel display TM	1.0
Enter second digit (TM + TK + TR)	1.9
Observe in channel display TM	1.0
Enter third digit (TM + TK + TR)	1.9
Observe in channel display	1.0
Total	7.7

Select from favorite’s method	Time
Enter favorite’s button (TK + TR)	0.9
Enter cursor up button (TM + TK + TR)	1.9
Enter cursor up button (TM + TK + TR)	1.9

(continued)

(continued)

Select from favorite's method	Time
Enter cursor up button (TM + TK + TR)	1.9
Enter cursor up button (TM + TK + TR)	1.9
Total	8.5

9.8.5 Case Study-3

You have to design the user interface for a motorized treadmill. User has to be provided below functions:

- Power ON and OFF.
- Start and stop the belt motion.
- Able to set any speed between 1 and 12 KMPH in steps of 0.1 KMPH.
- Quick selection of speeds.
- Safety key to stop belt motion in emergency.
- Display parameters: time, distance, no of laps, and calories.
- Able to select the display of one or all the above parameters.
- Continuously display pulse rate and speed.
- Monitor the running track through display.

Problem:

- Draw the interface on the given graph paper.
- Represent by GOMS model how you achieve the goal—"Start the machine to work out at 4.5KMPH and set the display to monitor time."
- Use key stroke-level model to compute the time for above goal

KLM parameters are given below:

K (Keystroking): 1 s;
 P (Pointing: Moving from one key to other): 1 s;
 H (Homing: Moving to power ON/OFF button): 4 s;
 M (Mental preparation): 1 s;
 R1 (system response to buttons): 0.2 s;
 R2 (system response to start): 5 s.

- b. Compute the cost of interface components.

Component cost in rupees is given below:

Led: 1;
 Seven segment LED display: 10;
 Seven segment LCD display: 5;

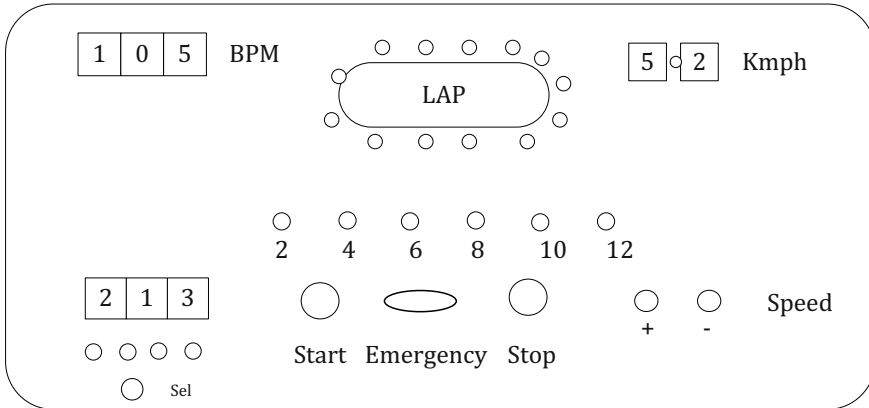


Fig. 9.10 User interface for a motorized treadmill

Push button: 2;
 Power on/off switch: 10;
 Assume some cost if not listed above.

Solution:

- Power button is back to the system. (not frequently used).
- Starting and stopping buttons control belt movement.
- Speed can be set by one click out of six values (reduce key strokes).
- Parameters to be displayed can be selected by *sel* button (to save space, cost, and avoid clutter.).
- Keyboard should not be used for speed test (because if one more digit is pressed, speed abruptly increases and the person falls).
- All treadmills have emergency stop which is strapped to the person (Fig. 9.10).

KLM

R2 = 5 (start)

M = 1 (mental prep)

P = 1 (pointing)

K + R1 = 1.2 (Speed 4 pressed)

5(K + R1) = 6.0 (Add 0.5 speed set)

3(K + R1) = 3.6 (Select parameter)

Total = 17.8 (20 s approx.)

9.9 Summary

This subject is multi-disciplinary starting with human physiology, cognitive science, psychology, systems hardware, and software.

From the brief coverage in this chapter, we summarize that the ultimate factor for success of a product is the usability. Several methods are discussed to improve this. Evaluating the interface through multiple methods is discussed for improving interface.

9.10 Further Reading

This chapter is a highly abridged version of human–computer interaction tuned toward interacting with embedded devices. Lots of books are available on HCI focusing on interfacing techniques with general-purpose computers. For a detailed coverage of this subject, please study reference (Dix et al. 2005), a Human Computer Interaction by Dix and Preece (1994) and Shneiderman (2000).

Advanced research is going on in implicit interfaces, pervasive and ubiquitous computing, and usability engineering.

9.11 Exercises

1. Why a message or event shown at the bottom of desktop is flashed or popped up? Why not it is a static message?
2. What is “visual acuity”? How is it dependent on luminance?
3. Why do most of CAD packages allow 5–10 colors for setting the color of a graphic object by default?
4. Why is it difficult to read text from a panel with rotating characters than a page with static text?
5. Numbers to be memorized are represented like this. “44-113-245-8920” Why?
6. Design appropriate user interface for below applications. Explain the system–user interaction for the workflows. You have to design a desktop/web application to know a train’s running status. Requirements are as given below:
 - i. One can select a train by its train name or its number.
 - ii. Able to know when it is going to arrive at any station in its route.
 - iii. Its actual schedule and deviations if any.
 - iv. Current position.
 - v. You may find multiple trains running at any instant which started on different dates from originating station.
 - vi. The interface can use standard user controls.
 - vii. Minimal clicks define the quality of design.

7. You are designing a mobile application to refuel your cooking gas. The constraint is that the app should have minimal clicks.
8. A system has to be designed with following requirements:
 - a. A grocery chain (GC) (say MORE or Reliance Fresh) wants to monitor the purchase pattern of customers ubiquitously for the benefit of customers and also GC.
 - b. GC has retail chain all over India.
 - c. All customers have to register to get benefits.
 - d. When customers enter any shop their purchase pattern and their interests based on the items they search are monitored.
 - e. Valued customers are identified intelligently.
 - f. Customers are given discount offers based on their value while they search for items. The offers are sent to the customers by SMS in real time.
 - g. GC builds knowledge base from all real-time events happening across retail chain.
 - h. Selected items are billed automatically by identifying all items in the basket.
 - i. Purchase patterns, inventory, etc. are updated in real time across GC.

You have to design all the blocks needed to implement the system.

9. You have to design a mobile app for geriatric needs. They are challenged with vision (blurred vision) and haptic (cannot identify a key and press). Can utter few words to call someone. The app should allow them to connect to their kin and talk. Apply your knowledge acquired till now in HCI and propose a solution. Show the user interface and Norman's model how the goal is achieved.

References

- arpost (2009) How augmented reality could revolutionize farming
- Dix A et al (2005) Human computer interaction, 3rd ed. Pearson Education
- Nielsen Norman Group (2020) 10 usability heuristics for user interface design
- Norman (2013) The design of everyday things. Basic Books
- Norman's 7 principles (2009): Google HCI-06129
- Preece J et al (1994) Human computer interaction. Addison-Wesley
- Schrepp M, Hardt A (2007) GOMS models to evaluate the quality of an user interface for disabled users. In: AAATE 2007 9th European conference for the advancement of assistive technology, 3–5 October 2007, San Sebastian, Spain, pp 646–651
- Shneiderman B (2000) Designing the user interface. Addison Wesley
- Zeepedia.com, Interaction: the terms of interaction, Donald normans model

Chapter 10

HW-SW Co-design



Abstract Traditionally, system design used to be done by hardware group and software group independently. The functional specifications used to be intuitively broken into hardware and software, and implementation used to proceed. Once the system is integrated, major problems used to rise in system integration. The worst-case scenario with such problems may even lead to re-do the complete system design again. Cooperative approach for design of HW and SW systems is well recognized now. This chapter will concentrate on the use of co-design in the development of embedded systems. In theory, several models and partitioning algorithms have been developed. Several benefits occur while adapting co-design strategy for embedded systems, viz., (a) it forces the developers to look into the problem in a holistic way; (b) design life cycle is well defined without surprises; and (C) reduces integration and test time. Current trend of designing system-on-chip needs co-design principles. Hardware–software partitioning is the critical activity in co-design. Major architectural decisions on the processor around which the system has to be designed and its interface to the hardware are important. The system partitioning problem is to allocate the components into partitioned subsystems. The partitioning challenge has major constraints of system cost, performance, size of the system, and power. The HW-SW partitioning problem is the process of deciding whether the required functionality is more advantageously implemented in hardware or software. This is a multivariate optimization problem which is NP-hard. Section 10.6 discusses basic partitioning approaches, viz., structural, functional, hardware oriented, and software oriented. Section 10.7 discusses important partitioning algorithms, viz., integer programming, hierarchical clustering, greedy partitioning, ratio cut, simulated annealing, and Kernighan-Lin algorithm. These are classified into constructive and iterative methods. In practice, a combination of constructive and iterative algorithms is often employed. To summarize, Chap. 3 is the basis for co-design where we studied exhaustively system-level modeling. Once a system is hierarchically broken into subsystems, modeled and analyzed, each subsystem’s functionality has to be transformed into architecture for implementation. It can be a hardware component like a processor, CDFG, GPU, etc. or a software module. A computation, modeled with fine grain parallelism, can be used both to develop software and to synthesize circuits.

Keywords Hardware–software partitioning · System partitioning · Structural partitioning · Functional partitioning · Software oriented partitioning · Hardware oriented partitioning · Hierarchical clustering · Greedy partitioning · Ratio cut · Simulated annealing · Kernighan-Lin algorithm

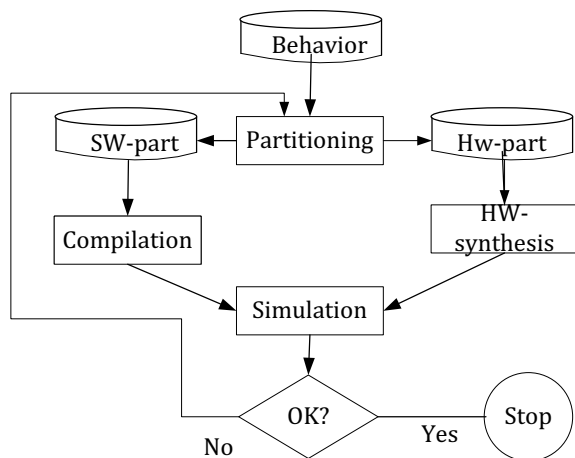
10.1 Introduction

This chapter deals with system-level design of embedded systems which constitutes both hardware and software components. The content explains how the systems after modeling and analysis are to be implemented with optimal partitioning of the functionality into hardware and software. This chapter covers algorithmic processes in hardware–software partitioning and optimal design decisions.

Top level design process for co-design is shown in Fig. 10.1. Section 10.5 explains the integrated co-design process which allows for incremental review throughout the design process, with interaction between hardware and software.

Figure 10.1 explains basic mechanism of hw-sw partitioning. When the system specifications are freezed and system behavior is modeled and verified, the system is ready to get implemented. We have seen all these phases in Chaps. 1, 2, 3, 4, 5, 6, 7 and 8. Now comes what portions are to be implemented in hardware and what portions in software. This is basically partitioning. Once a partition is freezed, proper hw-sw interfaces have to be defined for the behavior of the overall system design. This cannot be done after implementation because it will be very costly in terms of time and cost. So a partition has to be simulated and tested for the desired behavior. If the simulation results are satisfied, the partition is good and proceeds to real implementation. Else, it gets into next iteration.

Fig. 10.1 Conceptual hardware–software partitioning



Thus, the meeting of system-level objectives by exploiting the trade-offs between hardware and software in a system through their concurrent design with efficient partitioning is the basic objective of co-design. Hardware and software are developed at the same time on parallel paths concurrently. Interaction between hardware and software development will produce a design, meeting the performance criteria and functionality as specified.

10.2 Factors Driving Co-design

The major factor driving the need for hw-sw co-design is the fact that

- Most systems today include both dedicated hardware units and software units executing on microcontrollers or general-purpose processors.
- The increasing use of programmable processors being used in systems that formerly may have been all hardware.
- The availability of cheap microcontrollers for use in embedded systems and the availability of processor cores that can be easily embedded into an ASIC design.
- Increased efficiency of higher level language (C and C++) compilers that make writing efficient code for embedded processors much easier and less time-consuming.
- Increasing capacity of field programmable devices—some devices even able to be reprogrammed on-the-fly.
- Efficient tools for hardware synthesis capabilities.

Several benefits occur while adapting co-design strategy for embedded systems.

- Co-design forces the developers to look into the problem in a holistic way without partitioning the problem to hardware and software.
- Design life cycle is well defined without surprises.
- Growing complexity of embedded systems need this methodology for overall system performance, quality, design cycle time, reliability, and cost-effectiveness.
- Current trend of designing system on chip needs co-design principles.
- Design cycle time improves drastically because of less number of iterations.
- Take advantage of advances in tools and technologies.
- Reduces integration and test time.
- Support growing complexity of embedded systems.

Several technologies have evolved based on this theory, enabling co-design. Some are listed below:

- Hardware synthesis from high-level specification is possible with improved design automation tools.
- ASIC development allows complex algorithms to be implemented in silicon quickly and inexpensively.
- System-level development tools provide co-design philosophy.

10.3 Co-design Problems

The “best” solution for performing hw-sw partitioning and co-design depends on the type of system being designed. One type is any typical embedded system in manufacturing, control, defense, etc. The second type of systems is the ones having degree of flexibility for user programmability and customizing the system.

This leads to

- Co-design of embedded systems which are reactive systems with sensor inputs, control, and actuation.
- Co-design application-specific instruction set processors (ASIPs).
- Co-design of reconfigurable systems that can be personalized after manufacture for a specific application.

Components of the Co-design Problem

Drawing detailed system specifications is the major component to initiate co-design. There is no difference of this activity whether you go with co-design or not. The first few chapters on use cases and modeling deal with this activity. The choice of a model for co-design (or any other complex task) is a delicate balance between abstract and concrete. If the model is too concrete, the designer is constrained by low-level decisions even in the early phases of the design. On the other hand, if the model is too abstract, it may later become difficult to make an efficient realization. When doing co-design, the model should not favor a particular kind of realization, for example, a software realization.

Hardware–software partitioning is the critical activity in co-design. Major architectural decisions on the processor around which the system has to be designed and its interface to the hardware are important. The objective of partitioning should be clear in very early stages of co-design (see Fig. 10.2).

- partitioning to increase speed of execution;
- reduce latencies in task switching;
- reduce size of hardware and transfer the functionality to software without compromising the speed of execution;
- reduce overall cost of the system;
- is the partitioning manually or by automated tools?
- scheduling the execution of the system’s tasks to meet any timing constraints; and
- modeling the system throughout the design process to validate that it meets the original goals and functionality.

Scheduling the operations in hardware, scheduling the instructions in software, and scheduling the processes in RTOS are the scheduling activities in co-design.

Complexity issues and management in ES design

- The ES specifications are becoming complex with the system exploding into numerous tasks.

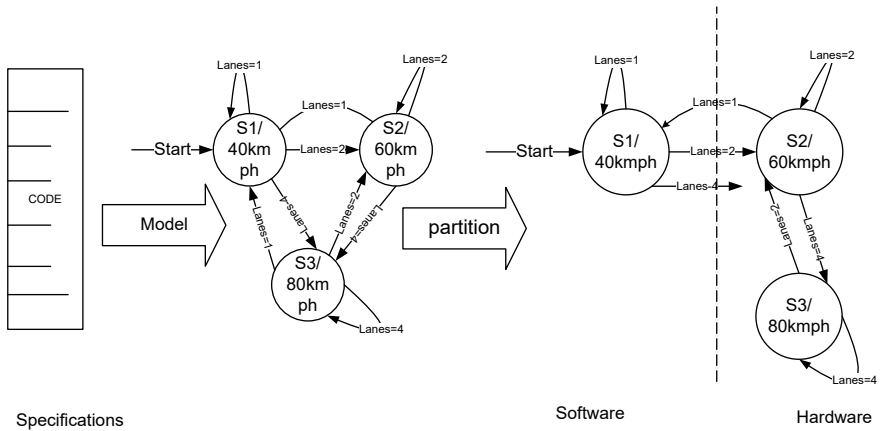


Fig. 10.2 Specifications to modeling to hw-sw partition

- System analysis, modeling, and analysis are exploding into exponentially complex hierarchy.
- Complexity management techniques are necessary to model and analyze these systems.
- Getting accurate realization of the system in the first iteration is becoming tough using conventional techniques.
- New issues rapidly emerging from new implementation technologies.

Some strategies to manage complexity in design are given below:

- Postpone as many decisions as possible that place constraints on the design.
- Apply abstractions and decomposition techniques.
- Develop incrementally through top-down design.
- Use executable system specification languages (ESL) as explained in earlier chapters.
- Apply partitioning interactively to achieve required specifications.

10.4 Conventional Model for HW-SW Design Process

Figure 10.3 represents conventional Hw-Sw design process which was published and standardized in department of defense, Standard 2167 (2009–2021).

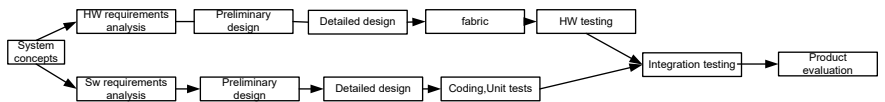


Fig. 10.3 Conventional model for hw-sw design process (courtesy DOD-std-2167A) (Defense software development STD-2167)

The requirements for hardware and software will be derived from system requirements and the processes go side by side without any interaction. The specification of the two components and interfaces will be designed at the beginning. The success of this model depends upon accuracy of system partitioning in the earliest part of the project. Else re-do from start. The separate development of HW and SW restricts the ability to study HW/SW trade-offs. Sometimes “Hardware-First” approach is often pursued assuming that hardware is specified without understanding the computational requirements of the software and software development does not influence hardware development and does not follow changes made to hardware design changes during its design process. With this type of process, problems encountered as a result of late integration can result in costly modifications and schedule slippage.

We normally assume with certain misconceptions like hardware and software can be acquired separately and independently, with successful and easy integration of the two later and hardware problems can be fixed with simple software modifications.

10.5 Integrated Co-design Process

Figure 10.4 shows one of the requirements for an efficient co-design process—an integrated substrate for modeling both the hardware and software and their interactions. The integrated modeling substrate allows for incremental review throughout the design process, with interaction between hardware and software. In this process, system specifications are the first phase. We have discussed in Chaps. 1, 2, 3, 4 and 5 the methodologies of system design. The next step would be partitioning of hardware and software. At this stage, certain architectural assumptions like the processor to be used, its architecture, and the interface are done. The basic objective of partitioning is to be clear whether it is being done for speedup, system size, cost of system, etc. It has also been decided whether the partitioning is to be done manually or using computer-aided partitioning tools.

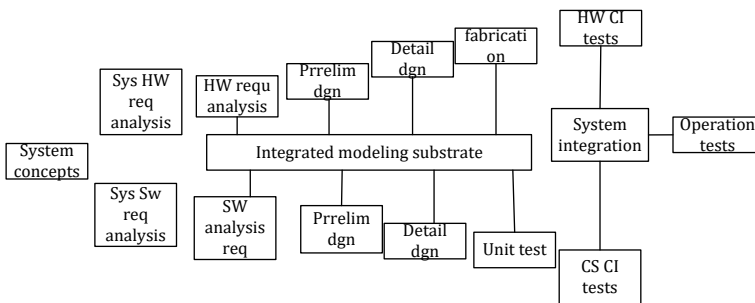


Fig. 10.4 Integrated co-design process (Courtesy Franke IEEE92] (Purvis and Franke 1992)

Specifically, hardware synthesis is done by transforming the partitioned hardware code through appropriate tool so that it becomes input to existing hardware synthesis tools (please correlate with System C in Chap. 4 for a detailed mechanism how it is implemented).

Automated system-level co-design tools should be selected in such a way that it allows un-biased and unified representation in hw-sw agnostic way during system-level analysis. It should allow system performance and evaluation at integrated design environment. It should allow certain modules to be transported to standard hardware synthesis tools. Also support performance with different partitioning options.

10.6 System Partitioning

Any embedded system is implemented on appropriate platform consisting one or more processor units, each having similar or heterogeneous architectures. The processor units may have local components like GPUs, ASICs, and memory banks all connected through local bus. The architecture may be with multiple processors connected to these components as global access over a bus. Whatever may be the architecture, we can assume each one as a component with all these components connected locally or globally through bus connectivity.

The system partitioning problem is to allocate these components into partitioned subsystems. At the same time, overall functionality of the system has to be partitioned and allocated to these components. The partitioning challenge has major constraints of system cost, performance, size of the system, and power. Partitioning is thus a centralized task.

The HW-SW partitioning problem is the process of deciding whether the required functionality is more advantageously implemented in hardware or software. This exercise is to be done for each major function to be implemented on the partitioned subsystem. The possible alternatives are too many. This is a multivariate optimization problem which is NP-hard. The goal is to achieve a partition that will give us the required performance within the overall system requirements like size, weight, power, cost, etc.

If the functionality is implemented in hardware, it provides higher performance via hardware speeds and parallel execution of operations. This is at the cost of additional hardware cost or design of specialized chips or GPUs or other types of accelerators. If the functionality is implemented in software, cost will be less for implementing but may incur costs of maintaining the software. This section discusses basic approaches of partitioning.

Partitioning approaches

Basic approaches are whether to do partitioning structurally or functionally.

Structural partitioning: If structural partitioning is done, the structure is freed and the functionality is mapped on to the structures. Structural partitioning is done by graph partitioning but functional partitioning is tougher.

Functional partitioning: If functional partitioning is done, the partition can be implemented in hardware or software. The system's functionality is described as collection of indivisible functional objects. Each system component's functionality is implemented in either hardware or software.

Software-oriented partitioning: Apply software-oriented approach where you start with all functionalities in software and move certain time critical portions onto hardware. This is software-oriented partitioning.

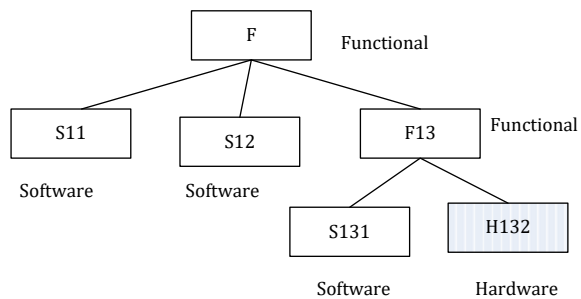
Hardware-oriented partitioning: Another approach is to start all functionalities in hardware and move portions to software when hardware implementation is not cost-effective, and there is no time criticality. This is hardware-oriented partitioning. Figure 10.5 shows how a complex problem at top level is partitioned into software (S11, s12) and another functional partition (f13). Functional partition F13 is broken into two leaves, one in software (s131) and one in hardware (H132). In the final partition, all the leaves will be implemented either in hardware or software. This shows how structurally the system is partitioned and finally each partition is assigned to software or hardware.

Binding is the mechanism of assigning the partitioned functional modules to hardware components. Certain functional partitioning is dependent on the overall architecture of the system. Some operations like numeric crunching are done in numeric co-processors. So the functional partition is tuned to the freezed hardware partition. When certain functionality is modeled suitable on CDFG model, it gets mapped closely to hardware. Thus, the functional module gets glued to the hardware structure.

Performance of a partition

- Let $HW(n)$ represent all functions marked for hardware implementation.
- Let $SW(m)$ represent all functions marked for software implementations.
- Let $Q(h)$ represent the cost of implementing all hardware partitions $HW(n)$. The cost can be in size and components.
- Let certain hardware components (h) and software components (s) form a group G and we have a constraint, C , so that this group has to execute the function within time T . This is our constraint.

Fig. 10.5 Functional versus HW-SW partitions



- Now the *performance satisfying partition* PSP (h, s) is the one which satisfies constraint C.
- The performance of the partition is the one which minimizes the cost Q out of all PSPs.

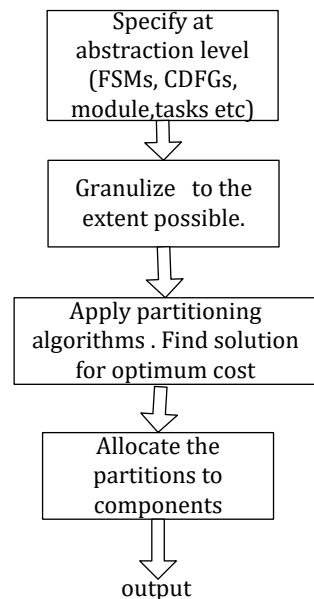
Basic partitioning process

Figure 10.6 represents basic partitioning process. Process starts with the abstraction-level analysis from the specifications, using modeling techniques we discussed in chapters earlier. Some are data flow graphs, finite state machines (FSM), hierarchical FSMs, CDFGs, etc. The specification is first decomposed into functional objects, which are then partitioned among system components.

Do this at certain granular level, so that the functionality is manageable. Coarse granularity means that each object contains a large amount of the specification. Fine granularity means that each object contains only a small amount of the specification. Fine granularity causes many more objects. You get more possible partitions and better optimizations can be achieved but at high computational costs. Apply partitioning algorithms with constraints and objective functionalities are defined. Speed and accuracy are competing goals of estimation.

Once partitioned, allocate them to the system components like processors, ASICs, etc. The set of selected components is called an allocation. Various allocations can be used to implement a specification, each differing primarily in monetary cost

Fig. 10.6 Basic partitioning process



and performance. Allocation is typically done manually or in conjunction with a partitioning algorithm.

Basic metrics of partitioning is performance and hardware size. Performance is generally improved by moving objects to hardware. Hardware size is generally improved by moving objects out of hardware. Computing accurate metrics is compute intensive. Heuristic methods, rough estimation by taking major components are different alternatives. Multiple metrics, such as cost, power, and performance, are weighed against one another. An objective function will be of the form

$$\text{Objfn} = k_1 * \text{area} + K_2 * \text{delay} + k_3 * \text{power}.$$

10.7 Partitioning Algorithms

Given a set of functional objects and a set of system components, a partitioning algorithm searches for the best partition, which has the lowest cost, as computed by an objective function. While the best partition can be found through exhaustive search, this method is impractical because of the inordinate amount of computation and time required.

A partitioning algorithm can be classified into two general categories. One is constructive algorithm which starts from decomposed functional objects. The partitioning is built and constructed by organizing or grouping these functional objects. The grouping of the objects is done on the defined closeness function. The computation time in a constructive algorithm is less because of small number of partitions.

Another class of algorithms belongs to iterative algorithm which will iteratively modify the initial partitioning in the hope to achieve a better result. Such algorithm uses an objective function to evaluate each partition, which yields more accurate evaluations than closeness function used by constructive algorithms. In practice, a combination of constructive and iterative algorithms is often employed. Basic partitioning algorithms are

Exact methods:

- Integer linear programming.

Heuristic methods:

Constructive methods:

- Random mapping.
- Clustering and multi-stage clustering.

Iterative methods:

- Kernighan/Lin algorithm.
- Ratio cut.

- Simulated annealing.
- Genetic evolution.

Some selected algorithms are described below.

Integer programming model

An integer program (IP) formulation consists of a set of variables $x_i, i = 1 \cdot N$ forming an integer expression and a set of constraints C_i and a single linear expression that serves as objective function O . An integer linear program is a linear program in which the variables x_i can only hold integers.

Let us look into a simple integer programming example, for the sake of understanding then we define the IP

Problem

Minimize $O = 5x_1 + 6x_2 + 4x_3$

Constraints: $x_1 + x_2 + x_3 \geq 2$ and $x_1, x_2, x_3 \in \{0, 1\}$,

where O is the objective function and we have to find the values x_1 to x_3 subject to constraints c_1, c_2 , and c_3 set above.

The constraints reduce the search space from 2^3 to 4 possible values as given below:

x_1	x_2	x_3	C
0	1	1	10
1	0	1	9
1	1	0	11
1	1	1	15

The optimum values are 1, 0, and 1 making the minimum value of 9.

Now defining the IP problem:

$$O = \sum_{xi \in X} a_i x_i \text{ where } a_i \in R \text{ and } x_i \in \{0, 1\} \quad (10.1)$$

$$\forall j \in J, \sum_{xi \in X} b_{i,j} x_i \geq c_j \quad (10.2)$$

Minimize objective function (10.1) subject to constraints (10.2).

Case study-1

A partitioning example

You have four tasks t_1 to t_4 each having different execution times on different processors. They have to be assigned on two processors p_1 and p_2 so that the execution time on the processors is equally balanced.

Execution times on p_1, p_2 are given for the tasks below:

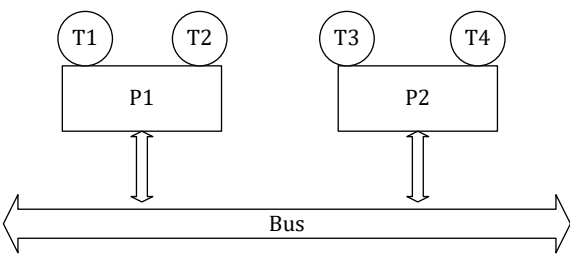
Task	P1	P2
T1	5	10
T2	15	20
T3	10	10
T4	30	10

Below is the combination of task assignments on different processors. Each cell shows the task-processor assignment and cost in brackets. Overall cost is shown on rightmost column. From the table you find, the first assignment (t1, t2 on P1 and t3, t4 on p2) balances.

	T1	T2	T3	T4	Cost
P1	1(5)	1(15)	0	0	20
P2	0	0	1(10)	1(10)	20
P1	1(5)	0	1(10)	0	15
P2	0	1(20)	0	1(10)	30
P1	1(5)	0	0	1(30)	45
P2	0	1(20)	1(10)	0	30
P1	0	1(15)	1(10)	0	25
P2	1(10)	0	0	1(10)	20
P1	0	1(15)	0	1(30)	45
P2	1(10)	0	1(10)	0	20
P1	0	0	1(10)	1(30)	40
P2	1(10)	1(20)	0	0	30

As you see from Fig. 10.7, the search space increases with the size of the problem. It is NP-complete problem. But problems of some thousands of variables can still be solved with commercial solvers (depending on the size/structure of the problem) or heuristic algorithms.

Fig. 10.7 Task assignment using processors using ILP



Hierarchical clustering

Clustering is a technique that groups similar data points such that the points in the same group are more similar to each other than the points in the other groups. The group of similar data points is called a cluster. Clustering is done in two types: agglomerative and divisive (see Fig. 10.8). In agglomerative clustering, initially, each data point is considered as an individual cluster. At each iteration, similar clusters merge with other clusters until one cluster or K clusters are formed.

Basic algorithm is given below:

- Compute the proximity matrix.
- Let each data point be a cluster.
- Repeat: Merge the two closest clusters and update the proximity matrix.
- Until only a single cluster remains.

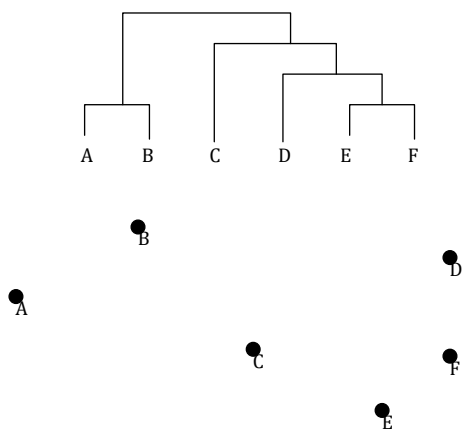
Figure 10.9 is a diagram showing the points to be clustered. The distances can be roughly estimated by the diagram: A, B clusters first; EF clusters next (EF); D is close to second cluster(EFD); and C is close to (EFD). Hence, clusters to (EFD C). Final cluster is (AB) to (EFD C).

Divisive clustering is the reverse process. Not much popular. We consider all the data points as a single cluster. In each iteration, we separate the data points from the cluster which are not similar. Multi-stage clustering is extended concept where hierarchical clustering is started with one metric and then continues with another metric. Each clustering with a particular metric is called a stage.

Greedy partitioning

Greedy partitioning is a heuristic method. You start with an initial partition. Compute the cost function on the parameters of your interest. Move the nodes from one partition to another heuristically till you go on gaining improvement in cost function. There

Fig. 10.8 Agglomerative clustering



is high probability, you get stuck at local minima or local maxima based on you are minimizing or maximizing the objective function. Also called as hill climbing or gradient descent search. Depends on start point (see Fig. 10.11). One alternate to improve the algorithm is the movement of objects between groups depending on whether it produces the greatest decrease or the smallest increase in cost. To prevent an infinite loop in the algorithm, each object can only be moved once.

In the example shown in Figs. 10.9 and 10.10, the cost function is number of connections across the two partitions. In Fig. 10.9, cost is 5. We have to minimize the cost. We swapped E and J into opposite partitions, respectively. The connections are now 4 and reduced cost by 1. This is iterative process.

Ratio cut

Given a graph $G = (v, E)$, partition into disjoint U and W such that $e(U, W)/(|U| \cdot |W|)$ is minimized. The ratio cut metric intuitively allows freedom to find natural partitions: the numerator captures the minimum-cut criterion, while the denominator favors an even partition.

The metric is $ratio = \frac{cut(p)}{size(p1)*size(p2)}$,

where $cut(p)$ = sum of weights of crossing edges and
 $size(p)$ = size of p .

The ratio metric balances the competing goals of grouping objects to reduce the cut size without grouping distance objects. Based on this new metric, the partition algorithms try to group objects to reduce the cut sizes without grouping objects that are not close.

Simulated annealing

Please refer to Fig. 10.11 where you are trying to reach the peak. Unfortunately when you do incremental trials at any point, you always take best value and proceed to next position. You will proceed till the point where all trials at that place give lower than the current value. You assume you have reached the peak. But it did not reach but stuck at local maxima. How can this be avoided to reach peak?

The concept of simulated annealing from metallurgy is used. The principle is a “structured” lattice structure of a solid is achieved by heating up the solid to its melting point and then slowly cooling down until it solidifies to a low-energy state. The concept states that

Fig. 10.9 Before swapping nodes E and J

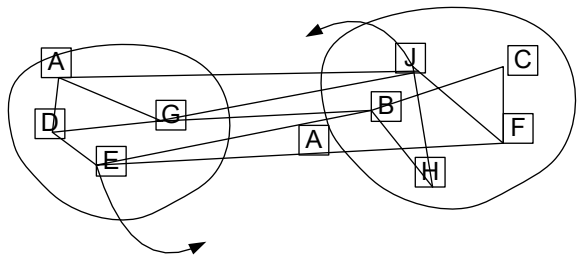


Fig. 10.10 After swapping E and J

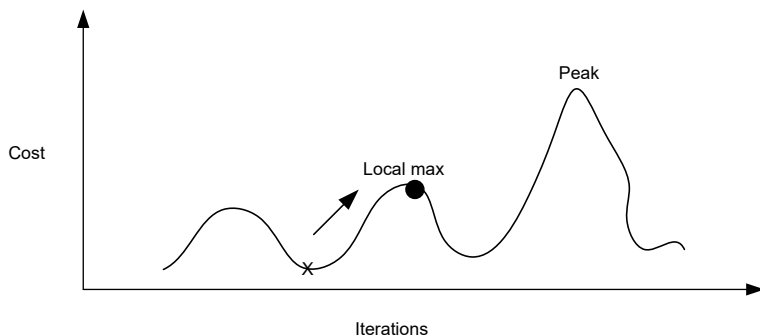
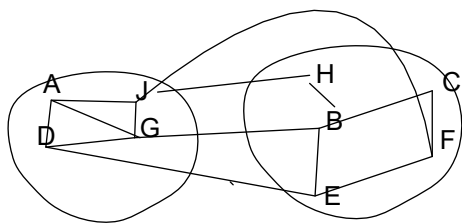


Fig. 10.11 Stuck at local peak while hill climbing

There is a non-zero probability that a particle “jumps” to a high-energy state ($e_{i+1} > e_i$)

$$p(e_i, e_{i+1}, T) = e^{\frac{e_i - e_{i+1}}{k_b T}},$$

where

p = the probability of jumping to higher energy state;

e_i = current energy;

e_{i+1} = next energy state; and

T = temperature.

By analogy with the physical process, replace the current solution with a nearby solution which reduces your objective function. Do this with certain probability (initially assume high temperature). This causes you to slip the peaks. Gradually decrease the probability of accepting worse cost (cooling) so that you look for better solutions. When T is large, selecting next solution is random but increasingly selects the better cost solution as T goes to zero. The process is

- Start with initial solution P randomly.
- Choose a random solution around P .
- Reduce temp(T).
- Repeat till $T = 0$ or finite number of iterations or no improvement.

Kernighan–Lin algorithm

- Make an initial partition of the objects.
- From all possible pairs, find best pair (reducing the cost) and regroup.
- From the remaining regroup best pair.
- Till all objects are paired.
- Repeat till there is no decrease in cost.

Case study-2

This heuristic can be best illustrated by taking a simple example (see Fig. 10.12), which shows a simple circuit with six objects. When you partition, the major communication costs are (a) across the partition (external costs) and (b) internal costs within the partition. Our objective is to reduce overall costs. Intuitively you can find the good partition of below circuit by partitioning D4 and d2. Let us see this algorithmically.

Let the communication costs from node X to Node Y $C(x, y)$ be as given below. We assume they are all constant in this example. Remaining node-to-node costs are zero as they are not directly connected.

Node to node	Communication cost
$C(5, 6)$	1
$C(5, 4)$	1
$C(6, 4)$	1
$C(4, 2)$	1
$C(2, 1)$	1
$C(2, 3)$	1

Let
 $Ext(i)$ = external communication costs of node i across partitions.
 $Int(i)$ = internal communication costs of node i across partitions.

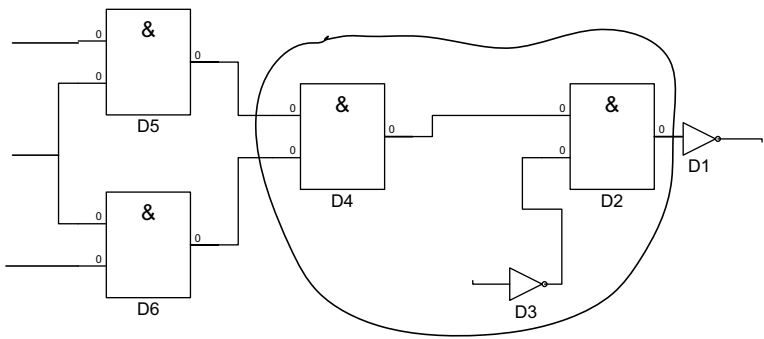
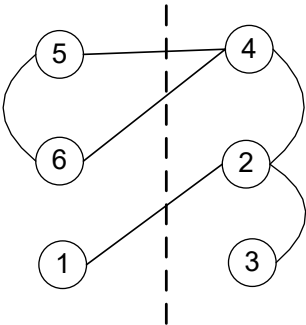


Fig. 10.12 A simple circuit with six objects to be partitioned

Fig. 10.13 Communication across partition



$D(i)$ = desirability to move node across partition to reduce communication costs across partition. $D i = Ext(i) - Int(i)$.

$gain (i, j) = D(i) + D(j) - 2*c(xi, j)$. $G (i, j)$ is the gain due to change in cut costs. If gain is high, i and j are highly desirable nodes for swapping.

Figure 10.13 shows the internal and external communication among the objects when partitioned as shown in Fig. 10.12. Let us now compute the costs $D_i, i = 1.0.6$. Assume external and internal communication. Cost for each edge is 1.

Obj	Ext(i)	Int(i)	$D(i) = Ext(i) - Int(i)$
1	1	0	1
2	1	2	-1
3	0	1	-1
4	2	1	1
5	1	1	0
6	1	1	0

Compute gains by regrouping.

Pair	$D(i)$	$D(j)$	$2C(x_i, x_j)$	$Gain = D(i) + D(j) - 2C(x_i, x_j)$
2, 1	-1	1	2	-2
2, 5	-1	0	0	-1
2, 6	-1	0	0	-1
3, 1	-1	1	0	0
3, 5	-1	0	0	-1
3, 6	-1	0	0	-1
<u>4, 1</u>	<u>1</u>	<u>1</u>	<u>0</u>	<u>2</u>
4, 6	1	0	2	-1
4, 5	1	0	2	-1

From Tables 4 and 1, give maximum gain $g (4, 1) = 2$. So swap 4 and 1 across partitions. Exclude 4 and 1 from the respective groups and re-compute D and gain.

Re computing D(i).

Obj	Ext(i)	Int(i)	D(i) = Ext(i) – Int(i)
1	0	1	–1
2	1	2	–1
3	0	1	–1
4	1	2	–1
5	0	2	–2
6	0	2	–2

Pair	D(i)	D(j)	2C(xi, xj)	Gain = D(i) + D(j) – 2C(xi, xj)
2, 5	–1	–2	0	–3
2, 6	–1	–2	0	–3
3, 5	–1	–2	0	–3
3, 6	–1	–2	0	–3

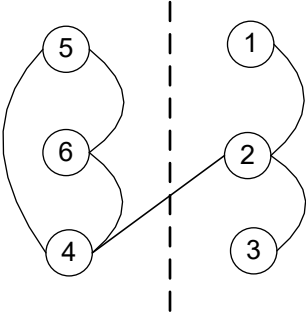
All values are –3. Arbitrarily pair (3, 6) and $g_2 = -3$. Remaining in the set are {2} and {5}.

Re-computing $D_2 = 1, D_5 = 0$.
Gain = $D_2 + d_5 - 2c(\times 2, \times 5) = 1 + 0 - 0 = 1$.
So $g_3 = 1$.

Out of three interactions $g_1 = +2$ with pair {4, 1}; $g_2 = -3$ with pair {6, 3}; and $g_1 = 1$ with pair {5, 2}.

So best partition is with first iteration. New partitions will thus be {5, 6, 4} and {1, 2, 3} as shown in Fig. 10.14.

Fig. 10.14 Partition remaining nodes



10.8 Summary

As the complexity of embedded systems is increasing, time-to-market constraints, strict cost margins, design time, and quality are becoming challenging. This leads to system-level design aborting old concept of HW and SW design separately. Chapter 3 is the basis for co-design where we studied exhaustively system-level modeling. Once a system is hierarchically broken into subsystems, modeled and analyzed, each subsystem's functionality has to be transformed into architecture for implementation. It can be a hardware component like a processor, CDFG, GPU, etc. or a software module. A computation, modeled with fine grain parallelism, can be used both to develop software and to synthesize circuits (Staunstrup and Wolf 1997).

During the co-design process, the designer will go through a series of well-defined design steps that include partitioning, scheduling and communication synthesis, which form the synthesis flow of the methodology. Allocation defines how the subsystems are allocated to available library of components. System partitioning problem is to allocate these components into partitioned subsystems. The partitioning problem is to find optimal partition such that design is implemented at optimal costs and meets other constraints.

Hardware–software co-synthesis tools (SystemC, System Verilog) provide system-level design which is then partitioned and synthesized on hardware synthesis tools like VHDL and programming languages.

Partitioning algorithms are fundamentally linear or combinatorial optimization techniques. Most of them are NP-complete problems. System partitioning at structural, functional hardware/software apply these techniques for efficient implementation. Because of complexity of the algorithms, heuristic methods are adapted in most of the cases. Readers will get further experience by practically experiencing with co-design tools and practice.

10.9 Further Reading

For in-depth understanding of this subject, readers are advised to go through the book by *Hardware/Software Co-Design: Principles and Practice* by Jorgen Staunstrup (1997), DeMicheli (1996, 2002), Gajski (1994), Kumar (1995). Similar to other topics, hands-on experience by working real-world projects using co-design tools would be very helpful.

10.10 Exercises

1. Write a SystemC code for the design of a 4-bit full adder using 2×1 multiplexers. Write a single module for 2×1 multiplexer and later instantiate the multiplexer module for construction of full adder. Map the co-design into hardware and software partition alternatives.
2. A portable iris and fingerprint scanner has to be developed. It has the following features:
 - a. Iris scanning.
 - b. Fingerprint scanning.
 - c. Identification of iris and fingerprint.
 - d. Data security/encryption.
 - e. Mobile data network connectivity.
 - f. Power management.

Assume each feature to be a process or task. Draw a data flow graph of the system and design the system.

Performance requirement and constraint for each feature are: task	Performance constraint (Min required)	Resource constraint
Iris scanning	500 MIPS	Hardware or software
Fingerprint scanning	400 MIPS	Software only
Identification of iris and fingerprint	20 MIPS	Hardware or software
Data security/encryption	450 MIPS	Hardware only
Data transmission over mobile network	100 MIPS	Hardware only
Power management	50 MIPS	Software only

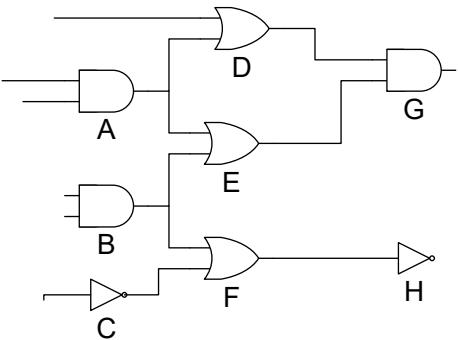
Off-the-shelf components can be used to design the system. Components available in the system are characterized by cost performance and power as given below:

Microprocessor 1: 500, 1000 MIPS, 5 watts;
 Microprocessor 2: 200, 700 MIPS, 10 watts;
 DSP 1: 200, 500 MIPS, 16 watts;
 DSP 2: 800, 1000 MIPS, 20 watts;
 FPGA 1: 100, 200 MIPS, 8 watts; and
 FPGA 2: 200, 500 MIPS, 12 watts.

Design an optimal configuration of the system. Provide an appropriate hardware and software partitioning with suitable scheduling of different processes. Also draw cost-time (MIPS) graph for the designed partition.

3. Obtain an optimal partition for the digital circuit, given below, using Kernighan–Lin algorithm. Assume all edges to be unit weight (Fig. 10.15).

Fig. 10.15 Digital circuit



- 4. Draw a co-design FSM (CFSM) for a simple seat-belt alarm system of a car that has the following specification:
“Five seconds after the key is turned on, if the belt has not been fastened, an alarm will beep for ten seconds or until the key is turned off.”
- 5. It is proposed to design a coffee vending machine using the co-design approach. While an 8086 microprocessor is given to you, what other components would you need for the design? Assuming they are available, draw the data flow graph (DFG), architecture graph, and specification graph for this design.
- 6. Architecture of available components and task graph of system is shown in figures below, respectively.
 - (a) For allocation constraint given by Table 10.1 construct a *data flow graph* by inserting communication nodes in task graph, draw the *architecture graph* for corresponding architecture, and illustrate all possible mapping by *specification graph*. Are there any other constraints apart from that mentioned in Table 10.1? If so what are they and how can those constraints can be solved (Fig. 10.16)?
 - (b) If the task allocation is restricted by Table 10.1, then explore all possible design points for the component and task set given by Table 10.2, plot time versus power graph and obtain pareto-optimal set.

Table 10.1 Task allocation constraint

Task	Component				
	ARM	ASIC	FPGA	RISC	DSP
T ₁	✓			✓	✓
T ₂		✓			
T ₃	✓			✓	
T ₄		✓	✓		✓
T ₅	✓			✓	

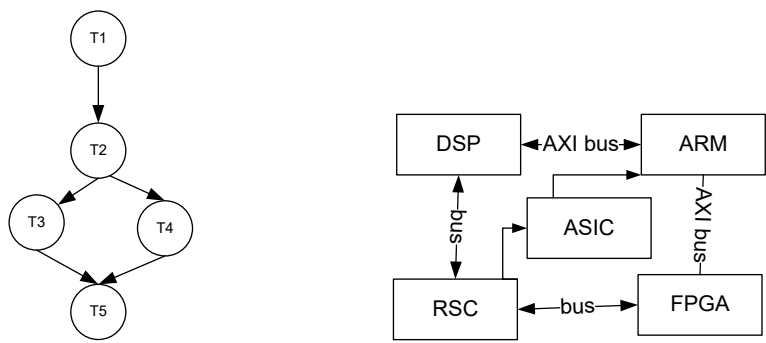


Fig. 10.16 Tasks and component architecture

Table 10.2 Estimation of task metrics for individual component

Task	ARM			ASIC			FPGA			RISC			DSP		
	a _s	τ	P	a _h	τ	P	a _h	τ	P	a _s	τ	P	a _s	τ	P
T ₁	10	3	1	20	2	8	30	3	12	10	4	3	15	8	2
T ₂	50	13	1	10	4	6	20	4	10	70	12	8	30	15	12
T ₃	8	6	3	30	5	10	40	10	15	10	10	5	35	9	7
T ₄	35	16	1	6	1	4	10	2	8	30	20	4	15	10	5
T ₅	5	2	8	16	5	3	11	6	6	4	2	3	22	14	17

References

Defense software development STD-2167 (2009–2021)

DeMicheli G, Sami M (1996) Hardware/software codesign. NATO ASI Series E, vol 310

DeMicheli G, Ernst R, Wolf W (2002) Readings in Hw/Sw co-design. M. Kaufmann

Gajski DD, Vahid F, Narayan S, Gong J (1994) Specification and design of embedded systems. Prentice Hall

Jerraya AA, Mermet J (eds) (1999) System level synthesis. Kluwer

Kumar S, Aylor JH, Johnson BW, Wulf WmA (1995) The codesign of embedded systems. Kluwer

Marwedel P, Embedded system design. Springer

Purvis MK, Franke DW (1992) An overview of hardware/software co-design. IEEE

Schaumont PR, A practical introduction to hardware/software codesign. Springer

Staunstrup J, Wolf W (1997) Hardware/software co-design: principles and practice. Kluwer Academic

Chapter 11

Energy Efficient Embedded Systems



Abstract Millions of embedded systems are hand held devices like mobiles, PDAs, remote controllers, audio systems, digital cameras, and son. They are battery operated. They are smart devices with rich functionality. Consumers now need high-performance and low-power consuming devices. Both the requirements are contradicting. Optimal design with contradicting requirements is challenging. Legacy methods of providing stable power to the devices are no more the way to design systems as on date. Intelligent techniques have to be implemented at each level in hardware, firmware, operating system, and applications to control the power consumption. This topic needs fundamental understanding of the power dissipation at transistor level. In Sect. 11.2, methods to optimize the dissipation without compromising the performance of the systems have to be developed. In this direction *dynamic voltage and frequency scaling* (DVFS) technique is getting successfully implemented in modern systems as described in Sect. 11.3. The idea behind DVFS is to scale the supply voltage and operational frequency to the performance requirements of the application at that instance. Thus energy dissipation is reduced to the appropriate level. Section 11.4 gives a brief overview of energy-aware real-time schedule algorithms by adding energy as another dimension in the optimization. While the above technique is the theoretical base for energy management and power management in the embedded systems, this has to be at each layer (i.e.,) hardware, BIOS, firmware, OS, and applications. *Advanced Configuration and Power Interface* (ACPI) is the specification evolved by Intel, Microsoft*, Toshiba*, HP*, and Phoenix* in the mid-1990s for power management, device discovery, and configuration. This chapter discusses in detail the concept of DVFS and the methods to implement, specifically in real-time embedded systems. We will discuss the detailed specification of ACPI and the implementation methods in Sect. 11.5.

Keywords Dynamic voltage and frequency scaling (DVFS) · Advanced Configuration and Power Interface (ACPI) · Activity allocation · Activity mapping · Activity scheduling · Power efficiency · Dynamic power management (DPM) · Dynamic voltage scaling (DVS) · Energy aware scheduling · Static DVFS scheduling · ACPI system

11.1 Introduction

During this chapter, we will only focus on embedded systems where power efficiency has to be optimized for good battery life. Most of the hand held devices, remotes; portable equipment comes into this category. One immediate solution which strikes to our mind is to optimize the hardware by minimizing the components and select components fabricated with low-power consuming technologies like CMOS. In fact, the solution lies at system-level design. To be more precise, it depends upon co-design strategies.

Given the application tasks, allocating the tasks to hardware or software, mapping the hardware tasks to appropriate components, mapping software tasks to appropriate processors finally leads to desired metrics (viz.,) performance, power consumption, and cost. It is a major optimization problem. This can be more elaborated with a real-life problem. One man has to reach a destination 3 km away in 20 mts. His objective is to complete the task with minimal cost, with minimal physical energy and reach in-time. One option he selects is to hire a motor and reach in 5 mts. Achieved at high cost. He is idle at the destination by 15 mts. Scheduling is bad. Next option is to hire a bicycle at lower cost and reach in 10 mts. He is still idle for 10 mts at the destination. Next option is to run for 3 km and reach by 15 mts. Still he is 5 min ahead. The next option is run for 1.5 km and walk remaining distance coolly and reach by 20 mts (just in time). Now see the cost, performance, energy, and real-time requirements. The last option is energy efficient, less cost and complies with the real-time deadlines.

The same applies to embedded systems. Map the problem to correct resources (hardware/software), define the speeds at which they should work, exploit idle times and slack times to work slow at less energy and meet the schedules. Idle and slack times are utilized by switching off the related components or reducing their performance to save energy. Thus energy management involves all the design aspects covered in the previous chapters specifically architecture allocation, application mapping, co-design, activity scheduling, and energy management.

11.1.1 Activity Allocation

Figure 11.1 illustrates the importance of selecting proper architecture keeping the cost, performance, energy, and real-time requirements in view. The system has major computational task dedicated to the ASIC and image processing task dedicated to

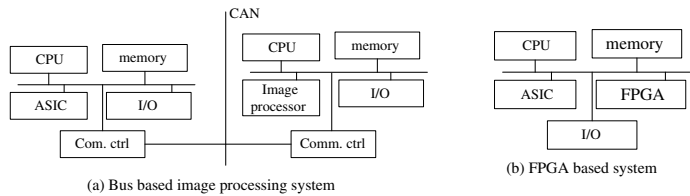


Fig. 11.1 Activity allocation

another processor-based unit. Both communicate over CAN bus. In alternate (a) allocating image processing task to separate unit may increase performance but energy costs due to additional hardware increases. Alternate (b) may have little lower performance but power requirements drastically reduce. Thus selecting the appropriate system components, in order to balance between these trade-offs, is of utmost importance for high-quality designs.

11.1.2 Activity Mapping

Mapping the functional activities to hardware or software plays a major role in energy management in addition to the requirements discussed in co-design chapter (see Fig. 11.2). When the software is allocated to different processors, the power consumptions of each processing unit must be taken care off. Inappropriate mapping of activities to hardware and software leads to poor utilization of the system. This was discussed elaborately in co-design chapter without considering energy aspects.

11.1.3 Activity Scheduling

We have studied task graphs in real-time systems chapter where the jobs are scheduled based on the precedence constraints and the timing constraints are satisfied (see Fig. 11.3). One can find one or more valid schedules. One can select an optimal schedule where the energy for overall execution is less.

Fig. 11.2 Activity mapping

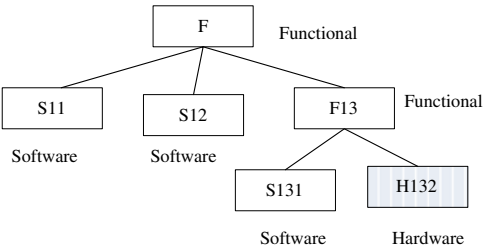
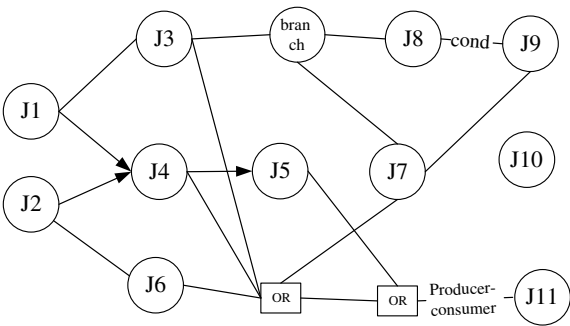


Fig. 11.3 Activity scheduling



11.1.4 Energy Management

Our goal is not to dissipate energy for no purpose. As in the real-life example stated above, one cannot run to the destination and wait at the destination. Rather he can walk dissipating lesser energy!! Same applies to tasks scheduled. We have discussed idle times and slack times in Chap. 6 (see Fig. 11.4a). For the given jobs scheduled, the processor has certain idle times where no job is scheduled. During this time, the concerned resources can be either put off or kept in low-power state, thus conserving energy.

Similarly observe the slack time in Fig. 11.4b. The deadline of the task is 40 units after its release. However the tasks execution time is 30 units. So the task has a slack time of 10 units. This time can be effectively utilized by reducing the speed of execution and utilize the slack time. This saves energy.

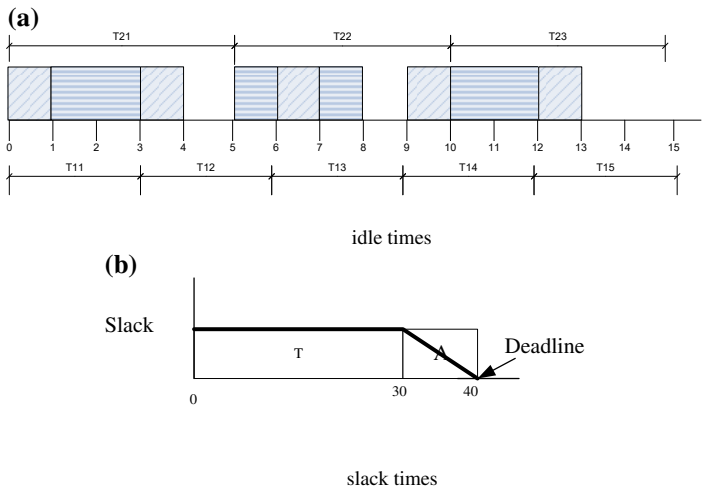


Fig. 11.4 a Idle times, b slack times

11.2 Energy Dissipation in Devices

Figure 11.5 is a simple inverter. When power is on, currents flow across the circuit causing static power dissipation. Static power is caused due to leakage currents and the bias power. $P(\text{static}) = P(\text{leakage}) + P(\text{bias})$. $I(\text{leak})$ is the leakage current which consists of both the subthreshold current and the reverse bias junction current in the CMOS circuit. Leakage current increases rapidly with the scaling of the devices. It becomes particularly significant with the reduction of the threshold voltage.

$$P(\text{leak}) = I(\text{leak}) * V.$$

The dynamic power consumption is due to short circuit power $P(\text{sc})$ and switching power $P(\text{sw})$. Out of these, switching power is very high and other power components can be neglected.

Static power $P(\text{static}) = P(\text{leakage}) + P(\text{bias})$ and

Dynamic power $P(\text{dyn}) = P(\text{short}) + P(\text{switch})$

$P(\text{total}) = P(\text{leakage}) + P(\text{bias}) + P(\text{short}) + P(\text{switch})$.

Short circuit power $P(\text{short})$ occurs for very small instance of time during switching of the two complementary transistors into ON state. For a short time both will conduct before settling to complimentary states. This causes a short circuit and high current flows across both.

Switching power is dissipated due to charging and discharging of the load capacitance of the output circuit. The power dissipation occurs when the output switches to high. The capacitance has to charge. The power for one charge is

$$P_{sw} = ic(t)V_{dd}$$

where $ic(t) = C \frac{\partial v}{\partial t}$.

Fig. 11.5 Inverter circuit

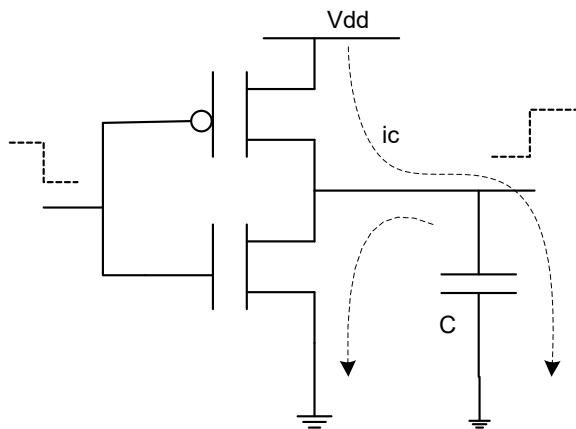
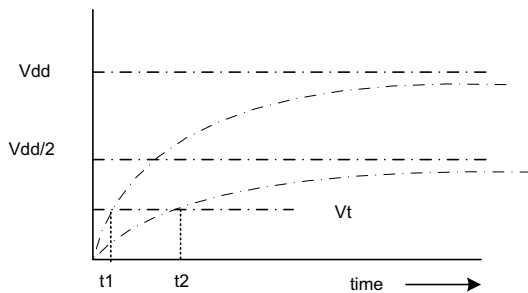


Fig. 11.6 Switching delay by reducing Vdd



Energy dissipated in one cycle is $E(sw) = Vdd \int_0^T i_c dt = CVdd^2$.

If a task needs N clock cycles energy dissipated $E = NCV_{dd}^2$.

Power dissipated for the task is $P = E/NT = kfCV_{dd}^2$ where f = frequency of the switching.

Hence, power dissipated is proportional to the switching frequency and square of supply voltage (because the load capacitance cannot be controlled). We have to play with frequency and Voltage only.

From the above equation, we can deduce that power dissipation can be reduced by reducing frequency and also voltage. This is true. But the energy dissipated for a task remains the same when frequency is reduced. As an example, assume a task needs 20 ms to complete on a processor at 100 MHz clock. Let the power dissipated is 5 mw. The energy consumed is 0.1 mJouls. Let us reduce the frequency to 50 MHz. the task takes 40 ms now. Power consumed is 2.5 mw but energy consumed is $2.5 \times 40 = 0.1$ mJouls. Hence the only way to reduce energy consumption is reducing Voltage (Fig. 11.6).

Now, let us study the effect of reducing Vdd. When supply voltage is reduced, the time for the gate voltage to reach the threshold to switch increases. Hence the transistor's switching time increases. Response of the system gets reduced. Effectively voltage scaling is trade-off between delay and energy.

11.2.1 Power Efficiency

When a device is powered, it consumes power at no load (i.e.,) the system is not utilized and when it is utilized to deliver useful output. The power efficiency of the device is the ratio of power utilized to deliver useful output to the total power consumed. A system is to be designed such that it has good power efficiency at any state. This is done through proper power management scheme.

Figure 11.7a illustrates an example with the device consuming 50% of peak power at no load. Thus, at 0% load, it consumes 50% of peak power and has an efficiency of 0%. As the system delivers 30 units of load total power consumed is 45, thus efficiency is 66%. As illustrated in the figure, the system is highly inefficient over

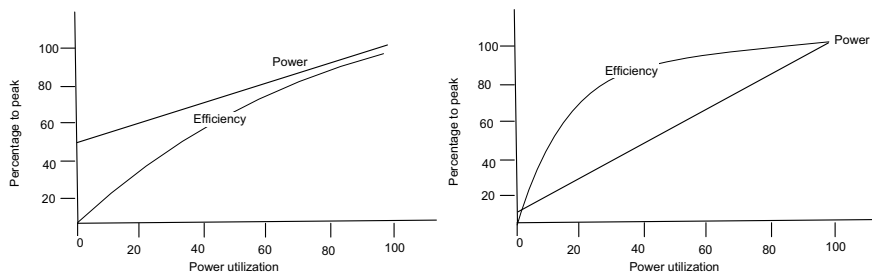


Fig. 11.7 Power efficiency

most of its operating range and does not achieve 80% efficiency until utilization rises to 70%.

Figure 11.7b illustrates an example where the power consumption at no load is 10% of peak power. But it reaches to good power efficiency even at 20 units of load consuming 28 units of total power and the efficiency is about 70%. The system reaches 90% efficiency at 50% of utilization.

11.3 Techniques for Energy Minimization

Based on the semiconductor behavior discussed above, two energy minimization techniques have gained considerable attention. They are dynamic power management and dynamic voltage scaling.

11.3.1 Dynamic Power Management (DPM)

The main strategy in DPM is to shut down the component when it gets into an idle state. A little advanced approach is to predict the system behavior, identify the component state and make it off when it will not be possibly used in near future. The later strategy works well if the prediction algorithm is accurate. Also depends upon the energy costs involved to wake up sleeping components to active state. This technique is used earlier as Advanced Power Management (APM) in most of the systems, particularly in laptops. The power management is done at BIOS level. Operating system is not aware of what APM does. APM observes device activities and determines when to transit them into low-power states and back to active state.

11.3.2 Dynamic Voltage Scaling (DVS)

The concept of voltage scaling and delay trade-off discussed above is termed as Dynamic Voltage Scaling. DVFS implemented systems dynamically vary the supply voltage and the frequency depending on the context and minimize unwanted energy dissipation. Also the technique is termed as **DVFS**. This process continues dynamically during run time of the application (see Fig. 11.8).

Supply voltage is controlled by DC to DC converter and the control signals are fed from processor-based logic. Similarly the clock frequency is controlled by Voltage controlled oscillator. The control signals are fed by processor controlled logic.

While DPM and DVFS have the same goal of energy optimization, DPM looks simpler to implement. DPM simply switches off system during idle times whereas DVFS controls the supply and frequency using complex control. Taking a simpler example, Fig. 11.9 there is a periodic task having 10 ms idle time periodically after executing 40 ms. DPM saves energy by putting off the system for 10 ms. In case of DVFS, system knows that there is a slack time of 10 ms and hence the task can be executed at lower frequency and extended by 10 ms more. Voltage can be reduced to an extent that the delay due to voltage reduction is within the slack times. It can be verified that the energy savings will be higher in case of DVFS for most of the cases. Today's domestic items like refrigerators use similar concept by making the compressor work continuously at lower voltages and lower frequencies and save energy rather making legacy ON/OFF control.

Fig. 11.8 Typical DVFS system

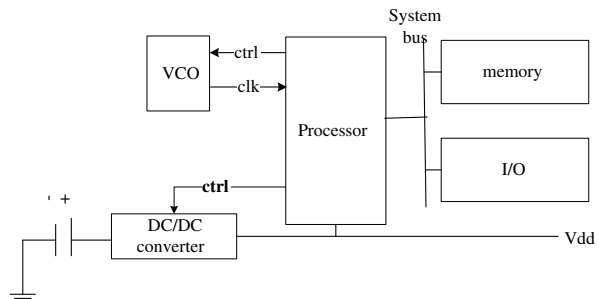
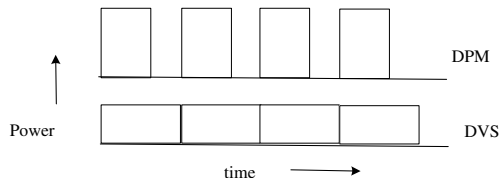


Fig. 11.9 DPM versus DVFS



11.3.3 DVFS in Heterogeneous Processing Elements (PEs)

Most of the embedded systems are no more stand-alone single processor-based systems. The systems have multiple processing elements. (PEs) They are heterogeneous having different power profiles. The heterogeneous systems are interconnected and the tasks are distributed among them. Each task has its own computation time and power profile. In single processor model we assume fixed power model. With heterogeneous multiple systems, we assume power variation model. In this case, power consumption varies during execution of different tasks. In such cases, voltage scaling strategy has to be based on mapping of the tasks across PEs and maintaining overall task schedule constraints.

The concept of energy gradient (EG) is defined in this context. EG is the change in the energy dissipation of a task execution when its task execution time is extended by a time quantum. Mathematically

$$\Delta E = E(t_{exe}) - E(t_{exe} + \Delta t)$$

Figure 11.10 gives a simple example where three PEs with distinct processing profiles have to execute an application with 5 tasks. A task executed on the different PE will have different execution time and energy consumption because of the characteristics of that PE. Each task has its deadline and the system as a whole. Problem is to map the tasks on different PEs and apply DVFS wherever the slack times are available and finally achieve global energy optimization. Algorithms are developed for dynamic voltage scaling using Energy gradient-based Voltage scaling (Schmitz 2004; Veeravalli et al. 2007).

11.4 Energy-Aware Scheduling

We have studied varieties of real-time scheduling methods in “Real-time systems” chapters. All the schedules aim at effective utilization of the processor, the deadlines of all the jobs are met; jobs are executed as per the priority. Effectively, we identified an optimum schedule out of the feasible schedules. In this context, we have not considered the power consumed by the processor. This section gives a brief

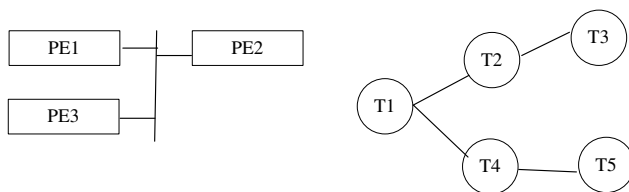


Fig. 11.10 DVFS in heterogeneous PEs

overview of energy-aware real-time schedule algorithms by adding energy as another dimension in the optimization (Veeravalli 2007; Yao et al. 1995).

The underlying principle of energy-aware scheduling techniques is to exploit the slack times and idle times during software execution by applying DVFS and DPM. The challenge is to make appropriate decisions on processor speeds to guarantee the timing requirements of the task while also considering the timing requirements of all the tasks in a system. DVFS reduces the system energy consumption by reducing the CPU supply voltage and the clock frequency (CPU speed) simultaneously. The problem becomes more complex when hard real-time tasks have to be scheduled on multi-processor system considering energy efficiency. It is NP hard problem (AlEnawy 2005). So, mostly heuristics are used to solve the problem. Multi-processor-based RT systems are scheduled using partitioned scheduling approach and global scheduling. In partitioned approach, the task is assigned to a processor and is not changed. In global scheduling, N high-priority tasks are scheduled optimally on M processors.

Another dimension is static and dynamic-priority scheduling. Although dynamic-priority algorithms lead to do better processor utilization, a majority of real-time embedded systems are implemented using static-priority schedules.

11.4.1 Case Study-1

Let there be two tasks T1 and T2 which are scheduled rate monotonically.

T1: period = 15 dead line = 15, release = 0, ci (no of execution clock cycles) = 5.

T2: period = 20 dead line = 20, release = 0, ci (no of execution clock cycles) = 5.

T1 has higher priority than T2 being RMA. Apply energy-aware schedule.

Solution:

Figure 11.11 shows the two tasks. One strategy is to apply DPM. Run the processor at its maximum speed and shut down the processor during idle times. Let the normalized maximum speed is represented as $S = 1$. Assume processor power $P = V^3$. The power

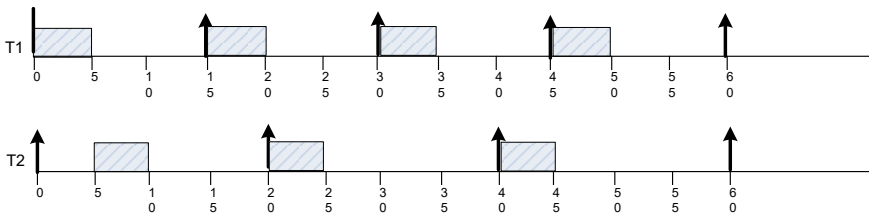


Fig. 11.11 Tasks T1 and T2 for energy-aware scheduling applying DPM = 35

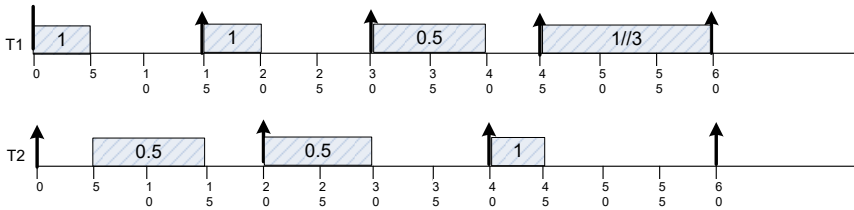


Fig. 11.12 Tasks T1 and T2 for energy-aware scheduling applying DVFS

consumed applying DPM is $7 * 5 = 35$ units during the common period $[0 \dots 60]$. One strategy for optimization is to reduce voltage and frequency so that the task can be extended and utilize the slack time and also maintains the deadlines (see Fig. 11.12). Applying this, the first job under T2 can be extended from 10 to 15 units of time. This is done by setting $S = 0.5$. J21 and J22 and j13 are set to 50% and j13 is set to $s/3$. Effectively, we have utilized the idle times. The total energy consumption by each job is as below.

Power by (J11, j12 and J23) = $(3 * 5) = 15$.

Power by (J221, J22, j13) = $(3 * 10 * (0.5^3)) = 3.75$.

Power by j14 = $(1/3)^3 * 15 = 0.56$.

Total power = 19.31, thus a reduced to about 55%.

This example demonstrates that exploiting the characteristics of voltage and frequency by DVFS technique can substantially reduce energy consumption.

11.4.2 Static DVFS Scheduling

We studied in “real-time systems” the task and job characteristic (viz) release, execution times, deadlines, priority are known in advance. The scheduler takes advantage of this fact to determine a valid schedule.

Let there be N independent jobs $J = \{J_1 \dots J_N\}$.

R_n = Release time of j_n .

D_n = deadline of job j_n .

C_n = maximum number of CPU cycles needed to complete the job for j_n (execution time measured in CPU clock cycles).

Using DVFS we can set the voltage and frequency at any time to schedule the jobs, which becomes the voltage schedule. Problem definition is to derive the *voltage schedule* meeting the job constraints defined above. We assume that setting the voltage can be done only at the release time and deadline of the job (Yao et al. 1995). We assume the jobs or scheduled based on EDF (Earliest deadline First).

The concept behind the algorithm is quite simple. When the scheduler finds certain periods where more jobs are released and compete for their deadlines, the processor must work hard to complete all of them. Hence voltage must be high. This parameter

is called *Intensity*. Once the voltage is fixed for an intensified duration, that period is removed and the other periods are considered.

The algorithm needs to define a parameter called Intensity over a time interval (t_a, t_b) as

$$I(t_a, t_b) = \frac{\sum_i c_i}{(t_b - t_a)}$$

where the set i are the jobs j_i for which their release and deadlines fall in the time period t_a, t_b . Represented mathematically. all j_i where $[r_i, d_i] \in [t_a, t_b]$.

The algorithm defines *critical interval* $[t_s, t_f]$ in which interval the intensity is maximum.

The algorithm states that the CPU will work at a maximum speed of $I(t_s, t_f)$ during the interval t_s to t_f . Thus the voltage and frequency of the DVFS is set for this interval.

11.4.3 Case Study-2

Below Fig. 11.13 shows five jobs, their release and deadlines $[r_i, d_i]$ and the CPU cycles (normalized) $[c_i]$. C_i value is shown in brackets by the side of the Job label. Let us compute the intensity for different ranges and find maximum intensity.

- $I(0, 4) = 1/4 = 0.25$
- $I(3, 18) = 10/15 = 0.66$
- $I(5, 12) = 5/7 = 0.714$
- $I(5, 15) = 7/10 = 0.7$
- $I(7, 18) = 3/11 = 0.272$
- $I(7, 15) = 3/8 = 0.375$.

The first critical interval out of all the jobs is $I(5, 12)$. During this period the DVFS is set to 0.714. The algorithm removes the interval together with the jobs in it. This adjusts the release and deadline of the remaining jobs (see Fig. 11.14).

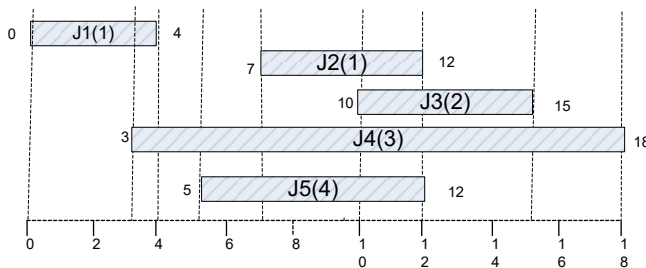


Fig. 11.13 Jobs to be voltage scheduled

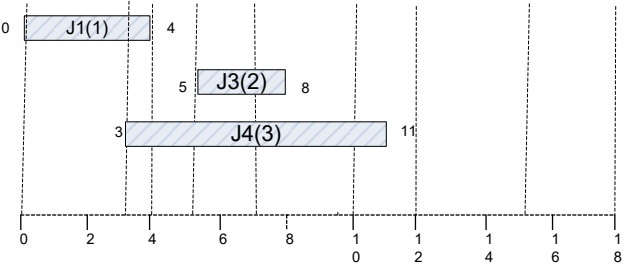


Fig. 11.14 Jobs after removing critical interval

Re-compute the intensity after removal of first intensity calculations.

- $I(0, 4) = 1/4 = 0.25$
- $I(3, 11) = 5/8 = 0.625$
- $I(5, 8) = 2/3 = 0.66$

Maximum intensity lies in the range 5–8. The DFS setting is 0.66 for this range (12–15).

In iteration 3, remove the intensity portion and readjust the jobs (see Fig. 11.15).

- $I(0, 4) = 1/4 = 0.25$
- $I(3, 8) = 3/5 = 0.6$
- $I(0, 8) = 4/8 = 0.5$.

Third intensity is during (15–18) and (3–5) is 0.6 and during (0–4) = 0.25. The final voltage schedule is shown in Fig. 11.15b.

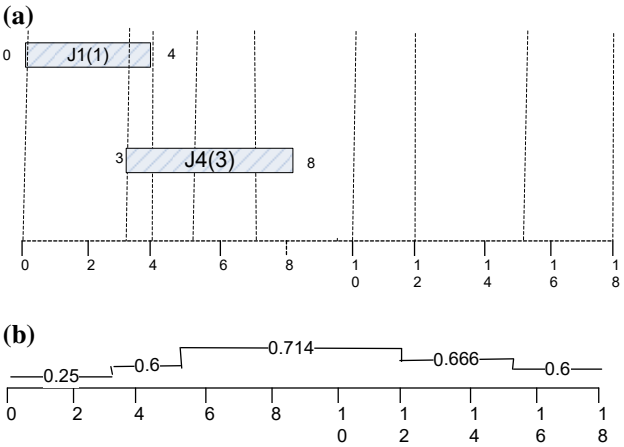


Fig. 11.15 **a** Jobs after removing second critical interval. **b** Final voltage

Above algorithm explains voltage scheduling using heuristic techniques. Similar approach can be applied for different static scheduling algorithms.

11.5 Advanced Configuration and Power Interface (ACPI)

ACPI is the collaborative effort of Microsoft*, Toshiba*, HP*, and Phoenix* to develop standard interface for power management, device discovery, and configuration ACPI (2019). This was developed in the mid-1990s and the specification is updated continuously. At the time of authoring the book, the version is Advanced Configuration and Power Interface (ACPI) Specification Release 6.3 Errata A (2020).

Power management approach normally uses the ability of the operating system to call system BIOS. BIOS discovers correct drivers and manages them and provides services to OS to manage power. The major problem in legacy techniques is that the interface between OS and BIOS is proprietary. Porting the system to different systems and updating BIOS needs thorough knowledge of interfacing.

ACPI has evolved as an architecture independent power management and configuration framework in any system. The framework defines standard power states (sleep, hibernate, wake, etc.). It defines certain registers to register the power states and accommodates the operations based on them. The direct calls by OS to BIOS are avoided. ACPI serves as an interface layer between OS and BIOS as shown in Fig. 11.16. ACPI is the key element in *Operating System-directed configuration and Power Management (OSPM)*.

11.5.1 ACPI Components

ACPI has three major components (see Fig. 11.17):

- **ACPI Registers** store the status of hardware resources. They can be written and read. They are at well-defined locations accessible to ACPI.

Fig. 11.16 ACPI subsystem
(Courtesy UEFI) (Unified Extensible Firmware Interface Forum. Specifications)

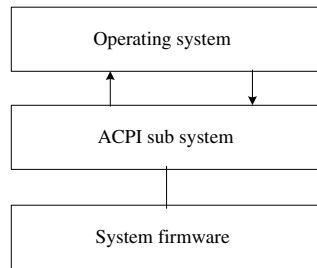
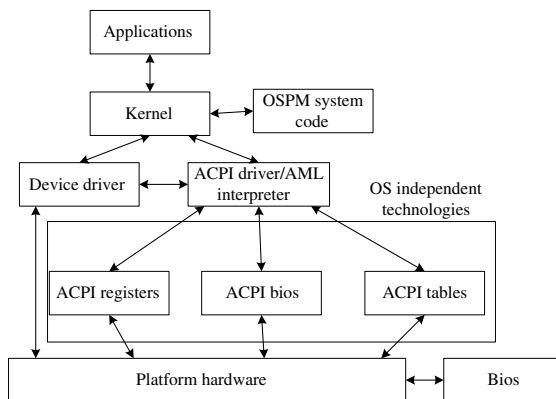


Fig. 11.17 OSPM/ACPI
Global system (Courtesy
UEFI, ACPI Specification
Version 6.3) (Unified
Extensible Firmware
Interface Forum.
Specifications)



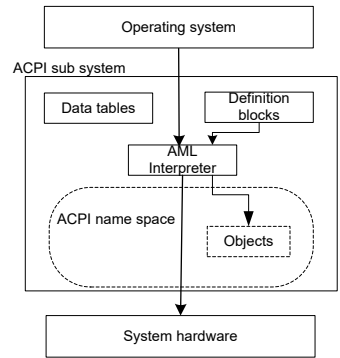
- **ACPI BIOS** is the firmware which manages booting the system and manages transition between sleep and active states.
- **ACPI Tables** store the interfaces to communicate with the underlying hardware. The tables represent system description. In order to keep hardware descriptions generic and extensible, a domain specific language has been defined within ACPI. The language, known as the *ACPI Machine Language (AML)*, is a compact, pseudo-code style of machine language. The operating system's ACPI driver includes an interpreter for AML. In ACPI parlance, hardware descriptions are called Definition Blocks..
- **ACPI control method Machine Language.** (AML) is the byte code in which, methods to control hardware are written using ASL (ACPI control method source language). Every ACPI compatible operating system uses these control methods provided by the virtual machine.
- **ACPI control method Source Language .** (ASL) is the programming language which gets compiled as AML code.

System firmware and the OS communicate through shared data tables and definition blocks. Data tables store raw data and are consumed by device drivers. ACPI Source Language (ASL) is the byte code used to define definition objects and control methods. This definition block byte code is compiled from the ASL code. When the system is initialized the ACPI machine language (AML) interpreter extracts the byte code from definition blocks as enumerable objects. These objects form the ACPI namespace. OS directs the AML interpreter to evaluate the objects and interfaces with system hardware to perform necessary operations (see Fig. 11.18).

The AML interpreter has read-write access to defined address spaces, system memory, I/O, PCI configuration, and more. It accesses these address spaces by defining entry points called objects. Devices that have a `_HID` object (*hardware identification* object) are enumerated and have their drivers loaded by ACPI.

After OS is initialized and during run time, OS handles any ACPI events which occur through an interrupt. The interrupt invokes fixed event or general purpose events (GPE). Fixed events are defined in ACPI specification itself. They are handled by OS

Fig. 11.18 ACPI structure
(Courtesy UEFI, ACPI
Specification Version 6.3)
(Unified Extensible
Firmware Interface Forum.
Specifications)

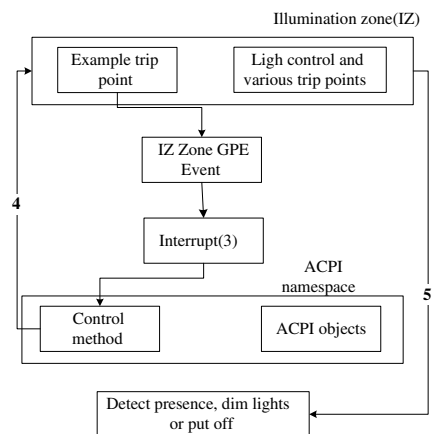


itself. GPE events are handled by control methods created using AML. The control methods are objects in the namespace. They access the system hardware and execute the needed operations through the hardware. The operation may involve invoking the respective driver to perform specified action (see Fig. 11.19).

Let us assume the system has to dim the lights when no one is present in a room. The system finds an illumination zone (IZ) in the namespace. Loads the IZ handler to dim the lights after detecting no activity. When there is no activity in the room, GPE event occurs. This causes an interrupt. When OS receives the interrupt, it searches for the control method object corresponding (like ISR) to the GPE interrupt. Upon finding, the IZ handler executes the actions. This runtime model is used throughout the system to manage all of the ACPI events that occur during system operation.

Thus ACPI is the interface between the system hardware/firmware and the OS for configuration and power management. This gives any OS a unified way to support power management and configuration via the ACPI namespace.

Fig. 11.19 Run time event
management



11.5.2 ACPI System

ACPI is the interface definition implemented using description tables, control objects, and the AML virtual machine. The interface defines how the system (hardware and software) must behave. ACPI provides low-level interfaces that allow Operating system directed power management (OSPM) to perform these functions. The functionality provided by the ACPI specification is as follows:

- Sets the computer into wakeup or sleep states. A device can wake up the computer.
- Places different devices connected into different power states. This enables the OS to put devices into low-power states based on application usage.
- When OS detects the processor in an idle state, it places the processor into low-power states.
- Keeps the devices and processors into different performance states, defined by ACPI, to achieve a desirable balance between performance and energy conservation.
- When the system is in active state, it will transition devices and processors into different performance states, defined by ACPI.
- Provides a general event processing mechanism used for system events such as thermal events, power management events, device insertion and removal, and so on.
- Battery management through ACPI embedded controller interface.

11.5.3 ACPI System States

Figure 11.20 shows the high-level view ACPI system states to implement overall system management strategy. The figure represents the states of each major component such as CPU, I/O devices.

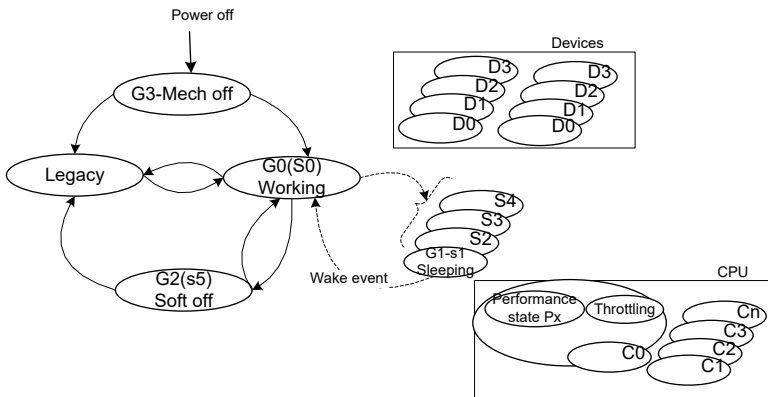


Fig. 11.20 ACPI system states (Courtesy UEFI, ACPI Specification Version 6.3) (Unified Extensible Firmware Interface Forum. Specifications)

The G states represent the system state. C states represent the CPU states. P states distinguish between performance and power consumption levels, D states do the same for I/O devices. A uniform interpretation can be applied to the number scheme: The 0-level state always corresponds to a fully operational state, and the higher number indicates increasing deep sleep states with correspondingly lower power consumption and higher return latencies. We now briefly discuss each in turn.

11.5.3.1 Global System States

- G0 is the *normal operating* state. Power consumption is maximum. No latency when an application is invoked.
- G1 is *sleeping* state in which applications do not execute. If any external event occurs the system wakes up. There are multiple substates. The power consumed depends upon the sleep state. OS reboot is not required.
- G2 is *soft off state*. In this state the power consumed is close to zero. It can move to a working state by booting in response to an electrical signal. System context is not stored, hence complete boot is necessary, and so has long latency.
- G3 is *mechanical off* state where it has to be mechanically made ON by a switch. System gets booted totally.

11.5.3.2 Sleep States

Within the global sleep state G1, several S sleep states are available. Multiple sleep states are needed in order to accommodate lulls in system activity across multiple time scales. We now briefly describe each state. The latency of the system to become active increases with sleep states s1 to s4.

- S1 has lowest latency. The system context is maintained by hardware. Main memory, cache contents, and chipset state are retained.
- In S2 the CPU and cache state is lost. OS will not be able to restore the context.
- In S3 system powers down more internal units than S2. However power to DRAM is maintained to retain the data.
- In S4 the system states including main memory are saved in non-volatile storage. The power consumption is very low but needs more latency to wake up the system from sleep.
- In S5 the system context is not stored. System loses the context and has to be electronically booted.

11.5.3.3 Device Power States

Device power states are states of particular device. The states can be applied to any device on the bus. The states are classified based on power consumption, the amount

of context stored by OS for the device, the time and effort needed to restore the device.

- D0 is fully on state. Highest power consuming state. Fully operational state.
- D1 does not provide normal service. Saves some amount of power. Preserves device context. Capable of waking itself or the entire system in response to an external event. Normally not defined on all devices.
- D2 definition saves more power and less device context. So the device loses its context when it has to power off. Capable of waking itself. Has greater latency to wake up.
- D3hot: Devices in this state will have long restore times. All classes of devices define this state. The device should have enough power to remain enumerable by software. Devices in this state are expected to save as much power as possible without affecting PNP Enumeration.
- D3-off: Power is fully removed. All device context is lost. OS will reinitialize the device. Need a long restore time.

11.5.3.4 Process Power States

Processor power states (Cx states) define power consumption and thermal management of processor. They are applicable within the global working state, G0.

- C0: When the processor is in this state, it executes instructions.
- C1: It puts the processor in non-executing power state supported by native instructions like HLT. It has lowest latency. This state has no visible software effects.
- C2 state provides improved power savings over c1. Apart from putting the processor in a non-executing power state, this state has no other software-visible effects.
- C3 offers great power reduction at the cost of increased transition latency. Processor caches maintain their state but do not emit cache coherence traffic. OS must ensure cache coherence.

11.5.3.5 Processor Performance States

In processor power state C0, the processor executes instructions. In other states it is in non-executing state. While in C0, ACPI defines multiple performance states by which the power consumption and performance can be set. This is done by DVFS, operating at different voltages and frequencies. The P states are as below.

- P0: Processor is at maximum performance with maximum power consumption.
- P1: Processor performance is limited and consumes less power.
- P2: processor functions at its lowest performance and power consumption possible. It still remains in active state.

- Processors may define support for an arbitrary number of performance states P0. Pn not exceeding 255.

The same states can be applied to device performance states also with the same definition.

11.6 Typical Guidelines for Power Management

We had a detailed discussion in the above sections, the basic techniques for energy management and established standards like ACPI for power management from device to application level. This section summarizes certain typical guidelines for power-aware embedded system design. Designers have to keep in mind some such guidelines as best practices for an efficient design. Only few guidelines out of the whole list may be suitable, as the context of the system development varies from application to application.

11.6.1 *Power Management at Design Time*

- When designing hardware with discrete components, select the components which satisfy the requirements and consume less power.
- Select the components working at the same voltage and clock domains as far as possible. This helps in applying DVFS.
- Follow co-design principles. Apply power consumed also as a cost function while allocating the function to hardware or software.
- Activity allocation, activity mapping and activity scheduling must be considered at the initial level design stages itself.
- Try to minimize output capacitance and load capacitances during design and fabrication stage.
- Consider ACPI compatible hardware, BIOS, and OS depending on cost and complexity of the system.
- Certain real time and embedded operating systems have built-in support for power management. If you are not looking for ACPI compatibility, select suitable OS based on the requirements.
- Exploit idle and slack times of static scheduling to minimize power using DVFS techniques.

11.6.2 Power Management at Run Time

- Apply some of these guidelines, when you are not using components (OS, BIOS, and firmware) with built-in capabilities for power management.
- Turn off or reduce the clock frequency when system is not needed.
- Turn off the components which are not needed for the current execution.
- Design different levels of sleep states in software and apply based on the context. You are effectively designing your own tiny and proprietary ACPI for your application.
- Brownout detection is used to monitor input voltage to the system. Turn it off when the system is in sleep mode and bring it up when system awakes.
- Embedded applications like gaming, video, and audio systems have long idle times. Apply dynamic power management techniques, which are very easy to implement.
- If the system is non-real-time system, when the system is booted, scale down the voltage and frequency gradually to an extent, that the system requirements are met. This helps in delivering requirements at optimum power.
- In non-real-time systems, monitor the utilization of the processor at periodic intervals and reduce voltage and frequency to get desired functionality. This can only be applied for static application and with less dynamism.
- The ultimate solution for real-time applications is applying DVFS to utilize idle and slack time of real-time tasks.

11.7 Summary

A power-aware system is one capable of providing the right power at the right place and at the right time. This chapter covered the theory behind device power consumption. Energy is consumed when the output switches from off to ON state. The energy is consumed in charging the output capacitance. The number of switchings in one second, which is the frequency of the system decides the power. So power consumed depends on frequency and supply voltage. However, total energy consumed in a finite time depends on Vdd. Reducing voltage causes delay due to switching threshold. Legitimate control of frequency and Voltage is the concept behind DVFS.

Dynamic power management (DPM), which puts off devices into idle state is replaced by DVFS concept in modern systems. Most embedded systems are no more unit processor based. Multiple processing elements, co-processors, ASIPs, ASICs, and SoCs constitute a complex embedded system. All these devices, the peripherals, and the communication devices have their own power profiles. Challenge lies in optimally allocating the functional tasks to appropriate devices to optimize the power consumption and meeting real-time constraints.

Power management is to be universally adapted in every system. The task needs coordination at each level (viz.) hardware, firmware, and OS. When OS is changed to another in a system, power management tasks need not be re-developed. In this

direction, Advanced Configuration and Power Interface (ACPI) is defined by UEFI which defines standard interface. This is described in detail in this chapter. ACPI compatible OS, drivers, and firmware help in developing energy efficient systems with easy portability of hardware and software components.

11.8 Further Reading

For designing energy efficient systems, ACPI specifications must be adapted. This needs thorough understanding of these specifications. Please refer to Advanced Configuration and Power Interface (ACPI) Specification, Version 6.3, January (2019). Study the books by Henkel (2007), Schmitz (2004).

11.9 Exercises

1. A processor performs a certain task in 20 ms. It dissipates a power of 500 mW when running at 33 MHz and supply voltage of 3.3 V and a threshold voltage of 0.8 V. The task is periodic at the rate of 30 ms. Apply DVS technique and estimate the reduction of power consumption.
2. Apply DPM as in case study-1 by changing the parameters as shown below. Re-work and find better optimization by at least 2 or 3 iterations.
 - The two tasks T1 and T2 are scheduled rate monotonically.
 - T1: period = 30 dead line = 30, release = 0, ci (no of execution clock cycles) = 10
 - T2: period = 30 dead line = 30, release = 0, ci (no of execution clock cycles) = 10.
3. Study case study-2. Apply same concept and work for the below EDF jobs.

Job Id	Release time	Deadline	Execution clock cycles
1	0	5	1
2	8	14	2
3	10	18	1
4	6	14	3
5	4	22	4

4. The minimum constant speed for a job is the lowest processor speed that if it is applied during the whole execution interval, the job can meet its deadline. Find the voltage profile for the below jobs.

Job Id	Release time	Deadline	Execution clock cycles
1	0	5	1
2	19	25	1
3	10	18	4
4	5	17	1
5	11	19	2

References

- Advanced configuration and power interface (ACPI) specification, Version 6.3, January (2019)
- AlEnawy TA (2005) Energy-aware task allocation for rate monotonic scheduling. In: Proceedings of the 11th IEEE real time on embedded technology and applications symposium
- Henkel J (2007) Designing embedded processors—a low power perspective. Springer
- Schmitz MT (2004) System level design techniques for energy efficient embedded systems. Kulwer Academic Publishers
- Unified Extensible Firmware Interface Forum. Specifications
- Veeravalli B et al (2007) An energy-aware gradient-based scheduling heuristic for heterogeneous multiprocessor embedded systems. In: International conference on high-performance computing HiPC 2007
- Yao F et al (1995) A scheduling model for reduced CPU energy. In: Annual symposium on foundations of computer science, Oct 1995

Chapter 12

Embedded Processor Architectures



Abstract Improvements in semiconductor technology enabled smaller feature sizes, better clock speeds, and high performance. Improvements in computer architectures were enabled by RISC architectures and efficient high-level language compilers. Together, we have enabled customized computer architectures from system-on-chips to powerful GPUs and high-performance processors. Users need that the CPU should be able to access unlimited amounts of memory with low latency. The cost of fast memory is multi-fold compared to lower speed memory. Another characteristic of CPU memory access is principle of spatial and temporal locality. The solution is to organize the memory into hierarchy by caching data at different levels. Section 12.3 covers cache basics in detail. All the memory addressable by CPU need not be in physical memory due to space and cost. It can reside in disk. The address range is mapped by virtual memory manager. Virtual address constitutes the page number and the offset within the page. This page is placed in the physical memory in the free page slot available. This is indexed in the page table. Thus, virtual memory is mapped into physical memory. Section 12.4 details the virtual memory management in detail. RISC stands for Reduced Instruction Set Computer. The clock per instruction (CPI) is one in RISC. This architecture uses optimized set of instructions executed in one cycle. This allows pipelining by which multiple instructions can be executed simultaneously in different stages. RISC has several registers; instruction decoding is simple and simple addressing modes. Section 12.5 explains RISC architectures in detail. An efficient implementation of instruction execution is to overlap the instruction executions by which each hardware unit is busy all the time. Section 12.6 explains in detail this concept of pipelining and hazards are controlled in the architecture. Several advances in pipelining architecture have been developed. But the performance improvements get saturated with new constraints and issues in implementation. When a single instruction operates on multiple data elements in a single instruction cycle, the instructions are called Single Instruction Multiple Data (SIMD) instructions. Section 12.7 introduces data-level parallelism with vector processing. Section 12.9 introduces Single instruction Multi-threading (SMT) in GPUs. We can exploit certain type of programs where they are inherently parallel and have very little dependence. We call them as threads of execution. Thread-Level Parallelism (TLP) is explained in detail in Sect. 12.10. FPGA-based technology has made system-on-chip designs a cake's walk. Systems with high-performance

requirements are possible with hardware configured to such requirements. Temporal re-configuration in FPGAs mimicking DLLs in software has made re-use of same FPGA fabric for just-in-time “use and throw” hardware blocks. Section 12.11 covers reconfigurable computing in detail. After reading this chapter, readers will be able to understand internal architecture of any processor which helps in selecting a processor for individual requirement.

Keywords Single instruction multiple data (SIMD) · Vector processing · Single instruction multi-threading (SMT) · Thread level parallelism (TLP) · Reconfigurable computing · Instruction level parallelism (ILP) · Data level parallelism (DLP) · Thread level parallelism (TLP) · Request level parallelism (RLP) · Memory hierarchy · Cache · Temporal locality · Spatial locality · Cache hit · Cache miss · Associative cache · Direct mapped cache · Virtual memory · RISC processors · Hazards in pipelining · Stride · Gather–scatter · Graphic processing units (GPU) · Simultaneous multi-threading (SMT) · System-on-chip · Constant folding

12.1 Introduction

All of us must have been introduced with basic processor architecture with examples through the course on microprocessors. The processor architectures are advancing day by day with the advances in VLSI technology. Powerful super computers like Titan are built with powerful processors and GPUs. At the same time, very low power miniature and smart processors for wearables are being built. As Moore’s law (the number of transistors in a dense Integrated Circuit (IC) doubles about every 2 years) is nearing to its end, architectural enhancements are being done since several years to achieve desired performance and limiting chip densities. Improvements in semiconductor technology enabled smaller feature sizes, better clock speeds, and high performance. Improvements in computer architectures were enabled by RISC architectures and efficient high-level language compilers. Together, we have enabled customized computer architectures from system on chips to powerful GPUs and high-performance processors.

Let us classify at broad level the computing platforms and the type of processors used.

- Majority of devices used in quantity are mobile/personal devices like smart mobiles, tablets which need energy efficiency and compactness.
- Laptops and desktop computers which need high performance at moderate cost.
- Servers with high performance, expandable and highly available.
- Clusters providing software as services with high performance and availability.
- Embedded systems where processors are needed with varied requirements based on application. These vary from size, cost, real-time performance, energy, and so on. We have dealt this topic in chapter one itself.

- Recent class of devices are wearables with very low power, compact, integrated sensors and networking, and moderate performance.

Whatever may be the classification, the processor architectures have to adapt enhancements for higher performance and memory architectures for higher access speeds. Current trends are implementing parallelism at architectural level by

- Instruction-level parallelism (ILP),
- Data-level parallelism (DLP),
- Thread-level parallelism (TLP), and
- Request-level parallelism (RLP).

12.2 Memory Hierarchy

Users need the CPU should be able to access unlimited amounts of memory with low latency. The cost of fast memory is multi-fold compared to lower speed memory. The solution is to organize the memory into hierarchy (see Fig. 12.1). The memory which is built with the processor should be at highest speeds. The memory which is immediately accessible by CPU external to the CPU should be at higher speed and the memory speeds to be slow as it is away from CPU. Thus, major chunk of memory will be in low-speed memory and gets *cached* into faster memory when that chunk is needed. Another characteristic of CPU memory access is the principle of spatial and temporal locality.

This means that CPU accesses nearby instruction in a program chunk (spatial locality) and same chunk is needed too often in a specific context (temporal locality). So if a chunk is cached into memory closer to the CPU it is mostly accessed multiple times before the chunk is no more needed. Cache architectures are based on this concept. This method gives an illusion that the processor is using large amounts of fast memory.

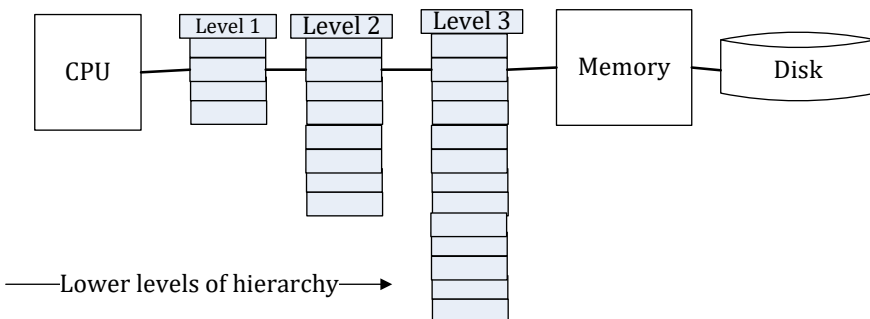


Fig. 12.1 Memory hierarchy

Design of memory hierarchy becomes crucial with processors with multiple cores. The peak bandwidth typically for an i7 processor can go up to 400 GB/s whereas the DRAM bandwidth is about 25 GB which is hardly 6%. Hence, multi-level cache for each core is very essential.

12.3 Cache Basics

Cache is synonymous to hoarding. This is like keeping some quantity handy for fast access, like keeping some money in your pocket (level 1 cache) and replenishing from you safe (level 2) and replenishing level 2 from bank with drawl. Hence, cache memory is the first memory bank addressed by CPU. Cache term is used in several contexts wherever the above *quick access* concept is needed.

Block is a fixed chunk of bytes retrieved from lower level cache (here lower level means level 2 or level 3 because they are at lower levels of hierarchy) or from main memory. The block has the desired word.

Temporal locality is the behavior where certain words in a block are needed in near time frame. Hence, the block need not be swapped immediately. Majority of software in a block has this property like loops.

Spatial locality is the behavior where certain words in a block are accessed which are within the range of currently accessing word. Examples are like branches and loops. Majority of software in a block have this property.

Cache hit occurs when the addressed word is already available in the cache.

Cache miss occurs when the addressed word is not available in the cache. A new block containing the desired word has to be placed in the cache.

Latency determines the time to retrieve the first word in the desired block. Based on memory bandwidth, the time taken to retrieve the block is determined. The time required to access a word when a cache miss occurs depends upon the latency and bandwidth of memory access.

Cache memory is organized as a sequence of blocks. Let us say it has a capacity of 16 blocks. When a block from memory has to be placed into one of the vacant blocks, there should be a mechanism (a) to identify in which vacant block it has to be placed and (b) we should be able to back track to which physical address this block belongs to.

Let us assume the memory is divided into 128 blocks. (2^7). Let the block size 64 words. (2^6).

The physical address is structured as shown in Fig. 12.2a.

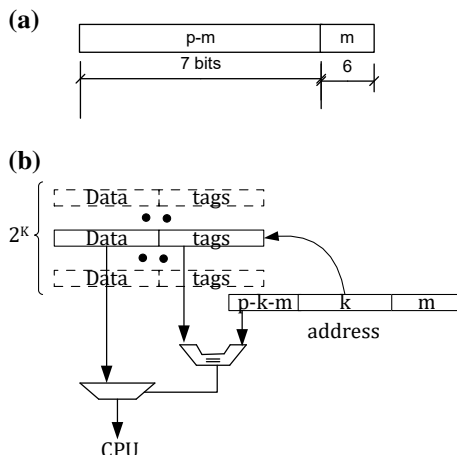
Total address space is 2^p .

Block size = 2^m .

Number of blocks are $2^{(p-m)} = 128$.

Available capacity in cache = $2^k = 16$ blocks.

Fig. 12.2 **a** Structure of physical address. **b** Direct mapped cache



12.3.1 Direct Mapped Cache

When any of the iword is to be placed in the cache, one mechanism is to *directly map* the address to one of the blocks. The block will be placed at $(p-m) \text{MOD } 16$. As an example if 71st block is to be placed in cache it will be placed at $71 \text{MOD } 16 = 7$. The block will be *tagged* with $(p-m-k)$ bits. In this case, it will be 100. (100-0111-<block>).

The cache read will check this tag value at block 7.

Same is shown in Fig. 12.2b.

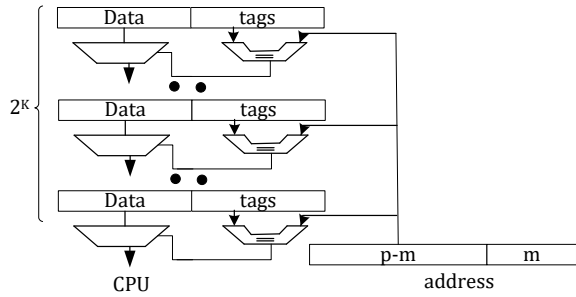
12.3.2 Fully Associative Cache

In direct mapping as explained above has one issue. A given block from memory can be placed at unique block in cache. Simply $2^{(p-m)} \text{MOD } 2^K$. If that block is already occupied, system has to swap the existing block with new block. If both the blocks are actively being used, several swaps will occur and performance reduces.

An alternate mechanism is to place the new block in any of the vacant blocks in the cache. This reduces block swaps. Now the issue is how to tag the block. It has to be tagged with $(p-m)$ bits. When accessing, the tag portion of the address is to be compared with all blocks to find a match. If a match occurs, it is a *cache hit* else it is a *cache miss*.

The same is shown in Fig. 12.3. The advantage here is placing a block is performant. But cache read is non-performant as this needs 2^K comparisons. Need more hardware.

Fig. 12.3 Fully associative cache



12.3.3 Set Associative Cache

This method is a compromise between direct mapped and fully associative cache. All the words in the block are grouped into sets. Normally a set consists of 2 or 4 words. When an address is to be cached it is placed in one of the locations in the set. The set is decided like direct mapping. Let us re-work the example of direct mapping. Assume it is two-way set associative (see Fig. 12.4). So we have $16/2 = 8$ sets. So the 71st block is placed at $71 \text{MOD} 8 = 7\text{th}$ set. If one word in the set is filled, it tries to place in the second word of the set. Advantage is that you are reducing the swaps by 50%.

Figure 12.5 shows how the data is placed in all three mappings. If you compare all mappings in terms of sets, direct mapping one word is one set. In fully associative, complete block is one set whereas N way set associative it is N words per set.

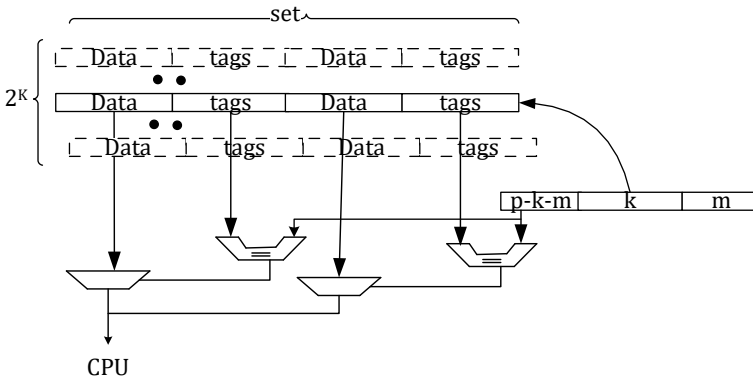


Fig. 12.4 Set associative cache

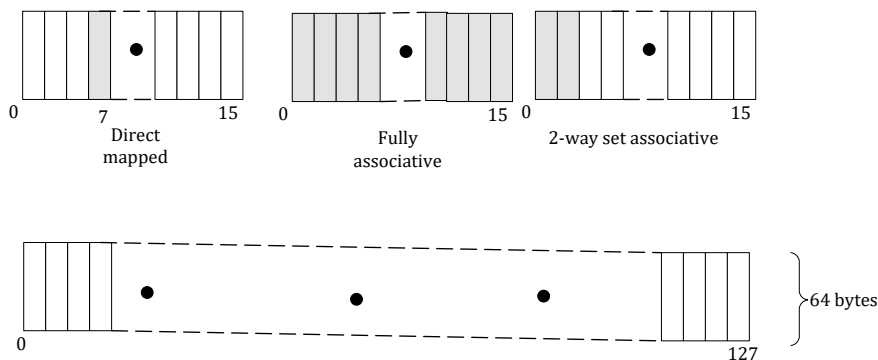


Fig. 12.5 Set associative and direct mapped caches

12.3.4 Writing to Cache

When CPU writes the word at highest level cache, it immediately updates in all lower level caches. This is *write-through cache*.

When CPU goes on updating the content in the highest level without updating the lower levels, it is called *write-back cache*. This causes in-coherency. But when the block is going to be replaced, then all the lower levels (including memory) gets updated. This is *write-back cache*.

Miss rate is one performance metric of the overall system which includes the application software, CPU cache architecture, and operating system. One cause of miss is when block is read for the first time. The second one is the capacity of the cache. When the number of cache blocks is less, more misses occur to accommodate new blocks. The third one can be carefully avoided, which is due to *cache conflicts*. This occurs when the software accesses multiple addresses which map to the same location in the cache. Miss rate plays very important role in average memory access time and because of the penalty, system has to pay for each miss.

12.3.5 Replacing a Block During Cache Miss

When cache miss occurs, one of the existing blocks has to be removed to find place for the incoming block. Lots of advanced algorithms are being used in latest processors (Hassidim 2010). Traditionally, the candidate block is randomly selected. Another meaningful technique is least recently used (LRU) where the block which has not been accessed for long time is the candidate for removal. The third one is first-in first-out (FIFO), the block which is oldest is the removed candidate. For more details refer Hassidim (2010) and Nagraj (2004).

12.3.6 Basic Cache Optimizations

Larger block sizes reduce the compulsory misses as the desired locations will be available in the spatial and temporal localities. This needs higher cache sizes. Increases conflict misses. Cost of miss penalty will be high due to the higher block size.

More blocks with more cache memory provides more blocks to be accommodated from different regions and hence miss rate will reduce. However, cache identification (hit time) will increase due to more blocks and need more power.

Higher associativity leads toward fully associative which reduces conflicts (multiple addresses map to same location). But this increases hit time because of more comparisons. Needs more hardware which increases power.

More cache levels reduce overall memory access time but additional overheads in write through. Additional space, cost, logic, and power requirements (Table 12.1).

12.4 Virtual Memory

All the memory addressable by CPU need not be in physical memory due to space and cost. It can reside in disk. The address range is mapped by virtual memory manager. The total address space is broken into pages of fixed size. At any time, each page resides either in main memory or on disk. A page fault occurs when CPU references an item in a page and that page is not in main memory. Then, the specific page is moved from disk into main memory. This process is done by the memory management software. During this time, CPU proceeds with another task. Such page faults take considerable time to load the page into memory. This process is close to the process of cache update from main memory (see Fig. 12.6).

Virtual address constitutes the page number and the offset within the page. This page is placed in the physical memory in the free page slot available. This is indexed in the page table. Thus, virtual memory is mapped into physical memory. Page tables are normally large. This table is stored in main memory for quick mapping. So every memory access logically takes at least twice as long, keeping address translations

Table 12.1 Cache optimization

Optimization	Compulsory misses	Higher cache size	Conflict misses	Miss penalty	Miss rate	Hit time	Power	Overall memory access
Larger block size	Reduce	Increase	Increase	Increase			Increase	
More blocks					Decrease	Increase	Increase	
Higher associativity			Reduce			Increase	Increase	
More cache levels	Reduce						Increase	Reduce

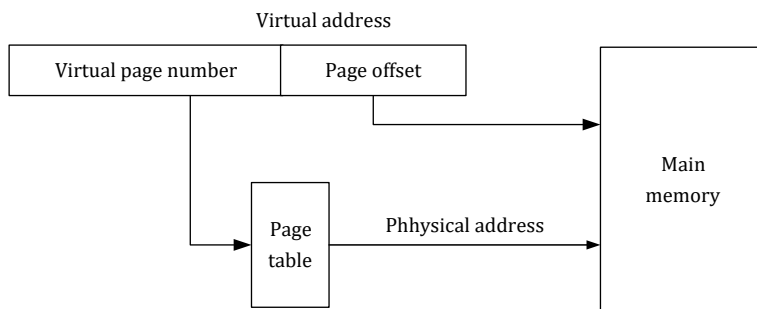


Fig. 12.6 Paging in virtual memory management

in a special cache, a memory access rarely requires a second access. This special address translation cache is referred to as a translation lookaside buffer (TLB).

12.4.1 Case Study-1

Here is the total memory organization with the parameters mentioned below. This includes virtual memory and cache organization.

- Page size of virtual memory = 8 KB.
- TLB direct mapped with 256 entries.
- L1 cache is a direct-mapped of size 8 KB.
- L2 cache is a direct-mapped of size 4 MB.
- Both use 64-byte blocks.
- Virtual address is 64 bits width.
- Physical address is 41 bits width.

Solution

Below is the organization (see Fig. 12.7). (B1) Processor addresses 64-bit virtual address to access memory. (B2) Page size is 8 KB (13 bits) and remaining are page number (51 bits). The physical memory is 41 bits. So page number has to be mapped into 28 bits (41–13). (B3) This is done by comparing the 256-byte TLB index (8 bits) and get the 28-bit physical address. We now got the physical address of 41 bits. (b4) Now it should be checked whether this physical address is in L1 cache. As the cache is direct mapped and block size is 64 bytes (6 bits), there are 128 blocks (7 bits). Each of these 128 blocks will have the tag value. Being direct mapped compares the physical address with the tag value at the predefined index (Block number MOD 7). If it matches, it is cache hit else it is miss.

If it misses it checks in L2 cache which is 8 MB (22 bits). The number of blocks is 2^{16} (16 bits). Being direct mapped cache verifies the appropriate location matches with the 19 bits of address. If it matches it is cache hit.

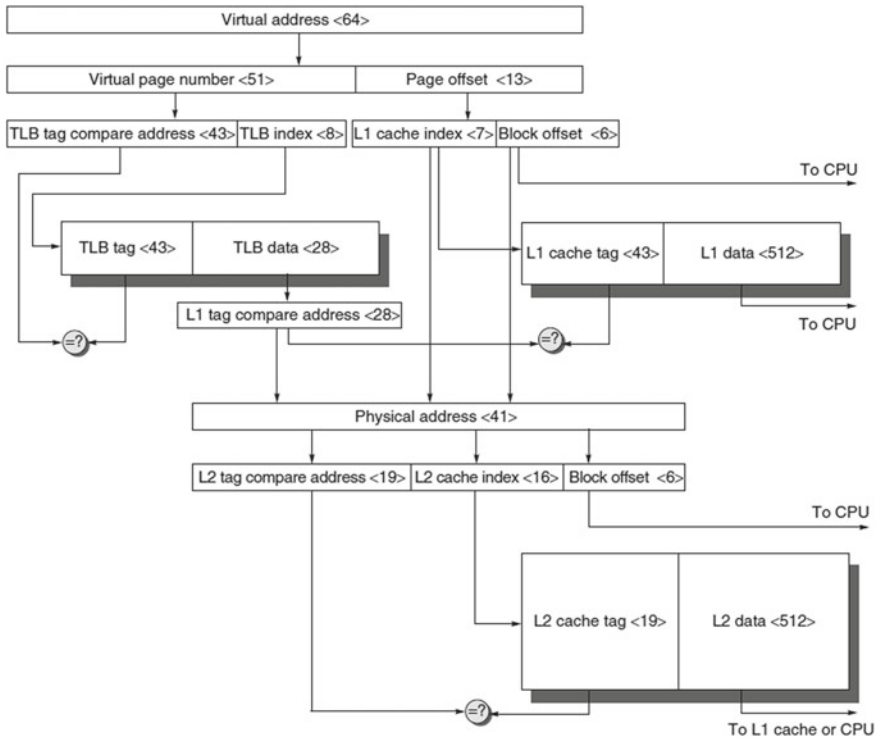


Fig. 12.7 Total picture of MMU and cache organization

ARM Cortex-A8 memory hierarchy

Arm Cortex-A8 supports two-level cache. There is separate cache for instructions and data (see Fig. 12.8).

Memory management is handled by separate TLBs for both. The page size is variable and helps in tuning the memory access depending on the type of application. The TLBs are fully associative, 32-entry data and instructions. TLB entries support 4 KB, 64 KB, 1 MB, and 16 MB pages. MMU performs a lookup for the requested virtual address in the relevant instruction or data TLB. If it misses hardware translation table walk is performed.

Level-1 cache is four-way set associative. Cache size is configurable to either 16 or 32 KB. Block size (also called line length) is 64 bytes.

Level-2 cache is eight-way set associative and configurable size of 128 KB to 1 MB. This uses random replacement policy. L2 cache is partitioned into multiple banks to enable parallel operations and improve access time. Two accesses can go simultaneously to the two banks separately.

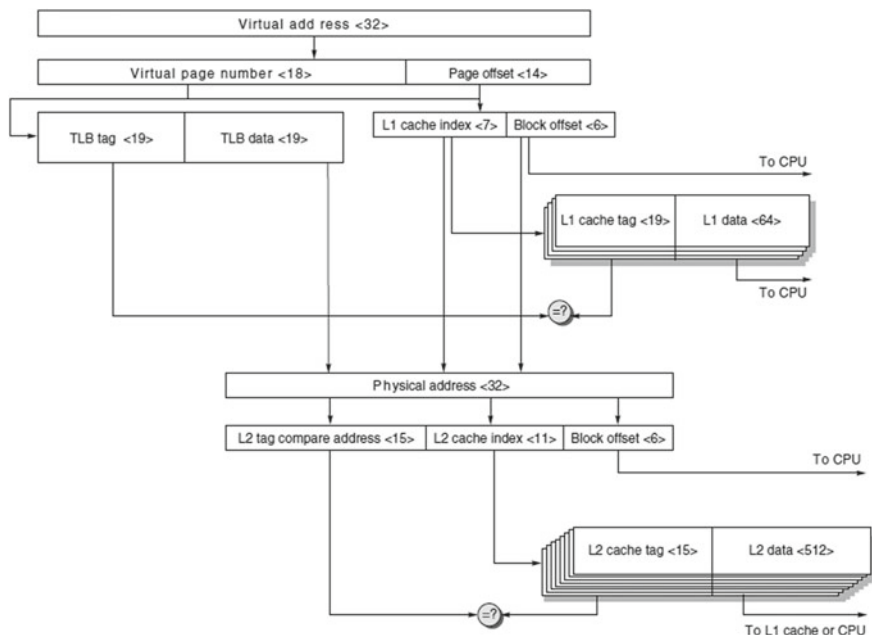


Fig. 12.8 ARM Cortex-A8 memory hierarchy (Arm copyright material kindly reproduced with permission of Arm Limited)

12.4.2 Few Techniques for Cache Performance

Modern processors apply one or more advanced techniques to improve cache performance.

Herewith some of them are listed and briefly described.

- Reduce cache hit time. Minimizing the critical timing path for cache hit. Use lower levels of associativity. Setting the multiplexor to choose the correct data item if the cache is set associative. Keep the TLB out of the critical path L1 caches should be virtually indexed. Results show that direct mapping is slightly faster than two-way set associative. Two way is 1.2 times faster than four-way set associative (approximately).
- Reduce hit time by keeping certain extra bits in the cache to *predict* the “way” in set associative. If the predictor bits do correct prediction, the correct “way” is selected and reduces hit time.
- Increase cache bandwidth by allowing data cache to continue to supply cache hits during a miss. Pentium Pro allows four outstanding memory misses.
- Increase cache bandwidth by dividing cache into independent banks that can support simultaneous access. Core i7 has four banks in L1.

- Reduce miss penalty. As soon as the requested word of the block arrives, send it to the CPU and let the CPU continue execution while filling the rest of the words in the block.
- Optimize compilers to reduce miss rates.

12.4.3 Compiler Optimizations

12.4.3.1 Merging Arrays

This improves spatial locality by single array of compound elements versus two arrays.

```
// Instead of two arrays
int ary1[SIZE];
int ary2 [SIZE];
//merge arrays
struct ary12 {
    int ary1;
    int ary2;;
};
struct ary12 merged_array[SIZE];
```

12.4.3.2 Loop Interchange

This improves spatial locality. Change nesting of loops to access data in the order as stored in memory. $x[i,j]$ and $x[i,j + 1]$ are adjacent (row major) sequential accesses instead of striding through memory every 100 words improves spatial locality.

```
/* Before */
for (k = 0; k < 100; k = k+1)
    for (j = 0; j < 100; j = j+1)
        for (i = 0; i < 5000; i = i+1)
            x[i][j] = 2 * x[i][j];
```

```
//after
for (k = 0; k < 100; k = k+1)
    for (i = 0; i < 5000; i = i+1)
        for (j = 0; j < 100; j = j+1)
            x[i][j] = 2 * x[i][j];
```

12.4.3.3 Loop Fusion

Combine two independent loops that have same looping and some variables overlap.

```

/* Before */
for (i = 0; i < N; i = i+1)
    for (j = 0; j < N; j = j+1)
        a[i][j] = 1/b[i][j] * c[i][j];
for (i = 0; i < N; i = i+1)
    for (j = 0; j < N; j = j+1)
        d[i][j] = a[i][j] + c[i][j];

```

```

/* After */
for (i = 0; i < N; i = i+1)
    for (j = 0; j < N; j = j+1)
        {a[i][j] = 1/b[i][j] * c[i][j];
         d[i][j] = a[i][j] + c[i][j];}

```

12.4.3.4 Blocking

Improve temporal locality by accessing “blocks” of data repeatedly versus going down whole columns or rows. Code is changed to compute on a submatrix of size B by B . B is called the *blocking factor*. Blocking exploits a combination of spatial and temporal localities, since y benefits from spatial locality and z benefits from temporal locality.

```

/* Before */
for (i = 0; i < N; i = i+1)
    for (j = 0; j < N; j = j+1)
        {r = 0;
         for (k = 0; k < N; k = k + 1)
             r = r + y[i][k]*z[k][j];
         x[i][j] = r;
        };

```

```

/* After */
for (jj = 0; jj < N; jj = jj+B)
    for (kk = 0; kk < N; kk = kk+B)
        for (i = 0; i < N; i = i+1)
            for (j = jj; j < min(jj+B,N); j = j+1)
                {r = 0;
                 for (k = kk; k < min(kk+B,N); k = k + 1)
                     r = r + y[i][k]*z[k][j];
                 x[i][j] = x[i][j] + r;
                };

```

12.5 RISC Processors

RISC stands for reduced instruction set computer. The conventional computer architecture is named as CISC which is complex instruction set computer which all of us know. In CSIC, as the name says, the instructions are complex. It means the size of instruction is larger than word. The instruction decoder decodes the instruction and breaks into several micro-instructions which gets executed by the micro-programmed control unit. Hence, one instruction execution involves several read–writes to memory, register to register, and ALU for completing the complex operation. The execution time varies with number of machine cycles needed per instruction. This architecture has evolved first when most of the programming was done in assembly language. So users were comfortable with more work done by each instruction and reduce code size.

The concept has changed with RISC where each instruction is executed per one cycle. A clock per instruction (CPI) is one in RISC. This architecture uses optimized set of instructions executed in one cycle. RISC architectures evolved in 1980s. Basic characteristics are single instruction per cycle, allows pipelining by which multiple instructions can be executed simultaneously in different stages, has several registers, instruction decoding is simple, and has simple addressing modes.

12.5.1 Instruction Cycle for RISC

Below is the typical cycle for RISC.

12.5.1.1 Instruction Fetch (IF)

- PC—address bus,
- Fetch instruction, and
- Increment PC.

12.5.1.2 Instruction Decode (ID)

- Decode the instruction,
- Fetch the data from register(s), and
- Compute branch target address if it is branch.

12.5.1.3 Execute (EX)

- ALU operates on the operands prepared in instruction decode,
- Reg to reg,
- Reg to ALU, and
- ALU to reg.

12.5.1.4 Memory (Mem)

- Load—reads data from effective address compute in execute cycle.
- Store—writes data at effective address.

12.5.1.5 Write Back (WB)

- Writes to register from ALU(Ex cycle).
- Writes to register from memory (Load in Mem).

A few instructions of MIPS and the instruction cycle are shown below:

add \$r12, \$r7, \$r8 (store r7 + r8 at r12)—IF→ID→EX→WB

Load R2,A (load (A) to R2)—IF→ID→EX→Mem

Store R2,A (R2 to (A)): IF→ID→EX→Mem.

12.6 Pipelining

One can observe from the sample instruction set, the instruction cycle for each instruction is very symmetric. If they are executed one after the other, the cycle for instructions 1 and 2 looks as shown in Fig. 12.9. One can observe that the instruction fetch hardware is free for three units of time. An efficient implementation would be to overlap the instruction executions by which each hardware unit is busy all the time. (But this is not possible always as you can see subsequently!).

Pipeline is the same technique used in assembly line (see Fig. 12.10). In assembly line, each unit executes the specific job and passes the job to the next section. The unit accepts next job and proceeds. Same happens in pipelined architecture. Each unit of instruction cycle is executed and data passed to the next unit. If the process cycles are symmetric the clocks per instruction reduces by the number of pipeline stages. Little more thought has to be placed how each stage comminutes with its next stage. The speed of execution of different stages may be different. One way is to make the previous stage wait till the next stage completes. Also another mechanism is to place a register between them so that both of them transfer the data asynchronously. This is normally done in assembly line in workshops where the completed job of previous stage is placed in a basket for the next stage to pick up. A similar approach is done in RISC as shown in Fig. 12.11.

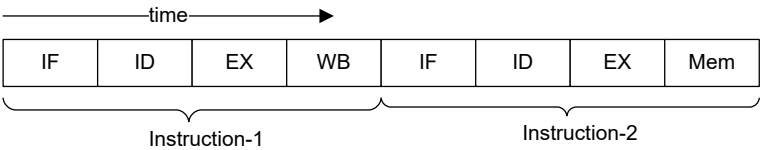


Fig. 12.9 Sequential execution

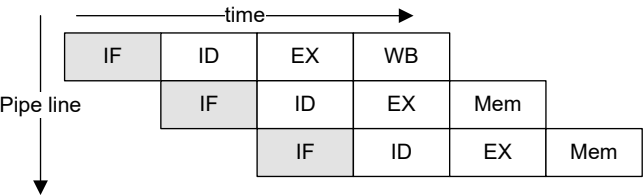


Fig. 12.10 Pipelined execution

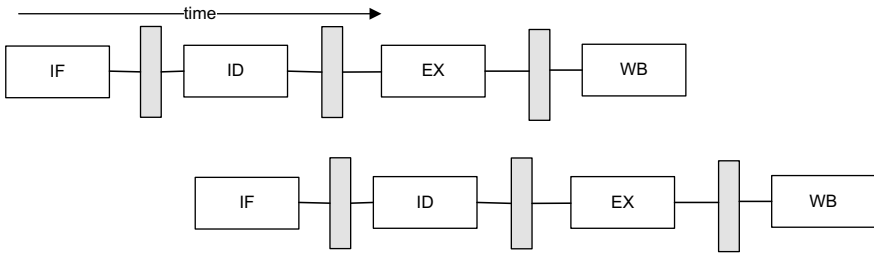


Fig. 12.11 Buffers in pipelining

Although it is critical to ensure that instructions in the pipeline do not attempt to use the hardware resources at the same time, we must also ensure that instructions in different stages of the pipeline do not interfere with one another. This interference is isolated by introducing pipeline registers between successive stages of the pipeline. After each operation in a stage, the results are stored in the buffer so that they are taken by the succeeding stage. The buffers thus keep the stage operations independent of each other and process at different speeds and share their data in asynchronous manner.

12.6.1 Hazards in Pipelining

12.6.1.1 Structural Hazards

Structural hazards occur when two instructions are in the pipeline at different stages and need a resource at the same instance. The processor is not able to provide simultaneous execution. For example, in a processor with single memory access for instruction and data, an instruction fetch (IF) gets stalled if load/store (Mem) instruction is fetching memory. In Fig. 12.12, instruction 1 is storing in memory whereas instruction 4 wants to fetch the instruction from memory. This happens when there is single

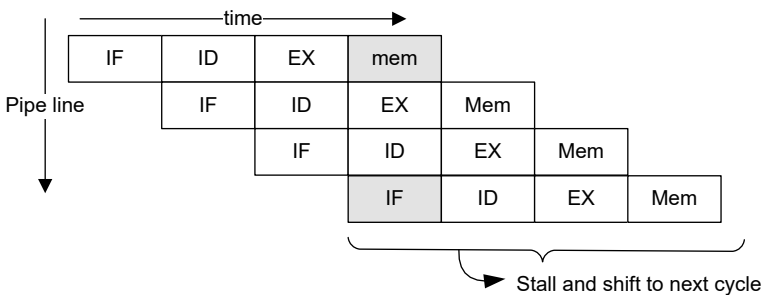


Fig. 12.12 Structural hazard due to single memory resource access

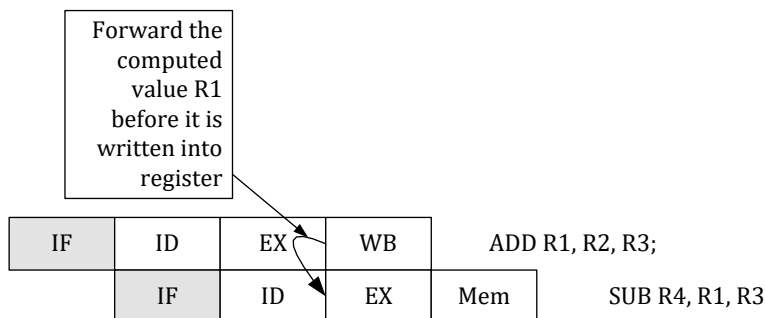


Fig. 12.13 Forwarding technique to avoid read-after-write hazard

cache for data and instructions. This causes a *stall* to instruction 4 and starts executing in next cycle. Structural hazards will occur when some functional units are not fully pipelined and when some resources have not been duplicated enough to allow all combinations of instructions in the pipeline to execute.

12.6.1.2 Data Hazards

Data hazards occur when a stage needs a data which is not processed yet by an earlier instruction. For example, the sequence of instructions are given below:

1. ADD R1, R2, R3; $R2 + R3 \rightarrow R1$
2. SUB R4, R1, R3; $R1 - R3 \rightarrow R4$

Instruction 2 can execute only after instruction 1 has updated R1 as instruction 2 needs updated value of R1. This is *read-after-write* data hazard.

Data hazards can be managed by *forward logic* (see Fig. 12.13). This logic detects whether the operand needed for the operation is not yet written to the register in the write back (WB) cycle, but the data is ready after Execute cycle, internally it forwards the data to the operation. In this case, R1 is already computed and can be forwarded to the next EX cycle. Forwarding can be generalized to include passing a result directly to the functional unit that requires it.

12.6.1.3 Control Hazards

Control hazards (also called as branch hazards) occur whether the instruction has to be executed or not. This happens when a decision for a branch has not been taken yet, but the next instruction fetch has already been done.



Fig. 12.14 Resolve branch hazard by modifying the instruction to NOP

In the example below, the stage executing instruction 1 has to change PC (assume $r1 = 0$ and it has to branch to L1) before the next instructions IF can take place. Some action has to be done to rectify such control hazards.

```
1. jz      r1, L1           // if r1 is 0, goto L1
2. load   r1, 1            // r1=11
3. L1:    Add r2, r1,2      // r2=r1+1
```

One way it is resolved is to re-do instruction fetch again so that correct PC after branch decision is taken (see Fig. 12.14). Another way is continue based on branch-taken or branch is not taken. If the branch is not taken there is no issue and the pipeline continues. If a branch is taken, change the fetched instruction to NOP. Lots of *branch prediction techniques* have been proved and successfully implemented in modern processors (Sweety and Chaudhary 2018).

12.6.2 Pipeline in MIPS Processor

MIPS32 is highly performant RISC architecture which was adapted in most of the products. The architecture is based on fixed length, regularly encoded instruction set suitable for RISC architecture. It has 32 general-purpose registers. Uses load/store data model (see Fig. 12.15).

MIPS pipeline executes each instruction in four or five clock cycles passing through the total pipeline of five units. If unit is busy the effective clocks per instruction (CPI) will be one clock cycle. After processioning in a stage the output is written into a temporary register for use in next stage. LMD, Imm, A, B, NPC, ALUout, and Cond are some temporary registers. In IF cycle, the instruction is fetched from instruction memory (cache). ID cycle reads the instruction from IF, and fetches if register data is involved. Based on the instruction the operands are made ready to be executed in the EX stage. The operands are passed through A and B and any new PC. EX unit gets the proper operands through multiplexer and execution completed in the ALUs. MEM unit writes to the data memory. For simplicity, data memory is shown as part of MEM unit but actually it exits outside and accessed through data cache. WB writes the data back into registers, if the instruction has to do so.

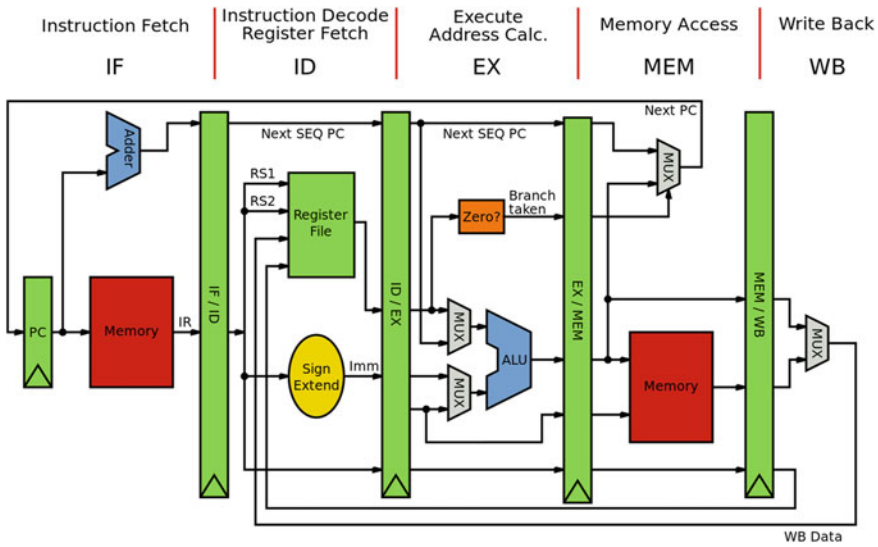


Fig. 12.15 MIPS pipeline architecture. (Courtesy author: Inductive load from Wikimedia Commons)

Theoretically for an efficient pipelining data should flow in one direction. Any backward flow like writing to registers of ID stage cause hazards. For more details study MIPS® Architecture for programmers (2020).

12.6.3 Pipeline in Arm Cortex-A8

Arm processors with 32-bit architectures are most widely used in embedded systems and mobile devices (see Fig. 12.16). Arm architecture V8-A supports 64-bit address space and 64-bit arithmetic. Arm Cortex-A8 series are designed for powerful mobile devices and high-ended embedded systems. The processors support all popular operating systems for mobile and embedded systems like Embedded Linux, Ubuntu, Android, windows Embedded, etc. Other vendors designed processors with Arm Cortex core with its instruction set like in Apple Ax series. Please see Cortex A8 technical reference manual (2006).

12.6.3.1 Instruction Fetch (IF)

IF predicts the instruction stream and fetches them from L1 instruction cache. Places fetched instructions in buffer for the instruction decode pipeline. Branch prediction is done by the IF unit and pre-fetches relevant instructions. L1 instruction cache is part of IF.

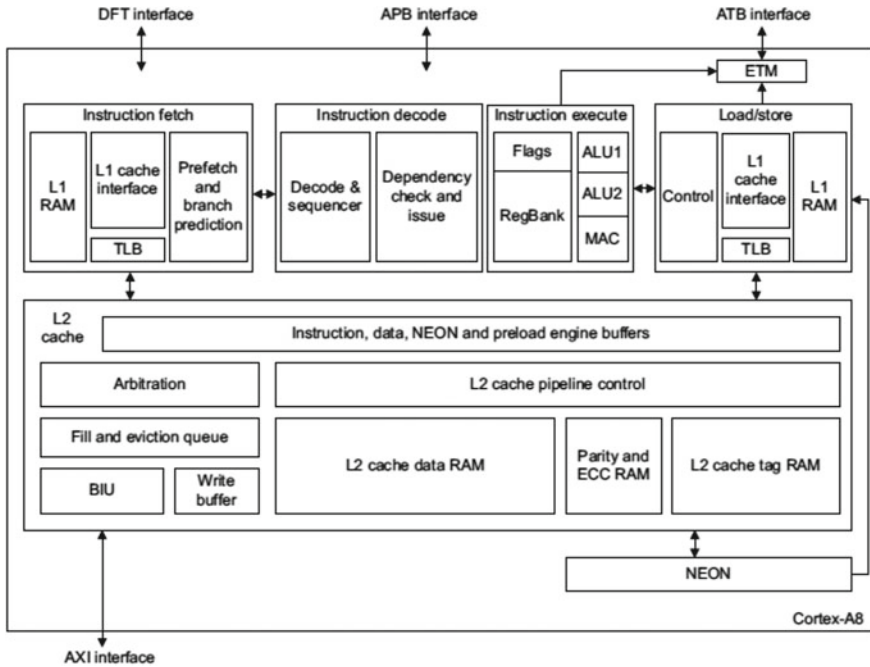


Fig. 12.16 ARM Cortex-A8 pipeline architecture (“Arm copyright material kindly reproduced with permission of Arm Limited”) (Cortex™-A8))

12.6.3.2 Instruction Decode (ID)

ID decodes and sequences all instructions. The sequencing process includes different types of exceptions, debug events, reset initializations, built-in self-tests, wait for interrupts, etc.

12.6.3.3 Instruction Execute (EX)

EX consists of two symmetric ALU, address generator for load/store instructions and multiply pipelines. EX executes all integers ALU and multiply operations and generates appropriate flags. EX generates virtual addresses for load/store operations and the base write-back value, when required. Formatted data is supplied for data to be stored. It processes branches and other changes of instruction stream and evaluates instruction condition codes.

12.6.3.4 Load/Store

Same as *MEM* we defined generically in describing pipeline. This unit contains L1 data cache, TLB, and the integer store buffer. This pipeline accepts one load or store per cycle.

Above explanation is for the basic pipeline in A8 processor. At this stage, we are not getting into NEON unit which is 10 stage pipeline unit that decodes and executes advanced SIMD (single instruction multiple data) instructions. The ETM unit is for instruction and data tracing. The unit filters data and instructions and passes through ATB (advanced trace bus) to trace the instructions for debugging purposes.

12.7 Data-Level Parallelism

Several advances in pipelining architecture have been developed. But the performance improvements get saturated with new constraints and issues in implementation. Increase in hardware is also a problem. The number of stages in the pipeline depends upon the type of workload. If the processing time of the task is small, we can have better performance without pipelining. Effectively, processor designers have to move toward other techniques for high performance. One promising direction is data-level parallelism.

When a single instruction operates on multiple data elements in a single instruction cycle, the instructions are called single instruction multiple data (SIMD) instructions. Also they are called vector instructions. For $\times 86$ architectures, the SIMD instruction set provides data processing for multimedia applications. They are MMX extensions. Similar instructions were implemented for streaming operations. This followed with advanced vector extensions (AVX) for processing vectored data.

12.7.1 Vector Architectures

Basic idea is to process data elements which are vectors. The data elements are read into vector registers process vector–vector and vector–scalar operations and place the results back into memory. A single instruction operates on vectors of data, which results in dozens of register–register operations on independent data elements (see Fig. 12.17).

12.7.2 Basic Structure

The basic structure of vector architecture is shown in Fig. 12.17. This architecture is very conceptual and is not a true representation of any commercial vector processor.

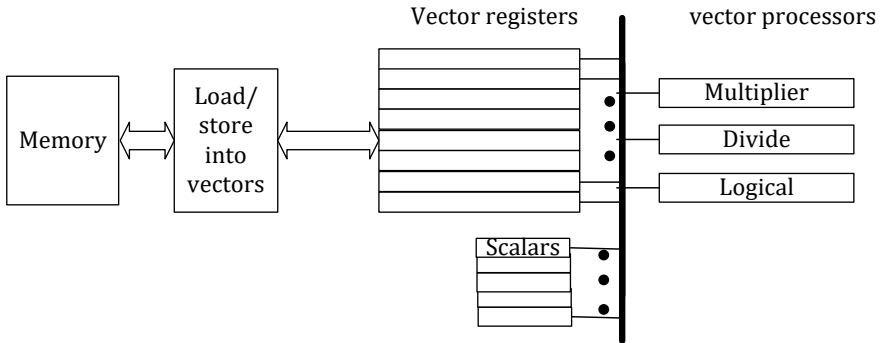


Fig. 12.17 Conceptual vector data processors

This has a set of vector registers which holds long sequence of elements like an array sequentially. Each vector register is of fixed length. Each element is normally of 32-bit or 64-bit wide. A bank of registers hold scalar values needed for operations. If each vector is represented as V , multiple operations happen across vector and vector or vector and scalar as shown below:

- OP: $V \rightarrow V$,
- OP: $V \rightarrow S$,
- OP: $V \times V \rightarrow V$, and
- OP: $V \times S \rightarrow V$.

Vector registers and scalar registers have multiple ports by which they are connected to the processing units. Simultaneous vector operations can occur using the processor units. Results are stored back in the vector registers.

The vector processors are fully pipelined for high performance. The pipeline handles all types of hazards discussed above. One important thing to be noted is that any vector operation is independent of the other.

The load/store unit moves data from memory into vector registers and back. This unit is also pipelined. The scalar register bank is used for scalar–vector operations.

12.7.3 Vector Instructions

A typical vector instruction has below structure:

- Opcode: Indicates the operation to be performed and the functional unit involved.
- Base address: Base address is the location from where the operands are to be fetched and results to be stored. This is in memory-referenced instructions. If it is a vector instruction, the vector register is referred to as the base address.
- Address increments to fetch next element from the vector.

- Address offset is the element fetched with reference to the base address added to the address offset.
- Vector length to indicate the vector length at which the vector operation gets terminated.

Sample code of a vector processor is shown below:

<i>LD</i>	<i>F0,s1</i>	<i>;load scalar s1</i>
<i>LV</i>	<i>V1,mx</i>	<i>;Load vector x</i>
<i>MULVS</i>	<i>V2,V1,F0</i>	<i>;vector-scalar multiply and store at V2</i>
<i>LV</i>	<i>V3,my</i>	<i>;Load vector y</i>
<i>ADDVV</i>	<i>V4,V3,V2</i>	<i>;Add vectors and store at V4</i>
<i>SV</i>	<i>V4,mz</i>	<i>;Store vector at z</i>

12.7.4 Lanes

In conventional architecture, the two elements from the vector register get processed in the functional unit per one element per cycle. One way to improve performance is to pipeline the functional unit into multiple lanes. Each lane will have one functional unit pipe. The vector register elements are also interleaved into multiple lanes so that each lane executes independently with the elements assigned to the lane using the functional unit. The performance improves by the number of lanes. In Fig. 12.18, the functional unit is structured as four units and each processing in one lane. Assuming the vector register is of length 16, the elements are interleaved into the four lanes. In one clock cycle, four elements are processed instead of one which happens without lane structure.

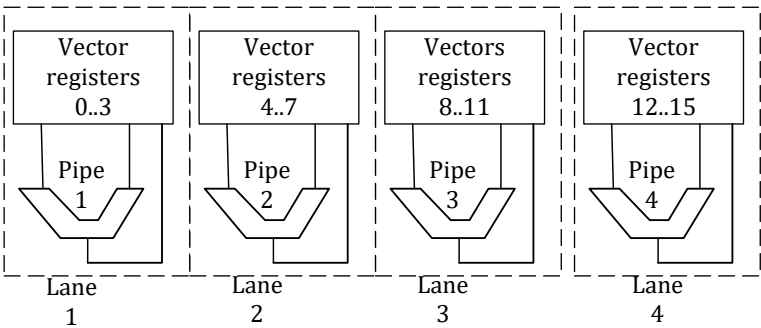


Fig. 12.18 Lanes in vector processing

12.7.5 Vector Length Register

The size of all the vector operations depends upon the vector length. This may not be known until run time. Vector length register (VLR) keeps size of vector length. If the data to be processed is higher than the VLR, dynamically the data is split into VLR size, load/store operations will be done from memory and processed in chunks.

12.7.6 Vector Mask Registers

Vector mask register helps in selective processing of elements in a vector. It is a Boolean value vector which decides what elements in the vector are to be processed. The elements in the vector where corresponding VMR has zero are unaffected by the operation. You can enable or disable using the VMR, set and clear operations in general.

12.7.7 Memory System and Memory Banks

Before vector processing, the vector has to be formed by accessing the appropriate elements from memory through load/store operations (see Fig. 12.17). The elements in the memory will not be sequential but spread across. The startup time for a load is the time to get the first word from memory into a register. The load/store operations get limited by the memory access time and the processor cycle time. This is because memory access time is several times higher than processor cycle time. If multiple processes are initiated, they get stalled due to memory access time. The solution is to have multiple memory banks so that multiple memory accesses can occur without a stall. PI refers to cache performance improvements through memory banks. As an example say you have 16 processors generating 4 loads and 2 stores/cycle, the total load/store cycles are 96 operations. If the processor cycle time is 2 ns and memory cycle time is 14 ns, you can do 7 operations if you have 7 banks to fully utilize throughputs of processor and memory. As you have 96 operations, you need $96 \times 7 = 672$ banks!

12.7.8 Stride

Stride is the distance separating the elements in memory which have to form a sequence and be adjacent in vector register. Unit stride means the elements in memory are already adjacent. It will have no issues. Most systems have stride register

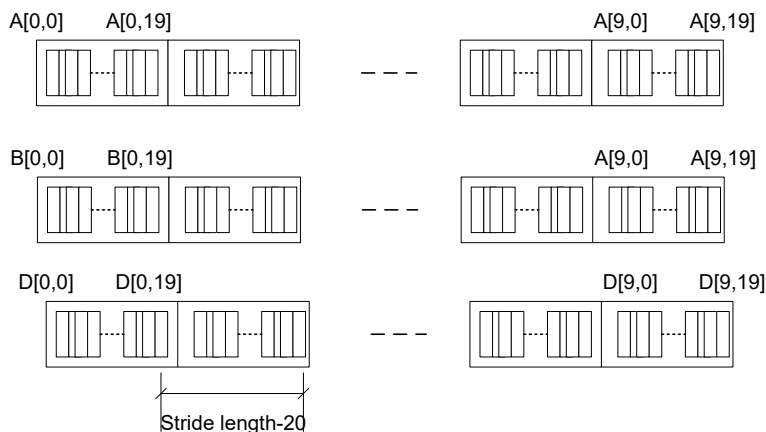


Fig. 12.19 Stride length for element $D[k, j]$

which helps to load the elements from memory from each stride length. Load/store operations with stride capability keep the data dance in vectors.

As an example, below is row major storage of data arrays A, B, D. Assume the size of each matrix is 10 by 20. Observe the operation in the last statement 5. It needs $D[k, j]$ meaning a sequence of elements $D[0, 0]$ then $D[1, 0]$, $D[2, 0]$, etc. The distance between $D[1, 0]$ and $D[2, 0]$ is 20 which is the stride length. The same is shown in Fig. 12.19.

```
for (i = 0; i < 10; i=i+1)
  for (j = 0; j < 20; j=j+1) {
    A[i][j] = 0.0;
    for (k = 0; k < 10; k=k+1)
      A[i][j] = A[i][j] + B[i][k] * D[k][j];
  }
```

12.7.9 Gather–Scatter

Let us say, you have a sparse matrix and have to operate on the non-zero elements. Doing normal load/store operation will be inefficient because majority of items are zero and the vector size increases abnormally. This is handled similar to sparse matrix algorithms by indexing non-zero elements. Here also the non-zero elements are indexed with respect to the base address. A *gather* operation fetches the element based on its index and base address. The result is a dense vector of all non-zero elements tagged with their respective index. After the vector is processed, the elements are scattered back to their respective locations using the index and base address. Hardware support for such operations is called gather–scatter and it appears on nearly all modern vector processors.

12.7.9.1 Vector Processing Architecture in ARM

Important features of ARM SVE architecture are listed below. These can be understood with the introduction from above paragraphs. [Courtesy Arm™].

- It has scalable vector length.
- Per lane prediction. Supports conditional execution for each of the lanes in the vector. The predication features make it possible to efficiently support unpredictable control flow within vectored loops.
- Gather–load and scatter–store.
- Fault-tolerant speculative vectorization.
- Horizontal and serialized vector operations
- Variable SVE vector width.
- Compiler to produce optimal auto-vectored output
- A new version (helium) supports below features.
- 128-bit vector size.
- Uses registers in the floating-point unit as vector registers.
- Supports many new features like loop predication, lane predication, complex math, and scatter–gather memory accesses.
- Support for vectored integer only, and with optional scalar FPU (double precision support also optional).
- Interleaving and de-interleaving load and store.
- Supporting conditional execution for each of the lanes in the vector.

12.8 SIMD Architecture

SIMD stands for single instruction multiple data which is easily understandable. A single instruction executes on multiple datasets. In fact, vector architecture is a subset of SIMD. SIMD instructions are mostly used for audio, video, 3D graphics, image, and speech processing applications. They are classified as MMX instructions. Basic difference with vector extensions has to be understood. SIMD extensions are simpler to implement. They are not vectors. There are no strides, lanes, and gather–scatter features. No vector length and vector mask registers.

In ARM, the extension can view Sixteen 128-bit registers, Thirty-two 64-bit registers, or a combination of registers.

Below is sample MIPS code:

```
• Example DXPY:
L.D      F0,v           ;load scalar v
MOV      F1, F0         ;Move scalar into F1
MOV      F2, F0         ;Move scalar into F2
MOV      F3, F0         ;Move scalar into F3
DADDIU   R4,Rx,#512     ;last address to load
Loop:    L.D F4,0[Rx]    ;load x[i] to x[i+3] to F4
          MUL.4D F4,F4,F0 ;compute a*x[i] i=1..3
          L.D F8,0[Ry]   ;load y[i] i=1..3 into F8
          ADD.4D F8,F8,F4 ; compute a*x[i]+y[i] i=1..3 and place in F8
          S.D 0[Ry],F8   ; scatter the value in Ry
          DADDIU Rx,Rx,#32 ;increment index to X
          DADDIU Ry,Ry,#32 ;increment index to Y
          DSUBU R20,R4,Rx ;compute bound
          BNEZ R20,Loop  ;check if done
```

This example shows MIPS SIMD code for the DAXPY loop. Assume that the starting addresses of X and Y are in Rx and Ry, respectively. The changes were replacing every MIPS double-precision instruction with its 4D equivalent, increasing the increment from 8 to 32, and changing the registers from F2 and F4 to F4 and F8 to get enough space in the register file for four sequential double-precision operands. So that each SIMD lane would have its own copy of the scalar a, we copied the value of F0 into registers F1, F2, and F3.

12.9 Graphic Processing Units (GPU) SIMT Architecture

This architecture evolved from graphic accelerators which were initially used to accelerate the graphic elements. 3D rendering engines were developed based on these accelerators. The model is extended to compute data elements in addition to graphics. The architecture is quite different from vector and SIMD architectures. It is based on heterogeneous execution which is illustrated in Fig. 12.20.

Assume you have to compute $f(a + b)$. You have multiple processors/processes which compute different tasks with different performance. So, the add function is assigned to one processor unit and the transformation to another appropriate unit. This is *heterogeneous computing*. This is similar to assigning a job to appropriate person who has expertise in executing that job. This paradigm gains performance not just by adding systems but by adding dissimilar processors.

GPU is the device which is programmed by the CPU. C-like languages are developed for vendor specific device like (CUDA-compute unified device architecture).

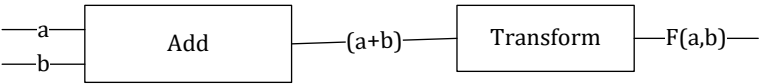


Fig. 12.20 Concept of heterogeneous computing

Vendor-independent languages like open CL are developed for ease of programming the GPU. The programming model can be classified as single instruction multiple threads model (SIMT). GPUs have similar traits as vector machines, viz., works well with data-level parallelism, gather scatter data transfers, mask registers, and large register files. Major differences are a) it has no scalar processors, uses multi-threading, has many functional units, as opposed to a few deeply pipelined units like a vector processor.

Figure 12.21 illustrates the single instruction and multiple thread approach. Please refer to the pipeline stages we studied earlier in Fig. 12.10. The instruction fetch is decoded and multiple threads are created to execute the multiple data elements. The results are stored in write-back cycle in memory. This brings basic difference between vector processing and SIMT processing.

In vector processing, one execution unit handles the bulk data which is in the form of a vector. In case of GPU, the instruction creates multiple threads each one executing on the block of data assigned to it (see Figs. 12.22 and 12.23).

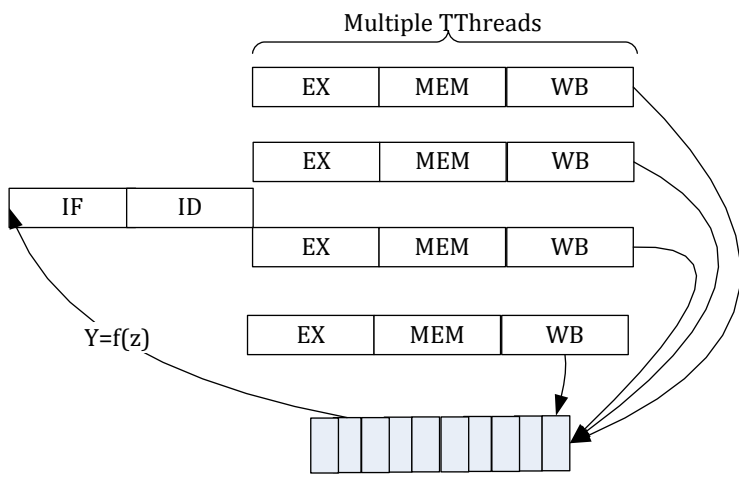


Fig. 12.21 Single instruction multiple threads (SIMT)

Fig. 12.22 Single instruction with wide data

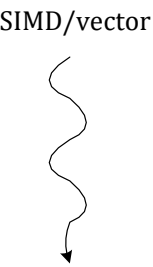
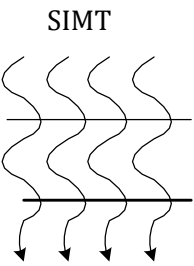


Fig. 12.23 Single instruction with multiple threads



Many SIMT threads are grouped together into one GPU core. A GPU contains such multiple cores. Hence, GPU is multicore multithreaded architecture (see Fig. 12.24). Figure 12.25 shows the logical structure of GPU which is realized in hardware. The terms and conventions used for different components in their respective architectures vary a lot. The terms used here are very generic. GPUs are not designed to replace CPUs. CPUs are aimed for applications with most of the work done by limited

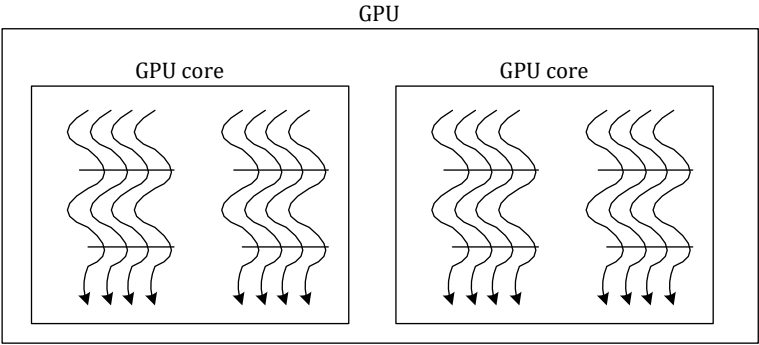


Fig. 12.24 SIMT threads grouped into multiple cores in GPU

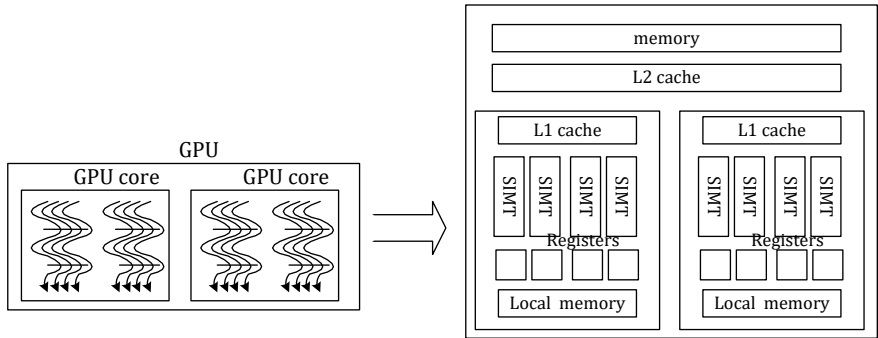


Fig. 12.25 GPU architecture mapped to GPU hardware

number of threads, each thread processing local data with different instruction set and conditional branches. GPUs are aimed at processing multiple threads using a sequence of computational instructions over sequential data.

12.9.1 Programming Model

GPUs are programmed using industry standard Open CL framework or NVIDIA's CUDA software platform. The computational programs are termed as *kernels*. An application may have one or more kernels. Note! It is not OS kernel. A kernel may consist of many threads which execute the same program in parallel. Each thread processes a chunk of data. A group of threads is termed as *thread block* (see Fig. 12.26). All the threads in a thread block run in same core and share the local memory. Thread blocks are grouped into *grids*. Each grid executes a unique kernel. Thread blocks and threads have identifiers (IDs). They specify their relationship to the kernel. Thread blocks cannot communicate each other directly. They can *coordinate* using memory operations from global memory.

Thread block scheduler assigns the thread blocks to the SIMD processors.

12.9.1.1 Example CUDA Code

Compute the sum of two arrays.

Sample program in C.

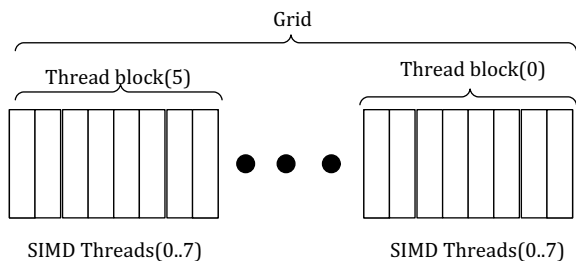
```
void sum (int n, double *x, double *y)
{
  for (int i = 0; i < n; ++i)
    y[i] = x[i] + y[i];
}
```

Assume you have eight threads per thread block

//Sum in CUDA. Invoke sum with 8 threads per Thread Block

int nblocks = (n + 7) / 8;

Fig. 12.26 Thread blocks and grid




```

__host__
sum < < < nblocks, 8 > > > (n, x, y);
__device__
void sum(int n, double *x, double *y)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;//get the element from block id and
    thread id
    if (i < n) y[i] = x[i] + y[i];
}

```

We launch n threads, one per vector element, with eight CUDA threads per thread block in a multithreaded SIMD processor. The GPU function starts by calculating the corresponding element index i based on the block ID, the number of threads per block, and the thread ID.

12.10 Thread-Level Parallelism (TLP)

We have studied instruction-level parallelism through pipelining and several mechanisms to reduce stalls, reduce hazards, branch predictions, and exception handling to get better CPI. There is a limit to increase number of pipelines because of dependency on the sequence of instructions. In spite of hardware and software techniques used in ILP, the hardware complexity increases as the logic of prediction, and hazards are to be managed in hardware. In short, we do not have ideal processor which can handle all the issues with optimal hardware complexity and less increase in power. To summarize, in an instruction stream, we do not find independent instructions for simultaneous execution due to data dependencies.

We have studied data-level parallelism where multiple data is processed as vectors or multiple SIMDs in GPUs. We can get better performance but these techniques handle certain type of data-like vectors, images, streamed audio, etc. In this technique, we are organizing the data such that the single instruction can handle bulk data efficiently. Such may not be the case all the time.

The way we exploited ILP by the characteristics of instructions through pipelining multiple stages and DLP by the data characteristics, we can exploit certain type of programs where they are inherently parallel and have very little dependence. We call them as threads of execution. Each thread has its own state and sequence of execution. It has its own set instructions and data, PC, registers, and so on. A thread may be part of a parallel program or independent program. TLP is more cost-effective than ILP or DLP. Threads are mostly controlled by a single operating system. Processing is by multiple cores on a single chip multicore processor. The parallelism is identified by the high-level software system or program.

ILP and TLP exploit two different levels of parallelism. It is more logical and meaningful to implement both the parallelisms. ILP gets stalls due to various hazards and dependencies. We can utilize those functional units by another thread which is

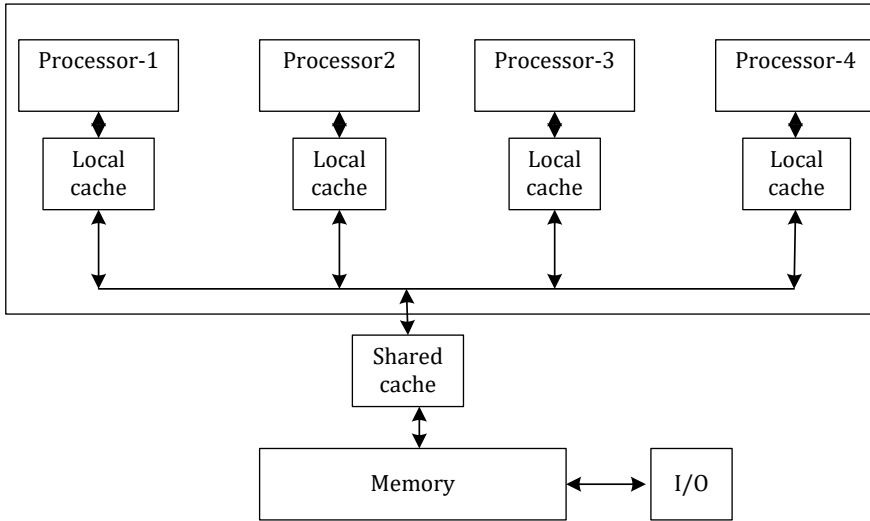


Fig. 12.27 Shared-memory multi-processor based on a multicore chip

totally independent of the stalled instructions. Thereby, all functional units are fully utilized.

12.10.1 Multicore Processor with Shared Memory

Figure 12.27 is the basic structure of a centralized shared-memory multi-processor based on a multicore chip. Multiple processor–cache subsystems share the same physical memory, typically with one level of shared cache, and one or more levels of private per-core cache. The key architectural property is the uniform access time to all of the memory from all of the processors. In a multichip version, the shared cache would be omitted and the bus or interconnection network connecting the processors to memory would run between chips as opposed to within a single chip.

12.10.2 Multi-processor System with Distributed Memory

The basic architecture of a distributed-memory multi-processor typically consists of a multicore multi-processor chip with memory and possibly I/O attached and an interface to an interconnection network that connects all the nodes. The processor cores share the memory which is at different levels. Each core gets faster access from its own cache and takes several cycles to access from other cores. For supporting

large number of cores, each core should have its private memory in different levels of cache and its own private memory. This is essential because accessing large common shared memory by all cores will have long latency and throughput of overall system falls down.

12.10.3 Multi-threaded Execution

ILP and TLP exploit different paradigms in parallel architectures in a program. ILP tries to keep the functional units pipelined and keep them occupied. CPI is improved by executing multiple instructions in sequence through pipeline. TLP tries to solve the stalls in ILP caused due to dependencies (hazards, resource constraints, etc.). This is done by executing independent chunk of instructions called threads by multiple cores in a coordinated way.

The question arises weather we can exploit both pipelining from ILP and threading from TLP simultaneously. The answer is “Yes.” When one instruction sequence of a thread gets executed through pipelined functional units, the idle units can be utilized by another thread. Hence, TLP is used as source of independent instructions that might keep the processor busy during stalls. By this approach, multiple threads utilize all functional units to maximum instant. This I concept is called *simultaneous multi-threading* (SMT) (see Fig. 12.28).

12.10.3.1 Fine-Grained Multi-threading

Here multiple threads execute simultaneously and utilize the pipelined functional units. Question now arises how the multiple threads switch and get executed. One mechanism is the threads switch for each instruction in a round robin way. This type

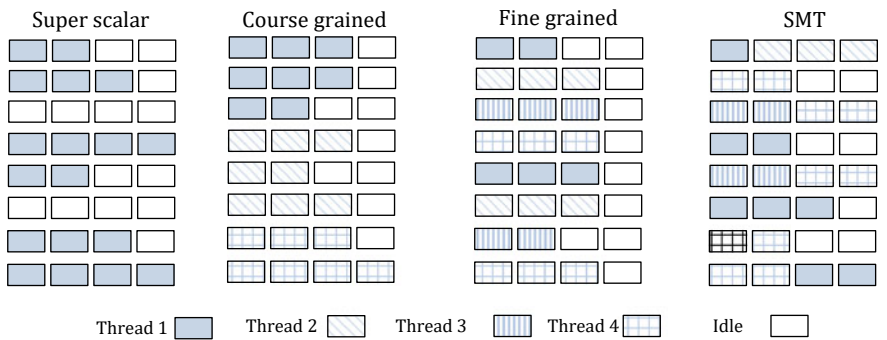


Fig. 12.28 Multi-threading variations

if interleaving instructions between threads is called fine-grained multi-threading. The processor must be able to switch the instructions per clock. Fine grained has major advantage by reducing throughput losses which are caused due to short and long stalls. This happens because during those stalls, other threads execute. The only disadvantage is that a thread having no stalls gets slower because of interleaving with other threads.

12.10.3.2 Coarse-Grained Multi-threading

In coarse-grained multi-threading, the thread switching takes place only on major stalls. The advantage is that a thread with no or minor stalls gets executed faster as there is no interleaving. But major disadvantage is that the pipeline must be purged to make the new thread to start executing. It is the startup overhead.

12.10.3.3 Simultaneous Multi-threading (SMT)

Simultaneous multi-threading is an improved fine-grained multi-threading where we issue instructions from multiple threads in the same clock cycle. This will be able to better utilize the functional units. This needs some scheduling mechanism which issues the instructions from multiple threads and schedules on the processor.

SMT uses thread-level parallelism to hide long-latency events in a processor, thereby increasing the usage of the functional units. The key insight in SMT is the dynamic scheduling which allows multiple issues of instructions from independent threads to be executed without regard to the dependences among them. The resolution of the dependences is handled by the dynamic scheduling capability.

Figure 12.28 shows pictorial representation of execution of variant multi-threading. Horizontal dimension represents execution capability in each clock cycle. Vertical dimension represents the sequence of clock cycles. In the superscalar approach without multi-threading support, the number of instructions issued per clock cycle is dependent on the ILP available. Additionally, a major stall, such as an instruction cache miss, can leave the entire processor idle. So we find the processor idle for two clock cycles in the super scalar figure. In the coarse-grained system, the long stalls are partially avoided by switching to another thread. Switching to other threads reduces the number of completely idle clock cycles. In a coarse-grained multi-threaded processor, however, thread switching occurs only when there is a stall. So we find effective processor utilization is better in coarse grained compared to super scalar. In the fine-grained case, all threads are interleaved which eliminates fully idle clock cycles. There can be multiple idle slots due to ILP limitations in a single clock. In SMT, multiple issues are done in a single clock cycle. So we find multiple threads in the same clock cycle in SMT figure.

12.11 Reconfigurable Computing—FPGAs

VLSI technology advances are scaling up device density for every 2 to 3 years. This progress is getting saturated because of theoretical limitations. Processor speeds and clock frequencies are also getting saturated. So the techniques like instruction-level pipe lining (ILP) evolved to get better CPI which we have studied. Even ILP has certain architectural limitations, so SIMD architectures and multi-threaded multicore architectures are currently implemented in every processor.

In spite of architectural enhancements, certain functionality has to be hardware based for high performance, hence application-specific integrated circuits (ASICs) have evolved long back and continue their presence in every system. ASICs are highly performant but major problem is that the functionality cannot be changed once the device is fabricated. Moreover, the cost of fabrication, NRE costs are high. If an improved algorithm has to be implemented in the chip, the process of re-fabrication costs is heavy.

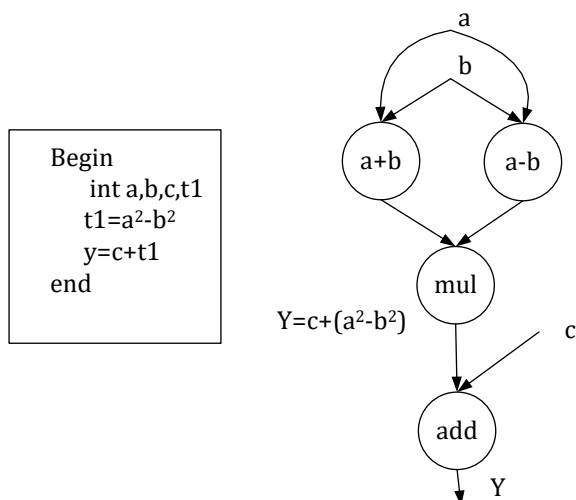
Due to this, programmable logic devices have evolved around 1970s. Different architectures with different densities evolved like PLA (programmable logic arrays), PALs (programmable array logics), PLDs (programmable logic devices), EPLDs (erasable programmable logic devices), etc. They are in commercial market even now. The main concept of all the devices is to connect an array of primitive devices like gates, flip-flops (1-bit memory devices) to derive the user-defined hardware logic. Several architectures evolved with different types of primitive devices, their density, their connectivity, and the tools to program them. Certain class of devices provide fuse-based connectivity, some do by programming the MOS switches by erasable programmable technique and some by volatile logic.

Currently, field programmable gate arrays (FPGAs) have evolved which can be programmed and configured for specific hardware logic by downloading the connectivity information as a bit map. Thus, the device can be programmed in the field, hence calling this device as FPGA. This device is reconfigurable to different hardware logics more or less instantly. This causes the application developers to think the paradigm of *spatial computing* and *reconfigurable computing*.

While it is not possible to cover all FPGA technologies, architectures, their programming languages, and configuration in this chapter, we will focus on basic FPGA architecture and its use in reconfigurable computing.

Figure 12.29 illustrates the way of processing temporally or spatially. The simple program shown in the left part of the figure can be executed in any language on a processor. The instructions get executed sequentially. The right side is shown with circles each representing a hardware block. The execution is done in hardware and the data flows from input to output as it gets processed. An FPGA can be configured to execute this logic spatially. Ability to extract parallelism (or concurrency) from algorithm descriptions is the key to acceleration using reconfigurable computing. Referring to the models we discussed in Chap. 3, processor-based computing is control flow driven and FPGA-based computing is dataflow driven.

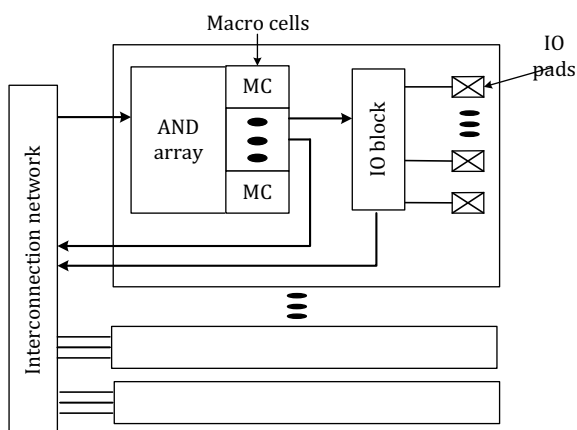
Fig. 12.29 Temporal versus spatial computing



12.11.1 Generic PLD Architecture

Figure 12.30 illustrates generic architecture of a programmable logic device. It consists of an array of multiple functional blocks (FBs). Each functional block provides programmable logic capability. It consists of AND/OR logic connected to macro-cells (MCs). The macro-cells are capable of implementing registered and combinational logic outputs. Macro-cell outputs go to I/O block. The I/O blocks provide buffering for input and output devices. Macro-cell outputs go to other FBs

Fig. 12.30 Generic PLD architecture



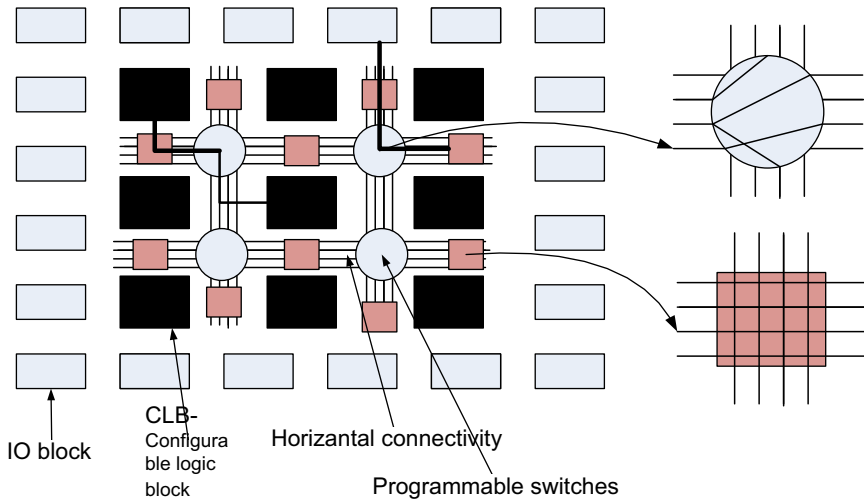


Fig. 12.31 Generic architecture of FPGA

through interconnection logic. Interconnection network connects all functional block inputs and outputs to the FB inputs.

12.11.2 Generic FPGA Architecture

Figure 12.31 shows field-programmable gate arrays (FPGAs) which are one example of reconfigurable devices. An FPGA consists of an array of *programmable logic blocks configured as logic tiles*. Xilinx names them as CLBs (configurable logic blocks). The functionality of these logic tiles is determined by programmable configuration bits. Each tile consists of a lookup table(s) (LUT), registers, multiply, and accumulator arithmetic units. Study FPGA architecture Overview by Xilinx (2020).

Routing resources in the channels between the logic tiles provide the connectivity between tiles, I/O, on-chip memory, and other resources. The routing resources are programmable. FPGAs can be dynamically reprogrammed in full or partially before runtime or during runtime which leads to virtual hardware.

12.11.3 Dynamic Configuration of FPGA

Figure 12.32 shows how the FPGA resources are dynamically configured to the needed functionality. FPGA will have certain CLBs unutilized, some already configured and active and some are inactive. The inactive CLBs can be configured for desired functionality. In this example, the area which is used by function B is no

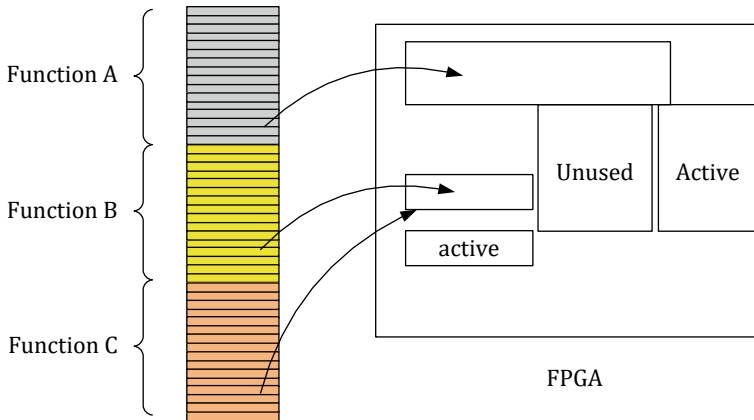


Fig. 12.32 Dynamic configuration of FPGA

more required and the CLBs are used by function C replacing the configuration bit map of function B. Thus, the FPGA resources can be dynamically utilized. This is very similar to DLLs in software development, where the library remains in disk. Based on the function calls, the DLLs get loaded into memory and get linked. In the case of FPGA, the hardware is reconfigured based on the desired functionality.

The major advantages in FPGA implementation are as follows:

- Temporal reconfigurability.
- Vast functionality in minimum hardware.
- Early prototyping.
- Configuration changes in the field itself.
- Low-volume requirements without going to ASIC fabrication.
- Provides spatial computational resources required to implement massively parallel computations directly in hardware and so on.

12.11.4 System on Chip with FPGA

In older FPGAs, the processor is interfaced to the FPGA through fat IO ports. The communication is through shared memory or IO-based communication. FPGA performs hardware-oriented computing and gets interfaced to for generic programming by the processor.

Today's scenario has moved for the complete system on chip with dense FPGA chips. The Zynq[®] series family integrates the Arm[™]-based processor with the configurable hardware of an FPGA (see Fig. 12.33). The series integrates CPU, DSP, ASSP, and mixed signal functionality on a single device. The device is a fully scalable SoC platform for unique application requirements.

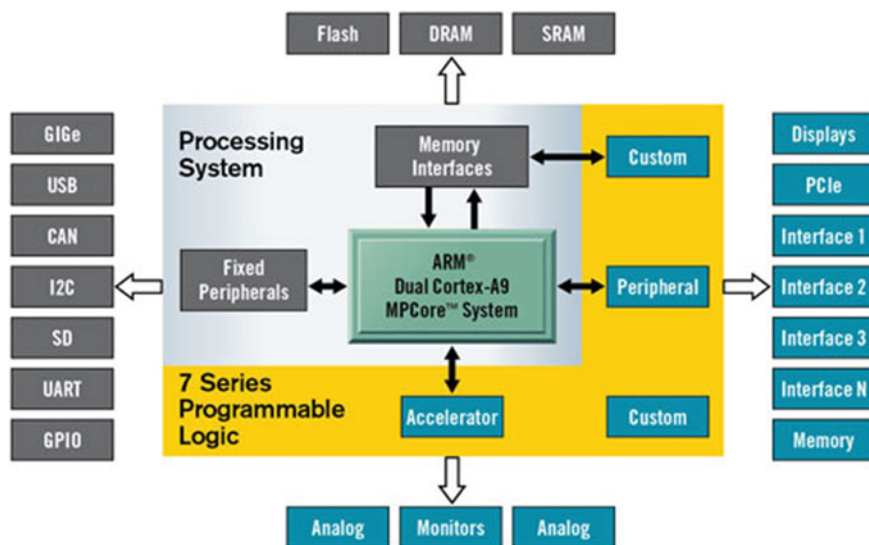


Fig. 12.33 Xilinx Zynq-7000 AP SoC (courtesy: Xilinx™ “File: Xilinx Zynq-7000 AP SoC.jpg” by Xilinx Inc. is licensed under CC BY-SA 3.0)

12.11.5 Typical Mapping Flow of FPGA

The FPGA-based development process is close to logic synthesis tools like Verilog, etc. All FPGA vendors provide development tools for their FPGAs (see Fig. 12.34). First step is to code the logic in any hardware description language like Verilog. Same is synthesized into hardware in a technology-independent manner (not specific to an FPGA model). Once basic verification is done, the logic is mapped onto the specific FPGA. In case of Xilinx™ tools, the CLBs and the interconnection network are generated. The layout created determines the logic delays, delays due to interconnections, parasitics, and energy consumption. Now the technology-based logic verification is done. The timings are verified as per specifications; the CLBs are placed and networked optimally. After verification, the bit stream is generated to program the MOS switches.

12.11.6 Logic Design with CLB

Digital logic basically contains the combinational logic and memory through flip-flops, registers. The CLB should be able to be programmable for any combinational logic with 1-bit memory devices so that any complex combinational or sequential logic can be implemented. The CLB consists of *lookup tables* (LUTs) shown in Fig. 12.35. An LUT is N-bit to 1-bit multiplexer thus implementing N-bit combi-

Fig. 12.34 Typical mapping flow of FPGA

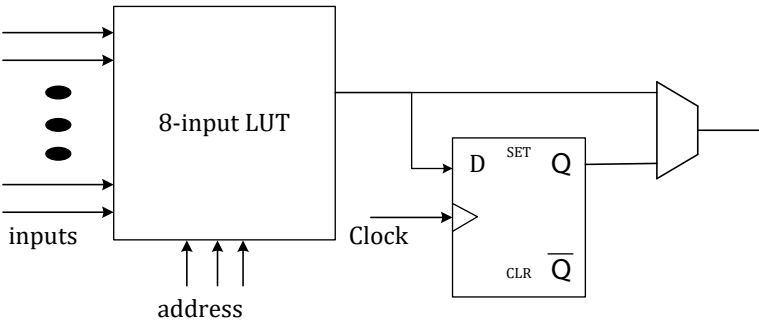
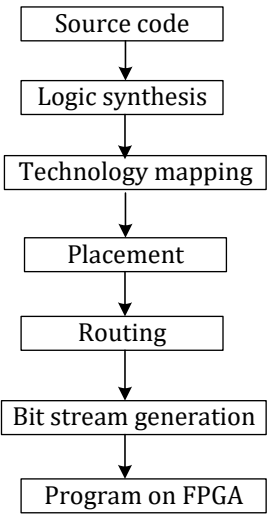


Fig. 12.35 A lookup table (LUT) with registered output

national logic. The LUT shown below has a D_flip-flop so that the combinational output can be registered. The output of the block shown can be the registered output or direct from LUT. A CLB may consist of multiple LUTs with different input sizes and different configurations.

12.11.7 Case Study–2

Design a full adder using input LUTs.

Solution

The truth table for full adder is given below. Given eight input LUTs, two LUTs are sufficient to implement the logic (see Fig. 12.36).

a	b	cy	sum	cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Given a Boolean function, the system has to partition it appropriately to utilize available LUT configurations. Logic synthesis takes care of breaking the function into logic elements and placing them in CLBs (see Fig. 12.37).

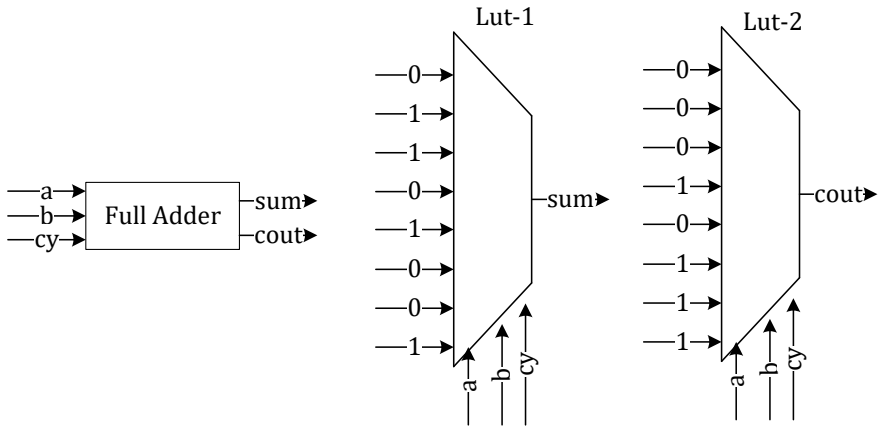


Fig. 12.36 Full adder implemented with three input LUTs

Fig. 12.37 Partitioning the logic for two-input LUTs

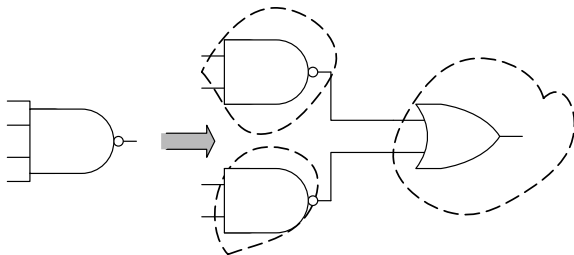
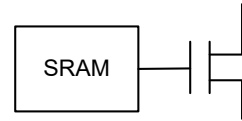


Fig. 12.38 A CMOS switch

If the function is $F = \overline{abcd}$ and you have only two-input LUTs the function has to be partitioned into $F1 = \overline{ab}$ and $F2 = \overline{cd}$ and $F3 = F1 + F2$.

12.11.8 Connectivity Programming

In Fig. 12.31, the logic implemented in the CLBs have to be interconnected and to the IO blocks using the programmable switches. The switches connect the devices in the fabric using the horizontal and vertical wires placed across the device.

The switches are programmed using one of the below technologies (see Fig. 12.38).

SRAM bit cell stores the programmability of the device. The connectivity is made when the device is conducting. SRAM-based configuration is quick and can be done repeatedly. No special fabrication steps are needed. The drawbacks are that it is volatile. At each power up, the device is to be programmed.

Antifuse is by blowing off the fuse. The switch is by default is OFF; when programmed it is ON. There are no delays due to the switch and less area overhead. This is not really a reconfigurable device as fuse cannot be reprogrammed. It is one time programmable.

Flash technology is similar to SRAM but it is flash memory. It is nonvolatile.

12.11.9 Exploiting Reconfigurability

While the computations are done using software the data formats are fixed based on the ALUs. We can exploit FPGA by reconfiguring the arithmetic or logical operations based on the type of data we are handling. This is one example. Several new techniques of computations originate with reconfigurable hardware. While this topic is highly exhaustive, we will study few cases where FPGA reconfigurability is exploited. For further study, please refer to book by Scott Hauck (Hauck and DeHon 2008).

12.11.9.1 Block Floating Format Computations

All computations in generic processors are either fixed point or floating point (Elam and Iovescu 2003). Floating point data is represented as per IEEE standard and

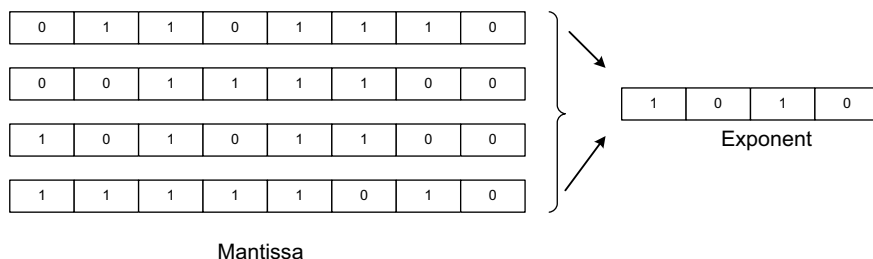


Fig. 12.39 Block floating point computation

complex hardware is needed to process each floating point operation. A floating value consists of mantissa and exponents. The exponents of the two operands have to be normalized and computation has to be done.

In most of the applications, the exponents of all the variables will be roughly close. If so, block point strategy makes all the values to share same exponent. The largest value in a block of data is located. The corresponding exponent and fractional part are normalized to that exponent. The mantissas of the entire data block are adjusted to this common exponent. Now the computations are done over the mantissa values which is simple fixed arithmetic. After certain operations, the exponent value is adjusted if necessary. Effectively we have converted the floating values to fixed-point operations (see Fig. 12.39).

This becomes possible as FPGA can dynamically configure the common floating point register. This mechanism is used in FFT chips using FPGAs.

12.11.9.2 Constant Folding

Let us say we have to compare two 4-bit values using a sequential program, it goes through several instructions and execution units. If the comparison has to be fast, this can be implemented in hardware where the two 4-bit values are given to comparator hardware and 1-bit output check is computed. However, we may not need this hardware when one of the operand $b(3.0.0)$ is a constant value, say, $a = 12$ at this *instance* and remains the same for large amount of computations.. Then the hardware gets mapped to simple 4-bit AND gate. FPGA can be configured to do the computation by a simple 4-bit AND gate. When the value of $b(3.0.0)$ changes, the hardware can be instantly reconfigured. This is called constant folding. This concept of computing is *called instance specific design* (see Fig. 12.40).

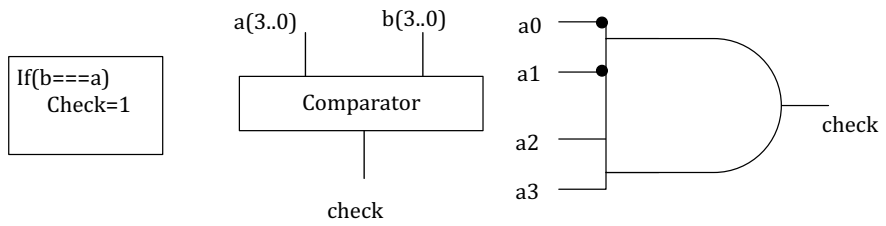


Fig. 12.40 Compare logic using software, hardware, and FPGA—constant folding

12.11.9.3 Constant Coefficient Multiplier

Let us say we have to compute $W = \sum_{i=1}^N a_i x_i$ where a_i is the coefficient with which it is to be multiplied. In normal computations same is implemented by multiplier hardware with a_i and x_i as two operands. If the coefficient is constant for long time, instance-specific design as explained above can be adapted. When a_i is constant, the multiplication can be done without multiplier by simple bit shift logic. Thus, multipliers are avoided. When the coefficient value changes the FPGA is reconfigured for the bit shifts (see Figs. 12.41 and 12.42).

In summary, reconfigurable paradigm of FPGA brings out very performant hardware design in minimal fabric space. Unique constraints and opportunities of the application must be understood to utilize these designs. Data formats need not be generic; they can be optimized with required word lengths based on application. Cordic algorithms, table lookup and additions, and distributed arithmetic are some more designs which can be tuned with reconfigurability.

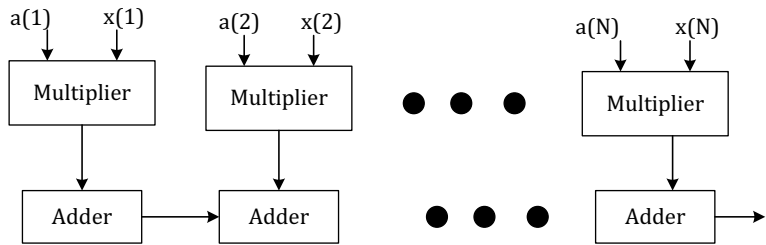


Fig. 12.41 Conventional computation of compute $W = \sum_{i=1}^N a_i x_i$ in hardware

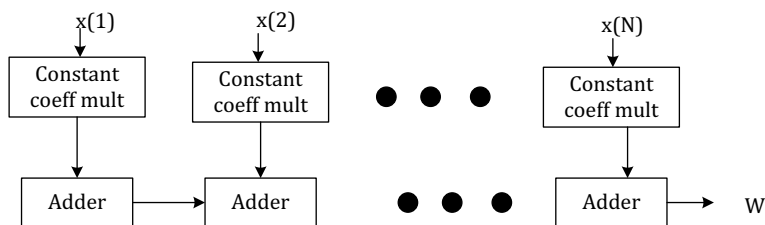


Fig. 12.42 Constant coeff multiplication using FPGA-instance-based design

12.12 Summary

In early days, an embedded system is designed using microcontrollers. The microcontrollers used to be mostly packed with a microprocessor, small amount of memory, certain I/O ports, and the glue logic to extend the hardware. Small systems could be designed with few more chips in a compact way. But as the technology has advanced, current microcontrollers have advanced processor architectures. This chapter covered most common architectures used in embedded processors and generic processors.

Instruction-level parallelism (ILP) has been exploited to maximum extent. Inherent problems in ILP have made to bring thread-level parallelism (TLP) which we have discussed in detail. In certain class of applications, where bulk data processing is involved like multimedia, audio, and streaming, SIMD architectures are very promising. GPUs are current state-of-the-art processors for such applications.

FPGA-based technology has made system-on-chip designs a cake's walk. Systems with high-performance requirements are possible with hardware configured to such requirements. Temporal re-configuration in FPGAs mimicking DLLs in software has made re-use of same FPGA fabric for just-in-time “use and throw” hardware blocks.

12.13 Further Reading

This topic is one-semester course. But, it is very essential to understand any commercial processor architecture. For further reading of this topic, please refer to *Computer Architecture-A quantitative approach*, John L. Hennessy, David A. Patterson, Elsevier (Hennessy and Patterson 2011). Study Gokhale (2005) and Hauck 2008 for more on re-configurable computing. (Developer guides of state-of-the-art processors will help to understand how these concepts are implemented in their architectures).

12.14 Exercises

- Design memory hierarchy for the below specifications:
 - Two-level cache.
 - Common cache for Instructions and Data.
 - L1: Four-way set associative with 64-byte blocks with 32 KB capacity, virtually indexed and physically tagged,
 - L2: Eight-way set associative with 64-byte blocks and 1 MB capacity. Physically indexed and tagged.
 - Memory management is handled by a TLB which is fully associative with 32 entries.
 - Page size: 4 KB.
 - 32-bit virtual address.
 - 32-bit physical address.

- A matrix has to be transposed where the elements are stored in row major order. Re-write the code below to improve the spatial locality in mind.

```

    for (i = 0; i < 256; i++) {
    for (j = 0; j < 256; j++) {
        output[j][i] = input[i][j];
    }
}

```

- Consider a two-level memory hierarchy made of L1 and L2 data caches. Both caches use write-back policy on write hit and both have the same block size. The hierarchy can be designed
 - Case 1: As inclusive hierarchy meaning that all data in the L1 cache must also be somewhere in the L2 cache (not vice versa).
 - Case 2: As exclusive hierarchy meaning that data is guaranteed to be in at most one of the L1 and L2 caches, never in both.

List the actions taken in response to the following events:

- An L1 cache miss when the caches are organized in an inclusive hierarchy.
 - An L1 cache miss when the caches are organized in an exclusive hierarchy.
- Suppose that in 1000 memory references there are 40 misses in the first-level cache and 20 misses in the second-level cache. What are the various miss rates? What is the average memory access time and average stall cycles per instruction? Ignore the impact of writes
Assume following parameters:
 - miss penalty from the L2 cache to memory 200 clock cycles,
 - hit time of the L2 cache is 10 clock cycles,
 - hit time of L1 is 1 clock cycle, and
 - there are 1.5 memory references per instruction.

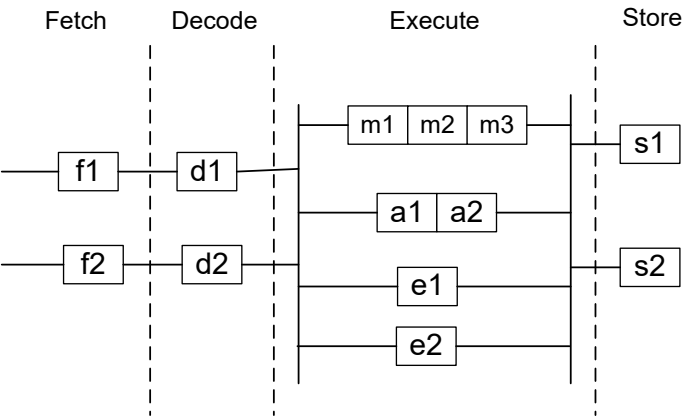


Fig. 12.43 Super scalar processor

5. A super scalar processor is capable of fetching and decoding two instructions at a time and two instances of write-back pipeline stage. It has three separate functional units. Six instructions (I1 to I6) with following constraints have to be executed. How many cycles does the processor take to execute the instructions if it has instruction issue policy of in-order issue with out-of-order completion. Pl fill-up the table below showing the pipeline status. Vertical cells represent clocks and horizontal the parallel units.
6. Figure 12.43 shows a super scalar processor organization. The processor can issue two instructions per cycle if there is no resource conflict and no data dependence problems. There are essentially two pipelines, with four processing stages fetch (f1, f2), decode (d1, d2), execute, and store (s1, s2)). Each pipeline has its own fetch decode and store unit. Four functional units (multiplier (m1, m2, m3), adder (a1, a2), logic unit (e1), and load unit (e2)) are available for use in the execute stage and are shared by the two pipelines on a dynamic basis. The two store units can be dynamically used by the two pipelines, depending on availability.
Consider the following program to be executed on this processor:

```
I1: Load R1, A (A)→R1
I2: Add R2, R1 R1+R2=R2
I3: Add R3, R4 R4+R3=R3
I4: Mul R4, R5 R5*R4=R4
I5: Comp R6 Complement R6
I6: Mul R6, R7 R7*R6=R6
```

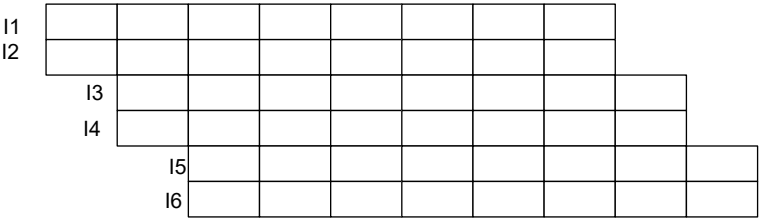


Fig. 12.44 Pipeline activity

Show the pipeline activity filling the slots below. Label each box with the pipeline unit name like f1, d1, m1, etc. If there is a stall, mark nothing. Assume in-order issue and in-order completion policy. Repeat same with in-order issue and out-of-order completion (Fig. 12.44).

7. Design using 4, 3, and 2 input LUTs the below Boolean function:

$$F = A0A1A3 + A1A2\overline{A}3 + \overline{A}0\overline{A}1\overline{A}2.$$

8. You have three-input two-output LUTs in the FPGA fabric. Implement the below logic function using minimum number of LUTs.

$$w = ab;$$

$$x = a\overline{b};$$

$$y = ab + cd;$$

$$z = ab + c\overline{d}$$

9. Below is the methodology by which a value (0...16) is represented symbolically (low, medium, high). From the below graph, the value can be classified into one or more symbols with certain membership (symbolic value). For example, a value of 5 means it is “Low” with 50% membership and “Medium” with 50% membership. Similarly, a value of 8 is “Medium” with 100% membership. We

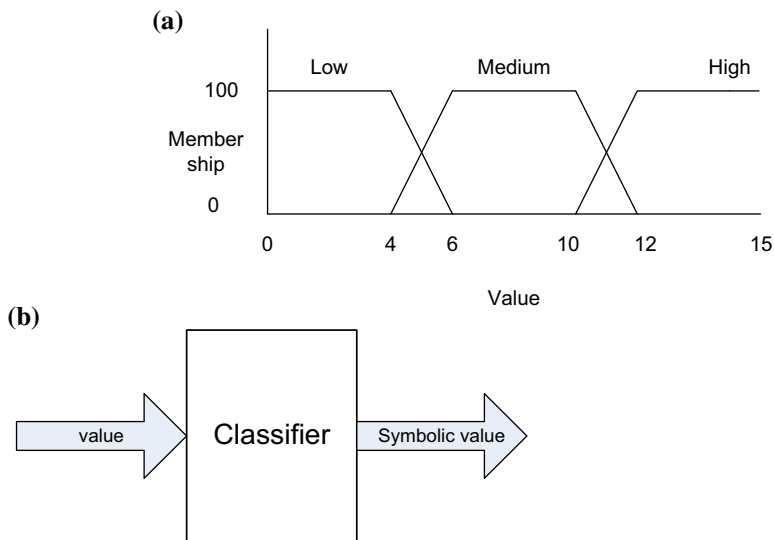


Fig. 12.45 a, b Classifier

need to design a circuit using FPGA to classify the input value which is a 4-bit integer value into corresponding symbolic value. The symbolic value is coded to represent the symbols and respective membership. Assume the percentage of membership is represented as integer value (0.0.100) (Fig. 12.45).

References

- Cortex™-A8 (2006) Revision: r3p2, technical reference manual
- Elam D, Iovescu C (2003) A block floating point implementation for an N-Point FFT on the TMS320C55x DSP. Application Report SPRA948
- FPGA architecture overview, Xilinx Corp (2020)
- Gokhale M, Graham PS (2005) Reconfigurable Computing. Springer
- Hassidim A (2010) Cache replacement policies for multicore processors. In: Innovations in computer science
- Hauck S, DeHon A (2008) Reconfigurable computing: the theory and practice off FPGA based computation. Elsevier Publications
- Hennessy JL, Patterson DA (2011) Computer architecture—a quantitative approach. Elsevier
- MIPS® Architecture for programmers, vol I-A: introduction to the MIPS32®. Architecture, Document Number: MD00083 (2020)
- Nagraj SV (2004) Cache replacement algorithms. The international series in engineering and computer science book series (SECS, vol 772)
- Sweety CP, Chaudhary P (2018) Branch prediction techniques used in pipeline processors: a review. Int J Pure Appl Math 119(15)

Chapter 13

Embedded Platform Architectures



Abstract We have studied the current processor architectures and the direction in which they are advancing for higher performance. Data crunching needs are bringing advances in complex GPUs. Multi core and multi-threaded processor architectures are executing more number of instructions per clock and high level of integration is realized through Systems-on-chip. But processors cannot perform independently unless they are interfaced with external peripherals having similar performance. The capabilities of SoC devices have to be expanded through certain interfaces. The peripherals can be like Input/Output devices, hard disk storage, extended memory, cache and memory controllers and so on. So all SoC devices have in-built interfaces to extend their capabilities and interconnect to multiple types of peripherals. Though the peripheral controllers are fast, the way they interconnect with processors should also be efficient. Communication among cores, processor to peripherals is done through bus. Bus architectures are also advancing for high throughput, fast event response and bus extension capabilities. Bus connectivity has been standardized so that multiple heterogeneous peripherals can be interconnected seamlessly. In this chapter, we will study some important peripheral interconnects and bus architectures which lead to efficient embedded platforms. Introduction to bus and basic modes of data transfers across processors and peripherals are described in Sect. 13.1 and 13.2. Typical Arm™ plat form with AMBA bus is described in Sect. 13.3 and 13.4. Important IO interface standards like USB, Bluetooth etc. are described in Sect. 13.5. Emerging IoT platform for embedded systems is introduced in 13.6. To summarize, challenge lies in selecting proper plat forms for distributed embedded systems. It depends on the data throughputs and real time nature of networks.

Keywords Synchronous data transfer · Asynchronous data transfer · Burst transfers · AMBA bus · Advanced System Bus (ASB) · Advanced Peripheral Bus (APB) · Universal Serial Bus (USB) · Bluetooth · I²C bus · Internet of Things (IoT)

13.1 Introduction to Bus

An embedded platform consists of a wide range of peripheral devices with different characteristics which get interconnected. This concept is nothing new and exists since the days a processor has evolved. Devices used to be connected through serial interfaces like RS232C, RS485, etc. Paralleled connectivity to printers used to be through Centronics Interface. Multiple processor-based hardware boards used to be connected through standard parallel bus interfaces like VME, Multibus I and II. PC-based systems are extended with ISA bus (Industry standard Architecture), EISA, PCI (Peripheral Component Interconnect), and so on. To illustrate a VME bus-based processor board and its chassis are shown in Fig. 13.1. This standard bus is used in industrial applications.

Typical I/O devices with which user interacts is like keyboard, display units, cameras, printers, etc. In embedded systems, the I/O devices are vast with heterogeneous characteristics like ADC, DAC, Codecs, communication controllers, interrupt controllers, etc. Several such devices need to intercommunicate in real time. These devices communicate in a standard way conceptually as shown below Fig. 13.2.

All the devices can be classified into masters and slaves. Masters have the capability to address any other slave device, apply control signals for a specific operation like read from slave or write to slave. The write data may be a coded command to the slave. At any instance, the bus is under control of one master. That device is called *bus master*. Once a master completes its transaction, it relinquishes and other master can gain bus master ship. This process is done through *bus arbitration*. Bus arbiter reads *bus requests* from masters and based on certain logic, it grants the bus mastership to one master. After bus transaction cycle, another master captures the



Fig. 13.1 Processor board, chassis, and block diagram of VME bus (Courtesy “File:3b2-vme.jpg” by Shieldforyoureyes Dave Fischer is licensed under CC BY-SA 3.0)

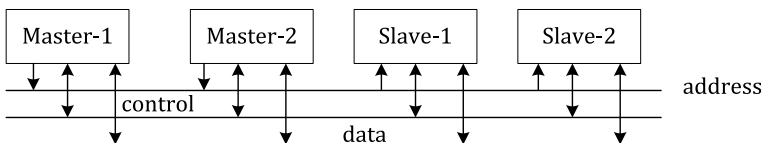


Fig. 13.2 Generic bus architecture

bus. If there are no requests, the bus will be idle. Bus arbitration logic can be with one device or the logic can be distributed among the masters. Slave devices can only be listeners on the bus. When a master addresses them and places a command through control signals, it responds by writing the data on data lines or read the data. This is multi-master bus architecture.

13.2 Data Transfers

Data transfers across devices are classified as synchronous and asynchronous.

13.2.1 Synchronous Transfers

In synchronous transfer, the master places the command and data to write into slave. Master expects that the slave is ready and removes the data at a specified time. This is synchronous write operation. In synchronous read, the master requests the slave through read command and expects data available on bus within a specified time and completes the transaction. This protocol assumes the slave will behave synchronously with master. This protocol is fast but can miss data as the slave may not be ready. Refer Fig. 13.3.

13.2.2 Asynchronous Transfers

Master sends a request to the slave device and waits for acknowledgement from the slave. When the slave is ready to provide the data, it places the data on the lines and sends an acknowledgement. Master reads the data and removes the request. Slave removes the acknowledgement and also the data. So the data transfer takes based on slave and master's response time and availability. This is essential because I/O devices (slaves) are very slow compared to the processor which is master. This is also called *handshaking*. Refer Fig. 13.4.

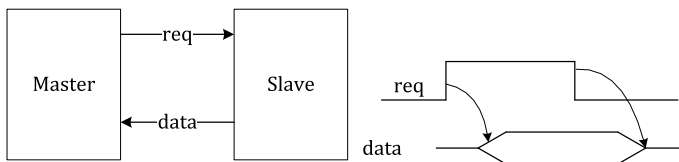
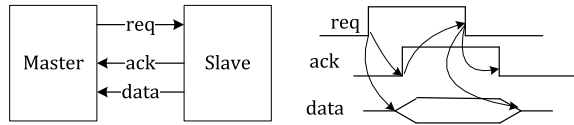


Fig. 13.3 Synchronous data transfer

Fig. 13.4 Asynchronous data transfer



13.2.3 Burst Transfers

Mostly used in data transfers from processor to memory. Normal memories read or write is a single operation where processor places the address and the control signal to read or write. The data is read or written by the processor. When a block of data is to be transferred which is in sequence, the data can be read or written within the same read/write cycle without placing address. This increases throughput heavily. Most modern processors use burst data transfer cycles.

13.2.4 IO Addressing

I/O devices are addressed by the master either port based or address based. Simple microcontrollers have few ports where the I/O device can be connected. These I/O devices are accessed by the port number. This mechanism provides limited number of devices for a simple system. When more devices are needed, the devices are mapped into the memory address of the processor. This is called *memory mapped I/O*. certain processors have a specific address range for I/O devices. In such cases, the IO devices are mapped to the IO address space. This is called *I/O mapped I/O*.

13.3 Typical ARM Platform with AMBA Bus

Advanced Microcontroller Bus Architecture (AMBA) is the specification which defines the communication across high-performance microcontrollers (AMBA™ Specification (Rev 2.0)) (2020). This includes three different bus specifications as below:

Figure 13.5 shows a typical AMBA-based microcontroller system.

13.3.1 Advanced High-Performance Bus

Advanced High-performance Bus (AHB) is high-speed bus providing communication among the connected processors and the slaves like on-chip memory and off-chip external memory. The data transfers are pipelined. It supports multiple bus masters.

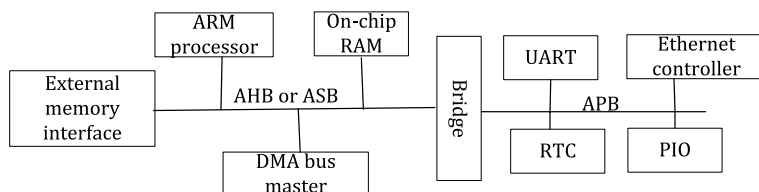


Fig. 13.5 Typical AMBA-based microcontroller

Supports burst mode data transfers. Supports split transaction. It has 32-bit system address bus. Data transfers can be 8, 16, or 32 bits per each cycle. The protocol allows doing transfers up to 1024 bits. A bus cycle can be a burst transfer. Master can specify the size of burst as 4, 8, or 16. During the burst, the address can be programmed to be incremented or wrapped at a particular boundary address. Each slave on the AHB is selected by the select signal generated from the combinational decode of the address bus.

AHB is multi-master supported. The centralized arbiter grants the bus to one of the bus requests done by bus masters. A bus request signal from a master indicates that it needs the bus. Arbiter supports up to 16 bus masters. When a bus is granted to a master, arbiter can preempt to a higher priority master in the next cycle. When a master is doing some critical operations (atomic operations), the master can request to LOCK the bus till it relinquishes the bus.

SPLIT transfers allow a bus transaction to be split. The story starts when a master (say master 12) requests a slave for data transfer. Assume the slave has no data and needs some time to get it. The slave informs the arbiter with a request that the data transaction may be split. This indicates to the arbiter that the master 12 should not be given bus grant at this stage, but it should be given when the slave is ready with the data as indicated to the arbiter. The arbiter masks the 12th master. When the data is ready, the slave asserts the 12th bit (indicating master 12) in the HSPLIT response. The arbiter then unmask the 12th master and master 12 gets the bus in due course to do the transaction. Thus split transfers improve overall utilization of the bus. The bus consists of 16-bit SPLIT bus used by the slave to indicate to the arbiter for the respective master attempt the transaction.

Bus transactions are not truly synchronous. When a bus request, the OKAY signal indicates to the master that the transaction is going normally and when HREADY signal goes high, it indicates that the transaction is completed.

In a simple bus cycle, master drives the address and control signals on the raising edge of bus clock. Slave samples address and control in next cycle. Slave drives the bus and places data on the third cycle. During the same third cycle master places next address and control for the next cycle. Thus address phase of any transfer occurs during data phase of previous cycle. Thus, it is pipelined (see Fig. 13.6).

- *Advanced System Bus (ASB)* provides communication among processors, on-chip memory, and off-chip memory interfaces. This is an alternative to AHB where you do not require high-performance features of AHB.

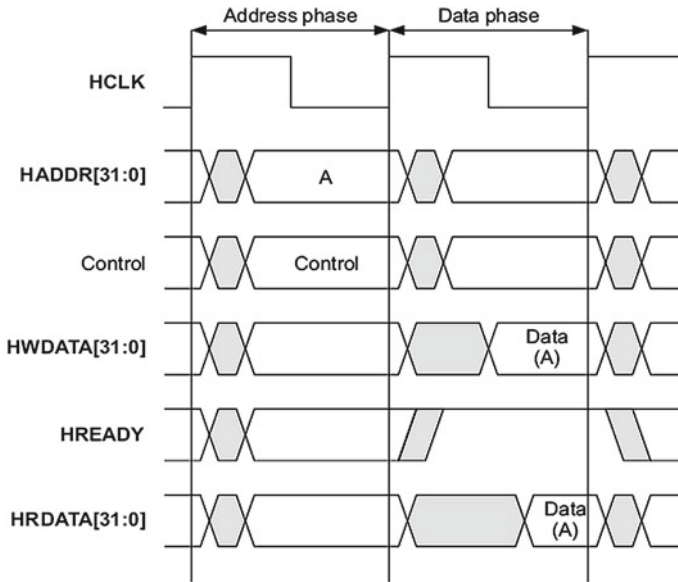


Fig. 13.6 Pipeline in Arm™ (“Arm copyright material kindly reproduced with permission of Arm Limited”)

- Advanced Peripheral Bus (APB) is the peripheral bus providing interface with multiple peripherals. APB works with both AHB and ASB. APB works as a secondary bus at lower speeds compared to system bus and provides communication interface with peripherals which have low data rates, and accessible as memory mapped registers.
- The *bridge* on the system bus connects the bus to the low speed APB. It converts AHB or ASB transfers for the slave devices on APB. The bridge latches the address, data, and control signals from the system bus and generates appropriate signals to select the slave and complete the transaction. The bridge appears as a slave on system bus. It provides hand shaking between the system bus and the peripherals on APB.

AMBA defines 4, 8, 16 beat burst. The bursts can be incremental where the sequential locations are accessed. In the case of wrapping burst transfers, the address of the transfers in the burst will wrap after the boundary is reached. Figure 13.7 shows four transfers to addresses 0×38 , $0 \times 3C$ and 0×40 , 0×44 .

13.4 Generic Interrupt Controller (GIC)

We have studied programmable interrupt controller devices used in Intel $\times 86$ family and in microcontrollers. In this chapter, we will study briefly the interrupt

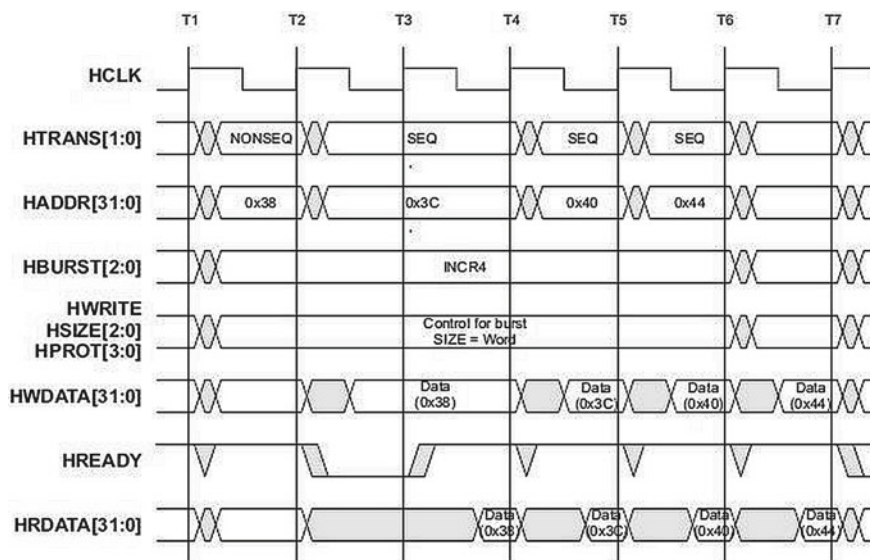


Fig. 13.7 4-beat incrementing burst operation (“Arm copyright material kindly reproduced with permission of Arm Limited”)

controller used with ARM processors having advanced functionality to support multiple processors and virtual machines.

Arm supports two types of hardware interrupts IRQ and FIQ. IRQ is general interrupt request processed by ISR. FIQ stands for fast interrupt request. FIQ is higher priority interrupt which disables IRQ and other FIQ handlers. So, no other interrupts can occur during FIQ servicing. GIC manages interrupt sources, interrupt behavior, routing the interrupt to one of the processors. Major functionality of GIC is listed below (ARM® Generic Interrupt Controller Architecture version 2.0, Architecture Specification 2013).

- Generates interrupts to the processor from peripheral interrupt sources.
- Enabling and disabling of interrupts.
- Interrupt masking and prioritization.
- Handle Software Generated Interrupts (SGI).
- Handle interrupts in uni-processor and multi-processor environments.

ARM provides certain features to develop secured software with built-in secured hardware extensions. The interrupt mechanism supports the secured and non-secured environment. As we have not dealt with security in embedded systems, this topic will be covered in secured embedded systems.

Virtual machines are created on a physical machine using hypervisor technology. Each virtual machine is an independent machine created in virtual environment. Such machines also need to handle certain interrupts from external peripherals. Hypervisors can generate virtual interrupts to the virtual machine or handle the physical

interrupt itself. As we are talking about embedded systems, virtual machine creation is not that common and so we do not get into these details.

GIC can manage the following types of interrupts.

Peripheral interrupts are physical signals given from external sources to the GIC. GIC handles two types of peripheral interrupts. *Private Peripheral Interrupts (PPI)* are specific to a single processor. *Shared Peripheral Interrupts (SPI)* are interrupts which can be serviced by any selected set of processors. The GIC handles SPIs, routes them to relevant processors, and gets them serviced. The interrupts can be edge triggered or level triggered.

Software generated Interrupts (SGI) are generated by software and communicated to the GIC to handle them. SGI can occur in uni-processor or multi-processor environment. When SGI occurs in multi-processor environment, the ID of the CPU identifies the processor requesting the interrupt. This mechanism is used to make inter-processor communication.

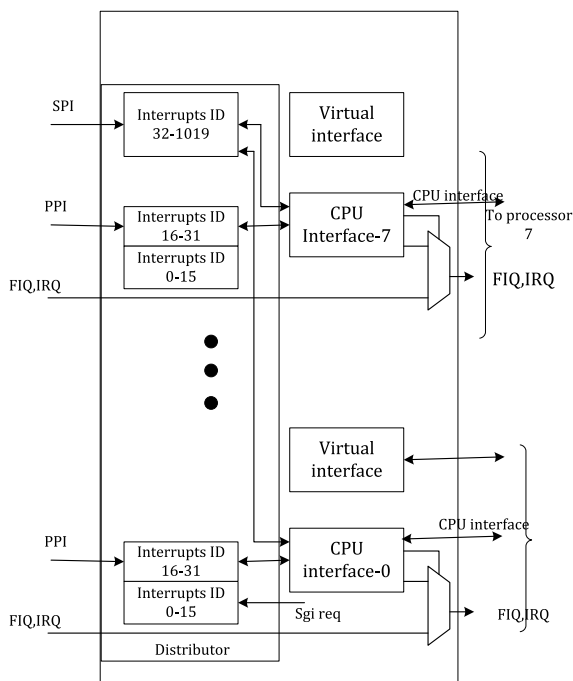
In multi-processor environment, the interrupts are handled in two ways. One way is that the interrupt is handled by one processor. The system has to configure which processor will handle this interrupt. The other way is that all the processors receive the interrupt. Any processor can acknowledge the interrupt. Once it acknowledges, the interrupt pending state of that processor gets cleared. All other processor's state remains as pending.

A processor may initially configure for an interrupt source and waiting for the interrupt. But, under some context, the processor does not require the interrupt anymore. In such situations, when the interrupt is received by the GIC and it signals to the processor, processor indicates that it does not require that interrupt anymore. In such contexts, GIC handles that as a *spurious interrupt*.

In multi-processor configuration, interrupts generated through PPIs and SGIs, the GIC can set same interrupt ID. Such an interrupt is called a *banked interrupt* and is identified uniquely by the combination of its interrupt ID and its associated CPU interface.

Figure 13.8 illustrates the simplified GIC architecture. It consists of distributor block and CPU interface block. GIC supports up to 8 CPU interfaces. The distributor block prioritizes the interrupts, enables and disables the interrupts, sets priority level, and distributes them to the CPU. CPU interface block performs priority masking and preemption handling for the connected processor. Each CPU interface block performs priority masking and preemption handling for a connected processor in the system. Each interrupt is identified by an ID. A CPU can service up to 1020 interrupts. IDs 0–31 are private to a CPU. (PPIs) A PPI is forwarded to a particular CPU interface and is private to that interface.

Each CPU interface enables interrupt requests to the processor. It acknowledges the interrupt, indicates completion of interrupt service, sets interrupt priority mask for the processor, and defines preemption policy. Version 3 of GIC supports high interrupt counts and more processors. For more details please study Arm™ GICv3 which offers support for much higher interrupt counts and larger numbers of processors (Arm® Generic Interrupt Controller Architecture Specification, GIC architecture version 3 and version 4).

Fig. 13.8 Simplified GIC architecture

13.5 Modern IO Interfaces

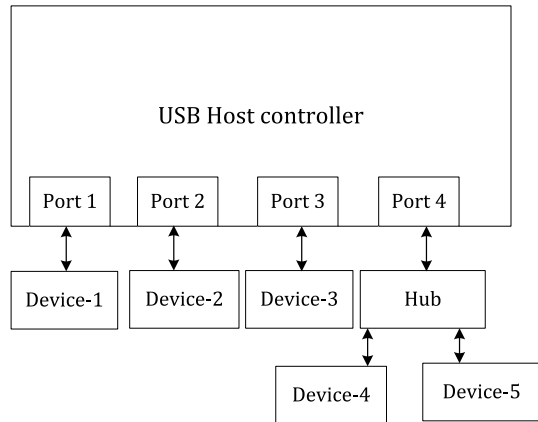
The interface of IO devices with processors is advancing day by day with new protocols and architectures. This is because the necessity to match the performance in both directions. As processors advanced with high throughput and high computing powers, the IO device interfaces also have to advance to match the throughputs. In this section we will study some important IO interfaces relevant to embedded systems.

When a processor connects to an external application specific I/O device, the device should be capable to get mapped into the address space of processor. The protocol should allow the external devices to read and write into the system. The IO device should be able to signal the system through interrupt mechanism for initiating a transaction. The device should allow the system to be expandable with more IO devices.

13.5.1 Universal Serial Bus (USB)

Universal Serial Bus (USB) is a serial bus. Current prevailing versions are USB 2.0 and 3.0. The current description in this section pertains to USB2.0. It communicates between a single host and multiple devices. The bus is controlled by host. There

Fig. 13.9 Multi-tiered star topology of USB



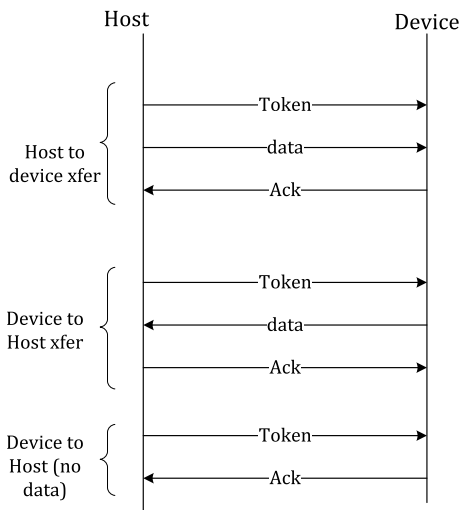
can be only one single host in the system. USB is a multi-tiered star topology (see Fig. 13.9). A maximum of 127 devices can be connected to the host in the network. Hubs provide additional fan out for the bus. For details study: USB in a Nutshell, beyond logic (2018) and USB—Universal Serial Bus 3.0 and 2.0 Specifications-Intel corporation (2010).

The physical connectivity is low voltage differential pair of wires. It is 4 wire system with +5 and ground and data over twisted pair differential signals and uses NRZ (non-return to zero) encoding scheme. D[−] and D⁺. Communication speeds are 1.5, 12, and 480 Mb/s. USB device indicates its speed by pulling either the D⁺ or D[−] line high (3.3 v). These pull up resistors at the device end will also be used by the host or hub to detect the presence of a device connected to its port.

13.5.1.1 Host Protocol

USB host undertakes all transactions and schedules bandwidth. Data transactions are done using token-based protocol. It is a polled bus. Most bus transactions need three separate packets to be exchanged (Fig. 13.10).

- Host starts the transaction with a token packet. An IN token solicits data from the device to the host. An OUT token indicates that host sends data to the device over the bus. The packet indicates the transaction type and the direction of the transaction. It contains the device ID and end point address. The end point is a logical channel identifier on the device. There can be 15 end points within a device.
- After the token has been received by the device, the data transaction is generated either by the host or by the device depending on the direction specified.
- Once the data transaction is complete, the handshake packet is generated. The ACK packet is generated by the device which receives the data.

Fig. 13.10 Transaction protocol

A pipe is a logical connection between an endpoint and the host. USB transfers data and control messages across the USB host and devices using a set of logical pipes. The pipes can be unidirectional or bidirectional. From the software point of view this is a direct connection and hides the details of the bus hierarchy. Pipes are set up with parameters like the bandwidth allocated to the pipe, the type of data transfer and maximum packet size.

13.5.1.2 Data Transfer Types

Control transfers

Control Transfers are the packets to configure the device which is discovered on the bus. They are usually used to set up the endpoints on the device.

Bulk Data Transfers

Bulk Data Transfers are used for large-scale transfer of data to and from the device. They are used when no special latency constraints exist in the transfers to or from the device. The data exchange is reliable. The bandwidth occupied by bulk transfers can vary depending on the other bus activities; these transfers are the lowest priority. Some bulk data transfer types are print jobs, image transfers from scanners, etc. Bulk transfers provide error correction using CRC16.

Interrupt Data Transfers

Interrupt Data Transfers are used for timely delivery of data to and from the device. These can be used for events such as mouse movements, or they can be used from a

device that wishes to indicate that data are available for a bulk transfer. This avoids constant polling of the bulk endpoint.

Isochronous Data Transfers

Isochronous transfers are continuous data transfers in real time. Guaranteed bandwidth is allocated for such data transfers to occur in real time. They are usually used for real-time transfers of audio and video.

13.5.1.3 Common USB Packet Fields

Figure 13.11 shows the fields in a USB packet which are explained briefly below.
Sync: All packets start with 8 bits for low and full speed and 32-bit long for high speeds. Used to synchronize the clock at the receiver end. The last two bits indicate where the PID fields start.

PID: packet ID identifies the type of packet.

Group	PID value	Packet identifier
Token	0001	OUT Token
	1001	IN Token
	0101	SOF Token
	1101	SETUP Token
Data	0011	DATA0
	1011	DATA1
	0111	DATA2
	1111	MDATA
Handshake	0010	ACK Handshake
	1010	NAK Handshake
	1110	STALL Handshake
	0110	NYET (No Response Yet)
Special	1100	Preamble
	1100	ERR
	1000	Split
	0100	Ping

ADDR: Specifies the device address to which this packet is designated.
ENDP: The endpoint.
CRC: Cyclic Redundancy Checks performed on the data.

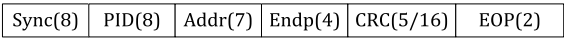


Fig. 13.11 Fields in a USB packet

EOP: End of packet. Indicated by a Single Ended Zero (SE0) for 2-bit times (approx.) followed by a J state for 1-bit time.

13.5.1.4 USB Packet Types

USB has four different packet types.

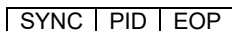
Token Packets: Host informs the device to read/write data or setup.



Data Packets There are two types of data packets each capable of transmitting up to 1024 bytes of data. Data packets: transmit data up to 1024 bytes. Low-speed devices transmit 8 bytes.



Handshake Packets There are three types of handshake packets which consist simply of the PID. They are ACK, NAK, and STALL.



Start of Frame Packets The SOF packet consisting of an 11-bit frame number is sent by the host every 1 ms on a full speed bus or every 125 μ s on a high-speed bus.



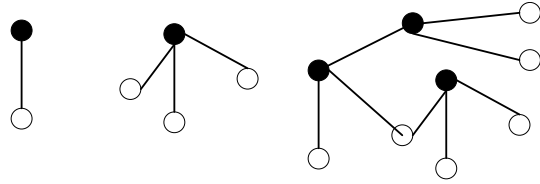
13.5.2 Bluetooth

When any two devices have to communicate, the first point which arises is the physical connectivity. Earlier several protocols like RS-232C, RS-485, etc. were used to connect nearby devices over twisted pair lines.

Bluetooth made a revolutionary change by providing wireless connectivity across any two local devices. Bluetooth is low cost, low-power radio frequency for short-range wireless communications. Bluetooth works with the broad specifications given below.

- 2.4 GHz ISM band, Frequency hopping
- Gaussian-shaped BFSK Modulation
- 723 Kbps Data rate

Fig. 13.12 Bluetooth connectivity configurations



- Operating range 10–100 m
- Power 0.1 W (Active)
- Security-Link layer authentication and encryption
- RF:
 - Carrier frequency: $f = 2402 + k \text{ MHz}$ $k = 0 \dots 78$
 - Hopping rate: 1 hop/packet. 1600 hop/s for 1 slot packet
 - Channel bandwidth: 1 MHz (–20 dB) 220 kHz (–3 dB)
 - uses spread spectrum.

There are two types of Bluetooth technology as of 2020: Bluetooth Low Energy (LE) and Bluetooth Classic. Mostly the devices use LE because it needs low energy.

13.5.2.1 Bluetooth Network

Figure 13.12 illustrates a Bluetooth providing ad hoc connectivity. Every Bluetooth unit can connect to other Bluetooth devices without the need of any infrastructure support or access points. The network formed with the Bluetooth-connected devices is called piconet. A piconet is sometimes called a personal area network (PAN) because the range of optimal operation for Bluetooth is 10 m.

13.5.2.2 Bluetooth Architecture

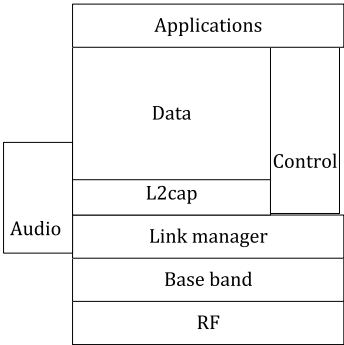
See Fig. 13.13.

13.5.2.3 Bluetooth Layers and Protocol Stack

RF Layer

Figure 13.13 shows that a Bluetooth RF layer is the physical layer of the network. Transmission is over 2.4 GHz ISM band in the range of 10 m across the devices. The frequency band is divided into 79 channels each of 1 MHz. Bluetooth uses Frequency Hopping Spread spectrum technique. The modulation frequency of the carrier is hop based algorithm. This avoids any interference from other devices. The hopping rate

Fig. 13.13 Bluetooth protocol stack



is 1600 times per second. The baseband data is modulated using GFSK, a derivative of FSK with Gaussian bandwidth Filtering.

Baseband Layer

Baseband layer is close to MAC layer of OSI. Bluetooth uses TDMA where the master and slave communicate in assigned time slots. It is half-duplex connection. If there is one slave only in the piconet, both master and slave use alternate time slots. In baseband layer, the master and slave can be linked as asynchronous connectionless link (ACL) or synchronous connection link (SCO). Using ACL, data is delivered through a link established with master. Frames can be lost as it is connectionless. Maximum 721 kbps data rates can be established. In connection-oriented link, a connection is established by reserving certain slots. This link provides fast but not accurate delivery. Used for audio streaming. A slave can have three SCO links with master.

Logical Link, Control Adaptation Protocol Layer (L2CAP)

This layer is similar to Logical Link Control in OSI. Gets packets up to 64 Kb from upper layer, and segments them into frames for transmission. Same layer re-assembles the packet at destination. L2CAP multiplexes the data from sender and at the receiving side. It extracts data addressed to that receiver by demultiplexing. L2CAP negotiates maximum payload across the devices during connection establishment.

13.5.2.4 Bluetooth States

The device can be in four states. The default state for a Bluetooth unit is *Standby*. The unit in the connection state can be in **active** mode, **sniff** mode, **hold** mode, or **park** mode.

In *active mode* the master connects to seven devices and will be in master/slave communication. The slave listens in master-to-slave slot if a frame is addressed to it. The master polls the slaves regularly. If not it will sleep till master-to-sleeve state. In *hold mode* it frees the slave for a predetermined time. The **hold** mode is negotiated between the slave and the master.

In *sniff mode*, it frees the slave at a periodic cycle. In the **sniff** mode, the slave reduces its activity by listening only to slots of interval Tsnif.

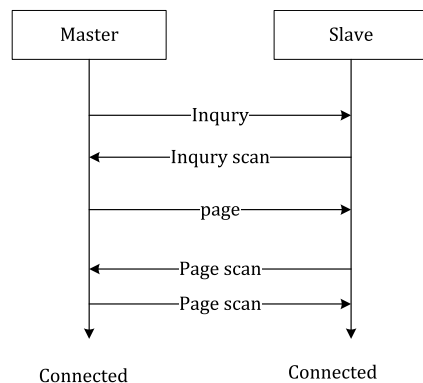
In *Park mode*, master enables to connect to as many as 255 devices and maintains 7 active devices only. In the park mode, the slave gives up its active-address. It gets a new 8-bit parked-address. A slave in parked state has very little activity. It only listens to the beacon channel to synchronize and checks for broadcast messages. The unit in park state has minimal energy consumption.

Bluetooth provides ad hoc connectivity. Every Bluetooth unit can connect to other Bluetooth devices without the need of any infrastructure support or access points. A member of one piconet could also be a member of another piconet. A unit participating in multiple Piconets does so on time division basis. When a unit is leaving a piconet, it indicates the master, it will not be available for a timed interval and places itself in sniff, hold or park mode. It synchronizes its clock to another piconet and joins the conversation there. Such a unit may act as the bridge between two piconets.

13.5.2.5 Connection Process

The unit (master) who wants to build a connection with other units enters the **inquiry** state to see if there are others nearby (see Fig. 13.14). If another unit happens to be in

Fig. 13.14 Bluetooth connection process



inquiry scan state and receives its inquiry message, it will respond to the master with information of its Bluetooth device address. The master unit then enters **page** state and uses the slave's Bluetooth device address to construct a paging message. The slave in the **page** scan state will be able to receive this paging and return a response. The master will send a FHS packet to help the slave to synchronize to the master clock. Then, a connection is established between the master and the slave.

13.5.2.6 Next-Generation Bluetooth

As stated by Bluetooth™

- LE Audio will include a new high-quality, low-power audio codec, the Low Complexity Communications Codec (LC3). Providing high-quality even at low data rates.
- The effective, reliable range between Bluetooth devices is anywhere from more than a kilometer down to less than a meter.
- Bluetooth mesh continues to revolutionize the IoT. It plays a pivotal role in the development of IoT applications like Smart Building, Smart Industry, Smart Cities, Smart Home, etc. For more detailed study refer Bluetooth architecture, AHIR labs (2017)

13.5.3 Low-Performance Device Interconnects

13.5.3.1 Inter-Integrated Circuit Bus (I²C Bus)

I²C bus was introduced by Philips in the early '80s to allow easy communication between multiple devices in a simplistic way. This is very much used in instrumentation to communicate across slow devices. Simplicity and flexibility are key characteristics that make this bus attractive to many applications. I²C is two-wire buses with clock and data transmitted serially. There can be multiple masters on the bus communicating with multiple slaves or multiple masters. The bus is bidirectional and driven by open-collector gates driven by low clock speeds.

Typical I²C bus for an embedded system is shown in Fig. 13.15 where multiple slave devices like IO expanders, sensors, etc. are controlled. This bus uses open collector with input buffer which enables bidirectional data transmission as shown in Fig. 13.16a, b. For more details study Understanding the I²C Bus Texas Instruments (2015).

Any master or slave will be in high impedance state when inactive. They will neither pull the bus high or low. The bus is pulled-up by the R_{up} resistor to high. When a master is transmitting, the FET pulls down the bus by going to conducting state. At any time only one can pull the bus down and indicates data transmission. Slaves read the signal.

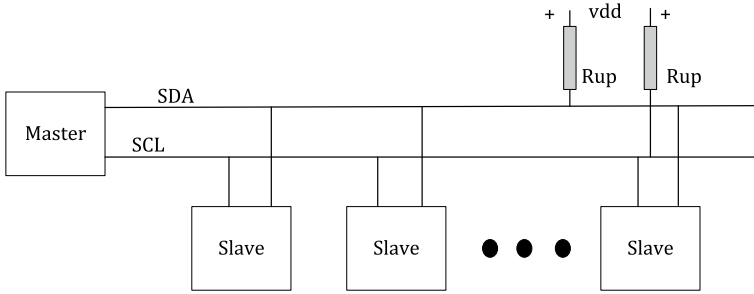


Fig. 13.15 I²C bus with a single master and multiple slaves

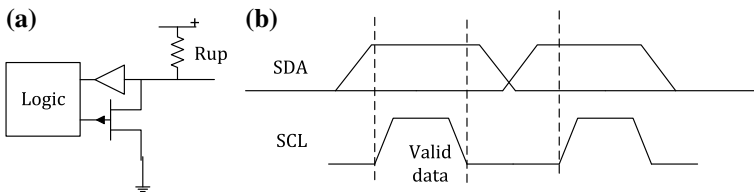


Fig. 13.16 a Pullup logic for each device b serial data transfer

Each device, either master or slave has two lines. SDA is the data line and SCL is the clock (Fig. 13.16b). Each device on the I²C bus has a specific device address to differentiate between other devices. The communication across the devices will be in the following four states.

Bus Not Busy—Neither master nor any slave are transmitting or receiving. The bus is idle when both SDA and SCL are high (logic one).

Starts Bus Transfer—All commands start with a start bus transfer signal. This is indicated with a high to low transition of SDA when SCL is high.

Data Transfer—Data is placed when SCL is low; when the SCL goes high, the data is considered valid. The first byte of data transfer consists of slave address (7 bits) and one read/write bit to indicate the type of transfer.

Acknowledge—Ack or Nak is done after every byte. The slave will generate an acknowledge cycle. Master releases the SDA line during the acknowledge clock pulse and slave pulls the SDA line low to indicate ACK. If the SDA line remains high during this clock phase, it is treated as a NACK.

Stop Bus Transfer—A rising edge of SDA while SCL is high indicates completion of bus transfer. Bus returns to Bus Not busy state.

Most of the slave devices require configuration upon startup. This is typically done when the master accesses the slave's internal register maps. A device can have one or multiple registers where data is stored, written, or read.

The communications protocol for a master to access a slave device is as follows:

- Master sends send data to a slave:

- Master sends a START bus transfer and addresses the slave.
 - Master sends data to slave.
 - Master terminates the transfer with a STOP bus transfer.
- Master wants to receive data from a slave:
 - Master sends a START bus transfer and addresses the slave.
 - Master sends the requested register to read to slave.
 - Master receives data from the slave.
 - Master terminates the transfer with a STOP bus transfer.

13.6 IOT Platform for Embedded Systems

We have studied several aspects of designing an embedded system. We studied, in depth, different processor architectures, interfacing methods to different peripheral devices, way to design peer-to-peer real-time networks with different protocols and real-time software to manage the complete system.

Till now we had classical look of embedded systems performing dedicated application. We assumed, they work stand-alone or utmost communicate with limited devices locally. The technology has advanced in other computing domains, (viz) networking any device through mobile technologies (mobile networks), communication through high-speed networks over Internet protocols (Internet technologies), computing over virtual machines (cloud-based computing) with software as services (SOS) and infrastructure as services (IOS) and certain derivatives of cloud services (Edge computing), and so on.

Obviously embedded systems have exploited all these advances by integrating all these advances and came out with new paradigm of Internet of Things (IoT). Each “thing” here is an embedded system which gets connected globally with other IoT devices and computing platforms and expands its functionality from local computing to global computing. IoT is the most trending technology by which a system is controlled locally and also globally through Internet. This section gives a brief introduction to the on-going research and advances in this direction. The focus is on how IOT platform is used in embedded system design.

The things in IoT are any system of any type starting from house old, (refrigerators, ovens, air conditioners...), mobile devices (tablets, mobile phones...), computers (servers, laptops...), Industrial equipment (process controllers, sensors, actuators, building automation systems...) and navigation systems (cars, trucks, trains...), and so on. In fact there is no limit. Any system which can communicate over Internet is an IoT. We can define the IoT as a network of smart objects that interact with other smart objects which are heterogeneous and are addressable uniquely based on standard communication protocols.

Simply communication is not enough. We need a common platform where the data is stored and the things are controlled and accessed transparently. As we have

cloud platform that contains all the data from IoTs, we can connect to our refrigerator at home, view and set the temperature as desired while moving in a car. Here the embedded devices, their connectivity through Internet cloud play different roles.

Basic features of the IoT architecture are autonomous functionality of the devices (things) which have the same characteristics of an embedded system which we described earlier. The things function even without placing in the IoT environment. Then comes the connectivity by which the things communicate and share appropriate data with “like” things after applying privacy and security norms. Analyzing and decision-making happens at “thing” level or at centralized system like “cloud services”. The last one is the “end point management” meaning that the things are not fully autonomous but managed by the end users.

IoT exhibits major advantages over independent systems. Existing resources are not utilized fully by independent users all the time. They can now be fully utilized. The resources can be globally distributed. One can access them with appropriate permissions. Users can interact with things more efficiently. One need not go near the system and operate. Remote communications on IoT do this job. Human efforts are thus minimized. Time is also saved in this process. For example, one can plan cooking before driving home. One can exploit intelligent and ubiquitous communication across the things (of course with privacy and security always embedded) to execute major jobs in a coordinated way. Communication across self-navigating cars is a classic example for this benefit. Collection of data and dissemination to the relevant things is seamless. This helps in appropriate decision-making.

One major challenge in designing such system is the robustness in the security and privacy at each stage of the system (vz.) at device level, network level, and data management level.

13.6.1 Privacy in IoT

Both the terms *trust* and *privacy* are used in personal life very often. We do not share our personal information with others unless we trust them and the context demands to share it. Thus trust and context controls the privacy. Today our personal data becomes commodity for marketers. This is the reason privacy laws protect us from limiting its access. Moreover the trust and privacy changes with time and also with the context. As IoTs have to make decisions, the major challenge is *when and what data can be shared with which thing*. As more and more things are connected, the threat on the system increases and the overall risk increases. Privacy establishment is thus a challenging and complex task in IoT.

13.6.2 Security in IoT

Once the trust levels and privacy needed are established between two IoT devices, secured access safeguards the connected devices from sharing or denial of data. If the devices are not properly secured, the device opens up several vulnerabilities in the network. What happens if some “car” hacks another “car” and disables its breaking system? Implementing security in each device and in the network is essential for the whole IoT network. In earlier days, most of the devices are not designed keeping security in mind. Now if a device gets into as an IoT, strict security has to be embedded in its design. Another challenge is that the devices may not have sufficient computing capability to implement complex security algorithms. The solution lies in building security at hardware, firmware, software, and integration levels. Security in embedded systems is getting the ultimate focus now. Refer Atlam (2020).

13.6.3 IoT Architecture

Generic architecture of an IoT device is shown in Fig. 13.17. It is an embedded system with local connectivity with local devices which were discussed in the networking of embedded systems. The device now has IPV6 and connectivity to Internet.

The layers of processing are shown in Fig. 13.18. There is no unique architecture for IoT framework. It depends mostly upon the application domain. The generic architecture can be defined as a layered model.

IoT framework can be represented as four-stage framework. The basic stage is the embedded system connected to the sensors and actuators either by wireless or by wired connections. Certain devices are interconnected by real-time networks or

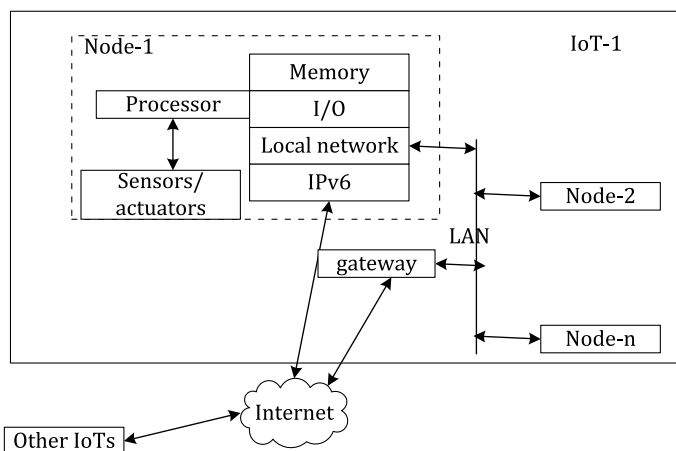
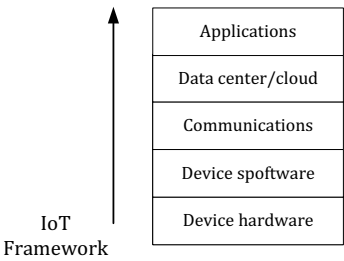


Fig. 13.17 IoT general architecture

Fig. 13.18 IoT framework



over LAN for local communication and coordination. In the next level (stage 2) such cells get connected by Internet for large data communication. Stage 1 devices get connected to Internet through gateways. They can use GSM, 5G, etc. Stage 3 is the EDGE IT. Edge system pre-processes the data before transferring to the cloud. The pre-process helps in data reduction by removing redundant and static data which was already passed to the cloud. Stage 4 is the cloud services where bulk data is processed by proper analytics based on the application domain.

13.6.3.1 IoT Development Hardware

Commercially microcontroller-based hardware is available with desired functionality to develop an IoT device. Arduino and Raspberry Pi are more popular whereas several other products are available. Raspberry includes built-in Wi-Fi and Bluetooth. Using GPIO and USB, one can connect to several peripherals. Development IDE includes Python, Java, and others. Arduino boards are equipped with digital input and output pins. Basic diagram of Raspberry Pi is shown in Fig. 13.19.

13.6.4 Communication in IoT

IETF (Internet Engineering Task Force) standardized RPL, the IPv6 *Routing Protocol for Low-Power and Lossy Networks*. The stack is shown in Fig. 13.20.

13.6.4.1 Datalink Layer

This layer provides services to the network layer. The device gets connected to the upper layers over Bluetooth layer which we discussed already. Also ZigBee protocols which are based on IEEE 802.15.4 are used at data link level. ZigBee Coordinator, ZigBee End Device, ZigBee Router provides services to upper layers.

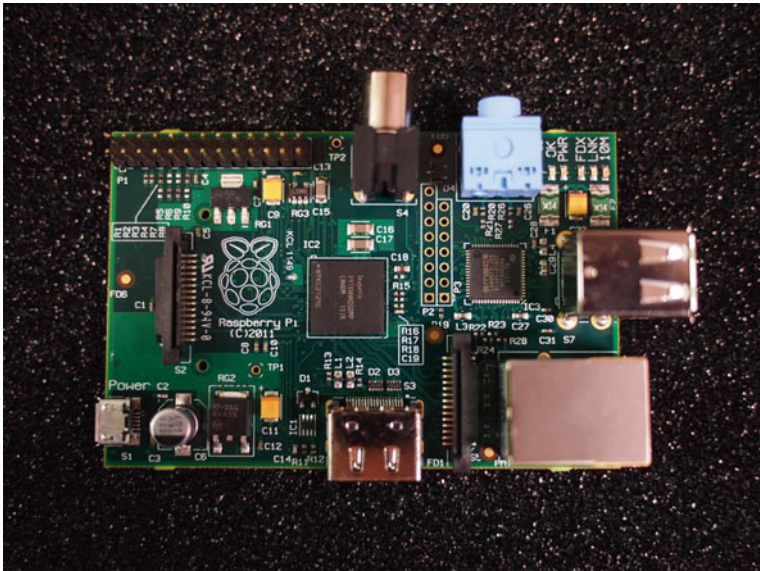


Fig. 13.19 Raspberry Pi diagram (Courtesy “Raspberry Pi: è davvero una rivoluzione?” by paz.ca is licensed under CC BY 2.0)

Fig. 13.20 IETF stack for IoT

IETF CoAP		Application
UDP		
IETF RPL		Network
IPv6	ICMP	
IETF 6LoWPAN		
IETF 6TOP		Link
IEEE802.15.4 TSCH MAC		
IEEE802.15.4 PHY		Physical

13.6.4.2 Network Layer

Network layer provides data transfer services from source to destination over the network using packets. **RPL** stands for Low-Power and Lossy Networks Routing Protocol on IPv6 even before emerging of IoTs. Now it is adapted to IoTs. RPL creates a routing topology in the form of a *Destination-Oriented Directed Acyclic Graph* (DODAG) (see Fig. 13.21a–d for illustrating this protocol). Figure 13.21a is a sample wireless network with 6 nodes having possible communication paths shown

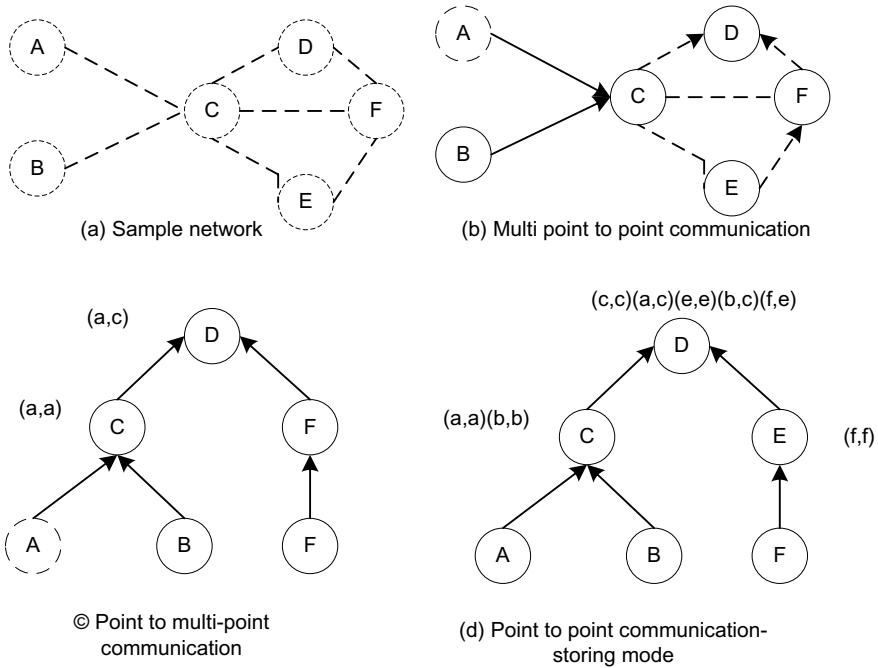


Fig. 13.21 Routing in RPL

in dotted lines. The routing is directed to a root node. In this case it is toward D. Each node maintains multiple parents for travel to root. Node E maintains paths through C and F to reach root D. Only one path is preferred to send data to root. Refer Iova (2016) and Salman (2016).

This is multi point-to-point communication by which any node can reach root in optimal path. Each node maintains the graph Information called as *Information Objects* (DIOs) and broadcast whenever topology changes. RPL also should support communication from the root as source and all other nodes as destinations. Root must have this information which it should get from its children. In figure (c) root D knows that it can reach A through its child C and node C knows it can reach A using the data through the destination advertisement objects (DAOs) (a, c) and (a, a). Each node stores this DAO information for flow in the other direction, similar to routing table. If this information is stored only at the root level, it is called non-storing mode. This information is sufficient for any node-to-node communication. For point-to-point communication, the data travels up to a node where it has possible path to its destination. As an example, if node A has to transmit to node B data travels from A to its parent C and finds (b, b) so data travels from C to B. This has become possible by storing mode where each node has routing information. In non-storing mode, if the same communication has to occur, data travels from A to C then to the root D, there it finds a path to B and data moves to C and then to B. It will not be efficient.

Several other network layer protocols exist whose description is given briefly below.

CARP (Channel-Aware Routing Protocol) (Aijaz 2015) is a distributed routing protocol. It has lightweight packets so that it can be used for Internet of Things (IoT). The network collects traffic data and its quality and decides forwarding nodes. Nodes do not support collecting previous data when data forwarding occurs. Not much useful in applications where the data is changing frequently.

6LoWPAN protocol refers to IPv6 Low-Power Personal Area Network which uses a lightweight IP-based communication to travel over low data rate networks. In IoT applications IPv6 addresses are too long and cannot fit in most IoT datalink frames which are relatively much smaller. Hence, IETF is developing a set of standards to encapsulate IPv6 datagrams in different datalink layer frames for use in IoT applications. 6LoWPAN protocol belongs to this class. This protocol efficiently encapsulates IPv6 long headers in IEEE802.15.4 small packets, which cannot exceed 128 bytes. For more details study Aijaz (2015)

13.6.4.3 Session Layer

IoT is a machine-to-machine connectivity and messaging. Some desirable features are simpler communication layer, low bandwidth, and low latency. One popular and mostly used session layer is MQTT from IBM.

MQTT (Message Queue Telemetry Transport) is introduced by IBM for monitoring sensor data from remote nodes and processing on remote computers. MQTT thus provides embedded connectivity across applications on one end and communication interface with the remote devices on the other end (see Fig. 13.22). Refer mqtt.org (2020).

MQTT uses publish-subscribe design pattern. The architecture consists of publishers, brokers, and subscribers. Publishers are any data-generating devices which will be consumed by the subscribers. Brokers are the middle men connecting publishers and subscribers. The IoT devices generate data and publish on the broker. The device will be idle when it has no data generated and need to be published.

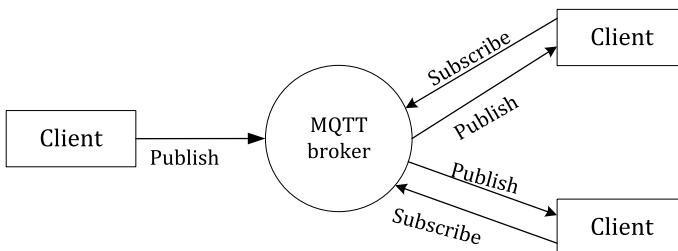


Fig. 13.22 MQTT broker architecture

Subscribers are applications which register with the broker and indicate the specific data they want to get and consume.

The broker gets the data from publishers and sends them to subscribers.

13.7 Summary

In this chapter we have studied some advances in embedded computing platforms. CPU manufacturers are releasing processors of multiple versions with increasing complexities from simpler to advanced applications. The vendors are also providing associated peripherals. The challenge lies with designers how to select and configure appropriate platform for a given application. Current trend is to develop dedicated applications with microcontrollers matching with the capabilities needed. Compact and energy sensitive applications are being developed using FPGAs with embedded processor IPs. Challenge lies in selecting platforms for distributed embedded systems. It depends on the data throughputs, real-time nature of networks. WSN is the platform when the nodes are volatile and mobile needing reconfigurable network topology. IoT platform is opted where each gadget is becoming smart and enhances its functionality by communicating over IPv6 with other smart devices globally. However privacy and security are challenging topics of advanced research for robust systems in every domain.

13.8 Exercises

1. An industrial system senses multiple parameters. It is uni-processor system. It has no cable connectivity to control room or local interface. Design a complete system to monitor and control the system using a smartphone with Bluetooth interface.
2. What is the major difference between computer architecture and computer platform? Distinguish between the two.
3. Give one example of synchronous, asynchronous, and burst mode transfer in popular computer platforms. Explain why that specific mode of transfer is chosen?
4. What are the specific reasons for separating APB (peripheral bus) and ASB (system bus) in ARM processor platform?
5. Assume there is no “split bus” in ARM architecture. Explain the consequences by an example.
6. Why is RPL protocol most suitable for IoT implementation?

13.9 Further Study

It is impossible to become an expert in all commercially available platforms. One has to understand basic architectures as described in this chapter and get into details of appropriate platforms of interest. Currently, multiple vendors provide comprehensive solutions for embedded systems design. Developer guides from them will be very helpful to fully understand.

References

- Aijaz A (2015) CORPL: a routing protocol for cognitive radio enabled AMI networks. *IEEE Trans Smart Grid* 6(1)
- AMBA™ Specification (Rev 2.0) (2020)
- Arm® Generic Interrupt Controller Architecture Specification, GIC architecture version 3 and version 4 (2013)
- ARM® Generic Interrupt Controller Architecture version 2.0, Architecture Specification
- Atlam HF (2020) IoT security, privacy, safety and ethics. Springer Nature Switzerland AG
- Bluetooth architecture, AHIR labs (2017)
- Bluetooth official website, Bluetooth[dot]com
- Iova O et al (2016) RPL, the routing standard for the internet of things . . . or is it? *IEEE Communications Magazine*, Institute of Electrical and Electronics Engineers
- MQTT: The standard for IoT messaging. mqtt.org (2020)
- Salman T. Networking protocols and standards for internet of things (2016)
- Understanding the I²C Bus Texas Instruments (2015)
- USB in a Nutshell, beyond logic (2018)
- USB—Universal Serial Bus 3.0 and 2.0 Specifications-Intel corporation (2010)

Chapter 14

Security in Embedded Systems



Abstract In past, most of the embedded systems are designed for dedicated functionality. They are stand-alone. However, with the advent of technological advances, most of these systems are not stand-alone but they are distributed. This caused everyone to think how to secure the embedded systems from hacking, intrusion, illegal data access, sabotage, and so on. These issues are well studied in Internet security. But, these techniques do not apply to different network protocols for embedded systems. No focus was given to the security of embedded hardware, firmware, embedded operating systems, and embedded applications, and embedded data. This chapter briefly introduces the security principles, the security issues in embedded systems, and the methodology to solve them. Section 14.2 introduces basic terminology, possible cyber-attacks on embedded systems, and needed security policies. Section 14.3 gets into details of security vulnerabilities in embedded systems and how to prevent them. Section 14.4 details basic security algorithms. Section 14.5 gives an example on how to implement security protocols on existing real-time network standards. The example is on authentication protocol implemented on CAN standard. Section 14.6 explains possible guidelines to secure embedded systems. The chapter concludes with current security standards for embedded systems and typical secured platform architecture.

Keywords Threat actor · Vulnerability · Attack vector · Attack surface · Attacker · Computer hacker · Authentication · Authorization · Confidentiality · Provenance · Mutable · Privacy · Security · Trust · Code injection · Reverse engineering · Malware · Eavesdropping · Buffer overflow · Hash algorithms · Asymmetric algorithms · Chain of trust · Trusted zone · Digital rights management

14.1 Motivation

In past, most of the embedded systems are designed for dedicated functionality. They are stand-alone. Most of the focus used to be on compactness, performance, reliability, energy consumption, and so on which we have discussed as the metrics in the first chapter. However, with the advent of technological advances, most of these

systems are not stand-alone but they are distributed systems. They Internetwork, coordinate, and execute functions in a coordinated way.

This caused everyone to think how to secure the embedded systems from hacking, intrusion, illegal data access, sabotage, and so on. These issues are well studied in Internet security. But, these techniques do not apply to different network protocols. No focus was given to the security of embedded hardware, firmware, embedded operating systems, embedded applications, and embedded data.

When an embedded system is powered off, there is no security issue. When it is powered on and not even interconnected, security issues arise where the programs can be hacked and data can be accessed by taking the control of processor execution by different techniques. Security is very important in embedded systems because of their roles in many mission and safety-critical systems. Attacks on cyber systems are proved to cause loss of data, physical damage to systems. However, comparing to conventional IT systems, security in embedded systems is not implemented in most of the systems. Even if it is implemented, it is not robust. Because of this, damage is much more serious like loss of life due to sabotage in industrial plants, faults in railway signaling traffic coordination, and so on. Hence, the topic of developing secured hardware, secured firmware, secure operating system for embedded systems has become hot topic of the day.

With the advent of IoT, very useful applications have come to reality. At the same time, the IoT has become more vulnerable to attacks. The problem has become more complex with the security aspects from the embedded system and also through the network. The number of possible attacks is growing exponentially, mostly because of interconnectivity. Today any smart device is most vulnerable to attacks. A hacker can take control of the system, if the system is not designed protecting from security threats. With increased functionalities in smart embedded systems, the complexity of the design increases, and the vulnerability to attacks increases. This chapter briefly introduces the security principles, the security issues in embedded systems, and the methodology to solve them.

14.2 Introduction

Let us understand the terms most often used in cyber security as we do use them too often in this chapter also.

14.2.1 Terminology

A **threat actor** is an individual or a team successfully conducting malicious activities against systems.

Vulnerability is a flaw in the system that can be exploited by a threat actor to perform an un-authorized action within an embedded system.

Attack vector is the technique by means of which the hacker gets un-authorized access to the system and compromises the system. Internet, flash drives, network protocols, etc. are some attack vectors for an embedded system.

Attack surface is the sum of all vulnerabilities which can be considered for an attack. The attack surface can be a digital or physical.

An **attacker** is a person or system performing a malicious action on the system.

A **computer hacker** is a knowledgeable person in computer system and its internals. The person utilizes his knowledge to verify the integrity of a system, or overcome certain obstacles or attack a system for certain malicious benefit. The hacking operation can be ethical or un-ethical.

Authentication is a process by which the user or computer proves its identity to other systems.

Authorization is to allow the control of resources by ensuring that a device is authorized to use a service before permitting it to do so.

Confidentiality is to prevent information compromise caused by eavesdropping. This is done by ensuring that only authorized devices can access and view data.

Provenance is the place of origin or earliest known of something. In security terms it provides a historical record of the data and its origins.

Mutable is the property of a data by which its value can be changed like Flash memory. Immutable data cannot be changed like ROM.

Privacy is the right to have some control over how personal information is collected and used.

Security refers to how one's personal information is protected.

Trust literally means "belief that the other party is good and honest and will not harm you" In security terms, a trusted system is the one whom it can be relied upon to enforce a specified security policy.

14.2.2 Cyber-Attacks on Embedded Systems

An embedded system is vulnerable to physical and software security. **Physical security** involves keeping the whole system in-accessible to un-authorized persons. Certain components can be accessed by the attacker to get the data off-line like reading the code from removable devices like EPROMS, flash drives, SD Rams, etc. Intentional damage to the system, stealing the hard drives, and other sensitive components from the system comes under physical category which must be protected from attackers. Damages from natural calamities, accidents have to be taken care by keeping proper stand-by to the data and systems. I came across certain corporates securing their software in bank lockers to mitigate the risk of fire!!

Software security protects the system from malicious behavior of the system by proper protection mechanisms built into the system during the design stage itself. Most secured systems are the ones which are not turned ON! Even if the system is turned ON if it is totally isolated and not connected with other systems, it is mostly secure. As the systems get connected and the connectivity domain increases

to global connectivity, systems become more and more un-secured. Some common cyberattacks are listed below (Papp et al. 2015).

Code injection: This type of attacks makes the normal control flow on the embedded device to be diverted to attacker's code and take control of the system.

Reverse engineering: Attacker gets sensitive information by monitoring the code execution and identifies vulnerabilities. Different debugging tools like logic analyzer, protocol analyzers, and code tracers can capture executing code at the assembly level and de-assemble by which reverse engineering can be done.

Malware: An attacker can infect an embedded device with malicious software (malware). The malicious code adds potentially harmful functionality to the infected system. Or it can modify the behavior of the device, which may have serious consequences.

Injecting crafted packets: Most embedded systems communicate over proprietary or standard protocols which we discussed in "networking of embedded systems" chapter. The attacker can modify the message frames over the bus or at the time of generation of the frame. This is malicious packet crafting and injecting into the system. By this mechanism, one device can send false message to other devices. Injected messages may be valid as per protocol but dangerous to the process. As an example, "close the water flow valve" in a cooling system may stop cooling and burn the complete system.

Eavesdropping: While packet crafting is an active attack, eavesdropping or sniffing is a passive attack whereby an attacker only reads the messages and extracts sensitive information. This may be used in packet crafting.

Exhaustive search: weak encryption and authentication can be broken by brute force exhaustive search. This is possible when the search space is small.

14.2.3 Security Policies

Embedded system security is a strategic approach to protect software running on embedded systems from attack and provide mechanisms to protect the system from all types of malicious behavior. While considering security, the focus used to lie on protecting data. But additional aspects must be considered in the security policy.

- All embedded systems are built on firmware. The firmware is not updated during the life cycle of the system. Gets vulnerable to security attacks. Firmware has to be updated periodically. Developers should consider obsolescence of hardware and

software components and do updates for better longevity and security. Basic issue is, software updates, security patches go very frequently, but hardware upgrades rarely happen.

- Certain systems are very difficult to get upgraded, even software. At the design time itself, the upgradability metric must be given high weightage in design. Embedded system architectures need to be flexible enough to support the rapid evolution of security mechanisms and standards.
- No un-trusted programs should be loadable into the system and take over execution.
- Programs should not share data with other systems unless both systems trust each other.
- The data should not be accessible to other systems or hackers. Devices should be uniquely identifiable.
- Devices should authenticate themselves before transmitting or receiving data
- Devices should check integrity of the boot code before the boot process.
- Embedded systems are manufactured in millions. If a security hole is exploited by a hacker, all the million devices are vulnerable. The fault has to be detected, rectified, and provided security update to all systems online. Such mechanism is available in mobile phones and computers today. It should get into every networked embedded system.
- Certain systems communicate using proprietary protocols. Systems are vulnerable because these protocols cannot be verified and validated.
- Security aspects should be considered during system modeling and design phase itself and the design should be validated and verified.
- Once a system is designed, finding the vulnerabilities in the system is a long drawn process. They get identified after long use of the system. During this time, attackers should not exploit vulnerabilities. Hence security protections like firewalls, intrusion detection systems are to be placed as additional layers.
- Devices should support security life cycle. It depends on software versions, hardware configuration, and on the product lifecycle. Product lifecycle phase includes development, deployment, returns, and end-of-life. Each security state defines the security properties of the device. The security state must be attestable.
- Devices should support security updates.
- Consider designing security at the processor design level. Currently modern SoCs include security at this stage.
- Use hardware-oriented techniques for detecting attacks. This improves detection at run time in real time.
- Dedicated processor can be added in the SoC with exclusive functionality to monitor security (Patel 2011). Offload security monitoring and control to security engines.
- Security is misunderstood as cryptography and network protocols. Security is now to be considered at all levels.

14.3 Security Vulnerabilities in ES

The attacker gets access to the system physically or through the network, understands the hardware, operating system, and the processes behind. They identify the vulnerabilities in the hardware and software components and the processes. Once identified, they exploit the vulnerabilities. The major attack vector is the Internet communications with other devices. The second one is operating system and boot process and finally the physical devices. Security in embedded systems starts at User identification, secure network access, secure communication, secure storage, and secure execution. At network level this is taken care by cryptography algorithms. Hardware architecture should support monitor data transactions over the bus and prevent illegal access to protected areas of memory or authenticate the firmware that executes on the system. Figure 14.1 classifies all the attacks (Qnx 2020).

Invasive attacks get into system internals, corrupt existing system, or take over system execution. This is done by probing the communication, monitoring bus transactions, etc. Noninvasive attacks do not get into internals but attack using side channels like power, clock, timing analysis, frequency, etc. By probing the execution times, power consumption patterns system behavior is predicted. Logical attacks which are described in detail below lies in sending false messages and getting responses, running malicious software, and exploiting weaknesses in the system implementation.

14.3.1 Buffer Overflow

This occurs when writing some data into a buffer or push data onto a stack, (*heap or stack overflow*). Theoretically the data should not overflow out of the allocated buffer or stack limit. If there is no check or there is no mechanism to generate an exception when such overflow occurs, the data gets overwritten outside the boundary. If the overwritten area has a valid code, the system hangs or malfunctions. Or if the overwritten data is malicious code, the hacker wins and the system gets into the hacking program. The overwritten program can cause erratic behavior without

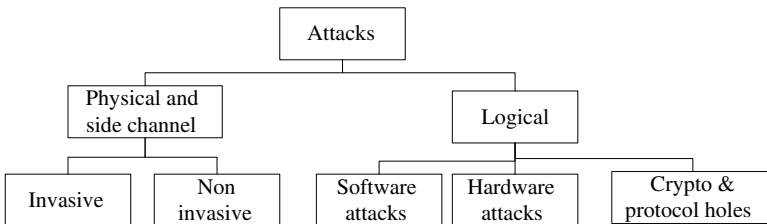


Fig. 14.1 Classification of attacks

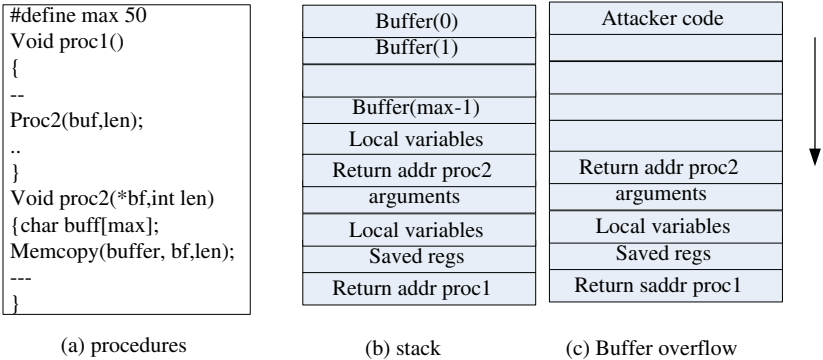


Fig. 14.2 Buffer overflow

the user’s notice through un-authorized memory access, alter execution paths, un-authorized control of peripherals, and crashes. If the attacker knows the memory map of the program, new code can be injected to gain un-authorized access. This is called *buffer overflow attack*. This is well known and common security hole.

In Fig. 14.2a function proc1 makes a call to function proc2. Return address of proc2 is stored on the stack. In proc2, buffer of size *max* has to be instantiated. It is done as local variables in proc2 and placed on stack. The attacker will push data more than *max* and thus causing the return value to corrupted. Thus, the control flow of the program is changed to execute malicious code.

14.3.2 Improper Input Data Validation

A system needs input from other devices or through users. Action takes place based on the input data. If the input validation is not done, the system will not have predicted behavior and may get into undefined and undesired states causing even the system to crash. This has to be taken up at design stage to validate all possible data and events so that system remains in predicted states.

14.3.3 Improper Authentication

Authentication proves that other system or user or process going to interact is correctly identified. If this process is improper, fake user gets identified or the attacker gets a chance to bypass the process.

14.3.4 Out of Bounds Memory Access

If the operating system and the compiler do not restrict the code from accessing privileged areas of memory, the threat actor may be able to take control of the system. Out of bounds access to programs must be restricted.

14.3.5 DMA Attacks

If an attacker compromises the firmware of a DMA capable I/O device, the compromised device might be able to access system memory during the DMA process. This could allow the attacker to interfere with the system's Trusted Boot process or corrupt memory.

14.3.6 Platform Reset Attacks

If the attacker is able to reset the system and during the system reset, the attacker changes the boot device like USB or to other boot devices, the system gets booted to the attacker's version of OS. All the critical data is now vulnerable to be accessed by the attacker.

14.4 Basic Security Algorithms

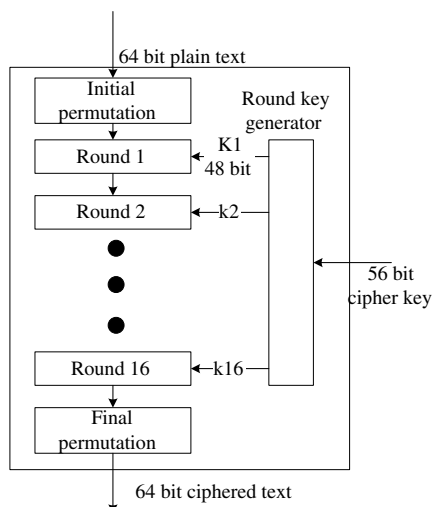
Implementing security is done through three classes of cryptographic algorithms. They are symmetric ciphers, asymmetric ciphers, and secure hash algorithms. The algorithms are discussed in brief below.

14.4.1 Symmetric Ciphers

Symmetric cipher requires the sender to use a secret key to encrypt the data and transmit to the receiver. On receiving the encrypted data, the receiver uses the same secret key and decrypts the original data. The quality of encryption is judged by the toughness to decrypt without the secret key. Thus, the data is transmitted confidentially.

Symmetric algorithms are of two types—stream and block ciphers. Stream ciphers encrypt plaintext bit by bit at a time. Block ciphers take a block of bits (normally 64 bits) and encrypt the block as a single unit. Several symmetric algorithms exist.

Fig. 14.3 DES implementation



One popular one is Digital Encryption Standard (DES) which is a symmetric block cipher with 64-bit block size that uses a 56-bit key (see Fig. 14.3).

DES is an implementation of a Feistel Cipher. It uses 16 round Feistel structure. The block size is 64-bit. Though, key length is 64-bit, DES has an effective key length of 56 bits, since 8 of the 64 bits of the key are not used by the encryption algorithm.

14.4.2 Secure Hash Algorithms

Hash algorithms convert messages into unique fixed-length values, thereby providing unique “fingerprints” for messages. Hash algorithms work by transforming the data using a hash function. The algorithm comprises bitwise operations, modular additions, and compression functions. The hash function generates a fixed-size string totally different from original data. These are one-way functions. It means, once they are hash coded, one cannot get back to original data. They are basically used to store passwords and critical data in hash coded form. When user enters the password again, it gets hash coded. System compares entered hash coded value with original for authentication. Hackers get the hash coded value from the system but it will not be useful because they cannot get back actual data.

Few algorithms SHA-1, SHA-2, and SHA-3 have been developed and standardized. SHA-2 is considered cryptographically strong enough to be used in modern commercial applications and standardized by NIST (see Fig. 14.4).

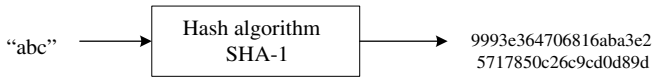


Fig. 14.4 Hash algorithms

14.4.3 Asymmetric Algorithms

Asymmetric algorithms use a pair of keys. One of the keys locks the data while the other unlocks it. Encryption of a message is done by generating a public key and using it. The same key is available to the public which is required during the decryption process. Data can be decrypted by a private key which is secretly transmitted to the recipient and kept as secret. RSA algorithm is asymmetric algorithm.

The private key provides host authentication as this is generated for the specific data transmitted and delivered by the transmitter. Digital signatures are generated by using public key cryptography and hash algorithms. User digitally signs a message by encrypting a hash of it with his private key.

RSA is based on the principle that it is difficult to factorize the product of two large prime integers. The public key consists of two numbers. The first number is generated by multiplication of two large prime numbers p and q (see Step 2). The second number, " e " is relatively prime to $(p - 1)$ and $(q - 1)$ (see Step 4). Private Key is derived from the same two prime numbers. (step 7). If someone can factorize the product of large number, the private key is compromised. RSA encryption strength lies on the key size. RSA keys are typically 1024 or 2048 bits long. The algorithm in brief is as below.

Generate public key:

1. Select two large prime numbers p, q .
2. First part of public key, $n = p * q$.
3. Let $\phi(n) = (p - 1)(q - 1)$.
4. Select an integer e where $1 < e < \phi(n)$ and relatively prime to $\phi(n)$.
5. Public key = (n, e) .
6. //Let D be the data to be encrypted.

Generate private key:

7. Private key = $d = \left(\frac{k * \phi(n) + 1}{e} \right)$ where k is an arbitrary integer.
- Encryption and decryption is done as
8. Encrypted data $c = D^e \text{Mod} n$.
9. Decrypted data = $c^d \text{Mod} n$.

As an example let the data be as $D = 72, p = 11, q = 17$.

- Data = 72.
- $n = 187$.
- $\phi(n) = 160$.
- Let $e = 3; k = 2$.
- Public key = $(187, 3)$.

- Encrypted data = 183.
- Private key = 107.
- Decrypted data = 72.

14.5 Security Protocols for Embedded Systems

Earlier, the focus was to consider safety and real-timeness as most important considerations in designing communication protocols for embedded systems. With increased connectivity of systems in a distributed way, and the attack surface has increased. The protocols need to be designed with security in view. Introducing security protocols in safety-critical systems has to be carefully planned. The system has safety and real-time constraints already. Security implementation may cause overheads in execution time. System should first meet safety and real time and then security (Bruni 2016).

Systematically reasoning about the correctness of security protocols is therefore important in design of secure systems. Formal methods provide theoretical frameworks and analysis techniques that can be used to reason about security properties in communication protocols.

As an example CAN bus protocol does not have any security aspects embedded in it.

Existing protocols have to be extended for providing security aspects like authentication. Let us assume the receiver CAN node has to receive only authenticated messages. The implementation should be done in such a way that an attacker sends a false message to the receiver; the key exchanges for authentication should not be revealed to the hacker. Thus receiver rejects. All this is done through embedding security keys within the message payload.

Such extensions impose several constraints on the compatibility with existing protocols. Since the protocol is running on microcontrollers with limited processing power, the cost of computing the cryptographic primitives must be limited in order to respect the deadlines imposed on the system. Other constraint is maximum frame size offered by CAN since authentication must limit within the frame size. CANAuth is an authentication protocol for CAN bus message authentication (see Fig. 14.5).

The protocol consists of two phases. The first one is key establishment phase. A designated master initiates authenticated communication. It establishes a session key (ks) that will be used to authenticate all messages. The message sent through the bus is signed with the session key.

All nodes connected to the CAN network have at least one pre-shared key kp installed.

1. (Fig. 14.5a) To establish a session key (ks) the designated master node (i) broadcasts a 24-bit count (cnt) and an 88 random number (rnd). The count must be greater than every value already used during key establishment in order to ensure new value. At this stage every node in possession of the pre-shared key

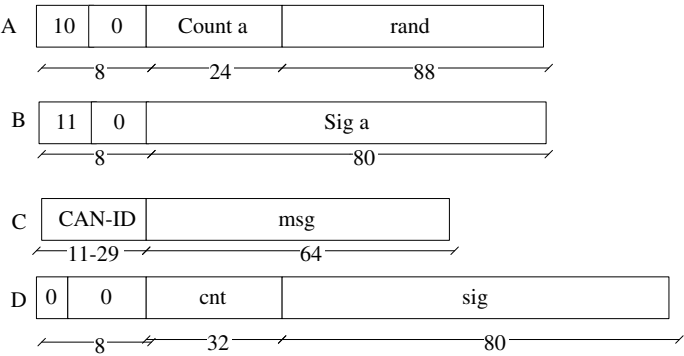


Fig. 14.5 Message authentication frame for CANAuth

can compute the session key (*ks*) and the signature (*sig*) using the received information as shown below.

$$ks = hash(kp; cnt; rnd) \bmod(2^{128})$$
$$sig = hash(ks; cnt; rnd) \bmod(2^{112})$$

2. (Fig. 14.5b) To confirm that the transmission succeeded, the master ECU again sends the signature so that the other nodes in the network can compare it with their own computed value and verify.
3. (Fig. 14.5c) Once a session key is established, messages are authenticated. The message format, shown in Fig. 14.5, shows the sizes of the bit fields, where the first row represents the CAN bus frame with 64-bit of payload and
4. (Fig. 14.5d) The second row represents the extension payload. To authenticate a message, the node sends a counter *cnt* and the signature *sig*. To ensure freshness *cnt* has to be greater than any other previously used value.

$$sig = hash(ks; cnt, M) \bmod(2^{80})$$

You observe in the CAN Auth protocol, the authentication is added without disturbing the standard CAN protocol. Please observe the additional overheads in the number of frames and data to provide authentication.

14.6 Guidelines for Secure Systems

Security must be considered from the inception of the design phase. It remains as a part of complete system development life cycle. This includes hardware, software, integration, and testing phases (see Fig. 14.6). The designs must ensure system-level threat modeling and analysis and determine appropriate use of security features. Figure 14.7 is an example of platform and software components that are typically found (Arm™ Server Base Security Guide). The firmware in the platform extends

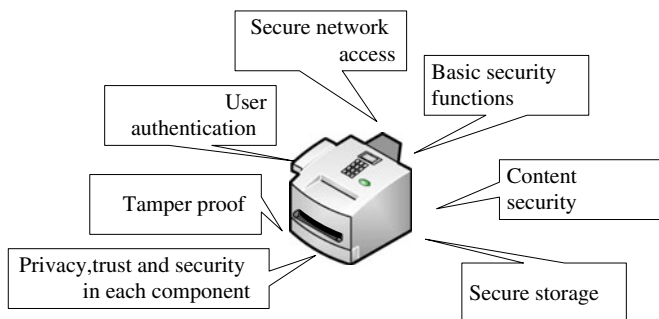
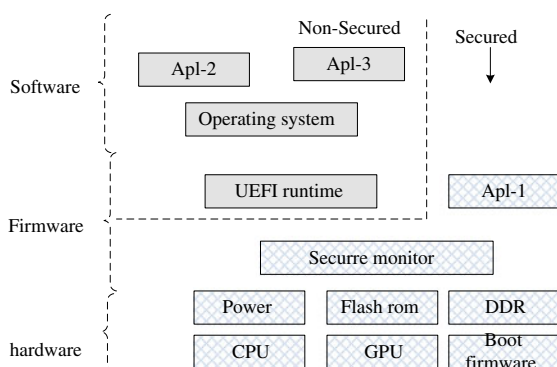


Fig. 14.6 A secured embedded system requirements from user perspective

Fig. 14.7 Secure and non-secure environment in a system (Courtesy Arm™)



beyond the host SoC firmware. Other hardware components may have their own mutable firmware components. Any compromise in the integrity of these platform components will make the complete system vulnerable to security attacks.

14.6.1 Trust Across Devices

Exchange of data through communication can go across mutually trusted devices. Trust building can go to the lowest level to components on a SoC. Trusted Services are collections of operations and assets that require protection from the system, and across peer services, to ensure confidentiality, authenticity, and integrity. During manufacturing, private key can be generated on the chip which serves as the root of trust. When this private key is certified by the Public key infra structure (PKI) that device can become root component for trusted device connectivity. Effectively, the secure software implementation through **PKI is mapped to hardware level**.

14.6.2 Secure Firmware Updates and Critical Data

A secure firmware update process must ensure that only authorized changes are permitted to the firmware in a system. Critical data includes configuration variables and policies which have to be validated and remain in valid state for it to be accessed during system boot and at any time. An attacker tries to execute by altering firmware and gains control of the system for executing an application, collecting some critical data. This can only be mitigated by verifying the firmware image or image metadata.

14.6.3 Secure Boot

All embedded systems get booted in a specified sequence. If any malicious code gets as boot code, the complete system gets into attacker's control. The boot sequence has to be authenticated and *integrity check* has to be made. The boot image is the code which gets booted. It has to be thoroughly authenticated and ensured that it is not tampered. The boot image is strongly encrypted and also ensured the image is not used elsewhere (*anti-cloning*). When the embedded system boots, the boot image is validated using this public key and the corresponding trust chain to ensure that boot-time software has not been tampered with.

Even after identifying the security issues, fixing and the firmware updated, attackers need earlier versions to exploit security holes. An attacker downgrades to a flawed version of firmware or software in order to exploit a vulnerability to gain partial or total control of the system. The *secure boot* layer ensures that previous boot images having vulnerabilities are not loaded by the attacker to exploit the security holes (*Anti-roll back*). The Secure Boot solution supports use of certificates.

When the mutable firmware and critical data have to be updated, the process must be authorized and verified. The mutable firmware and critical data must be digitally signed so that it is verified during the boot process. This forms *chain of trust*. First instruction integrity must be adapted. The first mutable firmware that is executed on the host SoC or other system component must be authenticated before use by an immutable bootloader.

Trusted Boot begins in an immutable bootloader component such as a boot ROM which loads the first mutable firmware image. The boot process continues with each component in the boot chain performing integrity and verification of the next component before it is executed or used. This forms a *chain of trust* in the immutable bootloader and continuing through all code that is executed up to the runtime environment (see Fig. 14.8).

In any system, booting with chain of trust starts from the embedded immutable boot ROM. Once a system is reset, the processor boots into secure state and executes the ROM code. During execution, it detects whether there is next stage of boot process exists. Accordingly, the next stage gets booted after proper authentication and verification. The trusted boot chain continues.

Fig. 14.8 Chain of trust approach

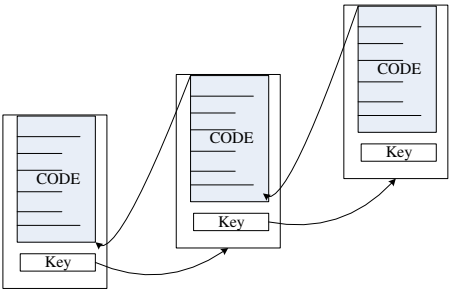
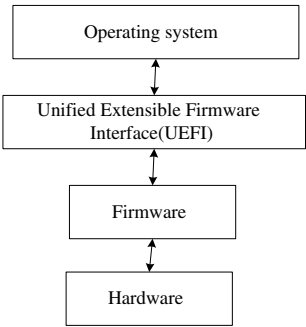


Fig. 14.9 UEFI interface



14.6.4 UEFI Security Guidance

UEFI stands for “*Unified Extensible Firmware Interface*” (www.uefi.org) UEFI (2019). This specification defines a secured model for the interface between standard operating systems and the firmware of the platform. It replaces legacy BIOS interface to the firmware. The interface consists of critical data and platform-related information. It contains secured boot and runtime service calls that are available to the operating system and its loader. These provide a secured and standard environment for pre-boot operations and booting the operating system. Thus UEFI brings security in devices, firmware, and operating systems. The specifications are used by BIOS vendors, add-on card developers, OS developers for unified way to develop secure systems. UEFI defines boot services and run time services. UEFI holds certain variables as non-volatile data which is shared between the firmware, OS, and UEFI applications (see Fig. 14.9).

14.6.5 Trusted Zone

Main processors are getting developed with specific portion of hardware and components as trusted zone. This zone is completely isolated and only trusted and secured

critical applications can be executed. Certain utilities in the OS mark the code in the files as trusted and ensures *trusted execution*. In today’s scenario every embedded system is built around an operating system. Moreover the OS functionality is causing more code size. This causes several security issues.

Hence partition the complete hardware and software into trusted and non-trusted zones (see Fig. 14.10). The trusted zone is highly protected. The assets in the trusted zone can be accessed by the trusted software only. However trusted zone software can access non-trusted zone assets but not the other way. In addition to separation into trusted and non-trusted zone, privilege levels like user mode/supervisor mode protect the execution. Privilege levels can vary from processor to processor. Higher levels are more secure. When lower level code tries to access higher level, exception is raised and processed.

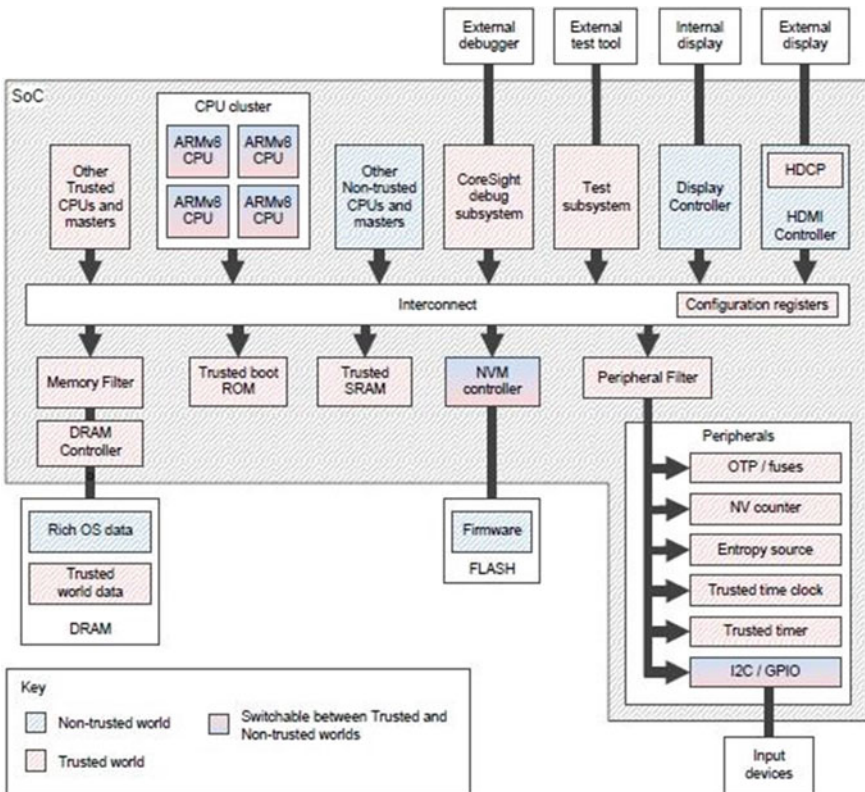


Fig. 14.10 Trust-based security architecture of ARM (Courtesy “Arm copyright material kindly reproduced with permission of Arm Limited”) (Arm 2018; Arm security guide 2019)

14.6.6 Secure OS

While commercial embedded operating systems have certain security provisions implemented, developers can check the features and configure for certain security mechanisms. Certain memory area can be protected as non-executable. Any attempt to execute any machine code in this area raises an exception. Every process should hop the stack, heap to different locations randomly. This leads to ***address space layout randomization***. Attacker finds it difficult to predict the locations for buffer flow and heap flow attackers.

Stack canaries is a mechanism to prevent stack overflow attacks. A small value is placed on stack whenever a function gets started. Before the function pulls the return value, it checks the value placed to ensure that the area is not overwritten by the attacker. The value is changed randomly for every such function call. Attacker tries to access kernel memory using a device like a DMA. OS prevents by ***DMA containment***, the system memory management layer tries to contain un-authorized access.

14.6.7 Secure Storage and Memory

Non-volatile memory which is mutable like flash memory is vulnerable to attacks and un-authorized change. The storage contains firmware images, critical data, and UEFI configuration settings.

The volatile memory which belongs to the security region is the memory to be protected. It is not accessible to non-secured region. The memory may be located in on-chip DDR or off-chip. It is needed for secured application execution.

14.6.8 System Recovery

If the attacker is to attack the system, there should be in-built mechanism to recover the system to a state of integrity.

14.6.9 Security Life Cycle

Every system should monitor the system's state. Safe states are like boot process, debug, secured application execution and non-secured application execution, etc. This helps to constantly monitor and take recovery action from attacks.

14.6.9.1 Hardware Identity

It is possible for an attacker to remove some off-chip hardware like flash memory. This may get replaced by the attacker. The hardware device has to be identified by Hardware Unique key (HUK). The HUK is stored on-chip, immutable non-volatile memory. The attacker may read off-chip data to do *reverse engineering*. This can be protected by encryption of data.

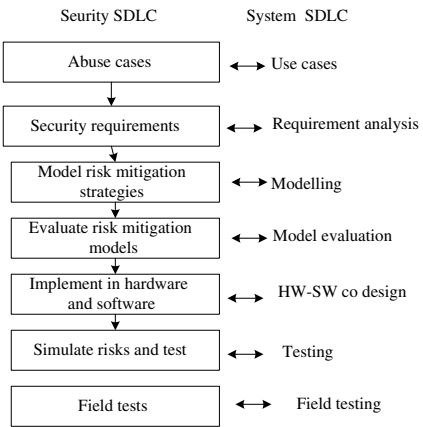
14.6.9.2 Secured Instruction Fetch

As in Fig. 14.10, the execution will be in secured code or un-secured code. If the application in secured region tries to branch to un-secured region, exception will rise. This setting is available in most modern processors.

14.6.9.3 Secured SDLC

As developers are becoming more concerned with security aspects, concurrent secured development life cycle must be executed in concurrence with SDLC (see Fig. 14.11). This includes *ab-use cases* which identifies possible abuses to the system and how to manage. Modeling takes care of the requirements to be taken care mitigating security attacks. The models have to be tested with all possible events. Development phase for hardware and software takes care of implementing all aspects discussed above. Testing phase should include secured test cases in concurrence with product test cases.

Fig. 14.11 Secured SDLC



14.7 Security Standards for Embedded Systems

SAE J3061_201601: Cybersecurity Guidebook for Cyber-Physical Vehicle Systems provides guidelines for security implementations in vehicles like cars, trucks, commercial and military vehicles, etc. It lists best practices for implementations in cyber-physical vehicle systems. This standard describes some techniques for Threat Analysis and Risk Assessment, Threat Modeling and Vulnerability Analysis (e.g., Attack Trees) and when to use them. It provides information on vulnerability classification and related information. Includes good design practices and describes test tools to test security aspects.

NIST-Cybersecurity Framework focusses on reliable functioning of critical infrastructure and contains cybersecurity threats which exploit the increased complexity and connectivity of critical infrastructure systems (NIST 2018).

The framework can be used by any organization. The guidelines are applicable to any type of system needing cybersecurity. These include information technology (IT), industrial systems (ICS), cyber-physical systems (CPS), or distributed systems, Internet of Things (IoT), and so on.

ISO/IEC 27001:2013, Information technology Security techniques: Information security management systems (ISMS) are the framework, through which any organization identifies, evaluates, and rectifies the security risks. For details study ISO/IEC 27001 (2013).

The ISMS ensures to keep pace with changes in the security threats, vulnerabilities, and business impacts. ISO/IEC 27001 2013 does not mandate specific security solution as the requirements vary markedly across the wide range of organizations adopting the standard. It lays out the design for ISMS, describing the important parts at a fairly high level.

14.7.1 Digital Rights Management

Digital Rights Management (DRM) (Murti and Tadimetti 2011) is a generic term for access control technologies that can be used by hardware manufacturers, publishers, copyright holders, and individuals to limit the usage of digital content and devices. The term is used to describe any technology that inhibits usage of digital content not desired or intended by the content provider. Digital Rights Management (DRM) technology attempts to control use of digital media by preventing access, copying, or conversion to other formats by end users. For details study Murti and Tadimetti (2011).

Figure 14.12 explains the role play of each actor and their interaction in the proposed model.

1. Owner of the data delegates the Licensing rights to the license manager by giving him a policy.

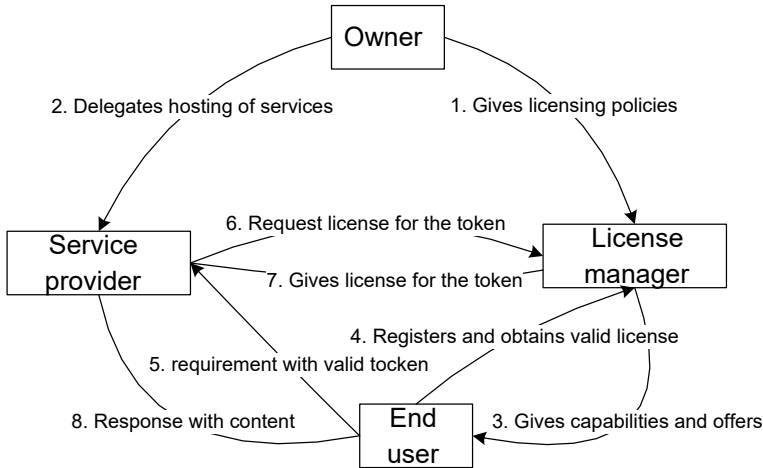


Fig. 14.12 Digital rights management model

2. Owner of the data delegates the service hosting rights to the service provider.
3. End user requests the license manager for the offer.
4. End user registers himself for some rights and operations with the license manager and obtains a token.
5. End user requests the service for an operation and sends the token along with his request.
6. The service provider requests its license manager with the token to give it a license.
7. The license manager gives a valid license if available and sends it to the service provider.
8. The service provider authorizes the request and enforces the license on the request (a). If the request is valid then the response is sent (b). Else an exception is thrown or a null response is sent depending on the configuration the response is sent back to the client.

Depending on the specific business model, roles may be combined in different ways.

14.7.2 Fast Identity Online Alliance (FIDO)

The Fast Identity Online (FIDO) (www.Fidoalliance.org) Alliance is an industry consortium that was formed to accelerate the adoption of strong online authentication via standardization. Implementing authentication beyond a password and /or an OTP, systems are using proprietary protocols beyond this conventional authentication.

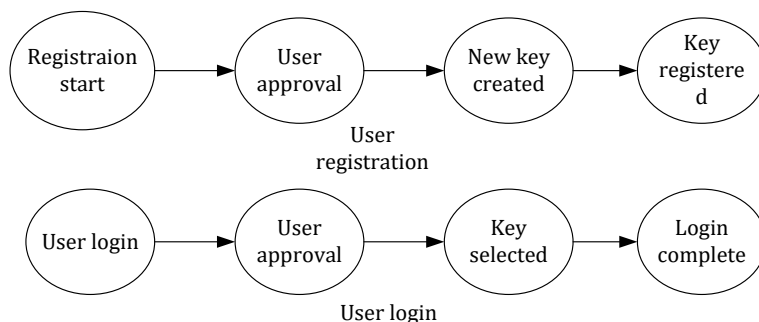


Fig. 14.13 Fast identity online alliance

FIDO provides standardized client and protocol layers. This enables second-level authentication using biometrics (see Fig. 14.13), see FIDO (2020).

FIDO uses standard public key cryptography (PKI) techniques for providing stronger authentication. A client initially registers to a service using the client device. During this process, the client creates public and private key pair. The client retains the private key and registers the public key with the online service. During registration,

- User is prompted to choose an available FIDO authenticator that matches the online service's acceptance policy.
- User unlocks the FIDO authenticator using a fingerprint reader, a button on a second-factor device, securely-entered PIN, or other methods.
- User's device creates a new public/private key pair unique for the local device, online service, and user's account.
- Public key is sent to the online service and associated with the user's account.

While logging in

- Online service challenges the user to login with a previously registered device that matches the service's acceptance policy.
- User unlocks the FIDO authenticator using the same method as at Registration time.
- Device uses the user's account identifier provided by the service to select the correct key and sign the service's challenge.
- Client device sends the signed challenge back to the service, which verifies it with the stored public key and logs in the user.

14.8 Typical Secured Platform Architecture

We have studied different embedded platform architectures in Chap. 13. Keeping security aspects in mind the layered representation of any generic system is shown in

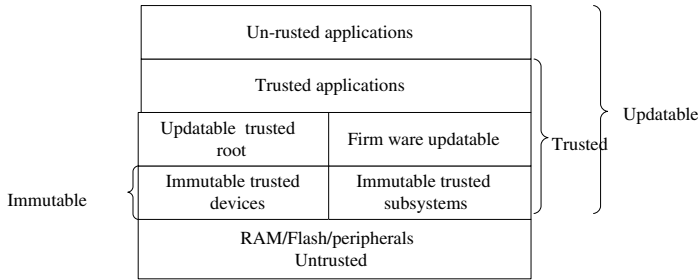


Fig. 14.14 Typical Secured platform architecture

Fig. 14.14. It consists of immutable trusted devices which never changes during the product life cycle. The updatable portion is trusted through verification and anchored to the immutable system. Trusted subsystems are protected off-chip memories, trusted peripherals, etc. Trusted applications use interfaces provided by trusted root. Untrusted components may include any off-chip device and code. Trusted root covers hardware and software to implement trusted services (Kocher 2004).

14.9 Summary

No security implementation is fully fool proof. But it has to be secured to an extent that any security attack on the system will be too costly for the attacker and time consuming. Introducing security protocols in embedded systems which were not initially designed with security in mind is a challenging issue. As explained in CANAuth protocol in this chapter as an example, any security has to be embedded in existing standards. This leads to loss of throughput, real-time performance, etc. New device development should include secured development life cycle in concurrence with SDLC.

14.10 Further Reading

As the topic is emerging, there is no comprehensive textbook for this topic. Majority of information can be obtained from the user guides of modern processors and how security is implemented at hardware, firmware, and OS level. Some important publications and standards are listed in the references for further study. Kocher (2004), Papp et al (2015), Patel (2011), SAE (2016) PhD thesis by Bruni (2016) are good coverage of security protocols.

14.11 Exercises

1. What is symmetric cypher? What are some examples of symmetric encryption?
2. What are the characteristics of the ideal encryption scheme?
3. What are the differences between privacy, trust, and security? Create a scenario and explain by an example.
4. List all possible vulnerabilities by example.
5. Give one application where you use symmetric cyphers, secure hash, and asymmetric techniques.
6. Give two applications where DRM can be suitably applied. Explain by scenarios.
7. What is denial of service? Explain by an example.
8. What are side channel attacks?
9. What are some practical applications where hash algorithms can be applied?
10. Take your nick name and encrypt using DES, Hash, and PKI. Try to get decrypt back to get original data.

References

- Arm™ Server base security guide (2019)
- Bruni A (2016) Analysis of security protocols in embedded systems. PhD dissertation, Technical University of Denmark. DTU, Compute PHD-2016, No. 389
- FIDO (2020) Simpler, stronger authentication. www.Fidoalliance.org
- Framework for Improving Critical Infrastructure Cybersecurity Version 1.1 (2018) National Institute of Standards and Technology
- ISO/IEC 27001 (2013) Information technology, security techniques, information security management systems, requirements
- Kocher P (2004) Security as a new dimension in embedded system design. In: Proceedings of the 41st annual design automation conference
- Murti KCS, Tadimetti V (2011) A Simplified GeoDRM model for SDI services. In: ICCCS'11: Proceedings of the 2011 international conference on communication, computing & security
- Papp D et al (2015) Embedded systems security: threats, vulnerabilities, and attack taxonomy. In: Annual conference on privacy, security and trust (PST). IEEE
- Patel K (2011) Architectural frameworks for security and reliability of MPSoCs. IEEE Trans Very Large Scale Integr (VLSI) Syst 19(9)
- QNX (2020) Ultimate guide to embedded systems security, Blackberry
- SAE International (2016) Cybersecurity guidebook for cyber-physical vehicle systems J3061_201601
- Trusted base system architecture, system hardware on Arm™ (2018)
- Unified Extensible Firmware Interface Forum (2019). www.uefi.org

Index

A

ACPI BIOS, 331
ACPI components, 330
ACPI control method Machine Language (AML), 331
ACPI control method Source Language (ASL), 331
ACPI registers, 330
ACPI structure, 332
ACPI subsystem, 330
ACPI system, 333
ACPI system states, 333
ACPI tables, 331
Active mode, 406
Activity allocation, 318
Activity diagram, 142
Activity mapping, 319
Activity oriented data flow graphs, 60
Activity-oriented models, 42
Activity scheduling, 319
Activity synchronization, 91
Advanced Configuration and Power Interface (ACPI), 330
Advanced High-performance Bus (AHB), 394
Advanced Microcontroller Bus Architecture (AMBA), 394
Advanced Peripheral Bus (APB), 396
Advanced System Bus (ASB), 395
Agglomerative clustering, 307
Aggregation, 128
AMBA-based microcontroller, 395
AND/OR precedencies, 166
Anti-cloning, 432
Anti dependency, 167
Anti-roll back, 432

Aperiodic but predictable, 158
Arbitration fields in CAN message, 237
Architecture, 40
ARM Cortex-A8 memory hierarchy, 350
Assembly, 133
Association, 125
Association class, 126
Asymmetric algorithms, 428
Asynchronous and unpredictable, 158
Asynchronous transfers, 393
Attacker, 421
Attack surface, 421
Attack vector, 421
Augmented Reality (AR), 274
Authentication, 421
Authorization, 421
Automotive NES, 228
A wireless sensor network, 253

B

BACnet protocol layers, 248
Bacnet—typical implementation, 247
Barrier, 215
Basic cache optimizations, 348
Basic cycle in TTCAN, 240
Basic design using RTOS, 195
Basic partitioning process, 303
Basic security algorithms, 426
Basic system level design process, 94
4-beat incrementing burst operation, 397
Behavioral diagrams, 135
Behavioral representation, 36
Behaviors, 22
Best practices, 31
Bit encoding and synchronization, 236

Bit stuffing and NRZ data transmission, 237
 Block, 344
 Block diagram of a sensor node, 254
 Block floating format computations, 382
 Blocking, 353
 Bluetooth, 403
 Bluetooth architecture, 405
 Bluetooth connection process, 406
 Bluetooth layers and protocol stack, 405
 Bluetooth network, 404
 Bluetooth states, 406
 Boundedness, 53
 Bridge, The, 396
 Broad classification of RTS, 157
 Broad segments of NES, 228
 Buffer overflow, 424
 Buffer overflow attack, 425
 Building Automation and Control Network (BACNET), 247
 Burst transfers, 394
 Bus based systems, 15

C

Cache basics, 344
 Cache conflicts, 347
 Cache hit, 344
 Cache miss, 344
 CANAuth, 429
 Cancel thread, 205
 CAN frame, 234
 CAN information exchange, 239
 CAN media access and arbitration, 237
 CAN messages, 235
 CAN physical layer, 235
 CAN protocol stack, 238
 CASE methodology, 119
 Casual vs structured version, 23
 Chain of trust, 432
 Channel-Aware Routing Protocol (CARP), 415
 Characteristics of ESL, 87
 Choice, 139
 Chunks, 270
 Class diagram, 124
 Classification of attacks, 424
 12 Classification of scheduling algorithms, 170
 Clock driven scheduling, 171
 Clock synchronization, 232
 Closure, 270
 Coarse-grained multi-threading, 374
 Code injection, 422

Co-design problems, 298
 Cognitive system, 270
 Cognitive walkthrough, 285
 Common characteristics, 3
 Common USB packet fields, 402
 Communication in IoT, 412
 Communication in PSM, 76
 Compact, 3
 Compiler optimizations, 352
 Completion of behaviors, 89
 Component, 133
 Composite states, 137
 Composition, 129
 Computer hacker, 421
 Concept-process and threads, 198
 Conceptual hardware–software partitioning, 296
 Concurrency, 87
 Conditional branches, 168
 Condition variable, 213
 Confidentiality, 421
 Constant coefficient multiplier, 384
 Constant folding, 383
 Control/Data Flow Graph (CDFG), 66
 Control dependent synchronization, 91
 Control flow driven, 88
 Control flow graphs, 62
 Control hazards, 357
 Controller Area Network (CAN), 233
 Conventional model for hw-sw design process, 299
 Cornea and lens, 265
 Counting semaphore, 193
 Create thread, 203
 Criticality, 161
 Customer requirements, 1
 Custom processors, 13
 Cyber-attacks on embedded systems, 421

D

Data dependency, 166
 Data dependent synchronization, 92
 Data driven, 87
 Data hazards, 357
 Data-level parallelism, 361
 Data oriented entity-relationship model, 63
 Data-oriented models, 42
 Data packets, 403
 Data transfers, 393
 Data transfer types, 401
 Data types, 108
 Deadline-Monotonic (DM) algorithm, 177

Dedicated systems, 14
 Deployment diagram, 134
 Design for usability, 282
 Design guidelines, 284
 Design metrics, 1
 Design patterns, 284
 Design strategies, 216
 DES implementation, 427
 Device power states, 334
 Devices-IC technology, 16
 Digital rights management, 437
 Direct mapped cache, 345
 Distributed systems, 15
 DMA attacks, 426
 DMA containment, 435
 Donald Norman's model, 276
 DPM vs DVFS, 324
 Driver module in systemC, 105
 DVFS in heterogeneous processing elements, 325
 Dynamic configuration of FPGA, 377
 Dynamic Power Management (DPM), 323
 Dynamic priority algorithms, 177
 Dynamic priority scheduling, 171
 Dynamic Voltage Scaling (DVS), 324

E

Earliest Deadline First (EDF), 178
 Eavesdropping, 422
 Echoic memory, 270
 19 EDF examples, 178
 Embedded systems, 1
 Encapsulation, 70
 Energy-aware scheduling, 325, 327
 Energy dissipation in devices, 321
 Energy efficiency, 6
 Energy efficient, 3
 Energy management, 320
 Entry and exit points, 139
 Entry point, 141
 Ergonomics, 278
 Evaluation, 285
 Evaluation through user participation, 286
 Event synchronization, 91
 Event triggered protocols, 231
 Example TT protocols, 232
 Exception handling, 92
 Executable Specification Languages (ESL), 85
 Execute (EX), 354
 Exhaustive search, 422
 Exit point, 141

External actor, 29
 Eye gaze, 273
 Eye structure, 265

F

Factors driving co-design, 297
 Fast identity online alliance, 438
 Fault isolation mechanisms, 233
 Few techniques for cache performance, 351
 Fieldbus, 243
 Fieldbus gateway to internet, 245
 Fieldbus-internet interface, 246
 Fieldbus topology, 243
 Fields in a USB packet, 402
 Final, 141
 Final pseudo state, 139
 Fine-grained multi-threading, 373
 Finite state machine (Mealy) model, 43
 Finite state machine (Moore model), 45
 Finite State Machine with Data path (FSMD), 45
 Flexibility, 283
 Flow dependency, 166
 Fork and Join, 140
 Fovea, 265
 Full adder using systemC, 103
 Fully associative cache, 345
 Functional updates, 6
 Functional vs. HW-SW partitions, 302
 Fundamental services of TT protocol, 232

G

Gather-scatter, 365
 Generalization, 129
 Generic bus architecture, 392
 Generic FPGA architecture, 377
 Generic Interrupt Controller (GIC), 396
 Generic PLD architecture, 376
 Global system states, 334
 Goal, 22
 GOMS model, 287
 GPU architecture mapped to GPU hardware, 369
 Graphic Processing Units (GPU) SIMT architecture, 367
 Greedy partitioning, 307
 Guidelines for secure systems, 430
 Gulf of evaluation, 277
 Gulf of execution, 277

H

Half adder module, 103
 Half adder using systemC, 101
 Handshake packets, 403
 Handwriting recognition, 272
 Hard RT systems, 159
 Hardware identity, 436
 Hardware-oriented partitioning, 302
 Hazards in pipelining, 356
 Heterogeneous models, 43, 66
 Hierarchical channels, 114
 Hierarchical clustering, 307
 Hierarchical concurrent FSMs, 56
 Hierarchy of behaviors, 88
 Higher associativity, 348
 Host protocol, 400
 Hub and spoke model, 26
 Human-agent interaction, 280
 Human system, 264
 HW-SW co-design, 295

I

IETF stack for IoT, 413
 Implicit interface, 281
 Improper authentication, 425
 Improper input data validation, 425
 Information Objects (DIOs), 414
 Inheritance, 70
 Initial pseudo state, 139
 Injecting crafted packets, 422
 Instruction cycle for RISC, 354
 Instruction Decode (ID), 354, 360
 Instruction execute (EX), 360
 Instruction Fetch (IF), 359
 Integer programming model, 305
 Integrated co-design process, 300
 Integrity check, 432
 Interaction concepts, 275
 Interaction model, 276
 Interest diffusion in WSN, 256
 Interface, 112, 131
 Inter-Integrated Circuit bus (I²C bus), 407
 Introduction to bus, 392
 Inverter circuit, 321
 IO addressing, 394
 IoT development hardware, 412
 IoT framework, 412
 IoT platform for embedded systems, 409
 ISO/IEC 27001:2013, 437

J

Jackson's structured programming model, 65
 Join command, 204
 Junctions, 140

K

Kernel, 199
 Kernighan-Lin algorithm, 310
 Keystroke-Level Model (KLM), 289

L

Lanes, 363
 Larger block sizes, 348
 Laxity function, 161
 Layered structure of fieldbus, 244
 Learnability, 282
 Least Slack Time first algorithm (LST), 178
 Levels, 28
 Lightweight processes, 206
 Liveness, 54
 Load/store, 361
 Localization, 256
 Localization in WSNs, 253
 Logical configuration of CAN bus, 234
 Logic design with CLB, 379
 Long-Term Memory (LTM), 270
 LON works, 248
 Loop fusion, 352
 Loop interchange, 352
 6LoWPAN protocol, 414

M

Main success scenario, 24
 Maintainability, 7
 Malware, 422
 Master-slave, 216
 Media access strategies in NES, 244, 245
 Member functions, 102
 Memory hierarchy, 343
 Memory (Mem), 270, 354
 Memory system and memory banks, 364
 Merging arrays, 352
 Message authentication frame for CANAuth, 430
 Message Queue Telemetry Transport (MQTT), 415
 Metaphors, 279
 Method (SC_METHOD), 102
 Mixed scheduling, 207
 Model, 38

Model-based evaluation, 286
 Modelling periodic tasks, 162
 Models and architectures, 35
 Models of computation in SystemC, 110
 Model taxonomy, 42
 Modern IO interfaces, 399
 Module, 98
 Module constructor, 99
 Module declaration, 98
 Module ports, 99
 Module signals, 99
 Monitor module, 105
 Moore's law, 342
 More blocks, 348
 More cache levels, 348
 Mostly periodic, 157
 Movement, 269
 MQTT broker architecture, 415
 Multicore processor with shared memory, 372
 Multimodality, 280
 Multi-tasking, 200
 Mutable, 421
 Mutex, 195
 Mutual exclusion, 90

N

Named connection, 101
 NES in building automation, 230
 NES in industrial automation, 241
 Networked Embedded Systems (NES), 225
 Network stack of WSN, 254
 Network topology, 235
 NIST-cybersecurity framework, 437
 Non recurring engineering costs, 1
 Norman's 7 principles, 284
 Notification process, 110

O

Object Management Group (OMG), 119
 Object oriented model, 69
 Objects in systemC, 107
 Open collector bus, 236
 Optimizations needed, 7
 Out of bounds memory access, 426

P

Paging in virtual memory management, 349
 Park mode, 406
 Partitioning algorithms, 304
 Partitioning approaches, 301

Performance, 6
 Performance of a partition, 302
 Periodic, 157
 Periodic exchange of state messages, 233
 Periodic schedule, 162
 Peripheral interrupts, 398
 Petri nets, 49
 Physical representation, 37
 Physical security, 421
 Physical system, 272
 Pipeline, 217
 Pipeline in ARM, 396
 Pipeline in Arm Cortex-A8, 359
 Pipeline in MIPS processor, 358
 Pipeline relationship, 168
 Pipelining, 355
 Platform reset attacks, 426
 Polling, 245
 Polymorphism, 70
 Positional connection, 100
 Posix, 202
 Power efficiency, 322
 Power management at design time, 336
 Power management at run time, 337
 Precautions while using semaphores, 194
 Precedence constraints and dependencies, 164
 Precedence graph, 164
 Precondition, 24
 Predictive p-persistent CSMA protocol, 249
 Preemptivity, 161
 Primary actor, 22, 24
 Primitive channel, 113
 Priority driven periodic tasks, 176
 Priority inheritance, 184
 Priority inversion, 183
 Privacy, 421
 Privacy in IoT, 410
 Process contention scope, 207
 Processes, 102, 200
 Process local scheduling, 207
 Processor performance states, 335
 Process power states, 335
 Process, threads, LWPs and kernel, 206
 Program state machines, 74
 Properties of petri nets, 52
 Provenance, 421
 Pseudo states, 138

Q

Quality metrics in ES design, 5

R

Random access, 245
 Rate Monotonic Algorithm (RMA), 176
 Ratio cut, 308
 Reachability and deadlock, 52
 Reactive, 4
 Reader's/writers lock, 214
 Reading interface, 268
 Real-time, 4
 Real time behavior, 1
 Real time extensions provided in Posix-4b, 202
 Real time operating systems, 189
 Real time systems, 155
 Reconfigurable computing—FPGAs, 375
 Reduced-Function Devices (RFDs), 250
 Re-entrancy, 192
 Register-transfer level model, 112
 Relation between model and architecture, 41
 Replacing a block during cache miss, 347
 Representation of a design, 36
 Resource access, 183
 Resource access and contention, 182
 Resource graph, 168
 Resource synchronization, 90
 Response time, 3
 Response to external events, 163
 Responsiveness, 284
 Reverse engineering, 422
 RISC processors, 353
 RMA example, 177
 Robustness, 283
 Rods and cones, 265
 Routing in RPL, 414
 Routing in WSNs, 255
 RTOS—basic organization, 192
 RTOS- concepts, 190
 Ruggedness, 7

S

SAE J3061_201601, 437
 Safety, 7
 Sc_clock, 107
 Scenarios, 28
 Sc_Event, 109
 SCHED_FIFO, 208
 SCHED_OTHER, 209
 SCHED_RR, 209
 Schedule matrixes TTCAN, 240
 Scheduler, 160
 Schedule threads, 206
 Scheduling classes, 208

Scheduling policies, 207
 Scheduling process, 169
 Scheduling sporadic jobs, 181
 Scheduling threads on LWPs, 207
 Sc_module coefficient multiplier, 100
 Scope, 24
 Sc_Start, 109
 Secure boot, 432
 Secured instruction fetch, 436
 Secured platform architecture, 439
 Secured SDLC, 436
 Secure firmware updates and critical data, 432
 Secure hash algorithms, 427
 Secure storage and memory, 435
 Security, 421
 Security in embedded systems, 419
 Security in IoT, 411
 Security life cycle, 435
 Security policies, 422
 Security protocols for embedded systems, 429
 Security standards for embedded systems, 437
 Security vulnerabilities, 424
 Semaphore, 193
 Sensing position in 3D space, 274
 Sensor to plant level network hierarchy, 242
 Sequence diagram, 143
 Session layer, 415
 Set associative cache, 346
 Shared communication, 89
 Short-Term Memory (STM), 270
 Signals, 133
 SIMD architecture, 366
 Simplified GIC architecture, 399
 Simulated annealing, 308
 Simultaneous Multi-Threading (SMT), 374
 Single Instruction Multiple Threads (SIMT), 368
 Slack time, 173
 Sleep states, 334
 Sniff mode, 406
 Soft RT systems, 160
 Software and scheduler modeling (more later), 97
 Software Generated Interrupts (SGI), 398
 Software-oriented partitioning, 302
 Software security, 421
 Spatial locality, 344
 Specifications to modeling to hw-sw partition, 299
 Speech recognition, 272

Spin locks, 215
 Sporadic jobs in clock driven scheduling, 175
 Stack canaries, 435
 Stake holder, 22
 802.15.4 standard, 250
 Standard use case template, 30
 Start of frame packets, 403
 State diagram, 137
 State-oriented models, 42
 Static DVFS scheduling, 327
 Static priority scheduling, 171
 Static scheduling, 170
 Stereoscopy principle, 267
 Stride, 364
 Structural diagrams, 123
 Structural hazards, 356
 Structural partitioning, 301
 Structural representation, 36
 Structure oriented models, 63
 Success scenario, 22
 Superframes, 251
 Supporting actor, 27
 Switching delay by reducing Vdd, 322
 Synchronization, 90
 Synchronous transfers, 393
 SystemC, 93
 SystemC features, 96
 System global scheduling, 207
 System on chip with FPGA, 378
 System partitioning, 301
 System recovery, 435
 System supported cooperative work, 280
 System under development, 21, 22, 26, 27, 29, 30

T

Task and task states, 191
 Task conformance, 284
 Task dependencies, 165
 Task graph, 165
 Task utilization, 163
 Techniques for energy minimization, 323
 Temporal dependency, 166
 Temporal locality, 344
 Terms in RT systems, 158
 Test bench, The, 106
 Thread, 201
 Thread blocks and grid, 370
 Thread ID, 201
 Thread-level parallelism (TLP), 371
 Thread per client, 217

Thread per request, 217
 Thread (SC_THREAD), 102
 Thread states, 208
 Thread synchronization, 204, 210
 Threat actor, 420
 Timed functional model, 111
 Time-slot-based, 245
 Time synchronization in WSNs, 256
 Time to market, 6
 Time triggered CAN, 239
 Time Triggered protocols (TT), 232
 Token packets, 403
 Token passing, 245
 Touch, 269
 Traits, 1
 Transaction-level model, 111
 Transaction protocol, 401
 Triangulation, 257
 Triggers, 22
 Trueness, 7
 Trust, 421
 Trust across devices, 431
 Trusted execution., 434
 Typical ARM platform with AMBA bus, 394
 Typical DVFS system, 324
 Typical guidelines for power management, 336
 Typical integrated building automation system, 246
 Typical mapping flow of FPGA, 379
 Typical tasks and roles in system engineering, 121

U

Ubiquitous computing, 281
 UEFI security guidance, 433
 UML diagrams, 121
 Unified Modelling Language (UML), 119
 Unique functionality, 3
 Unit cost, 5
 Universal Serial Bus (USB), 399
 Untimed functional model, 111
 USB packet types, 403
 Use case diagram, 136
 Use cases, 21–23, 25, 29, 31
 User driven requirements, 3

V

Valid and feasible schedule, 169
 Vector architectures, 361
 Vector instructions, 362
 Vector length register, 364

Vector mask registers, [364](#)
Vector processing architecture in ARM, [366](#)
Versatility factors for ES product, [9](#)
Virtual memory, [348](#)
Virtual reality, [273](#)
Virtual wire through NV binding, [249](#)
Visibility of system status, [285](#)
Vision, [265](#)
Visual angle, [266](#)
Visual interface, [266](#)
Vulnerability, [420](#)

W

Wait, [109](#)
Wait Until, [108](#)

Wireless Sensor Networks (WSN), [230](#)
Work queue, [217](#)
Write-back cache, [347](#)
Write Back (WB), [354](#)
Write-through cache., [347](#)
Writing to cache, [347](#)

X

X-cells, [265](#)

Z

ZigBee, [250](#)
Zigbee network, [250](#)
Zigbee network stack, [252](#)